Online Learning: A Modern Introduction Using Convex Optimization

Francesco Orabona
KAUST
francesco@orabona.com

arXiv:1912.13213v10 [cs.LG] 21 Jun 2026

To Irene and Dante

Contents

Foreword vi
Preface ix

1 What is Online Learning? 1
1.1 History Bits 6

2 Online Subgradient Descent 8
2.1 Online Learning with Convex Differentiable Losses 8
2.1.1 Convex Analysis Bits: Convexity 9
2.1.2 Online Gradient Descent 11
2.2 Online Subgradient Descent 16
2.2.1 Convex Analysis Bits: Subgradients 16
2.2.2 Analysis with Subgradients 19
2.2.3 Unit Analysis Bits 20
2.3 Linear Regret: From Convex Losses to Linear Losses 22
2.4 History Bits 22
2.5 Exercises 23

3 Online-to-Batch Conversions 24
3.1 From Online Learning to Stochastic Optimization 24
3.1.1 Bits on Concentration Inequalities 26
3.1.2 High-Probability Guarantees for Online-to-Batch Conversion 27
3.2 Application: Agnostic PAC Learning 29
3.3 History Bits 33
3.4 Exercises 33

4 Beyond $\sqrt{T}$ Regret 34
4.1 Strong Convexity and Online Subgradient Descent 34
4.1.1 Convex Analysis Bits: Strong Convexity 34
4.1.2 Online Subgradient Descent for Strongly Convex Losses 37
4.2 Adaptive Algorithms: $\mathscr{L}^\star$ bounds and AdaGrad 39
4.2.1 Adaptive Learning Rates for Online Subgradient Descent 39
4.2.2 Convex Analysis Bits: Dual Norms, Smooth, and Self-Bounded Functions 41
4.2.3 $\mathscr{L}^\star$ bounds 44
4.2.4 AdaGrad 45
4.3 History Bits 48
4.4 Exercises 49

5 Lower Bounds for Online Linear Optimization 50
5.1 Lower Bound for Bounded Online Linear Optimization 50
5.2 Lower Bound for Unconstrained Online Subgradient Descent 52
5.3 Lower Bound for Unconstrained Online Linear Optimization 53
5.4 History Bits 58
5.5 Exercises 59

6 Online Mirror Descent 60
6.1 Subgradients are not Informative 60
6.2 Reinterpreting the Online Subgradient Descent Algorithm 62
6.3 Convex Analysis Bits: Bregman Divergence 63
6.4 Online Mirror Descent 65
6.4.1 Convex Analysis Bits: Fenchel Conjugate 68
6.4.2 The "Mirror" Interpretation 74
6.4.3 Yet Another Way to Write the Online Mirror Descent Update 76
6.5 OMD Regret Bound using Local Norms 77
6.6 Example of OMD: Exponentiated Gradient 78
6.7 Example of OMD: p-norm Algorithms 82
6.8 Application: Learning with Expert Advice 84
6.9 Example of OMD: Multi-Scale Expert Algorithm 86
6.10 Application: Combining Online Algorithms to Adapt to the Learning Rate 88
6.11 Optimistic Online Mirror Descent 91
6.12 Application: OMD with Delayed Feedback 94
6.13 History Bits 95
6.14 Exercises 97

7 Follow-the-Regularized-Leader 99
7.1 The Follow-the-Regularized-Leader Algorithm 99
7.2 FTRL Regret Bound using Strong Convexity 101
7.2.1 Proximal Regularizers 102
7.3 FTRL with Linearized Losses 103
7.3.1 FTRL with Linearized Losses Can Be Equivalent to OMD 107
7.4 FTRL Regret Bound using Local Norms 107
7.5 Example of FTRL: Exponentiated Gradient without Knowing T 109
7.6 Example of FTRL: AdaHedge 110
7.7 Example of FTRL: Group Norms 115
7.8 Composite Losses and Proximal Operators 117
7.9 FTRL on Strongly Convex Functions 119
7.10 A Weaker Notion of Strong Convexity and the Online Newton Step 120
7.10.1 Convex Analysis Bits: Exp-concavity 123
7.11 Online Linear Regression: Vovk–Azoury–Warmuth Forecaster 126
7.12 Optimistic Follow-the-Regularized-Leader 129
7.12.1 Improved Optimistic FTRL Bound for Linear Losses 131
7.13 Application: FTRL with Delayed Feedback 133
7.14 Application: Regret that Depends on the Variance of the Subgradients 133
7.15 Application: Online Convex Optimization with Gradual Variations 134
7.16 History Bits 136
7.17 Exercises 139

8 Online Linear Classification 141
8.1 Online Randomized Classifier 141
8.2 The Perceptron Algorithm 142
8.3 The Gaptron Algorithm 146
8.4 History Bits 149
8.5 Exercises 150

9 Multi-Armed Bandit 151
9.1 Adversarial Multi-Armed Bandit 151
9.1.1 Exponential-weight algorithm for Exploration and Exploitation: Exp3 154
9.1.2 Optimal Regret Using Online Mirror Descent (OMD) with Tsallis Entropy 155
9.2 Stochastic Bandits 157
9.2.1 Concentration Inequalities Bits 157
9.2.2 Explore-Then-Commit Algorithm 159
9.2.3 Upper Confidence Bound Algorithm 161
9.3 Best-of-Both Worlds Guarantees with Tsallis-INF 165
9.4 History Bits 168
9.5 Exercises 169

10 Universal Portfolio Algorithms 170
10.1 Setting of Portfolio Selection 170
10.2 Portfolio Selection with the Exponentiated Gradient Algorithm 171
10.3 Universal Portfolio Selection with F-Weighted Portfolio Algorithms 171
10.3.1 Information Theory Bits 173
10.3.2 Proof of Theorem 10.1 174
10.4 Portfolio Selection with the Online Newton Step Algorithm 176
10.5 History Bits 178
10.6 Exercises 178

11 Weighted Average Algorithm and Aggregating Algorithm 179
11.1 Follow-the-Regularized-Leader with Distributions 179
11.2 The Weighted Average Algorithm 180
11.2.1 ONS and OGD as instantiations of WAA 183
11.3 The Aggregating Algorithm and Mixable Losses 186
11.4 Example of AA: Online Multiclass Logistic Regression 189
11.5 History Bits 190
11.6 Exercises 191

12 Black-Box Reductions 192
12.1 Solving Constrained Online Convex Optimization 192
12.1.1 Dealing with Constraints using (Non-Euclidean) Projections 193
12.1.2 Dealing with Constraints using Bregman Projections 199
12.2 Sleeping Experts 201
12.3 Reduction to Magnitude and Direction 203
12.4 History Bits 204
12.5 Exercises 205

13 Parameter-free Online Linear Optimization 206
13.1 The Coin-Betting Game 206

13.2 Parameter-free One-Dimensional Online Convex Optimization 211
13.2.1 KT as a One-Dimensional Online Convex Optimization Algorithm 213
13.3 Coordinate-wise Parameter-free Online Convex Optimization 215
13.4 Parameter-free Algorithms in Any Norm 216
13.5 Combining Parameter-free Algorithms Almost for Free 217
13.6 Reduction to Learning with Experts 217
13.7 Optimal Regret Bounds by Losing at Most a Constant Fraction of Money 221
13.8 Example: A Parameter-free Algorithm for Sleeping Experts 222
13.9 From Portfolio Selection to Continuous-Coin Betting 224
13.10 History Bits 224
13.11 Exercises 226

14 Dynamic, Strongly Adaptive, and Tracking Regret 227
14.1 Dynamic Regret 227
14.1.1 Dynamic Regret of Online Mirror Descent 228
14.1.2 ADER: Optimal Dependence on the Path Length for Online Subgradient Descent 230
14.1.3 Black-box Reduction for Dynamic Regret in Unbounded Feasible Sets 232
14.2 Strongly Adaptive Regret 234
14.2.1 CBCE: A Meta Algorithm for Strongly Adaptive Regret 234
14.2.2 Efficient Version of CBCE 236
14.3 Tracking Regret 240
14.4 History Bits 242
14.5 Exercises 243

15 Saddle-Point Optimization and Online Algorithms 244
15.1 Saddle-Point Problems 244
15.2 Solving Saddle-Point Problems with Online Convex Optimization 248
15.2.1 Variations with Best Response and Alternation 251
15.3 Game-Theory Interpretation of Saddle-Point Problems 253
15.3.1 Reducing Extensive to Normal-Form Zero-Sum Games 256
15.4 Application: Boosting as a Two-Person Game 258
15.5 Faster Rates Through Optimism 260
15.5.1 Prescient Online Mirror Descent and Be-The-Regularized-Leader 265
15.6 History Bits 267
15.7 Exercises 269

16 From Online Learning to X 270
16.1 From Online Learning to Rademacher Complexity 270
16.2 From Online Learning to PAC-Bayes 273
16.3 From Online Learning to Non-Convex Non-Smooth Optimization 276
16.4 From Online Learning to Time-Uniform Concentration Inequalities 281
16.5 History Bits 287

Appendix **Appendix** 291
References 299
Index 313

Foreword

This book is about online learning, a mathematical framework that guides the design of algorithms that learn by interacting with their environment. This approach to machine learning differs from the more traditional statistical view, which distinguishes between a training phase, in which a learning algorithm builds a prediction model from a training set, and a testing phase, in which the trained model is evaluated on a separate test set. In online learning, these two phases are interleaved: data are received sequentially, and the algorithm must first make a prediction on each incoming data point and only then update its model using the resulting feedback. The two theories are not in competition: they look at the same problem from different angles, and researchers choose the theory that offers the tools best suited to solving their problem. Indeed, both theories tackle the same fundamental machine learning question: what is the most efficient algorithmic way of using past observations to make future predictions? However, they differ with respect to the assumptions they make about the source of the learning data. In statistical learning, data are obtained through independent draws from a fixed and unknown distribution. In online learning, data are generated sequentially by a source that need not be stochastic and may even be adversarial. Still, one can obtain standard generalization guarantees by running an online learning algorithm with suitable regret guarantees on a data sequence obtained through independent draws (a so-called online-to-batch conversion), a technique that connects the online world to the statistical one. More generally, we can run online algorithms to solve stochastic convex optimization problems efficiently, which is just one of the many powerful applications of the online framework that the reader will find in the book.

The connection to convex optimization, one of the book's main themes, operates at a fundamental level. Indeed, online learning is often described in terms of online convex optimization, a setting in which the goal is to keep the algorithm's cumulative loss close to that of a suitable comparator, while facing an arbitrary sequence of convex loss functions. Building on this connection, one can transfer many powerful ideas and techniques from convex optimization—such as gradient descent and its generalizations—to the online world. Convex analysis thus becomes a unifying language for deriving the theory behind online algorithms, and the main technical tool used throughout the book. This brings us to one of the most important notions of online learning: regret. Regret addresses the following fundamental issue: what is the right way to measure the performance of an online learning algorithm when the data come from an arbitrary source? If we assume that data are the result of independent draws from a fixed distribution, the risk of a predictor is traditionally measured in terms of its expected loss. We then choose a reference class of predictors (e.g., linear models) and seek to bound the estimation error, expressed as the difference between the predictor's risk and

the risk of any comparator in the reference class. In the online world, however, there are two important differences: first, we cannot define statistical risk, because there is no statistical source generating the data; second, there is no single predictor, but rather an online algorithm outputting a sequence of predictions. To measure performance, we can then average all the losses accumulated by the algorithm when making predictions at each data point in the sequence (in contrast to statistical risk, I often call this quantity "sequential risk"). This empirical average reflects how well the algorithm adapts to any regularity present in the data sequence. Note that by doing so, we are not measuring the performance of a static prediction model generated after training a learning algorithm on a dataset, but rather the performance of a learning algorithm run on an individual sequence of data. Using this sequential risk, regret can be viewed as the online counterpart of estimation error. In normalized form, it is the difference between the algorithm's average loss and the average loss of a comparator. Multiplying by the number of rounds gives the more standard cumulative regret. Unlike in statistical learning, where risk is a property of predictors, online learning gives us more freedom in choosing the comparator: as the book explains, we can compare the algorithm not only with a fixed predictor, but also with dynamic comparator sequences that may use different models at different times. This makes online learning a natural framework for designing algorithms that remain robust under highly nonstationary data, especially when a fixed distributional model is inadequate.

In the online convex optimization setting, regret analysis copes with arbitrary loss sequences by requiring each loss function to be convex. This allows us to upper bound regret by its linearized version, thus reducing online convex optimization (OCO) to online linear optimization (OLO). One of the key algorithmic principles for controlling linearized regret is stability through regularization. After making a prediction on a new data point, the algorithm typically updates its model by balancing two goals: reducing the current loss and staying close to the previous model. The regularizer therefore shapes the trajectory followed by the algorithm as longer and longer prefixes of the data sequence are observed. Unfortunately, standard online algorithms face a difficult dilemma: as we explained, regret is the difference between the sequential risk of the algorithm and the sequential risk of any comparator in the reference class. Ideally, we would like regret bounds that are optimal simultaneously for all comparators. However, to achieve optimal regret against any single model, the algorithm would have to choose its regularizer (or the learning rate associated with it) based on information about the comparator (e.g., the norm of the vector of coefficients for a linear model). This seems to prevent simultaneous optimality, as the algorithm must choose a specific learning rate, which might be suboptimal for some of the comparators. One of the most striking contributions of the book is a theory showing that parameter-free online algorithms, achieving essentially optimal bounds simultaneously over all comparators, can be derived from algorithms designed to play a simple coin-betting game. This theory, developed by Francesco Orabona and his co-authors over the last decade, uses a connection between prediction and betting to turn optimal gambling algorithms into optimal online learners, so that a lower bound on the wealth of the gambler becomes an upper bound on the regret of the learner. Parameter-free algorithms have now become one of the most interesting and active areas of online learning research.

Gambling is just one of the many connections explored in the book. One chapter relates online learning to portfolio management and sequential investment; another explores its

interface with saddle point optimization and game theory. These interconnections with other fields reflect the way the ideas and techniques behind online learning developed over several decades in different disciplines, such as information theory and statistics, game theory, and computer science. Orabona does an excellent job of tracing the different sources, indicating where a definition first appeared and which authors should be credited for a given result or technique. The rigor and accuracy of the historical notes in the book are exemplary and should be taken as a model for surveys and monographs.

As noted at the beginning, online learning is an abstraction for an agent that learns by interacting with its environment. Regret makes this process quantitative: vanishing regret means that the algorithm's sequential risk approaches that of the comparator. A crucial aspect governing this process is the interface through which the algorithm obtains feedback from the environment. Consider, for example, a simple sequential classification problem in which, upon receiving a data point, the learner must predict a label from a fixed set of, say, ten labels. After each prediction, the environment could reveal the correct label (full feedback), reveal only whether the prediction was correct (bandit feedback), or the feedback itself may be selective, as in a spam filter that receives user feedback only on emails that reach the inbox. This adds a new dimension to the design of online learning algorithms; namely, identifying the best possible regret rate for a given feedback structure. Although, among partial-information models, the book focuses on the multi-armed bandit setting, the mathematical foundations of online learning are necessary background for studying online learning with partial feedback.

Speaking of mathematics, readers of this book will not be disappointed: every mathematical detail is laid out precisely, including all the preliminary notions needed to prove each result. Orabona strives to find the simplest possible proof, but he does not make it any simpler than that. The book is a treasure trove of inequalities, technical devices, and intermediate results that form the arsenal of any researcher interested in these topics.

I wrote a book on these topics two decades ago and have been delighted by the surprising conceptual and technical advances made since then. This book, which provides an accurate, rigorous, in-depth account of these advances, is the ideal starting point for all those who want to explore this garden of delights known as online learning.

Nicolò Cesa-Bianchi
Forte dei Marmi
June 1, 2026

Preface

Everything should be as simple as it can be, but not simpler
(attributed to Albert Einstein)

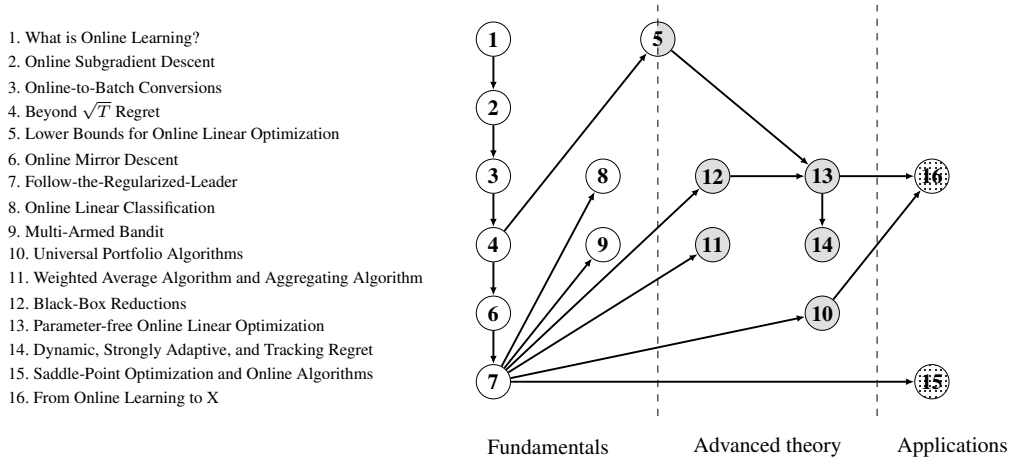
In this book, I introduce the concepts of online learning through a modern view based on convex optimization. Here, online learning refers to the framework of regret minimization under worst-case assumptions. I attempted to unify all the literature as instantiations of Online Mirror Descent and Follow-the-Regularized-Leader (and their variants). I paid particular attention to the issue of tuning the parameters of the algorithms, through *adaptive* and *parameter-free* online learning algorithms. The bandit setting is also briefly discussed, touching on the problem of adversarial and stochastic multi-armed bandits. Building on fundamental algorithms and concepts, I also cover advanced topics, including black-box reductions, saddle-point optimization, sequential investment, and non-stationary forms of regret analysis. Finally, I conclude with a selection of applications of online learning to domains far from it, such as generalization theory and concentration inequalities.

I attempted to maintain an informal, yet mathematically rigorous, tone throughout the book. Moreover, all the included proofs have been carefully chosen to be as simple and as short as possible. This also means that sometimes I have added one or two additional assumptions, just to simplify the proofs.

Why do we need another book on online learning? My belief is that offline and online learning are two sides of the same coin. So, I fully used the formalism of convex analysis, typical of the non-smooth offline optimization literature, to show the similarities and build bridges. Moreover, I tried to cover very recent and very advanced results, which are often not part of online learning books. For example, parameter-free methods are (by definition!) the optimal ones for online learning, yet they are not widely understood, maybe due to the lack of a reference book on the topic. I also spent a considerable amount of time to gather all the history of these methods in my "History Bits" sections. I hope the serious students will take advantage of these sections to further deepen their study of this topic. A note on citations: It is customary in the computer science literature to only cite the journal version of a result that first appeared in a conference. The rationale is that the conference version is only a preliminary version, while the journal one is often more complete and sometimes more correct. In this book, I will not use this custom. Instead, in the presence of the conference and journal versions of the same paper, I will cite both, to clearly delineate the history of the ideas, their first inventors, and the unavoidable rediscoveries.

Structure of the book. No prior knowledge of convex analysis is required. So, given the

amount of new tools and definitions I had to cover, I introduced the necessary math a bit at a time. Hence, the book is meant to be read in sequential order. In particular, the first 7 chapters cover the basics of a course on online learning, with a possible exception of Chapter 5 on lower bounds that can be omitted for an easier set of lectures. Chapters 8 (Online Linear Classification) and 9 (Multi-Armed Bandit) can be included or not, depending on which topics one wants to cover. Chapters 10 to 14 cover more advanced topics, such as universal portfolio, black-box reductions, and parameter-free algorithms, appropriate for a graduate-level class. Chapter 5 acts as a bridge between the foundational methods and the parameter-free ones, because it shows the *necessity* for better algorithms. Finally, Chapters 15 and 16 cover applications of online learning to other areas, such as saddle-point optimization, boosting, non-convex non-smooth optimization, and generalization theory, that might offer leads to new research directions. While all the chapters have a minimal amount of interconnection, the major dependencies are in the figure.



The dependence structure of the chapters.

Acknowledgments. I thank all the people who checked the proofs and reasoning in these notes. In particular, the students in my first class who mercilessly pointed out my mistakes, Nicolò Campolongo, who found all the typos in my formulas, and Jake Abernethy, for the brainstorming on presentation strategies. Other people that helped me with comments, feedback, references, and/or hunting typos (in alphabetical order): Zeyad Aljaali, Andreas Argyriou, Param Kishor Budhraja, Nicolò Cesa-Bianchi, Sahil Chaudhary, Keyi Chen, Mingyu Chen, Peiqing Chen, Ashok Cutkosky, Ryan D'Orazio, Gerardo Durán-Martín, Alon Gonen, Peijia Guo, Dirk van der Hoeven, Daniel Hsu, Gergely Imreh, Andrew Jacobsen, Emmeran Johnson, Kwang-Sung Jun, Michał Kempka, Ji-Ha Kim, Andrew Christian Kroer, Joon Kwon, Pierre Laforgue, Wei-Cheng Lee, Chuang-Chieh Lin, Haipeng Luo, Shashank Manjunath, David Martínez-Rubio, Valentina Masarotto, Aryan Mokhtari, Antoine Moulin, Gergely Neu, Ankit Pensia, Viacheslav D. Potapov, Yousef Radwan, Abed Razawy, Daniel Roy, Ludovic Schwartz, Alex Shtoff, Antonio Silveti-Falls, Yanze Song, Luca Viano, Guanghui Wang, Yulian Wu, Jiujia Zhang, Peng Zhao, and Xingyu Zhou.

1

What is Online Learning?

Consider the following repeated guessing game:
In each round $t = 1, \ldots, T$

- An adversary chooses a real number $y_t \in [0, 1]$ and keeps it secret;
- You try to guess the real number, choosing $x_t \in [0, 1]$;
- The adversary's number is revealed, and you pay the squared difference $(x_t - y_t)^2$.

Basically, we want to guess a sequence of numbers as precisely as possible. To make it a game, we must now define a "winning condition". Let's see what makes sense to consider as a winning condition.

Let's start by making the game easier for the player. Let's assume that the adversary is drawing independent and identically distributed (i.i.d.) numbers from some fixed distribution over $[0, 1]$. However, he is still free to decide which distribution to use at the beginning of the game. If we knew the distribution, we could just predict the mean of the distribution at each round, and in expectation we would pay $\sigma^2 T$, where σ^2 is the variance of the distribution. We cannot do better than that! However, given that we do not know the distribution, it is natural to benchmark our strategy with respect to the optimal one. That is, it is natural to measure the quantity

$$\mathbb{E}_{Y_1,\ldots,Y_T}\left[\sum_{t=1}^{T}(x_t - Y_t)^2\right] - \sigma^2 T, \tag{1.1}$$

or, equivalently, considering the average

$$\frac{1}{T}\mathbb{E}_{Y_1,\ldots Y_T}\left[\sum_{t=1}^{T}(x_t - Y_t)^2\right] - \sigma^2 \,. \tag{1.2}$$

Clearly, these quantities are nonnegative, and they seem to be a good measure, because they are somehow normalized with respect to the "difficulty" of the numbers generated by the adversary, through the variance of the distribution. This is not the only possible measure of our "success", but it is certainly a reasonable one. It would make sense to consider a strategy "successful" if the difference in (1.1) grows sublinearly over time and, equivalently, if the difference in (1.2) goes to zero as the number of rounds T goes to infinity. That is, on average over the rounds, we would like our algorithm to be able to approach the optimal performance.

Minimizing Regret. Having arrived at what seems to be a good measure of success of the

algorithm, let's now rewrite (1.1) in an equivalent way:

$$\mathbb{E}\left[\sum_{t=1}^{T}(x_t - Y_t)^2\right] - \min_{x\in[0,1]} \mathbb{E}\left[\sum_{t=1}^{T}(x - Y_t)^2\right] .$$

Now, the last step: let's remove the assumption on how the data is generated, consider any arbitrary sequence of y_t, and let's keep using the same measure of success. If the algorithm is deterministic, we can remove the expectation because there is no stochasticity anymore. So, we get that we will win the game if

$$\mathrm{Regret}_T := \sum_{t=1}^{T}(x_t - y_t)^2 - \min_{x\in[0,1]} \sum_{t=1}^{T}(x - y_t)^2$$

grows sublinearly with T. The quantity above is called the **regret**, because it measures how much the algorithm regrets not having played on every round the best single choice in hindsight. We will denote it by Regret_T.

Our reasoning should provide sufficient justification for this metric; however, throughout this book, we will see that it also makes sense from both a convex optimization and a machine learning perspective.

Note that in the stochastic case, the optimal strategy is given by a single best prediction, so it was natural to compare against it. Instead, with arbitrary sequences, it is not clear anymore that this is a good competitor. For example, we might consider a sequence of competitors instead of a single one. Indeed, it can be done, but the single competitor is still interesting in a variety of settings and simpler to explain. So, for most of this book, we will use a single competitor, while we will consider a sequence of competitors in Chapter 14.

Let's now generalize the online guessing game, considering that the algorithm outputs a vector $\boldsymbol{x}_t$ in the **valid set**[1] $\mathcal{V} \subseteq \mathbb{R}^d$ (also called **feasible set**), and it pays a **loss** $\ell_t : \mathcal{V} \to \mathbb{R}$ that measures how good the prediction of the algorithm was in each round. Also, let's consider a **comparator** (also called **competitor**), that is, an arbitrary predictor $\boldsymbol{u}$ in $\mathcal{V} \subseteq \mathbb{R}^d$ and let's parameterize the regret with respect to it: $\mathrm{Regret}_T(\boldsymbol{u})$. We can now define online learning: designing and analyzing algorithms to minimize the regret over a sequence of loss functions with respect to an arbitrary competitor $\boldsymbol{u} \in \mathcal{V} \subseteq \mathbb{R}^d$:

$$\mathrm{Regret}_T(\boldsymbol{u}) := \sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) .$$

It is worth stressing that an online algorithm does not know $\boldsymbol{u}$ or the value of the corresponding regret in order to guarantee an upper bound on the regret. We will say that the algorithm is **no-regret** when, for every fixed $\boldsymbol{u} \in \mathcal{V}$, its regret is at most sublinear, that is, $\lim_{T\to\infty} \mathrm{Regret}_T(\boldsymbol{u})/T \leq 0$.

Remark 1.1 *Strictly speaking, the regret is also a function of the losses $\ell_1, \ldots, \ell_T$. However, we will suppress this dependence for simplicity of notation.*

This framework is quite powerful, and it allows us to reformulate a bunch of different

[1] In some cases, we can make the game easier for the algorithm by letting it choose the prediction from a set $\mathcal{W} \supset \mathcal{V}$.

problems in machine learning and optimization as similar games. More generally, with the regret framework, we can analyze situations in which the data are not independent draws from a fixed distribution, yet we would like to guarantee that the algorithm is "learning" something. For example, online learning can be used to analyze

- Predicting clicks on banners on web pages;
- Routing on a network;
- Convergence to equilibrium of repeated games.

As we will see, it can *also* be used to analyze stochastic optimization algorithms, e.g., Stochastic Gradient Descent.

Let's now go back to our number-guessing game, and let's try a strategy to win it. Of course, this is one of the simplest examples of online learning, without a real application. Yet, going through it, we will uncover most of the key ingredients in online learning algorithms and their analysis.

A Winning Strategy. Can we win the number guessing game? We can, not only in the form we described it, but also assuming that the adversary picks his number *after* observing our choice. Moreover, we do not assume anything about how the adversary is deciding the numbers. In fact, the numbers can be chosen *adversarially*, that is, explicitly trying to make us lose the game. This is why we will call the mechanism generating the numbers the **adversary**.

Now, the fact that the numbers are adversarially chosen means that we can immediately rule out any strategy based on any statistical modeling of the data. In fact, it cannot work because the moment we estimate something and act on our estimate, the adversary can immediately change the way the data is generated, ruining us. So, we have to think about something else. Yet, surprisingly enough, many times online learning algorithms will look like classic ones from statistical estimation, even if they work for different reasons.

Now, let's try to design a strategy to make the regret provably sublinear in time, *regardless of how the adversary chooses the numbers*. The first thing we do is to take a look at the best strategy in hindsight, that is, the argmin of the second term of the regret. It should be immediate to see that

$$x_T^\star := \operatorname*{argmin}_{x\in[0,1]} \sum_{t=1}^{T}(x-y_t)^2 = \frac{1}{T}\sum_{t=1}^{T} y_t \,.$$

Now, given that we do not know the future, for sure, we cannot use $x_T^\star$ as our guess in each round. However, we do know the past, so a reasonable strategy in each round could be to output the best number over the past. Why would such a strategy work? For sure, the reason why it could work is not that we expect the future to be like the past, because it is not true! Instead, we want to leverage the fact that the optimal guess over time cannot change too much between rounds, so we can try to "track" it over time.

Hence, on each round $t \geq 2$, our strategy is to guess $x_t = x_{t-1}^\star = \frac{1}{t-1}\sum_{i=1}^{t-1} y_i$, and in the first round we guess any number between 0 and 1. Such a strategy is usually called **Follow-the-Leader (FTL)**, because you are following what would have been the optimal thing to do on the past rounds (i.e., the leader).

Let's now try to show that this strategy will allow us to win the game. Given that this is

a simple example, we will prove its regret guarantee using first principles, while in the next chapters we will introduce and use very general proof methods. First, we will need a small lemma.

Lemma 1.2 (Be-the-Leader Lemma) *Let $\mathcal{V} \subseteq \mathbb{R}^d$ and $\ell_t : \mathcal{V} \to \mathbb{R}$ be an arbitrary sequence of loss functions. Assume the existence of $\boldsymbol{x}_t^\star \in \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} \sum_{i=1}^t \ell_i(\boldsymbol{x})$, a minimizer in $\mathcal{V}$ of the cumulative loss over the first t rounds. Then, we have*

$$\sum_{t=1}^T \ell_t(\boldsymbol{x}_t^\star) \leq \sum_{t=1}^T \ell_t(\boldsymbol{x}_T^\star) \,.$$

Proof We prove it by induction on T. The base case is

$$\ell_1(\boldsymbol{x}_1^\star) \leq \ell_1(\boldsymbol{x}_1^\star),$$

which is trivially true. Now, for $T \geq 2$, we assume that $\sum_{t=1}^{T-1} \ell_t(\boldsymbol{x}_t^\star) \leq \sum_{t=1}^{T-1} \ell_t(\boldsymbol{x}_{T-1}^\star)$ is true, and we must prove the stated inequality, that is,

$$\sum_{t=1}^T \ell_t(\boldsymbol{x}_t^\star) \leq \sum_{t=1}^T \ell_t(\boldsymbol{x}_T^\star) \,.$$

This inequality is equivalent to

$$\sum_{t=1}^{T-1} \ell_t(\boldsymbol{x}_t^\star) \leq \sum_{t=1}^{T-1} \ell_t(\boldsymbol{x}_T^\star), \tag{1.3}$$

where we removed the last element of the sums because they are the same. Now observe that

$$\sum_{t=1}^{T-1} \ell_t(\boldsymbol{x}_t^\star) \leq \sum_{t=1}^{T-1} \ell_t(\boldsymbol{x}_{T-1}^\star),$$

by induction hypothesis, and

$$\sum_{t=1}^{T-1} \ell_t(\boldsymbol{x}_{T-1}^\star) \leq \sum_{t=1}^{T-1} \ell_t(\boldsymbol{x}_T^\star)$$

because $\boldsymbol{x}_{T-1}^\star$ is a minimizer of the l.h.s. in $\mathcal{V}$ and $\boldsymbol{x}_T^\star \in \mathcal{V}$. Chaining these two inequalities, we have that (1.3) is true, and so the lemma is proven. □

Basically, the above lemma quantifies the idea that knowing the future and being adaptive to it is typically better than not being adaptive to it.

With this lemma, we can now prove that the regret will grow sublinearly; in particular, it will be at most *logarithmic* in time. Note that we will not prove that our strategy is minimax optimal, even if it is possible to show that the logarithmic dependence on time is unavoidable for this problem.

Theorem 1.3 *Let $y_1, \dots, y_T \in [0,1]$ be an arbitrary sequence of real numbers. Let the algorithm's output be $x_t = x_{t-1}^\star := \frac{1}{t-1}\sum_{i=1}^{t-1} y_i$ for $t \geq 2$ and $x_1 = x_0^\star := 0.5$. Then, we have*

$$\operatorname{Regret}_T = \sum_{t=1}^T (x_t - y_t)^2 - \min_{x\in[0,1]} \sum_{t=1}^T (x - y_t)^2 \leq 4 + 4\ln T \,.$$

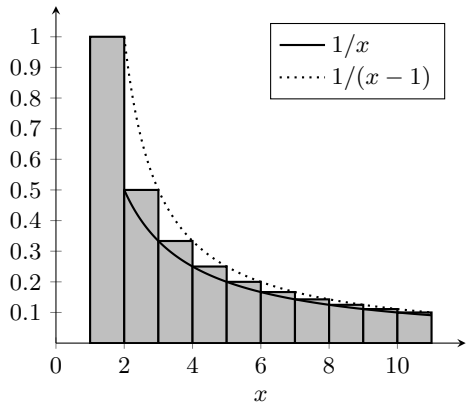


Figure 1.1 Upper bounding the sum with an integral.

Proof We use Lemma 1.2 to upper bound the regret:

$$\begin{aligned}\sum_{t=1}^{T}(x_t - y_t)^2 - \min_{x\in[0,1]} \sum_{t=1}^{T}(x - y_t)^2 &= \sum_{t=1}^{T}(x_{t-1}^\star - y_t)^2 - \sum_{t=1}^{T}(x_T^\star - y_t)^2 \\ &\le \sum_{t=1}^{T}(x_{t-1}^\star - y_t)^2 - \sum_{t=1}^{T}(x_t^\star - y_t)^2 \,.\end{aligned}$$

Now, let's take a look at each difference in the sum in the last equation. For $t = 1$, we have

$$(x_0^\star - y_1)^2 - (x_1^\star - y_1)^2 \le 0.5^2 \,.$$

For $t \ge 2$, we have that

$$\begin{aligned}(x_{t-1}^\star - y_t)^2 - (x_t^\star - y_t)^2 &= (x_{t-1}^\star)^2 - 2y_t x_{t-1}^\star - (x_t^\star)^2 + 2y_t x_t^\star \\ &= (x_{t-1}^\star + x_t^\star - 2y_t)(x_{t-1}^\star - x_t^\star) \\ &\le |x_{t-1}^\star + x_t^\star - 2y_t|\,|x_{t-1}^\star - x_t^\star| \le 2|x_{t-1}^\star - x_t^\star| \\ &= 2\left|\frac{1}{t-1}\sum_{i=1}^{t-1} y_i - \frac{1}{t}\sum_{i=1}^{t} y_i\right| \\ &= 2\left|\left(\frac{1}{t-1} - \frac{1}{t}\right)\sum_{i=1}^{t-1} y_i - \frac{y_t}{t}\right| \\ &\le 2\left|\frac{1}{t(t-1)}\sum_{i=1}^{t-1} y_i\right| + \frac{2|y_t|}{t} \le \frac{2}{t} + \frac{2|y_t|}{t} \le \frac{4}{t} \,.\end{aligned}$$

Hence, overall, we have

$$\sum_{t=1}^{T}(x_t - y_t)^2 - \min_{x\in[0,1]} \sum_{t=1}^{T}(x - y_t)^2 \le 0.25 + 4\sum_{t=2}^{T}\frac{1}{t} \le 4\sum_{t=1}^{T}\frac{1}{t} \,.$$

To upper bound the last sum, observe that we are trying to find an upper bound to the gray

area in Figure 1.1. As you can see from the picture, it can be upper bounded by 1 plus the integral of $\frac{1}{t-1}$ from 2 to $T+1$. So, we have

$$\sum_{t=1}^{T} \frac{1}{t} \leq 1 + \int_{2}^{T+1} \frac{1}{t-1} \, \mathrm{d}t = 1 + \ln T \, .$$

Putting everything together gives the stated bound. □

Let's write in words the steps of the proof: Lemma 1.2 allows us to upper bound the regret against the single best guess with the regret against the competitor sequence $x_1^\star, \dots, x_T^\star$. In turn, given that we produce in each round the prediction $x_{t-1}^\star$ and $|x_t^\star - x_{t-1}^\star|$ goes to zero like $1/t$, the total regret is sublinear in time.

There are a few things to stress about this strategy. The strategy does not have parameters to tune (e.g., learning rates, regularizers). Note that tuning parameters on validation data, as we typically do in machine learning, does not make sense in online learning: we have only one stream of data, and we cannot run our algorithm over it multiple times to select the best parameter! Also, this strategy does not need to maintain a complete record of the past, but only a "summary" of it, through the running average. This gives a computationally efficient algorithm. When we design online learning algorithms, we will strive to achieve all these characteristics. The final point to stress is that the algorithm does not use gradients: gradients are useful, and we will use them a lot, but they do not constitute the entire world of online learning.

Before going on, I want to remind the reader that, as seen above, this is different from the classic setting in statistical machine learning. So, for example, "overfitting" has no meaning here. The same holds for "generalization gap" and similar ideas linked to a training/testing scenario.

In the next chapters, we will introduce several algorithms for online learning, and one of them will be a strict generalization of the strategy we used in the example above.

1.1 History Bits

The concept of "regret" seems to have been proposed in Savage (1951), an exposition and review of the book by Wald (1950) on a foundation of statistical decision problems based on zero-sum two-person games. Savage (1951) introduces the idea of considering the difference between the utility of the best action in a given state and the utility obtained by any action under the same state. The proposed optimal strategy was the one minimizing such regret over the worst possible state. Savage (1951) called this concept "loss" and did not like the word "regret" because "that term seems to me charged with emotion and liable to lead to such misinterpretation as that the loss necessarily becomes known" (Savage, 1954, page 163). The name "regret" instead seems to have been suggested in Milnor (1951).

However, Savage's definition is a modification of the one proposed by Wald (1950), who instead proposed to maximize the utility directly, under the assumption that the utility of the best action for any state is 0. While minimizing the regret or minimizing the negative utility under the assumption of Wald (1950) are mathematically equivalent, Savage (1954, pages 169–170) explains that Wald considered the regret formulation different from what he proposed, while Savage attributed to his idea "little or no originality".

Extending the definition of Savage (1951, 1954) to a sequence of games, Hannan (1957) designed a randomized algorithm for zero-sum repeated games with a fixed loss matrix with a vanishing expected average regret. Hence, the concept of regret seems to originate from game theory, but, strangely enough, it passed through the work of two mathematical statisticians.

The Be-the-Leader lemma (Lemma 1.2) is due to Hannan (1957).

Exercises

Problem 1.1 *Extend the previous algorithm and analysis to the case when the adversary selects a vector $\boldsymbol{y}_t \in \mathbb{R}^d$ such that $\|\boldsymbol{y}_t\|_2 \leq 1$, the algorithm guesses a vector $\boldsymbol{x}_t \in \mathbb{R}^d$, and the loss function is $\|\boldsymbol{x}_t - \boldsymbol{y}_t\|_2^2$. Show an upper bound on the regret logarithmic in T and that does not depend on d. Among other things, you will probably need the Cauchy–Schwarz inequality: $|\langle \boldsymbol{x}, \boldsymbol{y} \rangle| \leq \|\boldsymbol{x}\|_2 \|\boldsymbol{y}\|_2$.*

Problem 1.2 *Consider the number-guessing game with absolute loss instead of squared loss: $\ell_t(x) = |x - y_t|$, where $x, y_t \in [0, 1]$. Show that the best fixed prediction in hindsight is any median of the sequence $y_1, \ldots, y_T$.*

2

Online Subgradient Descent

In this chapter, we will introduce the Online Subgradient Descent (OSD) algorithm: a generic online algorithm to solve online problems with convex losses. First, we will introduce Online Gradient Descent (OGD) for convex differentiable functions, then we will extend it to non-differentiable functions.

2.1 Online Learning with Convex Differentiable Losses

To summarize what we said in the first chapter, let's define online learning as the following general game:

- For $t = 1, \dots, T$
  - Output $\boldsymbol{x}_t \in \mathcal{V} \subseteq \mathbb{R}^d$
  - Pay the loss $\ell_t(\boldsymbol{x}_t)$, where $\ell_t : \mathcal{V} \to \mathbb{R}$
  - Receive some feedback on ℓ_t
- End for

The aim of this game is to minimize the regret with respect to any competitor $\boldsymbol{u} \in \mathcal{V}$:

$$\mathrm{Regret}_T(\boldsymbol{u}) := \sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \, .$$

We also said that the way the losses ℓ_t are decided is adversarial. Now, without making any additional assumptions, we cannot hope to solve this problem. Hence, we must understand what reasonable assumptions we can make. Typically, we will try to restrict the choice of the loss functions in some way. This is considered reasonable because most of the time, we have some say in deciding the set from which the loss functions are picked. So, for example, we will consider only *convex* loss functions. However, convexity might not be enough, so we might restrict the class a bit more to, for example, convex functions with bounded gradients. On the other hand, assuming knowledge of something about the future is not considered a reasonable assumption, because we very rarely have any control over the future. In general, the stronger the assumptions, the better the guarantee on the regret will be. The best algorithms we will see will guarantee a sublinear regret against the weakest assumption we can make, guaranteeing *at the same time* a smaller regret for *easy* adversaries.

It is also important to remember why minimizing regret is a good objective: given that we do not assume anything about how the adversary generates the loss functions, minimizing regret is a good metric that takes into account the difficulty of the problem. If an online

learning algorithm is able to guarantee a sublinear regret, it means that its performance on average will approach the performance of any fixed strategy. As said, we will see that in many situations, if the adversary is "weak", for example, it is a fixed stochastic distribution over the loss functions, being prepared for the worst-case scenario will not preclude us from getting the best guarantee anyway.

For a while, we will focus on the case that ℓ_t are convex, and this problem will be called **Online Convex Optimization (OCO)**. Later, we will see how to *convexify* some specific non-convex online problems.

Remark 2.1 *I will now introduce some math concepts. If you have a background in Convex Analysis, this will be easy stuff for you. On the other hand, if you have never seen these things before, they might look a bit scary. Let me tell you the right way to look at them:* these are tools that will make our job easier. *Without these tools, it would be basically impossible to design any online learning algorithm. And, no, it is not enough to test the algorithms on some machine learning dataset, because fixed datasets are not adversarial. Without a correct proof, you might not realize that your online algorithm fails on particular sequences of losses, as it happened with Adam (Reddi et al., 2018). I promise you that once you understand the key mathematical concepts, online learning is actually easy.*

2.1.1 Convex Analysis Bits: Convexity



Figure 2.1 Non-convex (left) and convex (right) sets.

Definition 2.2 *$\mathcal{V} \subseteq \mathbb{R}^d$ is **convex** if for any $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{V}$ and any $\lambda \in (0,1)$, we have $\lambda\boldsymbol{x} + (1-\lambda)\boldsymbol{y} \in \mathcal{V}$.*

In words, this means that the set $\mathcal{V}$ has no 'holes' or inward dents, see Figure 2.1.

We will make use of **extended-real-valued functions**, that is, functions that take values in $\mathbb{R} \cup \{-\infty, +\infty\}$. For f an extended-real-valued function on $\mathbb{R}^d$, its **domain** is the set $\operatorname{dom} f = \{\boldsymbol{x} \in \mathbb{R}^d : f(\boldsymbol{x}) < +\infty\}$.

Extended-real-valued functions allow us to easily consider constrained sets and are a standard notation in Convex Optimization (see, e.g., Boyd and Vandenberghe, 2004). For example, if I want the predictions of the algorithm $\boldsymbol{x}_t$ and the competitor $\boldsymbol{u}$ to be in a set $\mathcal{V} \subset \mathbb{R}^d$, I can just add $\iota_{\mathcal{V}}(\boldsymbol{x})$ to all the losses, where $\iota_{\mathcal{V}} : \mathbb{R}^d \to (-\infty, +\infty]$ is the **indicator function of the set** $\mathcal{V}$ defined as

$$\iota_{\mathcal{V}}(\boldsymbol{x}) = \begin{cases} 0, & \boldsymbol{x} \in \mathcal{V}, \\ +\infty, & \text{otherwise.} \end{cases}$$

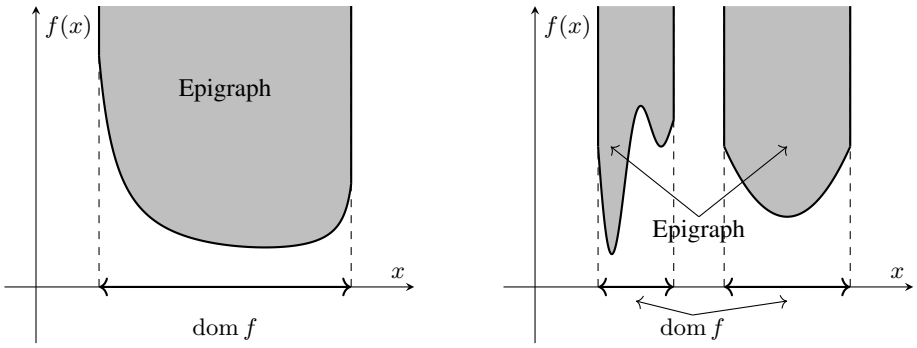


Figure 2.2 Convex (left) and nonconvex (right) functions.

In this way, the only way for the algorithm and for the competitor to suffer finite loss is to predict inside the set $\mathcal{V}$. Also, extended-real-valued functions will make the use of *Fenchel conjugates* more direct, see Section 6.4.1.

Convex functions will be an essential ingredient in online learning.

Definition 2.3 *Let $f : \mathbb{R}^d \to [-\infty, +\infty]$. f is **convex** if the epigraph of the function, $\{(\boldsymbol{x}, y) \in \mathbb{R}^{d+1} : y \geq f(\boldsymbol{x})\}$, is convex.*

We can see a visualization of this definition in Figure 2.2. Note that the definition implies that the domain of a convex function is convex. The $\iota_{\mathcal{V}}(\boldsymbol{x})$ is convex iff $\mathcal{V}$ is convex, so each convex set is associated with a convex function. Also, observe that if $f : \mathbb{R}^d \to (-\infty, +\infty]$ is convex, $f + \iota_{\mathcal{V}} : \mathbb{R}^d \to (-\infty, +\infty]$, where $\mathcal{V}$ is convex, is also convex.

The definition above gives rise to the following characterization for convex functions that do not assume the value $-\infty$.

Theorem 2.4 (Rockafellar, 1970, Theorem 4.1) *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be such that* $\operatorname{dom} f$ *is a convex set. Then f is convex iff, for any $0 < \lambda < 1$, we have*

$$f(\lambda \boldsymbol{x} + (1-\lambda)\boldsymbol{y}) \leq \lambda f(\boldsymbol{x}) + (1-\lambda) f(\boldsymbol{y}), \quad \forall \boldsymbol{x}, \boldsymbol{y} \in \operatorname{dom} f \, .$$

Example 2.5 Affine functions*, $f(\boldsymbol{x}) = \langle \boldsymbol{z}, \boldsymbol{x} \rangle + b$, are convex.*

Example 2.6 *Norms are always convex; the proof is left as an exercise.*

How to recognize a convex function? In the most general case, you have to rely on its definition. However, most of the time, we will recognize them as being obtained by operations that preserve convexity. For example:

- If f and g are convex, then their linear combination with non-negative weights is also convex.
- The composition with an affine transformation preserves the convexity.
- If $f : \mathbb{R}^d \to \mathbb{R}$ and $g : \mathbb{R} \to \mathbb{R}$ are convex functions and g is non-decreasing, then $h(\boldsymbol{x}) = g(f(\boldsymbol{x}))$ is convex.
- The pointwise supremum of convex functions is convex.

The proofs are left as exercises.

A very important property of *differentiable* convex functions is that we can construct a linear lower bound to the function.

Theorem 2.7 (Rockafellar, 1970, Theorem 25.1 and Corollary 25.1.1) *Suppose $f : \mathbb{R}^d \to (-\infty, +\infty]$ is a convex function and let $\boldsymbol{x} \in \operatorname{int} \operatorname{dom} f$. If f is differentiable at $\boldsymbol{x}$ then*

$$f(\boldsymbol{y}) \geq f(\boldsymbol{x}) + \langle \nabla f(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x} \rangle, \quad \forall \boldsymbol{y} \in \mathbb{R}^d \, .$$

We will also use the first-order optimality condition for differentiable convex functions:

Theorem 2.8 *Let $\mathcal{V}$ be a non-empty convex set, $\boldsymbol{x}^\star \in \mathcal{V}$, and f a function convex and differentiable over an open set that contains $\mathcal{V}$. Then, $\boldsymbol{x}^\star \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} f(\boldsymbol{x})$ iff $\langle \nabla f(\boldsymbol{x}^\star), \boldsymbol{y} - \boldsymbol{x}^\star \rangle \geq 0, \ \forall \boldsymbol{y} \in \mathcal{V}$.*

Proof Let us first assume that $\boldsymbol{x}^\star$ satisfies $\langle \nabla f(\boldsymbol{x}^\star), \boldsymbol{y} - \boldsymbol{x}^\star \rangle \geq 0, \ \forall \boldsymbol{y} \in \mathcal{V}$. Then, by Theorem 2.7, for any $\boldsymbol{y} \in \mathcal{V}$, we have that

$$f(\boldsymbol{y}) \geq f(\boldsymbol{x}^\star) + \langle \nabla f(\boldsymbol{x}^\star), \boldsymbol{y} - \boldsymbol{x}^\star \rangle \geq f(\boldsymbol{x}^\star),$$

that is, $\boldsymbol{x}^\star$ is the minimizer of f over $\mathcal{V}$.

Now, assume that $\boldsymbol{x}^\star$ is the minimizer of f over $\mathcal{V}$ and assume that there exists $\boldsymbol{y} \in \mathcal{V}$ such that $\langle \nabla f(\boldsymbol{x}^\star), \boldsymbol{y} - \boldsymbol{x}^\star \rangle < 0$. Consider $\boldsymbol{z}(\alpha) = \alpha \boldsymbol{y} + (1 - \alpha)\boldsymbol{x}^\star$ where $\alpha \in [0, 1]$. Note that $\boldsymbol{z}(\alpha) \in \mathcal{V}$ and denote by $h(\alpha) = f(\boldsymbol{z}(\alpha))$. We have that $h'(0) = \langle \nabla f(\boldsymbol{x}^\star), \boldsymbol{y} - \boldsymbol{x}^\star \rangle < 0$. Hence, by the definition of derivative, for $\alpha^\star > 0$ sufficiently small we have $h(\alpha^\star) < h(0)$, that is, $f(\boldsymbol{z}(\alpha^\star)) < f(\boldsymbol{x}^\star)$, which contradicts that $\boldsymbol{x}^\star$ is the minimizer over $\mathcal{V}$. □

In words, at the constrained minimum, the gradient makes an angle of 90° or less with all the feasible variations $\boldsymbol{y} - \boldsymbol{x}^\star$, hence we cannot decrease the function any further by moving inside $\mathcal{V}$. Moreover, if $\boldsymbol{x}^\star \in \operatorname{int} \mathcal{V}$, by choosing ϵ small enough such that $\boldsymbol{y} = \boldsymbol{x}^\star - \epsilon \nabla f(\boldsymbol{x}^\star) \in \mathcal{V}$, we obtain that $\boldsymbol{x}^\star$ is a minimum iff $\nabla f(\boldsymbol{x}^\star) = \boldsymbol{0}$.

Another critical property of convex functions is Jensen's inequality.

Theorem 2.9 (Jensen's inequality) *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be a measurable convex function and $\boldsymbol{x}$ be an $\mathbb{R}^d$-valued random element on some probability space such that $\mathbb{E}[\boldsymbol{x}]$ exists and $\boldsymbol{x} \in \operatorname{dom} f$ with probability 1. Then, we have*

$$\mathbb{E}[f(\boldsymbol{x})] \geq f(\mathbb{E}[\boldsymbol{x}]) \, .$$

We can now see our first OCO algorithm in the case that the functions are convex and differentiable.

2.1.2 Online Gradient Descent

In the first chapter, we saw a simple strategy to obtain a logarithmic regret in the guessing game. The strategy was to use the best prediction over the past, that is, the **Follow-the-Leader (FTL)** strategy. In formulas,

$$\boldsymbol{x}_t = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \sum_{i=1}^{t-1} \ell_i(\boldsymbol{x}),$$

Algorithm 2.1 Projected Online Gradient Descent (OGD)

Require: Non-empty closed convex set $\mathcal{V} \subseteq \mathbb{R}^d$, $\boldsymbol{x}_1 \in \mathcal{V}$, $\eta_1, \dots, \eta_T > 0$
1: **for** $t = 1$ **to** T **do**
2: Output $\boldsymbol{x}_t \in \mathcal{V}$
3: Pay the loss $\ell_t(\boldsymbol{x}_t)$, for a convex ℓ_t, differentiable on an open set containing $\mathcal{V}$
4: Set $\boldsymbol{g}_t = \nabla \ell_t(\boldsymbol{x}_t)$
5: $\boldsymbol{x}_{t+1} = \Pi_{\mathcal{V}}(\boldsymbol{x}_t - \eta_t \boldsymbol{g}_t) = \operatorname{argmin}_{\boldsymbol{y} \in \mathcal{V}} \|\boldsymbol{x}_t - \eta_t \boldsymbol{g}_t - \boldsymbol{y}\|_2$
6: **end for**

and, in the first round, we can play any admissible point. One might wonder if this strategy always works, but the answer is negative!

Example 2.10 (Failure of FTL) *Let $\mathcal{V} = [-1, 1]$ and consider the sequence of losses $\ell_t(x) = z_t x$, where*

$$z_1 = -0.5,$$
$$z_t = 1, \ t = 2, 4, \dots$$
$$z_t = -1, \ t = 3, 5, \dots$$

Then, apart from the first round where the prediction of FTL is arbitrary in $[-1, 1]$, the predictions of FTL will be $x_t = 1$ for t even and $x_t = -1$ for t odd. The cumulative loss of the FTL algorithm after T rounds will therefore be $T - 1 - \frac{x_1}{2}$ while the cumulative loss of the fixed solution $u = 0$ is 0. Thus, the regret of FTL with respect to $u = 0$ is $T - 1 - \frac{x_1}{2} \geq T - \frac{3}{2}$.

Hence, we will see an alternative strategy that guarantees sublinear regret for convex functions with bounded gradients. The strategy is called **Projected Online Gradient Descent**, or just **Online Gradient Descent (OGD)**, see Algorithm 2.1. In each step, we update the prediction of the algorithm by adding the negative gradient of the loss multiplied by a scalar $\eta_t > 0$ called the **learning rate** or **stepsize**. Then, we project the result back onto the feasible set $\mathcal{V}$. Some might notice that this algorithm is similar to Stochastic Gradient Descent, but it is not the same thing: here, the loss functions are different at each step, and they are not drawn from a fixed distribution but adversarially chosen. We will later see that OGD can *also* be used as Stochastic Gradient Descent.

First, we show the following two auxiliary results. The first proposition proves that Euclidean projections, the unique point $\Pi_{\mathcal{V}}(\boldsymbol{x}) := \operatorname{argmin}_{\boldsymbol{y} \in \mathcal{V}} \|\boldsymbol{x} - \boldsymbol{y}\|_2$, always decrease the distance to points inside the set.

Proposition 2.11 *Let $\boldsymbol{x} \in \mathbb{R}^d$ and $\boldsymbol{y} \in \mathcal{V}$, where $\mathcal{V} \subseteq \mathbb{R}^d$ is a non-empty closed convex set and define $\Pi_{\mathcal{V}}(\boldsymbol{x}) := \operatorname{argmin}_{\boldsymbol{y} \in \mathcal{V}} \|\boldsymbol{x} - \boldsymbol{y}\|_2$. Then, $\|\Pi_{\mathcal{V}}(\boldsymbol{x}) - \boldsymbol{y}\|_2 \leq \|\boldsymbol{x} - \boldsymbol{y}\|_2$.*

Proof First of all, observe that $\operatorname{argmin}_{\boldsymbol{y} \in \mathcal{V}} \|\boldsymbol{x} - \boldsymbol{y}\|_2 = \operatorname{argmin}_{\boldsymbol{y} \in \mathcal{V}} \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{y}\|_2^2$. So, from the optimality condition of Theorem 2.8 on the function $f(\boldsymbol{y}) = \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{y}\|_2^2$, we obtain

$$\langle \Pi_{\mathcal{V}}(\boldsymbol{x}) - \boldsymbol{x}, \boldsymbol{y} - \Pi_{\mathcal{V}}(\boldsymbol{x}) \rangle \geq 0 \, .$$

Therefore,

$$\begin{aligned}\|\boldsymbol{y}-\boldsymbol{x}\|_2^2 &= \|\boldsymbol{y}-\Pi_{\mathcal{V}}(\boldsymbol{x})+\Pi_{\mathcal{V}}(\boldsymbol{x})-\boldsymbol{x}\|_2^2 \\ &= \|\boldsymbol{y}-\Pi_{\mathcal{V}}(\boldsymbol{x})\|_2^2+2\langle \boldsymbol{y}-\Pi_{\mathcal{V}}(\boldsymbol{x}),\Pi_{\mathcal{V}}(\boldsymbol{x})-\boldsymbol{x}\rangle+\|\Pi_{\mathcal{V}}(\boldsymbol{x})-\boldsymbol{x}\|_2^2 \\ &\geq \|\boldsymbol{y}-\Pi_{\mathcal{V}}(\boldsymbol{x})\|_2^2 .\end{aligned}$$

□

The next lemma upper bounds the regret in a single iteration of Algorithm 2.1.

Lemma 2.12 *Let $\mathcal{V}\subseteq\mathbb{R}^d$ be a non-empty closed convex set, ℓ_t be convex and differentiable on an open set containing $\mathcal{V}$, and $\eta_t>0$. Set $\boldsymbol{g}_t=\nabla\ell_t(\boldsymbol{x}_t)$ and $\boldsymbol{x}_{t+1}=\Pi_{\mathcal{V}}(\boldsymbol{x}_t-\eta_t\boldsymbol{g}_t)$. Then, $\forall\boldsymbol{u}\in\mathcal{V}$, the following inequality holds*

$$\eta_t(\ell_t(\boldsymbol{x}_t)-\ell_t(\boldsymbol{u}))\leq\eta_t\langle\boldsymbol{g}_t,\boldsymbol{x}_t-\boldsymbol{u}\rangle\leq\frac{1}{2}\|\boldsymbol{x}_t-\boldsymbol{u}\|_2^2-\frac{1}{2}\|\boldsymbol{x}_{t+1}-\boldsymbol{u}\|_2^2+\frac{\eta_t^2}{2}\|\boldsymbol{g}_t\|_2^2 .$$

Proof From Proposition 2.11 and Theorem 2.7, we have that

$$\begin{aligned}\|\boldsymbol{x}_{t+1}-\boldsymbol{u}\|_2^2-\|\boldsymbol{x}_t-\boldsymbol{u}\|_2^2 &\leq \|\boldsymbol{x}_t-\eta_t\boldsymbol{g}_t-\boldsymbol{u}\|_2^2-\|\boldsymbol{x}_t-\boldsymbol{u}\|_2^2 \\ &= -2\eta_t\langle\boldsymbol{g}_t,\boldsymbol{x}_t-\boldsymbol{u}\rangle+\eta_t^2\|\boldsymbol{g}_t\|_2^2 \\ &\leq -2\eta_t(\ell_t(\boldsymbol{x}_t)-\ell_t(\boldsymbol{u}))+\eta_t^2\|\boldsymbol{g}_t\|_2^2 .\end{aligned}$$

Reordering, we have the stated bound. □

We can prove the following regret guarantee for Algorithm 2.1.

Theorem 2.13 *Let $\mathcal{V}\subseteq\mathbb{R}^d$ be a non-empty closed convex set with diameter D, that is, $D:=\sup_{\boldsymbol{x},\boldsymbol{y}\in\mathcal{V}}\|\boldsymbol{x}-\boldsymbol{y}\|_2$. Let $\ell_1,\ldots,\ell_T$ be an arbitrary sequence of convex and differentiable functions on an open set containing $\mathcal{V}$. Pick any $\boldsymbol{x}_1\in\mathcal{V}$ and assume $0<\eta_{t+1}\leq\eta_t,\ t=1,\ldots,T-1$. Then, $\forall\boldsymbol{u}\in\mathcal{V}$, Algorithm 2.1 satisfies the following regret bound*

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t)-\ell_t(\boldsymbol{u}))\leq\frac{D^2}{2\eta_T}+\sum_{t=1}^{T}\frac{\eta_t}{2}\|\boldsymbol{g}_t\|_2^2-\frac{1}{2\eta_T}\|\boldsymbol{x}_{T+1}-\boldsymbol{u}\|_2^2 .$$

Moreover, if η_t is constant, i.e., $\eta_t=\eta\ \forall t=1,\ldots,T$, for all $\boldsymbol{u}\in\mathcal{V}$, we have

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t)-\ell_t(\boldsymbol{u}))\leq\frac{\|\boldsymbol{u}-\boldsymbol{x}_1\|_2^2}{2\eta}+\frac{\eta}{2}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_2^2-\frac{1}{2\eta}\|\boldsymbol{x}_{T+1}-\boldsymbol{u}\|_2^2 .$$

Proof Dividing the inequality in Lemma 2.12 by η_t and summing over $t=1,\ldots,T$, we

have

$$\begin{aligned}
\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) &\leq \sum_{t=1}^{T}\left(\frac{\|\boldsymbol{x}_t - \boldsymbol{u}\|_2^2}{2\eta_t} - \frac{\|\boldsymbol{x}_{t+1} - \boldsymbol{u}\|_2^2}{2\eta_t}\right) + \sum_{t=1}^{T}\frac{\eta_t\|\boldsymbol{g}_t\|_2^2}{2} \\
&= \frac{\|\boldsymbol{x}_1 - \boldsymbol{u}\|_2^2}{2\eta_1} - \frac{\|\boldsymbol{x}_{T+1} - \boldsymbol{u}\|_2^2}{2\eta_T} + \sum_{t=1}^{T-1}\left(\frac{1}{\eta_{t+1}} - \frac{1}{\eta_t}\right)\frac{\|\boldsymbol{x}_{t+1} - \boldsymbol{u}\|_2^2}{2} + \sum_{t=1}^{T}\frac{\eta_t\|\boldsymbol{g}_t\|_2^2}{2} \\
&\leq \frac{D^2}{2\eta_1} + \frac{D^2}{2}\sum_{t=1}^{T-1}\left(\frac{1}{\eta_{t+1}} - \frac{1}{\eta_t}\right) + \sum_{t=1}^{T}\frac{\eta_t\|\boldsymbol{g}_t\|_2^2}{2} - \frac{\|\boldsymbol{x}_{T+1} - \boldsymbol{u}\|_2^2}{2\eta_T} \\
&= \frac{D^2}{2\eta_1} + D^2\left(\frac{1}{2\eta_T} - \frac{1}{2\eta_1}\right) + \sum_{t=1}^{T}\frac{\eta_t\|\boldsymbol{g}_t\|_2^2}{2} - \frac{\|\boldsymbol{x}_{T+1} - \boldsymbol{u}\|_2^2}{2\eta_T} \\
&= \frac{D^2}{2\eta_T} + \sum_{t=1}^{T}\frac{\eta_t}{2}\|\boldsymbol{g}_t\|_2^2 - \frac{1}{2\eta_T}\|\boldsymbol{x}_{T+1} - \boldsymbol{u}\|_2^2 .
\end{aligned}$$

In the same way, when η_t is constant, we have

$$\begin{aligned}
\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) &\leq \sum_{t=1}^{T}\left(\frac{1}{2\eta}\|\boldsymbol{x}_t - \boldsymbol{u}\|_2^2 - \frac{1}{2\eta}\|\boldsymbol{x}_{t+1} - \boldsymbol{u}\|_2^2\right) + \frac{\eta}{2}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_2^2 \\
&= \frac{1}{2\eta}\|\boldsymbol{x}_1 - \boldsymbol{u}\|_2^2 - \frac{1}{2\eta}\|\boldsymbol{x}_{T+1} - \boldsymbol{u}\|_2^2 + \frac{\eta}{2}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_2^2 .
\end{aligned}$$

□

We can immediately observe a few things.

- The terms $\|\boldsymbol{g}_t\|_2^2$ are due to the fact that we output $\boldsymbol{x}_t$ before knowing the loss ℓ_t. Indeed, these terms become non-positive if we were allowed to observe ℓ_t before producing $\boldsymbol{x}_t$, see Section 15.5.1.
- In the case of constant η, the bound holds even when $D = \infty$. Moreover, the first term in the regret depends on our *initial condition*, that is, how far $\boldsymbol{x}_1$ is from the competitor $\boldsymbol{u}$.
- The last term in both bounds is non-positive, so it is usually discarded. Yet, some advanced proofs require it, so it is good to be aware of it.
- If we want to use time-varying learning rates, we need to control the terms $\|\boldsymbol{x}_t - \boldsymbol{u}\|_2^2$. To do it, we assume a bounded domain $\mathcal{V}$. However, this assumption is false in most of the machine learning applications. We will also see in Section 5.2 that this is not an artifact of the proof, that is, OGD can fail in unbounded domains with time-varying learning rates. That said, in the stochastic setting, you can still use a time-varying learning rate in SGD with an unbounded domain, as we will see in Chapter 3.
- Another important observation is that the regret bound helps us choose the learning rates η_t. Indeed, it is the only guideline we have. Any other choice that is not justified by a regret analysis is not justified at all.

As we said, the presence of untuned parameters like the learning rates makes no sense in online learning. So, we have to decide on a strategy to set them. A simple choice is to find the constant learning rate that minimizes the bounds for a fixed number of iterations. We

have to consider the expression

$$\frac{\|\boldsymbol{u}-\boldsymbol{x}_1\|_2^2}{2\eta}+\frac{\eta}{2}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_2^2$$

and minimize with respect to η. It is easy to see that the optimal η is $\frac{\|\boldsymbol{u}-\boldsymbol{x}_1\|_2}{\sqrt{\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_2^2}}$, that would give the regret bound

$$\|\boldsymbol{u}-\boldsymbol{x}_1\|_2\sqrt{\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_2^2}\,.$$

However, we have a problem: in order to use this stepsize, we should know all the future gradients (that depend on η itself!), and the distance between the optimal solution and the initial point. This is clearly impossible: remember that the adversary can choose the sequence of functions. Hence, it can observe your choice of learning rates and decide the sequence so that your learning rate is no longer the correct one!

Indeed, it turns out that this kind of rate is completely impossible because it is ruled out by a lower bound we will show in Chapter 5. Yet, we will see that it is indeed possible to achieve very similar rates using *adaptive* (Section 4.2) and *parameter-free* algorithms (Chapter 13). For the moment, we can observe that we might be happy to minimize a loose upper bound. In particular, assume that the norm of the gradients is bounded by L, that is $\|\boldsymbol{g}_t\|_2 \leq L$. Also, assuming a bounded diameter, we can upper bound $\|\boldsymbol{u}-\boldsymbol{x}_1\|_2$ by D. Hence, we have

$$\eta^\star = \operatorname*{argmin}_{\eta} \frac{D^2}{2\eta}+\frac{\eta L^2 T}{2} = \frac{D}{L\sqrt{T}},$$

that gives a regret bound of

$$DL\sqrt{T}\,. \tag{2.1}$$

So, indeed, the regret is sublinear in time. In Chapter 5, we will also prove that the this regret upper bound is optimal up to constant multiplicative factors.

Example 2.14 *Consider the guessing game of the first chapter; we can easily solve it with OGD. Indeed, we just need to calculate the gradients, prove that they are bounded, and find a way to calculate the projection of a real number in* $[0,1]$*. So,* $\ell_t'(x) = 2(x-y_t)$*, that is bounded for* $x, y_t \in [0,1]$*. The projection on* $[0,1]$ *is just* $\Pi_{[0,1]}(x) = \min(\max(x,0),1)$*. With the optimal learning rate, the resulting regret would be* $\mathcal{O}(\sqrt{T})$*, which is worse than the one we found in the first chapter. Hence, OGD with this choice of the learning rate is suboptimal for this problem. We will see how to make it optimal in Chapter 4.*

Example 2.15 *Let's consider an example of the OCO setting. Consider the problem of predicting at day* t *the opening price of a stock based on a linear combination of the opening prices of the past* d *days, represented by a vector* $\boldsymbol{z}_t \in \mathbb{R}^d$*. So, our prediction at time* t *will be* $\langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle$ *and the feasible set is* $\mathcal{V} = \mathbb{R}^d$*. Once we make our prediction, we receive the true opening price* y_t*, and we pay the* ***Huber loss****, a convex loss function robust to outliers,*

on the difference between our prediction and the opening price:

$$\ell_t(\boldsymbol{x}) = \begin{cases} \frac{1}{2}(\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle - y_t)^2, & \text{for } |\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle - y_t| \le \delta, \\ \delta(|\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle - y_t| - \frac{1}{2}\delta), & \text{otherwise} \end{cases}$$

where δ is the threshold that decides what counts as an outlier, and it should be penalized with a linear error rather than a quadratic one. For this problem, we can use OGD with constant learning rate $\eta \propto 1/\sqrt{T}$. So, we just need to calculate the gradients:

$$\nabla\ell_t(\boldsymbol{x}) = \begin{cases} (\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle - y_t)\boldsymbol{z}_t, & \text{for } |\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle - y_t| \le \delta, \\ \delta\mathrm{sign}(\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle - y_t)\boldsymbol{z}_t, & \text{otherwise.} \end{cases}$$

Assuming that all $\|\boldsymbol{z}_t\|_2$ are bounded, we also have that the gradients are bounded, satisfying the assumptions of Theorem 2.13. Hence, running OGD, on average, the performance of the algorithm will approach the performance of any fixed linear predictor.

2.2 Online Subgradient Descent

In the previous section, we introduced Projected Online Gradient Descent. However, the differentiability assumption for the ℓ_t is quite strong. What happens when the losses are convex but not differentiable? For example, $\ell_t(x) = |x-10|$. Note that this situation is more common than one would think. For example, the hinge loss, $\ell_t(\boldsymbol{x}) = \max(1-y_t\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle, 0)$, is not differentiable. It turns out that we can just use OGD, substituting *subgradients* for the gradients. For this, we need some more convex analysis.

2.2.1 Convex Analysis Bits: Subgradients

First, we need a couple of technical definitions.

Definition 2.16 *A function $f : \mathbb{R}^d \to [-\infty, +\infty]$ is **closed** iff $\{\boldsymbol{x} : f(\boldsymbol{x}) \le \alpha\}$ is closed for every $\alpha \in \mathbb{R}$.*

Note that in any Euclidean space (and more generally in any Hausdorff space) a function is closed iff it is lower semicontinuous (Bauschke and Combettes, 2017, Lemma 1.24).

Example 2.17 *The indicator function of a set $\mathcal{V} \subset \mathbb{R}^d$ is closed iff $\mathcal{V}$ is closed.*

Definition 2.18 *If a function f is nowhere $-\infty$ and finite somewhere, then f is called **proper**.*

In this book, we are mainly interested in convex proper functions, which better conform to our intuition of what a convex function looks like.

Example 2.19 *The indicator function of a set $\mathcal{V} \subset \mathbb{R}^d$ is proper iff $\mathcal{V}$ is non-empty.*

Let's now define formally what a subgradient is.

Definition 2.20 *For a proper function $f : \mathbb{R}^d \to (-\infty, +\infty]$, we define a **subgradient** of f in $\boldsymbol{x} \in \mathbb{R}^d$ as a vector $\boldsymbol{g} \in \mathbb{R}^d$ that satisfies*

$$f(\boldsymbol{y}) \ge f(\boldsymbol{x}) + \langle \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{x}\rangle, \quad \forall \boldsymbol{y} \in \mathbb{R}^d\,.$$

Basically, a subgradient of f in $\boldsymbol{x}$ is any vector $\boldsymbol{g}$ that allows us to construct a linear lower bound to f. Note that the subgradient might not be unique, so we denote the *set* of subgradients of f in $\boldsymbol{x}$ by $\partial f(\boldsymbol{x})$, called the **subdifferential of f at $\boldsymbol{x}$**. A function f is **subdifferentiable** at $\boldsymbol{x}$ if $\partial f(\boldsymbol{x}) \neq \emptyset$.

Observe that if f is proper and convex, then $\partial f(\boldsymbol{x})$ is empty for $\boldsymbol{x} \notin \operatorname{dom} f$, because the inequality cannot be satisfied when $f(\boldsymbol{x}) = +\infty$. Also, the domain of ∂f, denoted by $\operatorname{dom} \partial f$, is the set of all $\boldsymbol{x} \in \mathbb{R}^d$ such that $\partial f(\boldsymbol{x})$ is non-empty; it is a subset of $\operatorname{dom} f$. A proper convex function f is always[1] subdifferentiable on $\operatorname{int} \operatorname{dom} f$ (Rockafellar, 1970, Theorem 23.4).

Note that we did not assume the function f to be convex in the definition of a subgradient. However, the following theorem tells us that if we have subgradients everywhere, then the function must be convex.

Theorem 2.21 *Let $f : \mathbb{R}^d \to \mathbb{R}$ and $\mathcal{V} \subseteq \mathbb{R}^d$ be a convex set. If $\partial f(\boldsymbol{x}) \neq \emptyset$ for all $\boldsymbol{x} \in \mathcal{V}$, then the restriction of f to $\mathcal{V}$ is convex.*

Proof For any $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{V}$ and for any $\lambda \in [0, 1]$, consider $\boldsymbol{y} = \lambda \boldsymbol{x}_1 + (1 - \lambda)\boldsymbol{x}_2 \in \mathcal{V}$ and $\boldsymbol{g} \in \partial f(\boldsymbol{y})$. Then, we have

$$f(\boldsymbol{x}_1) \geq f(\boldsymbol{y}) + \langle \boldsymbol{g}, \boldsymbol{x}_1 - \boldsymbol{y} \rangle = f(\boldsymbol{y}) + (1 - \lambda)\langle \boldsymbol{g}, \boldsymbol{x}_1 - \boldsymbol{x}_2 \rangle,$$
$$f(\boldsymbol{x}_2) \geq f(\boldsymbol{y}) + \langle \boldsymbol{g}, \boldsymbol{x}_2 - \boldsymbol{y} \rangle = f(\boldsymbol{y}) + \lambda\langle \boldsymbol{g}, \boldsymbol{x}_2 - \boldsymbol{x}_1 \rangle \ .$$

Multiplying the first inequality by λ and the second one by $1 - \lambda$ and adding them together, we have

$$\lambda f(\boldsymbol{x}_1) + (1 - \lambda) f(\boldsymbol{x}_2) \geq f(\boldsymbol{y}) = f(\lambda \boldsymbol{x}_1 + (1 - \lambda)\boldsymbol{x}_2),$$

that implies the convexity of f in $\mathcal{V}$. □

The unique subgradient of a differentiable function is just the gradient, as quantified in the next theorem.

Theorem 2.22 (Rockafellar, 1970, Theorem 25.1) *If the function $f : \mathbb{R}^d \to [-\infty, +\infty]$ is convex and finite in $\boldsymbol{x}$, it is differentiable in $\boldsymbol{x}$ iff the subdifferential consists of a single element, that turns out to be $\nabla f(\boldsymbol{x})$.*

We can also calculate subgradients of the sum of functions.

Theorem 2.23 *Let $f_1, \ldots, f_m$ be proper functions on $\mathbb{R}^d$, and $F = f_1 + \cdots + f_m$. Then, $\partial F(\boldsymbol{x}) \supseteq \partial f_1(\boldsymbol{x}) + \cdots + \partial f_m(\boldsymbol{x}), \forall \boldsymbol{x}$. Note that if one of the sets $\partial f_i(\boldsymbol{x})$ is empty, the right-hand side is empty and the inclusion is vacuous. Moreover, if $f_1, \ldots, f_m$ are also convex, closed, and $\operatorname{dom} f_m \cap \bigcap_{i=1}^{m-1} \operatorname{int} \operatorname{dom} f_i \neq \emptyset$, then actually $\partial F(\boldsymbol{x}) = \partial f_1(\boldsymbol{x}) + \cdots + \partial f_m(\boldsymbol{x}), \forall \boldsymbol{x}$.*

Proof For any $\boldsymbol{z}$, when the right-hand side is non-empty, choose $\boldsymbol{g}_i \in \partial f_i(\boldsymbol{z})$ for $i = 1, \ldots, m$. From the definition of subgradient, we have

$$F(\boldsymbol{x}) = \sum_{i=1}^{m} f_i(\boldsymbol{x}) \geq \sum_{i=1}^{m} (f_i(\boldsymbol{z}) + \langle \boldsymbol{g}_i, \boldsymbol{x} - \boldsymbol{z} \rangle) = F(\boldsymbol{z}) + \left\langle \sum_{i=1}^{m} \boldsymbol{g}_i, \boldsymbol{x} - \boldsymbol{z} \right\rangle \ .$$

[1] The stronger version of this theorem uses the *relative interior* of the domain of f, however, we never need to use it, so we will keep it simple.

Hence, $\sum_{i=1}^m \boldsymbol{g}_i \in \partial F(\boldsymbol{z})$. Since this holds for every choice of $\boldsymbol{g}_i \in \partial f_i(\boldsymbol{z})$, we have $\partial F(\boldsymbol{z}) \supseteq \partial f_1(\boldsymbol{z}) + \cdots + \partial f_m(\boldsymbol{z})$.

For the second statement, see Bauschke and Combettes (2017, Corollary 16.50). □

Example 2.24 *Let $f(x) = |x|$. Then, the subdifferential set $\partial f(x)$ is*

$$\partial f(x) = \begin{cases} \{1\}, & x > 0, \\ [-1, 1], & x = 0, \\ \{-1\}, & x < 0\,. \end{cases}$$

Example 2.25 *Let's calculate the subgradient of the indicator function for a non-empty convex set $\mathcal{V} \subset \mathbb{R}^d$. By definition, $\boldsymbol{g} \in \partial \iota_{\mathcal{V}}(\boldsymbol{x})$ if*

$$\iota_{\mathcal{V}}(\boldsymbol{y}) \geq \iota_{\mathcal{V}}(\boldsymbol{x}) + \langle \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{x} \rangle, \quad \forall \boldsymbol{y} \in \mathbb{R}^d\,.$$

*This condition implies that $\boldsymbol{x} \in \mathcal{V}$ and $0 \geq \langle \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{x} \rangle$ for all $\boldsymbol{y} \in \mathcal{V}$ (because for $\boldsymbol{y} \notin \mathcal{V}$ the inequality is always verified). The set of all $\boldsymbol{g}$ that satisfy the above inequality is called the **normal cone to $\mathcal{V}$ at $\boldsymbol{x}$** and it is denoted by $\mathcal{N}_{\mathcal{V}}(\boldsymbol{x})$. Note that the normal cone for any $\boldsymbol{x} \in \operatorname{int} \mathcal{V}$ is $\{\boldsymbol{0}\}$ (Hint: take $\boldsymbol{y} = \boldsymbol{x} + \epsilon \boldsymbol{g}$). For example, for $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \leq 1\}$, $\mathcal{N}_{\mathcal{V}}(\boldsymbol{x}) = \{\alpha \boldsymbol{x} : \alpha \geq 0\}$ for all $\boldsymbol{x} : \|\boldsymbol{x}\|_2 = 1$.*

Another useful theorem is to calculate the subdifferential of the pointwise maximum of convex functions.

Theorem 2.26 (Bauschke and Combettes, 2017, Theorem 18.5) *Let $(f_i)_{i \in I}$ be a finite set of proper convex functions from $\mathbb{R}^d$ to $(-\infty, +\infty]$. Suppose $\boldsymbol{x} \in \bigcap_{i \in I} \operatorname{dom} f_i$ and each f_i is continuous at $\boldsymbol{x}$. Set $F = \max_{i \in I} f_i$ and let $A(\boldsymbol{x}) = \{i \in I | f_i(\boldsymbol{x}) = F(\boldsymbol{x})\}$ be the set of the active functions. Then, denoting by* conv *the **convex hull**, we have*

$$\partial F(\boldsymbol{x}) = \operatorname{conv} \bigcup_{i \in A(\boldsymbol{x})} \partial f_i(\boldsymbol{x})\,.$$

Example 2.27 (Subgradients of the Hinge loss) *Consider the hinge loss $\ell(\boldsymbol{x}) = \max(1 - \langle \boldsymbol{z}, \boldsymbol{x} \rangle, 0)$ for $\boldsymbol{z} \in \mathbb{R}^d$. The subdifferential set is*

$$\partial \ell(\boldsymbol{x}) = \begin{cases} \{\boldsymbol{0}\}, & 1 - \langle \boldsymbol{z}, \boldsymbol{x} \rangle < 0 \\ \{-\alpha \boldsymbol{z} : \alpha \in [0, 1]\}, & 1 - \langle \boldsymbol{z}, \boldsymbol{x} \rangle = 0 \\ \{-\boldsymbol{z}\}, & \textit{otherwise} \end{cases}.$$

Finally, we can show a result on the subgradient of affine transformations.

Theorem 2.28 *Let $f : \mathbb{R}^m \to (-\infty, +\infty]$ be proper. Define $h(\boldsymbol{x}) = f(\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b})$, where $\boldsymbol{A} \in \mathbb{R}^{m \times d}$ and $\boldsymbol{b} \in \mathbb{R}^m$. Then, we have $\boldsymbol{A}^\top \partial f(\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}) \subseteq \partial h(\boldsymbol{x})$.*

Proof For any $\boldsymbol{g} \in \partial f(\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b})$, we want to show that $\boldsymbol{A}^\top \boldsymbol{g}$ is a subgradient of h in $\boldsymbol{x}$. From the definition of subgradient, for all $\boldsymbol{y} \in \mathbb{R}^d$ we have

$$\begin{aligned} h(\boldsymbol{y}) &= f(\boldsymbol{A}\boldsymbol{y} + \boldsymbol{b}) \geq f(\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}) + \langle \boldsymbol{g}, \boldsymbol{A}\boldsymbol{y} + \boldsymbol{b} - (\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}) \rangle \\ &= h(\boldsymbol{x}) + \langle \boldsymbol{A}^\top \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{x} \rangle, \end{aligned}$$

that implies our stated result. □

We also have a handy result that upper bounds the norm of subgradients of convex *Lipschitz* functions.

Definition 2.29 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$. We say that f is **L-Lipschitz** over a set $\mathcal{V} \subseteq \operatorname{dom} f$ with respect to a norm $\|\cdot\|$ if $|f(\boldsymbol{x}) - f(\boldsymbol{y})| \leq L\|\boldsymbol{x} - \boldsymbol{y}\|$, $\forall \boldsymbol{x}, \boldsymbol{y} \in \mathcal{V}$.*

Theorem 2.30 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be convex and proper. Then, f is L-Lipschitz in $\operatorname{int}\operatorname{dom} f$ with respect to the L_2 norm iff for all $\boldsymbol{x} \in \operatorname{int}\operatorname{dom} f$ and $\boldsymbol{g} \in \partial f(\boldsymbol{x})$ we have $\|\boldsymbol{g}\|_2 \leq L$.*

Proof First, we assume f to be L-Lipschitz, then $|f(\boldsymbol{x}) - f(\boldsymbol{y})| \leq L\|\boldsymbol{x} - \boldsymbol{y}\|_2$, $\forall \boldsymbol{x}, \boldsymbol{y} \in \operatorname{int}\operatorname{dom} f$. Let $\boldsymbol{g} \in \partial f(\boldsymbol{x})$. If $\|\boldsymbol{g}\|_2 = 0$, then the theorem is true. So, we can safely assume it to be different from 0. For $\epsilon > 0$ small enough $\boldsymbol{y} = \boldsymbol{x} + \epsilon \frac{\boldsymbol{g}}{\|\boldsymbol{g}\|_2} \in \operatorname{int}\operatorname{dom} f$, then

$$L\epsilon = L\|\boldsymbol{x} - \boldsymbol{y}\|_2 \geq |f(\boldsymbol{y}) - f(\boldsymbol{x})| \geq f(\boldsymbol{y}) - f(\boldsymbol{x}) \geq \langle \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{x} \rangle = \epsilon \|\boldsymbol{g}\|_2,$$

that implies that $\|\boldsymbol{g}\|_2 \leq L$.

For the other implication, the subgradient definition and Cauchy–Schwarz inequality give

$$f(\boldsymbol{x}) - f(\boldsymbol{y}) \leq \|\boldsymbol{g}\|_2 \|\boldsymbol{x} - \boldsymbol{y}\|_2 \leq L\|\boldsymbol{x} - \boldsymbol{y}\|_2,$$

for any $\boldsymbol{x}, \boldsymbol{y} \in \operatorname{int}\operatorname{dom} f$. Taking $\boldsymbol{g} \in \partial f(\boldsymbol{y})$, we also get

$$f(\boldsymbol{y}) - f(\boldsymbol{x}) \leq L\|\boldsymbol{x} - \boldsymbol{y}\|_2 \,. \qquad \square$$

Finally, let's dispel the common misconception that a convex function must be differentiable on its domain except possibly at countably many points. For example, consider the convex function $f : \mathbb{R}^2 \to \mathbb{R}$ defined as $f(\boldsymbol{x}) = |x_1|$ that is not differentiable on the line segment between $\boldsymbol{v} = (0, 0)$ and $\boldsymbol{w} = (0, 1)$.

2.2.2 Analysis with Subgradients

As I promised you, with the proper mathematical tools, analyzing online algorithms becomes easy. Indeed, switching from gradient to subgradient comes for free! In fact, our analysis of OGD with differentiable losses holds as is when using subgradients instead of gradients. The reason is that the only property of the gradients that we used in the proof of Theorem 2.13 was that

$$\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u}) \leq \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle,$$

where $\boldsymbol{g}_t = \nabla \ell_t(\boldsymbol{x}_t)$. However, the exact same property holds when $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$. Hence, the regret bounds we proved hold as well, just replacing differentiability with subdifferentiability and gradients with subgradients. In particular, we have the following lemma.

Lemma 2.31 *Let $\mathcal{V} \subseteq \mathbb{R}^d$ be a non-empty closed convex set, $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ a function subdifferentiable on $\mathcal{V}$, and $\eta_t > 0$. Set $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$ and $\boldsymbol{x}_{t+1} = \Pi_{\mathcal{V}}(\boldsymbol{x}_t - \eta_t \boldsymbol{g}_t)$. Then, $\forall \boldsymbol{u} \in \mathcal{V}$, we have*

$$\eta_t(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \eta_t \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \leq \frac{1}{2}\|\boldsymbol{x}_t - \boldsymbol{u}\|_2^2 - \frac{1}{2}\|\boldsymbol{x}_{t+1} - \boldsymbol{u}\|_2^2 + \frac{\eta_t^2}{2}\|\boldsymbol{g}_t\|_2^2 \,.$$

Algorithm 2.2 Projected Online Subgradient Descent (OSD)

Require: Non-empty closed convex set $\mathcal{V} \subseteq \mathbb{R}^d$, $\boldsymbol{x}_1 \in \mathcal{V}$, $\eta_1, \ldots, \eta_T > 0$
1: **for** $t = 1$ **to** T **do**
2: Output $\boldsymbol{x}_t \in \mathcal{V}$
3: Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable on $\mathcal{V}$
4: Set $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$
5: $\boldsymbol{x}_{t+1} = \Pi_{\mathcal{V}}(\boldsymbol{x}_t - \eta_t \boldsymbol{g}_t) = \operatorname{argmin}_{\boldsymbol{y} \in \mathcal{V}} \|\boldsymbol{x}_t - \eta_t \boldsymbol{g}_t - \boldsymbol{y}\|_2$
6: **end for**

So, we can state the **Online Subgradient Descent (OSD)** algorithm in Algorithm 2.2, where the only difference from Algorithm 2.1 is line 4.

Example 2.32 *Consider the guessing game of the first chapter again, but now change the loss function to the absolute loss of the difference: $\ell_t(x) = |x - y_t|$. Now we will need to use OSD, because the functions are non-differentiable. We can easily see that*

$$\partial \ell_t(x) = \begin{cases} \{1\}, & x > y_t \\ [-1, 1], & x = y_t \\ \{-1\}, & x < y_t \, . \end{cases}$$

Again, running OSD with the optimal learning rate on this problem will give us immediately a regret of $\mathcal{O}(\sqrt{T})$ as $T \to \infty$, without having to design a specific strategy for it.

2.2.3 Unit Analysis Bits

One might wonder why I did not "simplify the math" by removing some of the constants in the bounds, for example, by setting the Lipschitz constant L to 1. While this is very common in machine learning papers, this is a bad idea because it makes it (i) difficult to check the correctness of an equation, and (ii) difficult to correctly implement the algorithm. To explain why this is the case, we will have to consider the concepts of "units".

When we look at a mathematical formula coming from a physical model, we are used to the idea that each quantity has a "unit" of measurement, e.g., meters, seconds, Joules. A fundamental principle is that any physically meaningful equation must be dimensionally consistent. That is, the units on the left-hand side of an equation must be the same as the units on the right-hand side. For example, you cannot sum a quantity measured in meters with one measured in seconds.

It turns out that this simple idea is not limited to physical models and it can be used as a sanity check for *any* mathematical formula, including the ones we see in online learning. Let's assign a symbolic unit to each quantity we are dealing with. For example, let's denote the units of our predictions $\boldsymbol{x}$ by $[\boldsymbol{x}]$ and the units of the loss ℓ by $[\ell]$. We can now check each formula by following a few simple rules. First, we have to make sure that each term has the same units. Another rule to keep in mind is that transcendental functions are only defined on unitless quantities. So, for example, we can take logarithms and exponentials of ratios of quantities with the same units, to obtain a unitless quantity, but not of quantities

with units. Finally, while probabilities are unitless, the probability density function has the units of inverse of the random variable it represents, because it represents the probability per unit of the variable it is defined over.

The importance of this check cannot be overstated. When we see an equation where the units do not match, either it is plainly wrong, or there is a constant with units hidden somewhere. Moreover, the dimensional analysis will immediately tell us the dependence of a quantity on the others. Let's do a practical example considering OSD.

Consider the OSD update in Algorithm 2.2, where $\mathcal{V} = \mathbb{R}^d$: $\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \eta \boldsymbol{g}_t$. For this equation to be coherent, the term $\eta \boldsymbol{g}_t$ must have the same units as $\boldsymbol{x}_t$. Let's see if this gives us some constraints on the units of the learning rate η. The units of $\boldsymbol{g}_t$, from the definition of subgradient, are the units of the loss divided by the units of the variable we are subdifferentiating with respect to. So, we have

$$[\boldsymbol{g}_t] = \frac{[\ell]}{[\boldsymbol{x}]} \,.$$

Now, from the consistency of the update rule, we must have $[\boldsymbol{x}_t] = [\eta \boldsymbol{g}_t] = [\eta][\boldsymbol{g}_t]$. Hence, we can derive the units of the learning rate:

$$[\eta] = \frac{[\boldsymbol{x}]}{[\boldsymbol{g}_t]} = \frac{[\boldsymbol{x}]}{[\ell]/[\boldsymbol{x}]} = \frac{[\boldsymbol{x}]^2}{[\ell]} \,.$$

This shows that *the learning rate is not a unitless quantity*. So, even without deriving the regret guarantee, from the unit analysis, we immediately know that the learning rate must depend on the problem characteristics in a certain way.

Finally, let's perform a sanity check on the regret bound for OSD with a constant learning rate (that is, the one in Theorem 2.13):

$$\sum_{t=1}^{T} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \frac{\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2}{2\eta} + \frac{\eta}{2} \sum_{t=1}^{T} \|\boldsymbol{g}_t\|_2^2 \,.$$

The left-hand side is a sum of losses, so its unit is $[\ell]$. Let's check the two terms on the right-hand side, to verify that they also have the unit of $[\ell]$. For the first term, we have

$$\left[\frac{\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2}{\eta}\right] = \frac{[\boldsymbol{x}]^2}{[\eta]} = \frac{[\boldsymbol{x}]^2}{[\boldsymbol{x}]^2/[\ell]} = [\ell] \,.$$

For the second term, we have

$$\left[\eta \sum_{t=1}^{T} \|\boldsymbol{g}_t\|_2^2\right] = [\eta][\boldsymbol{g}_t]^2 = \frac{[\boldsymbol{x}]^2}{[\ell]} \frac{[\ell]^2}{[\boldsymbol{x}]^2} = [\ell] \,.$$

Let me stress that having a formula with wrong units is not only a theoretical issue, but a very practical one. Indeed, constants with hidden units in any algorithm can be problematic. Let's show it with an example. For example, suppose our variable $\boldsymbol{x}_t$ represents a location in meters, and we use a learning rate schedule such as $\eta = 1/\sqrt{T}$. Now, suppose we change our unit of measurement for $\boldsymbol{x}_t$ from meters to kilometers. Our new variable is $\boldsymbol{x}'_t = \boldsymbol{x}_t/1000$. Since this is merely a change of units, the underlying learning problem is identical, and we should expect the algorithm's behavior to be invariant. However, the gradient with respect to the new variable is $\boldsymbol{g}'_t = \nabla_{\boldsymbol{x}'} \ell_t(1000\boldsymbol{x}'_t) = 1000 \nabla_{\boldsymbol{x}} \ell_t(\boldsymbol{x}_t) = 1000 \boldsymbol{g}_t$.

If we still use the dimensionless learning rate $\eta = 1/\sqrt{T}$, the update in the new coordinate system becomes $\boldsymbol{x}'_{t+1} = \boldsymbol{x}'_t - \eta \boldsymbol{g}'_t = \boldsymbol{x}_t/1000 - (1/\sqrt{T})(1000\boldsymbol{g}_t)$. Converting back to meters by multiplying by 1000, we get $\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - (1000^2/\sqrt{T})\boldsymbol{g}_t$. The effective learning rate has been scaled by a factor of one million, which will drastically change the algorithm's path, all because of a simple change in units.

This failure occurs because of the constant with hidden units in the learning rate, which hides the fact that we should change the learning rate when we change the units of $\boldsymbol{x}_t$. In fact, the correct learning rate η has units of $[\boldsymbol{x}]^2/[\ell]$ and should have been scaled by a factor of $(1/1000)^2 = 1/1000000$ when we switched the units of $\boldsymbol{x}_t$ from meters to kilometers.

2.3 Linear Regret: From Convex Losses to Linear Losses

Let's take a deeper look at this step:

$$\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u}) \leq \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle, \quad \forall \boldsymbol{u} \in \mathcal{V} .$$

Summing over time, we have

$$\sum_{t=1}^{T} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle, \quad \forall \boldsymbol{u} \in \mathcal{V} .$$

Now, define the linear (and convex) losses $\tilde{\ell}_t(\boldsymbol{x}) := \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$, so we have

$$\sum_{t=1}^{T} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \sum_{t=1}^{T} (\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{u})), \quad \forall \boldsymbol{u} \in \mathcal{V} .$$

This is more powerful than it seems: we upper-bounded the regret with respect to the convex losses ℓ_t with a regret with respect to another sequence of linear losses. This is important because it implies that we can build online algorithms that deal only with linear losses, and through the reduction above, they can be seamlessly used as OCO algorithms! Note that this does not imply that this reduction is always optimal, as we saw in Example 2.14. But, it allows us to easily construct optimal OCO algorithms in many interesting cases.

So, we will often consider just the problem of minimizing the **linear regret**

$$\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle, \quad \forall \boldsymbol{u} \in \mathcal{V},$$

for an arbitrary sequence of vectors $\boldsymbol{g}_1, \ldots, \boldsymbol{g}_T \in \mathbb{R}^d$. This problem is called **Online Linear Optimization (OLO)**.

2.4 History Bits

Shor (1964) first introduced the subgradient descent method using a related concept of generalized gradients for non-differentiable functions. The concept of subgradients and the possibility of developing a calculus for them appeared for the first time in Rockafellar (1963).

Projected OGD with time-varying learning rate and the name "Online Convex Optimization" were introduced by Zinkevich (2003), but the OCO framework was introduced earlier by Gordon (1999a,b), which also introduced the idea of reducing OCO to OLO.

Before the OCO framework, the online learning community focused on specific losses and mostly linear predictors, see Cesa-Bianchi and Lugosi (2006). Moreover, the concept of subgradients is also a recent addition to the online learning literature, and the earliest papers using subgradients of loss functions seem to be Zhang (2004); Shalev-Shwartz and Singer (2006b). However, subgradients were implicitly used in previous analyses too, see for example Cesa-Bianchi (1999, Theorem 4).

It is impossible not to notice that the literature on OCO is heavily biased toward the use of bounded domains. Yet, there are very few practical applications of OCO where the domain is bounded. Moreover, as shown in Theorem 2.13, it is possible to have a sublinear regret even in unbounded domains. I speculate that the insistence on using bounded domains is due to two main reasons: (i) the less general definition of regret (derived from the setting of learning with experts (Section 6.8)) as

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \min_{\boldsymbol{u} \in \mathcal{V}} \sum_{t=1}^{T} \ell_t(\boldsymbol{u})$$

that requires the existence of a minimizer of the sum of the losses, and (ii) the fact that OSD with time-varying learning rates has a vacuous regret upper bound on unbounded domains. However, we now know that both issues can easily be fixed: (i) by defining the regret with respect to arbitrary competitors, and (ii) by using different algorithms, for example Follow-the-Regularized-Leader (FTRL) which we will see in Chapter 7. Indeed, even in offline optimization, the existence of a minimizer is not required to guarantee that the suboptimality gap goes to zero, as we will see in Example 3.2. So, it is probably time for the community to stop using the crutch of bounded domains just to avoid technical difficulties.

2.5 Exercises

Problem 2.1 *Prove that* $\sum_{t=1}^{T} \frac{1}{\sqrt{t}} \leq 2\sqrt{T} - 1$.

Problem 2.2 *Using the inequality in the previous exercise, prove that the learning rate* $\eta_t = \frac{D}{L\sqrt{t}}$ *gives rise to a regret only a constant multiplicative factor worse than the one in* (2.1).

Problem 2.3 *Calculate the subdifferential set of the* ϵ*-insensitive loss:* $f(x) = \max(|x - y| - \epsilon, 0)$. *It is a loss used in regression problems where we do not want to penalize predictions* x *within* $\pm\epsilon$ *of the correct value* y.

Problem 2.4 *Using the definition of subgradient, find the subdifferential set of* $f(\boldsymbol{x}) = \|\boldsymbol{x}\|_2$, $\boldsymbol{x} \in \mathbb{R}^d$.

Problem 2.5 *Consider Projected Online Subgradient Descent for the Example 2.10 on the failure of Follow-the-Leader: can we use it on that problem? Would it guarantee sublinear regret? How would the behaviour of the algorithm differ from FTL?*

3

Online-to-Batch Conversions

In this chapter, we take a break from online learning theory and see some applications of online learning to other domains. For example, we may wonder what the connection is between online learning and stochastic optimization. Given that projected Online Subgradient Descent (OSD) looks basically the same as projected stochastic (sub)gradient descent, they must have something in common. Indeed, for example, we can reduce stochastic optimization of convex functions to Online Convex Optimization (OCO). Let's see how.

3.1 From Online Learning to Stochastic Optimization

Theorem 3.1 *Let $\mathcal{V}$ be a non-empty closed convex set of $\mathbb{R}^d$, $F(\boldsymbol{x}) = \mathbb{E}[f(\boldsymbol{x}, \boldsymbol{\xi})]$ where the expectation is with respect to $\boldsymbol{\xi}$ drawn from a distribution ρ over some vector space $\mathcal{D}$, and $f : \mathbb{R}^d \times \mathcal{D} \to (-\infty, +\infty]$ is convex and subdifferentiable in the first argument on $\mathcal{V}$. Draw T samples $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_T$ i.i.d. from ρ and construct the sequence of losses $\ell_t(\boldsymbol{x}) = \alpha_t f(\boldsymbol{x}, \boldsymbol{\xi}_t)$, where $\alpha_t > 0$ are deterministic. Run any Online Convex Optimization (OCO) algorithm over the losses ℓ_t on $\mathcal{V}$, to construct the sequence of predictions $\boldsymbol{x}_1, \dots, \boldsymbol{x}_T$. The algorithm may be randomized, but its internal randomness is independent of $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_T$. Then, we have*

$$\mathbb{E}\left[F\left(\frac{1}{\sum_{t=1}^T \alpha_t}\sum_{t=1}^T \alpha_t \boldsymbol{x}_t\right)\right] \leq F(\boldsymbol{u}) + \frac{\mathbb{E}[\mathrm{Regret}_T(\boldsymbol{u})]}{\sum_{t=1}^T \alpha_t}, \quad \forall \boldsymbol{u} \in \mathcal{V},$$

where the expectation is with respect to $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_T$ and the internal randomness of the algorithm.

Proof We first show that

$$\mathbb{E}\left[\sum_{t=1}^T \alpha_t F(\boldsymbol{x}_t)\right] = \mathbb{E}\left[\sum_{t=1}^T \ell_t(\boldsymbol{x}_t)\right] . \tag{3.1}$$

In fact, from the linearity of the expectation we have

$$\mathbb{E}\left[\sum_{t=1}^T \ell_t(\boldsymbol{x}_t)\right] = \sum_{t=1}^T \mathbb{E}\left[\ell_t(\boldsymbol{x}_t)\right] .$$

Then, from the law of total expectation, letting $\mathscr{F}_{t-1}$ be the sigma-algebra generated by $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_{t-1}$ and the internal random bits used by the algorithm to compute $\boldsymbol{x}_t$, we have

$$\mathbb{E}\left[\ell_t(\boldsymbol{x}_t)\right] = \mathbb{E}\left[\mathbb{E}\left[\ell_t(\boldsymbol{x}_t) \mid \mathscr{F}_{t-1}\right]\right] = \mathbb{E}\left[\mathbb{E}\left[\alpha_t f(\boldsymbol{x}_t, \boldsymbol{\xi}_t) \mid \mathscr{F}_{t-1}\right]\right] = \mathbb{E}\left[\alpha_t F(\boldsymbol{x}_t)\right],$$

where we used the fact that $\boldsymbol{x}_t$ is $\mathscr{F}_{t-1}$-measurable, and that $\boldsymbol{\xi}_t$ is independent of $\mathscr{F}_{t-1}$. Hence, (3.1) is proved.

It remains only to use Jensen's inequality (Theorem 2.9), using the fact that F is convex, to have

$$F\left(\frac{1}{\sum_{t=1}^T \alpha_t}\sum_{t=1}^T \alpha_t \boldsymbol{x}_t\right) \leq \frac{1}{\sum_{t=1}^T \alpha_t}\sum_{t=1}^T \alpha_t F(\boldsymbol{x}_t)\ .$$

Dividing the regret by $\sum_{t=1}^T \alpha_t$ and using the above inequalities gives the stated theorem. □

Let's now see some applications of this result. Let's use the above theorem to transform Online Subgradient Descent (OSD) into Stochastic Subgradient Descent to minimize the training loss of a classifier.

Example 3.2 *Consider a problem of binary classification, with inputs $\boldsymbol{z}_i \in \mathbb{R}^d$ and outputs $y_i \in \{-1, 1\}$. The loss function is the hinge loss: $f(\boldsymbol{x}, (\boldsymbol{z}, y)) = \max(1 - y\langle \boldsymbol{z}, \boldsymbol{x}\rangle, 0)$. Suppose that you want to minimize the training loss over a training set of N samples, $\{(\boldsymbol{z}_i, y_i)\}_{i=1}^N$. Also, assume the maximum L_2 norm of the samples is R. That is, we want to minimize*

$$\min_{\boldsymbol{x}} \left(F(\boldsymbol{x}) := \frac{1}{N}\sum_{i=1}^N \max(1 - y_i\langle \boldsymbol{z}_i, \boldsymbol{x}\rangle, 0)\right)\ .$$

Run the reduction described in Theorem 3.1 for T iterations using OSD. In each iteration, construct $\ell_t(\boldsymbol{x}) = \max(1 - y_t\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle, 0)$ by sampling a training point uniformly at random from $\{1, \ldots, N\}$. Set $\boldsymbol{x}_1 = \mathbf{0}$ and $\eta = \frac{1}{R\sqrt{T}}$. Assuming $\operatorname{argmin}_{\boldsymbol{x}} F(\boldsymbol{x}) \neq \emptyset$, we have that

$$\mathbb{E}\left[F\left(\frac{1}{T}\sum_{t=1}^T \boldsymbol{x}_t\right)\right] - F(\boldsymbol{x}^\star) \leq R\frac{\|\boldsymbol{x}^\star\|_2^2 + 1}{2\sqrt{T}},$$

for all $\boldsymbol{x}^\star \in \operatorname{argmin}_{\boldsymbol{x}} F(\boldsymbol{x})$. In words, we used an OCO algorithm to stochastically optimize a function, transforming the regret guarantee into a convergence rate guarantee.

In this last example, we have to use a constant learning rate to minimize the training loss over the entire space $\mathbb{R}^d$. In the next one, we will see a different approach that allows us to *implicitly* use a varying learning rate without the need of a bounded feasible set.

Example 3.3 *Consider the same setting of the previous example, and let's change the way in which we construct the online losses. Now, use $\ell_t(\boldsymbol{x}) = \frac{1}{R\sqrt{t}}\max(1 - y_t\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle, 0)$ and step size $\eta = 1$. Hence, we have*

$$\begin{aligned}\mathbb{E}\left[F\left(\frac{1}{\sum_{t=1}^T \frac{1}{\sqrt{t}}}\sum_{t=1}^T \frac{1}{\sqrt{t}}\boldsymbol{x}_t\right)\right] - F(\boldsymbol{x}^\star) &\leq \frac{\|\boldsymbol{x}^\star\|_2^2}{2\sum_{t=1}^T \frac{1}{R\sqrt{t}}} + \frac{1}{2\sum_{t=1}^T \frac{1}{R\sqrt{t}}}\sum_{t=1}^T \frac{1}{t} \\ &\leq R\frac{\|\boldsymbol{x}^\star\|_2^2 + 1 + \ln T}{4\sqrt{T+1} - 4},\end{aligned}$$

where we used $\sum_{t=1}^T \frac{1}{\sqrt{t}} \geq 2\sqrt{T+1} - 2$.

Remark 3.4 *Using the online-to-batch conversion with online subgradient descent to minimize the expectation of convex Lipschitz functions we can obtain a convergence rate of $\mathcal{O}(\frac{1}{\sqrt{T}})$, that is optimal for this class of problems. This should dispel the common misconception that online algorithms are suboptimal in the stochastic setting because they are designed to work in the adversarial case. Indeed, the opposite is true: virtually any optimal guarantee for offline optimization can be recovered using online learning algorithms.*

I stressed the fact that the only meaningful way to define a regret is with respect to an arbitrary point in the feasible set. This is obvious in the case where we consider unconstrained Online Linear Optimization (OLO), because the optimal competitor is unbounded. But, it is also true in unconstrained OCO. Let's see an example of this.

Example 3.5 *Consider a problem of binary classification, with inputs $\boldsymbol{z}_i \in \mathbb{R}^d$ and outputs $y_i \in \{-1, 1\}$. The loss function is the logistic loss: $f(\boldsymbol{x}, (\boldsymbol{z}, y)) = \ln(1+\exp(-y\langle \boldsymbol{z}, \boldsymbol{x}\rangle))$. Suppose that you want to minimize the training loss over a training set of N samples, $\{(\boldsymbol{z}_i, y_i)\}_{i=1}^N$. Also, assume the maximum L_2 norm of the samples is R. That is, we want to minimize*

$$\min_{\boldsymbol{x}} \left(F(\boldsymbol{x}) := \frac{1}{N} \sum_{i=1}^{N} \ln(1 + \exp(-y_i \langle \boldsymbol{z}_i, \boldsymbol{x}\rangle)) \right) .$$

So, run the reduction described in Theorem 3.1 for T iterations using OSD. In each iteration, construct $\ell_t(\boldsymbol{x}) = \ln(1 + \exp(-y_t\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle))$ by sampling a training point uniformly at random from $\{1, \dots, N\}$. Set $\boldsymbol{x}_1 = \boldsymbol{0}$ and $\eta = \frac{1}{R\sqrt{T}}$. We have that

$$\mathbb{E}\left[F\left(\frac{1}{T} \sum_{t=1}^{T} \boldsymbol{x}_t \right) \right] \leq \frac{R}{2\sqrt{T}} + \min_{\boldsymbol{u} \in \mathbb{R}^d} \left(F(\boldsymbol{u}) + R\frac{\|\boldsymbol{u}\|_2^2}{2\sqrt{T}} \right) .$$

In words, we will be $\frac{R}{2\sqrt{T}}$ away from the optimal value of the function F plus a regularizer *$\|\boldsymbol{u}\|_2^2$, where the weight of the regularization is $\frac{R}{2\sqrt{T}}$. Now, let's consider the case that the training set is linearly separable, which means that the infimum of F is 0 and the optimal solution does not exist, i.e., informally speaking, it has norm equal to infinity. So, any convergence guarantee that depends on $\boldsymbol{x}^\star$ would be vacuous. On the other hand, our guarantee above still makes perfect sense.*

Note that the above examples only deal with training loss. However, in the next sections we show a more interesting application of the online-to-batch conversion, that is to directly minimize the generalization error. Moreover, we will see guarantees in high probability, rather than just in expectation.

3.1.1 Bits on Concentration Inequalities

We will use a concentration inequality to prove the high probability guarantee, but we will need to go beyond the sum of i.i.d. random variables. In particular, we will use the concept of *martingales*.

Let $(\Omega, (\mathscr{F}_t)_{t=0}^T, \mathbb{P})$ be a filtered probability space, where $T \in \mathbb{N} \cup \{\infty\}$, and let $Z_0, Z_1, \dots$ be random variables on Ω. Recall that the sequence $(Z_t)_{t=0}^T$ is adapted if Z_t is $\mathscr{F}_t$-measurable for all $0 \leq t \leq T$.

Definition 3.6 *A sequence of random variables $Z_0, Z_1, \dots$ is a **martingale** with respect to the filtration $(\mathscr{F}_t)_{t=0}^{\infty}$ if it is adapted to $(\mathscr{F}_t)_{t=0}^{\infty}$ and, for all $t \geq 0$, it satisfies:*

$$\mathbb{E}[|Z_t|] < \infty, \qquad \mathbb{E}[Z_{t+1}|\mathscr{F}_t] = Z_t, \text{ almost surely.}$$

Definition 3.7 *A sequence of random variables $Z_0, Z_1, \dots$ is a **supermartingale** with respect to the filtration $(\mathscr{F}_t)_{t=0}^{\infty}$ if it is adapted to $(\mathscr{F}_t)_{t=0}^{\infty}$ and, for all $t \geq 0$, it satisfies:*

$$\mathbb{E}[|Z_t|] < \infty, \qquad \mathbb{E}[Z_{t+1}|\mathscr{F}_t] \leq Z_t, \text{ almost surely.}$$

Example 3.8 *Consider a fair coin where each $c_t \in \{-1, 1\}$ is independent of the past, and an online betting algorithm that bets $|x_t|$ money on each round on the side of the coin equal to $\operatorname{sign}(x_t)$. We win or lose money at even odds, so the net gain up to round $t \geq 1$ is $Z_t = \sum_{i=1}^{t} c_i x_i$ and $Z_0 = 0$. Define $\mathscr{F}_t = \sigma(c_1, \dots, c_t)$. Since we use a deterministic online betting algorithm, x_t depends only on the past, and therefore x_t is $\mathscr{F}_{t-1}$-measurable. So, we have*

$$\mathbb{E}[Z_t|\mathscr{F}_{t-1}] = \mathbb{E}[Z_{t-1} + x_t c_t|\mathscr{F}_{t-1}] = Z_{t-1} + \mathbb{E}[x_t c_t|\mathscr{F}_{t-1}] = Z_{t-1} \ .$$

Hence, $(Z_t)_t$ is a martingale. If we throw away part of the wealth in each round, we obtain a supermartingale.

For martingales with bounded differences we can prove high probability guarantees as for bounded i.i.d. random variables. The following theorem will be the key result we will need.

Theorem 3.9 (Hoeffding–Azuma inequality) *Let $Z_0, \dots, Z_T$ be a martingale, where $|Z_t - Z_{t-1}| \leq B, t = 1, \dots, T$ almost surely. Then, we have*

$$\mathbb{P}\{Z_T - Z_0 \geq \epsilon\} \leq \exp\left(-\frac{\epsilon^2}{2B^2T}\right) \ .$$

Also, the same upper bound holds on $\mathbb{P}\{Z_0 - Z_T \geq \epsilon\}$.

3.1.2 High-Probability Guarantees for Online-to-Batch Conversion

We now show how the online-to-batch conversion we have introduced can be strengthened to produce guarantees in high probability.

Theorem 3.10 *Let $\mathcal{V} \subset \mathbb{R}^d$, $F(\boldsymbol{x}) = \mathbb{E}[f(\boldsymbol{x}, \boldsymbol{\xi})]$, where the expectation is with respect to $\boldsymbol{\xi}$ drawn from ρ with support over some set $\mathcal{D}$, $f : \mathcal{V} \times \mathcal{D} \to [0, 1]$, and $f(\boldsymbol{x}, \cdot)$ measurable for every fixed $\boldsymbol{x}$. Draw T samples $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_T$ i.i.d. from ρ and construct the sequence of losses $\ell_t(\boldsymbol{x}) = f(\boldsymbol{x}, \boldsymbol{\xi}_t)$. Let $\mathscr{A}$ be any deterministic online learning algorithm over the losses ℓ_t that outputs the sequence of predictions $\boldsymbol{x}_1, \dots, \boldsymbol{x}_{T+1}$ and guarantees $\operatorname{Regret}_T(\boldsymbol{u}) \leq R(\boldsymbol{u}, T)$ for all $\boldsymbol{u} \in \mathcal{V}$, for a function $R : \mathcal{V} \times \mathbb{N} \to \mathbb{R}$. Let $\delta \in (0, 1)$. Assume that $F(\boldsymbol{u}) + \frac{R(\boldsymbol{u},T)}{T}$ has a minimizer in $\mathcal{V}$. Then, with probability at least $1 - \delta$, it holds that*

$$\frac{1}{T}\sum_{t=1}^{T} F(\boldsymbol{x}_t) \leq \min_{\boldsymbol{u} \in \mathcal{V}} F(\boldsymbol{u}) + \frac{R(\boldsymbol{u}, T)}{T} + 2\sqrt{\frac{2\ln\frac{2}{\delta}}{T}} \ .$$

Proof Let $\mathscr{F}_{t-1}$ be the sigma-algebra generated by $\boldsymbol{\xi}_1, \ldots, \boldsymbol{\xi}_{t-1}$. Since $\mathscr{A}$ is an online algorithm, $\boldsymbol{x}_t$ depends only on the past samples $\boldsymbol{\xi}_1, \ldots, \boldsymbol{\xi}_{t-1}$, and therefore $\boldsymbol{x}_t$ is $\mathscr{F}_{t-1}$-measurable.

Define

$$Z_t := \sum_{i=1}^{t} \big(F(\boldsymbol{x}_i) - \ell_i(\boldsymbol{x}_i)\big), \quad t = 1, \ldots, T,$$

and $Z_0 := 0$. We claim that $(Z_t)_{t=0}^T$ is a martingale with respect to the filtration $(\mathscr{F}_t)_{t=0}^T$.

First, Z_t is $\mathscr{F}_t$-measurable for every t, since for each $i \le t$, the random variable $\boldsymbol{x}_i$ is $\mathscr{F}_{i-1}$-measurable and $\ell_i(\boldsymbol{x}_i) = f(\boldsymbol{x}_i, \boldsymbol{\xi}_i)$ is $\mathscr{F}_i$-measurable. Moreover, $|F(\boldsymbol{x}_i) - \ell_i(\boldsymbol{x}_i)| \le 1$, because both $F(\boldsymbol{x}_i)$ and $\ell_i(\boldsymbol{x}_i)$ belong to $[0,1]$, so Z_t is integrable.

Now observe that

$$\mathbb{E}[\ell_t(\boldsymbol{x}_t) \mid \mathscr{F}_{t-1}] = \mathbb{E}[f(\boldsymbol{x}_t, \boldsymbol{\xi}_t) \mid \mathscr{F}_{t-1}] = F(\boldsymbol{x}_t),$$

where we used that $\boldsymbol{x}_t$ is $\mathscr{F}_{t-1}$-measurable and that $\boldsymbol{\xi}_t$ is independent of $\mathscr{F}_{t-1}$. Hence,

$$\begin{aligned}\mathbb{E}[Z_t \mid \mathscr{F}_{t-1}] &= \mathbb{E}\left[Z_{t-1} + F(\boldsymbol{x}_t) - \ell_t(\boldsymbol{x}_t) | \mathscr{F}_{t-1}\right] \\ &= Z_{t-1} + F(\boldsymbol{x}_t) - \mathbb{E}[\ell_t(\boldsymbol{x}_t) \mid \mathscr{F}_{t-1}] = Z_{t-1}\,.\end{aligned}$$

So $(Z_t)_{t=0}^T$ is indeed a martingale.

Also, for every t, $|Z_t - Z_{t-1}| = |F(\boldsymbol{x}_t) - \ell_t(\boldsymbol{x}_t)| \le 1$. Therefore, by Theorem 3.9, we have

$$\mathbb{P}\left\{\sum_{t=1}^{T} \big(F(\boldsymbol{x}_t) - \ell_t(\boldsymbol{x}_t)\big) \ge \epsilon\right\} = \mathbb{P}\{Z_T - Z_0 \ge \epsilon\} \le \exp\left(-\frac{\epsilon^2}{2T}\right)\,.$$

Thus, with probability at least $1 - \delta/2$,

$$\frac{1}{T}\sum_{t=1}^{T} F(\boldsymbol{x}_t) \le \frac{1}{T}\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) + \sqrt{\frac{2\ln\frac{2}{\delta}}{T}}\,.$$

Now fix any $\boldsymbol{u} \in \mathcal{V}$. By the regret guarantee,

$$\frac{1}{T}\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) = \frac{\mathrm{Regret}_T(\boldsymbol{u})}{T} + \frac{1}{T}\sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \le \frac{R(\boldsymbol{u},T)}{T} + \frac{1}{T}\sum_{t=1}^{T} \ell_t(\boldsymbol{u})\,.$$

It remains to upper bound $\frac{1}{T}\sum_{t=1}^{T} \ell_t(\boldsymbol{u})$ by $F(\boldsymbol{u})$ with high probability. Choose now $\boldsymbol{u}$ to be any fixed minimizer of $F(\boldsymbol{x}) + \frac{R(\boldsymbol{x},T)}{T}$ over $\mathcal{V}$. Since this $\boldsymbol{u}$ is deterministic, the random variables $\ell_t(\boldsymbol{u}) = f(\boldsymbol{u}, \boldsymbol{\xi}_t)$ are i.i.d. Define $Y_t := \sum_{i=1}^{t} \big(F(\boldsymbol{u}) - \ell_i(\boldsymbol{u})\big)$ for $t = 1, \ldots, T$, and $Y_0 := 0$. Then, $(Y_t)_{t=0}^T$ is a martingale with respect to the same filtration $(\mathscr{F}_t)_{t=0}^T$, because

$$\mathbb{E}[\ell_t(\boldsymbol{u}) \mid \mathscr{F}_{t-1}] = \mathbb{E}[\ell_t(\boldsymbol{u})] = F(\boldsymbol{u})\,.$$

Moreover, $|Y_t - Y_{t-1}| \le 1$, so Azuma's inequality gives that, with probability at least $1 - \delta/2$,

$$\frac{1}{T}\sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \le F(\boldsymbol{u}) + \sqrt{\frac{2\ln\frac{2}{\delta}}{T}}\,.$$

Putting everything together and using the *union bound* (for events $A_1, \ldots, A_n$, we have $\mathbb{P}(\cup_{i=1}^n A_i) \leq \sum_{i=1}^n \mathbb{P}(A_i)$), we have the stated bound. □

The theorem above upper bounds the average value of the T different solutions, while we are interested in producing a single solution. If F is a convex function and $\mathcal{V}$ is convex, then we can lower bound the l.h.s. of the inequalities in the theorem with the function evaluated on the average of the $\boldsymbol{x}_t$. That is

$$F\left(\frac{1}{T}\sum_{t=1}^{T}\boldsymbol{x}_t\right) \leq \frac{1}{T}\sum_{t=1}^{T}F(\boldsymbol{x}_t)\ .$$

Remark 3.11 *Note that using the online-to-batch conversion to optimize a population risk, i.e., $F(\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{\xi}\sim\rho}[f(\boldsymbol{x},\boldsymbol{\xi})]$, is not the same as using the online-to-batch conversion to minimize the empirical risk, $\hat{F}(\boldsymbol{x}) = \frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{x},\boldsymbol{\xi}_t)$. In both cases, we first sample T random vectors $\mathcal{S} = \{\boldsymbol{\xi}_1, \ldots, \boldsymbol{\xi}_T\}$ from ρ. Then, to minimize the empirical risk we sample* with replacement *from $\mathcal{S}$. Instead, to minimize the population risk we sample uniformly* without replacement *from $\mathcal{S}$, which is equivalent to using T i.i.d. samples from ρ. This is not a minor difference because one can show that there are cases where sampling without replacement will not minimize the empirical risk (Vansover-Hager et al., 2025).*

3.2 Application: Agnostic PAC Learning

In this section, we show another application of online-to-batch methods to obtain statistical learning guarantees. Here, we assume that we have a prediction strategy $\phi_{\boldsymbol{x}}$ parametrized by a vector $\boldsymbol{x}$ and we want to learn the relationship between an input $\boldsymbol{z}$ and its associated label y. Moreover, we will assume that $(\boldsymbol{z}, y)$ is drawn from a joint probability distribution ρ. Also, we are equipped with a loss function, $\ell(\hat{y}, y)$, that measures how good our prediction $\hat{y} = \phi_{\boldsymbol{x}}(\boldsymbol{z})$ is, compared to the true label y. So, learning the relationship can be cast as minimizing the expected loss of our predictor

$$\min_{\boldsymbol{x}\in\mathcal{V}} \mathbb{E}_{(\boldsymbol{z},y)\sim\rho}[\ell(\phi_{\boldsymbol{x}}(\boldsymbol{z}), y)]\ .$$

In machine learning terms, the object above is nothing else than the *expected test loss* of our predictor.

Note that the above setting assumes labeled samples, but we can generalize it further by considering *Vapnik's general setting of learning*, where we collapse the prediction function and the loss into a single function. This allows, for example, to treat supervised and unsupervised learning in the same unified way. So, we want to minimize the **risk**

$$\min_{\boldsymbol{x}\in\mathcal{V}} \left(\mathrm{Risk}(\boldsymbol{x}) := \mathbb{E}_{\boldsymbol{\xi}\sim\rho}[f(\boldsymbol{x},\boldsymbol{\xi})]\right),$$

where ρ is an unknown distribution over $\mathcal{D}$ and $f : \mathcal{V}\times\mathcal{D} \to \mathbb{R}$ is such that for every fixed $\boldsymbol{x}$, $f(\boldsymbol{x},\cdot)$ is measurable. Also, the set $\mathcal{H}$ of the composition of a loss with all predictors that can be expressed by vectors $\boldsymbol{x}$ in $\mathcal{V}$ is called the **hypothesis class**.

Example 3.12 *In a linear regression task where the loss is the square loss, we have $\boldsymbol{\xi} = (\boldsymbol{z}, y) \in \mathbb{R}^d \times \mathbb{R}$ and $\phi_{\boldsymbol{x}}(\boldsymbol{z}) = \langle \boldsymbol{z}, \boldsymbol{x}\rangle$. Hence, $f(\boldsymbol{x},\boldsymbol{\xi}) = (\langle \boldsymbol{z}, \boldsymbol{x}\rangle - y)^2$.*

Example 3.13 *In linear binary classification where the loss is the hinge loss, we have* $\boldsymbol{\xi} = (\boldsymbol{z}, y) \in \mathbb{R}^d \times \{-1, 1\}$ *and* $\phi_{\boldsymbol{x}}(\boldsymbol{z}) = \langle \boldsymbol{z}, \boldsymbol{x} \rangle$*. Hence,* $f(\boldsymbol{x}, \boldsymbol{\xi}) = \max(1 - y\langle \boldsymbol{z}, \boldsymbol{x} \rangle, 0)$.

Example 3.14 *In binary classification with a neural network with the logistic loss, we have* $\boldsymbol{\xi} = (\boldsymbol{z}, y) \in \mathbb{R}^d \times \{-1, 1\}$ *and* $\phi_{\boldsymbol{x}}$ *is the network corresponding to the weights* $\boldsymbol{x}$*. Hence,* $f(\boldsymbol{x}, \boldsymbol{\xi}) = \ln(1 + \exp(-y\phi_{\boldsymbol{x}}(\boldsymbol{z})))$.

The key difficulty of the above problem is that we do not know the distribution ρ. Hence, there is no hope to exactly solve this problem. Instead, we are interested in understanding *what is the best we can do if we have access to* T *samples drawn i.i.d. from* ρ. More precisely, we want to upper bound the **excess risk**

$$\mathrm{Risk}(\boldsymbol{x}_T) - \inf_{\boldsymbol{x} \in \mathcal{V}} \mathrm{Risk}(\boldsymbol{x}),$$

where $\boldsymbol{x}_T$ is a predictor that was *learned* using T samples.

It should be clear that this is just an optimization problem and the one above is just the suboptimality gap. In this view, the objective of machine learning can be considered as a particular optimization problem.

Remark 3.15 *Note that this is not the only way to approach the problem of learning. Indeed, the regret minimization model is an alternative model to learning. Moreover, another approach would be to try to estimate the distribution* ρ *and then solve the risk minimization problem. No approach is superior to the other and each of them has its pros and cons.*

Given that we have access to the distribution ρ through samples drawn from it, any procedure we might think to use to minimize the risk will be stochastic in nature. This means that we cannot provide a deterministic guarantee. Instead, *we can try to prove that with high probability our minimization procedure will return a solution that is close to the minimizer of the risk*. It is also intuitive that the precision and probability we can guarantee must depend on how many samples we draw from ρ.

Quantifying the dependence of precision and probability of failure on the number of samples used is the objective of the *agnostic Probably Approximately Correct* (PAC) framework, where the keyword "agnostic" refers to the fact that we do not assume anything about the best possible predictor. More precisely, given a precision parameter ϵ and a probability of failure δ, we are interested in characterizing the *sample complexity of the hypothesis class* $\mathcal{H}$. A sample complexity of T implies the existence of a learning algorithm using the hypothesis class $\mathcal{H}$ and outputting a solution $\boldsymbol{x}_T$ using T samples that has an excess risk upper bounded by ϵ with probability at least $1 - \delta$ for every distribution ρ over $\mathcal{D}$. Note that the sample complexity does not depend on ρ, so it is a worst-case measure with respect to all the possible distributions. This makes sense if you think that we know nothing about the distribution ρ, so if your guarantee holds for the worst distribution it will also hold for any other distribution. Mathematically, we will say that the hypothesis class is agnostic PAC-learnable if such a sample-complexity function exists.

Definition 3.16 *We will say that a hypothesis class* $\mathcal{H} = \{f(\boldsymbol{x}, \cdot) : \boldsymbol{x} \in \mathcal{V}\}$ *is* ***Agnostic-PAC-learnable*** *if there exists an algorithm* $\mathscr{A}$ *and a function* $T(\epsilon, \delta) : \mathbb{R}_{>0} \times (0, 1) \to \mathbb{N}$ *such that, for every distribution* ρ *over* $\mathcal{D}$*, when* $\mathscr{A}$ *is used with* $T \geq T(\epsilon, \delta)$ *samples drawn*

i.i.d. from ρ, with probability at least $1-\delta$ the solution returned by the algorithm has excess risk at most ϵ.

Note that the Agnostic PAC learning setting does not say what procedure we should follow to find such sample complexity. The approach most commonly used in machine learning to solve the learning problem is the so-called **Empirical Risk Minimization (ERM)** procedure. It consists of drawing T samples i.i.d. from ρ and minimizing the **empirical risk** defined as

$$\widehat{\mathrm{Risk}}(\boldsymbol{x}) := \frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{x}, \boldsymbol{\xi}_t) \; .$$

The minimizer $\hat{\boldsymbol{x}}_T = \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} \widehat{\mathrm{Risk}}(\boldsymbol{x})$ is called the *empirical risk minimizer*. In words, ERM is nothing else than the minimization of some loss function on a training set. However, in many interesting cases $\operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} \frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{x}, \boldsymbol{\xi}_t)$ can be very far from the true optimum $\operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} \mathbb{E}[f(\boldsymbol{x}, \boldsymbol{\xi})]$. So, we need to modify the ERM formulation in some way, e.g., using a *regularization* term, a Bayesian prior of $\boldsymbol{x}$, or more generally find conditions under which ERM works.

The ERM approach is so widespread that machine learning itself is often wrongly identified with some kind of minimization of the training loss. We now show that ERM is not the entire world of machine learning, showing that *the existence of a no-regret algorithm, that is an online learning algorithm with sublinear regret, guarantees Agnostic PAC learnability*. More precisely, we will see that an online algorithm with sublinear regret can be used to solve machine learning problems. This is not just a curiosity, for example this gives rise to computationally efficient algorithms, that can be achieved through ERM only by running a two-step procedure, i.e., running ERM with different parameters and selecting the best solution among them.

We can use Theorem 3.10 to produce a solution with small risk. In particular, if the risk is convex, we can output the average of the $\boldsymbol{x}_t$, using Jensen's inequality.

If the risk is not a convex function, we need a different way. An alternative solution is to construct a *stochastic predictor* that samples one of the $\boldsymbol{x}_t$ uniformly at random and predicts with it. So, defining $I \sim \mathrm{Unif}(\{1, \dots, T\})$, we immediately have

$$\mathbb{E}_I[\mathrm{Risk}(\boldsymbol{x}_I)] = \frac{1}{T}\sum_{t=1}^{T} \mathrm{Risk}(\boldsymbol{x}_t) \; .$$

Yet another way is to try to find among the T predictors the one with the smallest risk. This works because the average is lower bounded by the minimum. We can use $T/2$ samples for the online learning procedure and $T/2$ samples to generate a validation set to evaluate the solution and pick the best one. The following theorem shows that selecting the predictor with the smallest empirical risk on a validation set will give us a predictor close to the best one with high probability.

Theorem 3.17 *Let $\mathcal{V} \subset \mathbb{R}^d$, $\mathrm{Risk}(\boldsymbol{x}) = \mathbb{E}[f(\boldsymbol{x}, \boldsymbol{\xi})]$, where the expectation is with respect to $\boldsymbol{\xi}$ drawn from ρ with support over some set $\mathcal{D}$, and $f : \mathcal{V} \times \mathcal{D} \to [0, 1]$. Let $\delta \in (0, 1)$. We have a finite set of vectors $\mathcal{S} = \{\boldsymbol{x}_1, \dots, \boldsymbol{x}_{|\mathcal{S}|}\}$ fixed before seeing T random vectors $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_T$ drawn i.i.d. from ρ. Denote by $\hat{\boldsymbol{x}} = \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{S}} \widehat{\mathrm{Risk}}(\boldsymbol{x})$, where $\widehat{\mathrm{Risk}}(\boldsymbol{x}) =$*

$\frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{x}, \boldsymbol{\xi}_t)$. *Then, with probability at least* $1-\delta$*, we have*

$$\operatorname{Risk}(\hat{\boldsymbol{x}}) \leq \min_{\boldsymbol{x}\in\mathcal{S}} \operatorname{Risk}(\boldsymbol{x}) + 2\sqrt{\frac{2\ln(2|\mathcal{S}|/\delta)}{T}}\,.$$

Proof We want to calculate the probability that the hypothesis that minimizes the validation error is far from the best hypothesis in the set. We cannot do it directly because we do not have the required independence to use a concentration inequality. Instead, *we will upper bound the probability that there exists at least one function whose empirical risk is far from the risk.* So, using the union bound, we have

$$\begin{aligned}\mathbb{P}\left\{\exists \boldsymbol{x}\in\mathcal{S} : |\operatorname{Risk}(\boldsymbol{x}) - \widehat{\operatorname{Risk}}(\boldsymbol{x})| > \frac{\epsilon}{2}\right\} &\leq \sum_{i=1}^{|\mathcal{S}|} \mathbb{P}\left\{|\operatorname{Risk}(\boldsymbol{x}_i) - \widehat{\operatorname{Risk}}(\boldsymbol{x}_i)| > \frac{\epsilon}{2}\right\} \\ &\leq 2|\mathcal{S}|\exp\left(-\frac{\epsilon^2 T}{8}\right)\,.\end{aligned}$$

Hence, with probability at least $1-\delta$, we have that

$$|\operatorname{Risk}(\boldsymbol{x}) - \widehat{\operatorname{Risk}}(\boldsymbol{x})| \leq \frac{\epsilon}{2}, \quad \forall \boldsymbol{x}\in\mathcal{S},$$

where $\epsilon = 2\sqrt{\frac{2\ln(2|\mathcal{S}|/\delta)}{T}}$.

We are now able to upper bound the risk of $\hat{\boldsymbol{x}}$, just using the fact that the above applies to $\hat{\boldsymbol{x}}$ too. Defining $\boldsymbol{x}^\star \in \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{S}} \operatorname{Risk}(\boldsymbol{x})$, we have

$$\operatorname{Risk}(\hat{\boldsymbol{x}}) \leq \widehat{\operatorname{Risk}}(\hat{\boldsymbol{x}}) + \epsilon/2 \leq \widehat{\operatorname{Risk}}(\boldsymbol{x}^\star) + \epsilon/2 \leq \operatorname{Risk}(\boldsymbol{x}^\star) + \epsilon,$$

where in the second inequality we used the fact that $\hat{\boldsymbol{x}}$ minimizes the empirical risk. □

Using this theorem, we can use $T/2$ samples for the training and $T/2$ samples for the validation, where $T \geq 2$ and even. Denoting by $\hat{\boldsymbol{x}}_T$ the predictor with the best empirical risk on the validation set among the $T/2$ generated during the online procedure, we have with probability at least $1-2\delta$ that

$$\operatorname{Risk}(\hat{\boldsymbol{x}}_T) \leq \min_{\boldsymbol{u}\in\mathcal{V}} \operatorname{Risk}(\boldsymbol{u}) + \frac{2R(\boldsymbol{u}, T/2)}{T} + 8\sqrt{\frac{\ln(T/\delta)}{T}}\,.$$

It is important to note that with any of the above three methods to produce one predictor from the T generated ones by the online learning procedure, the sample complexity guarantee we get matches the one we would have obtained by ERM, up to polylogarithmic factors. In other words, there is nothing special about ERM compared to the online learning approach to statistical learning. Moreover, ERM implies the existence of a hypothetical procedure that perfectly minimizes the training loss. In reality, we should take into account the optimization error in the analysis of ERM. On the other hand, in the online learning approach we have a guarantee directly for the computed solution.

Another important point is that the above guarantee does not imply the existence of online learning algorithms with sublinear regret for any learning problem. It just says that, if it exists, it can be used in the statistical setting too.

3.3 History Bits

The specific shape of Theorem 3.1 is new, but I would not be surprised if it appeared somewhere in the literature. In particular, the uniform averaging is from Cesa-Bianchi et al. (2004), but was proposed for the absolute loss in Blum et al. (1999). The non-uniform averaging of Example 3.3 is from Zhang (2004), even though not proposed explicitly as an online-to-batch conversion.

A more recent method for online-to-batch conversion was introduced in Cutkosky (2019a), which independently rediscovered and generalized the averaging method in Nesterov and Shikhman (2015). This new method allows us to prove the convergence of the last iterate rather than that of the weighted average, with a small change in any online learning algorithm.

Theorem 3.10 is from Cesa-Bianchi et al. (2004), but here I used a second concentration to state it in terms of the competitor's true risk rather than its empirical risk. Theorem 3.17 is just the Agnostic PAC learning guarantee for ERM for hypothesis classes with finite cardinality. Cesa-Bianchi et al. (2004) also gives an alternative procedure to select a single hypothesis among the T generated during the online procedure that does not require splitting the data in training and validation. However, the obtained guarantee matches the one we have proved.

3.4 Exercises

Problem 3.1 *Derive an explicit rate of convergence for SGD in Example 3.5 by upper bounding the value of the minimum on the r.h.s. of the convergence rate guarantee. Hint: see Ji and Telgarsky (2019).*

Problem 3.2 *Let $F(\boldsymbol{x}) = \mathbb{E}[f(\boldsymbol{x}, \boldsymbol{\xi})]$, where $f(\cdot, \boldsymbol{\xi})$ is convex and L-Lipschitz with respect to $\|\cdot\|_2$ for every $\boldsymbol{\xi}$. Assume that $\mathcal{V}$ is convex and has diameter at most D. For $a \in \mathbb{R}$, run OSD on the losses $\ell_t(\boldsymbol{x}) = t^a f(\boldsymbol{x}, \boldsymbol{\xi}_t)$, where $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_T$ are i.i.d., and output the weighted average*

$$\bar{\boldsymbol{x}}_T^{(a)} = \frac{\sum_{t=1}^T t^a \boldsymbol{x}_t}{\sum_{t=1}^T t^a} .$$

Choose the best constant learning rate in hindsight and prove an upper bound on

$$\mathbb{E}\left[F(\bar{\boldsymbol{x}}_T^{(a)})\right] - \min_{\boldsymbol{u} \in \mathcal{V}} F(\boldsymbol{u})$$

as a function of a and T. In particular, show that for $a > -\frac{1}{2}$ the rate is of order $\frac{DL}{\sqrt{T}} \frac{a+1}{\sqrt{2a+1}}$, up to lower order terms. What happens when $a = -\frac{1}{2}$ and when $a < -\frac{1}{2}$?

4

Beyond $\sqrt{T}$ Regret

The guessing game in the first chapter showed us that it is possible to get logarithmic regret in time. However, in Chapter 2, we saw that we get only $\sqrt{T}$-regret with Online Subgradient Descent (OSD) on the same game. What is the reason? In this chapter, we will see that some sequences of losses can possess properties that make it easier for OSD to learn, but we will have to change the learning rate a bit.

4.1 Strong Convexity and Online Subgradient Descent

The losses in the guessing game in Chapter 1, $\ell_t(x) = (x - y_t)^2$ on $[0, 1]$, are not just Lipschitz. They possess some *curvature* that can be exploited to achieve a better asymptotic regret. In a moment, we will see that the only change we will need to Online Subgradient Descent (OSD) is a different learning rate, dictated as usual by the regret analysis.

The key concept we will need is *strong convexity*.

4.1.1 Convex Analysis Bits: Strong Convexity

Here, we introduce a stronger concept of convexity that allows us to build a better lower bound to a function. Instead of the linear lower bound achievable using subgradients, we will make use of a *quadratic* lower bound.

Definition 4.1 *Let $\lambda \geq 0$. A proper function $f : \mathbb{R}^d \to (-\infty, +\infty]$ is **λ-strongly convex** with respect to $\|\cdot\|$ over a convex set $\mathcal{V} \subseteq \operatorname{dom} f$ if*

$$f(\alpha \boldsymbol{x} + (1-\alpha)\boldsymbol{y}) \leq \alpha f(\boldsymbol{x}) + (1-\alpha) f(\boldsymbol{y}) - \frac{1}{2}\lambda\alpha(1-\alpha)\|\boldsymbol{x} - \boldsymbol{y}\|^2,$$

for all $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{V}$ and all $\alpha \in (0, 1)$.

*We will also say that f is **strongly convex in** $\mathcal{V}$, if there exists $\lambda > 0$ and a norm such that the above holds.*

From the definition, it is clear that if a function is λ-strongly convex, it is also λ'-strongly convex for any $0 \leq \lambda' < \lambda$. Moreover, convex functions are 0-strongly convex.

We can also obtain an equivalent characterization in terms of subgradients.

Lemma 4.2 *Let $\lambda \geq 0$. We have that $f : \mathbb{R}^d \to (-\infty, +\infty]$ is λ-strongly convex over a convex set $\mathcal{V} \subseteq \operatorname{dom} \partial f$ with respect to $\|\cdot\|$ iff*

$$\forall \boldsymbol{x}, \boldsymbol{y} \in \mathcal{V}, \forall \boldsymbol{g} \in \partial f(\boldsymbol{y}), \quad f(\boldsymbol{x}) \geq f(\boldsymbol{y}) + \langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{y} \rangle + \frac{\lambda}{2}\|\boldsymbol{x} - \boldsymbol{y}\|^2 .$$

Proof Let's first assume that f is λ-strongly convex over $\mathcal{V}$ with respect to $\|\cdot\|$. Then, for any $\alpha \in (0,1)$ and any $\boldsymbol{g} \in \partial f(\boldsymbol{y})$, we have

$$\langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{y} \rangle \leq \frac{f(\alpha \boldsymbol{x} + (1-\alpha)\boldsymbol{y}) - f(\boldsymbol{y})}{\alpha} \leq f(\boldsymbol{x}) - f(\boldsymbol{y}) - \frac{1}{2}\lambda(1-\alpha)\|\boldsymbol{x} - \boldsymbol{y}\|^2 .$$

Taking the limit as $\alpha \to 0$, we obtain the statement.

Let's now assume that the inequality in the lemma holds, and let's prove that f is λ-strongly convex. Setting $\boldsymbol{v} = \alpha \boldsymbol{x} + (1-\alpha)\boldsymbol{y}$, for any $\boldsymbol{g} \in \partial f(\boldsymbol{v})$, we have

$$\begin{aligned}
\langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{v} \rangle &\leq f(\boldsymbol{x}) - f(\boldsymbol{v}) - \frac{\lambda}{2}\|\boldsymbol{x} - \boldsymbol{v}\|^2, \\
\langle \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{v} \rangle &\leq f(\boldsymbol{y}) - f(\boldsymbol{v}) - \frac{\lambda}{2}\|\boldsymbol{y} - \boldsymbol{v}\|^2 .
\end{aligned}$$

Summing these two inequalities with coefficients α and $1-\alpha$ and using the definition of $\boldsymbol{v}$, we have

$$\begin{aligned}
0 &= \langle \boldsymbol{g}, \alpha \boldsymbol{x} - \alpha \boldsymbol{v} + (1-\alpha)\boldsymbol{y} - (1-\alpha)\boldsymbol{v} \rangle \\
&= \alpha f(\boldsymbol{x}) - f(\alpha \boldsymbol{x} + (1-\alpha)\boldsymbol{y}) + (1-\alpha) f(\boldsymbol{y}) - \alpha(1-\alpha)\frac{\lambda}{2}\|\boldsymbol{x} - \boldsymbol{y}\|^2 . \qquad \square
\end{aligned}$$

In words, the lemma above tells us that a strongly convex function can be lower bounded by a quadratic, where the linear term is the usual one constructed through the subgradient, and the quadratic term depends on the strong convexity. Hence, we have a tighter lower bound to the function than the one obtained by using convexity alone. This is what we would expect using a Taylor expansion on a twice-differentiable convex function and lower-bounding the smallest eigenvalue of the Hessian. Indeed, we have the following theorem.

Theorem 4.3 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be proper and convex.*

- *f is λ-strongly convex with respect to $\|\cdot\|$ in $\operatorname{dom} \partial f$ iff*

$$\langle \boldsymbol{g}_{\boldsymbol{x}} - \boldsymbol{g}_{\boldsymbol{y}}, \boldsymbol{x} - \boldsymbol{y} \rangle \geq \lambda \|\boldsymbol{x} - \boldsymbol{y}\|^2, \quad \forall \boldsymbol{x}, \boldsymbol{y} \in \operatorname{dom} \partial f, \boldsymbol{g}_{\boldsymbol{x}} \in \partial f(\boldsymbol{x}), \boldsymbol{g}_{\boldsymbol{y}} \in \partial f(\boldsymbol{y}). \quad (4.1)$$

- *Let $\boldsymbol{x}, \boldsymbol{y} \in \operatorname{dom} f$. If f is continuously twice differentiable on an open set containing the segment between $\boldsymbol{y}$ and $\boldsymbol{x}$, and it holds that*

$$\langle \nabla^2 f(\boldsymbol{y} + \alpha(\boldsymbol{x} - \boldsymbol{y}))(\boldsymbol{x} - \boldsymbol{y}), \boldsymbol{x} - \boldsymbol{y} \rangle \geq \lambda \|\boldsymbol{x} - \boldsymbol{y}\|^2, \quad \forall \alpha \in [0,1], \qquad (4.2)$$

then we have

$$f(\boldsymbol{x}) \geq f(\boldsymbol{y}) + \langle \nabla f(\boldsymbol{y}), \boldsymbol{x} - \boldsymbol{y} \rangle + \frac{\lambda}{2}\|\boldsymbol{x} - \boldsymbol{y}\|^2 .$$

If the assumption holds for any $\boldsymbol{x}, \boldsymbol{y} \in \operatorname{dom} f$, then f is λ-strongly convex in $\operatorname{dom} f$ with respect to $\|\cdot\|$.

Proof For the first statement, first assume that f is λ-strongly convex with respect to $\|\cdot\|$.

Then, from Lemma 4.2, we have

$$\langle \boldsymbol{g}_{\boldsymbol{x}}, \boldsymbol{y} - \boldsymbol{x} \rangle \leq f(\boldsymbol{x}) - f(\boldsymbol{y}) - \frac{\lambda}{2}\|\boldsymbol{x} - \boldsymbol{y}\|^2, \quad \forall \boldsymbol{g}_{\boldsymbol{x}} \in \partial f(\boldsymbol{x}),$$
$$\langle \boldsymbol{g}_{\boldsymbol{y}}, \boldsymbol{x} - \boldsymbol{y} \rangle \leq f(\boldsymbol{y}) - f(\boldsymbol{x}) - \frac{\lambda}{2}\|\boldsymbol{x} - \boldsymbol{y}\|^2, \quad \forall \boldsymbol{g}_{\boldsymbol{y}} \in \partial f(\boldsymbol{y}) \,.$$

Summing the two inequalities, we have the stated bound.

Now, assume that the (4.1) holds. Define $h(\alpha) = f(\boldsymbol{y}+\alpha(\boldsymbol{x}-\boldsymbol{y}))$, $\boldsymbol{w}_\alpha = \boldsymbol{y}+\alpha(\boldsymbol{x}-\boldsymbol{y})$, and $\boldsymbol{g}_{\boldsymbol{w}_\alpha} \in \partial f(\boldsymbol{w}_\alpha)$. From Theorem 2.28, we have $\langle \boldsymbol{g}_{\boldsymbol{w}_\alpha}, \boldsymbol{x} - \boldsymbol{y} \rangle \in \partial h(\alpha)$. Moreover,

$$\langle \boldsymbol{g}_{\boldsymbol{w}_\alpha}, \boldsymbol{x} - \boldsymbol{y} \rangle - \langle \boldsymbol{g}_{\boldsymbol{y}}, \boldsymbol{x} - \boldsymbol{y} \rangle = \frac{1}{\alpha}\langle \boldsymbol{g}_{\boldsymbol{w}_\alpha} - \boldsymbol{g}_{\boldsymbol{y}}, \boldsymbol{w}_\alpha - \boldsymbol{y} \rangle \geq \frac{\lambda}{\alpha}\|\boldsymbol{w}_\alpha - \boldsymbol{y}\|^2 = \lambda\alpha\|\boldsymbol{x} - \boldsymbol{y}\|^2,$$

where we used (4.1) in the inequality. Using the fundamental theorem of calculus for extended-real-valued convex functions (Theorem A.1) and this last inequality, we have

$$\begin{aligned} f(\boldsymbol{x}) - f(\boldsymbol{y}) - \langle \boldsymbol{g}_{\boldsymbol{y}}, \boldsymbol{x} - \boldsymbol{y} \rangle &= h(1) - h(0) - \langle \boldsymbol{g}_{\boldsymbol{y}}, \boldsymbol{x} - \boldsymbol{y} \rangle \\ &= \int_0^1 (\langle \boldsymbol{g}_{\boldsymbol{w}_\alpha}, \boldsymbol{x} - \boldsymbol{y} \rangle - \langle \boldsymbol{g}_{\boldsymbol{y}}, \boldsymbol{x} - \boldsymbol{y} \rangle) \,\mathrm{d}\alpha \geq \frac{\lambda}{2}\|\boldsymbol{x} - \boldsymbol{y}\|^2 \,. \end{aligned}$$

Hence, by Lemma 4.2, f is λ-strongly convex with respect to $\|\cdot\|$.

For the second statement, assume that f is twice differentiable and (4.2) holds. Then,

$$h''(\alpha) = \langle \nabla^2 f(\boldsymbol{y} + \alpha(\boldsymbol{x} - \boldsymbol{y}))(\boldsymbol{x} - \boldsymbol{y}), \boldsymbol{x} - \boldsymbol{y} \rangle \geq \lambda\|\boldsymbol{x} - \boldsymbol{y}\|^2 \,.$$

Moreover, from Taylor's remainder theorem, we have that $h(1) = h(0)+h'(0)+h''(\beta)/2$, where $\beta \in [0, 1]$. So, we have

$$f(\boldsymbol{x}) = h(1) = h(0) + h'(0) + h''(\beta)/2 \geq f(\boldsymbol{y}) + \langle \nabla f(\boldsymbol{y}), \boldsymbol{x} - \boldsymbol{y} \rangle + \frac{\lambda}{2}\|\boldsymbol{x} - \boldsymbol{y}\|^2 \,.$$

If it holds for all $\boldsymbol{x}, \boldsymbol{y}$, then, by Lemma 4.2, f is λ-strongly convex with respect to $\|\cdot\|$. □

Example 4.4 *Let $f(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{x}\|_2^2$. Using Theorem 4.3, we have that f is 1-strongly convex with respect to $\|\cdot\|_2$ in $\mathbb{R}^d$.*

However, a strongly convex function does not need to be twice differentiable. Indeed, we do not even need plain differentiability. Hence, the use of the subgradient implies that the quadratic lower bound does not have to be uniquely determined, as in the next example.

Example 4.5 *Consider the strongly convex function $f(x) = |x| + x^2$. In Figure 4.1, we show two possible quadratic lower bounds to the function at $x = 0$.*

We also have the following useful property of sums of strongly convex functions.

Theorem 4.6 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be μ_1-strongly convex and $g : \mathbb{R}^d \to (-\infty, +\infty]$ be μ_2-strongly convex in a non-empty convex set $\mathcal{V} \subseteq \operatorname{dom} f \cap \operatorname{dom} g$ with respect to $\|\cdot\|$. Then, $f + g$ is $\mu_1 + \mu_2$-strongly convex in $\mathcal{V}$ with respect to $\|\cdot\|$.*

Proof It is enough to sum the inequality in the definition of strong convexity (Definition 4.1) for both functions. □

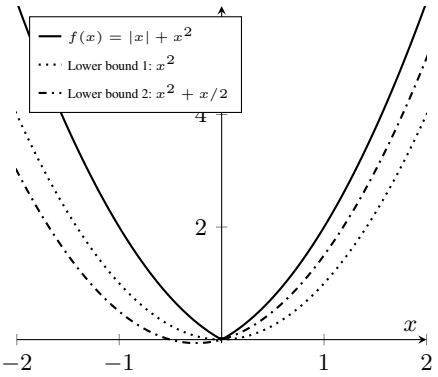


Figure 4.1 Possible lower bounds to the strongly convex non-differentiable function $f(x) = |x| + x^2$.

4.1.2 Online Subgradient Descent for Strongly Convex Losses

Theorem 4.7 *Let $\mathcal{V}$ be a non-empty closed convex set in $\mathbb{R}^d$. Assume that the functions $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ are μ_t-strongly convex with respect to $\|\cdot\|_2$ and subdifferentiable on $\mathcal{V}$, where $\mu_t > 0$. Use OSD in Algorithm 2.2 with stepsizes equal to $\eta_t = \frac{1}{\sum_{i=1}^t \mu_i}$. Then, for any $\boldsymbol{u} \in \mathcal{V}$, we have the following regret guarantee*

$$\sum_{t=1}^{T} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \frac{1}{2} \sum_{t=1}^{T} \frac{\|\boldsymbol{g}_t\|_2^2}{\sum_{i=1}^t \mu_i} \,.$$

Proof From the assumption of μ_t-strong convexity of the functions ℓ_t, we have that

$$\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u}) \leq \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle - \frac{\mu_t}{2} \|\boldsymbol{x}_t - \boldsymbol{u}\|_2^2 \,.$$

From the fact that $\eta_t = \frac{1}{\sum_{i=1}^t \mu_i}$, we have

$$\begin{aligned} \frac{1}{2\eta_1} - \frac{\mu_1}{2} &= 0, \\ \frac{1}{2\eta_t} - \frac{\mu_t}{2} &= \frac{1}{2\eta_{t-1}}, \; t = 2, \ldots, T \,. \end{aligned}$$

Hence, use Lemma 2.31 and sum from $t = 1, \ldots, T$, to obtain

$$\begin{aligned} &\sum_{t=1}^{T} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \\ &\quad \leq \sum_{t=1}^{T} \left(\frac{\|\boldsymbol{x}_t - \boldsymbol{u}\|_2^2}{2\eta_t} - \frac{\|\boldsymbol{x}_{t+1} - \boldsymbol{u}\|_2^2}{2\eta_t} - \frac{\mu_t \|\boldsymbol{x}_t - \boldsymbol{u}\|_2^2}{2} + \frac{\eta_t \|\boldsymbol{g}_t\|_2^2}{2} \right) \\ &\quad = -\frac{\|\boldsymbol{x}_2 - \boldsymbol{u}\|_2^2}{2\eta_1} + \sum_{t=2}^{T} \left(\frac{\|\boldsymbol{x}_t - \boldsymbol{u}\|_2^2}{2\eta_{t-1}} - \frac{\|\boldsymbol{x}_{t+1} - \boldsymbol{u}\|_2^2}{2\eta_t} \right) + \sum_{t=1}^{T} \frac{\eta_t \|\boldsymbol{g}_t\|_2^2}{2} \,. \end{aligned}$$

Observing that the first sum on the r.h.s. is a telescopic sum, we have the stated bound. □

Corollary 4.8 *Under the assumptions of Theorem 4.7, if in addition we have $\mu_t = \mu > 0$ and ℓ_t is L-Lipschitz with respect to $\|\cdot\|_2$ on an open set containing $\mathcal{V}$, for $t = 1, \dots, T$, then we have*

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \leq \frac{L^2}{2\mu} (1 + \ln T) \ .$$

Remark 4.9 *Notice that this corollary implicitly requires a bounded domain, otherwise the loss functions will not be Lipschitz, given that they are also strongly convex.*

Remark 4.10 *Corollary 4.8* does not *imply that for any T the regret will be smaller than using learning rates $\propto \frac{1}{L\sqrt{t}}$. Indeed, depending on the other quantities in the upper bounds, the upper bound in Corollary 4.8 is better than that of OSD with Lipschitz losses only for T large enough. Moreover, a better upper bound does not necessarily imply a better regret.*

Example 4.11 *Consider once again the guessing game in the first chapter: $\ell_t(x) = (x - y_t)^2$. Note that the loss functions are 2-strongly convex with respect to $|\cdot|$. Hence, setting $\eta_t = \frac{1}{2t}$ and $\ell_t'(x) = 2(x - y_t)$ gives a regret of $\ln(T) + 1$.*

We can also use the online-to-batch conversion on strongly convex stochastic problems.

Example 4.12 *As done before, we can use the online-to-batch conversion to use Corollary 4.8 to obtain stochastic subgradient descent algorithms for strongly convex stochastic functions. For example, consider the classic Support Vector Machine objective,*

$$\min_{\boldsymbol{x}} \ F(\boldsymbol{x}) := \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 + \frac{1}{N}\sum_{i=1}^{N} \max(1 - y_i\langle \boldsymbol{z}_i, \boldsymbol{x}\rangle, 0),$$

or any other regularized formulation like regularized logistic regression,

$$\min_{\boldsymbol{x}} \ F(\boldsymbol{x}) := \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 + \frac{1}{N}\sum_{i=1}^{N} \ln(1 + \exp(-y_i\langle \boldsymbol{z}_i, \boldsymbol{x}\rangle)),$$

where $\boldsymbol{z}_i \in \mathbb{R}^d$, $\|\boldsymbol{z}_i\|_2 \leq R$, and $y_i \in \{-1, 1\}$. First, notice that the minimizer of both expressions has to be in the L_2 ball of radius proportional to $\sqrt{\frac{1}{\lambda}}$ (proof left as an exercise). Hence, we can set $\mathcal{V}$ equal to this set. Then, setting $\ell_t(\boldsymbol{x}) = \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 + \max(1 - y_t\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle, 0)$ or $\ell_t(\boldsymbol{x}) = \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 + \ln(1 + \exp(-y_t\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle))$ results in λ-strongly convex loss functions. Using Corollary 4.8 and Theorem 3.1 gives immediately

$$\mathbb{E}\left[F\left(\frac{1}{T}\sum_{t=1}^{T} \boldsymbol{x}_t\right)\right] - \min_{\boldsymbol{x}} \ F(\boldsymbol{x}) = \mathcal{O}\left(\frac{\ln T}{\lambda T}\right) \ .$$

However, we can do better! We can use non-uniform weights in Theorem 3.1 to remove the log term and obtain the optimal convergence rate for the stochastic optimization of strongly convex functions. Observe that $\ell_t(\boldsymbol{x}) = \frac{\lambda t}{2}\|\boldsymbol{x}\|_2^2 + t\max(1 - y_t\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle, 0)$ or $\ell_t(\boldsymbol{x}) = \frac{\lambda t}{2}\|\boldsymbol{x}\|_2^2 + t\ln(1 + \exp(-y_t\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle))$ are λt-strongly convex loss functions. So, using Theorem 4.7, we have that $\eta_t = \frac{2}{\lambda t(t+1)}$ and Theorem 3.1 gives immediately

$$\mathbb{E}\left[F\left(\frac{2}{T(T+1)}\sum_{t=1}^{T} t\,\boldsymbol{x}_t\right)\right] - \min_{\boldsymbol{x}} \ F(\boldsymbol{x}) = \mathcal{O}\left(\frac{1}{\lambda T}\right) \ .$$

4.2 Adaptive Algorithms: $\mathscr{L}^\star$ bounds and AdaGrad

In this section, we will explore different conditions where we can get better regret upper bounds than $\mathcal{O}(DL\sqrt{T})$ as $T \to \infty$. Also, we will obtain these improved guarantees in an *automatic* way. That is, the algorithm will be *adaptive* to the characteristics of the sequence of loss functions, without having to rely on information about the future.

4.2.1 Adaptive Learning Rates for Online Subgradient Descent

Consider the minimization of the regret with linear losses:

$$\text{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle \,.$$

Using OSD, in Chapter 2, we said that the regret for bounded domains can be upper bounded by

$$\sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle \leq \frac{D^2}{2\eta_T} + \frac{1}{2}\sum_{t=1}^T \eta_t \|\boldsymbol{g}_t\|_2^2 \,.$$

With a fixed learning rate $\eta_t = \eta$, the learning rate that minimizes this upper bound on the regret is

$$\eta^\star = \frac{D}{\sqrt{\sum_{t=1}^T \|\boldsymbol{g}_t\|_2^2}} \,.$$

Unfortunately, as we said, this learning rate cannot be used because it assumes knowledge of the future rounds. However, we might be lucky, and we might try to just approximate it in each round using the knowledge up to time t. That is, we might try to use

$$\eta_t = \frac{D}{\sqrt{\sum_{i=1}^t \|\boldsymbol{g}_i\|_2^2}}, \tag{4.3}$$

and just skip the rounds in which $\boldsymbol{g}_t = \boldsymbol{0}$ to avoid possible divisions by 0. Observe that $\eta_T = \eta^\star$, so the first term of the regret would be exactly what we need! For the other term, the optimal learning rate would give us

$$\frac{1}{2}\sum_{t=1}^T \eta_T^\star \|\boldsymbol{g}_t\|_2^2 = \frac{1}{2} D \sqrt{\sum_{t=1}^T \|\boldsymbol{g}_t\|_2^2} \,.$$

Now, let's see what we obtain with our approximation in the other term of the regret:

$$\frac{1}{2}\sum_{t=1}^T \eta_t \|\boldsymbol{g}_t\|_2^2 = \frac{1}{2} D \sum_{t=1}^T \frac{\|\boldsymbol{g}_t\|_2^2}{\sqrt{\sum_{i=1}^t \|\boldsymbol{g}_i\|_2^2}} \,.$$

We need a way to upper-bound that sum. The way to treat these sums, as we did in Chapter 1, is to try to approximate them with integrals. So, we can use the following very handy lemma that generalizes a lot of similar specific ones.

Lemma 4.13 *Let $a_0, \dots, a_T \geq 0$ and $f : [0, +\infty) \to [0, +\infty)$ a non-increasing continuous function. Then*

$$\sum_{t=1}^{T} a_t f\left(a_0 + \sum_{i=1}^{t} a_i\right) \leq \int_{a_0}^{\sum_{t=0}^{T} a_t} f(x)\,\mathrm{d}x\,.$$

Proof Denote by $s_t = \sum_{i=0}^{t} a_i$.

$$a_t f\left(a_0 + \sum_{i=1}^{t} a_i\right) = a_t f(s_t) = \int_{s_{t-1}}^{s_t} f(s_t)dx \leq \int_{s_{t-1}}^{s_t} f(x)dx\,.$$

Summing over $t = 1, \dots, T$, we have the stated bound. □

Using this lemma with $f(x) = \frac{1}{\sqrt{x}}$ and $a_0 \to 0$, we have that

$$\frac{1}{2} D \sum_{t=1}^{T} \frac{\|\boldsymbol{g}_t\|_2^2}{\sqrt{\sum_{i=1}^{t} \|\boldsymbol{g}_i\|_2^2}} \leq D \sqrt{\sum_{t=1}^{T} \|\boldsymbol{g}_t\|_2^2}\,.$$

Surprisingly, this term is only a factor of 2 worse than what we would have got from the optimal choice of $\eta^\star$. However, this learning rate can be computed without knowledge of the future, and it can actually be used! Overall, with this choice, we get

$$\mathrm{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \leq \frac{3}{2} D \sqrt{\sum_{t=1}^{T} \|\boldsymbol{g}_t\|_2^2}\,. \tag{4.4}$$

Note that it is possible to improve the constant in front of the bound to $\sqrt{2}$ by multiplying the learning rates by $\frac{\sqrt{2}}{2}$. So, putting everything together, we have the following theorem.

Theorem 4.14 *Let $\mathcal{V} \subset \mathbb{R}^d$ be a closed non-empty convex set with diameter D, that is, $\max_{\boldsymbol{x},\boldsymbol{y} \in \mathcal{V}} \|\boldsymbol{x} - \boldsymbol{y}\|_2 \leq D$. Let $\ell_1, \dots, \ell_T$ be an arbitrary sequence of convex functions $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ subdifferentiable on $\mathcal{V}$ for $t = 1, \dots, T$. Pick any $\boldsymbol{x}_1 \in \mathcal{V}$, and run OSD with $\eta_t = \frac{\sqrt{2}D}{2\sqrt{\sum_{i=1}^{t} \|\boldsymbol{g}_i\|_2^2}}$, $t = 1, \dots, T$, and do not update on rounds when $\boldsymbol{g}_t = \boldsymbol{0}$. Then, $\forall \boldsymbol{u} \in \mathcal{V}$, the following regret bound holds*

$$\sum_{t=1}^{T} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq D \sqrt{2 \sum_{t=1}^{T} \|\boldsymbol{g}_t\|_2^2} = \sqrt{2} \min_{\eta > 0} \left(\frac{D^2}{2\eta} + \frac{\eta}{2} \sum_{t=1}^{T} \|\boldsymbol{g}_t\|_2^2\right).$$

The second equality in the theorem clearly shows the advantage of this learning rate: we obtain (almost) the same guarantee we would have obtained knowing the future gradients!

This is an interesting result on its own: it gives a principled way to set the learning rates with an almost optimal guarantee. Observe that this approach avoids knowing the Lipschitz constants of the functions, so the algorithm adapts to them. The downside is that it works only in the bounded case. Another important observation is that the sum of the squared gradients acts as an *intrinsic notion of time*, better suited than T to capture the dependence on time.

There are also other consequences of this simple regret upper bound as we will now see by specializing this result to the case that the losses are *smooth* or *self-bounded*.

4.2.2 Convex Analysis Bits: Dual Norms, Smooth, and Self-Bounded Functions

We now consider a family of loss functions that have the characteristic of being lower bounded by the squared norm of their subgradients. We will also introduce the concept of *dual norms*. While dual norms are not strictly needed for this topic, they give more generality, and at the same time, they allow me to slowly introduce some of the concepts that will be needed for the chapter on Online Mirror Descent.

Definition 4.15 *The **dual norm** $\|\cdot\|_\star$ of a norm $\|\cdot\|$ is defined as $\|\boldsymbol{\theta}\|_\star = \max_{\boldsymbol{x}:\|\boldsymbol{x}\|\leq 1} \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle$.*

Remark 4.16 *The definition of dual norm immediately implies $\langle \boldsymbol{\theta}, \boldsymbol{x}\rangle \leq \|\boldsymbol{\theta}\|_\star \|\boldsymbol{x}\|$.*

Example 4.17 *The dual norm of the L_2 norm is the L_2 norm. We can easily prove it. First of all, if $\boldsymbol{\theta} = \mathbf{0}$, then the dual norm is 0 too. Hence, let's assume that $\boldsymbol{\theta} \neq \mathbf{0}$. We have $\|\boldsymbol{\theta}\|_\star = \max_{\boldsymbol{x}:\|\boldsymbol{x}\|_2\leq 1} \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle \leq \|\boldsymbol{\theta}\|_2$ by Cauchy–Schwarz inequality. Also, set $\boldsymbol{v} = \frac{\boldsymbol{\theta}}{\|\boldsymbol{\theta}\|_2}$, so $\max_{\boldsymbol{x}:\|\boldsymbol{x}\|_2\leq 1} \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle \geq \langle \boldsymbol{\theta}, \boldsymbol{v}\rangle = \|\boldsymbol{\theta}\|_2$.*

Example 4.18 *Let $p \geq 1$. The L_p norm of a vector $\boldsymbol{x} \in \mathbb{R}^d$ is defined as $\|\boldsymbol{x}\|_p = (\sum_{i=1}^d |x_i|^p)^{1/p}$. The dual norm is the q-norm where $\frac{1}{p} + \frac{1}{q} = 1$. Note that the dual of the L_1 norm is the L_∞ norm, defined as $\|\boldsymbol{x}\|_\infty = \max_{i=1,\dots,d} |x_i|$. The proof is left as an exercise to the reader.*

Example 4.19 *Let $\boldsymbol{A}$ be a positive definite matrix, then it is possible to show that $\|\boldsymbol{x}\|_{\boldsymbol{A}} := \sqrt{\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}}$ is a norm. The dual norm is $\|\boldsymbol{x}\|_{\boldsymbol{A}^{-1}} = \sqrt{\boldsymbol{x}^\top \boldsymbol{A}^{-1}\boldsymbol{x}}$. In fact, we have that the dual norm of $\|\cdot\|_{\boldsymbol{A}}$ is defined as*

$$\begin{aligned}\|\boldsymbol{\theta}\|_\star &= \max_{\boldsymbol{x}:\|\boldsymbol{x}\|_{\boldsymbol{A}}\leq 1} \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle = \max_{\boldsymbol{x}:\boldsymbol{x}^\top\boldsymbol{A}\boldsymbol{x}\leq 1} \boldsymbol{\theta}^\top \boldsymbol{x} = \max_{\boldsymbol{y}:\boldsymbol{y}^\top\boldsymbol{y}\leq 1} \boldsymbol{\theta}^\top \boldsymbol{A}^{-1/2}\boldsymbol{y} = \max_{\boldsymbol{y}:\|\boldsymbol{y}\|_2\leq 1} (\boldsymbol{A}^{-1/2}\boldsymbol{\theta})^\top \boldsymbol{y} \\ &= \|\boldsymbol{A}^{-1/2}\boldsymbol{\theta}\|_2 = \sqrt{\boldsymbol{\theta}^\top \boldsymbol{A}^{-1}\boldsymbol{\theta}},\end{aligned}$$

where we have used the change of variable $\boldsymbol{y} = \boldsymbol{A}^{1/2}\boldsymbol{x}$ in the third equality and the dual norm of the L_2 norm from Example 4.17 in the second to last equality.

If you do not know the concept of *operator norms*, the concept of dual norm can be a bit weird at first. One way to understand it is that it is a way to measure how "big" linear functionals are. For example, consider the linear function $f(\boldsymbol{x}) = \langle \boldsymbol{z}, \boldsymbol{x}\rangle$, we want to try to understand how big it is. So, we can measure $\max_{\boldsymbol{x}\neq 0} \frac{\langle \boldsymbol{z}, \boldsymbol{x}\rangle}{\|\boldsymbol{x}\|}$, that is, we measure how big the output of the linear functional is compared to its input $\boldsymbol{x}$, where $\boldsymbol{x}$ is measured with some norm. Now, you can show that the above is equivalent to the dual norm of $\boldsymbol{z}$.

We have the following lemma that will be useful in the later chapters.

Lemma 4.20 *Let $\|\cdot\|$ be a norm on $\mathbb{R}^d$, and let $\|\cdot\|_\star$ be its dual norm. Then, for every $\boldsymbol{\theta} \in \mathbb{R}^d$ and every $\alpha \in \mathbb{R}_{\geq 0}$, there exists $\boldsymbol{x} \in \mathbb{R}^d$ such that $\|\boldsymbol{x}\| = \alpha$ and $\langle \boldsymbol{\theta}, \boldsymbol{x}\rangle = \|\boldsymbol{\theta}\|_\star \|\boldsymbol{x}\|$.*

Proof If $\alpha = 0$, choose $\boldsymbol{x} = \mathbf{0}$. Hence, assume $\alpha > 0$.

If $\boldsymbol{\theta} = \mathbf{0}$, choose any $\boldsymbol{x}$ such that $\|\boldsymbol{x}\| = \alpha$. Otherwise, by the definition of dual norm, there exists $\boldsymbol{v}$ with $\|\boldsymbol{v}\| \leq 1$ such that

$$\langle \boldsymbol{\theta}, \boldsymbol{v}\rangle = \max_{\boldsymbol{u}:\|\boldsymbol{u}\|\leq 1} \langle \boldsymbol{\theta}, \boldsymbol{u}\rangle = \|\boldsymbol{\theta}\|_\star .$$

Since $\boldsymbol{\theta} \neq \mathbf{0}$, the function $\boldsymbol{v} \mapsto \langle \boldsymbol{\theta}, \boldsymbol{v} \rangle$ is nonconstant and linear, and so it attains its maximum over $\{\boldsymbol{v} : \|\boldsymbol{v}\| \leq 1\}$ on the boundary. Thus, we can choose $\boldsymbol{v}$ such that $\|\boldsymbol{v}\| = 1$. So, $\boldsymbol{x} = \alpha \boldsymbol{v}$ satisfies $\|\boldsymbol{x}\| = \alpha$ and $\langle \boldsymbol{\theta}, \boldsymbol{x} \rangle = \|\boldsymbol{\theta}\|_\star \|\boldsymbol{x}\|$. □

We can also extend Theorem 2.30 to generic dual norms.

Theorem 4.21 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be proper and convex. Then, the following are equivalent:*

1. *f is L-Lipschitz on $\operatorname{int} \operatorname{dom} f$ with respect to $\|\cdot\|$, i.e., $|f(\boldsymbol{x}) - f(\boldsymbol{y})| \leq L\|\boldsymbol{x} - \boldsymbol{y}\| \quad \forall \boldsymbol{x}, \boldsymbol{y} \in \operatorname{int} \operatorname{dom} f$.*
2. *For every $\boldsymbol{x} \in \operatorname{int} \operatorname{dom} f$ and every $\boldsymbol{g} \in \partial f(\boldsymbol{x})$, $\|\boldsymbol{g}\|_\star \leq L$.*

Proof First assume that f is L-Lipschitz on $\operatorname{int} \operatorname{dom} f$.

Fix $\boldsymbol{x} \in \operatorname{int} \operatorname{dom} f$ and $\boldsymbol{g} \in \partial f(\boldsymbol{x})$. By the definition of dual norm,

$$\|\boldsymbol{g}\|_\star = \sup_{\|\boldsymbol{u}\| \leq 1} \langle \boldsymbol{g}, \boldsymbol{u} \rangle \ .$$

So, it suffices to show that $\langle \boldsymbol{g}, \boldsymbol{u} \rangle \leq L$ for every $\boldsymbol{u}$ with $\|\boldsymbol{u}\| \leq 1$.

Let $\boldsymbol{u} \in \mathbb{R}^d$ satisfy $\|\boldsymbol{u}\| \leq 1$. Since $\boldsymbol{x} \in \operatorname{int} \operatorname{dom} f$, for $\epsilon > 0$ small enough we have

$$\boldsymbol{y} := \boldsymbol{x} + \epsilon \boldsymbol{u} \in \operatorname{int} \operatorname{dom} f \ .$$

Using L-Lipschitz of f and the subgradient inequality, we obtain

$$L\epsilon\|\boldsymbol{u}\| = L\|\boldsymbol{y} - \boldsymbol{x}\| \geq |f(\boldsymbol{y}) - f(\boldsymbol{x})| \geq f(\boldsymbol{y}) - f(\boldsymbol{x}) \geq \langle \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{x} \rangle = \epsilon \langle \boldsymbol{g}, \boldsymbol{u} \rangle \ .$$

Hence, $\langle \boldsymbol{g}, \boldsymbol{u} \rangle \leq L\|\boldsymbol{u}\| \leq L$. Taking the supremum over all $\boldsymbol{u}$ with $\|\boldsymbol{u}\| \leq 1$ gives $\|\boldsymbol{g}\|_\star \leq L$.

Now assume conversely that for every $\boldsymbol{z} \in \operatorname{int} \operatorname{dom} f$ and every $\boldsymbol{h} \in \partial f(\boldsymbol{z})$, $\|\boldsymbol{h}\|_\star \leq L$. Take any $\boldsymbol{x}, \boldsymbol{y} \in \operatorname{int} \operatorname{dom} f$. Since f is convex and $\boldsymbol{x} \in \operatorname{int} \operatorname{dom} f$, the subdifferential $\partial f(\boldsymbol{x})$ is nonempty, so choose $\boldsymbol{g} \in \partial f(\boldsymbol{x})$. Then, by the subgradient inequality, we have

$$f(\boldsymbol{x}) - f(\boldsymbol{y}) \leq \langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{y} \rangle \leq \|\boldsymbol{g}\|_\star \|\boldsymbol{x} - \boldsymbol{y}\| \leq L\|\boldsymbol{x} - \boldsymbol{y}\| \ .$$

Exchanging the roles of $\boldsymbol{x}$ and $\boldsymbol{y}$ and combining the two inequalities, we obtain

$$|f(\boldsymbol{x}) - f(\boldsymbol{y})| \leq L\|\boldsymbol{x} - \boldsymbol{y}\| \ .$$ □

Now, we can introduce smooth functions, using the dual norms defined above.

Definition 4.22 *Let f be differentiable on an open set containing $\mathcal{V}$. We say that f is s-**smooth** with respect to $\|\cdot\|$ if $\|\nabla f(\boldsymbol{x}) - \nabla f(\boldsymbol{y})\|_\star \leq s\|\boldsymbol{x} - \boldsymbol{y}\|$ for all $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{V}$.*

Keeping in mind the intuition above on dual norms, taking the dual norm of a gradient makes sense if you associate each gradient with the linear functional $\langle \nabla f(\boldsymbol{y}), \boldsymbol{x} \rangle$, that is, the one needed to create a linear approximation of f.

Remark 4.23 *Note that smoothness does not imply convexity.*

Smooth functions have many properties, for example, a smooth function can be upper and lower bounded by a quadratic.

Lemma 4.24 *Let $f : \mathcal{V} \to \mathbb{R}$ be s-smooth with respect to $\|\cdot\|$. Then, for any $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{V}$ such that the line segment between $\boldsymbol{x}$ and $\boldsymbol{y}$ is in $\mathcal{V}$, we have*

$$|f(\boldsymbol{y}) - f(\boldsymbol{x}) - \langle \nabla f(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x}\rangle| \leq \frac{s}{2}\|\boldsymbol{y} - \boldsymbol{x}\|^2 .$$

Proof First, notice that by the definition of smoothness, $\nabla f : \mathcal{V} \to \mathbb{R}^d$ is Lipschitz and so continuous. Hence, by the fundamental theorem of calculus, we have

$$\begin{aligned} f(\boldsymbol{y}) &= f(\boldsymbol{x}) + \int_0^1 \langle \nabla f(\boldsymbol{x} + \tau(\boldsymbol{y} - \boldsymbol{x})), \boldsymbol{y} - \boldsymbol{x}\rangle \,\mathrm{d}\tau \\ &= f(\boldsymbol{x}) + \langle \nabla f(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x}\rangle + \int_0^1 \langle \nabla f(\boldsymbol{x} + \tau(\boldsymbol{y} - \boldsymbol{x})) - \nabla f(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x}\rangle \,\mathrm{d}\tau . \end{aligned}$$

Therefore,

$$\begin{aligned} |f(\boldsymbol{y}) - f(\boldsymbol{x}) - \langle \nabla f(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x}\rangle| &= \left| \int_0^1 \langle \nabla f(\boldsymbol{x} + \tau(\boldsymbol{y} - \boldsymbol{x})) - \nabla f(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x}\rangle \,\mathrm{d}\tau \right| \\ &\leq \int_0^1 |\langle \nabla f(\boldsymbol{x} + \tau(\boldsymbol{y} - \boldsymbol{x})) - \nabla f(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x}\rangle| \,\mathrm{d}\tau \\ &\leq \int_0^1 \|\nabla f(\boldsymbol{x} + \tau(\boldsymbol{y} - \boldsymbol{x})) - \nabla f(\boldsymbol{x})\|_\star \|\boldsymbol{y} - \boldsymbol{x}\| \,\mathrm{d}\tau \\ &\leq \int_0^1 \tau s \|\boldsymbol{y} - \boldsymbol{x}\|^2 \,\mathrm{d}\tau = \frac{s}{2}\|\boldsymbol{y} - \boldsymbol{x}\|^2 . \end{aligned}$$

□

In the following, we will also need the following property.

Theorem 4.25 *Let $f : \mathbb{R}^d \to \mathbb{R}$ be s-smooth with respect to $\|\cdot\|$, and bounded from below. Then, for all $\boldsymbol{x} \in \mathbb{R}^d$, we have*

$$\|\nabla f(\boldsymbol{x})\|_\star^2 \leq 2s(f(\boldsymbol{x}) - \inf_{\boldsymbol{y} \in \mathbb{R}^d} f(\boldsymbol{y})) .$$

Proof From Lemma 4.24, for any $\boldsymbol{x}, \boldsymbol{v} \in \mathbb{R}^d$, we have

$$\langle -\nabla f(\boldsymbol{x}), \boldsymbol{v}\rangle - \frac{s}{2}\|\boldsymbol{v}\|^2 \leq f(\boldsymbol{x}) - f(\boldsymbol{x} + \boldsymbol{v}) \leq f(\boldsymbol{x}) - \inf_{\boldsymbol{y} \in \mathbb{R}^d} f(\boldsymbol{y}) .$$

Given that this holds for any $\boldsymbol{v}$, we can take the supremum of the l.h.s. with respect to $\boldsymbol{v}$. Using Example 6.19, we have

$$\frac{1}{2s}\|\nabla f(\boldsymbol{x})\|_\star^2 = \sup_{\boldsymbol{v}} \langle -\nabla f(\boldsymbol{x}), \boldsymbol{v}\rangle - \frac{s}{2}\|\boldsymbol{v}\|^2 \leq f(\boldsymbol{x}) - \inf_{\boldsymbol{y} \in \mathbb{R}^d} f(\boldsymbol{y}) .$$ □

Sometimes we do not need smoothness or differentiability, but only the property of the above theorem. We call convex functions that satisfy such an inequality *self-bounded*.

Definition 4.26 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be bounded from below, and subdifferentiable on a set $\mathcal{V}$. We say that f is **s-self-bounded** in $\mathcal{V}$ with respect to $\|\cdot\|$ if*

$$\|\boldsymbol{g}\|_\star^2 \leq 2s(f(\boldsymbol{x}) - \inf_{\boldsymbol{y} \in \mathbb{R}^d} f(\boldsymbol{y})), \quad \forall \boldsymbol{x} \in \mathcal{V}, \forall \boldsymbol{g} \in \partial f(\boldsymbol{x}) .$$

Remark 4.27 *If $\mathcal{V}$ is convex, then self-bounded functions are also convex, because we are assuming that they are subdifferentiable on $\mathcal{V}$ (Theorem 2.21).*

Clearly, a convex s-smooth function is also s-self-bounded, but the converse is not true, as shown in the next example.

Example 4.28 *Let $f : \mathbb{R} \to \mathbb{R}$ be defined as $f(x) = \frac{1}{2}x^2 + |x - 2|$. The function f is not differentiable at 2, hence it is not smooth. However, it is easy to verify that it is 9-self-bounded.*

4.2.3 $\mathscr{L}^\star$ bounds

We now introduce the $\mathscr{L}^\star$ bounds, which depend on the cumulative competitor loss that is usually denoted by $\mathscr{L}^\star$.

Assume that the loss functions $\ell_1, \dots, \ell_T$ are s-self-bounded on $\mathcal{V}$ with respect to $\|\cdot\|_2$. From this assumption, without loss of generality, we can assume that each of them is bounded from below by 0. Under these assumptions, we can obtain bounds that depend on the cumulative loss of the competitor rather than time.

From the regret of OSD (that is the same as in Theorem 2.13) and Definition 4.26, for a constant learning rate η we obtain

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \le \frac{\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2}{2\eta} + \eta \sum_{t=1}^{T} s\ell_t(\boldsymbol{x}_t), \quad \forall \boldsymbol{u} \in \mathcal{V} .$$

Reordering, it implies

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \le \frac{\eta s}{1 - \eta s} \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) + \frac{\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2}{2\eta(1 - \eta s)}, \quad \forall \boldsymbol{u} \in \mathcal{V} .$$

Assuming $\eta \le \frac{1}{2s}$, we simplify this regret upper bound in

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \le 2\eta s \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) + \frac{\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2}{\eta}, \quad \forall \boldsymbol{u} \in \mathcal{V} .$$

This is already an interesting result because it guarantees that a fixed learning rate that depends only on s can achieve a vanishing average regret if there exists a competitor $\boldsymbol{u} \in \mathcal{V}$ whose cumulative loss grows sublinearly. However, we could do better. In fact, for a fixed $\boldsymbol{u} \in \mathcal{V}$, setting $\eta = \min(\frac{\alpha}{\sqrt{2s\sum_{t=1}^{T}\ell_t(\boldsymbol{u})}}, \frac{1}{2s})$ for $\alpha > 0$, we obtain

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \le \max\left[\left(\frac{\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2}{\alpha} + \alpha\right)\sqrt{2s\sum_{t=1}^{T}\ell_t(\boldsymbol{u})}, 2s(\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2 + \alpha^2)\right].$$

Comparing this bound to the one of Online Gradient Descent (OGD) with Lipschitz losses, we see that here the dependence is on $\sqrt{\sum_{t=1}^{T}\ell_t(\boldsymbol{u})}$ instead of $\sqrt{T}$. The cumulative loss of the competitor can be much smaller than T and, in particular, can be even 0. In this case, the regret is upper-bounded by a constant. Moreover, in this latter case, we can afford to use a learning rate η that depends only on the self-boundedness constant. However, this result is not fully satisfactory because it requires the knowledge of the future through the cumulative

loss of the competitor. In the following, we show how to easily get rid of this limitation with a different choice of the learning rate.

In fact, under the same assumptions on the losses, from the regret in Theorem 4.14 and Definition 4.26, we immediately obtain

$$\text{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}) \leq 2D\sqrt{s\sum_{t=1}^T \ell_t(\boldsymbol{x}_t)}, \quad \forall \boldsymbol{u} \in \mathcal{V},$$

where D is the diameter of $\mathcal{V}$, assumed to be bounded. This is an implicit bound, in the sense that $\sum_{t=1}^T \ell_t(\boldsymbol{x}_t)$ appears on both sides of the inequality. To make it explicit, we will use the following simple lemma (proof left as an exercise).

Lemma 4.29 *Let $a, b, c \geq 0$, and $x \geq 0$ such that $x - \sqrt{ax+b} \leq c$. Then $x \leq \frac{a}{2} + c + \sqrt{\frac{a^2}{4} + b + ac} \leq a + c + \sqrt{b + ac}$.*

So, we have the following theorem.

Theorem 4.30 *Let $\mathcal{V} \subset \mathbb{R}^d$ be a closed non-empty convex set with diameter D, that is, $D := \max_{\boldsymbol{x},\boldsymbol{y}\in\mathcal{V}} \|\boldsymbol{x} - \boldsymbol{y}\|_2$. Let $\ell_1, \dots, \ell_T$ be an arbitrary sequence of non-negative convex functions $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ s-self-bounded in $\mathcal{V}$ with respect to $\|\cdot\|_2$. Pick any $\boldsymbol{x}_1 \in \mathcal{V}$, run projected OSD with $\eta_t = \frac{\sqrt{2}D}{2\sqrt{\sum_{i=1}^t \|\boldsymbol{g}_i\|_2^2}}$, $t = 1, \dots, T$, and do not update on rounds in which $\boldsymbol{g}_t = \boldsymbol{0}$. Then, $\forall \boldsymbol{u} \in \mathcal{V}$, the following regret bound holds*

$$\text{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}) \leq 4sD^2 + 2D\sqrt{s\sum_{t=1}^T \ell_t(\boldsymbol{u})}\,.$$

This regret guarantee is very interesting because in the worst case it is $\mathcal{O}(\sqrt{T})$, but in the best case scenario it becomes a constant! In fact, if there exists a $\boldsymbol{u} \in \mathcal{V}$ such that $\sum_{t=1}^T \ell_t(\boldsymbol{u}) = 0$ we get a constant regret. Basically, if the losses are "easy", the algorithm *adapts* to this situation and gives us a better regret.

Example 4.31 *Consider an online linear classification problem with the squared hinge loss. So, each loss is defined as $\ell_t(\boldsymbol{x}) = \max(1 - y_t\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle, 0)^2$ for labels $y_t \in \{-1, 1\}$ and features $\boldsymbol{z}_t \in \mathbb{R}^d$. If we assume that $\|\boldsymbol{z}_t\|_2$ is bounded for all t, then the losses are self-bounded (and even smooth) (proof left as an exercise). Under this assumption, if the problem is linearly separable in $\mathcal{V}$, i.e., there exists $\boldsymbol{u} \in \mathcal{V}$ such that $\ell_t(\boldsymbol{u}) = 0$ for all t, Theorem 4.30 will guarantee a constant regret.*

4.2.4 AdaGrad

We now present another application of the regret bound in (4.4). *AdaGrad*, that stands for Adaptive Gradient, is an Online Convex Optimization (OCO) algorithm that aims at being adaptive to the sequence of (sub)gradients.

We will present a proof that only covers hyperrectangles as feasible sets $\mathcal{V}$. On the other hand, the restriction makes the proof almost trivial. Let's see how it works.

AdaGrad has two key ingredients:

Algorithm 4.1 AdaGrad for Hyperrectangles

Require: $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : a_i \le x_i \le b_i\}$, $\boldsymbol{x}_1 \in \mathcal{V}$
for $t = 1$ **to** T **do**
  Output $\boldsymbol{x}_t$
  Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable on $\mathcal{V}$
  Set $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$
  for $i = 1$ **to** d **do**
    if $g_{t,i} \neq 0$ **then**
      $x_{t+1,i} = \max(\min(x_{t,i} - \eta_{t,i} g_{t,i}, b_i), a_i)$ where $\eta_{t,i} = \frac{\sqrt{2}D_i}{2\sqrt{\sum_{j=1}^t g_{j,i}^2}}$
    else
      $x_{t+1,i} = x_{t,i}$
    end if
  end for
end for

- A coordinate-wise learning process;
- The adaptive learning rates in (4.3).

For the first ingredient, as we said in Section 2.3, the regret of any OCO problem can be upper bounded by the regret of the Online Linear Optimization (OLO) problem. That is,

$$\sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}) \le \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle \ .$$

Now, the essential observation is to explicitly write the inner product as a sum of products over the individual coordinates:

$$\begin{aligned}\sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle &= \sum_{t=1}^T \sum_{i=1}^d g_{t,i} x_{t,i} - \sum_{t=1}^T \sum_{i=1}^d g_{t,i} u_i \\ &= \sum_{i=1}^d \left(\sum_{t=1}^T g_{t,i} x_{t,i} - \sum_{t=1}^T g_{t,i} u_i \right) = \sum_{i=1}^d \mathrm{Regret}_{T,i}(u_i),\end{aligned}$$

where we denoted by $\mathrm{Regret}_{T,i}(u_i)$ the regret of the one-dimensional Online Linear Optimization (OLO) problem over coordinate i, that is $\sum_{t=1}^T g_{t,i} x_{t,i} - \sum_{t=1}^T g_{t,i} u_i$. In words, *we can decompose the original linear regret as the sum of d OLO regret minimization problems, and we can try to focus on each one of them separately*.

A good candidate for the one-dimensional problems is OSD with the learning rates in (4.3). We can specialize the regret in (4.4) to the one-dimensional case for linear losses, so we get for each coordinate i

$$\sum_{t=1}^T g_{t,i} x_{t,i} - \sum_{t=1}^T g_{t,i} u_i \le \sqrt{2} D_i \sqrt{\sum_{t=1}^T g_{t,i}^2} \ .$$

This choice gives us the **AdaGrad** algorithm in Algorithm 4.1.

Putting everything together, we have the following regret guarantee immediately.

Theorem 4.32 *Let $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : a_i \le x_i \le b_i\}$ with diameters along each coordinate equal to $D_i = b_i - a_i < \infty$. Let $\ell_1, \dots, \ell_T$ be an arbitrary sequence of convex functions $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ subdifferentiable on $\mathcal{V}$ for $t = 1, \dots, T$. Pick any $\boldsymbol{x}_1 \in \mathcal{V}$. Then, $\forall \boldsymbol{u} \in \mathcal{V}$, Algorithm 4.1 guarantees*

$$\mathrm{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}) \le \sqrt{2} \sum_{i=1}^d D_i \sqrt{\sum_{t=1}^T g_{t,i}^2} \,.$$

Is this a better regret bound compared to the one in Theorem 4.14? It depends! To compare the two, let's first consider the case that $\mathcal{V}$ is a hyperrectangle. Then, we have to compare

$$D \sqrt{\sum_{t=1}^T \|\boldsymbol{g}_t\|_2^2} \qquad \text{versus} \qquad \sum_{i=1}^d D_i \sqrt{\sum_{t=1}^T g_{t,i}^2} \,.$$

From Cauchy–Schwarz, we have that $\sum_{i=1}^d D_i \sqrt{\sum_{t=1}^T g_{t,i}^2} \le D \sqrt{\sum_{t=1}^T \|\boldsymbol{g}_t\|_2^2}$. So, *assuming the same sequence of subgradients*, AdaGrad never has worse regret on hyperrectangles. For a more precise quantification of the gain of AdaGrad, let's now assume that $\mathcal{V}$ is a hypersquare. Also, note that

$$\sqrt{\sum_{t=1}^T \|\boldsymbol{g}_t\|_2^2} \le \sum_{i=1}^d \sqrt{\sum_{t=1}^T g_{t,i}^2} \le \sqrt{d} \sqrt{\sum_{t=1}^T \|\boldsymbol{g}_t\|_2^2}, \tag{4.5}$$

where the lower bound is by the fact that the L_1 norm is bigger than the L_2 norm, and the upper bound is given by Cauchy–Schwarz inequality. So, in the case that $\mathcal{V}$ is a hypercube we have $D_i = D_\infty = \max_{\boldsymbol{x},\boldsymbol{y}} \|\boldsymbol{x} - \boldsymbol{y}\|_\infty$ and $D = \sqrt{d} D_\infty$, the bound of AdaGrad is between $1/\sqrt{d}$ and 1 times the bound of Theorem 4.14. In other words, if we are lucky with the subgradients, we might save a factor of $\sqrt{d}$ in the guarantee.

However, what happens if the domain is an L_2 ball? First of all, it is possible to generalize AdaGrad to work on L_2 balls, and the guarantee remains the same. Hence, in this case, we have $D_i = D_\infty = D$, so from (4.5), we have

$$\frac{1}{\sqrt{d}} \sum_{i=1}^d D_i \sqrt{\sum_{t=1}^T g_{t,i}^2} \le D \sqrt{\sum_{t=1}^T \|\boldsymbol{g}_t\|_2^2} \le \sum_{i=1}^d D_i \sqrt{\sum_{t=1}^T g_{t,i}^2} \,.$$

Hence, for an L_2 ball, the opposite happens: the bound in Theorem 4.14 is never worse than the one of AdaGrad and, depending on the subgradients, we can gain a $\sqrt{d}$ factor. Overall, the *shape of the domain* determines the potential gain of one approach over the other, and the specific sequence of subgradients determines the actual gain. It is possible to show that hyperrectangles are indeed the best domains for AdaGrad. We will explore this issue of choosing the online algorithm based on the shape of the feasible set $\mathcal{V}$ when we introduce Online Mirror Descent in Chapter 6.

Another big advantage of AdaGrad is the property of being *coordinate-wise scale-free*, according to the following definition.

Definition 4.33 *We say that an online learning algorithm is* ***coordinate-wise scale-free***

*when the iterates of the algorithm do not change if, assuming the same sequence of subgradients, each coordinate of the subgradients is multiplied by a different positive constant. We will say that the algorithm is simply **scale-free** if the same property holds when each coordinate is multiplied by the same positive constant.*

Another way to say it is that the update of AdaGrad is invariant to the units of each coordinate of the subgradients. This fact is not immediately apparent from the regret because by scaling the coordinate of the subgradients, the optimal solution $\boldsymbol{u}$ would also scale accordingly, but the fixed diameters of the feasible set hide it. This might be useful in the case that the ranges of coordinates of the gradients are vastly different from one another. Indeed, this does happen in many machine learning problems, for example, in the stochastic optimization of deep neural networks, where the first layers have gradients of different magnitudes compared to the last layers.

Remark 4.34 *AdaGrad is usually known as a stochastic optimization algorithm, but in reality, it was proposed for the online setting. To use it as a stochastic algorithm, you should use an online-to-batch conversion, otherwise you do not have any guarantee of convergence.*

4.3 History Bits

The concept of strong convexity was first defined in Polyak (1966).

The logarithmic regret in Corollary 4.8 was shown for the first time in the seminal paper Hazan et al. (2006, 2007). The general statement in Theorem 4.7 was proven by Hazan et al. (2008).

The non-uniform averaging for the online-to-batch conversion of Example 4.12 is from Lacoste-Julien et al. (2012), but there it is not proposed as an online-to-batch conversion. The basic idea of solving the Support Vector Machine (SVM) problem with OSD and online-to-batch conversion of Example 4.12 was the Pegasos algorithm (Shalev-Shwartz et al., 2007), for many years the most used optimizer for SVMs.

The adaptive learning rate in (4.3) first appeared in Streeter and McMahan (2010). However, similar methods were used a long time before. Indeed, the key observation to approximate oracle quantities with estimates up to time t was first proposed in the self-confident algorithms (Auer et al., 2002c), where the learning rate is inversely proportional to the square root of the cumulative loss of the algorithm, and for self-bounded losses it implies the $\mathscr{L}^\star$ bounds similar to the one in Theorem 4.30. The $\mathscr{L}^\star$ bound for the square loss and linear predictors was introduced by Cesa-Bianchi et al. (1996). In the past, several works focused on obtaining regret upper bounds depending on constant times $\mathscr{L}^\star$ (see, e.g., Kivinen and Warmuth, 1997), however, these guarantees are meaningful only if $\mathscr{L}^\star$ is sublinear in T. Zhang (2004) explored the use of $\mathscr{L}^\star$ bounds in stochastic optimization. The observation that the cumulative sum of the squared gradients acts as an intrinsic notion of time comes from the statistics literature, see the discussion in Blackwell and Freedman (1973).

AdaGrad was proposed in basically identical form independently by two groups at the same conference: McMahan and Streeter (2010) and Duchi et al. (2010). The analysis presented here is the one in Streeter and McMahan (2010) that does not handle generic feasible sets and does not support the "full-matrices" proposed in Duchi et al. (2010), i.e., full-matrix learning rates instead of diagonal ones. However, in machine learning applications, AdaGrad

is usually used without a projection step (even if doing so provably destroys the worst-case performance, see Theorem 5.4). Also, in the adversarial setting, full matrices do not seem to offer advantages in terms of regret compared to diagonal ones, see the discussion in Cutkosky (2020b, Section 5).

The AdaGrad learning rate is usually written as

$$\eta_{t,i} = \frac{D_i}{\epsilon + \sqrt{\sum_{j=1}^{t} g_{j,i}^2}},$$

where $\epsilon > 0$ is a small constant used to prevent division by zero. In reality, ϵ is not necessary: there should be no update when the coordinate of the gradient is 0 (Orabona and Pál, 2015, 2018; Agarwal et al., 2020). Moreover, removing ϵ makes the updates coordinate-wise scale-free, as stressed in Orabona and Pál (2015, 2018). Scale-freeness in online learning has been introduced in Cesa-Bianchi et al. (2005, 2007) for the setting of Learning with Expert Advice (LEA) and in Orabona and Pál (2015, 2018) for OCO.

AdaGrad inspired an incredible number of clones, most of them with similar, worse, or no regret guarantees. The keyword "adaptive" itself has shifted its meaning over time. It used to denote the ability of the algorithm to obtain the same guarantee as if it knew in advance a particular property of the data (i.e., adaptive to the gradients/noise/scale = (almost) same performance as if it knew the gradients/noise/scale in advance). Indeed, in statistics, this keyword is used with the same meaning. Nowadays, instead "adaptive learning rates" seems to denote any kind of coordinate-wise learning rates that do not guarantee anything in particular.

4.4 Exercises

Problem 4.1 *Prove that OSD in Example 4.11 with $x_1 = 0$ is exactly the Follow-the-Leader strategy for that particular problem.*

Problem 4.2 *Prove that $\ell_t(\boldsymbol{x}) = \|\boldsymbol{x} - \boldsymbol{z}_t\|_2^2$ is 2-strongly convex with respect to $\|\cdot\|_2$, derive the OSD update for it, and its regret guarantee.*

Problem 4.3 *Show that online subgradient descent on a bounded domain $\mathcal{V}$ with learning rates $\eta_t \propto 1/t$ obtains a $\mathcal{O}(\ln(1 + \mathscr{L}^\star))$ regret bound for Lipschitz, self-bounded, and strongly convex losses.*

Problem 4.4 *Prove that the logistic loss $\ell(\boldsymbol{x}) = \ln(1+\exp(-y\langle \boldsymbol{z}, \boldsymbol{x}\rangle))$, where $\|\boldsymbol{z}\|_2 \leq 1$ and $y \in \{-1, 1\}$ is $\frac{1}{4}$-smooth with respect to $\|\cdot\|_2$.*

Problem 4.5 *Prove Lemma 4.29.*

5

Lower Bounds for Online Linear Optimization

In this chapter, we will present some lower bounds for Online Linear Optimization (OLO). Since linear losses are convex, this immediately gives us lower bounds for Online Convex Optimization (OCO). We will consider both the constrained and the unconstrained case. The lower bounds in this chapter are important because they inform us about which algorithms are optimal and where the gaps in our knowledge are.

5.1 Lower Bound for Bounded Online Linear Optimization

We will first consider the constrained bounded case. Finding a lower bound amounts to finding a strategy for the adversary that forces a certain regret onto the algorithm, *no matter what the algorithm does*. We will use the probabilistic method to construct our lower bound.

The basic method relies on the fact that if for a given $K \in \mathbb{R}$ we can construct a sequence of random vectors $\tilde{\boldsymbol{g}}_1, \dots, \tilde{\boldsymbol{g}}_T$ such that

$$\mathbb{E}\left[\sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t \rangle\right] \geq K,$$

this implies that there exists a sequence $\boldsymbol{g}_1, \dots, \boldsymbol{g}_T$ among all the possible random sequences such that

$$\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \geq K \ .$$

It is easy to see why this is true: if for *all* sequences $\sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t \rangle < K$, then the expectation would also be strictly less than K, contradicting our assumption.

For us, it means that we prove the existence of a "difficult" sequence of functions through a result on the expectation with respect to a distribution over stochastic functions. Why do we rely on expectations rather than actually constructing an adversarial sequence? Because the use of stochastic loss functions makes it very easy to deal with arbitrary algorithms. In particular, we will choose a distribution over stochastic loss functions that makes the expected loss of the algorithm equal to 0, independently of the strategy of the algorithm.

Theorem 5.1 *Let $\mathcal{V} \subset \mathbb{R}^d$ be any non-empty bounded closed convex subset. Let $D = \max_{\boldsymbol{v},\boldsymbol{w} \in \mathcal{V}} \|\boldsymbol{v} - \boldsymbol{w}\|_2 > 0$ be the diameter of $\mathcal{V}$. Let $\mathscr{A}$ be any deterministic algorithm for Online Linear Optimization (OLO) on $\mathcal{V}$. Let T be any positive integer. Then, there exists a sequence of vectors $\boldsymbol{g}_1, \dots, \boldsymbol{g}_T$ with $\|\boldsymbol{g}_t\|_2 \leq L$ and $\boldsymbol{u} \in \mathcal{V}$ such that the regret of algorithm*

$\mathscr{A}$ satisfies

$$\mathrm{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle \geq \frac{\sqrt{2}LD\sqrt{T}}{4} \, .$$

Proof Let's denote by $\mathrm{Regret}_T := \max_{\boldsymbol{u} \in \mathcal{V}} \mathrm{Regret}_T(\boldsymbol{u})$. Let $\boldsymbol{v}, \boldsymbol{w} \in \mathcal{V}$ such that $\|\boldsymbol{v} - \boldsymbol{w}\|_2 = D$. Let $\boldsymbol{z} = \frac{\boldsymbol{v}-\boldsymbol{w}}{\|\boldsymbol{v}-\boldsymbol{w}\|_2}$, so that $\langle \boldsymbol{z}, \boldsymbol{v} - \boldsymbol{w} \rangle = D$. Let $\epsilon_1, \dots, \epsilon_T$ be i.i.d. Rademacher random variables, that is $\mathbb{P}\{\epsilon_t = 1\} = \mathbb{P}\{\epsilon_t = -1\} = 1/2$, and set the vector of the stochastic linear losses $\tilde{\boldsymbol{g}}_t = L\epsilon_t \boldsymbol{z}$. So, we have

$$\begin{aligned}
&\mathbb{E}_{\tilde{\boldsymbol{g}}_1, \dots, \tilde{\boldsymbol{g}}_T} \left[\sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t \rangle - \min_{\boldsymbol{u} \in \mathcal{V}} \sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{u} \rangle \right] \\
&= \mathbb{E}_{\epsilon_1, \dots, \epsilon_T} \left[\sum_{t=1}^{T} L\epsilon_t \langle \boldsymbol{z}, \boldsymbol{x}_t \rangle - \min_{\boldsymbol{u} \in \mathcal{V}} \sum_{t=1}^{T} L\epsilon_t \langle \boldsymbol{z}, \boldsymbol{u} \rangle \right] \\
&= \mathbb{E}_{\epsilon_1, \dots, \epsilon_T} \left[- \min_{\boldsymbol{u} \in \mathcal{V}} \sum_{t=1}^{T} L\epsilon_t \langle \boldsymbol{z}, \boldsymbol{u} \rangle \right] = \mathbb{E}_{\epsilon_1, \dots, \epsilon_T} \left[\max_{\boldsymbol{u} \in \mathcal{V}} \sum_{t=1}^{T} -L\epsilon_t \langle \boldsymbol{z}, \boldsymbol{u} \rangle \right] \\
&= \mathbb{E}_{\epsilon_1, \dots, \epsilon_T} \left[\max_{\boldsymbol{u} \in \mathcal{V}} \sum_{t=1}^{T} L\epsilon_t \langle \boldsymbol{z}, \boldsymbol{u} \rangle \right] \geq \mathbb{E}_{\epsilon_1, \dots, \epsilon_T} \left[\max_{\boldsymbol{u} \in \{\boldsymbol{v}, \boldsymbol{w}\}} \sum_{t=1}^{T} L\epsilon_t \langle \boldsymbol{z}, \boldsymbol{u} \rangle \right] \\
&= \mathbb{E}_{\epsilon_1, \dots, \epsilon_T} \left[\frac{1}{2} \sum_{t=1}^{T} L\epsilon_t \langle \boldsymbol{z}, \boldsymbol{v} + \boldsymbol{w} \rangle + \frac{1}{2} \left| \sum_{t=1}^{T} L\epsilon_t \langle \boldsymbol{z}, \boldsymbol{v} - \boldsymbol{w} \rangle \right| \right] \\
&= \frac{L}{2} \mathbb{E}_{\epsilon_1, \dots, \epsilon_T} \left[\left| \sum_{t=1}^{T} \epsilon_t \langle \boldsymbol{z}, \boldsymbol{v} - \boldsymbol{w} \rangle \right| \right] = \frac{LD}{2} \mathbb{E}_{\epsilon_1, \dots, \epsilon_T} \left[\left| \sum_{t=1}^{T} \epsilon_t \right| \right] \geq \frac{\sqrt{2}LD\sqrt{T}}{4} \, .
\end{aligned}$$

where in the first equality we used $\mathbb{E}[\epsilon_t] = 0$ and the independence of ϵ_t and $\boldsymbol{x}_t$, the fact that ϵ_t and $-\epsilon_t$ follow the same distribution in the fourth equality, $\max(a, b) = \frac{a+b}{2} + \frac{|a-b|}{2}$ in the fifth equality, and Khintchine inequality (Theorem A.14) in the last inequality.

Now, given that the expectation is lower bounded by a positive constant, there exists a sequence of realizations of the random variables that gives the same lower bound. □

Remark 5.2 *Unlike similar proofs, we do not assume $\mathcal{V}$ to be symmetric with respect to $\mathbf{0}$.*

Remark 5.3 *If the algorithm is randomized, one can easily extend the proof for a lower bound on the expected regret of the algorithm with respect to its internal randomization.*

The lower bound is within a constant multiplicative factor from the upper bound we proved for Online Subgradient Descent (OSD) with learning rates $\eta_t = \frac{D}{L\sqrt{t}}$ or $\eta = \frac{D}{L\sqrt{T}}$. This means that OSD is asymptotically optimal with both settings of the learning rate.

At this point, there is an important consideration: how can this be the optimal regret when we managed to prove a better regret, for example, with adaptive learning rates in Section 4.2? The subtlety is that, constraining the adversary to play L-Lipschitz losses, the adversary could always force on the algorithm at least the regret in Theorem 5.1. However, we can design algorithms that take advantage of *suboptimal plays of the adversary*. Indeed, for example, if the adversary plays in a way that all the subgradients have the same norm equal to L, there is nothing to adapt to!

5.2 Lower Bound for Unconstrained Online Subgradient Descent

Here, we will focus on a specific algorithm, that is, OSD. We want to show that the limitation we saw in Chapter 2 of OSD with time-varying stepsizes to be used only on bounded domains is real. In fact, we can prove the following lower bound.

Theorem 5.4 *Let $\alpha \in (0,1)$, $\phi : (0,1) \to (0, 1-\ln 2)$ defined as $\phi(\alpha) := \frac{1}{2-\alpha} + \frac{(1/2)^{1-\alpha}-1}{1-\alpha}$, and $T \geq \frac{2}{(1-\alpha)\phi(\alpha)}$. For unprojected OSD with stepsizes $\eta_t = t^{-\alpha}$ and $\boldsymbol{x}_1 = \mathbf{0}$, there exists a sequence of T convex and 1-Lipschitz losses such that*

$$\mathrm{Regret}_T(\mathbf{0}) \geq \frac{1}{2}\phi(\alpha)T^{2-\alpha} .$$

Also, we have that $\lim_{\alpha\to 1}\ \phi(\alpha) = 1 - \ln 2 \geq 0.3$.

Proof We assume $d = 1$. For $d \geq 2$, we simply embed the one-dimensional loss vectors into the first coordinate of $\mathbb{R}^d$. Note that the condition on T implies $T \geq 2$. Consider the sequence

$$(\ell_1(x), \dots, \ell_T(x)) = (\underbrace{-x, \dots, -x}_{\lceil T/2\rceil}, \underbrace{x, \dots, x}_{\lfloor T/2\rfloor}) .$$

That is, the first half consists of $-x$'s, the second of $+x$'s. For $t \leq \lceil T/2\rceil$, we have $x_{t+1} = x_t + t^{-\alpha}$. Unrolling the recurrence and using $x_1 = 0$ we get

$$x_t = \sum_{i=1}^{t-1} i^{-\alpha}, \quad t \leq \lceil T/2\rceil + 1 .$$

On the other hand, for $t \geq \lceil T/2\rceil + 1$, we have $x_{t+1} = x_t - t^{-\alpha}$. Unrolling the recurrence up to $x_{\lceil T/2\rceil+1}$ we get

$$x_t = x_{\lceil T/2\rceil+1} \ - \sum_{i=\lceil T/2\rceil+1}^{t-1} i^{-\alpha} = \sum_{i=1}^{\lceil T/2\rceil} i^{-\alpha} \ - \sum_{i=\lceil T/2\rceil+1}^{t-1} i^{-\alpha}, \quad t \geq \lceil T/2\rceil + 1 .$$

We are ready to lower bound the regret:

$$\begin{aligned}
\mathrm{Regret}_T(0) &= -\sum_{t=1}^{\lceil T/2\rceil} x_t + \sum_{t=\lceil T/2\rceil+1}^{T} x_t \\
&= -\sum_{t=1}^{\lceil T/2\rceil}\sum_{i=1}^{t-1} i^{-\alpha} + \sum_{t=\lceil T/2\rceil+1}^{T}\left(\sum_{i=1}^{\lceil T/2\rceil} i^{-\alpha} - \sum_{i=\lceil T/2\rceil+1}^{t-1} i^{-\alpha}\right) \\
&= -\sum_{i=1}^{\lceil T/2\rceil} \frac{\lceil T/2\rceil - i}{i^{\alpha}} + \lfloor T/2\rfloor \sum_{i=1}^{\lceil T/2\rceil} i^{-\alpha} - \sum_{i=\lceil T/2\rceil+1}^{T} \frac{T-i}{i^{\alpha}} \\
&= -\sum_{i=1}^{\lceil T/2\rceil} \frac{\lceil T/2\rceil - \lfloor T/2\rfloor}{i^{\alpha}} + \sum_{i=1}^{T} i^{1-\alpha} - T\sum_{i=\lceil T/2\rceil+1}^{T} i^{-\alpha} .
\end{aligned}$$

Since $\lceil T/2 \rceil - \lfloor T/2 \rfloor \leq 1$ and using the integral bounds, we have

$$\begin{aligned}
\text{Regret}_T(0) &\geq -1 - \int_{i=1}^{\lceil T/2 \rceil} x^{-\alpha} \, \mathrm{d}x + \int_0^T x^{1-\alpha} \, \mathrm{d}x - T \int_{\lceil T/2 \rceil}^T x^{-\alpha} \, \mathrm{d}x \\
&= -1 - \frac{\lceil T/2 \rceil^{1-\alpha} - 1}{1-\alpha} + \frac{T^{2-\alpha}}{2-\alpha} - T \frac{T^{1-\alpha} - \lceil T/2 \rceil^{1-\alpha}}{1-\alpha} \\
&\geq -1 + \frac{1}{1-\alpha} - \frac{\lceil T/2 \rceil^{1-\alpha}}{1-\alpha} + \frac{T^{2-\alpha}}{2-\alpha} - \frac{T^{2-\alpha} - T(T/2)^{1-\alpha}}{1-\alpha} \\
&\geq -\frac{T^{1-\alpha}}{1-\alpha} + \left(\frac{1}{2-\alpha} + \frac{(1/2)^{1-\alpha}}{1-\alpha} - \frac{1}{1-\alpha} \right) T^{2-\alpha} \\
&= -\frac{T^{1-\alpha}}{1-\alpha} + \phi(\alpha) T^{2-\alpha} \geq \frac{1}{2} \phi(\alpha) T^{2-\alpha} \, .
\end{aligned}$$

□

This lower bound tells us that OSD can indeed fail in unbounded domains when used with a polynomially decreasing stepsize. However, it does not rule out the possibility of another algorithm working in the same setting. Indeed, we will see in Chapter 7 that Follow-the-Regularized-Leader achieves sublinear regret on unbounded domains with time-varying regularizers. Yet, its dependence on the other quantities will still be suboptimal, and we will obtain the optimal bound only with parameter-free algorithms in Chapter 13. Indeed, in the next section, we prove that unconstrained OLO is actually more difficult than OLO in bounded domains, *for any algorithm*.

5.3 Lower Bound for Unconstrained Online Linear Optimization

In the unconstrained setting, we proved that OSD with $\boldsymbol{x}_1 = \boldsymbol{0}$ and constant learning rate of $\eta = \frac{1}{L\sqrt{T}}$ gives a regret of $\frac{1}{2} L (\|\boldsymbol{u}\|_2^2 + 1) \sqrt{T}$ for any $\boldsymbol{u} \in \mathbb{R}^d$. Is this regret optimal? It is clear that the regret must be at least linear in $\|\boldsymbol{u}\|_2$. In fact, we could select a specific $\boldsymbol{u}$, pass the information of $\|\boldsymbol{u}\|_2$ to the online learning algorithm and make the problem constrained in $\mathcal{V} = \{\boldsymbol{x} : \|\boldsymbol{x}\|_2 \leq \|\boldsymbol{u}\|_2\}$, so that the lower bound for bounded OLO would hold. However, we now show that the correct dependence in $\|\boldsymbol{u}\|_2$ is more than linear, so the unconstrained setting is strictly more difficult than the bounded one.

The approach I will follow is to *reduce the OLO game to an online game of betting on a coin*, where the lower bounds are easier to prove. So, let's introduce the coin-betting online game:

- Start with an initial amount of money $\epsilon > 0$.
- In each round, the algorithm bets a fraction of its current wealth on the outcome of a coin.
- The outcome of the coin is revealed, and the algorithm wins or loses its bet at even odds.

The aim of this online game is to win as much money as possible. Also, as in all the online games we consider, we do not assume anything about how the outcomes of the coin are decided.

We will denote by $c_t \in \{-1, 1\}, t = 1, \dots, T$ the outcomes of the coin. The absolute value of the signed betting fraction $\beta_t \in [-1, 1]$ will denote the fraction of money to bet, and its sign will denote on which side we are betting. The net gain of the algorithm from the

beginning of the game until the end of round t will be denoted by r_t. Given that the money is won or lost at even odds, we have

$$\overbrace{r_t + \epsilon}^{\text{Money at the end of round } t} = \overbrace{r_{t-1} + \epsilon}^{\text{Money at the beginning of round } t} + \overbrace{c_t \beta_t (r_{t-1} + \epsilon)}^{\text{Money won or lost}} = \epsilon \prod_{i=1}^{t} (1 + \beta_i c_i),$$

where we used the fact that $r_0 = 0$. We will also denote by $x_t = \beta_t(\epsilon + r_{t-1})$ the signed bet of the algorithm on round t.

If we got all the outcomes of the coin correct, we would double our money in each round, so that $\epsilon + r_T = \epsilon 2^T$. On the other hand, if the adversary can always select a coin outcome that is the opposite of our bet, then we would lose money in each round. Hence, we are interested in the best any algorithm can do when the adversary is constrained to give us a sequence of coins where one side appears more often than the other.

To facilitate the use of this theorem later on, we will also slightly generalize the problem assuming that the coins are in $\{-L, L\}$ instead of $\{-1, 1\}$, where $L > 0$, consequently we impose $\beta_t \in [-\frac{1}{L}, \frac{1}{L}]$.

Theorem 5.5 *Let $T \geq 1$ be even and $0 < q \leq \frac{T}{2}$ integer. Then, for any online coin-betting algorithm that guarantees non-negative wealth on any sequence of T coins in $\{-L, L\}$ and starts with initial wealth ϵ, there exists a sequence of coins such that $|\sum_{t=1}^{T} c_t| \geq 2qL$ and the wealth of the algorithm is upper bounded by*

$$\frac{3\epsilon}{2}\left(\frac{2q}{\sqrt{T}} + 1\right) \exp\left(T \cdot \mathrm{KL}_{\textit{Bern}}\left(\frac{1}{2} + \frac{q}{T}; \frac{1}{2}\right)\right) \leq \frac{3\epsilon}{2}\left(\frac{2q}{\sqrt{T}} + 1\right) \exp\left(2\frac{q^2}{T} + 3.1\frac{q^4}{T^3}\right),$$

where $\mathrm{KL}_{\textit{Bern}}(p; q)$ denotes the Kullback–Leibler (KL) divergence between two Bernoulli distributions with parameters p and q:

$$\mathrm{KL}_{\textit{Bern}}(p; q) := p \ln\left(\frac{p}{q}\right) + (1 - p) \ln\left(\frac{1 - p}{1 - q}\right) .$$

Proof Let $Y_1, \ldots, Y_T$ be independent random variables that take the value of 1 with probability 0.5 and -1 with probability 0.5. Hence, we have that $\mathbb{E}[\sum_{t=1}^{T} x_t L Y_t] = 0$, and also $\sum_{t=1}^{T} x_t L Y_t \geq -\epsilon$ for the hypothesis on the betting algorithm. For any $q \geq 0$, it follows that

$$\begin{aligned}
0 &= \mathbb{E}\left[\sum_{t=1}^{T} x_t Y_t\right] \\
&= \mathbb{E}\left[\sum_{t=1}^{T} x_t Y_t \middle| \left|\sum_{t=1}^{T} Y_t\right| < 2q\right] \mathbb{P}\left\{\left|\sum_{t=1}^{T} Y_t\right| < 2q\right\} \\
&\quad + \mathbb{E}\left[\sum_{t=1}^{T} x_t Y_t \middle| \left|\sum_{t=1}^{T} Y_t\right| \geq 2q\right] \mathbb{P}\left\{\left|\sum_{t=1}^{T} Y_t\right| \geq 2q\right\} \\
&\geq -\frac{\epsilon}{L} + \left(\frac{\epsilon}{L} + \mathbb{E}\left[\sum_{t=1}^{T} x_t Y_t \middle| \left|\sum_{t=1}^{T} Y_t\right| \geq 2q\right]\right) \mathbb{P}\left\{\left|\sum_{t=1}^{T} Y_t\right| \geq 2q\right\},
\end{aligned}$$

hence

$$\mathbb{E}\left[\sum_{t=1}^{T} x_t Y_t \middle| \left|\sum_{t=1}^{T} Y_t\right| \geq 2q\right] \leq \frac{\epsilon}{L\,\mathbb{P}\left\{\left|\sum_{t=1}^{T} Y_t\right| \geq 2q\right\}} - \frac{\epsilon}{L} = \frac{\epsilon}{2L\,\mathbb{P}\left\{\sum_{t=1}^{T} Y_t \geq 2q\right\}} - \frac{\epsilon}{L}\,.$$

Using the fact that $\mathbb{P}\left\{\sum_{t=1}^{T} Y_t \geq 2q\right\} = \mathbb{P}\left\{\sum_{t=1}^{T} \frac{Y_t+1}{2} \geq \frac{1}{2}T + q\right\}$, where $\frac{Y_t+1}{2}$ are Bernoulli random variables, we can apply Lemma A.17, to lower bound the tail of the distribution:

$$\mathbb{P}\left\{\sum_{t=1}^{T} Y_t \geq 2q\right\} \geq \frac{1}{3}\frac{1}{\frac{2q}{\sqrt{T}}+1}\exp\left(-T\cdot \mathrm{KL}_{\text{Bern}}\left(\frac{1}{2}+\frac{q}{T};\frac{1}{2}\right)\right)\,.$$

Hence, we have

$$\mathbb{E}\left[\sum_{t=1}^{T} x_t L Y_t \middle| \left|\sum_{t=1}^{T} Y_t\right| \geq 2q\right] \leq \frac{3}{2}\epsilon\left(\frac{2q}{\sqrt{T}}+1\right)\exp\left(T\cdot \mathrm{KL}_{\text{Bern}}\left(\frac{1}{2}+\frac{q}{T};\frac{1}{2}\right)\right) - \epsilon\,.$$

Given that the minimum over a set is smaller than or equal to the expectation with respect to any distribution over the set, there exists a sequence of $c_1,\ldots,c_T \in \{-L,L\}^T$ such that $\frac{1}{L}|\sum_{t=1}^{T} c_t| \geq 2q$ and the wealth of the algorithm is deterministically upper bounded by

$$\frac{3}{2}\epsilon\left(\frac{2q}{\sqrt{T}}+1\right)\exp\left(T\cdot \mathrm{KL}_{\text{Bern}}\left(\frac{1}{2}+\frac{q}{T};\frac{1}{2}\right)\right)\,.$$

For the second upper bound, it is enough to use the elementary inequality

$$\mathrm{KL}_{\text{Bern}}\left(\frac{1}{2}+x;\frac{1}{2}\right) \leq 2x^2 + 3.1x^4, \quad |x| \leq \frac{1}{2}\,. \qquad \square$$

Remark 5.6 *It is also possible to upper bound the l.h.s. of the inequality in the above theorem by a quantity that depends on the wealth of the best constant betting fraction.*

For the expression of the optimal wealth on the $c_1,\ldots,c_T$, consider the wealth of a strategy that bets a constant signed fraction of money β. Starting with initial money ϵ, after T rounds the wealth is $\epsilon\prod_{t=1}^{T}(1+\beta c_t)$. By taking the derivative of the logarithm of the wealth, it is immediate to verify that the $\beta^\star$ that maximizes the above quantity is $\frac{\sum_{t=1}^{T} c_t}{L^2 T}$. Denote by $k = |\{c_t : c_t = L\}|$, hence we have $\beta^\star = \frac{2k-T}{LT}$. Hence, the optimal wealth is

$$\epsilon(1+\beta^\star L)^k(1-\beta^\star L)^{T-k} = \epsilon\exp\left(k\ln\frac{2k}{T} + (T-k)\ln\left(2-\frac{2k}{T}\right)\right) = \epsilon\exp\left(T\cdot \mathrm{KL}_{\textit{Bern}}\left(\frac{k}{T};\frac{1}{2}\right)\right)\,.$$

Equivalently, given that $\frac{1}{L}\sum_{t=1}^{T} c_t = 2k - T$*, we also have that*

$$\begin{aligned}\max_{-1/L\le\beta\le 1/L}\prod_{t=1}^{T}(1+\beta c_t) &= \exp\left(T\cdot \mathrm{KL}_{\mathit{Bern}}\left(\frac{\sum_{t=1}^{T} c_t}{2LT}+\frac{1}{2};\frac{1}{2}\right)\right)\\ &= \exp\left(T\cdot \mathrm{KL}_{\mathit{Bern}}\left(\frac{|\sum_{t=1}^{T} c_t|}{2LT}+\frac{1}{2};\frac{1}{2}\right)\right),\end{aligned}$$

where in the second equality we used the fact that $\mathrm{KL}_{\mathit{Bern}}(x+\frac{1}{2};\frac{1}{2}) = \mathrm{KL}_{\mathit{Bern}}(-x+\frac{1}{2};\frac{1}{2})$*. Hence, we have*

$$\begin{aligned}&\frac{3}{2}\epsilon\left(\frac{2q}{\sqrt{T}}+1\right)\exp\left(T\cdot \mathrm{KL}_{\mathit{Bern}}\left(\frac{1}{2}+\frac{q}{T};\frac{1}{2}\right)\right)\\ &\quad\le \frac{3}{2}\epsilon\left(\frac{|\sum_{t=1}^{T} c_t|}{L\sqrt{T}}+1\right)\exp\left(T\cdot \mathrm{KL}_{\mathit{Bern}}\left(\frac{1}{2}+\frac{|\sum_{t=1}^{T} c_t|}{2LT};\frac{1}{2}\right)\right)\\ &\quad= \frac{3}{2}\left(\frac{|\sum_{t=1}^{T} c_t|}{L\sqrt{T}}+1\right)\max_{\beta}\epsilon\prod_{t=1}^{T}(1+c_t\beta)\,.\end{aligned}$$

Now, let's connect the coin-betting game with OLO, thanks to the next theorem.

Theorem 5.7 *Let* ϵ_t *be a non-negative non-decreasing sequence and* $\mathscr{A}$ *an OLO algorithm that guarantees* $\mathrm{Regret}_t(\mathbf{0}) \le \epsilon_t$ *for any sequence of* $\boldsymbol{g}_1,\dots,\boldsymbol{g}_t \in \mathbb{R}^d$ *with* $\|\boldsymbol{g}_i\|_2 \le L$*. Then, for any* $T \ge 0$ *there exists* $\boldsymbol{\beta}_t$ *such that* $\boldsymbol{x}_t = \boldsymbol{\beta}_t(\epsilon_T - \sum_{i=1}^{t-1}\langle \boldsymbol{g}_i, \boldsymbol{x}_i\rangle)$ *and* $\|\boldsymbol{\beta}_t\|_2 \le \frac{1}{L}$ *for* $t = 1,\dots,T$*.*

Proof Define $r_t = -\sum_{i=1}^{t}\langle \boldsymbol{g}_i, \boldsymbol{x}_i\rangle$ as the "reward" of the algorithm. So, we have

$$\mathrm{Regret}_t(\boldsymbol{u}) = \sum_{i=1}^{t}\langle \boldsymbol{g}_i, \boldsymbol{x}_i\rangle - \sum_{i=1}^{t}\langle \boldsymbol{g}_i, \boldsymbol{u}\rangle = -r_t + \left\langle \sum_{i=1}^{t}\boldsymbol{g}_i, \boldsymbol{u}\right\rangle\,.$$

Since we assumed that $\mathrm{Regret}_t(\mathbf{0}) \le \epsilon_t$, we always have $r_t \ge -\epsilon_t$. Using this, we claim that $L\|\boldsymbol{x}_t\|_2 \le r_{t-1} + \epsilon_t$ for all $t = 1,\dots,T$. To see this, assume that there is a sequence $\boldsymbol{g}_1,\dots,\boldsymbol{g}_{t-1}$ that gives $L\|\boldsymbol{x}_t\|_2 > r_{t-1} + \epsilon_t$. If $\|\boldsymbol{x}_t\|_2 = 0$ then the stated inequality holds. Instead, for $\|\boldsymbol{x}_t\|_2 \ne 0$, set $\boldsymbol{g}_t = L\frac{\boldsymbol{x}_t}{\|\boldsymbol{x}_t\|_2}$. For this sequence, we would have $r_t = r_{t-1} - L\|\boldsymbol{x}_t\|_2 < -\epsilon_t$, that contradicts the observation that $r_t \ge -\epsilon_t$.

So, from the fact that $L\|\boldsymbol{x}_t\|_2 \le r_{t-1} + \epsilon_t \le r_{t-1} + \epsilon_T$ we have that there exists $\boldsymbol{\beta}_t$ such that $\boldsymbol{x}_t = \boldsymbol{\beta}_t(\epsilon_T + r_{t-1})$ and $\|\boldsymbol{\beta}_t\|_2 \le \frac{1}{L}$. □

This theorem informs us of something important: *any OLO algorithm that suffers at most a non-decreasing regret in* t *against the competitor* $\boldsymbol{u} = \mathbf{0}$ *predicts in the form of a "vectorial" coin-betting algorithm.*

From this connection to online betting, we now derive a lower bound for OLO.

Theorem 5.8 *Let* $T \in \mathbb{N}$ *be even and let* $\mathscr{A}$ *be any OLO algorithm that guarantees regret at most* $\epsilon_T > 0$ *against the null competitor on any sequence of* T *linear and* L*-Lipschitz losses* $\ell_t : \mathbb{R} \to \mathbb{R}$ *for* $t = 1,\dots,T$*. Let* $U > 0$ *be such that* $1 \le W(\frac{\sqrt{eT}UL}{8\epsilon_T}) \le \frac{\sqrt{T}}{2}$*. Then,*

for this algorithm, there exists a sequence of g_t with $|g_t| \le L$ and a competitor $u \in \mathbb{R}$ with $|u| = U$, such that

$$\sum_{t=1}^{T} g_t(x_t - u) \ge R_T(U) := UL\sqrt{T}\left(\sqrt{2W\left(\frac{\sqrt{T}UL}{5\epsilon_T}\right) - 1}\right) - 2UL + \epsilon_T,$$

where $W : \mathbb{R}_{\ge 0} \to \mathbb{R}_{\ge 0}$ is the Lambert function (see Appendix A.2.1).

Moreover, assuming there exist $0 < K_1 \le K_2 < \infty$ such that $K_1 \le \epsilon_T \le K_2$ for all T, we have $\lim_{T\to+\infty} \frac{R_T(U)}{UL\sqrt{T \ln T}} = 1$.

Proof Consider the linear losses $\ell_t(x) = g_t x$, where $|g_t| \le L$.

Given that the algorithm guarantees a regret of at most ϵ_T against the null competitor, from Theorem 5.7 we have that, if we feed the algorithm with linear losses, we can reinterpret the algorithm as a betting algorithm with initial wealth ϵ_T and that guarantees non-negative wealth, where the wealth at time T is defined as $\epsilon_T - \sum_{t=1}^{T} g_t x_t$.

Set $A = W\left(\frac{\sqrt{eT}UL}{8\epsilon_T}\right)$, $\tilde{q} = \frac{\sqrt{T}}{2}\sqrt{2A-1}$, and $q = \lfloor \tilde{q} \rfloor$, so that $\tilde{q} \ge q \ge \tilde{q} - 1$.

Observe that the constraint on U ensures that

$$q \le \tilde{q} \le \frac{T^{3/4}}{2} \le \frac{T}{2}, \tag{5.1}$$

so we can safely use Theorem 5.5, that tells us that there exists a sequence of $c_1, \dots, c_T \in \{-L, L\}$ such that $|\sum_{t=1}^{T} c_t| \ge 2qL$ and

$$\sum_{t=1}^{T} c_t x_t \le \frac{3}{2}\left(\frac{2q}{\sqrt{T}} + 1\right)\epsilon_T \exp\left(2\frac{q^2}{T}\right)\exp\left(3.1\frac{q^4}{T^3}\right) - \epsilon_T\,.$$

Moreover, observe that

$$\begin{aligned}\exp\left(2\frac{q^2}{T}\right)\exp\left(3.1\frac{q^4}{T^3}\right) &\le \exp\left(2\frac{\tilde{q}^2}{T}\right)\exp\left(3.1\frac{\tilde{q}^4}{T^3}\right) \le \exp\left(2\frac{\tilde{q}^2}{T}\right)\exp\left(\frac{3.1}{16}\right)\\ &\le \frac{4}{3}\exp\left(2\frac{\tilde{q}^2}{T}\right),\end{aligned}$$

where in second inequality we used $\frac{\tilde{q}^4}{T^3} \le \frac{1}{16}$ from (5.1).

Set $g_t = -c_t$ and choose $u = -U\mathrm{sign}(\sum_{t=1}^{T} g_t)$, so we have

$$\begin{aligned}\sum_{t=1}^{T} g_t(x_t - u) &= U\left|\sum_{t=1}^{T} g_t\right| + \sum_{t=1}^{T} g_t x_t\\ &\ge U \cdot 2qL - 2\left(\frac{2\tilde{q}}{\sqrt{T}} + 1\right)\epsilon_T \exp\left(2\frac{\tilde{q}^2}{T}\right) + \epsilon_T\\ &\ge UL \cdot (2\tilde{q} - 2) - 2\left(\frac{2\tilde{q}}{\sqrt{T}} + 1\right)\epsilon_T \exp\left(2\frac{\tilde{q}^2}{T}\right) + \epsilon_T\,.\end{aligned}$$

Now, from the condition on U, we have $\frac{2\tilde{q}}{\sqrt{T}} \geq 1$, hence we have

$$\sum_{t=1}^{T} g_t(x_t - u) = U \left| \sum_{t=1}^{T} g_t \right| + \sum_{t=1}^{T} g_t x_t$$
$$\geq 4\epsilon_T \left[\frac{\sqrt{T}UL}{4\epsilon_T} \cdot \frac{2\tilde{q}}{\sqrt{T}} - \frac{2\tilde{q}}{\sqrt{T}} \exp\left(2\frac{\tilde{q}^2}{T}\right) \right] - 2UL + \epsilon_T \,. \tag{5.2}$$

Let $a \geq 1$, then

$$x^\star := \operatorname*{argmax}_{x \geq 0} \; ax - x\exp(x^2/2) = \sqrt{2W\left(\frac{a\sqrt{e}}{2}\right) - 1} \,.$$

So, we have

$$ax^\star - x^\star \exp((x^\star)^2/2) = a\sqrt{2W\left(\frac{a\sqrt{e}}{2}\right) - 1}\left(1 - \frac{1}{2W\left(\frac{a\sqrt{e}}{2}\right)}\right) \,.$$

Hence, our choice of $\tilde{q}$ maximizes the lower bound in (5.2). Hence, we obtain

$$\sum_{t=1}^{T} g_t(x_t - u) \geq UL\sqrt{T}\sqrt{2W\left(\frac{\sqrt{e\,T}UL}{8\epsilon_T}\right) - 1}\left(1 - \frac{1}{2W\left(\frac{\sqrt{e\,T}UL}{8\epsilon_T}\right)}\right) - 2UL + \epsilon_T \,.$$

The condition on U ensures that $W(\frac{\sqrt{eT}UL}{8\epsilon_T}) \geq 1$. So, using the elementary inequality $(1 - 1/(2x))\sqrt{2x-1} \geq \sqrt{2x} - 1$ for $x \geq 1$ and simplifying the numerical constants, we get the stated bound.

The limit statement is obtained using Theorem A.4. □

Remark 5.9 *The leading constant $\sqrt{2}$ is asymptotically optimal because there exist algorithms with a matching upper bound, see Section 5.4.*

This theorem implies that OSD with learning rate $\eta = \frac{\alpha}{L\sqrt{T}}$ does not have the optimal dependence on $\|\boldsymbol{u}\|_2$ for any $\alpha > 0$.

In Chapter 13, we will see that the connection between coin betting and OLO can also be used to design an OLO algorithm. This will give us *optimal unconstrained OLO algorithms with the surprising property of not requiring a learning rate at all.*

5.4 History Bits

The lower bound for bounded OLO is quite standard, and the proof presented is a simplified version of the one in Orabona and Pál (2015, 2018). One could also use the function in the lower bound for offline optimization in Nesterov (2004, Section 3.2.1), but it would require the additional assumption that $d > T$ and that $\boldsymbol{x}_t$ lies in the span of the previous subgradients. This limitation on d is unavoidable: without it, offline convex optimization becomes easier while Online Convex Optimization (OCO) is equally hard for any number of dimensions.

The lower bound for OSD in Theorem 5.4 is a minor generalization of the one in Orabona

and Pál (2015, 2018). They also provide a similar lower bound for the Exponentiated Gradient (EG) algorithm.

Strangely enough, both the online learning and the optimization literature have almost ignored the issue of lower bounds for the unconstrained case. The connection between coin betting and OLO was first unveiled in Orabona and Pál (2016). Theorem 5.7 is a mild generalization of McMahan and Abernethy (2013, Theorem 9). A similar, but more general, result was also rediscovered in Cutkosky (2018). The first lower bound for unconstrained OLO is from Streeter and McMahan (2012), but their proof relied on using the value of $\inf_T \mathbb{P}\{\sum_{i=1}^T X_i \geq \sqrt{T}\}$ where X_i are Rademacher random variables. Streeter and McMahan (2012) claim that this value is $7/64$ that corresponds to $T = 6$, but they do not provide a proof for it. The same value was also conjectured by Hitczenko and Kwapień (1994) and proved formally only in 2023 by Hollom and Portier (2023). A proof avoiding that step completely was given by Orabona (2013). Theorem 5.8 is new: it is a more precise version of the lower bound in Orabona (2013) and it has the asymptotically optimal constant $\sqrt{2}$. One way to achieve the optimal constant in lower bounds using the tail of Binomial distributions was shown in Orabona and Pál (2015). Independently, Zhang et al. (2022) proved a lower bound for unconstrained OCO with the optimal constant but only when $\epsilon_T = \mathcal{O}(\sqrt{T})$, with an algorithm with a matching upper bound. The specific method used here to obtain the optimal constant is new. McMahan and Orabona (2014) proposed an algorithm that matches the lower bound up to a multiplicative constant, while Zhang et al. (2022) matched the multiplicative constant too. More recently, Carmon and Hinder (2024) proved a lower bound of $\Omega(\|\boldsymbol{x}^\star - \boldsymbol{x}_1\| L \sqrt{\frac{1}{T} \ln \frac{L\|\boldsymbol{x}^\star - \boldsymbol{x}_1\|}{\epsilon}})$ in the stochastic setting on the expected suboptimality gap. Their lower bound implies the same lower bound we stated on the regret of OCO through the online-to-batch conversion. However, their lower bound requires the use of convex nonlinear Lipschitz functions, while the result here just requires linear losses. Their results also imply that the parameter-free algorithm in McMahan and Orabona (2014) coupled with the online-to-batch conversion is optimal for the setting of stochastic unconstrained optimization of Lipschitz convex functions.

5.5 Exercises

Problem 5.1 *Fix $U > 0$ and $\mathcal{V} = \mathbb{R}^d$. Mimicking the proof of Theorem 5.1, prove that for any OCO algorithm there exists a $\boldsymbol{u}^\star$ and a sequence of loss functions such that $\mathrm{Regret}_T(\boldsymbol{u}^\star) \geq \frac{1}{2}\|\boldsymbol{u}^\star\|_2 L\sqrt{T}$ where $\|\boldsymbol{u}^\star\|_2 = U$ and the loss functions are L-Lipschitz with respect to $\|\cdot\|_2$.*

Problem 5.2 *Extend the proof of Theorem 5.1 to an arbitrary norm $\|\cdot\|$ to measure the diameter of $\mathcal{V}$ and with $\|\boldsymbol{g}_t\|_\star \leq L$.*

6

Online Mirror Descent

In this chapter, we will introduce the Online Mirror Descent (OMD) algorithm. To explain its genesis, I think it is essential to understand what subgradients do. In particular, the negative subgradients are not always pointing towards a direction that minimizes the function. Then, we will introduce the Bregman divergences to generalize the notion of distance implicitly used in online subgradient descent. Finally, we will see extensions and applications of OMD.

6.1 Subgradients are not Informative

We have seen that in online learning, we receive a sequence of loss functions, and we have to output a vector before observing the loss function on which we will be evaluated. However, we can gain a lot of intuition if we consider the easy case that the sequence of loss functions is always a fixed function, i.e., $\ell_t(\boldsymbol{x}) = \ell(\boldsymbol{x})$. If our hypothetical online algorithm does not work in this situation, then it will not work on the more general case.

Hence, considering the case of fixed loss functions, let's take a look at the key step in the proof of the upper bound to the regret for Online Subgradient Descent (OSD) in Lemma 2.31. We used the following property of the subgradients:

$$\ell(\boldsymbol{x}_t) - \ell(\boldsymbol{u}) \leq \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle, \quad \forall \boldsymbol{u} \in \mathcal{V} \,. \tag{6.1}$$

In words, to minimize the l.h.s. of this equation, it is enough to minimize the r.h.s., that is nothing else than the instantaneous linear regret on the linear function $\langle \boldsymbol{g}_t, \cdot \rangle$. This is the only reason why OSD works! However, I am sure you have heard a million times the (wrong) intuition that gradient points towards the minimum, and you might be tempted to think that the same (even more wrong) intuition holds for subgradients. Indeed, I am sure that even if we proved the regret guarantee based on (6.1), in the back of your mind, you keep thinking "yeah, sure, it works because the subgradient tells me where to go to minimize the function". Typically, this idea is so strong that I have to present explicit counterexamples to fully convince a person.

So, take a look at the following examples that illustrate the fact that a subgradient does not always point in a direction where the function decreases.

Example 6.1 *Let $f(\boldsymbol{x}) = \max[-x_1, x_1 - x_2, x_1 + x_2]$, see Figure 6.1. The vector $\boldsymbol{g} = (1, 1)$ is a subgradient in $\boldsymbol{x} = (1, 0)$ of $f(\boldsymbol{x})$. No matter how we choose the stepsize, moving in the negative direction of this subgradient will not decrease the objective function. An even more extreme example is in Figure 6.2, with the function $f(\boldsymbol{x}) = \max[x_1^2 + (x_2 + 1)^2, x_1^2 +$*

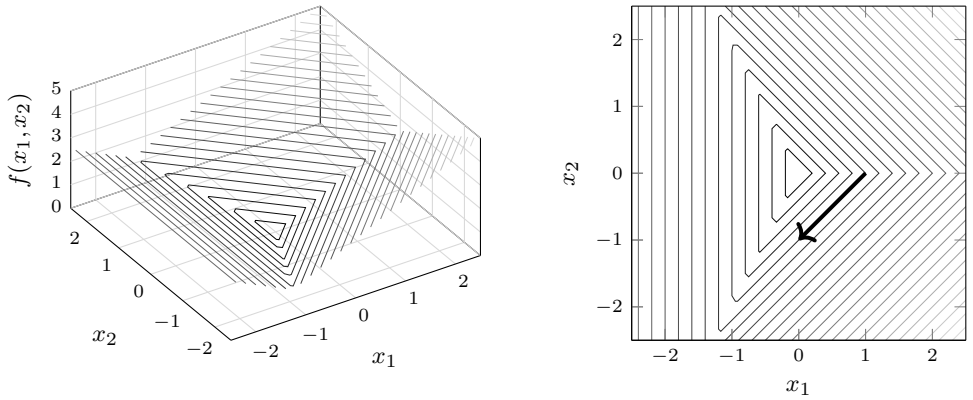


Figure 6.1 3D plot (left) and level sets (right) of $f(\boldsymbol{x}) = \max[-x_1, x_1 - x_2, x_1 + x_2]$. A negative subgradient is indicated by the black arrow.

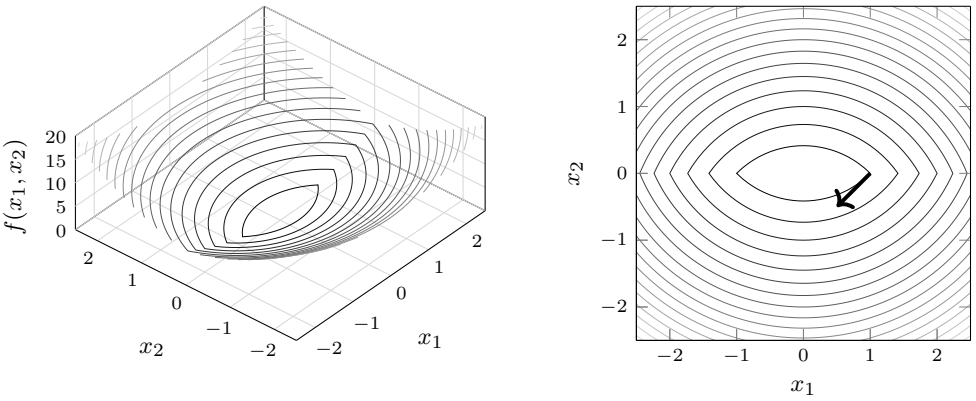


Figure 6.2 3D plot (left) and level sets (right) of $f(\boldsymbol{x}) = \max[x_1^2 + (x_2 + 1)^2, x_1^2 + (x_2 - 1)^2]$. A negative subgradient is indicated by the black arrow.

$(x_2 - 1)^2]$. *Here, in the point* $(1, 0)$*, any positive step in the direction of the displayed negative subgradient will* increase *the objective function.*

In both examples, one might think that it would be enough to add a little bit of noise to move away from the "corners". However, we have to remember that we are in the adversarial setting. So, assuming the adversary sees our randomization, it can always present us a function such that our prediction is exactly on a "corner". This means that our analysis of subgradient descent will not have to use the fact that the subgradients point towards descending directions, because it can be false in every single iteration.

Remark 6.2 *Given the above, one might wonder how much "information" the subgradients carry. It turns out, quite a lot! In fact, for bounded domains in the offline case, i.e., when all the functions are the same, the cutting plane method can optimize the function exponen-*

tially fast just using subgradients (see, e.g., Chapter 3 in Nemirovski, 1995). However, in the adversarial setting, subgradients become much weaker than in the offline setting, exactly because the adversary has the power to change the function in each round.

6.2 Reinterpreting the Online Subgradient Descent Algorithm

How does OSD work? It works exactly as I told you before: thanks to (6.1). But what does that inequality really mean?

A way to understand how the OSD algorithm works is to think that it minimizes a local approximation of the original objective function. This is not unusual for optimization algorithms. For example, the Newton algorithm constructs an approximation with a Taylor expansion truncated to the second term. Thanks to the definition of subgradients, we can immediately build a linear lower bound to a function f around $\boldsymbol{x}_0$:

$$f(\boldsymbol{x}) \geq \tilde{f}(\boldsymbol{x}) := f(\boldsymbol{x}_0) + \langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{x}_0 \rangle, \quad \forall \boldsymbol{x} \in \mathcal{V} \, .$$

So, in our setting, this would mean that we update the online algorithm with the minimizer of a linear approximation of the loss function you received. Unfortunately, minimizing a linear function is unlikely to give us a good online algorithm. Indeed, over unbounded domains, the infimum of a linear function is $-\infty$.

So, let's introduce the other key concept: we constrain the minimization of this lower bound only in a neighborhood of $\boldsymbol{x}_0$, where we have good reason to believe that the approximation is more precise. Moreover, in online learning, it makes sense not to go too far from the previous iteration because the losses are different in each step, and we do not want to give too much importance to the current loss. Coding the neighborhood constraint with an L_2 squared distance from $\boldsymbol{x}_0$ less than some positive number h, we might think to use the following update

$$\begin{aligned} \boldsymbol{x}_{t+1} = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \; & f(\boldsymbol{x}_t) + \langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{x}_t \rangle \\ & \text{s.t. } \|\boldsymbol{x}_t - \boldsymbol{x}\|^2 \leq h \, . \end{aligned}$$

Equivalently, for some $\eta > 0$, we can consider the unconstrained formulation

$$\operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \left(\hat{f}(\boldsymbol{x}) := f(\boldsymbol{x}_0) + \langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{x}_0 \rangle + \frac{1}{2\eta} \|\boldsymbol{x}_0 - \boldsymbol{x}\|_2^2 \right) \, . \tag{6.2}$$

This is a well-defined update scheme that hopefully moves $\boldsymbol{x}_t$ closer to the optimum of f. See Figure 6.3 for a graphical representation in one dimension.

And now the final element of our story: the argmin in (6.2) is exactly the update we used in OSD! Indeed, solving the argmin and completing the square, we get

$$\begin{aligned} & \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle + \frac{1}{2\eta_t} \|\boldsymbol{x}_t - \boldsymbol{x}\|_2^2 \\ & \quad = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \; \|\eta_t \boldsymbol{g}_t\|^2 + 2\eta_t \langle \boldsymbol{g}_t, \boldsymbol{x} - \boldsymbol{x}_t \rangle + \|\boldsymbol{x}_t - \boldsymbol{x}\|_2^2 \\ & \quad = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \; \|\boldsymbol{x} - \boldsymbol{x}_t + \eta_t \boldsymbol{g}_t\|_2^2 = \Pi_{\mathcal{V}}(\boldsymbol{x}_t - \eta_t \boldsymbol{g}_t), \end{aligned} \tag{6.3}$$

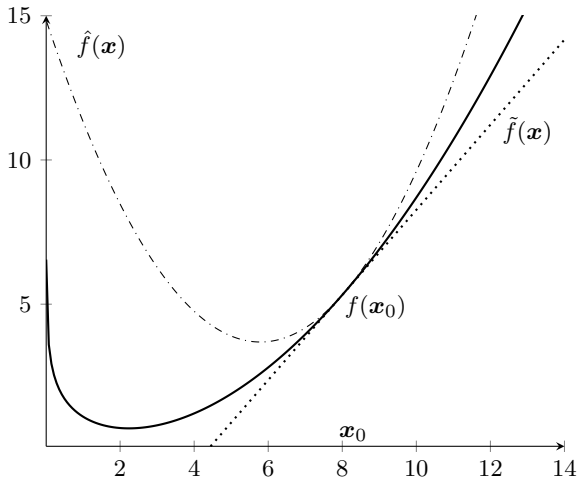


Figure 6.3 Approximations of $f(\boldsymbol{x})$.

where we used the fact that the argmin is independent of additive constants and positive rescalings, and $\Pi_{\mathcal{V}}$ is the Euclidean projection onto $\mathcal{V}$, i.e., $\Pi_{\mathcal{V}}(\boldsymbol{x}) = \operatorname{argmin}_{\boldsymbol{y}\in\mathcal{V}} \|\boldsymbol{x}-\boldsymbol{y}\|_2$.

The new way to write the update of OSD in (6.2) will be the core ingredient for designing *Online Mirror Descent (OMD)*. In fact, OMD is a strict generalization of that update when we use a different way to measure the locality of $\boldsymbol{x}$ from $\boldsymbol{x}_t$. That is, we measured the distance to the current point with the squared L_2 norm. What happens if we change the norm? Do we even have to use a norm?

To answer these questions, we have to introduce another useful mathematical object: the *Bregman divergence*.

6.3 Convex Analysis Bits: Bregman Divergence

We first give a new definition, a slightly stronger notion of convexity.

Definition 6.3 *Let $f : \mathcal{V} \subseteq \mathbb{R}^d \to \mathbb{R}$ and $\mathcal{V}$ be a convex set. f is **strictly convex** if*

$$f(\alpha\boldsymbol{x} + (1-\alpha)\boldsymbol{y}) < \alpha f(\boldsymbol{x}) + (1-\alpha) f(\boldsymbol{y}), \quad \forall \boldsymbol{x}, \boldsymbol{y} \in \mathcal{V}, \boldsymbol{x} \neq \boldsymbol{y}, 0 < \alpha < 1 \,.$$

From the definition, it is immediate to see that strong convexity with respect to any norm implies strict convexity. Note that for a differentiable function, strict convexity also implies that $f(\boldsymbol{y}) > f(\boldsymbol{x}) + \langle \nabla f(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x} \rangle$ for $\boldsymbol{x} \neq \boldsymbol{y}$ (Bauschke and Combettes, 2017, Proposition 17.10).

We now define our new notion of "distance".

Definition 6.4 *Let $\psi : \mathcal{X} \to \mathbb{R}$ be strictly convex and differentiable on $\operatorname{int} \mathcal{X} \neq \emptyset$, where $\mathcal{X}$ is the domain of ψ. The **Bregman divergence** with respect to ψ is denoted by $B_\psi : \mathcal{X} \times \operatorname{int} \mathcal{X} \to \mathbb{R}$ defined as*

$$B_\psi(\boldsymbol{x}; \boldsymbol{y}) := \psi(\boldsymbol{x}) - \psi(\boldsymbol{y}) - \langle \nabla\psi(\boldsymbol{y}), \boldsymbol{x} - \boldsymbol{y} \rangle \,.$$

We will call the function ψ associated to B_ψ the **distance generating function**.

From the definition, we see that the Bregman divergence is always non-negative for $\boldsymbol{x}, \boldsymbol{y} \in \operatorname{int} \mathcal{X}$, from the convexity of ψ. However, something stronger holds. By the strict convexity of ψ, for a fixed point $\boldsymbol{y} \in \operatorname{int} \mathcal{X}$ we have that $\psi(\boldsymbol{x}) \geq \psi(\boldsymbol{y}) + \langle \nabla\psi(\boldsymbol{y}), \boldsymbol{x} - \boldsymbol{y}\rangle$, $\forall \boldsymbol{x} \in \mathcal{X}$, with equality only for $\boldsymbol{y} = \boldsymbol{x}$. Hence, the strict convexity allows us to use the Bregman divergence as a similarity measure between $\boldsymbol{x}$ and $\boldsymbol{y}$. Moreover, this similarity measure *changes* with the reference point $\boldsymbol{y}$. This also implies that, as you can see from the definition, the Bregman divergence is not symmetric.

Let me give you some more intuition on the concept of the Bregman divergence. Consider the case that ψ is twice differentiable in an open ball $\mathcal{B}$ around $\boldsymbol{y}$ and $\boldsymbol{x} \in \mathcal{B}$. So, by Taylor's theorem, there exists $0 \leq \alpha \leq 1$ such that

$$B_\psi(\boldsymbol{x}; \boldsymbol{y}) = \psi(\boldsymbol{x}) - \psi(\boldsymbol{y}) - \nabla\psi(\boldsymbol{y})^\top(\boldsymbol{x} - \boldsymbol{y}) = \frac{1}{2}(\boldsymbol{x} - \boldsymbol{y})^\top \nabla^2\psi(\boldsymbol{z})(\boldsymbol{x} - \boldsymbol{y}),$$

where $\boldsymbol{z} = \alpha\boldsymbol{x} + (1 - \alpha)\boldsymbol{y}$. Hence, we are using a *squared local norm* that depends on the Hessian of ψ. *Different areas of the space will have a different value of the Hessian, and so the Bregman will behave differently*. We will use this exact idea in the local norm analyses of OMD (Section 6.5) and Follow-the-Regularized-Leader (FTRL) (Section 7.4).

We can also lower bound the Bregman divergence if the function ψ is strongly convex. In particular, if ψ is λ-strongly convex with respect to a norm $\|\cdot\|$ in $\operatorname{int} \mathcal{X}$, then we have

$$B_\psi(\boldsymbol{x}; \boldsymbol{y}) \geq \frac{\lambda}{2}\|\boldsymbol{x} - \boldsymbol{y}\|^2, \quad \forall \boldsymbol{x}, \boldsymbol{y} \in \operatorname{int} \mathcal{X} \,. \tag{6.4}$$

Example 6.5 *If $\psi(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{x}\|_2^2$, then $B_\psi(\boldsymbol{x}; \boldsymbol{y}) = \frac{1}{2}\|\boldsymbol{x}\|_2^2 - \frac{1}{2}\|\boldsymbol{y}\|_2^2 - \langle \boldsymbol{y}, \boldsymbol{x} - \boldsymbol{y}\rangle = \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{y}\|_2^2$.*

Example 6.6 *Let $\mathcal{X} = \mathbb{R}^d_{\geq 0}$ and $\psi(\boldsymbol{x}) = \sum_{i=1}^d x_i \ln x_i$, the negative Shannon entropy. Then, for all $\boldsymbol{x} \in \mathcal{X}$ and $\boldsymbol{y} \in \operatorname{int} \mathcal{X}$ we have*

$$B_\psi(\boldsymbol{x}; \boldsymbol{y}) = \sum_{i=1}^d (x_i \ln x_i - y_i \ln y_i - (\ln(y_i) + 1)(x_i - y_i)) = \sum_{i=1}^d \left(x_i \ln \frac{x_i}{y_i} - x_i + y_i \right),$$

*where we define $0 \ln 0 := 0$. This is called the **generalized Kullback–Leibler divergence**, where "generalized" is because $\boldsymbol{x}$ and $\boldsymbol{y}$ do not have to be discrete probability distributions.*

We also have the following immediate lemma that links the Bregman divergences between 3 points.

Lemma 6.7 (Chen and Teboulle, 1993) *Let B_ψ be the Bregman divergence with respect to $\psi : \mathcal{X} \to \mathbb{R}$. Then, for any three points $\boldsymbol{x}, \boldsymbol{y} \in \operatorname{int} \mathcal{X}$ and $\boldsymbol{z} \in \mathcal{X}$, the following identity holds*

$$B_\psi(\boldsymbol{z}; \boldsymbol{x}) + B_\psi(\boldsymbol{x}; \boldsymbol{y}) - B_\psi(\boldsymbol{z}; \boldsymbol{y}) = \langle \nabla\psi(\boldsymbol{y}) - \nabla\psi(\boldsymbol{x}), \boldsymbol{z} - \boldsymbol{x}\rangle \,.$$

Algorithm 6.1 Online Mirror Descent (OMD)

Require: Non-empty closed convex $\mathcal{V} \subseteq \mathcal{X} \subseteq \mathbb{R}^d$, $\psi : \mathcal{X} \to \mathbb{R}$ strictly convex and differentiable on $\operatorname{int} \mathcal{X}$, $\boldsymbol{x}_1 \in \operatorname{int} \mathcal{X} \cap \mathcal{V}$, $\eta_1, \dots, \eta_T > 0$

1: **for** $t = 1$ **to** T **do**
2: Output $\boldsymbol{x}_t \in \mathcal{V}$
3: Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable on $\mathcal{V}$
4: Set $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$
5: Set $\boldsymbol{x}_{t+1} \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle + \frac{1}{\eta_t} B_\psi(\boldsymbol{x}; \boldsymbol{x}_t)$
6: **end for**

6.4 Online Mirror Descent

Based on what we said before, we can start from the equivalent formulation of the OSD update,

$$\boldsymbol{x}_{t+1} = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle + \frac{1}{2\eta_t} \|\boldsymbol{x}_t - \boldsymbol{x}\|_2^2,$$

and we can change the last term with another measure of distance. In particular, using the Bregman divergence with respect to a function ψ, we have

$$\boldsymbol{x}_{t+1} = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle + \frac{1}{\eta_t} B_\psi(\boldsymbol{x}; \boldsymbol{x}_t),$$

where we assumed that the argmin exists and it is unique. These two updates are exactly the same when $\psi(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{x}\|_2^2$.

So, we get **Online Mirror Descent** in Algorithm 6.1.

However, without an additional assumption, this algorithm has a problem. Can you see it? The problem is that $\boldsymbol{x}_{t+1}$ might be on the boundary of $\mathcal{V}$, and in the next step, we would have to evaluate $B_\psi(\boldsymbol{x}; \boldsymbol{x}_{t+1})$ for a point on the boundary of $\mathcal{V}$. Given that $\mathcal{V} \subseteq \mathcal{X}$, we might end up on the boundary of $\mathcal{X}$ where the Bregman divergence is not defined!

To fix this problem, different sufficient conditions can be used. In the following, we will use one of the following assumptions:

$$\lim_{\lambda \to 0} \langle \nabla \psi((1-\lambda)\boldsymbol{x} + \lambda \boldsymbol{y}), \boldsymbol{y} - \boldsymbol{x} \rangle = -\infty, \quad \forall \boldsymbol{x} \in \operatorname{bdry} \mathcal{X}, \forall \boldsymbol{y} \in \operatorname{int} \mathcal{X} \tag{6.5}$$

$$\mathcal{V} \subseteq \operatorname{int} \mathcal{X} \,. \tag{6.6}$$

If either of these conditions is true and the argmin exists on each round, then the algorithm is well-defined as proved in the following theorem.

Theorem 6.8 *Let B_ψ be the Bregman divergence with respect to $\psi : \mathcal{X} \to \mathbb{R}$. Let $\mathcal{V} \subseteq \mathcal{X}$ be a non-empty closed convex set. Assume* (6.5) *or* (6.6) *hold and, with the notation and initialization in Algorithm 6.1, the argmin exists on all rounds. Then, $\boldsymbol{x}_{t+1} \in \operatorname{int} \mathcal{X}$.*

Proof In the case that (6.6) holds, we have that $\boldsymbol{x}_{t+1} \in \mathcal{V}$ implies immediately that $\boldsymbol{x}_{t+1} \in \operatorname{int} \mathcal{X}$.

Let's now assume that (6.5) holds and let's prove it by induction. The base case is true by the definition of $\boldsymbol{x}_1$. Let's now assume that $\boldsymbol{x}_t \in \operatorname{int} \mathcal{X}$ and let's prove that $\boldsymbol{x}_{t+1} \in \operatorname{int} \mathcal{X}$.

We will prove it by contradiction. So, assume that $\boldsymbol{x}_{t+1} \in \operatorname{bdry} \mathcal{X}$. Set $\boldsymbol{z} \in \operatorname{int} \mathcal{X} \cap \mathcal{V}$ and define $\phi(\lambda) = \langle \eta_t \boldsymbol{g}_t, (1-\lambda)\boldsymbol{x}_{t+1} + \lambda \boldsymbol{z} \rangle + B_\psi((1-\lambda)\boldsymbol{x}_{t+1} + \lambda \boldsymbol{z}; \boldsymbol{x}_t)$ for $\lambda \in (0,1)$. From (6.5), we have that

$$\begin{aligned}\lim_{\lambda \to 0} \phi'(\lambda) &= \lim_{\lambda \to 0} (\langle \eta_t \boldsymbol{g}_t, \boldsymbol{z} - \boldsymbol{x}_{t+1} \rangle + \langle \nabla \psi(\boldsymbol{x}_{t+1} + \lambda(\boldsymbol{z} - \boldsymbol{x}_{t+1})) - \nabla \psi(\boldsymbol{x}_t), \boldsymbol{z} - \boldsymbol{x}_{t+1} \rangle) \\ &= -\infty.\end{aligned}$$

Hence, there exists $\epsilon > 0$ such that

$$\langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_\epsilon \rangle + B_\psi(\boldsymbol{x}_\epsilon; \boldsymbol{x}_t) = \phi(\epsilon) < \phi(0) = \langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_{t+1} \rangle + B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t),$$

where $\boldsymbol{x}_\epsilon := (1-\epsilon)\boldsymbol{x}_{t+1} + \epsilon \boldsymbol{z} \in \operatorname{int} \mathcal{X} \cap \mathcal{V}$. However, this contradicts the definition of $\boldsymbol{x}_{t+1}$ as an argmin, proving that $\boldsymbol{x}_{t+1}$ must be in $\operatorname{int} \mathcal{X}$. □

When (6.5) holds, this theorem implies that the predictions of the algorithm always stay in the interior of $\mathcal{X}$ without the need for any projection. If in addition $\mathcal{V} = \mathcal{X}$, the update of the algorithm is the solution of an unconstrained problem because the feasible set is implicit in the Bregman divergence.

Now we have a well-defined algorithm, but does it guarantee a sublinear regret? We know that at least in one case, it recovers the OSD algorithm, which does work. So, from an intuitive point of view, how well the algorithm works should depend on some characteristic of ψ. In particular, a sufficient property will be the *strong convexity* of ψ. The strong convexity also takes care of the existence of the argmin in the algorithm, thanks to the next theorem.

Theorem 6.9 *Let $\lambda > 0$ and $f : \mathbb{R}^d \to (-\infty, +\infty]$ proper, closed, and λ-strongly convex with respect to $\|\cdot\|$ on its domain. Assume $\operatorname{dom} \partial f \neq \emptyset$. Then, f has exactly one minimizer.*

Proof Let $\boldsymbol{y} \in \operatorname{dom} \partial f$ and $\boldsymbol{g} \in \partial f(\boldsymbol{y})$. From Lemma 4.2, for any $\boldsymbol{x} \in \mathbb{R}^d$, we have

$$\begin{aligned}f(\boldsymbol{x}) &\geq f(\boldsymbol{y}) + \langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{y} \rangle + \frac{\lambda}{2}\|\boldsymbol{x} - \boldsymbol{y}\|^2 \\ &\geq f(\boldsymbol{y}) - \|\boldsymbol{g}\|_\star \|\boldsymbol{x}\| - \langle \boldsymbol{g}, \boldsymbol{y} \rangle + \frac{\lambda}{2}(\|\boldsymbol{x}\| - \|\boldsymbol{y}\|)^2 \\ &= f(\boldsymbol{y}) - \|\boldsymbol{g}\|_\star \|\boldsymbol{x}\| - \langle \boldsymbol{g}, \boldsymbol{y} \rangle + \frac{\lambda}{2}(\|\boldsymbol{x}\|^2 + \|\boldsymbol{y}\|^2 - 2\|\boldsymbol{x}\|\|\boldsymbol{y}\|),\end{aligned}$$

where in the second inequality we used the reverse triangle inequality and the definition of dual norms. From the above, we have that $\lim_{\|\boldsymbol{x}\| \to \infty} f(\boldsymbol{x}) = +\infty$. In turn, this implies that the level sets of f are bounded. From the assumption that f is closed, we get that the level sets are compact. Hence, for any $\boldsymbol{y}$ in $\operatorname{dom} f$, the minimum of f is the same as the minimum of f over the set $\{\boldsymbol{x} : f(\boldsymbol{x}) \leq f(\boldsymbol{y})\}$, that is, the minimum over a compact set, that exists by the Weierstrass theorem, Theorem A.3. The uniqueness is given by the fact that strongly convex functions are strictly convex. □

To analyze OMD, we first prove a one-step relationship, similar to the ones we proved for Online Gradient Descent (OGD) and OSD. Note how in this lemma, we will use a lot of the concepts we introduced till now: strong convexity, dual norms, subgradients, etc. In a way, over the past sections, I slowly prepared you to be able to prove this lemma.

Lemma 6.10 *Let B_ψ be the Bregman divergence with respect to $\psi : \mathcal{X} \to \mathbb{R}$ and assume ψ to be proper, closed, and λ-strongly convex with respect to $\|\cdot\|$ in $\mathcal{V}$, where $\mathcal{V} \subseteq \mathcal{X}$ is a non-empty closed convex set. Assume* (6.5) *or* (6.6) *hold. Then, with the notation and initialization in Algorithm 6.1, for all t we have that $\boldsymbol{x}_{t+1}$ exists, it is unique, and it is in the interior of $\mathcal{X}$. Moreover, $\forall \boldsymbol{u} \in \mathcal{V}$, the following inequality holds*

$$\begin{aligned}\eta_t(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) &\le \eta_t \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle \\ &\le B_\psi(\boldsymbol{u}; \boldsymbol{x}_t) - B_\psi(\boldsymbol{u}; \boldsymbol{x}_{t+1}) - B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) + \langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle \\ &\le B_\psi(\boldsymbol{u}; \boldsymbol{x}_t) - B_\psi(\boldsymbol{u}; \boldsymbol{x}_{t+1}) + \frac{\eta_t^2}{2\lambda}\|\boldsymbol{g}_t\|_\star^2 .\end{aligned}$$

Proof First of all, in each round $\boldsymbol{x}_{t+1}$ exists using Theorem 6.9 and the fact that $B_\psi(\cdot; \boldsymbol{x}_t)$ is proper, closed, and strongly convex. Moreover, from Theorem 6.8, $\boldsymbol{x}_t \in \operatorname{int} \mathcal{X}$ for all t.

Now, from the optimality condition of Theorem 2.8 for the update of OMD, we have

$$\langle \eta_t \boldsymbol{g}_t + \nabla\psi(\boldsymbol{x}_{t+1}) - \nabla\psi(\boldsymbol{x}_t), \boldsymbol{u} - \boldsymbol{x}_{t+1}\rangle \ge 0, \quad \forall \boldsymbol{u} \in \mathcal{V} . \tag{6.7}$$

Hence, we have that

$$\begin{aligned}\langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle &\le \langle \nabla\psi(\boldsymbol{x}_{t+1}) - \nabla\psi(\boldsymbol{x}_t), \boldsymbol{u} - \boldsymbol{x}_{t+1}\rangle + \langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle \\ &= B_\psi(\boldsymbol{u}; \boldsymbol{x}_t) - B_\psi(\boldsymbol{u}; \boldsymbol{x}_{t+1}) - B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) + \langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle \\ &\le B_\psi(\boldsymbol{u}; \boldsymbol{x}_t) - B_\psi(\boldsymbol{u}; \boldsymbol{x}_{t+1}) - \frac{\lambda}{2}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 + \eta_t \|\boldsymbol{g}_t\|_\star \|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\| \\ &\le B_\psi(\boldsymbol{u}; \boldsymbol{x}_t) - B_\psi(\boldsymbol{u}; \boldsymbol{x}_{t+1}) + \frac{\eta_t^2}{2\lambda}\|\boldsymbol{g}_t\|_\star^2,\end{aligned}$$

where in the first inequality we used (6.7), in the equality we used Lemma 6.7, in the second inequality we used the definition of dual norm, (6.4) (because ψ is λ-strong convex with respect to $\|\cdot\|$), and Cauchy–Schwarz inequality, finally in the last inequality we used the fact that $ax - \frac{b}{2}x^2 \le \frac{a^2}{2b}$ for all $x \in \mathbb{R}$ and $a, b > 0$.

The lower bound with the function values is due to the definition of subgradients. □

We now see how to use this one-step relationship to prove a regret bound that will finally show us if and when this entire construction is a good idea.

We can now prove a regret bound for OMD.

Theorem 6.11 *Assume $0 < \eta_{t+1} \le \eta_t,\ t = 1, \dots, T-1$. Then, under the assumptions of Lemma 6.10 and for all $\boldsymbol{u} \in \mathcal{V}$, the following regret bound holds*

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \le \max_{1\le t\le T} \frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_t)}{\eta_T} + \frac{1}{2\lambda}\sum_{t=1}^{T}\eta_t\|\boldsymbol{g}_t\|_\star^2 .$$

Moreover, if η_t is constant, i.e., $\eta_t = \eta\ \forall t = 1, \dots, T$, we also have

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \le \frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_1)}{\eta} + \frac{\eta}{2\lambda}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_\star^2 .$$

Proof Fix $\boldsymbol{u} \in \mathcal{V}$. As in the proof of OGD, dividing the inequality in Lemma 6.10 by η_t

and summing from $t = 1, \dots, T$, we get

$$\begin{aligned}
\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) &\leq \sum_{t=1}^{T}\left(\frac{B_\psi(\boldsymbol{u};\boldsymbol{x}_t)}{\eta_t} - \frac{B_\psi(\boldsymbol{u};\boldsymbol{x}_{t+1})}{\eta_t}\right) + \sum_{t=1}^{T}\frac{\eta_t\|\boldsymbol{g}_t\|_\star^2}{2\lambda} \\
&= \frac{B_\psi(\boldsymbol{u};\boldsymbol{x}_1)}{\eta_1} - \frac{B_\psi(\boldsymbol{u};\boldsymbol{x}_{T+1})}{\eta_T} + \sum_{t=1}^{T-1}\left(\frac{1}{\eta_{t+1}} - \frac{1}{\eta_t}\right)B_\psi(\boldsymbol{u};\boldsymbol{x}_{t+1}) + \sum_{t=1}^{T}\frac{\eta_t\|\boldsymbol{g}_t\|_\star^2}{2\lambda} \\
&\leq \frac{D^2}{\eta_1} + D^2\sum_{t=1}^{T-1}\left(\frac{1}{\eta_{t+1}} - \frac{1}{\eta_t}\right) + \sum_{t=1}^{T}\frac{\eta_t\|\boldsymbol{g}_t\|_\star^2}{2\lambda} \\
&= \frac{D^2}{\eta_1} + D^2\left(\frac{1}{\eta_T} - \frac{1}{\eta_1}\right) + \sum_{t=1}^{T}\frac{\eta_t\|\boldsymbol{g}_t\|_\star^2}{2\lambda} = \frac{D^2}{\eta_T} + \sum_{t=1}^{T}\frac{\eta_t\|\boldsymbol{g}_t\|_\star^2}{2\lambda},
\end{aligned}$$

where we denoted by $D^2 = \max_{1\leq t\leq T} B_\psi(\boldsymbol{u};\boldsymbol{x}_t)$.

The second statement is left as an exercise. □

In words, OMD allows us to prove regret guarantees that depend on an arbitrary pairs of dual norms $\|\cdot\|$ and $\|\cdot\|_\star$. In particular, the primal norm will be used to measure the feasible set $\mathcal{V}$ or the distance between the competitor and the initial point, and the dual norm will be used to measure the gradients. If you happen to know something about these quantities, we can choose the most appropriate pair of norms to guarantee a small regret. The only thing you need is a function ψ that is strongly convex with respect to the primal norm you have chosen.

Overall, the regret bound is still of the order of $\sqrt{T}$ for Lipschitz functions, and the only difference is that now the Lipschitz constant is measured with respect to a different norm. Also, everything we did for OSD can be trivially used here. For example, we can slightly generalize the stepsizes we saw in Section 4.2. Assuming $D^2 = \max_{\boldsymbol{x},\boldsymbol{y}\in\mathcal{V}} B_\psi(\boldsymbol{x};\boldsymbol{y}) < \infty$, set

$$\eta_t = \frac{D\sqrt{\lambda}}{\sqrt{\sum_{i=1}^{t}\|\boldsymbol{g}_i\|_\star^2}}$$

to achieve a regret upper bound of $2\frac{D}{\sqrt{\lambda}}\sqrt{\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_\star^2}$.

In Sections 6.6 and 6.7, we will see practical examples of OMD that guarantee strictly better regret than OSD. As we did in the case of AdaGrad, the better guarantee will depend on the shape of the domain and the characteristics of the subgradients.

Next, we see the meaning of the "Mirror", but first we need another mathematical tool: *Fenchel conjugates*.

6.4.1 Convex Analysis Bits: Fenchel Conjugate

Definition 6.12 *For a function $f : \mathbb{R}^d \to [-\infty, \infty]$, we define the* ***Fenchel conjugate*** *$f^\star : \mathbb{R}^d \to [-\infty, \infty]$ as*

$$f^\star(\boldsymbol{\theta}) = \sup_{\boldsymbol{x}\in\mathbb{R}^d} \langle\boldsymbol{\theta}, \boldsymbol{x}\rangle - f(\boldsymbol{x}) \,.$$

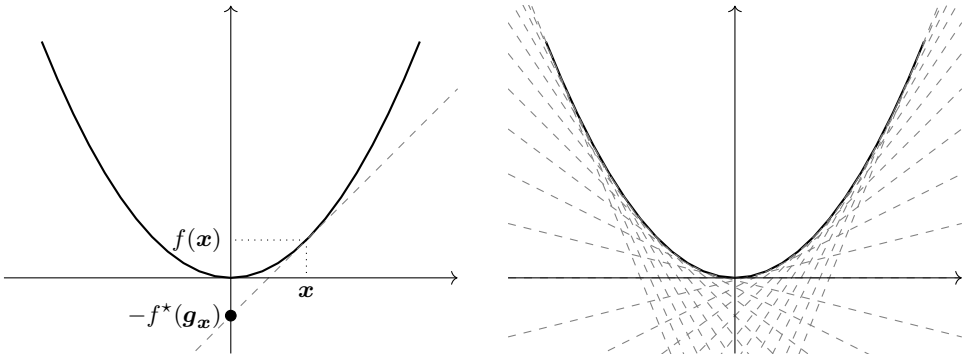


Figure 6.4 A geometric interpretation of the Fenchel conjugate.

From the definition we immediately obtain the **Fenchel–Young's inequality** for proper functions:

$$\langle \boldsymbol{\theta}, \boldsymbol{x} \rangle \leq f(\boldsymbol{x}) + f^\star(\boldsymbol{\theta}), \quad \forall \boldsymbol{x}, \boldsymbol{\theta} \in \mathbb{R}^d \, .$$

Moreover, $f^\star$ is always convex and closed, regardless of the convexity of f (Bauschke and Combettes, 2017, Proposition 13.13).

We have the following useful properties for the Fenchel conjugate.

Theorem 6.13 (Rockafellar, 1970, Theorem 12.2) *Let f be a convex function. Then, $f^\star$ is a closed convex function, proper iff f is proper. Moreover, if f is also closed then $f^{\star\star} = f$.*

Theorem 6.14 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be proper. Then, the following conditions are equivalent:*

(a) $\boldsymbol{\theta} \in \partial f(\boldsymbol{x})$.
(b) $\langle \boldsymbol{\theta}, \boldsymbol{y} \rangle - f(\boldsymbol{y})$ *achieves its supremum in* $\boldsymbol{y}$ *at* $\boldsymbol{y} = \boldsymbol{x}$.
(c) $f(\boldsymbol{x}) + f^\star(\boldsymbol{\theta}) = \langle \boldsymbol{\theta}, \boldsymbol{x} \rangle$.

Moreover, if f is also convex and closed, we have an additional equivalent condition

(d) $\boldsymbol{x} \in \partial f^\star(\boldsymbol{\theta})$.

Proof Let's prove (a)⇔(b). From the definition of subgradient, we immediately have

$$\langle \boldsymbol{\theta}, \boldsymbol{x} \rangle - f(\boldsymbol{x}) \geq \langle \boldsymbol{\theta}, \boldsymbol{y} \rangle - f(\boldsymbol{y}), \quad \forall \boldsymbol{y} \, .$$

Then, (b)⇔(c) by definition of $f^\star(\boldsymbol{\theta})$.

If f is also convex and closed, then $f^{\star\star} = f$ is proper by Theorem 6.13. Hence, (c) is equivalent to $f^{\star\star}(\boldsymbol{x}) + f^\star(\boldsymbol{\theta}) = \langle \boldsymbol{\theta}, \boldsymbol{x} \rangle$, which is equivalent to (d) by following the same reasoning as above. □

In the following, we will not need a geometric intuition of the concept of Fenchel conjugates in order to use them. However, for some people, geometric intuitions help them remember better, so let's briefly discuss it. Let f be convex, closed, and proper. Let $\boldsymbol{x} \in$

$\operatorname{dom} \partial f$ and use a subgradient $\boldsymbol{g}_{\boldsymbol{x}} \in \partial f(\boldsymbol{x})$ to construct a linear lower bound around $\boldsymbol{x}$ passing through $f(\boldsymbol{x})$ as

$$\tilde{f}(\boldsymbol{y}) := f(\boldsymbol{x}) + \langle \boldsymbol{g}_{\boldsymbol{x}}, \boldsymbol{y} - \boldsymbol{x} \rangle \, .$$

Moreover, from Theorem 6.14(c), we have

$$\tilde{f}(\mathbf{0}) = f(\boldsymbol{x}) - \langle \boldsymbol{g}_{\boldsymbol{x}}, \boldsymbol{x} \rangle = -f^\star(\boldsymbol{g}_{\boldsymbol{x}}) \, .$$

Hence, we have that $-f^\star(\boldsymbol{g}_{\boldsymbol{x}})$ is the value of $\tilde{f}$ at $\boldsymbol{y} = \mathbf{0}$, see Figure 6.4 (left). This is sometimes mentioned as a geometric intuition for Fenchel conjugates, but I do not find it very illuminating, so let's dig deeper.

We can show that we can recover f as the point-wise maximum of all its tangents, see Figure 6.4 (right). In turn, the tangents can be expressed using only the knowledge of $f^\star$. Let's consider the family of linear functions $\tilde{f}_{\boldsymbol{g}}(\boldsymbol{y}) = -f^\star(\boldsymbol{g}) + \langle \boldsymbol{g}, \boldsymbol{y} \rangle$ parametrized by a generic $\boldsymbol{g}$. By Fenchel–Young's inequality, we immediately have $\tilde{f}_{\boldsymbol{g}}(\boldsymbol{y}) \leq f(\boldsymbol{y})$. Moreover, from Theorem 6.14(c), if $\boldsymbol{g}_{\boldsymbol{y}} \in \partial f(\boldsymbol{y})$, then $\tilde{f}_{\boldsymbol{g}_{\boldsymbol{y}}}(\boldsymbol{y}) = f(\boldsymbol{y})$. This means that this family of functions lower bounds f and it contains the tangents to f. Now, observe that for any $\boldsymbol{y}$ we obtain

$$\sup_{\boldsymbol{g} \in \mathbb{R}^d} \tilde{f}_{\boldsymbol{g}}(\boldsymbol{y}) = \sup_{\boldsymbol{g} \in \mathbb{R}^d} -f^\star(\boldsymbol{g}) + \langle \boldsymbol{g}, \boldsymbol{y} \rangle = f^{\star\star}(\boldsymbol{y}) = f(\boldsymbol{y}),$$

where in last equality we used Theorem 6.13. So, the supremum over this family of linear functions is equal to the function f. Overall, this means that the Fenchel conjugate allows us to reason about functions using their tangent hyperplanes, whose information is in the Fenchel conjugate, without losing anything.

Another way to quantify the above is the fact that the domain of $f^\star$ contains all the possible subgradients of f.

Corollary 6.15 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be proper. Then, $\{\boldsymbol{g} \in \partial f(\boldsymbol{x}) : \boldsymbol{x} \in \mathbb{R}^d\} \subseteq \operatorname{dom} f^\star$.*

Proof From Theorem 6.14 (a) $\Leftrightarrow$ (c) and the fact that $\operatorname{dom} \partial f \subseteq \operatorname{dom} f$, we have

$$\boldsymbol{g} \in \partial f(\boldsymbol{x}) \Leftrightarrow f(\boldsymbol{x}) + f^\star(\boldsymbol{g}) = \langle \boldsymbol{g}, \boldsymbol{x} \rangle \Rightarrow f^\star(\boldsymbol{g}) < +\infty \, . \qquad \square$$

Note that there are cases where the domain of $f^\star$ contains vectors that are not subgradients, as shown in the next example.

Example 6.16 *Let $f(x) = \exp(x)$, hence we have $f^\star(\theta) = \sup_x \; x\theta - \exp(x)$. Solving it, we have $x^\star = \ln(\theta)$ if $\theta > 0$. Hence, $f^\star(\theta) = \begin{cases} \theta \ln \theta - \theta, & \text{if } \theta > 0 \\ 0, & \text{if } \theta = 0 \\ +\infty, & \text{if } \theta < 0 \end{cases}$. So, $f^\star(0) = \sup_x \; -\exp(x) = 0 < \infty$, but 0 is not a derivative of f.*

Example 6.17 (Conjugate of the inner product) *Let $f(\boldsymbol{x}) = \langle \boldsymbol{z}, \boldsymbol{x} \rangle$ where $\boldsymbol{z} \neq \mathbf{0} \in \mathbb{R}^d$. Then*

$$f^\star(\boldsymbol{\theta}) = \sup_{\boldsymbol{x} \in \mathbb{R}^d} \langle \boldsymbol{\theta} - \boldsymbol{z}, \boldsymbol{x} \rangle = \begin{cases} 0, & \boldsymbol{\theta} = \boldsymbol{z} \\ +\infty, & \text{otherwise.} \end{cases}$$

Example 6.18 (Conjugate of the hinge loss) *Let $f(\boldsymbol{x}) = \max(1 - \langle \boldsymbol{z}, \boldsymbol{x} \rangle, 0)$ where $\boldsymbol{z} \in \mathbb{R}^d \setminus \{\mathbf{0}\}$ and let's calculate $f^\star(\boldsymbol{\theta})$. If $\boldsymbol{\theta}$ has a component orthogonal to $\boldsymbol{z}$, I can choose $\boldsymbol{x}$ along that component, and the supremum in the definition of $f^\star$ is $+\infty$. Hence, let's consider the case that $\boldsymbol{\theta} = \alpha \boldsymbol{z}$. In this case, we have*

$$f^\star(\boldsymbol{\theta}) = \sup_{\boldsymbol{x}} \ \alpha \langle \boldsymbol{z}, \boldsymbol{x} \rangle - \max(1 - \langle \boldsymbol{z}, \boldsymbol{x} \rangle, 0) = \sup_{u} \ \alpha u - \max(1 - u, 0) \ .$$

If $\alpha > 0$ or $\alpha < -1$, again the supremum is $+\infty$. Hence, we only need to consider the case that $-1 \leq \alpha \leq 0$. From a case analysis on u, it is easy to see that in this case the supremum is attained in $u = 1$. Putting everything together, we have

$$f^\star(\boldsymbol{\theta}) = \begin{cases} \alpha, & \text{if } \boldsymbol{\theta} = \alpha \boldsymbol{z}, \alpha \in [-1, 0], \\ +\infty, & \text{otherwise.} \end{cases}$$

Example 6.19 (Conjugate of squared norms) *Consider the function $f(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{x}\|^2$, where $\|\cdot\|$ is a norm in $\mathbb{R}^d$, with dual norm $\|\cdot\|_\star$. We can show that its conjugate is $f^\star(\boldsymbol{\theta}) = \frac{1}{2}\|\boldsymbol{\theta}\|_\star^2$. Let's see how. First, we have*

$$\langle \boldsymbol{\theta}, \boldsymbol{x} \rangle - \frac{1}{2}\|\boldsymbol{x}\|^2 \leq \|\boldsymbol{\theta}\|_\star \|\boldsymbol{x}\| - \frac{1}{2}\|\boldsymbol{x}\|^2$$

for all $\boldsymbol{x}$. The r.h.s. is a quadratic function of $\|\boldsymbol{x}\|$, which has maximum value $\frac{1}{2}\|\boldsymbol{\theta}\|_\star^2$. Therefore for all $\boldsymbol{x}$, we have

$$\langle \boldsymbol{\theta}, \boldsymbol{x} \rangle - \frac{1}{2}\|\boldsymbol{x}\|^2 \leq \frac{1}{2}\|\boldsymbol{\theta}\|_\star^2,$$

which shows that $f^\star(\boldsymbol{\theta}) \leq \frac{1}{2}\|\boldsymbol{\theta}\|_\star^2$. To show the other inequality, by Lemma 4.20, let $\boldsymbol{x}$ be any vector with $\langle \boldsymbol{\theta}, \boldsymbol{x} \rangle = \|\boldsymbol{\theta}\|_\star \|\boldsymbol{x}\|$ and $\|\boldsymbol{x}\| = \|\boldsymbol{\theta}\|_\star$. Then, for this $\boldsymbol{x}$, we have

$$\langle \boldsymbol{\theta}, \boldsymbol{x} \rangle - \frac{1}{2}\|\boldsymbol{x}\|^2 = \frac{1}{2}\|\boldsymbol{\theta}\|_\star^2,$$

which shows that $f^\star(\boldsymbol{\theta}) \geq \frac{1}{2}\|\boldsymbol{\theta}\|_\star^2$.

Example 6.20 (Young's inequality) *Let $p > 1$ and $f : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$ defined as $f(x) = \frac{1}{p}x^p$. Let's calculate the conjugate. We have*

$$f^\star(\theta) = \sup_{x \geq 0} \ \theta x - \frac{1}{p}x^p \ .$$

If $\theta < 0$, then the supremum is 0. For $\theta \geq 0$, by differentiation, we have that $x^\star = \theta^{\frac{1}{p-1}}$. So, we have

$$f^\star(\theta) = \begin{cases} 0, & \text{if } \theta < 0, \\ \frac{p-1}{p}\theta^{\frac{p}{p-1}}, & \text{if } \theta \geq 0 \ . \end{cases}$$

Denoting by $q > 1$ the positive number such that $\frac{1}{p} + \frac{1}{q} = 1$, we can rewrite it as

$$f^\star(\theta) = \begin{cases} 0, & \text{if } \theta < 0, \\ \frac{1}{q}\theta^q, & \text{if } \theta \geq 0 \ . \end{cases}$$

Using the Fenchel–Young's inequality, for $x, y \geq 0$ and $p, q > 1$ such that $1/p + 1/q = 1$, we have

$$xy \leq \frac{1}{p}x^p + \frac{1}{q}y^q,$$

that is called Young's inequality.

Example 6.21 (Conjugate of norms) *Consider the function $f(\boldsymbol{x}) = \|\boldsymbol{x}\|$, where $\|\cdot\|$ is a norm in $\mathbb{R}^d$, with dual norm $\|\cdot\|_\star$. Then, we have*

$$f^\star(\boldsymbol{\theta}) = \sup_{\boldsymbol{x}\in\mathbb{R}^d} \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle - \|\boldsymbol{x}\| \leq \sup_{\boldsymbol{x}\in\mathbb{R}^d} \|\boldsymbol{\theta}\|_\star\|\boldsymbol{x}\| - \|\boldsymbol{x}\| = \begin{cases} 0, & \|\boldsymbol{\theta}\|_\star \leq 1, \\ +\infty, & \|\boldsymbol{\theta}\|_\star > 1\,. \end{cases}$$

To show the other inequality, by Lemma 4.20, let $\boldsymbol{x}$ be any vector with $\langle \boldsymbol{\theta}, \boldsymbol{x}\rangle = \|\boldsymbol{\theta}\|_\star\|\boldsymbol{x}\|$ and $\|\boldsymbol{x}\| = \alpha$. Then, we have

$$f^\star(\boldsymbol{\theta}) = \sup_{\boldsymbol{x}\in\mathbb{R}^d} \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle - \|\boldsymbol{x}\| \geq \sup_{\alpha\in\mathbb{R}_{\geq 0}} \alpha\|\boldsymbol{\theta}\|_\star - \alpha = \begin{cases} 0, & \|\boldsymbol{\theta}\|_\star \leq 1, \\ +\infty, & \|\boldsymbol{\theta}\|_\star > 1\,. \end{cases}$$

Lemma 6.22 *Let f be a function and let $f^\star$ be its Fenchel conjugate. For $a > 0$ and $b \in \mathbb{R}$, the Fenchel conjugate of $h(\boldsymbol{x}) = af(\boldsymbol{x}) + b + \langle \boldsymbol{g}, \boldsymbol{x}\rangle$ is $h^\star(\boldsymbol{\theta}) = af^\star((\boldsymbol{\theta} - \boldsymbol{g})/a) - b$.*

Proof From the definition of conjugate function, we have

$$\begin{aligned} h^\star(\boldsymbol{\theta}) &= \sup_{\boldsymbol{x}\in\mathbb{R}^d} \langle \boldsymbol{\theta} - \boldsymbol{g}, \boldsymbol{x}\rangle - af(\boldsymbol{x}) - b = -b + a \sup_{\boldsymbol{x}\in\mathbb{R}^d} \left\langle \frac{\boldsymbol{\theta} - \boldsymbol{g}}{a}, \boldsymbol{x} \right\rangle - f(\boldsymbol{x}) \\ &= -b + af^\star\left(\frac{\boldsymbol{\theta} - \boldsymbol{g}}{a}\right)\,. \end{aligned}$$

□

Lemma 6.23 *Let f_1 and f_2 such that $f_1(\boldsymbol{x}) \leq f_2(\boldsymbol{x})$ for all $\boldsymbol{x}$. Then, $f_1^\star(\boldsymbol{\theta}) \geq f_2^\star(\boldsymbol{\theta})$ for all $\boldsymbol{\theta}$.*

Proof

$$f_2^\star(\boldsymbol{\theta}) = \sup_{\boldsymbol{x}} \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle - f_2(\boldsymbol{x}) \leq \sup_{\boldsymbol{x}} \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle - f_1(\boldsymbol{x}) = f_1^\star(\boldsymbol{\theta})\,.$$

□

Lemma 6.24 (Bauschke and Combettes, 2017, Example 13.8) *Let $f : \mathbb{R} \to (-\infty, +\infty]$ be even, i.e., $f(x) = f(-x)$. Then $(f \circ \|\cdot\|_2)^\star = f^\star \circ \|\cdot\|_2$.*

Remark 6.25 *There is a tight connection between the dual function in convex optimization and the Fenchel conjugate. Indeed, let $f : \mathbb{R}^d \to (-\infty, +\infty]$ and consider the constrained optimization problem*

$$\begin{aligned} \min_{\boldsymbol{x}} \quad & f(\boldsymbol{x}) \\ \text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x} \leq \boldsymbol{b} \\ & \boldsymbol{C}\boldsymbol{x} = \boldsymbol{d}, \end{aligned}$$

where the inequality is component-wise and $\boldsymbol{A}, \boldsymbol{C}$ are matrices. The dual *function is defined*

as

$$\begin{aligned} g(\boldsymbol{\lambda}, \boldsymbol{\nu}) &= \inf_{\boldsymbol{x}} \left(f(\boldsymbol{x}) + \boldsymbol{\lambda}^\top (\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}) + \boldsymbol{\nu}^\top (\boldsymbol{C}\boldsymbol{x} - \boldsymbol{d}) \right) \\ &= -\boldsymbol{b}^\top \boldsymbol{\lambda} - \boldsymbol{d}^\top \boldsymbol{\nu} + \inf_{\boldsymbol{x}} \left(f(\boldsymbol{x}) + (\boldsymbol{A}^\top \boldsymbol{\lambda} + \boldsymbol{C}^\top \boldsymbol{\nu})^\top \boldsymbol{x} \right) \\ &= -\boldsymbol{b}^\top \boldsymbol{\lambda} - \boldsymbol{d}^\top \boldsymbol{\nu} - f^\star(-\boldsymbol{A}^\top \boldsymbol{\lambda} - \boldsymbol{C}^\top \boldsymbol{\nu}) \, . \end{aligned}$$

For closed, convex, and proper functions, Theorem 6.14 implies that $\boldsymbol{x} \in \partial f^\star(\boldsymbol{\theta})$ iff $\boldsymbol{\theta} \in \partial f(\boldsymbol{x})$, that in words means that $(\partial f)^{-1} = \partial f^\star$ in the sense of multivalued mappings. Now, we show that for strongly convex functions, the Fenchel conjugate is smooth and hence differentiable.

Theorem 6.26 (Duality Strong Convexity/Smoothness) *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be a proper, closed, convex function, and $\operatorname{dom} \partial f$ be non-empty. Then, f is $\lambda > 0$ strongly convex with respect to $\|\cdot\|$ on $\operatorname{dom} \partial f$ iff $f^\star$ is $\frac{1}{\lambda}$-smooth with respect to $\|\cdot\|_\star$ on $\mathbb{R}^d$.*

Proof Let's first prove the implication from left to right. First, let's show that $f^\star$ is differentiable. Since f is proper, closed, and strongly convex, the maximizer of $\max_{\boldsymbol{x}} \langle \boldsymbol{\theta}, \boldsymbol{x} \rangle - f(\boldsymbol{x})$ exists, and it is unique by Theorem 6.9. Denote by $\boldsymbol{x}^\star$ the argmax of this expression. Hence, from Theorem 6.14, we have $\boldsymbol{x}^\star \in \partial f^\star(\boldsymbol{\theta})$. Let's now show that this is the only element in the subdifferential. Assume there exists $\boldsymbol{x}' \in \partial f^\star(\boldsymbol{\theta})$, then from Theorem 6.14, we have $f^\star(\boldsymbol{\theta}) = \langle \boldsymbol{\theta}, \boldsymbol{x}' \rangle - f(\boldsymbol{x}')$ but from the uniqueness of the maximizer we have that $\boldsymbol{x}^\star = \boldsymbol{x}'$.

Now, let's prove that the gradient of $f^\star$ is $\frac{1}{\lambda}$-Lipschitz with respect to $\|\cdot\|_\star$. For any $\boldsymbol{\theta}_1$ and $\boldsymbol{\theta}_2$, set $\boldsymbol{x}_1 = \nabla f^\star(\boldsymbol{\theta}_1)$ and $\boldsymbol{x}_2 = \nabla f^\star(\boldsymbol{\theta}_2)$. From Theorem 6.14, we have that $\boldsymbol{\theta}_1 \in \partial f(\boldsymbol{x}_1)$ and $\boldsymbol{\theta}_2 \in \partial f(\boldsymbol{x}_2)$. Hence, by Lemma 4.2, we have

$$\begin{aligned} f(\boldsymbol{x}_2) &\geq f(\boldsymbol{x}_1) + \langle \boldsymbol{\theta}_1, \boldsymbol{x}_2 - \boldsymbol{x}_1 \rangle + \frac{\lambda}{2} \|\boldsymbol{x}_1 - \boldsymbol{x}_2\|^2, \\ f(\boldsymbol{x}_1) &\geq f(\boldsymbol{x}_2) + \langle \boldsymbol{\theta}_2, \boldsymbol{x}_1 - \boldsymbol{x}_2 \rangle + \frac{\lambda}{2} \|\boldsymbol{x}_1 - \boldsymbol{x}_2\|^2 \, . \end{aligned}$$

Summing these two inequalities, we have

$$\|\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2\|_\star \|\boldsymbol{x}_1 - \boldsymbol{x}_2\| \geq \langle \boldsymbol{\theta}_2 - \boldsymbol{\theta}_1, \boldsymbol{x}_1 - \boldsymbol{x}_2 \rangle \geq \lambda \|\boldsymbol{x}_1 - \boldsymbol{x}_2\|^2,$$

where in the first inequality we used the definition of dual norms. Solving the inequality, we get that

$$\|\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2\|_\star \geq \lambda \|\boldsymbol{x}_1 - \boldsymbol{x}_2\| = \lambda \|\nabla f^\star(\boldsymbol{\theta}_1) - \nabla f^\star(\boldsymbol{\theta}_2)\| \, .$$

Let's now prove the other direction. Assume that $f^\star$ is $\frac{1}{\lambda}$-smooth with respect to $\|\cdot\|_\star$ on $\mathbb{R}^d$. Set $\boldsymbol{y} \in \operatorname{dom} \partial f$ and $\boldsymbol{u} \in \partial f(\boldsymbol{y})$. Hence, by Theorem 6.14 and the differentiability of $f^\star$, we also have $\boldsymbol{y} = \nabla f^\star(\boldsymbol{u})$. Define $\phi(\boldsymbol{\theta}) := f^\star(\boldsymbol{\theta} + \boldsymbol{u}) - f^\star(\boldsymbol{u}) - \langle \boldsymbol{\theta}, \nabla f^\star(\boldsymbol{u}) \rangle$. From the $\frac{1}{\lambda}$-smoothness and Lemma 4.24, we have $\phi(\boldsymbol{\theta}) \leq \frac{1}{2\lambda} \|\boldsymbol{\theta}\|_\star^2$. From Lemma 6.23 and

Example 6.19, we have that $\phi^\star(\boldsymbol{x}) \geq \frac{\lambda}{2}\|\boldsymbol{x}\|^2$. Let's now calculate $\phi^\star(\boldsymbol{x})$.

$$\begin{aligned}
\phi^\star(\boldsymbol{x}) &= \sup_{\boldsymbol{\theta}} \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle - f^\star(\boldsymbol{\theta}+\boldsymbol{u}) + f^\star(\boldsymbol{u}) + \langle \boldsymbol{\theta}, \nabla f^\star(\boldsymbol{u})\rangle \\
&= f^\star(\boldsymbol{u}) - \langle \boldsymbol{u}, \boldsymbol{x} + \nabla f^\star(\boldsymbol{u})\rangle + \sup_{\boldsymbol{v}} \langle \boldsymbol{v}, \boldsymbol{x} + \nabla f^\star(\boldsymbol{u})\rangle - f^\star(\boldsymbol{v}) \\
&= f^\star(\boldsymbol{u}) - \langle \boldsymbol{u}, \boldsymbol{x} + \nabla f^\star(\boldsymbol{u})\rangle + f(\boldsymbol{x} + \nabla f^\star(\boldsymbol{u})) \\
&= -\langle \boldsymbol{u}, \boldsymbol{x}\rangle - f(\nabla f^\star(\boldsymbol{u})) + f(\boldsymbol{x} + \nabla f^\star(\boldsymbol{u})),
\end{aligned}$$

where we used Theorem 6.13 in the third equality and Theorem 6.14 in the last one. Putting everything together, we have

$$f(\boldsymbol{x}+\boldsymbol{y}) - f(\boldsymbol{y}) - \langle \boldsymbol{u}, \boldsymbol{x}\rangle \geq \frac{\lambda}{2}\|\boldsymbol{x}\|^2, \quad \forall \boldsymbol{u} \in \partial f(\boldsymbol{y})\,. \qquad \square$$

We will also use the following theorem on the first-order optimality condition.

Theorem 6.27 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be proper. Then $\boldsymbol{x}^\star \in \operatorname{argmin}_{\boldsymbol{x}\in\mathbb{R}^d} f(\boldsymbol{x})$ iff $\mathbf{0} \in \partial f(\boldsymbol{x}^\star)$.*

Proof We have that

$$\begin{aligned}
\boldsymbol{x}^\star \in \operatorname*{argmin}_{\boldsymbol{x}\in\mathbb{R}^d} f(\boldsymbol{x}) &\Leftrightarrow \forall \boldsymbol{y} \in \mathbb{R}^d,\ f(\boldsymbol{y}) \geq f(\boldsymbol{x}^\star) = f(\boldsymbol{x}^\star) + \langle \mathbf{0}, \boldsymbol{y} - \boldsymbol{x}^\star\rangle \\
&\Leftrightarrow \mathbf{0} \in \partial f(\boldsymbol{x}^\star)\,.
\end{aligned}$$

$\square$

6.4.2 The "Mirror" Interpretation

Here, we explain the "mirror" interpretation of OMD, using the following theorem.

Theorem 6.28 *Let B_ψ be the Bregman divergence with respect to $\psi : \mathcal{X} \to \mathbb{R}$, where ψ is $\lambda > 0$ strongly convex and closed. Let $\mathcal{V} \subseteq \mathcal{X}$ be a non-empty closed convex set and $\boldsymbol{x}_t \in \mathcal{V}$. Define*

$$\boldsymbol{x}_{t+1} = \operatorname*{argmin}_{\boldsymbol{x}\in\mathcal{V}} \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle + \frac{1}{\eta_t} B_\psi(\boldsymbol{x}; \boldsymbol{x}_t),$$

and assume ψ to be differentiable in $\boldsymbol{x}_t$ and $\boldsymbol{x}_{t+1}$. Then, for any $\boldsymbol{g}_t \in \mathbb{R}^d$, we have

$$\boldsymbol{x}_{t+1} = \nabla \psi_{\mathcal{V}}^\star(\nabla\psi(\boldsymbol{x}_t) - \eta_t \boldsymbol{g}_t), \tag{6.8}$$

where $\psi_{\mathcal{V}}$ is the restriction of ψ to $\mathcal{V}$, that is, $\psi_{\mathcal{V}} := \psi + \iota_{\mathcal{V}}$.

Proof We have that

$$\begin{aligned}
\boldsymbol{x}_{t+1} &= \operatorname*{argmin}_{\boldsymbol{x}\in\mathcal{V}} \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle + \frac{1}{\eta_t} B_\psi(\boldsymbol{x}; \boldsymbol{x}_t) \\
&= \operatorname*{argmin}_{\boldsymbol{x}\in\mathcal{V}} \eta_t \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle + B_\psi(\boldsymbol{x}; \boldsymbol{x}_t) \\
&= \operatorname*{argmin}_{\boldsymbol{x}\in\mathcal{V}} \eta_t \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle + \psi(\boldsymbol{x}) - \psi(\boldsymbol{x}_t) - \langle \nabla\psi(\boldsymbol{x}_t), \boldsymbol{x} - \boldsymbol{x}_t\rangle \\
&= \operatorname*{argmin}_{\boldsymbol{x}\in\mathcal{V}} \langle \eta_t \boldsymbol{g}_t - \nabla\psi(\boldsymbol{x}_t), \boldsymbol{x}\rangle + \psi(\boldsymbol{x})\,.
\end{aligned}$$

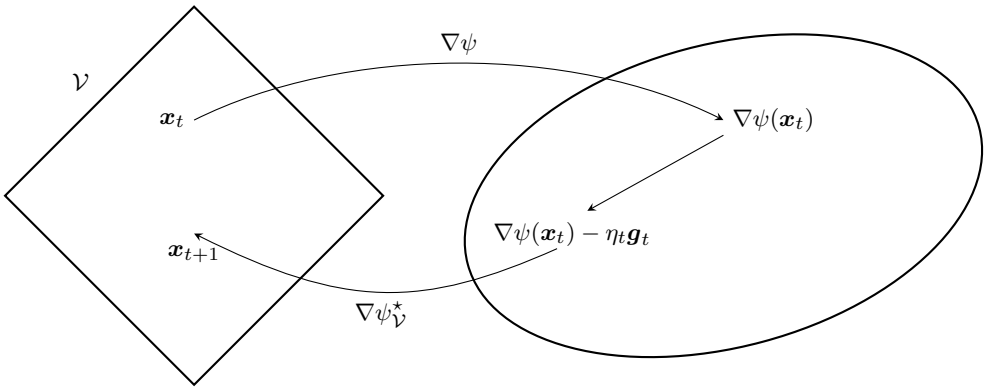


Figure 6.5 OMD update in terms of duality maps.

Now, we use the first-order optimality condition in Theorem 6.27 to have

$$\mathbf{0} \in \eta_t \boldsymbol{g}_t + \nabla\psi(\boldsymbol{x}_{t+1}) - \nabla\psi(\boldsymbol{x}_t) + \partial\iota_{\mathcal{V}}(\boldsymbol{x}_{t+1}),$$

that is

$$\nabla\psi(\boldsymbol{x}_t) - \eta_t \boldsymbol{g}_t \in (\nabla\psi + \partial\iota_{\mathcal{V}})(\boldsymbol{x}_{t+1}) \subseteq \partial\psi_{\mathcal{V}}(\boldsymbol{x}_{t+1}),$$

where in the last inclusion we used Theorem 2.23. Hence, from Theorem 6.14, we have

$$\boldsymbol{x}_{t+1} \in \partial\psi^\star_{\mathcal{V}}(\nabla\psi(\boldsymbol{x}_t) - \eta_t \boldsymbol{g}_t)\ .$$

Using the fact that $\psi_{\mathcal{V}} := \psi + \iota_{\mathcal{V}}$ is λ-strongly convex, proper, and closed, from Theorem 6.26 we have that $\partial\psi^\star_{\mathcal{V}} = \{\nabla\psi^\star_{\mathcal{V}}\}$. Hence,

$$\boldsymbol{x}_{t+1} = \nabla\psi^\star_{\mathcal{V}}(\nabla\psi(\boldsymbol{x}_t) - \eta_t \boldsymbol{g}_t)\ . \qquad \square$$

Let's explain what this theorem says. We said that OMD extends the OSD method to non-Euclidean norms. Hence, the regret bound we proved contains a pair of dual norms: one for distances between iterates and one for gradients. We also said that it makes sense to use a dual norm to measure a gradient, because it is a natural way to measure how "big" the linear functional $\boldsymbol{x} \mapsto \langle \nabla f(\boldsymbol{y}), \boldsymbol{x}\rangle$ is. More precisely, the gradient is naturally an element of the *dual space*, that is, a different space from the one where the predictions live. Hence, in general, summing iterates and gradients is not the right geometric operation, in the same way in which we cannot sum pears and apples together. In the Euclidean case, the dual of the L_2 norm is again the L_2 norm, so the distinction between the geometry used for predictions and the geometry used for gradients is hidden. For a general norm, predictions and gradients are measured with different, dual norms, and OMD uses the mirror maps to move between these two geometries.

So, in OMD we need a way to go from one space to the other. This is exactly the role of $\nabla\psi$ and $\nabla\psi^\star_{\mathcal{V}}$, which are called **duality maps** or **mirror** and **inverse mirror** maps. We can now understand that the theorem tells us that OMD takes the primal vector $\boldsymbol{x}_t$, transforms it into a dual vector through $\nabla\psi$, does a subgradient descent step in the dual space, and finally transforms the vector back to the primal space through $\nabla\psi^\star_{\mathcal{V}}$. This reasoning is summarized in Figure 6.5.

Example 6.29 *Let $\psi : \mathbb{R}^d \to \mathbb{R}$ equal to $\psi(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{x}\|_2^2$ and $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \leq 1\}$. Define $\psi_{\mathcal{V}} = \psi + \iota_{\mathcal{V}}$. Then, we have*

$$\psi_{\mathcal{V}}^\star(\boldsymbol{\theta}) = \sup_{\boldsymbol{x}\in\mathcal{V}} \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle - \frac{1}{2}\|\boldsymbol{x}\|_2^2 \,.$$

Let's compute this conjugate. First of all, if $\boldsymbol{\theta} = \mathbf{0}$ we have that $\psi_{\mathcal{V}}^\star(\boldsymbol{\theta}) = 0$. So, in the following, we assume $\boldsymbol{\theta} \neq \mathbf{0}$. For any $\boldsymbol{x} \in \mathcal{V}$ there exist $\boldsymbol{q}$ and α such that $\boldsymbol{x} = \alpha\frac{\boldsymbol{\theta}}{\|\boldsymbol{\theta}\|_2} + \boldsymbol{q}$ and $\langle \boldsymbol{q}, \boldsymbol{\theta}\rangle = 0$. Hence, we have

$$\sup_{\boldsymbol{x}\in\mathcal{V}} \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle - \frac{1}{2}\|\boldsymbol{x}\|_2^2 = \sup_{\alpha,\boldsymbol{q}:\alpha\frac{\boldsymbol{\theta}}{\|\boldsymbol{\theta}\|_2}+\boldsymbol{q}\in\mathcal{V},\langle \boldsymbol{q},\boldsymbol{\theta}\rangle=0} \alpha\|\boldsymbol{\theta}\|_2 - \frac{\alpha^2}{2} - \frac{1}{2}\|\boldsymbol{q}\|_2^2 = \sup_{-1\leq\alpha\leq 1} \alpha\|\boldsymbol{\theta}\|_2 - \frac{\alpha^2}{2}.$$

Solving the constrained optimization problem, we have $\alpha^\star = \min(1, \|\boldsymbol{\theta}\|_2)$. Hence, we have

$$\psi_{\mathcal{V}}^\star(\boldsymbol{\theta}) = \begin{cases} \frac{1}{2}\|\boldsymbol{\theta}\|_2^2, & \|\boldsymbol{\theta}\|_2 \leq 1 \\ \|\boldsymbol{\theta}\|_2 - \frac{1}{2}, & \|\boldsymbol{\theta}\|_2 > 1 \end{cases},$$

that is finite everywhere and differentiable.

So, the two duality maps are $\nabla\psi(\boldsymbol{x}) = \boldsymbol{x}$ and

$$\nabla\psi_{\mathcal{V}}^\star(\boldsymbol{\theta}) = \begin{cases} \boldsymbol{\theta}, & \|\boldsymbol{\theta}\|_2 \leq 1 \\ \frac{\boldsymbol{\theta}}{\|\boldsymbol{\theta}\|_2}, & \|\boldsymbol{\theta}\|_2 > 1 \end{cases} = \Pi_{\mathcal{V}}(\boldsymbol{\theta}) \,.$$

Using (6.8)*, we obtain exactly the update of projected online subgradient descent.*

6.4.3 Yet Another Way to Write the Online Mirror Descent Update

There exists yet another way to write the update of OMD. This third method uses the concept of *Bregman projections*. Extending the definition of Euclidean projections, we can define the projection with respect to a Bregman divergence.

Definition 6.30 *Let $\psi : \mathcal{X} \to \mathbb{R}$ be strictly convex and differentiable in the interior of $\mathcal{X}$. Assume that $\mathcal{V} \subseteq \mathcal{X}$ is convex and closed. Then, for any $\boldsymbol{x} \in \operatorname{int}\mathcal{X}$, we define the* ***Bregman projection*** *as*

$$\Pi_{\mathcal{V},B_\psi}(\boldsymbol{x}) := \operatorname*{argmin}_{\boldsymbol{y}\in\mathcal{V}} \; B_\psi(\boldsymbol{y}; \boldsymbol{x}),$$

when the argmin exists.

In the online learning literature, the OMD algorithm is typically presented with a two-step update: first, solving the argmin over the entire space and then projecting back over $\mathcal{V}$ with respect to the Bregman divergence. In the following, we show that most of the time the two-step update is equivalent to the one-step update in (6.8).

First, we prove a general theorem that allows us to break the constrained minimization of functions in the minimization over the entire space, plus a Bregman projection step.

Theorem 6.31 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be proper, closed, strictly convex, and differentiable in $\operatorname{int}\operatorname{dom} f$. Also, let $\mathcal{V} \subset \mathbb{R}^d$ be a non-empty, closed convex set with $\mathcal{V} \cap \operatorname{dom} f \neq \emptyset$ and assume that $\tilde{\boldsymbol{y}} = \operatorname{argmin}_{\boldsymbol{z}\in\mathbb{R}^d} \; f(\boldsymbol{z})$ exists and $\tilde{\boldsymbol{y}} \in \operatorname{int}\operatorname{dom} f$. Then, we have*

1 $\operatorname{argmin}_{\boldsymbol{z}\in\mathcal{V}}\ f(\boldsymbol{z})$ *contains exactly one element.*
2 $\operatorname{argmin}_{\boldsymbol{z}\in\mathcal{V}}\ f(\boldsymbol{z}) = \operatorname{argmin}_{\boldsymbol{z}\in\mathcal{V}}\ B_f(\boldsymbol{z};\tilde{\boldsymbol{y}})$.

Proof For the first point, from Bauschke and Combettes (2017, Proposition 11.13) and the existence of $\tilde{\boldsymbol{y}}$, we have that f is coercive. So, from Bauschke and Combettes (2017, Proposition 11.15), the minimizer of f in $\mathcal{V}$ exists. Given that f is strictly convex, the minimizer must be unique too.

For the second point, denote by $\boldsymbol{y}' = \operatorname{argmin}_{\boldsymbol{z}\in\mathcal{V}}\ B_f(\boldsymbol{z};\tilde{\boldsymbol{y}})$ and $\boldsymbol{y} = \operatorname{argmin}_{\boldsymbol{z}\in\mathcal{V}}\ f(\boldsymbol{z})$. From the definition of $\boldsymbol{y}$, we have $f(\boldsymbol{y}) \leq f(\boldsymbol{y}')$. On the other hand, from the first-order optimality condition, we have $\nabla f(\tilde{\boldsymbol{y}}) = \boldsymbol{0}$. So, we have

$$f(\boldsymbol{y}') - f(\tilde{\boldsymbol{y}}) = B_f(\boldsymbol{y}';\tilde{\boldsymbol{y}}) \leq B_f(\boldsymbol{y};\tilde{\boldsymbol{y}}) = f(\boldsymbol{y}) - f(\tilde{\boldsymbol{y}}),$$

that is $f(\boldsymbol{y}') \leq f(\boldsymbol{y})$. Given that f is strictly convex, $\boldsymbol{y}' = \boldsymbol{y}$. □

Now, note that, if $\tilde{\psi}(\boldsymbol{x}) = \psi(\boldsymbol{x}) + \langle \boldsymbol{g}, \boldsymbol{x}\rangle$, then

$$\begin{aligned} B_{\tilde{\psi}}(\boldsymbol{x};\boldsymbol{y}) &= \tilde{\psi}(\boldsymbol{x}) - \tilde{\psi}(\boldsymbol{y}) - \langle \nabla\tilde{\psi}(\boldsymbol{y}), \boldsymbol{x}-\boldsymbol{y}\rangle \\ &= \psi(\boldsymbol{x}) - \psi(\boldsymbol{y}) - \langle \boldsymbol{g} + \nabla\psi(\boldsymbol{y}), \boldsymbol{x}-\boldsymbol{y}\rangle + \langle \boldsymbol{g}, \boldsymbol{x}-\boldsymbol{y}\rangle \\ &= \psi(\boldsymbol{x}) - \psi(\boldsymbol{y}) - \langle \nabla\psi(\boldsymbol{y}), \boldsymbol{x}-\boldsymbol{y}\rangle = B_\psi(\boldsymbol{x};\boldsymbol{y})\ . \end{aligned}$$

Now, define $f(\boldsymbol{x}) = \langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}\rangle + B_\psi(\boldsymbol{x};\boldsymbol{x}_t)$, so that $f(\boldsymbol{x}) = \psi(\boldsymbol{x}) + \langle \boldsymbol{z}, \boldsymbol{x}\rangle + K$ for some $K \in \mathbb{R}$ and $\boldsymbol{z} \in \mathbb{R}^d$. This implies that $B_f(\boldsymbol{x};\boldsymbol{y}) = B_\psi(\boldsymbol{x};\boldsymbol{y})$. Hence, under the assumption of the above theorem, we have that $\boldsymbol{x}_{t+1} = \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle + \frac{1}{\eta_t} B_\psi(\boldsymbol{x};\boldsymbol{x}_t)$ is equivalent to

$$\begin{aligned} \tilde{\boldsymbol{x}}_{t+1} &= \operatorname*{argmin}_{\boldsymbol{x}\in\mathcal{X}} \langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}\rangle + B_\psi(\boldsymbol{x};\boldsymbol{x}_t), \\ \boldsymbol{x}_{t+1} &= \operatorname*{argmin}_{\boldsymbol{x}\in\mathcal{V}}\ B_\psi(\boldsymbol{x};\tilde{\boldsymbol{x}}_{t+1})\ . \end{aligned}$$

The advantage of this update is that sometimes it gives two easier problems to solve rather than a single difficult one.

6.5 OMD Regret Bound using Local Norms

In Lemma 6.10, strong convexity basically tells us that there is some minimum curvature in all the directions, which allows us to upper bound the difference between $\boldsymbol{x}_t$ and $\boldsymbol{x}_{t+1}$. However, it turns out that we can still get a meaningful regret upper bound without this assumption. In particular, we can get an interesting expression for the regret that involves the use of **local norms**. We will use these ideas with multi-scale experts (Section 6.9) and with multi-armed bandits (Chapter 9).

Lemma 6.32 *Let $\mathcal{X} \subseteq \mathbb{R}^d$ be closed and convex, B_ψ be the Bregman divergence with respect to $\psi : \mathcal{X} \to \mathbb{R}$, and assume ψ twice differentiable and with the Hessian positive definite in* $\operatorname{int}\mathcal{X}$. *Let $\mathcal{V} \subseteq \mathcal{X}$ be a non-empty closed convex set. Assume* (6.5) *or* (6.6) *to hold. Define $\|\boldsymbol{x}\|_{\boldsymbol{A}} := \sqrt{\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}}$. Also, with the notation and initialization in Algorithm 6.1, assume $\boldsymbol{x}_{t+1}$ and $\tilde{\boldsymbol{x}}_{t+1} \in \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{X}} \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle + \frac{1}{\eta_t} B_\psi(\boldsymbol{x};\boldsymbol{x}_t)$ exist. Then, $\forall \boldsymbol{u} \in \mathcal{V}$, there*

exists $\boldsymbol{z}_t$ on the line segment between $\boldsymbol{x}_t$ and $\boldsymbol{x}_{t+1}$, and $\boldsymbol{z}'_t$ on the line segment between $\boldsymbol{x}_t$ and $\tilde{\boldsymbol{x}}_{t+1}$, such that the following inequality holds

$$\eta_t \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \leq B_\psi(\boldsymbol{u}; \boldsymbol{x}_t) - B_\psi(\boldsymbol{u}; \boldsymbol{x}_{t+1}) + \frac{\eta_t^2 \min\left(\|\boldsymbol{g}_t\|^2_{(\nabla^2 \psi(\boldsymbol{z}_t))^{-1}}, \|\boldsymbol{g}_t\|^2_{(\nabla^2 \psi(\boldsymbol{z}'_t))^{-1}} \right)}{2} .$$

Proof First of all, from Theorem 6.8 used with two different feasible sets, $\boldsymbol{x}_{t+1}$ and $\tilde{\boldsymbol{x}}_{t+1}$ are in the interior of $\mathcal{X}$ for all $t \geq 1$. Then, reasoning as in the proof of Lemma 6.10, we have

$$\langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \leq B_\psi(\boldsymbol{u}; \boldsymbol{x}_t) - B_\psi(\boldsymbol{u}; \boldsymbol{x}_{t+1}) - B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) + \langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle . \quad (6.9)$$

From the Taylor's theorem, $B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) = \frac{1}{2}(\boldsymbol{x}_{t+1} - \boldsymbol{x}_t)^\top \nabla^2 \psi(\boldsymbol{z}_t)(\boldsymbol{x}_{t+1} - \boldsymbol{x}_t)$ for some $\boldsymbol{z}_t$ on the line segment between $\boldsymbol{x}_t$ and $\boldsymbol{x}_{t+1}$. Observe that this is $\frac{1}{2}\|\boldsymbol{x}_{t+1} - \boldsymbol{x}_t\|^2_{\nabla^2 \psi(\boldsymbol{z}_t)}$ and it is indeed a norm because we assumed the Hessian of ψ to be positive definite. Hence, by Fenchel–Young's inequality and Examples 4.19 and 6.19, we have

$$\begin{aligned}
&\langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) \\
&\leq \frac{\eta_t^2}{2} \|\boldsymbol{g}_t\|^2_{(\nabla^2 \psi(\boldsymbol{z}_t))^{-1}} + \frac{1}{2}(\boldsymbol{x}_{t+1} - \boldsymbol{x}_t)^\top \nabla^2 \psi(\boldsymbol{z}_t)(\boldsymbol{x}_{t+1} - \boldsymbol{x}_t) - B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) \\
&= \frac{\eta_t^2}{2} \|\boldsymbol{g}_t\|^2_{(\nabla^2 \psi(\boldsymbol{z}_t))^{-1}},
\end{aligned}$$

that gives the first term in the minimum.

For the second term in the minimum, we instead observe that

$$\begin{aligned}
\langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) &\leq \max_{\boldsymbol{x} \in \mathcal{X}} \langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x} \rangle - B_\psi(\boldsymbol{x}; \boldsymbol{x}_t) \\
&= \langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \tilde{\boldsymbol{x}}_{t+1} \rangle - B_\psi(\tilde{\boldsymbol{x}}_{t+1}; \boldsymbol{x}_t) .
\end{aligned}$$

Then, we proceed as in the first bound. □

Despite the apparent more difficult formulation, the second term in the minimum is often easier to use, especially in constrained settings, because $\tilde{\boldsymbol{x}}_{t+1}$ is defined over $\mathcal{X}$ rather than over $\mathcal{V}$. Also, under the assumptions of Theorem 6.31, it is easy to recognize that $\boldsymbol{x}_{t+1}$ is the Bregman projection of $\tilde{\boldsymbol{x}}_{t+1}$ onto $\mathcal{V}$.

6.6 Example of OMD: Exponentiated Gradient

Let $\Delta^{d-1} := \{\boldsymbol{x} \in \mathbb{R}^d : x_i \geq 0, \|\boldsymbol{x}\|_1 = 1\}$ be the **probability simplex** and set $\mathcal{V} = \Delta^{d-1}$. So, in words, we want to output vectors in the probability simplex. Also, let $\mathcal{X} = \mathbb{R}^d_{\geq 0}$ and $\psi(\boldsymbol{x}) : \mathcal{X} \to \mathbb{R}$ defined as $\psi(\boldsymbol{x}) = \sum_{i=1}^d x_i \ln x_i$, where we define $0 \ln(0) = 0$. Note that the restriction of ψ to $\mathcal{V}$ is the negative Shannon entropy of the discrete distributions in Δ^{d-1}. It is possible to verify that ψ satisfies the first condition in Theorem 6.8, hence the update is well defined.

The Fenchel conjugate $\psi^\star_{\mathcal{V}}(\boldsymbol{\theta})$ is defined as

$$\psi^\star_{\mathcal{V}}(\boldsymbol{\theta}) = \sup_{\boldsymbol{x} \in \mathcal{V}} \langle \boldsymbol{\theta}, \boldsymbol{x} \rangle - \psi(\boldsymbol{x}) = \sup_{\boldsymbol{x} \in \mathcal{V}} \langle \boldsymbol{\theta}, \boldsymbol{x} \rangle - \sum_{i=1}^d x_i \ln x_i .$$

It is a constrained optimization problem, we could solve it using the Karush–Kuhn–Tucker conditions. However, there is a simpler way to do it: we will remove the probability simplex constraint, rephrasing the problem over $d-1$ variables. In fact, the maximization problem is equivalent to

$$\min_{\boldsymbol{x}\in\mathbb{R}^{d-1}} \sum_{i=1}^{d-1} x_i \ln x_i + \left(1-\sum_{i=1}^{d-1} x_i\right) \ln\left(1-\sum_{i=1}^{d-1} x_i\right) - \sum_{i=1}^{d-1} \theta_i x_i - \theta_d \left(1-\sum_{i=1}^{d-1} x_i\right) .$$

Note that a constraint on $x_1,\dots,x_{d-1}$ and $1-\sum_{i=1}^{d-1} x_i$ to be non-negative is enforced by the domain of the logarithm. This is now an unconstrained convex optimization problem, so we can solve it by equating the gradient of the objective function to zero. Hence, we have

$$\ln \frac{x_i}{1-\sum_{j=1}^{d-1} x_j} = \theta_i - \theta_d, \quad i = 1,\dots,d-1 .$$

That is

$$x_i = \exp(\theta_i-\theta_d)\left(1-\sum_{j=1}^{d-1} x_j\right), \quad i=1,\dots,d-1 . \tag{6.10}$$

Summing this equality over $i=1,\dots,d-1$, we obtain

$$\sum_{i=1}^{d-1} x_i = \sum_{i=1}^{d-1} \exp(\theta_i-\theta_d)\left(1-\sum_{j=1}^{d-1} x_j\right),$$

that can be solved to obtain

$$1-\sum_{j=1}^{d-1} x_j = \frac{1}{1+\sum_{j=1}^{d-1}\exp(\theta_j-\theta_d)} .$$

Substituting it back in (6.10), we have

$$x_i = \frac{\exp(\theta_i-\theta_d)}{1+\sum_{j=1}^{d-1}\exp(\theta_j-\theta_d)} = \frac{\exp(\theta_i)}{\sum_{j=1}^{d}\exp(\theta_j)}, \quad i=1,\dots,d . \tag{6.11}$$

Denoting with $\alpha = \sum_{i=1}^{d}\exp(\theta_i)$, and substituting in the definition of the conjugate function, we get

$$\begin{aligned}\psi_{\mathcal{V}}^{\star}(\boldsymbol{\theta}) &= \sum_{i=1}^{d}\left(\frac{1}{\alpha}\theta_i\exp(\theta_i) - \frac{1}{\alpha}\exp(\theta_i)(\theta_i-\ln(\alpha))\right) = \ln(\alpha)\frac{1}{\alpha}\sum_{i=1}^{d}\exp(\theta_i) = \ln(\alpha) \\ &= \ln\left(\sum_{i=1}^{d}\exp(\theta_i)\right) .\end{aligned}$$

We also have $(\nabla\psi_{\mathcal{V}}^{\star}(\boldsymbol{\theta}))_j = \frac{\exp(\theta_j)}{\sum_{i=1}^{d}\exp(\theta_i)}$ and $(\nabla\psi(\boldsymbol{x}))_j = \ln(x_j)+1$ for $\boldsymbol{x}\in\mathbb{R}^d_{>0}$. Note that we could have also derived the gradient of $\psi_{\mathcal{V}}^{\star}$ directly from (6.11), using Theorem 6.14.

Putting everything together, we have the online mirror descent update rule for the entropic distance generating function.

$$x_{t+1,j} = \frac{\exp(\ln x_{t,j} + 1 - \eta_t g_{t,j})}{\sum_{i=1}^{d}\exp(\ln x_{t,i} + 1 - \eta_t g_{t,i})} = \frac{x_{t,j}\exp(-\eta_t g_{t,j})}{\sum_{i=1}^{d} x_{t,i}\exp(-\eta_t g_{t,i})} .$$

Algorithm 6.2 Exponentiated Gradient (EG)

Require: $\eta > 0$
1: Set $\boldsymbol{x}_1 = [1/d, \dots, 1/d]$
2: **for** $t = 1$ **to** T **do**
3: Output $\boldsymbol{x}_t \in \Delta^{d-1}$
4: Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable on Δ^{d-1}
5: Set $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$
6: $x_{t+1,j} = \frac{x_{t,j} \exp(-\eta g_{t,j})}{\sum_{i=1}^d x_{t,i} \exp(-\eta g_{t,i})}$, $j = 1, \dots, d$
7: **end for**

The algorithm is summarized in Algorithm 6.2. This algorithm is called **Exponentiated Gradient (EG)** because in the update rule we take the component-wise exponential of the (sub)gradient vector.

Let's take a look at the regret bound we get. From Example 6.6, for all $\boldsymbol{x} \in \Delta^{d-1}$ and $\boldsymbol{y} \in \{\boldsymbol{x} \in \mathbb{R}^d : x_i > 0, \|\boldsymbol{x}\|_1 = 1\}$, we have $\mathrm{KL}(\boldsymbol{x}; \boldsymbol{y}) := B_\psi(\boldsymbol{x}; \boldsymbol{y}) = \sum_{i=1}^d x_i \ln \frac{x_i}{y_i}$, that is the Kullback–Leibler (KL) divergence between the discrete distributions $\boldsymbol{x}$ and $\boldsymbol{y}$. Now, we prove the strong convexity of ψ through a slightly more general statement. Setting $c_i = 1$ for $i = 1, \dots, d$ gives the result we need here.

Lemma 6.33 *Let $c_1, \dots, c_d \in \mathbb{R}_{>0}$. Then, the negative weighted entropy function $\psi(\boldsymbol{x}) = \sum_{i=1}^d c_i x_i \ln x_i$ is 1-strongly convex with respect to the L_1 norm defined over the set $\mathcal{K} = \{\boldsymbol{x} \in \mathbb{R}^d : x_i > 0, \sum_{i=1}^d x_i/c_i = 1\}$.*

Proof Let $\phi(u) = (u-1)\ln u - \frac{2(u-1)^2}{u+1}$ for $u > 0$. Observe that $\phi''(u) > 0$ for $u > 0$ so the function is convex. Moreover, $\phi(1) = \phi'(1) = 0$. So, we have $\phi(u) \geq \phi(1) + \phi'(1)(u-1) = 0$ for all $u > 0$.

Using this inequality, we have

$$\begin{aligned}
&\langle \nabla\psi(\boldsymbol{x}) - \nabla\psi(\boldsymbol{y}), \boldsymbol{x} - \boldsymbol{y} \rangle \\
&= \sum_{i=1}^d c_i (x_i - y_i) \ln \frac{x_i}{y_i} = \sum_{i=1}^d c_i y_i \left(\frac{x_i}{y_i} - 1 \right) \ln \frac{x_i}{y_i} \geq \sum_{i=1}^d c_i \frac{2(x_i - y_i)^2}{x_i + y_i} \\
&= \sum_{i=1}^d \frac{x_i + y_i}{2c_i} \left(\frac{|x_i - y_i|}{\frac{x_i+y_i}{2c_i}} \right)^2 \geq \left(\sum_{i=1}^d \frac{x_i + y_i}{2c_i} \frac{|x_i - y_i|}{\frac{x_i+y_i}{2c_i}} \right)^2 = \|\boldsymbol{x} - \boldsymbol{y}\|_1^2,
\end{aligned}$$

where in the last inequality we used Jensen's inequality (Theorem 2.9) with the distribution $[\frac{x_1+y_1}{2c_1}, \dots, \frac{x_d+y_d}{2c_d}]$. Using Theorem 4.3 completes the proof. □

Another thing to do is to decide the initial point $\boldsymbol{x}_1$. A reasonable choice is to set $\boldsymbol{x}_1$ to be the minimizer of ψ in $\mathcal{V}$. Hence, we set $\boldsymbol{x}_1 = [1/d, \dots, 1/d] \in \mathbb{R}^d$, because the uniform distribution minimizes the negative entropy. Hence, we have that $B_\psi(\boldsymbol{u}; \boldsymbol{x}_1)$ is equal to $\psi(\boldsymbol{u}) - \min_{\boldsymbol{x} \in \mathcal{V}} \psi(\boldsymbol{x})$. So, we have $B_\psi(\boldsymbol{u}; \boldsymbol{x}_1) = \sum_{i=1}^d u_i \ln u_i + \ln d \leq \ln d$.

Putting everything together, we have

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \frac{\ln d}{\eta} + \frac{\eta}{2}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_\infty^2, \quad \forall \boldsymbol{u} \in \Delta^{d-1} .$$

Assuming $\|\boldsymbol{g}_t\|_\infty \leq L_\infty$, we can set $\eta = \sqrt{\frac{2\ln d}{L_\infty^2 T}}$, to obtain an upper bound on the regret of $L_\infty\sqrt{2T\ln d}$.

Remark 6.34 *Note that the time-varying version of OMD with entropic distance generating function would give rise to a vacuous bound. Can you see why? In Chapter 7, we will see how FTRL overcomes this issue using a time-varying regularizer rather than a time-varying learning rate.*

We can also get a *tighter* bound using the local norms. Let's use the additional assumption that $g_{t,i} \geq 0$, for all $t = 1, \ldots, T$ and $i = 1, \ldots, d$. Summing the inequality of Lemma 6.32 from $t = 1$ to T, we have for all $\boldsymbol{u} \in \mathcal{V}$ that

$$\sum_{t=1}^{T}\ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T}\ell_t(\boldsymbol{u}) \leq \frac{\ln d}{\eta} + \frac{\eta}{2}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|^2_{(\nabla^2\psi(\boldsymbol{z}'_t))^{-1}},$$

where $\boldsymbol{z}'_t$ is on the line segment between $\boldsymbol{x}_t$ and $\tilde{\boldsymbol{x}}_{t+1}$. In this case, it is easy to calculate $\tilde{x}_{t+1,i}$ as $x_{t,i}\exp(-\eta g_{t,i})$ for $i = 1, \ldots, d$. Moreover, $\nabla^2\psi(\boldsymbol{z}'_t)$ is a diagonal matrix whose elements on the diagonal are $\frac{1}{z'_{t,i}}, i = 1, \ldots, d$. Hence, we have that

$$\|\boldsymbol{g}_t\|^2_{(\nabla^2\psi(\boldsymbol{z}'_t))^{-1}} = \sum_{i=1}^{d} g_{t,i}^2 z'_{t,i} \leq \sum_{i=1}^{d} g_{t,i}^2 x_{t,i} .$$

Putting everything together, the final bound would be

$$\sum_{t=1}^{T}\ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T}\ell_t(\boldsymbol{u}) \leq \frac{\ln d}{\eta} + \frac{\eta}{2}\sum_{t=1}^{T}\sum_{i=1}^{d} g_{t,i}^2 x_{t,i}, \quad \forall \boldsymbol{u} \in \Delta^{d-1} . \tag{6.12}$$

This is indeed a tighter bound because $\sum_{i=1}^{d} g_{t,i}^2 x_{t,i} \leq \|\boldsymbol{g}_t\|_\infty^2$.

One can also weaken the assumption on the subgradients at the cost of a slight increase in the constant in front of them. In fact, for example, assume that $\|\boldsymbol{g}_t\|_\infty \leq L_\infty$ for all $t = 1, \ldots, T$ and $\eta \leq \frac{1}{L_\infty}$. In this case, we can upper bound $\tilde{x}_{t+1,i}$ as $x_{t,i}\exp(-\eta g_{t,i}) \leq e x_{t,i}$. Hence, the final bound in this case is

$$\sum_{t=1}^{T}\ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T}\ell_t(\boldsymbol{u}) \leq \frac{\ln d}{\eta} + \frac{e\eta}{2}\sum_{t=1}^{T}\sum_{i=1}^{d} g_{t,i}^2 x_{t,i}, \quad \forall \boldsymbol{u} \in \Delta^{d-1} .$$

How would OSD work on the same problem? First, it is important to realize that nothing prevents us from using OSD on this problem. We just have to implement the Euclidean projection onto the probability simplex, which does not have a closed form, but it can be implemented in $\mathcal{O}(d\ln d)$ time, see Condat (2016). The regret bound we would get from

OSD is

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t)-\ell_t(\boldsymbol{u}))\leq\frac{2}{\eta}+\frac{\eta}{2}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_2^2\leq\frac{2}{\eta}+\frac{\eta}{2}TL_2^2,\quad\forall\boldsymbol{u}\in\Delta^{d-1},$$

where $L_2\geq\|\boldsymbol{g}_t\|_2$ for all t. Optimally tuning the learning rate, we get

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t)-\ell_t(\boldsymbol{u}))\leq 2L_2\sqrt{T},\quad\forall\boldsymbol{u}\in\Delta^{d-1}\,.$$

To compare it with the bound of EG, it is sufficient to observe that

$$\frac{1}{\sqrt{d}}\|\boldsymbol{g}_t\|_2\leq\|\boldsymbol{g}_t\|_\infty\leq\|\boldsymbol{g}_t\|_2,$$

and the inequalities are tight. Hence, ignoring the numerical constants, the bound of EG is between $\sqrt{\frac{\ln d}{d}}$ and $\sqrt{\ln d}$ times the one of OSD, depending on the structure of the subgradients $\boldsymbol{g}_t$. Hence, in a worst-case sense with respect to the infinity norm of the subgradients $\boldsymbol{g}_t$, using an entropic distance generating function can transform a dependence on the dimension from $\sqrt{d}$ to $\sqrt{\ln d}$ for Online Convex Optimization (OCO) over the probability simplex. So, as we already saw analyzing AdaGrad (Section 4.2.4), the shape of the domain and the structure of the subgradients are the important ingredients when we move from the Euclidean norm to other norms.

6.7 Example of OMD: p-norm Algorithms

We now present a class of algorithms that interpolates between the regret guarantee of OSD and that of EG.

Consider the distance generating function $\psi(\boldsymbol{x})=\frac{1}{2}\|\boldsymbol{x}\|_p^2$, for $1<p\leq 2$ over $\mathcal{X}=\mathcal{V}=\mathbb{R}^d$. Let's remind the reader that the p-norm of a vector $\boldsymbol{x}$ is defined as $(\sum_{i=1}^{d}|x_i|^p)^{1/p}$. From Examples 4.18 and 6.19, we have that $\psi_{\mathcal{V}}^{\star}(\boldsymbol{\theta})=\frac{1}{2}\|\boldsymbol{\theta}\|_q^2$, where $\frac{1}{p}+\frac{1}{q}=1$, so that $q\geq 2$. Let's calculate the dual maps: $(\nabla\psi(\boldsymbol{x}))_j=\operatorname{sign}(x_j)|x_j|^{p-1}\|\boldsymbol{x}\|_p^{2-p}$ and, for $\boldsymbol{x}\neq\boldsymbol{0}$, $(\nabla\psi_{\mathcal{V}}^{\star}(\boldsymbol{x}))_j=\operatorname{sign}(x_j)|x_j|^{q-1}\|\boldsymbol{x}\|_q^{2-q}$, while $\nabla\psi_{\mathcal{V}}^{\star}(\boldsymbol{0})=\boldsymbol{0}$. Hence, we can write the update rule as

$$\begin{aligned}
&\tilde{x}_{t+1,j}=\operatorname{sign}(x_{t,j})|x_{t,j}|^{p-1}\|\boldsymbol{x}_t\|_p^{2-p}-\eta g_{t,j},\quad j=1,\dots,d,\\
&x_{t+1,j}=\begin{cases}\operatorname{sign}(\tilde{x}_{t+1,j})|\tilde{x}_{t+1,j}|^{q-1}\,\|\tilde{\boldsymbol{x}}_{t+1}\|_q^{2-q}\,, & \tilde{\boldsymbol{x}}_{t+1}\neq\boldsymbol{0},\\ 0, & \tilde{\boldsymbol{x}}_{t+1}=\boldsymbol{0},\end{cases}\quad j=1,\dots,d,
\end{aligned}$$

where we broke the update into two steps to simplify the notation (and the implementation). Starting from $\boldsymbol{x}_1=\boldsymbol{0}$, we have that

$$B_\psi(\boldsymbol{u};\boldsymbol{x}_1)=\psi(\boldsymbol{u})-\psi(\boldsymbol{x}_1)-\langle\nabla\psi(\boldsymbol{x}_1),\boldsymbol{u}-\boldsymbol{x}_1\rangle=\psi(\boldsymbol{u})\,.$$

The last ingredient is the fact that $\psi(\boldsymbol{x})$ is $p-1$ strongly convex with respect to $\|\cdot\|_p$.

Lemma 6.35 *$\psi(\boldsymbol{x})=\frac{1}{2}\|\boldsymbol{x}\|_p^2$ is $(p-1)$-strongly convex with respect to $\|\cdot\|_p$, for $1\leq p\leq 2$.*

Proof Observe that for $p = 1$ the function is convex, hence it is 0-strongly convex with respect to any norm. So, we now assume $p > 1$.

We want to show that

$$\psi(\boldsymbol{y}) \geq \psi(\boldsymbol{x}) + \langle \nabla\psi(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x}\rangle + \frac{p-1}{2}\|\boldsymbol{x} - \boldsymbol{y}\|_p^2, \quad \forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d\,. \tag{6.13}$$

Rather than studying ψ directly, we study the Hessian of a surrogate function: $\psi_a(\boldsymbol{x}) = \frac{1}{2}\left(\sum_{i=1}^d (x_i^2 + a)^{\frac{p}{2}}\right)^{2/p}$ where $a > 0$. Observe that ψ_a is continuously twice differentiable for $a > 0$.

Define the auxiliary functions $f : \mathbb{R} \to \mathbb{R}$ convex and twice differentiable and $h : \mathbb{R}^d \to \mathbb{R}$ differentiable, and define $\phi(\boldsymbol{x}) = f(h(\boldsymbol{x}))$. So, we have

$$\nabla^2\phi(\boldsymbol{x}) = f''(h(\boldsymbol{x}))\nabla h(\boldsymbol{x})\nabla h(\boldsymbol{x})^\top + f'(h(\boldsymbol{x}))\nabla^2 h(\boldsymbol{x}) \succeq f'(h(\boldsymbol{x}))\nabla^2 h(\boldsymbol{x}),$$

where in the inequality we used the fact that f is convex, so $f''(x) \geq 0$ for all $x \in \mathbb{R}$.

Now, let's specialize the above result to $h(\boldsymbol{x}) = \sum_{i=1}^d (x_i^2 + a)^{\frac{p}{2}}$ and $f(x) = \frac{1}{2}x^{\frac{2}{p}}$. So, $\nabla^2 h(\boldsymbol{x})$ is a diagonal matrix and the element i on the diagonal is

$$\begin{aligned} p(x_i^2 + a)^{\frac{p}{2}-1} + p(p-2)x_i^2(x_i^2 + a)^{\frac{p}{2}-2} &= p(x_i^2 + a)^{\frac{p}{2}-1}(1 + (p-2)x_i^2(x_i^2 + a)^{-1}) \\ &\geq p(x_i^2 + a)^{\frac{p}{2}-1}(1 + (p-2)) \\ &= p(p-1)(x_i^2 + a)^{\frac{p}{2}-1}, \end{aligned}$$

where in the inequality we have used the fact that $a > 0$ and $p \leq 2$. Hence, we have

$$\nabla^2\psi_a(\boldsymbol{x}) \succeq \frac{1}{p}\left(\sum_{i=1}^d (x_i^2 + a)^{\frac{p}{2}}\right)^{\frac{2}{p}-1} p(p-1)\mathrm{diag}\left\{(x_1^2 + a)^{\frac{p}{2}-1}, \ldots, (x_d^2 + a)^{\frac{p}{2}-1}\right\}\,.$$

Hence, denoting $w_i = (x_i^2 + a)^{(2-p)\frac{p}{4}} > 0$, we have

$$\begin{aligned} \langle \nabla^2\psi_a(\boldsymbol{x})\boldsymbol{y}, \boldsymbol{y}\rangle &\geq (p-1)\left(\sum_{i=1}^d (x_i^2 + a)^{\frac{p}{2}}\right)^{\frac{2-p}{p}} \sum_{i=1}^d (x_i^2 + a)^{\frac{p}{2}-1} y_i^2 \\ &= (p-1)\left[\left(\sum_{i=1}^d (x_i^2 + a)^{\frac{p}{2}}\right)^{\frac{2-p}{2}} \left(\sum_{i=1}^d (x_i^2 + a)^{\frac{p}{2}-1} y_i^2\right)^{\frac{p}{2}}\right]^{\frac{2}{p}} \\ &= (p-1)\left[\left(\sum_{i=1}^d w_i^{\frac{2}{2-p}}\right)^{\frac{2-p}{2}} \left(\sum_{i=1}^d \frac{y_i^2}{w_i^{\frac{2}{p}}}\right)^{\frac{p}{2}}\right]^{\frac{2}{p}} \\ &\geq (p-1)\left(\sum_{i=1}^d w_i \frac{y_i^p}{w_i}\right)^{\frac{2}{p}} = (p-1)\|\boldsymbol{y}\|_p^2, \end{aligned}$$

where we used Hölder inequality with dual norms $\|\cdot\|_{\frac{2}{p}}$ and $\|\cdot\|_{\frac{2}{2-p}}$.

Hence, we have that

$$\psi_a(\boldsymbol{y}) \geq \psi_a(\boldsymbol{x}) + \langle \nabla\psi_a(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x}\rangle + \frac{p-1}{2}\|\boldsymbol{x} - \boldsymbol{y}\|_p^2, \quad \forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d\,.$$

Now, given that ψ_a and $\nabla\psi_a$ are continuous in a, taking the limit for $a \to 0^+$, we get (6.13). By Lemma 4.2, this implies the strong convexity of ψ. □

Hence, the regret bound will be

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \frac{\|\boldsymbol{u}\|_p^2}{2\eta} + \frac{\eta}{2(p-1)}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_q^2 \,.$$

Setting $p = 2$, we get the (unprojected) OSD. However, we can set p to achieve a logarithmic dependence in the dimension d as in EG. Let's assume again that $\|\boldsymbol{g}_t\|_\infty \leq L_\infty$, so we have $\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_q^2 \leq L_\infty^2 d^{2/q} T$. Also, note that $\|\boldsymbol{u}\|_p \leq \|\boldsymbol{u}\|_1$, so we have an upper bound on the regret of

$$\mathrm{Regret}_T(\boldsymbol{u}) \leq \frac{\|\boldsymbol{u}\|_1^2}{2\eta} + \frac{L_\infty^2 d^{2/q} T\eta}{2(p-1)}, \quad \forall \boldsymbol{u} \in \mathbb{R}^d \,.$$

Setting $\eta = \frac{\alpha\sqrt{p-1}}{L_\infty d^{1/q}\sqrt{T}}$, we get an upper bound on the regret of

$$\begin{aligned}\frac{1}{2}\left(\frac{\|\boldsymbol{u}\|_1^2}{\alpha} + \alpha\right) L_\infty\sqrt{T}\frac{d^{1/q}}{\sqrt{p-1}} &= \frac{1}{2}\left(\frac{\|\boldsymbol{u}\|_1^2}{\alpha} + \alpha\right) L_\infty\sqrt{T}\sqrt{q-1}d^{1/q} \\ &\leq \frac{1}{2}\left(\frac{\|\boldsymbol{u}\|_1^2}{\alpha} + \alpha\right) L_\infty\sqrt{T}\sqrt{q}d^{1/q} \,.\end{aligned}$$

Assuming $d \geq 3$, the choice of q that minimizes the last term is $q = 2\ln d$ that makes the term $\sqrt{q}d^{1/q} = \sqrt{2e\ln d}$. Hence, we have regret bound of $\mathcal{O}(\sqrt{T\ln d})$ as $T \to \infty$.

Note that here the set $\mathcal{V}$ is the entire space, however we could still set $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : x_i \geq 0, \|\boldsymbol{x}\|_1 = 1\}$. While this would allow us to get the same asymptotic bound of EG, the update would not be in a closed form anymore.

6.8 Application: Learning with Expert Advice

Let's introduce a particular OCO game called **Learning with Expert Advice (LEA)**.

In this setting, on each round, we have d experts who give us some advice, and we have to decide which expert we want to follow. After we made our choice, the losses associated with each expert are revealed, and we pay the loss associated with the expert we picked. The aim of the game is to minimize the losses we suffer compared to the cumulative losses of the best expert. This is a general setting that allows us to model many interesting cases. For example, we have a number of different online learning algorithms, and we would like to choose the best among them.

Is this problem solvable? If we put ourselves in the adversarial setting, unfortunately, it cannot be solved! Indeed, even with 2 experts, the adversary can force the algorithm to incur linear regret. Let's see how. In each round, we have to pick expert 1 or expert 2. In each round, the adversary can decide that the expert we pick has a loss of 1 and the other one has a loss of 0. This means that the cumulative loss of the algorithm over T rounds is T. On the other hand, the best cumulative loss over experts 1 and 2 is less than $T/2$. This means that our regret, no matter what we do, can be as big as $T/2$.

The problem above is due to the fact that the adversary has too much power. One way to

Algorithm 6.3 Learning with Expert Advice through Randomization

Require: $\boldsymbol{x}_1 \in \Delta^{d-1}$
1: **for** $t = 1$ **to** T **do**
2: Draw A_t according to $\mathbb{P}\{A_t = i\} = x_{t,i}$
3: Select expert A_t
4: Observe all the experts' losses $\boldsymbol{g}_t \in \mathbb{R}^d$ and pay the loss g_{t,A_t}
5: Get $\boldsymbol{x}_{t+1}$ from an OCO algorithm with feasible set Δ^{d-1}
6: **end for**

reduce its power is by using *randomization*. We can allow the algorithm to be randomized *and* force the adversary to decide the losses at time t without knowing the outcome of the randomization of the algorithm at time t (but it can depend on the past randomization). This is enough to make the problem solvable. Another view to look at it is that randomization makes the problem convex, allowing us to use any OCO algorithm on it.

First, let's write the problem in the original formulation. We set a discrete feasible set $\mathcal{V} = \{\boldsymbol{e}_i\}_{i=1}^d$, where $\boldsymbol{e}_i$ is the vector with all zeros but a 1 in the coordinate i. Our predictions and the competitor are from $\mathcal{V}$. The losses are linear losses: $\ell_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$, for $t = 1, \dots, T$. The regret is

$$\mathrm{Regret}_T(\boldsymbol{e}_i) = \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{e}_i \rangle, \quad i = 1, \dots, d\,. \tag{6.14}$$

The only thing that makes this problem non-convex is the feasibility set, which is clearly a non-convex one.

Let's now see how the randomization makes this problem convex. Let's extend the feasible set to $\Delta^{d-1} := \{\boldsymbol{x} \in \mathbb{R}^d : x_i \geq 0, \|\boldsymbol{x}\|_1 = 1\}$. Note that $\boldsymbol{e}_i \in \Delta^{d-1}$. For this problem, we can use an OCO algorithm to minimize the regret

$$\mathrm{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle, \quad \forall \boldsymbol{u} \in \Delta^{d-1}\,.$$

Can we find a way to transform an upper bound on this regret to the one we care about in (6.14)? One way is the following one: on each time step, construct a random variable A_t that is equal to i with probability $x_{t,i}$ for $i = 1, \dots, d$. Then, select the expert according to the outcome of A_t. Now, using the law of total expectation, we have

$$\mathbb{E}[g_{t,A_t}] = \mathbb{E}_{A_1,\dots,A_{t-1}}[\mathbb{E}_{A_t}[g_{t,A_t} | A_1, \dots, A_{t-1}]] = \mathbb{E}[\langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle],$$

and

$$\mathbb{E}\left[\sum_{t=1}^T (g_{t,A_t} - g_{t,i})\right] = \mathbb{E}\left[\sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{e}_i \rangle\right], \quad \forall i \in \{1, \dots, d\}\,.$$

This means that we can minimize in expectation the non-convex regret with a randomized OCO algorithm. We can summarize this reasoning in Algorithm 6.3.

For example, assume that $\|\boldsymbol{g}_t\|_\infty \leq L_\infty$ for all t. Then, using the EG algorithm from

Section 6.6, we obtain the following update rule

$$x_{t+1,j} = \frac{x_{t,j}\exp(-\eta g_{t,j})}{\sum_{i=1}^{d} x_{t,i}\exp(-\eta g_{t,i})}, \quad j = 1,\dots,d,$$

setting $\boldsymbol{x}_1 = [1/d,\dots,1/d]$ and $\eta = \frac{\sqrt{2\ln d}}{L_\infty\sqrt{T}}$. For such an algorithm, the regret will be

$$\mathbb{E}[\mathrm{Regret}_T(\boldsymbol{e}_i)] \le L_\infty\sqrt{2T\ln d}, \quad \forall i \in \{1,\dots,d\}\ .$$

It is worth stressing the importance of the result just obtained: we can design an algorithm that in expectation, is close to the best expert in a set, *paying only a logarithmic penalty in the size of the set*. The p-norm Algorithm in Section 6.7 would give a similar guarantee.

Later, in Section 13.7, we will see algorithms that achieve the even better regret guarantee of $\mathcal{O}(\sqrt{T\cdot \mathrm{KL}(\boldsymbol{u};\boldsymbol{x}_1)})$ as $T\to\infty$, for any $\boldsymbol{u}$ in the probability simplex. You should be able to convince yourself that no setting of η in EG allows us to achieve such a regret guarantee. Indeed, these algorithms will be based on a very different strategy.

6.9 Example of OMD: Multi-Scale Expert Algorithm

We now consider an application of OMD to the problem of prediction with multi-scale expert advice. In this setting, at each time step t, an algorithm selects a probability distribution $\boldsymbol{x}_t$ over a set of d experts (or actions). After observing a loss vector $\boldsymbol{g}_t \in \mathbb{R}^d$, the algorithm incurs a loss of $\langle \boldsymbol{g}_t, \boldsymbol{x}_t\rangle$. The goal is to minimize the regret with respect to the best single expert in hindsight. However, unlike in the usual learning with expert advice setting, here the losses associated with different experts can have vastly different scales. For example, one expert might correspond to a low-risk, low-reward strategy with losses in $[-0.1, 0.1]$, while another might be a high-risk, high-reward strategy with losses in $[-100, 100]$. Thus, we would like to prove a regret guarantee that depends on the range of the best coordinate.

One might be tempted to use the EG algorithm, but it has a regret bound that scales with the largest range of the loss coordinates, so it is not suitable for this problem. We need a different algorithm.

Here, we assume that we have prior knowledge of the approximate scale of losses for each expert. Specifically, for each expert $i \in \{1,\dots,d\}$, we know a value $c_i > 0$ such that the loss $g_{t,i}$ is guaranteed to be in the range $[-c_i, c_i]$ for all t. The goal is to leverage this scale information to achieve better regret bounds.

Here, we derive an algorithm for this setting as an instance of OMD with a specific choice of surrogate losses. In particular, we run OMD with distance generating function $\psi(\boldsymbol{x}) = \sum_{i=1}^{d} c_i x_i \ln x_i$ on the shifted losses $\tilde{g}_{t,i} = g_{t,i} + \eta\frac{g_{t,i}^2}{c_i}$, see Algorithm 6.4.

Theorem 6.36 *Let $c_1,\dots,c_d > 0$ such that $|g_{t,i}| \le c_i$ for all $t = 1,\dots T$ and $i = 1,\dots d$. In Algorithm 6.4, set $\eta \le \frac{1}{5}$. Then, we have*

$$\sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle \le \frac{B_\psi(\boldsymbol{u};\boldsymbol{x}_1)}{\eta} + \eta\sum_{t=1}^{T}\sum_{i=1}^{d}\frac{u_i g_{t,i}^2}{c_i}, \quad \forall \boldsymbol{u}\in\Delta^{d-1}\ .$$

Proof First, observe that

$$|\tilde{g}_{t,i}| \le |g_{t,i}|\,(1+\eta)\ . \tag{6.15}$$

Algorithm 6.4 OMD for Multi-Scale Experts

Require: Per-expert scales $c_1, \ldots, c_d > 0$, learning rate $\eta > 0$, $\boldsymbol{x}_1 \in \Delta^{d-1}$
1: Set $\psi(\boldsymbol{x}) = \sum_{i=1}^{d} c_i x_i \ln x_i$
2: **for** $t = 1$ **to** T **do**
3: Output $\boldsymbol{x}_t \in \Delta^{d-1}$
4: Receive $\boldsymbol{g}_t \in \mathbb{R}^d$ where $|g_{t,i}| \leq c_i$ for $i = 1, \ldots, d$
5: Pay the loss $\langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle$
6: Set $\tilde{g}_{t,i} = g_{t,i} + \eta \frac{g_{t,i}^2}{c_i}$ for $i = 1, \ldots, d$
7: Set $\boldsymbol{x}_{t+1} = \operatorname{argmin}_{\boldsymbol{x} \in \Delta^{d-1}} B_\psi(\boldsymbol{x}; \boldsymbol{x}_t) + \eta \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x} \rangle$
8: **end for**

Using the local norm regret upper bound for OMD in Lemma 6.32, we obtain

$$\sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \leq \frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_1)}{\eta} + \frac{\eta}{2} \sum_{t=1}^{T} \sum_{i=1}^{d} \frac{z_{t,i} \tilde{g}_{t,i}^2}{c_i}, \quad \forall \boldsymbol{u} \in \Delta^{d-1},$$

where $\boldsymbol{z}_t$ is between $\boldsymbol{x}_t$ and $\boldsymbol{x}'_{t+1} = \operatorname{argmin}_{\boldsymbol{x} \in \mathbb{R}^d_{\geq 0}} B_\psi(\boldsymbol{x}; \boldsymbol{x}_t) + \eta \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x} \rangle$. Using (6.15), we have that

$$x'_{t+1,i} = x_{t,i} \exp\left(-\eta \frac{\tilde{g}_{t,i}}{c_i}\right) \leq x_{t,i} \exp\left(\eta + \eta^2\right) .$$

So, we can upper bound each $z_{t,i}$ with $\exp\left(\eta + \eta^2\right) x_{t,i}$.

Using again (6.15), we have that the right-hand side of the regret upper bound becomes

$$\frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_1)}{\eta} + \frac{1}{2} \exp\left(\eta + \eta^2\right) (1 + \eta)^2 \, \eta \sum_{t=1}^{T} \sum_{i=1}^{d} \frac{x_{t,i} g_{t,i}^2}{c_i} .$$

Observe that $\eta \leq \frac{1}{5}$, so $\frac{1}{2} \exp\left(\eta + \eta^2\right) (1 + \eta)^2 \leq 1$. Hence, for all $\boldsymbol{u} \in \Delta^{d-1}$, we obtain

$$\begin{aligned} \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle + \sum_{t=1}^{T} \sum_{i=1}^{d} \eta \frac{g_{t,i}^2}{c_i} (x_{t,i} - u_i) &= \sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \\ &\leq \frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_1)}{\eta} + \eta \sum_{t=1}^{T} \sum_{i=1}^{d} \frac{x_{t,i} g_{t,i}^2}{c_i} . \end{aligned}$$

Simplifying, we have the stated result. □

Let's now see how we can choose the prior π and the initial point $\boldsymbol{x}_1 \in \Delta^{d-1}$. First, let's calculate the value of the Bregman divergence:

$$\begin{aligned} B_\psi(\boldsymbol{u}; \boldsymbol{x}_1) &= \sum_{i=1}^{d} c_i u_i \ln u_i - \sum_{i=1}^{d} c_i x_{1,i} \ln x_{1,i} - \sum_{i=1}^{d} c_i (\ln x_{1,i} + 1)(u_i - x_{1,i}) \\ &= \sum_{i=1}^{d} c_i u_i \ln \frac{u_i}{x_{1,i}} - \sum_{i=1}^{d} c_i u_i + \sum_{i=1}^{d} c_i x_{1,i} . \end{aligned}$$

Define $c_{\min} = \min_i \ c_i$, $c_{\max} = \max_i \ c_i$, and $i_{\min}$ to be any index in $\operatorname{argmin}_i \ c_i$. If we set $\boldsymbol{u} = \boldsymbol{e}_j$, we have

$$B_\psi(\boldsymbol{u}; \boldsymbol{x}_1) = c_j \ln \frac{1}{x_{1,j}} + \sum_{i=1}^d c_i x_{1,i} - c_j \ .$$

Now, set $x_{1,i} = \frac{c_{\min}}{c_i}\pi_i$ for $i \neq i_{\min}$ and $x_{1,i_{\min}} = 1 - \sum_{i \neq i_{\min}} x_{1,i}$. Also, set $\pi_i = \frac{c_i}{\sum_{j=1}^d c_j}$. In this way, for $j \neq i_{\min}$, we have

$$\begin{aligned}
B_\psi(\boldsymbol{u}; \boldsymbol{x}_1) &= c_j \ln \frac{1}{x_{1,j}} + \sum_{i=1}^d c_i x_{1,i} - c_j \\
&= c_j \ln \frac{c_j}{c_{\min}\pi_j} + c_{\min}(1 - \pi_{i_{\min}}) + c_{\min} x_{1,i_{\min}} - c_j \\
&\leq c_j \ln \frac{c_j}{c_{\min}\pi_j} + 2c_{\min} - c_j \\
&\leq c_j \ln \frac{c_j}{c_{\min}\pi_j} + c_j = c_j \ln \frac{\sum_{k=1}^d c_k}{c_{\min}} + c_j \ .
\end{aligned}$$

On the other hand, for $j = i_{\min}$, we have

$$x_{1,i_{\min}} = 1 - \sum_{i \neq i_{\min}} x_{1,i} \geq 1 - \sum_{i \neq i_{\min}} \pi_i = \pi_{i_{\min}} = \frac{c_{\min}}{\sum_{k=1}^d c_k} \ .$$

Hence, the previous bound holds for all j. In this case, choosing $\eta = \frac{1}{5 + \frac{\sqrt{T}}{\sqrt{1 + \ln \frac{\sum_{i=1}^d c_i}{c_{\min}}}}} \leq \frac{1}{5}$, for all $j = 1, \dots, d$, we have

$$\sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{e}_j \rangle \leq 2c_j \sqrt{T\left(1 + \ln \frac{\sum_{i=1}^d c_i}{c_{\min}}\right)} + 5c_j \left(1 + \ln \frac{\sum_{i=1}^d c_i}{c_{\min}}\right) \ .$$

Note that if $c_1 = c_2 = \cdots = c_d$ then $\ln \frac{\sum_{i=1}^d c_i}{c_{\min}} = \ln d$, so we recover the bound of the EG algorithm, up to constant factors. On the other hand, if the scales c_i are different, then the bounds depend on the range of the expert with whom we compete and only logarithmically on the ratio between the largest and smallest scale.

6.10 Application: Combining Online Algorithms to Adapt to the Learning Rate

We now show an interesting application of the multi-scale expert algorithm.

In our analysis of OSD, we have seen that the choice of the learning rate η_t is critical for performance. For instance, with a constant learning rate η, the optimal choice that minimizes the regret bound is $\eta^\star = \frac{\|\boldsymbol{u} - \boldsymbol{x}_1\|_2}{\sqrt{\sum_{t=1}^T \|\boldsymbol{g}_t\|_2^2}}$, which unfortunately depends on the competitor $\boldsymbol{u}$ and the entire sequence of future gradients. One might be tempted to use a grid of learning rates and select the best one in hindsight, but unfortunately, this is not a valid online learning procedure.

In this section, we demonstrate how to use the LEA framework to design a meta-algorithm

Algorithm 6.5 Combining OSD Instances with a Multi-Scale Expert Algorithm

Require: Number of OSD instances N, number of rounds T, sequence of learning rates $\eta_t^{(1)}, \dots, \eta_t^{(N)}$, initial radius R

Initialize N copies of OSD: $\boldsymbol{x}_1^{(i)} = \mathbf{0}$ for $i = 1, \dots, N$
Set $\mathcal{V}^{(i)} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \le R2^{i-1}\}$
Set $c_i = LR2^{i-1}$ for $i = 1, \dots, N$
Set the prior in the Multi-scale Expert algorithm: $\pi_i = \frac{c_i}{\sum_{j=1}^N c_j}$ for $i = 1, \dots, N$
Initialize the Multi-scale Expert algorithm: $p_{1,i} = \frac{c_1}{c_i}\pi_i$ for $i = 2, \dots, N$ and $p_{1,1} = 1 - \sum_{j=2}^N p_{1,j}$
for $t = 1$ **to** T **do**
  Get $\boldsymbol{p}_t \in \Delta^{N-1}$ from the Multi-scale Expert algorithm
  Output combined prediction $\boldsymbol{x}_t = \sum_{i=1}^N p_{t,i}\boldsymbol{x}_t^{(i)}$
  Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable on $\mathbb{R}^d$
  Set $\boldsymbol{g}_t \in \partial\ell_t(\boldsymbol{x}_t)$
  Define loss vector: $z_{t,i} = \langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(i)} \rangle$ for $i = 1, \dots, N$
  Pass $\boldsymbol{z}_t$ to the Multi-scale Expert algorithm
  Update each OSD instance: $\boldsymbol{x}_{t+1}^{(i)} = \Pi_{\mathcal{V}^{(i)}}(\boldsymbol{x}_t^{(i)} - \eta_t^{(i)}\boldsymbol{g}_t)$ for $i = 1, \dots, N$
end for

that automatically adapts to the best learning rate from a given set, paying only a small price in the regret. The core idea is to treat each instance of an online learning algorithm with a fixed learning rate as an "expert". We then use a multi-scale expert algorithm to combine the predictions of these experts. The resulting ensemble algorithm will have a regret guarantee that is close to the regret of the best expert—and thus the best learning rate—in hindsight.

Let us consider running N parallel instances of the OSD algorithm, each with a different learning rate $\eta^{(i)}$ for $i = 1, \dots, N$. At each round t, each OSD instance i produces a prediction $\boldsymbol{x}_t^{(i)}$. We can view these N predictions as advice from N experts. Our goal is to combine them into a single prediction $\boldsymbol{x}_t$ that performs nearly as well as the best prediction $\boldsymbol{x}_t^{(i^\star)}$ from the best OSD instance $i^\star$.

A straightforward approach would be to compute the loss $\ell_t(\boldsymbol{x}_t^{(i)})$ for each expert i and use this as the loss vector for the multi-scale expert algorithm. However, this would require computing N separate subgradients $\boldsymbol{g}_t^{(i)} \in \partial\ell_t(\boldsymbol{x}_t^{(i)})$ at each round, which can be computationally expensive.

To create an efficient algorithm that requires only one subgradient evaluation per round, we can use the linearization technique. The controller algorithm forms its combined prediction $\boldsymbol{x}_t = \sum_{i=1}^N p_{t,i}\boldsymbol{x}_t^{(i)}$. We then receive a single subgradient $\boldsymbol{g}_t \in \partial\ell_t(\boldsymbol{x}_t)$ at this combined point. This single subgradient is then used to define a linear surrogate loss, $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$, which is passed to all expert algorithms and to the controller algorithm. Because all experts receive the same linear loss function, they all use the same subgradient $\boldsymbol{g}_t$ for their updates. The loss for the i-th expert, used by the multi-scale expert algorithm, is simply $\langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(i)} \rangle$. This procedure is summarized in Algorithm 6.5.

We can now prove a regret bound for this ensemble algorithm.

Theorem 6.37 *Let $R > 0$ and fix T, the number of rounds. With the notation in Algorithm 6.5, assume that the losses ℓ_t are L-Lipschitz with respect to $\|\cdot\|_2$. Let $\eta_t^{(i)} = \frac{R2^{i-1}}{L\sqrt{T}}$ be the set of learning rates for the OSD experts, $N = \lceil \log_2 \sqrt{T} \rceil + 1$, and the learning rate of the multi-scale expert algorithm in Algorithm 6.4 to $\alpha = \frac{\sqrt{1+\ln(2^N-1)}}{5\sqrt{1+\ln(2^N-1)}+\sqrt{T}}$. Then, for all $\boldsymbol{u}$ such that $\|\boldsymbol{u}\|_2 \leq R\sqrt{T}$, Algorithm 6.5 satisfies*

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t)-\ell_t(\boldsymbol{u})) \leq \max(2\|\boldsymbol{u}\|_2, R)L\left[\sqrt{T}\left(1+2\sqrt{3+\ln\sqrt{T}}\right)+15+5\ln\sqrt{T}\right].$$

Proof The choice of N ensures that for any $\boldsymbol{u}$ such that $\|\boldsymbol{u}\|_2 \leq R\sqrt{T}$ there exists an expert $j_{\boldsymbol{u}} \in \{1,\dots,N\}$ such that $\|\boldsymbol{u}\|_2 \leq R2^{j_{\boldsymbol{u}}-1} \leq \max(2\|\boldsymbol{u}\|_2, R)$. Indeed, if $\|\boldsymbol{u}\|_2 \leq R$ we take $j_{\boldsymbol{u}} = 1$, while otherwise we take $j_{\boldsymbol{u}} = \left\lceil \log_2 \frac{\|\boldsymbol{u}\|_2}{R} \right\rceil + 1$.

By the definition of subgradient, the regret of the ensemble algorithm can be bounded by the regret on the linearized losses:

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle .$$

We can now decompose the regret as

$$\sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle = \sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_t^{(j_{\boldsymbol{u}})}\rangle + \sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(j_{\boldsymbol{u}})} - \boldsymbol{u}\rangle . \tag{6.16}$$

The second sum in r.h.s. of (6.16) is the regret of the best OSD expert on the sequence of linear losses $\langle \boldsymbol{g}_t, \cdot\rangle$. From the regret bound of OSD with constant learning rate $\eta^{(j_{\boldsymbol{u}})} = \frac{R2^{j_{\boldsymbol{u}}-1}}{L\sqrt{T}}$ and initial point equal to zero, we have

$$\sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(j_{\boldsymbol{u}})} - \boldsymbol{u}\rangle \leq R2^{j_{\boldsymbol{u}}-1}L\sqrt{T} \leq \max(2\|\boldsymbol{u}\|_2, R)L\sqrt{T} .$$

For the first sum in the r.h.s. of (6.16), we have

$$\begin{aligned}\sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_t^{(j_{\boldsymbol{u}})}\rangle &= \sum_{t=1}^{T}\left\langle \boldsymbol{g}_t, \sum_{i=1}^{N} p_{t,i}\boldsymbol{x}_t^{(i)} - \boldsymbol{x}_t^{(j_{\boldsymbol{u}})}\right\rangle \\ &= \sum_{t=1}^{T}\left(\sum_{i=1}^{N} p_{t,i}\langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(i)}\rangle - \langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(j_{\boldsymbol{u}})}\rangle\right) .\end{aligned}$$

This is the regret of the multi-scale expert algorithm against the expert $j_{\boldsymbol{u}}$ on the sequence of loss vectors $\boldsymbol{z}_t$ where $|z_{t,i}| = |\langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(i)}\rangle| \leq \|\boldsymbol{g}_t\|_2\|\boldsymbol{x}_t^{(i)}\|_2 \leq L2^{i-1}R$ for all $i = 1,\dots,N$, where we used that ℓ_t is L-Lipschitz and $\mathcal{V}^{(i)}$ has radius $R2^{i-1}$. This is exactly the setting we used for the scales c_i. So, using the regret bound for the multi-scale expert algorithm, we get

$$\sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_t^{(j_{\boldsymbol{u}})}\rangle \leq 2LR2^{j_{\boldsymbol{u}}-1}\sqrt{T(1+\ln(2^N-1))}+5LR2^{j_{\boldsymbol{u}}-1}\left(1+\ln(2^N-1)\right) .$$

Using the fact that $R2^{j_{\boldsymbol{u}}-1} \leq \max(2\|\boldsymbol{u}\|_2, R)$, the expression of N, and combining the bounds for both terms completes the proof. □

This result shows that by combining OSD instances, we can achieve a regret that is close to the one obtained by the best learning rate in the chosen set. The constraint on $\boldsymbol{u}$ is motivated by the fact that for larger competitors, the regret would not be sublinear in T, even with prior knowledge of the norm of the competitor. Hence, it is impossible to compete with $\boldsymbol{u}$ that are too large. The additional multiplicative price for the adaptivity is $\mathcal{O}(\sqrt{\ln T})$, which we know to be necessary by the lower bound for unconstrained OCO in Section 5.3.

This technique provides a principled method for automating the selection of learning rates in an online fashion. However, its interest is mostly theoretical: it shows that such adaptivity to $\boldsymbol{u}$ is possible, but one should not expect a strong empirical performance. In Chapter 13, we will see how to obtain learning-rate-free algorithms for unbounded domains, without the need to combine several base online learners.

6.11 Optimistic Online Mirror Descent

Algorithm 6.6 Optimistic Online Mirror Descent (Optimistic OMD)

Require: Non-empty closed convex $\mathcal{V} \subset \mathcal{X} \subseteq \mathbb{R}^d$, $\psi : \mathcal{X} \to \mathbb{R}$ strictly convex and differentiable on $\operatorname{int} \mathcal{X}$, $\boldsymbol{x}_1 \in \operatorname{int} \mathcal{X} \cap \mathcal{V}$, $\eta_1, \dots, \eta_T > 0$

$\tilde{\boldsymbol{g}}_1 = \boldsymbol{0}$
for $t = 1$ **to** T **do**
  Output $\boldsymbol{x}_t \in \mathcal{V}$
  Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable on $\mathcal{V}$
  Set $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$
  if $t < T$ **then**
    Get hint $\tilde{\boldsymbol{g}}_{t+1} \in \mathbb{R}^d$ on the subgradient
  else
    Get hint $\tilde{\boldsymbol{g}}_{T+1} = \boldsymbol{0}$
  end if
  $\boldsymbol{x}_{t+1} \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \eta_t \langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t + \tilde{\boldsymbol{g}}_{t+1}, \boldsymbol{x} \rangle + B_\psi(\boldsymbol{x}; \boldsymbol{x}_t)$
end for

Till now, we have mainly considered the adversarial model as our model of the environment. However, the world is never completely adversarial. So, we might be tempted to model the environment in some way, but that would leave our algorithm vulnerable to attacks. An alternative is to consider the data as generated by some *predictable process plus an adversarial signal*. In this view, it might be beneficial to try to model the predictable part, without compromising the robustness to the adversarial signal.

In this section, we will explore this possibility through a particular version of OMD, where we *predict* the next gradient. In very intuitive terms, if our predicted gradient is correct, we can expect the regret to decrease. However, if our prediction is wrong, we still want to recover the worst-case guarantee. Such an algorithm is called **Optimistic OMD**.

The pseudo-code of Optimistic OMD is summarized in Algorithm 6.6. At round t, the

algorithm receives a hint $\tilde{\boldsymbol{g}}_{t+1}$ on the next subgradient $\boldsymbol{g}_{t+1}$ and uses it to construct the update. At the same time, you have to remove the hint you used at the previous time step, $\tilde{\boldsymbol{g}}_t$. Note that for the sake of the analysis, it does not matter how the prediction is generated. It can even be generated by another online learning procedure!

To gain some intuition on why this update makes sense, consider the case that $\psi(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{x}\|_2^2$, $\eta_t = \eta$, and $\mathcal{V} = \mathbb{R}^d$. In this case, $\boldsymbol{x}_{t+1} = \boldsymbol{x}_t + \eta\tilde{\boldsymbol{g}}_t - \eta\boldsymbol{g}_t - \eta\tilde{\boldsymbol{g}}_{t+1}$. Unrolling the update, we get $\boldsymbol{x}_{t+1} = \boldsymbol{x}_1 - \eta(\tilde{\boldsymbol{g}}_{t+1} + \sum_{i=1}^t \boldsymbol{g}_i)$. Without hints, that is in plain OMD, under the same assumptions the unrolled update would be $\boldsymbol{x}_{t+1} = \boldsymbol{x}_1 - \eta\sum_{i=1}^t \boldsymbol{g}_i$ and $\boldsymbol{x}_{t+2} = \boldsymbol{x}_1 - \eta\sum_{i=1}^{t+1} \boldsymbol{g}_i$. Hence, $\tilde{\boldsymbol{g}}_{t+1}$ is a proxy for the next (unknown) subgradient $\boldsymbol{g}_{t+1}$.

One might be tempted to multiply $\tilde{\boldsymbol{g}}_t$ by η_{t-1}, because in the previous iteration we used the learning rate η_{t-1}. However, the OMD proof reveals that the correct approach is to think of the learning rate as attached to the Bregman divergence rather than to the subgradients.

One might also be tempted to find a way to study this algorithm with a special proof. However, the one-step lemma we proved for OMD is essentially tight: we only used two inequalities, one to deal with the set $\mathcal{V}$ and the other one to linearize the losses. But, both steps can be made tight, considering $\mathcal{V} = \mathbb{R}^d$ and linear losses. Hence, if the update is just OMD with a different sequence of subgradients, the proof *must* follow from that of OMD with a different set of subgradients.

Observe that setting $\tilde{\boldsymbol{g}}_1 = \boldsymbol{0}$ is not a limitation because setting it to any other value would be equivalent to changing the initial point $\boldsymbol{x}_1$. In the same way, setting $\tilde{\boldsymbol{g}}_{T+1} = \boldsymbol{0}$ does not change the behaviour of the algorithm in rounds $t = 1, \dots, T$, but it simplifies the analysis.

Theorem 6.38 *With the notation and initialization in Algorithm 6.6, assume $\eta_{t+1} \leq \eta_t,\ t = 1, \dots, T-1$. Define $\delta_t = \max(\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t\|_\star - \|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t + \tilde{\boldsymbol{g}}_{t+1}\|_\star, 0)$. Then, under the assumptions of Lemma 6.10, for all $\boldsymbol{u} \in \mathcal{V}$, the following regret bounds hold*

$$\begin{aligned}&\sum_{t=1}^T (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u}))\\ &\leq \frac{\max_{1\leq t\leq T} B_\psi(\boldsymbol{u}; \boldsymbol{x}_t)}{\eta_T} + \sum_{t=1}^T \left(\langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{1}{\eta_t} B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t)\right).\end{aligned}$$

Moreover, if η_t is constant, i.e., $\eta_t = \eta\ \forall t = 1, \dots, T$, we have

$$\sum_{t=1}^T (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_1)}{\eta} + \sum_{t=1}^T \left(\langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{1}{\eta} B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t)\right).$$

Moreover, in both cases for all t we have

$$\begin{aligned}&\langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{1}{\eta_t} B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t)\\ &\quad\leq \frac{\eta_t(\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t\|_\star^2 - \delta_t^2)}{2\lambda}\\ &\quad\leq \frac{\eta_t\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t\|_\star \min(\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t\|_\star, 2\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t + \tilde{\boldsymbol{g}}_{t+1}\|_\star)}{2\lambda}.\end{aligned}$$

To prove this theorem, we will use the following technical lemmas.

Lemma 6.39 *Assume $\psi : \mathcal{X} \to \mathbb{R}$ to be λ-strongly convex with respect to $\|\cdot\|$ and $\boldsymbol{u} \in \mathcal{V} \cap \operatorname{int}\mathcal{X}$. Let $\mathcal{V} \subseteq \mathcal{X}$ be a non-empty convex closed set. Assume that $\boldsymbol{x}' = \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} B_\psi(\boldsymbol{x};\boldsymbol{u}) + \eta\langle \boldsymbol{g}, \boldsymbol{x}\rangle$ exists and belongs to $\operatorname{int}\mathcal{X}$. Then, we have*

$$\|\boldsymbol{x}' - \boldsymbol{u}\| \leq \frac{\eta}{\lambda}\|\boldsymbol{g}\|_\star \,.$$

Proof Observe that from the optimality condition of Theorem 2.8 for $\boldsymbol{x}'$, we have

$$\langle \eta\boldsymbol{g} + \nabla\psi(\boldsymbol{x}') - \nabla\psi(\boldsymbol{u}), \boldsymbol{v} - \boldsymbol{x}'\rangle \geq 0, \quad \forall \boldsymbol{v} \in \mathcal{V}\,. \tag{6.17}$$

Hence, we have

$$\begin{aligned}\lambda\|\boldsymbol{x}' - \boldsymbol{u}\|^2 &\leq B_\psi(\boldsymbol{u};\boldsymbol{x}') + B_\psi(\boldsymbol{x}';\boldsymbol{u}) = \langle \nabla\psi(\boldsymbol{x}') - \nabla\psi(\boldsymbol{u}), \boldsymbol{x}' - \boldsymbol{u}\rangle \leq \langle \eta\boldsymbol{g}, \boldsymbol{x}' - \boldsymbol{u}\rangle \\ &\leq \eta\|\boldsymbol{g}\|_\star\|\boldsymbol{x}' - \boldsymbol{u}\|\,.\end{aligned}$$

where in the first inequality we used the strong convexity of ψ, (6.17) in the second one, and the definition of dual norms in the last one. Reordering, we have the stated bound. □

Lemma 6.40 *Let $\boldsymbol{g} \in \mathbb{R}^d$ and $a, c > 0$. Then, we have*

$$\sup_{\boldsymbol{v}\in\mathbb{R}^d:\|\boldsymbol{v}\|\leq c/a} \langle \boldsymbol{g}, \boldsymbol{v}\rangle - \frac{a}{2}\|\boldsymbol{v}\|^2 = \frac{\|\boldsymbol{g}\|_\star^2 - \max(\|\boldsymbol{g}\|_\star - c, 0)^2}{2a} \leq \frac{\|\boldsymbol{g}\|_\star}{2a}\min\left(\|\boldsymbol{g}\|_\star, 2c\right).$$

Proof

$$\begin{aligned}\sup_{\boldsymbol{v}\in\mathbb{R}^d:\|\boldsymbol{v}\|\leq c/a} \langle \boldsymbol{g}, \boldsymbol{v}\rangle - \frac{a}{2}\|\boldsymbol{v}\|^2 &= \sup_{0\leq x\leq c/a}\ \sup_{\boldsymbol{v}\in\mathbb{R}^d:\|\boldsymbol{v}\|=x} \langle \boldsymbol{g}, \boldsymbol{v}\rangle - \frac{a}{2}\|\boldsymbol{v}\|^2 \\ &= \sup_{0\leq x\leq c/a} x\|\boldsymbol{g}\|_\star - \frac{a}{2}x^2 \\ &= \frac{1}{a}\min(\|\boldsymbol{g}\|_\star, c)\left[\|\boldsymbol{g}\|_\star - \frac{1}{2}\min(\|\boldsymbol{g}\|_\star, c)\right] \\ &= \frac{1}{2a}\left[\|\boldsymbol{g}\|_\star^2 - \max(\|\boldsymbol{g}\|_\star - c, 0)^2\right]\,.\end{aligned}$$

The second inequality is obtained by lower-bounding the max with its two possible values and overapproximating. □

We can now prove the theorem.

Proof of Theorem 6.38 We use Lemma 6.10 with $\boldsymbol{g}_t \to \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t + \tilde{\boldsymbol{g}}_{t+1}$, that gives us that $\boldsymbol{x}_{t+1}$ exists, and it is in $\operatorname{int}\mathcal{X}$. Moreover, we also have

$$\begin{aligned}&\langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t + \tilde{\boldsymbol{g}}_{t+1}, \boldsymbol{x}_t - \boldsymbol{u}\rangle \\ &\leq \frac{1}{\eta_t}\left(B_\psi(\boldsymbol{u};\boldsymbol{x}_t) - B_\psi(\boldsymbol{u};\boldsymbol{x}_{t+1}) - B_\psi(\boldsymbol{x}_{t+1};\boldsymbol{x}_t)\right) + \langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t + \tilde{\boldsymbol{g}}_{t+1}, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle\,.\end{aligned}$$

Summing over $t = 1, \dots, T$ the l.h.s., we obtain

$$\begin{aligned}
&\sum_{t=1}^{T} \langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t + \tilde{\boldsymbol{g}}_{t+1}, \boldsymbol{x}_t - \boldsymbol{u} \rangle \\
&\quad = \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle + \sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_{t+1} - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \\
&\quad = \sum_{t=1}^{T} (\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle + \langle \tilde{\boldsymbol{g}}_{t+1} - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t \rangle) + \langle \tilde{\boldsymbol{g}}_1 - \tilde{\boldsymbol{g}}_{T+1}, \boldsymbol{u} \rangle \, .
\end{aligned}$$

Summing the r.h.s., we have that

$$\sum_{t=1}^{T} \langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t + \tilde{\boldsymbol{g}}_{t+1}, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle = \sum_{t=1}^{T} \langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle + \sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_{t+1}, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle \, .$$

Finally, observe that

$$\begin{aligned}
\sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_{t+1} - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_{t+1}, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle &= \sum_{t=1}^{T} (\langle \tilde{\boldsymbol{g}}_{t+1}, \boldsymbol{x}_{t+1} \rangle - \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t \rangle) \\
&= \langle \tilde{\boldsymbol{g}}_{T+1}, \boldsymbol{x}_{T+1} \rangle - \langle \tilde{\boldsymbol{g}}_1, \boldsymbol{x}_1 \rangle \, .
\end{aligned}$$

Telescoping the terms with the Bregman divergences gives the first bound. The bound for fixed η is proved similarly.

Now, using the strong convexity of ψ, we have

$$\langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - \frac{1}{\eta_t} B_\psi(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) \leq \langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - \frac{\lambda}{2\eta_t} \|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 \, .$$

From Lemma 6.39, we know that $\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|$ is upper bounded by $\frac{\eta_t}{\lambda} \|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t + \tilde{\boldsymbol{g}}_{t+1}\|_\star$. Hence, we have

$$\langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - \frac{\lambda}{2\eta_t} \|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 \leq \max_{\boldsymbol{v} \in \mathbb{R}^d : \|\boldsymbol{v}\| \leq \frac{\eta_t}{\lambda} \|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t + \tilde{\boldsymbol{g}}_{t+1}\|_\star} \langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{v} \rangle - \frac{\lambda}{2\eta_t} \|\boldsymbol{v}\|^2 \, .$$

Using Lemma 6.40 finishes the proof. □

In the next section, we show an immediate application of Optimistic OMD to study OMD with delayed feedback. In Section 15.5, we describe the applications of optimistic algorithms for saddle-point optimization.

6.12 Application: OMD with Delayed Feedback

We now consider online learning with *delayed feedback*. For simplicity, we consider a constant delay of length τ, where at time t the learner has only observed $\ell_1, \dots, \ell_{t-1-\tau}$ before producing $\boldsymbol{x}_t$. In other words, the learner observes ℓ_t at time $t + \tau$. This means that the algorithm receives its first feedback at round $\tau + 1$ and it receives no feedback before that round.

Instead of designing online algorithms specifically for the case of delayed feedback, we will reduce the setting of online learning with delays to that of optimistic online learning.

In OMD with delays we predict the same $\boldsymbol{x}_1$ for the first $\tau + 1$ rounds and

$$\boldsymbol{x}_{t+1} = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \; B_\psi(\boldsymbol{x}; \boldsymbol{x}_t) + \eta_t \langle \boldsymbol{g}_{t-\tau}, \boldsymbol{x} \rangle, \quad \forall t \geq \tau + 1 \; .$$

On the other hand, Optimistic OMD without delays updates with

$$\boldsymbol{x}_{t+1} = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \; B_\psi(\boldsymbol{x}; \boldsymbol{x}_t) + \eta_t \langle \boldsymbol{g}_t + \tilde{\boldsymbol{g}}_{t+1} - \tilde{\boldsymbol{g}}_t, \boldsymbol{x} \rangle,$$

where $\tilde{\boldsymbol{g}}_1 = \boldsymbol{0}$. So, the two are *equivalent* by setting $\tilde{\boldsymbol{g}}_{t+1} - \tilde{\boldsymbol{g}}_t = \boldsymbol{g}_{t-\tau} - \boldsymbol{g}_t$. So, unrolling, we have $\tilde{\boldsymbol{g}}_{t+1} = -\sum_{i=\max(1,t-\tau+1)}^{t} \boldsymbol{g}_i$. In other words, *we are receiving a very poor hint that corresponds to the delayed feedback*.

The above observation is very powerful because it immediately gives us the regret guarantee of OMD with delays. Let's consider OMD with delayed feedback and constant learning rate. Following the delay-to-optimism conversion in the previous section, we set $\tilde{\boldsymbol{g}}_{t+1} = -\sum_{i=\max(1,t-\tau+1)}^{t} \boldsymbol{g}_i$ for $t = 1, \dots, T-1$, yet we must also set $\tilde{\boldsymbol{g}}_{T+1} = \boldsymbol{0}$. Hence, for all $\boldsymbol{u} \in \mathcal{V}$, we obtain a regret upper bound with reduced terms in the sum for the first $T - 1$ rounds:

$$\begin{aligned}
&\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \\
&\qquad \leq \frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_1)}{\eta} + \frac{\eta}{\lambda} \sum_{t=\tau+1}^{T-1} \left\| \sum_{i=\max(1,t-\tau)}^{t} \boldsymbol{g}_i \right\|_\star \|\boldsymbol{g}_{t-\tau}\|_\star + \frac{\eta}{2\lambda} \left\| \sum_{i=\max(1,T-\tau)}^{T} \boldsymbol{g}_i \right\|_\star^2 .
\end{aligned}$$

That is, assuming $\boldsymbol{g}_t$ is bounded for all t, we improved the worst-case dependence in the dominant term of the bound from τ^2 to τ. Optimally choosing the learning rate, we obtain the optimal regret guarantee of $\mathcal{O}(\sqrt{\tau T})$.

6.13 History Bits

The Bregman divergence was introduced by Bregman (1967) as a particular example of a distance-like function satisfying certain properties, to generalize the cyclic projection algorithm to general topological vector spaces. Often people drop the condition on the strict convexity (e.g., Bauschke et al., 2003), but in reality it is part of the original definition by Bregman (1967).

Mirror Descent (MD) was introduced by Nemirovskij and Yudin (1983) in the *offline* setting. The description of MD with Bregman divergence that I described here (with minor changes) was done by Beck and Teboulle (2003). The minor changes are in decoupling the domain $\mathcal{X}$ of ψ from the feasibility set $\mathcal{V}$. This allows us to use functions ψ that do not satisfy the condition (6.5), but they satisfy (6.6). In the online setting, the mirror descent scheme was used for the first time by Warmuth and Jagota (1997).

Most of the online learning literature for OMD assumes ψ to be *Legendre* (see, e.g., Cesa-Bianchi and Lugosi, 2006) that corresponds to assuming (6.5) (or $\lim_{\boldsymbol{x} \to \operatorname{bdry} \mathcal{X}} \|\nabla \psi(\boldsymbol{x})\|_2 = +\infty$, see (Rockafellar, 1970, Theorem 26.1 and Lemma 26.2)). This condition allows us to prove that $\nabla \psi_{\mathcal{V}}^\star = (\nabla \psi_{\mathcal{V}})^{-1}$. However, it turns out that the Legendre condition is not necessary, and we only need the function ψ to be differentiable on the predictions $\boldsymbol{x}_t$. For

example, we only need one of the two conditions in (6.5) or (6.6) to hold. Removing the Legendre assumption makes it easier to use OMD with different combinations of feasibility sets/Bregman divergences. So, I did not introduce the concept of Legendre functions at all, relying instead on (a minor modification of) OMD as described by Beck and Teboulle (2003). Theorem 6.8 is derived from (Bauschke and Borwein, 1997, Theorem 3.12).

The proof of Theorem 6.26 is based on the one in Kakade et al. (2009).

The concept of local norms was introduced in Abernethy et al. (2008) for FTRL with self-concordant regularizers.

The EG algorithm was introduced by Kivinen and Warmuth (1997), but not as a specific instantiation of OMD. Beck and Teboulle (2003) rediscover EG for the offline case as an example of MD. Later, Cesa-Bianchi and Lugosi (2006) shows that EG is just an instantiation of OMD. The p-norm algorithms for online prediction were originally introduced by Grove et al. (1997, 2001). Lemma 6.35 is well-known, but I could not find a good proof for it,[1] so I wrote one. The trick to set $q = 2 \ln d$ is from Gentile and Littlestone (1999); Gentile (2003) (online learning) and apparently rediscovered in Ben-Tal et al. (2001) (optimization). The LEA setting was introduced by Littlestone and Warmuth (1994) and Vovk (1990). The ideas in Algorithm 6.3 are based on the Multiplicative Weights algorithm (Littlestone and Warmuth, 1994) and the Hedge algorithm (Freund and Schapire, 1995, 1997), both being just OMD/FTRL with the entropic distance generating function/regularizer. As a side note, the weighted majority algorithm was also discovered independently in the game theory literature by Fudenberg and Levine (1995). For two experts with losses in $[0, 1]$, Cover (1965) showed that the asymptotic minimax regret is $\sqrt{\frac{T}{2\pi}}$ and proposed an algorithm achieving it. Notably, the approach in Cover (1965) is based on online betting. On the other hand, for more than 2 experts and losses in $[0, 1]$, the minimax regret is $(1 + o(1))\sqrt{\frac{T \ln d}{2}}$, where $o(1) \to 0$ when $d, T \to \infty$ (Cesa-Bianchi et al., 1993, 1997). By now, the literature on LEA is huge, with tons of variations over algorithms and settings.

The multi-scale setting was introduced independently[2] and around the same time by Bubeck et al. (2017, 2019) and Foster et al. (2017). Bubeck et al. (2017, 2019) propose two algorithms, one for non-positive losses and one for generic losses. Unfortunately, their proof for the generic losses appears to be wrong and not easily fixable. Their bound is similar to the one we proved here. Foster et al. (2017) proved a stronger result, where the term in the logarithm depends on c_i/π_i rather than on the sum of the scales and the minimal value. However, this algorithm has a computational complexity at step t of $\mathcal{O}(t)$. Cutkosky and Orabona (2018) proposed another solution that achieves the same bound of Foster et al. (2017), but with constant computational complexity. The algorithm I describe here is a simplification of the one in Chen et al. (2021). The bound is worse than the one in Cutkosky and Orabona (2018) and Foster et al. (2017), but it is simpler to describe, and it also allowed me to describe the method of the shifted surrogate loss. The use of shifted surrogate losses in LEA setting was proposed in Hazan and Kale (2008, 2010) and later used in Steinhardt and Liang (2014). Interestingly, these shifted surrogates can be understood as an approximation of the coin-betting instantaneous log wealth, as one can see by comparing the update

[1] People often cite Shalev-Shwartz (2007, Lemma 17), but it has a wrong proof because it ignores the fact that the function is not twice differentiable.

[2] Dylan Foster, personal communication, 2026

of Squint (Koolen and van Erven, 2015) with the update of the KT-based parameter-free algorithm for LEA.

The construction in Section 6.10 is from Foster et al. (2017), but they require N function values per step, while we use only one subgradient per step using the linearization idea proposed in MetaGrad (van Erven and Koolen, 2016; van Erven et al., 2021). The idea of restricting the norm of the competitor to the "meaningful ones" is not necessary in the original formulation by Foster et al. (2017), but it is standard, and it has been rediscovered many times in different forms, e.g., in a method privately proposed by Nemirovski in 2013 and described in the ICML 2020 tutorial[3] on "Parameter-free Online Optimization" by Orabona and Cutkosky, in Orabona (2014, Theorem 2), in Cutkosky (2019c, Section 5).

The idea of "hallucinating" future losses used in Optimistic OMD is originally from Azoury and Warmuth (2001) in the Forward Algorithm. Apparently, this idea was forgotten and rediscovered by Chiang et al. (2012) that used the previous loss function as an estimate of the next one, showing smaller regret in the case that the losses have small temporal variation. Later, Rakhlin and Sridharan (2013b) generalized this idea in the Optimistic OMD algorithm. Surprisingly enough, the procedure using two Optimistic OGD algorithms to solve saddle-point problems was already proposed by Popov (1980), see also Section 15.6. Optimistic OMD was proposed in Rakhlin and Sridharan (2013b) with a two-step update. It was then simplified to the one-step updates I presented here by Joulani et al. (2017). However, Malitsky (2015) presented a version of Popov's algorithm for variational inequalities with only one projection that is essentially Optimistic OGD with one projection. The proof I present here is based on the one I proposed for Optimistic FTRL in Section 7.12.

Weinberger and Ordentlich (2002) were the first to analyze the delayed feedback problem. They considered the adversarial full information setting with a fixed, known delay τ. They achieved the optimal rate of $\mathcal{O}(\sqrt{\tau T})$ with a black-box reduction, by running $\tau + 1$ online algorithms on subsampled sequences. Zinkevich et al. (2009) analyzed OMD with delayed feedback and obtained the optimal regret. Joulani et al. (2013) unified most of the prior research on the effect of delay in adversarial and stochastic problems. McMahan and Streeter (2014) studied a variant of AdaGrad with delays. Quanrud and Khashabi (2015) studied OGD and FTRL with adversarially chosen delays, while Joulani et al. (2016) studied OMD and FTRL-Proximal with adaptive learning rates.

The equivalence of delays/bad hints and the improved regret guarantees for Optimistic OMD and Optimistic FTRL are due to Flaspohler et al. (2021). However, their OMD bound contains a small mistake: they are missing the last terms in the bound, because we set $\tilde{\boldsymbol{g}}_{T+1} = \mathbf{0}$, breaking the equivalence between optimistic and delayed updates for $\boldsymbol{x}_{T+1}$. Note that while the choice of $\boldsymbol{x}_{T+1}$ does not influence the algorithm in the rounds $1, \dots, T$, its value appears in the regret upper bound. Flaspohler et al. (2021) also show that Regret Matching and Regret Matching+ algorithms automatically adapts to the delay τ.

Lemma 6.39 is from Joulani et al. (2016).

6.14 Exercises

Problem 6.1 *Prove the three-points equality for Bregman divergences in Lemma 6.7.*

[3] `https://parameterfree.com/icml-tutorial/`

Problem 6.2 *Let $\boldsymbol{A} \in \mathbb{R}^{d\times d}$ be a positive definite matrix. Define $\|\boldsymbol{x}\|_{\boldsymbol{A}}^2 = \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}$. Prove that $\frac{1}{2}\|\boldsymbol{x}-\boldsymbol{y}\|_{\boldsymbol{A}}^2$ is the Bregman divergence $B_\psi(\boldsymbol{x};\boldsymbol{y})$ associated with $\psi(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{x}\|_{\boldsymbol{A}}^2$.*

Problem 6.3 *Let $\psi : \mathcal{X} \to \mathbb{R}$ be a distance generating function and B_ψ its associated Bregman divergence. Fix $\boldsymbol{y} \in \operatorname{int}\mathcal{X}$ and define $f(\boldsymbol{x}) = B_\psi(\boldsymbol{x};\boldsymbol{y})$. For any $\boldsymbol{v} \in \operatorname{int}\mathcal{X}$ and $\boldsymbol{x} \in \mathcal{X}$, prove that $B_f(\boldsymbol{x};\boldsymbol{v}) = B_\psi(\boldsymbol{x};\boldsymbol{v})$.*

Problem 6.4 *We want to show an equality quantifying the gap in Fenchel–Young's inequality with a Bregman divergence term. Assume that f and $f^\star$ are differentiable, f strictly convex, and $\operatorname{dom} f = \mathbb{R}^d$. Prove that*

$$f(\boldsymbol{x}) + f^\star(\boldsymbol{\theta}) = \langle \boldsymbol{\theta}, \boldsymbol{x}\rangle + B_f\left(\boldsymbol{x}; \nabla f^\star(\boldsymbol{\theta})\right) \ .$$

Problem 6.5 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be even. Prove that $f^\star$ is even.*

Problem 6.6 *In the proof of OMD, we have the terms $-B_\psi(\boldsymbol{x}_{t+1};\boldsymbol{x}_t) + \langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle$. Prove that they can be lower bounded by $B_\psi(\boldsymbol{x}_t;\boldsymbol{x}_{t+1})$.*

Problem 6.7 *Generalize the concept of strong convexity to Bregman divergences, instead of norms, and prove a logarithmic regret guarantee for such functions using OMD.*

Problem 6.8 *Consider the setting of Theorem 6.11, assume that $\sup_{\boldsymbol{x},\boldsymbol{y}\in\mathcal{V}} B_\psi(\boldsymbol{x};\boldsymbol{y}) \le D^2$, and that $\|\boldsymbol{g}_t\|_\star \le L$ for all t. Suppose that the time horizon T is not known in advance.*

Use the following doubling trick. *Split time into epochs of lengths $1, 2, 4, \dots$. At the beginning of each epoch of length 2^k, restart OMD from an arbitrary point in $\operatorname{int}\mathcal{X} \cap \mathcal{V}$ and run it with the constant learning rate $\eta_k = \frac{D\sqrt{2\lambda}}{L\sqrt{2^k}}$. Prove that, for every $T \ge 1$ and every $\boldsymbol{u} \in \mathcal{V}$, this algorithm satisfies*

$$\sum_{t=1}^{T} \left(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})\right) \le \frac{2}{\sqrt{2}-1}\frac{DL}{\sqrt{\lambda}}\sqrt{T} \ .$$

Problem 6.9 *Derive the EG update rule and regret bound in the case that the algorithm starts from an arbitrary vector $\boldsymbol{x}_1$ in the probability simplex.*

Problem 6.10 *Show that EG is invariant to additive constants added to the loss vectors. Use this observation to show that the terms $\sum_{i=1}^d g_{t,i}^2 x_{t,i}$ for $t = 1, \dots, T$ in the regret upper bound can be tightened to $\sum_{i=1}^d (g_{t,i} - m_t)^2 x_{t,i}$ for any $m_t \in \mathbb{R}$.*

Problem 6.11 *Extend Theorem 5.1 to arbitrary norms, measuring the diameter with respect to a norm $\|\cdot\|$ and considering losses L-Lipschitz with respect to $\|\cdot\|$.*

Problem 6.12 *In this problem, we will tackle Online Non-Convex Optimization. Assume that $\mathcal{V} \subset \mathbb{R}^d$ is the feasible set and it is convex and bounded. The losses $\ell_t : \mathbb{R}^d \to [0,1]$ are non-convex and 1-Lipschitz with respect to $\|\cdot\|_2$. Prove that there exists a randomized algorithm that achieves sublinear regret on this problem, assuming knowledge of the total number of rounds T. Hint: aim for something like $\mathbb{E}[\operatorname{Regret}_T(\boldsymbol{u})] = \mathcal{O}(\sqrt{dT\ln T})$ and do not worry about the efficiency of the algorithm.*

7

Follow-the-Regularized-Leader

Until now, we focused only on Online Subgradient Descent and its generalization, Online Mirror Descent, with a brief ad-hoc analysis of Follow-the-Leader (FTL) in the first chapter. In this chapter, we will extend FTL to a powerful and generic algorithm to do Online Convex Optimization (OCO): Follow-the-Regularized-Leader (FTRL).

FTRL is a very intuitive algorithm: at each time step, it will play the minimizer of the sum of the past losses *plus* a time-varying *regularizer*, ψ_t. We will see that the regularization is needed to make the algorithm "more stable" with linear losses and avoid the jumping back and forth that we saw in Example 2.10.

7.1 The Follow-the-Regularized-Leader Algorithm

Algorithm 7.1 Follow-the-Regularized-Leader (FTRL)

Require: A sequence of regularizers $\psi_1, \dots, \psi_T : \mathcal{X} \to \mathbb{R}$, closed non-empty set $\mathcal{V} \subseteq \mathcal{X} \subseteq \mathbb{R}^d$

1: **for** $t = 1$ **to** T **do**
2: Output $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1} \ell_i(\boldsymbol{x})$
3: Receive $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ and pay the loss $\ell_t(\boldsymbol{x}_t)$
4: **end for**

In **Follow-the-Regularized-Leader (FTRL)**, in each round we output the minimizer of the regularized cumulative past losses, see Algorithm 7.1. It should be clear that FTRL is not an algorithm, but a family of algorithms, in the same way that Online Mirror Descent (OMD) is a family of algorithms.

Before analyzing FTRL, let's get some intuition on it. In OMD, we saw that the "state" of the algorithm is stored in the current iterate $\boldsymbol{x}_t$, in the sense that the next iterate $\boldsymbol{x}_{t+1}$ depends on $\boldsymbol{x}_t$ and the loss received at time t, plus obviously the choice of the learning rate. Instead, in FTRL, the next iterate $\boldsymbol{x}_{t+1}$ depends on the entire history of losses received up to time t and on the regularizer used at time t. This has an immediate consequence: in the case that $\mathcal{V}$ is bounded, OMD will only "remember" the last $\boldsymbol{x}_t$ and nothing else. On the other hand, FTRL keeps in memory the entire history of the past, which in principle allows us to recover the iterates before the projection in $\mathcal{V}$.

This difference in behavior might make the reader think that FTRL is more computationally and memory-expensive. And indeed it is! But, in Section 7.3 we will also see that there

is a way to consider approximate losses that makes the algorithm as expensive as OMD, yet retaining the same or more information than OMD.

For FTRL, we prove an *equality* for the regret. This equality factors the regret in three terms that have precise meanings and can be easily upper-bounded with some familiar quantities.

Lemma 7.1 *Let $\psi_1, \dots, \psi_{T+1} : \mathcal{X} \to \mathbb{R}$ be a sequence of regularization functions and $\mathcal{V} \subseteq \mathcal{X} \subseteq \mathbb{R}^d$ a closed and non-empty set. Denote by $F_t(\boldsymbol{x}) = \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1} \ell_i(\boldsymbol{x})$, where $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ for all t. Assume that $\operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} F_t(\boldsymbol{x})$ is not empty and set $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} F_t(\boldsymbol{x})$. Then, for all $\boldsymbol{u} \in \mathcal{X}$, we have*

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) = \psi_{T+1}(\boldsymbol{u}) - \min_{\boldsymbol{x}\in\mathcal{V}} \psi_1(\boldsymbol{x}) + \sum_{t=1}^{T}[F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t)] + F_{T+1}(\boldsymbol{x}_{T+1}) - F_{T+1}(\boldsymbol{u}) \,.$$

In addition, the choice of ψ_{T+1} does not change the iterates $\boldsymbol{x}_1, \dots, \boldsymbol{x}_T$ of the algorithm.

Proof Given that the terms $\ell_t(\boldsymbol{x}_t)$ appear on both sides of the equality, we just have to verify that

$$-\sum_{t=1}^{T}\ell_t(\boldsymbol{u}) = \psi_{T+1}(\boldsymbol{u}) - \min_{\boldsymbol{x}\in\mathcal{V}} \psi_1(\boldsymbol{x}) + \sum_{t=1}^{T}[F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1})] + F_{T+1}(\boldsymbol{x}_{T+1}) - F_{T+1}(\boldsymbol{u}) \,.$$

Remembering that $F_1(\boldsymbol{x}_1) = \min_{\boldsymbol{x}\in\mathcal{V}} \psi_1(\boldsymbol{x})$ and using the fact that the sum with F_t is telescopic, we have

$$\begin{aligned}-\sum_{t=1}^{T}\ell_t(\boldsymbol{u}) &= \psi_{T+1}(\boldsymbol{u}) - F_1(\boldsymbol{x}_1) + F_1(\boldsymbol{x}_1) - F_{T+1}(\boldsymbol{x}_{T+1}) + F_{T+1}(\boldsymbol{x}_{T+1}) - F_{T+1}(\boldsymbol{u}) \\ &= \psi_{T+1}(\boldsymbol{u}) - F_{T+1}(\boldsymbol{u}),\end{aligned}$$

that is true by the definition of F_{T+1}. The second statement is immediate. □

Remark 7.2 *We basically did not assume anything on ℓ_t nor on ψ_t, hence the above equality holds even for non-convex losses and regularizers. Yet, solving the minimization problem at each step might be computationally infeasible.*

Remark 7.3 *The FTRL algorithm is invariant to any constant added to the regularizers, hence we can always state the regret guarantee with $\psi_t(\boldsymbol{u}) - \min_{\boldsymbol{x}\in\mathcal{V}} \psi_t(\boldsymbol{x})$ instead of $\psi_t(\boldsymbol{u})$. However, for clarity, we will sometimes explicitly choose the regularizers such that their minimum in $\mathcal{V}$ is 0.*

Remark 7.4 *The terms $\ell_t(\boldsymbol{x}_t)$ appear on the left and right of the equality. Hence, in some proofs we can substitute them with some other terms, for example $\ell_t(\tilde{\boldsymbol{x}}_t)$ where $\tilde{\boldsymbol{x}}_t$ is* not *the FTRL prediction. In other words, we can write the same equality for the prediction of a generic algorithm whose regret will depend on the prediction of the FTRL algorithm.*

Let's take a closer look at the equality. If $\boldsymbol{u} \in \mathcal{V}$, we have that the sum of the last two terms on the r.h.s. is negative. On the other hand, the first two terms on the r.h.s. are similar to what we got in OMD. The interesting part is the sum of the terms $F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t)$. To give an intuition of what is going on, let's consider the case when the regularizer is constant over time, i.e., $\psi_t = \psi$. Hence, the terms in the sum can be rewritten as

$$F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t) = \psi(\boldsymbol{x}_t) + \sum_{i=1}^{t} \ell_i(\boldsymbol{x}_t) - \left(\psi(\boldsymbol{x}_{t+1}) + \sum_{i=1}^{t} \ell_i(\boldsymbol{x}_{t+1}) \right) .$$

Hence, we are measuring the distance between the value of the regularized losses in two consecutive predictions of the algorithms. Roughly speaking, if $\boldsymbol{x}_t \approx \boldsymbol{x}_{t+1}$ and the losses plus regularization are "nice" then this term will be small. This should remind you exactly of the OMD update, where we *constrain* $\boldsymbol{x}_{t+1}$ to be close to $\boldsymbol{x}_t$. Instead, here the two predictions will be close one to the other if the minimizer of the regularized losses up to time t is close to the minimizer of the losses up to time $t+1$. So, like in OMD, the regularizer here will play the critical role of *stabilizing* the predictions, if the losses do not possess enough curvature.

However, while surprising, the above equality is not yet a regret bound, because it is "implicit". In fact, the losses are appearing on both sides of the equation. So, in the following, we will see different ways to get an explicit upper bound from Lemma 7.1.

7.2 FTRL Regret Bound using Strong Convexity

An easy case to get a regret upper bound for FTRL is when the losses plus the regularizer are strongly convex, as we prove in the next lemma.

Lemma 7.5 *With the notation in Algorithm 7.1, denote by $F_t(\boldsymbol{x}) = \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1} \ell_i(\boldsymbol{x})$. Assume that $\mathcal{V} \subseteq \mathcal{X}$ is non-empty, closed, and convex. If F_t is closed, subdifferentiable, and strongly convex in $\mathcal{V}$, then $\boldsymbol{x}_t$ exists and is unique. In addition, assume $\partial \ell_t(\boldsymbol{x}_t)$ to be non-empty and $F_t + \ell_t$ to be closed, subdifferentiable, and λ_t-strongly convex with respect to $\|\cdot\|$ in $\mathcal{V}$. Then, we have*

$$\begin{aligned}
&F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t) \\
&\quad \le \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - \frac{\lambda_t}{2} \|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 + \psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1}) \\
&\quad \le \frac{\|\boldsymbol{g}_t\|_\star^2}{2\lambda_t} + \psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1}), \quad \forall \boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t) .
\end{aligned}$$

Proof The existence and uniqueness are given by Theorem 6.9. Then, we have

$$\begin{aligned}
&F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t) \\
&\quad = (F_t(\boldsymbol{x}_t) + \ell_t(\boldsymbol{x}_t)) - (F_t(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_{t+1})) + \psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1}) \\
&\quad = (F_t(\boldsymbol{x}_t) + \ell_t(\boldsymbol{x}_t) + \iota_{\mathcal{V}}(\boldsymbol{x}_t)) - (F_t(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_{t+1}) + \iota_{\mathcal{V}}(\boldsymbol{x}_{t+1})) \\
&\qquad + \psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1}) \\
&\quad \le \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - \frac{\lambda_t}{2} \|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 + \psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1}),
\end{aligned}$$

where in the second inequality we used Lemma 4.2 for the function $F_t + \ell_t + \iota_{\mathcal{V}}$ with $\boldsymbol{g}_t \in \partial(F_t+\ell_t+\iota_{\mathcal{V}})(\boldsymbol{x}_t)$. Observing that $\boldsymbol{x}_t = \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} F_t(\boldsymbol{x})$, we have $\mathbf{0} \in \partial(F_t+\iota_{\mathcal{V}})(\boldsymbol{x}_t)$ by Theorem 6.27. Hence, using Theorem 2.23, we have $\partial\ell_t(\boldsymbol{x}_t) \subseteq \partial(F_t + \ell_t + \iota_{\mathcal{V}})(\boldsymbol{x}_t)$.

The second inequality is obtained observing that $\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle \leq \|\boldsymbol{g}_t\|_\star \|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|$ and using the elementary inequality $ax - \frac{b}{2}x^2 \leq \frac{a^2}{2b}$ for $x \in \mathbb{R}$ and $a, b > 0$. □

Let's see an immediate application of this lemma.

Corollary 7.6 *With the notation in Algorithm 7.1, let $\mathcal{V}$ be non-empty, closed, and convex. Let $\psi : \mathcal{V} \to \mathbb{R}$ be a closed and μ-strongly convex function with respect to $\|\cdot\|$. Set the sequence of regularizers as $\psi_t(\boldsymbol{x}) = \frac{1}{\eta_{t-1}}(\psi(\boldsymbol{x}) - \min_{\boldsymbol{z}\in\mathcal{V}} \psi(\boldsymbol{z}))$ for $t = 1, \dots, T$ and $\psi_{T+1} = \psi_T$. Assume each ℓ_t subdifferentiable in $\mathcal{V}$. Then, for all $\boldsymbol{u} \in \mathcal{V}$, FTRL guarantees*

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \frac{\psi(\boldsymbol{u}) - \min_{\boldsymbol{x}\in\mathcal{V}} \psi(\boldsymbol{x})}{\eta_{T-1}} + \sum_{t=1}^{T} \frac{\eta_{t-1}}{2\mu}\|\boldsymbol{g}_t\|_\star^2 + \sum_{t=1}^{T-1}\left(\frac{1}{\eta_{t-1}} - \frac{1}{\eta_t}\right)(\psi(\boldsymbol{x}_{t+1}) - \min_{\boldsymbol{z}\in\mathcal{V}} \psi(\boldsymbol{z})), \quad \forall \boldsymbol{g}_t \in \partial\ell_t(\boldsymbol{x}_t)\,.$$

Moreover, if the functions ℓ_t are L-Lipschitz on an open set containing $\mathcal{V}$, setting $\eta_{t-1} = \frac{\alpha\sqrt{\mu}}{L\sqrt{t}}$ we get

$$\sum_{t=1}^{T}\ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T}\ell_t(\boldsymbol{u}) \leq \left(\frac{\psi(\boldsymbol{u}) - \min_{\boldsymbol{x}} \psi(\boldsymbol{x})}{\alpha} + \alpha\right)\frac{L\sqrt{T}}{\sqrt{\mu}}, \quad \forall \boldsymbol{u} \in \mathcal{V}\,.$$

Proof The corollary is immediate from Lemma 7.1, Lemma 7.5, and the observation that from the assumptions we have $\psi_t(\boldsymbol{x}) - \psi_{t+1}(\boldsymbol{x}) \leq 0,\ \forall \boldsymbol{x} \in \mathcal{V}$. □

This might look like the same regret guarantee of OMD; however, here there is a very important difference: the last term contains a time-varying element (η_t), but the domain does not have to be bounded! Also, I used the regularizer $\frac{1}{\eta_{t-1}}\psi(\boldsymbol{x})$ and not $\frac{1}{\eta_t}\psi(\boldsymbol{x})$ to remind you of another important difference: In OMD the learning rate η_t is chosen after receiving information about ℓ_t, through the subgradient $\boldsymbol{g}_t$, while here you have to choose it *before* observing ℓ_t.

7.2.1 Proximal Regularizers

We can obtain a slightly stronger guarantee in the case that the regularizer is **proximal**, that is, it satisfies that $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} \psi_{t+1}(\boldsymbol{x}) - \psi_t(\boldsymbol{x})$ for all t.

Lemma 7.7 *With the notation in Algorithm 7.1, denote by $F_t(\boldsymbol{x}) = \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1}\ell_i(\boldsymbol{x})$ for all t. Assume that $\mathcal{V} \subseteq \mathcal{X}$ is non-empty, closed, and convex. Also, assume that F_{t+1} is closed, subdifferentiable, and λ_{t+1}-strongly convex with respect to $\|\cdot\|$ in $\mathcal{V}$, the regularizer is such that $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} \psi_{t+1}(\boldsymbol{x}) - \psi_t(\boldsymbol{x})$. Then, $\boldsymbol{x}_{t+1}$ exists and is unique. Moreover,*

if $\partial \ell_t(\boldsymbol{x}_t)$ is non-empty, we have

$$\begin{aligned} & F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t) \\ & \quad \leq \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - \frac{\lambda_{t+1}}{2} \|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 + \psi_t(\boldsymbol{x}_t) - \psi_{t+1}(\boldsymbol{x}_t) \\ & \quad \leq \frac{\|\boldsymbol{g}_t\|_\star^2}{2\lambda_{t+1}} + \psi_t(\boldsymbol{x}_t) - \psi_{t+1}(\boldsymbol{x}_t), \quad \forall \boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t) \, . \end{aligned}$$

Proof The existence and uniqueness are given by Theorem 6.9. We have

$$\begin{aligned} & F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t) \\ & \quad = (F_t(\boldsymbol{x}_t) + \ell_t(\boldsymbol{x}_t) + \psi_{t+1}(\boldsymbol{x}_t) - \psi_t(\boldsymbol{x}_t)) - F_{t+1}(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_t) + \psi_t(\boldsymbol{x}_t) \\ & \quad = F_{t+1}(\boldsymbol{x}_t) + \iota_{\mathcal{V}}(\boldsymbol{x}_t) - (F_{t+1}(\boldsymbol{x}_{t+1}) + \iota_{\mathcal{V}}(\boldsymbol{x}_{t+1})) - \psi_{t+1}(\boldsymbol{x}_t) + \psi_t(\boldsymbol{x}_t) \\ & \quad \leq \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - \frac{\lambda_{t+1}}{2} \|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 - \psi_{t+1}(\boldsymbol{x}_t) + \psi_t(\boldsymbol{x}_t), \end{aligned}$$

where in the second inequality we used Lemma 4.2 on the strongly convex function $F_{t+1}+\iota_{\mathcal{V}}$ with $\boldsymbol{g}_t \in \partial(F_{t+1} + \iota_{\mathcal{V}})(\boldsymbol{x}_t)$. Observing that, from the proximal property, we have that $\boldsymbol{x}_t = \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} F_t(\boldsymbol{x}) + \psi_{t+1}(\boldsymbol{x}) - \psi_t(\boldsymbol{x})$, from Theorem 6.27 we have $\mathbf{0} \in \partial(F_t + \iota_{\mathcal{V}} + \psi_{t+1} - \psi_t)(\boldsymbol{x}_t)$. Hence, using Theorem 2.23, and remembering that $F_{t+1}(\boldsymbol{x}) = F_t(\boldsymbol{x}) + \ell_t(\boldsymbol{x}) + \psi_{t+1}(\boldsymbol{x}) - \psi_t(\boldsymbol{x})$, we have that $\partial \ell_t(\boldsymbol{x}_t) \subseteq \partial(F_{t+1} + \iota_{\mathcal{V}})(\boldsymbol{x}_t)$.

The second inequality is obtained observing that $\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle \leq \|\boldsymbol{g}_t\|_\star \|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|$ and using the elementary inequality $ax - \frac{b}{2}x^2 \leq \frac{a^2}{2b}$ for $x \in \mathbb{R}$ and $a, b > 0$. □

Remark 7.8 *Note that a constant regularizer is proximal because any point is the minimizer of the zero function. On the other hand, a constant regularizer makes Lemmas 7.5 and 7.7 the same.*

Example 7.9 *An example of proximal regularization is $\psi_t(\boldsymbol{x}) = \frac{1}{2}\sum_{i=1}^{t-1} \|\boldsymbol{x}_i - \boldsymbol{x}\|_2^2$. It is easy to see that this is $t-1$ strongly convex with respect to the L_2 norm. Using this regularizer with linear losses gives an algorithm known as **FTRL-Proximal**. This algorithm lies in between FTRL and OMD, indeed it solves the issue of being "one-step-behind" compared to OMD explained in the previous section, but now its regret depends on $\frac{1}{2}\sum_{t=1}^{T} \|\boldsymbol{x}_t - \boldsymbol{u}\|_2^2$ that is not easily controlled, unless $\mathcal{V}$ has a bounded diameter. However, the main advantage of FTRL-Proximal is the possibility to deal with non-smooth regularizers (see Section 7.8), which is not possible in OMD.*

7.3 FTRL with Linearized Losses

An important difference between OMD and FTRL is that here the update rule seems way more expensive than in OMD, because we need to solve a convex optimization problem at each step. However, it turns out we can use FTRL on *linearized losses* and obtain the same bound with the same computational complexity as that of OMD.

Consider the case in which the losses are linear, i.e., $\ell_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle, \; t = 1, \ldots, T$, we

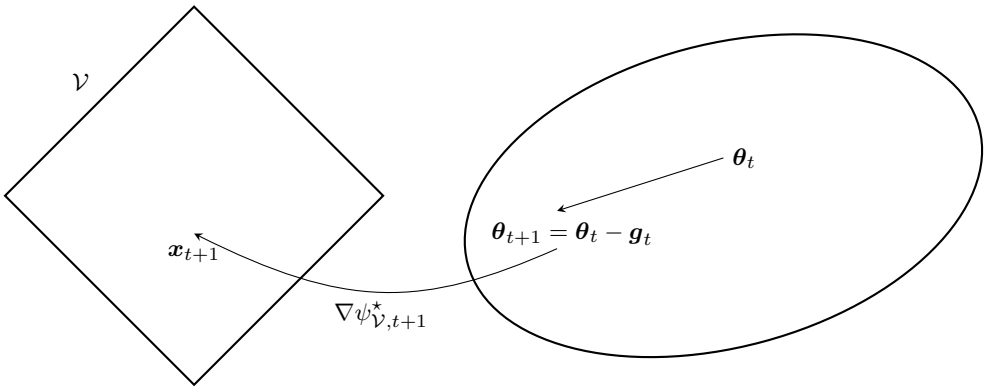


Figure 7.1 Dual map for FTRL with linear losses.

have that the prediction of FTRL is

$$\boldsymbol{x}_{t+1} \in \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \psi_{t+1}(\boldsymbol{x}) + \sum_{i=1}^{t} \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle = \operatorname*{argmax}_{\boldsymbol{x} \in \mathcal{V}} \ \left\langle -\sum_{i=1}^{t} \boldsymbol{g}_i, \boldsymbol{x} \right\rangle - \psi_{t+1}(\boldsymbol{x}) \ .$$

Denote by $\psi_{\mathcal{V},t}(\boldsymbol{x}) := \psi_t(\boldsymbol{x}) + \iota_{\mathcal{V}}(\boldsymbol{x})$. Now, if we assume $\psi_{\mathcal{V},t}$ to be proper, convex, and closed, using Theorem 6.14, we have that $\boldsymbol{x}_{t+1} \in \partial \psi^\star_{\mathcal{V},t+1}(-\sum_{i=1}^{t} \boldsymbol{g}_i)$. Moreover, if $\psi_{\mathcal{V},t+1}$ is also strongly convex, by Theorem 6.26 we know that $\psi^\star_{\mathcal{V},t+1}$ is differentiable and we get

$$\boldsymbol{x}_{t+1} = \nabla \psi^\star_{\mathcal{V},t+1} \left(-\sum_{i=1}^{t} \boldsymbol{g}_i \right) \ . \tag{7.1}$$

In turn, this update can be written in the following way

$$\begin{aligned} \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \boldsymbol{g}_t, \\ \boldsymbol{x}_{t+1} &= \nabla \psi^\star_{\mathcal{V},t+1}(\boldsymbol{\theta}_{t+1}) \ . \end{aligned}$$

This corresponds to Figure 7.1.

Compare it to the mirror update of OMD, rewritten similarly:

$$\begin{aligned} \boldsymbol{\theta}_{t+1} &= \nabla \psi(\boldsymbol{x}_t) - \eta_t \boldsymbol{g}_t, \\ \boldsymbol{x}_{t+1} &= \nabla \psi^\star_{\mathcal{V}}(\boldsymbol{\theta}_{t+1}) \ . \end{aligned}$$

They are very similar, but with important differences:

- In OMD, the state is kept in $\boldsymbol{x}_t$, so we need to use a duality map to transform it into a dual variable before making the update, and then back to the primal variable.
- In FTRL with linear losses, the state is kept directly in the dual space, updated, and then transformed into the primal variable. The primal variable is only used to predict, but not directly in the update.
- In OMD, the samples are weighted by the learning rates, which typically decrease.
- In FTRL with linear losses, all the subgradients have the same weight, but the regularizer is typically increasing over time.

Algorithm 7.2 Follow-the-Regularized-Leader (FTRL) on Linearized Losses

Require: A sequence of regularizers $\psi_1, \dots, \psi_T : \mathcal{X} \to \mathbb{R}$, closed non-empty convex set $\mathcal{V} \subseteq \mathcal{X} \subseteq \mathbb{R}^d$

1: **for** $t = 1$ **to** T **do**
2: Output $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1} \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle$
3: Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable on $\mathcal{V}$
4: Set $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$
5: **end for**

Also, we will not lose anything in the worst-case regret bound! Indeed, we can run **FTRL on linearized losses** $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$, where $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$, guaranteeing exactly the same worst-case upper bound on the regret with the losses ℓ_t. The algorithm for such a procedure is in Algorithm 7.2.

In fact, using the definition of the subgradients and the assumptions of Corollary 7.6, we have

$$\begin{aligned}\operatorname{Regret}_T(\boldsymbol{u}) &= \sum_{t=1}^T (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \sum_{t=1}^T (\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{u})) \\ &\leq \frac{\psi(\boldsymbol{u}) - \min_{\boldsymbol{x} \in \mathcal{V}} \psi(\boldsymbol{x})}{\eta_{T-1}} + \frac{1}{2\mu} \sum_{t=1}^T \eta_{t-1} \|\boldsymbol{g}_t\|_\star^2, \quad \forall \boldsymbol{u} \in \mathcal{V} .\end{aligned}$$

The only difference with respect to Corollary 7.6 is that here the $\boldsymbol{g}_t$ are the specific ones we use in the algorithm, while in Corollary 7.6 the statement holds for any choice of the $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$. Moreover, FTRL with exact losses can adapt to any "nice" properties of the losses, for example, strong convexity, while this information is lost in the linearized case. We also have to remember that this is just a worst-case guarantee: on real problems FTRL with full losses typically performs better than both FTRL with linearized losses and OMD.

More generally, we can also specialize the FTRL regret equality to this case, obtaining the following lemma, which is usually considered a "dual analysis" of FTRL for linear losses.

Lemma 7.10 *Under the assumptions of Lemma 7.1, further assume for all t that $\ell_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$. Then, for any $\boldsymbol{u} \in \mathcal{V}$, we have*

$$\begin{aligned}&\sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \\ &\quad = \psi_{\mathcal{V},T+1}(\boldsymbol{u}) + \psi_{\mathcal{V},1}^\star(\boldsymbol{0}) + \sum_{t=1}^T \left[\psi_{\mathcal{V},t+1}^\star \left(-\sum_{i=1}^t \boldsymbol{g}_i \right) - \psi_{\mathcal{V},t}^\star \left(-\sum_{i=1}^{t-1} \boldsymbol{g}_i \right) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \right] \\ &\qquad - \psi_{\mathcal{V},T+1}^\star \left(-\sum_{i=1}^T \boldsymbol{g}_i \right) - \psi_{\mathcal{V},T+1}(\boldsymbol{u}) - \left\langle \sum_{i=1}^T \boldsymbol{g}_i, \boldsymbol{u} \right\rangle , \end{aligned} \tag{7.2}$$

where $\psi_{\mathcal{V},t}^\star$ is the Fenchel conjugate of $\psi_{\mathcal{V},t} = \psi_t + \iota_{\mathcal{V}}$. Also, by Fenchel–Young's inequality, the sum of the last three terms is non-positive. In addition, the choice of ψ_{T+1} does not change the iterates $\boldsymbol{x}_1, \dots, \boldsymbol{x}_T$ of the algorithm.

Proof The proof is immediate by just summing up the terms on both sides.

We can also obtain it from Lemma 7.1, observing that

$$\begin{aligned} F_t(\boldsymbol{x}_t) &= \min_{\boldsymbol{x}\in\mathcal{V}} \ \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1} \ell_i(\boldsymbol{x}) = -\max_{\boldsymbol{x}\in\mathcal{V}} \left\langle -\sum_{i=1}^{t-1} \boldsymbol{g}_i, \boldsymbol{x} \right\rangle - \psi_t(\boldsymbol{x}) \\ &= -\psi^\star_{\mathcal{V},t}\left(-\sum_{i=1}^{t-1} \boldsymbol{g}_i\right), \end{aligned}$$

and using the fact that $\psi^\star_{\mathcal{V},1}(\mathbf{0}) = -\inf_{\boldsymbol{x}\in\mathbb{R}^d} \ \psi_{\mathcal{V},1}(\boldsymbol{x})$. □

The function $\psi^\star_{\mathcal{V},t}$ in the above lemma is also known as the "potential function" in the online learning literature.

Remark 7.11 *While the regularizer ψ_t in FTRL plays the same role as the distance generating function in OMD, it is important to note that ψ must be differentiable in OMD. Indeed, it is necessary for the existence of the Bregman divergence, and for the uniqueness of the dual map $\nabla\psi$. Instead, in FTRL the dual map $\nabla\psi$ is not used, as it can be seen in Figure 7.1. We will use this property of FTRL in Section 7.8, when we will use regularizers that contain a non-differentiable component. Instead, we can express the above bound using the notion of* ***generalized Bregman divergence*** *$\bar{B}_\psi : \mathbb{R}^d \times \mathbb{R}^d \to [0, +\infty]$ defined as $\bar{B}_\psi(\boldsymbol{x};\boldsymbol{\theta}) = \psi(\boldsymbol{x}) + \psi^\star(\boldsymbol{\theta}) - \langle\boldsymbol{\theta}, \boldsymbol{x}\rangle$. By the Fenchel–Young's inequality, $\bar{B}_\psi(\boldsymbol{x};\boldsymbol{\theta}) \geq 0$. Observe that we do not require ψ to be differentiable. However, if ψ is differentiable, then from Theorem 6.14 we have $B_\psi(\boldsymbol{x};\boldsymbol{y}) = \bar{B}_\psi(\boldsymbol{x};\nabla\psi(\boldsymbol{y}))$. With this definition and assuming that $\inf_{\boldsymbol{x}\in\mathcal{V}} \psi_1(\boldsymbol{x}) = \inf_{\boldsymbol{x}\in\mathcal{V}} \psi_{T+1}(\boldsymbol{x})$, (7.2) becomes*

$$\begin{aligned} \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle = \bar{B}_{\psi_{\mathcal{V},T+1}}(\boldsymbol{u};\mathbf{0}) + \sum_{t=1}^{T} \left[\bar{B}_{\psi_{\mathcal{V},t+1}}(\boldsymbol{u};\boldsymbol{\theta}_{t+1}) - \bar{B}_{\psi_{\mathcal{V},t}}(\boldsymbol{u};\boldsymbol{\theta}_t) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t\rangle\right] \\ - \bar{B}_{\psi_{\mathcal{V},T+1}}(\boldsymbol{u};\boldsymbol{\theta}_{T+1}), \end{aligned}$$

where $\boldsymbol{\theta}_t = -\sum_{i=1}^{t-1} \boldsymbol{g}_i$.

In the next example, we can see the different behavior of FTRL and OMD.

Example 7.12 *Consider $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \leq 1\}$. With Online Subgradient Descent (OSD) with learning rate $\eta_t = \frac{1}{\sqrt{t}}$ and $\boldsymbol{x}_1 = \mathbf{0}$, the update is*

$$\begin{aligned} \tilde{\boldsymbol{x}}_{t+1} &= \boldsymbol{x}_t - \frac{1}{\sqrt{t}}\boldsymbol{g}_t, \\ \boldsymbol{x}_{t+1} &= \tilde{\boldsymbol{x}}_{t+1} \min\left(\frac{1}{\|\tilde{\boldsymbol{x}}_{t+1}\|_2}, 1\right). \end{aligned}$$

On the other hand in FTRL with linearized losses, we can use $\psi_t(\boldsymbol{x}) = \frac{\sqrt{t}}{2}\|\boldsymbol{x}\|_2^2$ and it is easy to verify that the update in (7.1) becomes

$$\begin{aligned} \tilde{\boldsymbol{x}}_{t+1} &= \frac{-\sum_{i=1}^{t} \boldsymbol{g}_i}{\sqrt{t}}, \\ \boldsymbol{x}_{t+1} &= \tilde{\boldsymbol{x}}_{t+1} \min\left(\frac{1}{\|\tilde{\boldsymbol{x}}_{t+1}\|_2}, 1\right). \end{aligned}$$

While the regret guarantee would be the same for these two updates, from an intuitive point of view, OMD seems to be losing a lot of potential information due to the fact that we only memorize the projected iterate.

7.3.1 FTRL with Linearized Losses Can Be Equivalent to OMD

Even if FTRL and OMD seem very different, in certain cases they are actually equivalent. Let's consider an example.

Let $\psi : \mathcal{X} \to \mathbb{R}$ and consider that case that $\mathcal{V} = \mathcal{X} = \operatorname{dom} \psi$. The output of OMD is

$$\boldsymbol{x}_{t+1} = \operatorname*{argmin}_{\boldsymbol{x}} \langle \eta \boldsymbol{g}_t, \boldsymbol{x} \rangle + B_\psi(\boldsymbol{x}; \boldsymbol{x}_t) \; .$$

Assume that $\boldsymbol{x}_{t+1} \in \operatorname{int} \operatorname{dom} \psi$ for all $t = 1, \dots, T$. This implies that $\eta \boldsymbol{g}_t + \nabla \psi(\boldsymbol{x}_{t+1}) - \nabla \psi(\boldsymbol{x}_t) = \boldsymbol{0}$, that is $\nabla \psi(\boldsymbol{x}_{t+1}) = \nabla \psi(\boldsymbol{x}_t) - \eta \boldsymbol{g}_t$. Assuming $\boldsymbol{x}_1 \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \psi(\boldsymbol{x})$, we have

$$\nabla \psi(\boldsymbol{x}_{t+1}) = -\eta \sum_{i=1}^{t} \boldsymbol{g}_i \; .$$

On the other hand, consider FTRL with linearized losses with regularizers $\psi_t = \frac{1}{\eta}\psi$, then

$$\boldsymbol{x}_{t+1} = \operatorname*{argmin}_{\boldsymbol{x}} \; \frac{1}{\eta}\psi(\boldsymbol{x}) + \sum_{i=1}^{t} \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle = \operatorname*{argmin}_{\boldsymbol{x}} \; \psi(\boldsymbol{x}) + \eta \sum_{i=1}^{t} \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle \; .$$

Assuming that $\boldsymbol{x}_{t+1} \in \operatorname{int} \operatorname{dom} \psi$, this implies that $\nabla \psi(\boldsymbol{x}_{t+1}) = -\eta \sum_{i=1}^{t} \boldsymbol{g}_i$. Further, assuming that $\nabla \psi$ is invertible implies that the predictions of FTRL and OMD are the same.

This equivalence immediately gives us some intuition on the role of ψ in both algorithms: the same function is inducing the Bregman divergence, that is, our similarity measure, and is the regularizer in FTRL. Moreover, the inverse of the growth rate over time of the regularizers in FTRL takes the role of the learning rate in OMD.

Example 7.13 *Consider $\psi(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{x}\|_2^2$ and $\mathcal{V} = \mathbb{R}^d$, then it satisfies the conditions above to have the predictions of OMD equal to the ones of FTRL.*

Remark 7.14 *Based on the above observation and on the observations in Section 7.3, a common misunderstanding even among experts is that FTRL and OMD differ* only *in the constrained setting. This is clearly false: the regularizer of FTRL can vary over time in arbitrary ways, not just as a scaled version of a fixed regularizer. Indeed, general time-varying regularizers are used, for example, in parameter-free algorithms (Chapter 13) and in Vovk–Azoury–Warmuth forecaster (Section 7.11).*

7.4 FTRL Regret Bound using Local Norms

In Lemma 7.5, strong convexity basically tells us that the losses plus regularizer have some minimum curvature in all directions. However, as in the OMD case, it turns out that we can get a regret upper bound using again the notion of local norms.

Lemma 7.15 *Under the same assumptions of Lemma 7.1, assume $\psi_1, \dots, \psi_T$ twice differentiable and with the Hessian positive definite in the interior of their domains. Also, assume $\ell_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle,\ t = 1, \dots, T$, for arbitrary vectors $\boldsymbol{g}_t$. Define $\|\boldsymbol{x}\|_{\boldsymbol{A}} := \sqrt{\boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x}}$. For $t = 1, \dots, T$, assume that $\boldsymbol{x}_t \in \operatorname{int} \operatorname{dom} \psi_t$ and that $\tilde{\boldsymbol{x}}_{t+1} := \operatorname{argmin}_{\boldsymbol{x} \in \mathbb{R}^d} \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle + B_{\psi_t}(\boldsymbol{x}; \boldsymbol{x}_t)$ exists and $\tilde{\boldsymbol{x}}_{t+1} \in \operatorname{int} \operatorname{dom} \psi_t$. Then, there exists $\boldsymbol{z}_t$ on the line segment between $\boldsymbol{x}_t$ and $\boldsymbol{x}_{t+1}$ and $\boldsymbol{z}'_t$ on the line segment between $\boldsymbol{x}_t$ and $\tilde{\boldsymbol{x}}_{t+1}$, such that the following inequality holds for any $\boldsymbol{u} \in \mathcal{V}$*

$$\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \le \psi_T(\boldsymbol{u}) - \min_{\boldsymbol{x} \in \mathcal{V}} \psi_1(\boldsymbol{x}) + \sum_{t=1}^{T-1} [\psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1})] + \sum_{t=1}^{T} \min \left(\frac{\|\boldsymbol{g}_t\|^2_{(\nabla^2 \psi_t(\boldsymbol{z}_t))^{-1}}}{2}, \frac{\|\boldsymbol{g}_t\|^2_{(\nabla^2 \psi_t(\boldsymbol{z}'_t))^{-1}}}{2} \right)$$

Proof We start from Lemma 7.1 with $\psi_{T+1} = \psi_T$ and observe that

$$\begin{aligned} &F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t) \\ &\quad = F_t(\boldsymbol{x}_t) - F_t(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{x}_{t+1}) + \psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1}) \,. \end{aligned}$$

Then, observe that ψ_t are strictly convex because the Hessians are positive definite. Hence, they can be used to define Bregman divergences. Moreover, from the optimality condition of Theorem 2.8 for $\boldsymbol{x}_t$, we have

$$\langle \nabla F_t(\boldsymbol{x}_t), \boldsymbol{v} - \boldsymbol{x}_t \rangle \ge 0, \quad \forall \boldsymbol{v} \in \mathcal{V} \,.$$

Hence, in particular, we have

$$\langle \nabla F_t(\boldsymbol{x}_t), \boldsymbol{x}_{t+1} - \boldsymbol{x}_t \rangle \ge 0 \,.$$

Using this inequality, we have

$$B_{F_t}(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) = F_t(\boldsymbol{x}_{t+1}) - F_t(\boldsymbol{x}_t) - \langle \nabla F_t(\boldsymbol{x}_t), \boldsymbol{x}_{t+1} - \boldsymbol{x}_t \rangle \le F_t(\boldsymbol{x}_{t+1}) - F_t(\boldsymbol{x}_t) \,.$$

This last inequality implies that

$$\begin{aligned} F_t(\boldsymbol{x}_t) - F_t(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{x}_{t+1}) &= F_t(\boldsymbol{x}_t) - F_t(\boldsymbol{x}_{t+1}) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle \\ &\le \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - B_{F_t}(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) \,. \end{aligned}$$

Bregman divergences are independent of linear terms, so $B_{F_t}(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) = B_{\psi_t}(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t)$. From the Taylor's theorem, $B_{\psi_t}(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) = \frac{1}{2}(\boldsymbol{x}_{t+1} - \boldsymbol{x}_t)^\top \nabla^2 \psi_t(\boldsymbol{z}_t)(\boldsymbol{x}_{t+1} - \boldsymbol{x}_t)$, where $\boldsymbol{z}_t$ is on the line segment between $\boldsymbol{x}_t$ and $\boldsymbol{x}_{t+1}$. Observe that this is $\frac{1}{2}\|\boldsymbol{x}_{t+1} - \boldsymbol{x}_t\|^2_{\nabla^2 \psi_t(\boldsymbol{z}_t)}$ and it is indeed a norm because we assumed the Hessian of ψ_t to be positive definite. Hence, by Fenchel–Young's inequality and Examples 4.19 and 6.19, we have

$$\begin{aligned} &F_t(\boldsymbol{x}_t) - F_t(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{x}_{t+1}) \\ &\quad \le \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - B_{\psi_t}(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) \qquad (7.3) \\ &\quad \le \frac{1}{2}\|\boldsymbol{g}_t\|^2_{(\nabla^2 \psi_t(\boldsymbol{z}_t))^{-1}} + \frac{1}{2}(\boldsymbol{x}_{t+1} - \boldsymbol{x}_t)^\top \nabla^2 \psi_t(\boldsymbol{z}_t)(\boldsymbol{x}_{t+1} - \boldsymbol{x}_t) - B_{\psi_t}(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) \\ &\quad = \frac{1}{2}\|\boldsymbol{g}_t\|^2_{(\nabla^2 \psi_t(\boldsymbol{z}_t))^{-1}}, \end{aligned}$$

that gives the first term in the minimum.

For the second term in the minimum, we start from (7.3) to get

$$\begin{aligned}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - B_{\psi_t}(\boldsymbol{x}_{t+1}; \boldsymbol{x}_t) &\leq \max_{\boldsymbol{x}\in\mathbb{R}^d} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}\rangle - B_{\psi_t}(\boldsymbol{x}; \boldsymbol{x}_t) \\ &= \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \tilde{\boldsymbol{x}}_{t+1}\rangle - B_{\psi_t}(\tilde{\boldsymbol{x}}_{t+1}; \boldsymbol{x}_t)\ .\end{aligned}$$

Then, we proceed as in the first bound. □

Observe that $\boldsymbol{z}'_t$ is defined as a sort of OMD update using ψ_t as distance generating function.

7.5 Example of FTRL: Exponentiated Gradient without Knowing T

As we did in Section 6.6 for OMD, let's see an example of an instantiation of FTRL with linearized losses to have the FTRL version of Exponentiated Gradient (EG).

Let $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_1 = 1, x_i \geq 0\}$ and the sequence of loss functions $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ be convex, subdifferentiable on $\mathcal{V}$, and L_∞-Lipschitz with respect to the L_1 norm. Let $\psi : R^d_{\geq 0} \to \mathbb{R}$ be defined as $\psi(\boldsymbol{x}) = \sum_{i=1}^d x_i \ln x_i$, where we define $0 \ln 0 = 0$. Set $\psi_t(\boldsymbol{x}) = \alpha L_\infty \sqrt{t}\psi(\boldsymbol{x})$, that is $\alpha L_\infty \sqrt{t}$-strongly convex with respect to the L_1 norm by Lemma 6.33, where $\alpha > 0$ is a parameter of the algorithm.

Given that the regularizers are strongly convex and defining $\psi_{\mathcal{V},t} = \psi_t + \iota_{\mathcal{V}}$, from (7.1) we have

$$\boldsymbol{x}_t = \nabla \psi^\star_{\mathcal{V},t}\left(-\sum_{i=1}^{t-1} \boldsymbol{g}_i\right)\ .$$

We already saw in Section 6.6 that $\psi^\star_{\mathcal{V}}(\boldsymbol{\theta}) = \ln\left(\sum_{i=1}^d \exp(\theta_i)\right)$, that implies that $\psi^\star_{\mathcal{V},t}(\boldsymbol{\theta}) = \alpha L_\infty \sqrt{t} \ln\left(\sum_{i=1}^d \exp\left(\frac{\theta_i}{\alpha L_\infty \sqrt{t}}\right)\right)$. So, running FTRL with linearized losses, we have that

$$x_{t,j} = \frac{\exp\left(-\frac{\sum_{k=1}^{t-1} g_{k,j}}{\alpha L_\infty \sqrt{t}}\right)}{\sum_{i=1}^d \exp\left(-\frac{\sum_{k=1}^{t-1} g_{k,i}}{\alpha L_\infty \sqrt{t}}\right)}, \quad \forall j = 1, \dots, d,$$

where $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$. Note that this is exactly the same update of EG based on OMD, but here we are effectively using time-varying learning rates.

We also get that the regret guarantee is

$$\begin{aligned}\sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}) &\leq L_\infty \sqrt{T}\alpha\left(\sum_{i=1}^d u_i \ln u_i + \ln d\right) + \frac{1}{2\alpha L_\infty}\sum_{t=1}^T \frac{\|\boldsymbol{g}_t\|_\infty^2}{\sqrt{t}} \qquad (7.4)\\ &\leq L_\infty \sqrt{T}\left(\alpha\left(\sum_{i=1}^d u_i \ln u_i + \ln d\right) + \frac{1}{\alpha}\right)\\ &\leq L_\infty \sqrt{T}\left(\alpha \ln d + \frac{1}{\alpha}\right), \quad \forall \boldsymbol{u} \in \mathcal{V},\end{aligned}$$

where we used the fact that using $\psi_t(\boldsymbol{x}) = \alpha L_\infty \sqrt{t}\psi(\boldsymbol{x})$ and $\psi_t(\boldsymbol{x}) = \alpha L_\infty \sqrt{t}(\psi(\boldsymbol{x}) -$

$\min_{\boldsymbol{x}} \psi(\boldsymbol{x})$) give the same iterates in FTRL. The optimal choice of α to minimize the regret upper bound is $\frac{1}{\sqrt{\ln d}}$. This regret guarantee is similar to the one we proved for OMD, but with an important difference: we do not have to know in advance the number of rounds T. In OMD, a similar bound would be vacuous because it would depend on the $\max_{\boldsymbol{u},\boldsymbol{x}\in\mathcal{V}} B_\psi(\boldsymbol{u};\boldsymbol{x})$ that is infinite.

As we did in the OMD case, we can also get a bound using the local norms. Let's use the additional assumption that $g_{t,i} \geq 0$, for all $t = 1,\dots,T$ and $i = 1,\dots,d$. Using Lemma 7.15, we have for all $\boldsymbol{u} \in \mathcal{V}$ that

$$\begin{aligned}\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) &\leq L_\infty \sqrt{T}\alpha \left(\sum_{i=1}^{d} u_i \ln u_i + \ln d\right) + \frac{1}{2}\sum_{t=1}^{T} \|\boldsymbol{g}_t\|^2_{(\nabla^2\psi_t(\boldsymbol{z}'_t))^{-1}} \\ &= L_\infty \sqrt{T}\alpha \left(\sum_{i=1}^{d} u_i \ln u_i + \ln d\right) + \frac{1}{2\alpha L_\infty}\sum_{t=1}^{T} \frac{\|\boldsymbol{g}_t\|^2_{(\nabla^2\psi(\boldsymbol{z}'_t))^{-1}}}{\sqrt{t}},\end{aligned}$$

where $\boldsymbol{z}'_t$ is on the line segment between $\boldsymbol{x}_t$ and $\tilde{\boldsymbol{x}}_{t+1}$. In this case, it is easy to calculate $\tilde{x}_{t+1,i}$ as $x_{t,i}\exp(-\frac{g_{t,i}}{\alpha L_\infty \sqrt{t}})$ for $i = 1,\dots,d$. Moreover, $\nabla^2\psi(\boldsymbol{z}'_t)$ is a diagonal matrix whose elements on the diagonal are $\frac{1}{z'_{t,i}}, i = 1,\dots,d$. Hence, we have that

$$\|\boldsymbol{g}_t\|^2_{(\nabla^2\psi(\boldsymbol{z}'_t))^{-1}} = \sum_{i=1}^{d} g_{t,i}^2 z'_{t,i} \leq \sum_{i=1}^{d} g_{t,i}^2 x_{t,i}, \tag{7.5}$$

that is less than or equal to the terms $\|\boldsymbol{g}_t\|^2_\infty$ we have in (7.4).

7.6 Example of FTRL: AdaHedge

In this section, we explain a variation of the EG algorithm, called *AdaHedge*. The basic idea is to design an algorithm that is adaptive to the sum of the squared L_∞ norm of the losses, without any prior information on the range of the losses.

First, consider the case in which we use as a constant regularizer the negative Shannon entropy $\psi_t(\boldsymbol{x}) = \lambda \sum_{i=1}^{d} x_i \ln x_i$ defined over $\mathbb{R}^d_{\geq 0}$, where $\lambda > 0$ will be determined in the following. Set $\mathcal{V} = \Delta^{d-1}$, the probability simplex in $\mathbb{R}^d$. Using FTRL with linear losses with this regularizer, for any $\boldsymbol{u} \in \Delta^{d-1}$, we immediately obtain

$$\begin{aligned}\mathrm{Regret}_T(\boldsymbol{u}) &\leq \lambda\left(\ln d + \sum_{i=1}^{d} u_i \ln u_i\right) + \sum_{t=1}^{T}\left(F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t\rangle\right) \\ &\leq \lambda \ln d + \sum_{t=1}^{T}\left(F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t\rangle\right),\end{aligned}$$

where we upper bounded the negative entropy of $\boldsymbol{u}$ with 0. Using the strong convexity of the regularizer with respect to the L_1 norm and Lemma 7.5, we further upper bound it as

$$\mathrm{Regret}_T(\boldsymbol{u}) \leq \lambda \ln d + \sum_{t=1}^{T} \frac{\|\boldsymbol{g}_t\|^2_\infty}{2\lambda}\,.$$

This suggests that the optimal λ should be $\lambda = \sqrt{\frac{\sum_{t=1}^{T}\|\boldsymbol{g}_t\|^2_\infty}{2\ln d}}$. However, as we have seen

in Section 4.2, this choice is not feasible. Hence, exactly as we did in Section 4.2, we might think of using an online version of this choice

$$\psi_t(\boldsymbol{x}) = \lambda_t \sum_{i=1}^{d} x_i \ln x_i \quad \text{where} \quad \lambda_t = \frac{1}{\alpha}\sqrt{\sum_{i=1}^{t-1} \|\boldsymbol{g}_i\|_\infty^2}, \tag{7.6}$$

where $\alpha > 0$ is a constant that will be determined later. An important property of such a choice is that it gives rise to an algorithm that is scale-free (Definition 4.33), that is, its predictions $\boldsymbol{x}_t$ are invariant to the scaling of the losses by any constant factor. This is easy to see because

$$x_{t,j} \propto \exp\left(-\frac{\alpha \sum_{i=1}^{t-1} g_{i,j}}{\sqrt{\sum_{i=1}^{t-1} \|\boldsymbol{g}_i\|_\infty^2}}\right), \quad \forall j = 1, \dots, d\,.$$

This choice makes the regularizer non-decreasing over time and immediately gives us

$$\text{Regret}_T(\boldsymbol{u}) \leq \lambda_T \ln d + \sum_{t=1}^{T} \frac{\|\boldsymbol{g}_t\|_\infty^2}{2\lambda_t} = \frac{\ln d}{\alpha}\sqrt{\sum_{t=1}^{T} \|\boldsymbol{g}_t\|_\infty^2} + \alpha \sum_{t=1}^{T} \frac{\|\boldsymbol{g}_t\|_\infty^2}{2\sqrt{\sum_{i=1}^{t-1} \|\boldsymbol{g}_i\|_\infty^2}}\,.$$

At this point, we might be tempted to use Lemma 4.13 to upper bound the last sum in the upper bound, but unfortunately, we cannot! Indeed, the denominator does not contain the term $\|\boldsymbol{g}_t\|_\infty^2$. We might add a constant to λ_t, but that would destroy the scale-freeness of the algorithm. However, it turns out that we can still prove our bound without any change to the regularizer. The key observation is that we can bound the term $F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle$ in two different ways. One way is using Lemma 7.5. The other one is

$$\begin{aligned} F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle &\leq F_t(\boldsymbol{x}_{t+1}) - F_{t+1}(\boldsymbol{x}_{t+1}) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \\ &= \psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1}) - \langle \boldsymbol{g}_t, \boldsymbol{x}_{t+1} \rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \\ &\leq -\langle \boldsymbol{g}_t, \boldsymbol{x}_{t+1} \rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \leq 2\|\boldsymbol{g}_t\|_\infty, \end{aligned}$$

where we used the definition of $\boldsymbol{x}_{t+1}$ and the fact that the regularizer is non-decreasing over time. So, we can now write

$$\begin{aligned} \sum_{t=1}^{T} F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle &\leq \sum_{t=1}^{T} \min\left(\frac{\alpha\|\boldsymbol{g}_t\|_\infty^2}{2\sqrt{\sum_{i=1}^{t-1} \|\boldsymbol{g}_i\|_\infty^2}}, 2\|\boldsymbol{g}_t\|_\infty\right) \\ &= 2\sum_{t=1}^{T} \sqrt{\min\left(\frac{\alpha^2\|\boldsymbol{g}_t\|_\infty^4}{16\sum_{i=1}^{t-1} \|\boldsymbol{g}_i\|_\infty^2}, \|\boldsymbol{g}_t\|_\infty^2\right)} \\ &\leq 2\sum_{t=1}^{T} \sqrt{\frac{2}{\frac{16\sum_{i=1}^{t-1} \|\boldsymbol{g}_i\|_\infty^2}{\alpha^2\|\boldsymbol{g}_t\|_\infty^4} + \frac{1}{\|\boldsymbol{g}_t\|_\infty^2}}} \\ &= 2\sum_{t=1}^{T} \sqrt{2}\frac{\alpha\|\boldsymbol{g}_t\|_\infty^2}{\sqrt{\alpha^2\|\boldsymbol{g}_t\|_\infty^2 + 16\sum_{i=1}^{t-1} \|\boldsymbol{g}_i\|_\infty^2}}, \end{aligned}$$

where we used the fact that the minimum between two numbers is less than their harmonic mean. Assuming $\alpha \geq 4$ and using Lemma 4.13, we have

$$\sum_{t=1}^{T} F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \leq \frac{\sqrt{2}}{2} \sum_{t=1}^{T} \frac{\alpha \|\boldsymbol{g}_t\|_\infty^2}{\sqrt{\sum_{i=1}^{t} \|\boldsymbol{g}_i\|_\infty^2}} \leq \alpha \sqrt{2 \sum_{t=1}^{T} \|\boldsymbol{g}_t\|_\infty^2}$$

and

$$\text{Regret}_T(\boldsymbol{u}) \leq \left(\frac{\ln d}{\alpha} + \alpha\sqrt{2} \right) \sqrt{\sum_{t=1}^{T} \|\boldsymbol{g}_t\|_\infty^2}, \quad \forall \boldsymbol{u} \in \Delta^{d-1} \,. \tag{7.7}$$

The bound and the assumption on α suggest to set $\alpha = \max(4, 2^{-1/4}\sqrt{\ln d})$.

We might consider ourselves happy, but there is a clear problem in the above algorithm: the choice of λ_t in the time-varying regularizer strictly depends on our upper bound. So, a loose bound will result in a poor choice of the regularization! In general, every time we use a part of the proof in the design of an algorithm, we cannot expect an exciting empirical performance, unless our upper bound is really tight. So, can we design a better regularizer? Well, we need a better upper bound!

Let's consider a generic regularizer $\psi_t(\boldsymbol{x}) = \lambda_t \psi(\boldsymbol{x})$ and its corresponding FTRL with linear losses regret upper bound

$$\text{Regret}_T(\boldsymbol{u}) \leq \lambda_T(\psi(\boldsymbol{u}) - \inf_{\boldsymbol{x} \in \mathcal{V}} \psi(\boldsymbol{x})) + \sum_{t=1}^{T} \left(F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \right),$$

where we assume λ_t to be non-decreasing in time.

Now, observe that the sum is unlikely to disappear for this kind of algorithm, so we could try to make the term $\lambda_T(\psi(\boldsymbol{u}) - \inf_{\boldsymbol{x} \in \mathcal{V}} \psi(\boldsymbol{x}))$ of the same order as the sum. So, we would like to set λ_t of the same order of $\sum_{i=1}^{t} \left(F_i(\boldsymbol{x}_i) - F_{i+1}(\boldsymbol{x}_{i+1}) + \langle \boldsymbol{g}_i, \boldsymbol{x}_i \rangle \right)$. However, this approach would cause an annoying recurrence. So, using the fact that λ_t is non-decreasing, let's upper bound the terms in the sum just a little bit:

$$\begin{aligned}
&F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \\
&\quad = F_t(\boldsymbol{x}_t) - \lambda_{t+1}\psi(\boldsymbol{x}_{t+1}) - \sum_{i=1}^{t} \langle \boldsymbol{g}_i, \boldsymbol{x}_{t+1} \rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \\
&\quad \leq F_t(\boldsymbol{x}_t) - \lambda_t\psi(\boldsymbol{x}_{t+1}) - \sum_{i=1}^{t} \langle \boldsymbol{g}_i, \boldsymbol{x}_{t+1} \rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \\
&\quad \leq F_t(\boldsymbol{x}_t) - \min_{\boldsymbol{x} \in \mathcal{V}} \left(\lambda_t\psi(\boldsymbol{x}) + \sum_{i=1}^{t} \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle \right) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle := \delta_t \,.
\end{aligned}$$

Now, we can set $\lambda_t = \frac{1}{\alpha^2} \sum_{i=1}^{t-1} \delta_i$ for $t \geq 2$, $\lambda_1 = 0$, and $\boldsymbol{x}_1 = \text{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \psi(\boldsymbol{x})$. This immediately implies that

$$\text{Regret}_T(\boldsymbol{u}) \leq \left(\psi(\boldsymbol{u}) - \inf_{\boldsymbol{x} \in \mathcal{V}} \psi(\boldsymbol{x}) + \alpha^2 \right) \lambda_{T+1} \,.$$

Setting ψ to be equal to the negative entropy, we get an algorithm known as **AdaHedge**.

Algorithm 7.3 AdaHedge

Require: $\alpha > 0$

$\lambda_1 = 0$

$\boldsymbol{x}_1 = [1/d, \dots, 1/d] \in \Delta^{d-1}$

$\boldsymbol{\theta}_1 = \mathbf{0} \in \mathbb{R}^d$

for $t = 1$ **to** T **do**

  Output $\boldsymbol{x}_t$

  Receive $\boldsymbol{g}_t \in \mathbb{R}^d$ and pay $\langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle$

  Update $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \boldsymbol{g}_t$

  Set $\delta_t = \begin{cases} -\max_{j=1,\dots,d} \theta_{t,j} + \max_{j=1,\dots,d} \theta_{t+1,j} + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle, & \lambda_t = 0 \\ \lambda_t \ln \left(\sum_{j=1}^d x_{t,j} \exp\left(-g_{t,j}/\lambda_t\right) \right) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle, & \text{otherwise} \end{cases}$

  Update $\lambda_{t+1} = \lambda_t + \frac{1}{\alpha^2}\delta_t$

  Set $\boldsymbol{x}_{t+1} = \begin{cases} [1/d, \dots, 1/d], & \lambda_{t+1} = 0 \\ x_{t+1,j} \propto \exp\left(\frac{\theta_{t+1,j}}{\lambda_{t+1}}\right), j = 1, \dots, d, & \text{otherwise} \end{cases}$

end for

With this choice of the regularizer, we can simplify a bit the expression of δ_t. For $\lambda_t = 0$, we have $\delta_t = -\max_{j=1,\dots,d} \theta_{t,j} + \max_{j=1,\dots,d} \theta_{t+1,j} + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle$. Instead, for $\lambda_t > 0$, using the properties of the Fenchel conjugates, we have that

$$\delta_t = \lambda_t \ln \frac{\sum_{j=1}^d \exp\left(\theta_{t+1,j}/\lambda_t\right)}{\sum_{j=1}^d \exp\left(\theta_{t,j}/\lambda_t\right)} + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle = \lambda_t \ln \left(\sum_{j=1}^d x_{t,j} \exp\left(-g_{t,j}/\lambda_t\right) \right) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \,.$$

Overall, we get the pseudo-code of AdaHedge in Algorithm 7.3.

So, now we need an upper bound for λ_{T+1}. Observe that $\lambda_{t+1} = \lambda_t + \frac{1}{\alpha^2}\delta_t$. Moreover, as we have done before, we can upper bound δ_t in two different ways. In fact, from Lemma 7.5, we have $\delta_t \leq \frac{\|\boldsymbol{g}_t\|_\infty^2}{2\lambda_t}$ whenever $\lambda_t > 0$. Also, denoting by $\tilde{\boldsymbol{x}}_{t+1} = \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \lambda_t \psi(\boldsymbol{x}) + \sum_{i=1}^t \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle$, we have

$$\begin{aligned}
\delta_t &= F_t(\boldsymbol{x}_t) - \min_{\boldsymbol{x} \in \mathcal{V}} \left(\lambda_t \psi(\boldsymbol{x}) + \sum_{i=1}^t \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle \right) + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \\
&= F_t(\boldsymbol{x}_t) - \lambda_t \psi(\tilde{\boldsymbol{x}}_{t+1}) - \sum_{i=1}^t \langle \boldsymbol{g}_i, \tilde{\boldsymbol{x}}_{t+1} \rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \\
&\leq F_t(\tilde{\boldsymbol{x}}_{t+1}) - \lambda_t \psi(\tilde{\boldsymbol{x}}_{t+1}) - \sum_{i=1}^t \langle \boldsymbol{g}_i, \tilde{\boldsymbol{x}}_{t+1} \rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \\
&= -\langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_{t+1} \rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \leq 2\|\boldsymbol{g}_t\|_\infty \,.
\end{aligned}$$

Hence, we have

$$\begin{aligned}\lambda_1 &= 0,\\ \lambda_2 &\le \frac{2\|\boldsymbol{g}_1\|_\infty}{\alpha^2},\\ \lambda_{t+1} = \lambda_t + \frac{\delta_t}{\alpha^2} &\le \lambda_t + \frac{1}{\alpha^2}\min\left(2\|\boldsymbol{g}_t\|_\infty, \frac{\|\boldsymbol{g}_t\|_\infty^2}{2\lambda_t}\right), \quad \forall t \ge 1,\end{aligned}$$

where we use the convention that $\frac{\|\boldsymbol{g}_t\|_\infty^2}{2\lambda_t} = +\infty$ when $\lambda_t = 0$. We can solve this recurrence using the following lemma, where $\Delta_t = \lambda_{t+1}$ and $a_t = \|\boldsymbol{g}_t\|_\infty$.

Lemma 7.16 *Let $\{a_t\}_{t=1}^\infty$ be any sequence of non-negative real numbers. Suppose that $\{\Delta_t\}_{t=0}^\infty$ is a non-decreasing sequence of non-negative real numbers satisfying*

$$\Delta_0 = 0 \qquad \textit{and} \qquad \Delta_t \le \Delta_{t-1} + \min\left\{b a_t,\ c\frac{a_t^2}{2\Delta_{t-1}}\right\} \quad \textit{for any } t \ge 1\,.$$

When $\Delta_{t-1} = 0$, we use the convention $\frac{a_t^2}{2\Delta_{t-1}} = +\infty$. Then, for any $T \ge 0$, $\Delta_T \le \sqrt{(b^2+c)\sum_{t=1}^T a_t^2}$.

Proof Observe that

$$\Delta_T^2 = \sum_{t=1}^T (\Delta_t^2 - \Delta_{t-1}^2) = \sum_{t=1}^T \left[(\Delta_t - \Delta_{t-1})^2 + 2(\Delta_t - \Delta_{t-1})\Delta_{t-1}\right]\,.$$

We bound each term in the sum separately. The left term of the minimum inequality in the definition of Δ_t gives $(\Delta_t - \Delta_{t-1})^2 \le b^2 a_t^2$, while the right term gives $2(\Delta_t - \Delta_{t-1})\Delta_{t-1} \le c a_t^2$. The latter inequality is also true when $\Delta_{t-1} = 0$. So, we conclude $\Delta_T^2 \le (b^2+c)\sum_{t=1}^T a_t^2$. □

Overall, for any $\boldsymbol{u} \in \Delta^{d-1}$, we got

$$\begin{aligned}\mathrm{Regret}_T(\boldsymbol{u}) &\le \left(\psi(\boldsymbol{u}) - \inf_{\boldsymbol{x}\in\mathcal{V}}\psi(\boldsymbol{x}) + \alpha^2\right)\lambda_{T+1}\\ &\le \left(\frac{\psi(\boldsymbol{u}) - \inf_{\boldsymbol{x}\in\mathcal{V}}\psi(\boldsymbol{x})}{\alpha^2} + 1\right)\sqrt{(4+\alpha^2)\sum_{t=1}^T \|\boldsymbol{g}_t\|_\infty^2},\end{aligned}$$

and setting $\alpha = \sqrt{\ln d}$, we have

$$\mathrm{Regret}_T(\boldsymbol{u}) \le 2\sqrt{(4+\ln d)\sum_{t=1}^T \|\boldsymbol{g}_t\|_\infty^2}\,.$$

Note that this is roughly the same regret in (7.7), but the very important difference is that this new regret bound depends on the much tighter quantity λ_{T+1}, that we upper bounded with a term proportional to $\sqrt{\sum_{t=1}^T \|\boldsymbol{g}_t\|_\infty^2}$, but it can be much smaller than that. For example, λ_{T+1} could be upper bounded using the tighter local norms, see Section 7.5. Instead, in

the first solution, the regret will always be dominated by the term $\sqrt{\sum_{t=1}^T \|\boldsymbol{g}_t\|_\infty^2}$ because we explicitly use it in the regularizer.

Remark 7.17 *There is an important lesson to be learned from AdaHedge: the regret is not the full story, and algorithms with the same worst-case guarantee can exhibit vastly different empirical behaviours. Unfortunately, this message is rarely heard, and sometimes one focuses too much on the worst-case guarantee rather than on the empirical performance. Even worse, sometimes people favor algorithms with a "more elegant analysis" completely ignoring the likely worse empirical performance.*

7.7 Example of FTRL: Group Norms

Suppose you want to do linear regression and you have n different groups of features in $\mathbb{R}^d$ associated with each sample. For example, they could represent different modalities (audio, video, etc.) associated with the same sample. You could just concatenate all the features in a long vector and learn with them using FTRL with a squared L_2 regularizer. Alternatively, we could use FTRL with p-norm (see Problem 7.8), and tune p to have a logarithmic dependence on $n\,d$. However, there is something in between: we may want to learn a linear combination of predictors, one for each group, trying to use a very large number of groups and paying a small price for it. In particular, we might not want d to appear in the bound because it might be arbitrarily large, potentially infinity. This is the equivalent of the Learning with Expert Advice (LEA) setting, where each expert is now a linear predictor. In fact, one might be tempted to use one online learning algorithm on each group and a LEA algorithm on top that combines the predictions. However, this approach is very clunky because the LEA algorithm and the linear predictors are learning somehow independently, hence we can do much better. In fact, we can use an L_2 norm over each group and a p-norm over the groups with p very close to 1. In this way, we expect a regret upper bound that depends on the maximum L_2 norm of the subgradients with respect to each single group, but no explicit dependence on d. Let's see how this would work.

First, we introduce the definition of *group norms*.

Definition 7.18 *Let $\boldsymbol{X} = \begin{bmatrix}\boldsymbol{x}_1 & \boldsymbol{x}_2 & \dots & \boldsymbol{x}_n\end{bmatrix}$ be a $d \times n$ real matrix with columns $\boldsymbol{x}_i \in \mathbb{R}^d$. Given norms Ψ and Φ on $\mathbb{R}^d$ and $\mathbb{R}^n$, we define the* ***group norm*** *$\|\boldsymbol{X}\|_{\Psi,\Phi}$ as*

$$\|\boldsymbol{X}\|_{\Psi,\Phi} := \Phi([\Psi(\boldsymbol{x}_1), \dots, \Psi(\boldsymbol{x}_n)]) \,.$$

It is possible to check that this is indeed a norm, and we can also calculate the dual norm. We will need an additional definition.

Definition 7.19 *We say that $f : \mathbb{R}^d \to \mathbb{R}$ is* ***absolutely symmetric*** *if it is invariant under permutations and sign changes of the components of its input.*

Lemma 7.20 (Kakade et al., 2009, Lemma 17) *Let Φ be an absolutely symmetric norm on $\mathbb{R}^n$. Then,*

$$(\|\cdot\|_{\Psi,\Phi})_\star = \|\cdot\|_{\Psi^\star,\Phi^\star} \,.$$

Algorithm 7.4 FTRL with Group Norms for Linear Regression

Require: $q \geq 2, \alpha > 0$

$\boldsymbol{\theta}_{1,i} = \boldsymbol{x}_{1,i} = \mathbf{0} \in \mathbb{R}^d,\ i = 1, \dots, n$

for $t = 1$ **to** T **do**

  Receive matrix of features $\boldsymbol{Z}_t = \begin{bmatrix} \boldsymbol{z}_{t,1} & \dots & \boldsymbol{z}_{t,n} \end{bmatrix}$

  Predict $\hat{y}_t = \sum_{i=1}^n \boldsymbol{z}_{t,i}^\top \boldsymbol{x}_{t,i}$

  Receive label y_t and pay $|\hat{y}_t - y_t|$

  Update $\boldsymbol{\theta}_{t+1,i} = \boldsymbol{\theta}_{t,i} - \boldsymbol{z}_{t,i}\mathrm{sign}(\hat{y}_t - y_t),\ i = 1, \dots, n$

  Update $\boldsymbol{x}_{t+1,i} = \begin{cases} \mathbf{0}, & \boldsymbol{\theta}_{t+1,j} = \mathbf{0},\ \forall j, \\ \frac{\boldsymbol{\theta}_{t+1,i}\|\boldsymbol{\theta}_{t+1,i}\|_2^{q-2}}{\alpha\sqrt{t+1}(\sum_{j=1}^n \|\boldsymbol{\theta}_{t+1,j}\|_2^q)^{1-2/q}}, & \text{otherwise}, \end{cases} \quad i = 1, \dots, n$

end for

From this lemma and Example 6.19, we immediately have that the Fenchel conjugate of $\frac{1}{2}\|\cdot\|_{\Psi,\Phi}^2$ is $\frac{1}{2}\|\cdot\|_{\Psi^\star,\Phi^\star}^2$.

We can also construct strongly convex functions based on group norms.

Theorem 7.21 (Kakade et al., 2009, Theorem 18) *Let Ψ, Φ be absolutely symmetric norms on $\mathbb{R}^d$, $\mathbb{R}^n$. Let $\Phi^2 \circ \sqrt{} : \mathbb{R}^n \to (-\infty, +\infty]$ denote the following function,*

$$(\Phi^2 \circ \sqrt{})(\boldsymbol{x}) := \Phi^2(\sqrt{x_1}, \dots, \sqrt{x_n})\ .$$

Suppose, $\Phi^2 \circ \sqrt{}$ satisfies the properties of a norm on $\mathbb{R}^n_{\geq 0}$. Further, let the functions Ψ^2 and Φ^2 be λ_1- and λ_2-smooth with respect to Ψ and Φ respectively. Then, $\|\cdot\|_{\Psi,\Phi}^2$ is $(\lambda_1 + \lambda_2)$-smooth with respect to $\|\cdot\|_{\Psi,\Phi}$.

We can now specialize this result to *group p-norms*.

Corollary 7.22 *Let $q, s \geq 2$. The function $\frac{1}{2}\|\cdot\|_{q,s}^2$ is $(q + s - 2)$-smooth with respect to $\|\cdot\|_{q,s}$ on $\mathbb{R}^{d\times n}$.*

Assuming $n \geq 2$, the above suggests to use FTRL with linearized losses and regularizer $\psi_t(\boldsymbol{X}) = \frac{\alpha\sqrt{t}}{2}\|\boldsymbol{X}\|_{2,p}^2$, where $1 < p \leq 2$ and $\alpha > 0$ will be decided in the following. From the above and Example 6.19, the dual is $\psi_t^\star(\boldsymbol{\Theta}) = \frac{1}{2\alpha\sqrt{t}}\|\boldsymbol{\Theta}\|_{2,q}^2$ and it is $\frac{q}{\alpha\sqrt{t}}$-smooth, where $\frac{1}{p} + \frac{1}{q} = 1$. Moreover,

$$[\nabla\psi_t^\star(\boldsymbol{\Theta})]_i = \frac{\boldsymbol{\theta}_i\|\boldsymbol{\theta}_i\|_2^{q-2}}{\alpha\sqrt{t}(\sum_{j=1}^n \|\boldsymbol{\theta}_j\|_2^q)^{1-2/q}}, \quad i = 1, \dots, n\ .$$

For simplicity, we will use the absolute loss as a loss function. We are now ready to introduce Algorithm 7.4.

From the analysis of FTRL with linearized losses, the properties of the group norms above, and Theorem 6.26, it is immediate to state a regret upper bound. With the notation in

Algorithm 7.4, for any $\boldsymbol{U} \in \mathbb{R}^{d \times n}$ we have

$$\sum_{t=1}^{T} |\hat{y}_t - y_t| - \sum_{t=1}^{T} \left| \sum_{i=1}^{n} \boldsymbol{z}_{t,i}^\top \boldsymbol{u}_i - y_t \right| \leq \frac{\alpha\sqrt{T}}{2} \|\boldsymbol{U}\|_{2,p}^2 + \frac{q}{2\alpha} \sum_{t=1}^{T} \frac{\|\boldsymbol{Z}_t\|_{2,q}^2}{\sqrt{t}}$$
$$\leq \frac{\alpha\sqrt{T}}{2} \|\boldsymbol{U}\|_{2,p}^2 + \frac{q\sqrt{T}}{\alpha} \max_{t \leq T} \|\boldsymbol{Z}_t\|_{2,q}^2 .$$

To obtain a logarithmic dependence in n, we can now proceed as in Section 6.6. Assuming $\|\boldsymbol{z}_{t,i}\|_2 \leq L$, we have that $\|\boldsymbol{Z}_t\|_{2,q}^2 \leq L^2 n^{2/q}$. Hence, setting $\alpha = qLn^{1/q}$ and $q = \max(2 \ln n, 2)$ we have that $\frac{q}{\alpha} \|\boldsymbol{Z}_t\|_{2,q}^2 \leq L \max(\sqrt{2e \ln n}, 2)$. Moreover, we can upper bound $\|\boldsymbol{U}\|_{2,p}^2$ with $\|\boldsymbol{U}\|_{2,1}^2$. Hence, the final regret upper bound is logarithmic in the number of groups: $L \max(\sqrt{2e \ln n}, 2) \left(\frac{\|\boldsymbol{U}\|_{2,1}^2}{2} + 1 \right) \sqrt{T}$.

7.8 Composite Losses and Proximal Operators

Let's now see a variant of the linearization of the losses: *partial linearization of composite losses*.

Suppose that the losses we receive are composed of two terms: one convex function changing over time, and another part is fixed and known. These losses are called **composite**. For example, we might have $\ell_t(\boldsymbol{x}) = \tilde{\ell}_t(\boldsymbol{x}) + \lambda \|\boldsymbol{x}\|_1$. Using the linearization, we might just take the subgradient of ℓ_t. However, in this particular case, we might lose the ability of the L_1 norm to produce sparse solutions.

There is a better way to deal with these kinds of losses: move the constant part of the loss inside the regularization term. In this way, that part will not be linearized but used exactly in the argmin of the update. Moreover, the regret of the algorithm will depend only on the subgradient of $\tilde{\ell}_t$ rather than subgradients of the full loss ℓ_t. Assuming that the argmin is still easily computable, you can always expect better performance from this approach. In particular, in the case of adding an L_1 norm to the losses, you will be predicting in each step with the solution of an L_1 regularized optimization problem.

Remark 7.23 *Given that we do not pay for the gradient of the regularizer in the regret upper bound of FTRL, one may wonder why we do not put the entire loss function in the regularizer. This is not possible because it would require knowledge of the future: at time t, we construct the regularizer ψ_t when we do not have access to the future loss ℓ_t.*

Practically speaking, in the example above, we will define $\psi_t(\boldsymbol{x}) = \frac{L}{\alpha}\sqrt{t}(\psi(\boldsymbol{x}) - \min_{\boldsymbol{y}} \psi(\boldsymbol{y})) + \lambda t \|\boldsymbol{x}\|_1$, where we assume ψ to be 1-strongly convex and the losses $\tilde{\ell}_t$ to be L-Lipschitz. Note that we use at time t a term $\lambda t \|\boldsymbol{x}\|_1$ because we anticipate the next term in the next round. Given that $\psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1} \ell_i(\boldsymbol{x})$ is $\frac{L}{\alpha}\sqrt{t}$-strongly convex, using Lemma 7.5

and setting $\psi_{T+1} = \psi_T$, we have

$$\begin{aligned}
&\sum_{t=1}^{T} \tilde{\ell}_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \tilde{\ell}_t(\boldsymbol{u}) \leq \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle \\
&\leq \frac{L}{\alpha}\sqrt{T}\left(\psi(\boldsymbol{u}) - \min_{\boldsymbol{x}\in\mathcal{V}} \psi(\boldsymbol{x})\right) + \lambda T\|\boldsymbol{u}\|_1 - \min_{\boldsymbol{x}\in\mathcal{V}} \left[\lambda\|\boldsymbol{x}\|_1 + \frac{L}{\alpha}(\psi(\boldsymbol{x}) - \min_{\boldsymbol{y}\in\mathcal{V}} \psi(\boldsymbol{y}))\right] \\
&\quad + \frac{\alpha}{2}\sum_{t=1}^{T} \frac{\|\boldsymbol{g}_t\|_\star^2}{L\sqrt{t}} - \lambda \sum_{t=1}^{T-1} \|\boldsymbol{x}_{t+1}\|_1 \\
&\leq L\sqrt{T}\left(\alpha + \frac{\psi(\boldsymbol{u}) - \min_{\boldsymbol{x}\in\mathcal{V}} \psi(\boldsymbol{x})}{\alpha}\right) + \lambda T\|\boldsymbol{u}\|_1 - \lambda\|\boldsymbol{x}_1\|_1 - \lambda \sum_{t=1}^{T-1} \|\boldsymbol{x}_{t+1}\|_1,
\end{aligned}$$

where $\boldsymbol{g}_t \in \partial\tilde{\ell}_t(\boldsymbol{x}_t)$. Reordering the terms, we have

$$\begin{aligned}
\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) &= \sum_{t=1}^{T} (\lambda\|\boldsymbol{x}_t\|_1 + \tilde{\ell}_t(\boldsymbol{x}_t)) - \sum_{t=1}^{T} (\lambda\|\boldsymbol{u}\|_1 + \tilde{\ell}_t(\boldsymbol{u})) \\
&\leq L\sqrt{T}\left(\frac{\psi(\boldsymbol{u}) - \min_{\boldsymbol{x}\in\mathcal{V}} \psi(\boldsymbol{x})}{\alpha} + \alpha\right) .
\end{aligned}$$

Example 7.24 *Let's also take a look at the update rule in that case, that $\psi(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{x}\|_2^2$ and we get composite losses with the L_1 norm. Also, assume that $\mathcal{V} = \mathbb{R}^d$. So, we have*

$$\boldsymbol{x}_t = \operatorname*{argmin}_{\boldsymbol{x}\in\mathbb{R}^d} \frac{L\sqrt{t}}{2\alpha}\|\boldsymbol{x}\|_2^2 + \lambda t\|\boldsymbol{x}\|_1 + \sum_{i=1}^{t-1} \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle .$$

We can solve this problem by observing that the minimization decomposes over each coordinate of $\boldsymbol{x}$. Denote by $\boldsymbol{\theta}_t = \sum_{i=1}^{t-1} \boldsymbol{g}_i$. Hence, we know from first-order optimality condition in Theorem 6.27 that $x_{t,i}$ is the solution for the coordinate i iff there exists $v_i \in \partial|x_{t,i}|$ such that

$$\frac{L}{\alpha}\sqrt{t}x_{t,i} + \lambda t v_i + \theta_{t,i} = 0 .$$

The difficulty comes from the fact that $\boldsymbol{v}$ is a subgradient in the next point $\boldsymbol{x}_t$, which we do not know because we do not know $\boldsymbol{v}$. Unfortunately, there is no automatic way to find v_i, so we can "guess" the correct expression of v_i and verify its correctness in the above expression. In particular, considering 3 different cases, we have

- *$|\theta_{t,i}| \leq \lambda t$, then $x_{t,i} = 0$ and $v_i = -\frac{\theta_{t,i}}{\lambda t}$.*
- *$\theta_{t,i} > \lambda t$, then $x_{t,i} = -\alpha\frac{\theta_{t,i} - \lambda t}{L\sqrt{t}}$ and $v_i = -1$.*
- *$\theta_{t,i} < -\lambda t$, then $x_{t,i} = -\alpha\frac{\theta_{t,i} + \lambda t}{L\sqrt{t}}$ and $v_i = 1$.*

So, overall, we have

$$x_{t,i} = -\alpha\frac{\operatorname{sign}(\theta_{t,i})\max(|\theta_{t,i}| - \lambda t, 0)}{L\sqrt{t}} .$$

Observe that this update is a soft-thresholding operator and it will produce sparse solutions, while just taking the subgradient of the L_1 norm would have never produced sparse predictions.

In the example above, we calculated something like

$$\begin{aligned}\operatorname*{argmin}_{\boldsymbol{x}\in\mathbb{R}^d} \|\boldsymbol{x}\|_1 - \langle \boldsymbol{v}, \boldsymbol{x}\rangle + \frac{1}{2}\|\boldsymbol{x}\|_2^2 &= \operatorname*{argmin}_{\boldsymbol{x}\in\mathbb{R}^d} \|\boldsymbol{x}\|_1 - \langle \boldsymbol{v}, \boldsymbol{x}\rangle + \frac{1}{2}\|\boldsymbol{x}\|_2^2 + \frac{1}{2}\|\boldsymbol{v}\|_2^2 \\ &= \operatorname*{argmin}_{\boldsymbol{x}\in\mathbb{R}^d} \|\boldsymbol{x}\|_1 + \frac{1}{2}\|\boldsymbol{x}-\boldsymbol{v}\|_2^2 \,.\end{aligned}$$

This operation is known in the optimization literature as **Proximal Operator** of the L_1 norm. In general, a proximal operator of a convex, proper, and closed function $f : \mathbb{R}^d \to (-\infty, +\infty]$ is defined as

$$\operatorname{Prox}_f(\boldsymbol{v}) = \operatorname*{argmin}_{\boldsymbol{x}\in\mathbb{R}^d} f(\boldsymbol{x}) + \frac{1}{2}\|\boldsymbol{x}-\boldsymbol{v}\|_2^2 \,. \tag{7.8}$$

Proximal operators are used in optimization in the same way as we used them: they allow us to minimize the entire function rather than a linear approximation of it. Also, proximal operators generalize the concept of Euclidean projection. Indeed, $\operatorname{Prox}_{\iota_{\mathcal{V}}}(\boldsymbol{v}) = \Pi_{\mathcal{V}}(\boldsymbol{v})$.

From the definition, we also derive the important property of proximal operators that they minimize the function in each update. In other words, if $\boldsymbol{x}_{t+1} = \operatorname{Prox}_f(\boldsymbol{x}_t)$, then $f(\boldsymbol{x}_{t+1}) \leq f(\boldsymbol{x}_t)$. Another interesting property is the fact that we can write the proximal operator as an *implicit* subgradient descent update. Let $\eta > 0$ and consider the following update

$$\boldsymbol{x}_{t+1} = \operatorname{Prox}_{\eta f}(\boldsymbol{x}_t) = \operatorname*{argmin}_{\boldsymbol{x}\in\mathbb{R}^d} \eta f(\boldsymbol{x}) + \frac{1}{2}\|\boldsymbol{x}-\boldsymbol{x}_t\|_2^2 \,.$$

Then, from Theorem 6.27 we have

$$\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \eta \boldsymbol{g}_t,$$

where $\boldsymbol{g}_t \in \partial f(\boldsymbol{x}_{t+1})$. Hence, we do an update step using the subgradient in the *next* iteration. This also reveals a connection between OMD and proximal updates. Indeed, the update in (7.8) resembles the one in (6.3), where the function is not linearized. We will use this kind of update in Section 15.5.1.

7.9 FTRL on Strongly Convex Functions

We will now use Lemma 7.5 to prove a logarithmic regret bound for Follow-the-Leader (FTL) with strongly convex losses. Note that the existence and uniqueness (for $t \geq 2$) of $\boldsymbol{x}_t$ is guaranteed by Theorem 6.9.

Corollary 7.25 *Let $\mathcal{V} \subseteq \mathbb{R}^d$ be a non-empty convex closed set. Let $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ be μ_t-strongly convex with respect to $\|\cdot\|$, closed, and subdifferentiable on $\mathcal{V}$ for $t = 1, \ldots, T$. Set the sequence of regularizers to zero. Then, FTL guarantees a regret of*

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \leq \frac{1}{2}\sum_{t=1}^{T} \frac{\|\boldsymbol{g}_t\|_\star^2}{\sum_{i=1}^{t}\mu_i}, \quad \forall \boldsymbol{u} \in \mathcal{V},\ \forall \boldsymbol{g}_t \in \partial\ell_t(\boldsymbol{x}_t) \,.$$

Algorithm 7.5 Follow-the-Regularized-Leader with "Quadratized" Losses

Require: A sequence of regularizers $\psi_1, \dots, \psi_T : \mathcal{X} \to \mathbb{R}$, closed non-empty convex set $\mathcal{V} \subseteq \mathcal{X} \subseteq \mathbb{R}^d$

for $t = 1$ **to** T **do**
  Output $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1} \left(\langle \boldsymbol{g}_i, \boldsymbol{x} \rangle + \frac{\mu_i}{2} \|\boldsymbol{x} - \boldsymbol{x}_i\|^2 \right)$
  Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable on $\mathcal{V}$ and μ_t strongly convex in $\mathcal{V}$
  Set $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$
end for

The above regret guarantee is the same as the one of OMD over strongly convex losses, but here we do not need to know the strong convexity of the losses. In fact, we just need to output the minimizer over the past losses. However, this might be undesirable because now each update requires the minimization of a convex function.

Hence, we can again use the idea of replacing the losses with simpler *surrogates*. In the Lipschitz case, it made sense to use linear losses because they were used to upper bound the regret of the true losses. However, here you can do better and use *quadratic* losses, because the losses are strongly convex. In fact, assuming ℓ_t to be μ_t strongly convex with respect to $\|\cdot\|$, we have

$$\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u}) \leq \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle - \frac{\mu_t}{2} \|\boldsymbol{x}_t - \boldsymbol{u}\|^2 = \tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{u}), \quad \forall \boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t) \,.$$

So, we could run FTRL on the quadratic losses $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle + \frac{\mu_t}{2} \|\boldsymbol{x} - \boldsymbol{x}_t\|^2$, where $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$. Depending on the norm, these quadratic losses might be strongly convex, allowing us to use Corollary 7.25 and obtain Algorithm 7.5.

For example, consider the case that the losses are strongly convex with respect to $\|\cdot\|_2$ and there is no regularizer. Using Theorem 6.31, the update in Algorithm 7.5 for $t \geq 2$ becomes

$$\boldsymbol{x}_t = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \sum_{i=1}^{t-1} \left(\langle \boldsymbol{g}_i, \boldsymbol{x} \rangle + \frac{\mu_i}{2} \|\boldsymbol{x} - \boldsymbol{x}_i\|_2^2 \right) = \Pi_{\mathcal{V}} \left(\frac{\sum_{i=1}^{t-1} (\mu_i \boldsymbol{x}_i - \boldsymbol{g}_i)}{\sum_{i=1}^{t-1} \mu_i} \right), \tag{7.9}$$

where the only expensive operation is the projection, which can be computed in polynomial time. Moreover, we will get exactly the same regret bound as in Corollary 7.25 because $\boldsymbol{g}_t \in \partial \tilde{\ell}_t(\boldsymbol{x}_t)$. However, there are two important differences: 1) here the guarantee holds for a specific choice of the $\boldsymbol{g}_t$ rather than for any subgradient in $\partial \ell_t(\boldsymbol{x}_t)$; 2) we need to know the strong convexity parameters μ_i, while it was not needed for FTL with exact losses.

Example 7.26 *Going back to the example in the first chapter, where $\mathcal{V} = [0, 1]$ and $\ell_t(\boldsymbol{x}) = (x - y_t)^2$ are strongly convex, we now see immediately that FTRL without a regularizer, that is FTL, gives logarithmic regret.*

7.10 A Weaker Notion of Strong Convexity and the Online Newton Step

In the previous section, we saw that the notion of strong convexity allows us to build quadratic surrogate loss functions, on which FTRL has smaller regret. Can we find a more

general notion of strong convexity that allows us to get a small regret for a larger class of functions? We can start from strong convexity and try to generalize it. So, instead of asking that the function is strongly convex with respect to a norm, we might be happy requiring that strong convexity holds at a particular point with respect to a seminorm that depends on the points themselves.

In particular, we can require that for a loss $\ell : \mathbb{R}^d \to (-\infty, +\infty]$ subdifferentiable on $\mathcal{V}$, for a $\boldsymbol{x} \in \mathcal{V}$, and for a $\boldsymbol{g} \in \partial \ell(\boldsymbol{x})$, there exists $\boldsymbol{A} \in \mathbb{R}^{d \times d}$ such that $\boldsymbol{A} \succeq 0$ and

$$\ell(\boldsymbol{y}) \geq \ell(\boldsymbol{x}) + \langle \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{x} \rangle + \frac{1}{2} \|\boldsymbol{x} - \boldsymbol{y}\|_{\boldsymbol{A}}^2, \quad \forall \boldsymbol{y} \in \mathcal{V}, \tag{7.10}$$

where $\|\boldsymbol{x}\|_{\boldsymbol{A}}$ is defined as $\sqrt{\boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x}}$. Note that this is a weaker property than strong convexity because $\boldsymbol{A}$ depends on $\boldsymbol{x}$ and $\boldsymbol{g}$. On the other hand, in the characterization of strong convexity in Lemma 4.2, we want the last term to be the same norm everywhere in the space. See also Problem 6.7 for a similar generalization of strong convexity.

The rationale of this new definition is that it still allows us to build surrogate loss functions, but without requiring strong convexity over the entire space. In particular, for a suitable choice of $\boldsymbol{A}_t$, we can think to use FTRL on the surrogate losses

$$\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle + \frac{1}{2} \|\boldsymbol{x}_t - \boldsymbol{x}\|_{\boldsymbol{A}_t}^2$$

and the regularizers $\psi_t(\boldsymbol{x}) = \frac{\lambda}{2} \|\boldsymbol{x}\|_2^2$, where $\lambda > 0$. Observe that, as in the surrogate losses for the strongly convex case, $\boldsymbol{g}_t \in \partial \tilde{\ell}_t(\boldsymbol{x}_t)$. We will denote by $\boldsymbol{S}_t = \lambda \boldsymbol{I}_d + \sum_{i=1}^t \boldsymbol{A}_i$.

Remark 7.27 *Note that $\|\boldsymbol{x}\|_{\boldsymbol{S}_t} := \sqrt{\boldsymbol{x}^\top \boldsymbol{S}_t \boldsymbol{x}}$ is a norm because $\boldsymbol{S}_t$ is positive definite. Also, $f(\boldsymbol{x}) = \frac{1}{2} \boldsymbol{x}^\top \boldsymbol{S}_t \boldsymbol{x}$ is 1-strongly convex with respect to $\|\cdot\|_{\boldsymbol{S}_t}$ (from Theorem 4.3 given that the Hessian is $\boldsymbol{S}_t$ and $\boldsymbol{x}^\top \nabla^2 f(\boldsymbol{y}) \boldsymbol{x} = \|\boldsymbol{x}\|_{\boldsymbol{S}_t}^2$). Also, from Example 4.19, the dual norm of $\|\cdot\|_{\boldsymbol{S}_t}$ is $\|\cdot\|_{\boldsymbol{S}_t^{-1}}$.*

From the above remark, we have that the $\psi_t(\boldsymbol{x}) + \sum_{i=1}^t \tilde{\ell}_i(\boldsymbol{x})$ is 1-strongly convex with respect to $\|\cdot\|_{\boldsymbol{S}_t}$. Hence, using the FTRL regret equality in Lemma 7.1 and Lemma 7.5, we immediately get the following guarantee

$$\begin{aligned}\sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}) &\leq \sum_{t=1}^T \left(\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle - \frac{1}{2} \|\boldsymbol{u} - \boldsymbol{x}_t\|_{\boldsymbol{A}_t}^2 \right) = \sum_{t=1}^T (\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{u})) \\ &\leq \frac{\lambda}{2} \|\boldsymbol{u}\|_2^2 - \min_{\boldsymbol{x} \in \mathcal{V}} \frac{\lambda}{2} \|\boldsymbol{x}\|_2^2 + \frac{1}{2} \sum_{t=1}^T \|\boldsymbol{g}_t\|_{\boldsymbol{S}_t^{-1}}^2, \quad \forall \boldsymbol{u} \in \mathcal{V} . \end{aligned} \tag{7.11}$$

Note how the proof and the algorithm mirror what we did for FTRL with strongly convex losses in Section 7.9.

Remark 7.28 *It is possible to generalize Lemma 7.5 to hold in this generalized notion of strong convexity. This would allow us to get exactly the same bound running FTRL over the original losses with regularizer $\psi(\boldsymbol{x}) = \frac{\lambda}{2} \|\boldsymbol{x}\|_2^2$.*

Let's now see a practical instantiation of this idea. Consider the case that the sequence of

Algorithm 7.6 Online Newton Step (ONS)

Require: $\mathcal{V} \subset \mathbb{R}^d$ closed non-empty convex set, $\lambda, \mu > 0$
for $t = 1$ **to** T **do**
  Set $\boldsymbol{x}_t = \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 + \sum_{i=1}^{t-1} \left(\langle \boldsymbol{g}_i, \boldsymbol{x}\rangle + \frac{\mu}{2}(\langle \boldsymbol{g}_i, \boldsymbol{x} - \boldsymbol{x}_i\rangle)^2\right)$
  Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable on $\mathcal{V}$
  Set $\boldsymbol{g}_t \in \partial\ell_t(\boldsymbol{x}_t)$
end for

loss functions we receive satisfies

$$\ell_t(\boldsymbol{x}) - \ell_t(\boldsymbol{u}) \leq \langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{u}\rangle - \frac{\mu}{2}(\langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{u}\rangle)^2, \quad \forall \boldsymbol{x}, \boldsymbol{u} \in \mathcal{V}, \forall \boldsymbol{g} \in \partial\ell_t(\boldsymbol{x})\,. \quad (7.12)$$

In words, *we assume to have a class of functions that can be lower bounded by a quadratic that depends on the current subgradient*. In particular, these functions guarantee a minimal curvature only in the direction of (any of) their subgradients. Denoting by $\boldsymbol{A}_t = \mu \boldsymbol{g}_t \boldsymbol{g}_t^\top$, we can use the above idea using

$$\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle + \frac{1}{2}\|\boldsymbol{x}_t - \boldsymbol{x}\|_{\boldsymbol{A}_t}^2 = \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle + \frac{\mu}{2}(\langle \boldsymbol{g}_t, \boldsymbol{x} - \boldsymbol{x}_t\rangle)^2\,.$$

Hence, the update rule would be

$$\boldsymbol{x}_t = \operatorname*{argmin}_{\boldsymbol{x}\in\mathcal{V}} \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 + \sum_{i=1}^{t-1} \left(\langle \boldsymbol{g}_i, \boldsymbol{x}\rangle + \frac{\mu}{2}(\langle \boldsymbol{g}_i, \boldsymbol{x} - \boldsymbol{x}_i\rangle)^2\right)\,.$$

This update is called the **Online Newton Step (ONS)**, see Algorithm 7.6.

Denoting by $\boldsymbol{S}_t = \lambda \boldsymbol{I}_d + \sum_{i=1}^t \boldsymbol{A}_i$ and using (7.11), we have

$$\sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}) \leq \frac{\lambda}{2}\|\boldsymbol{u}\|_2^2 - \min_{\boldsymbol{x}\in\mathcal{V}} \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 + \frac{1}{2}\sum_{t=1}^T \|\boldsymbol{g}_t\|_{\boldsymbol{S}_t^{-1}}^2, \quad \forall \boldsymbol{u} \in \mathcal{V}\,.$$

To bound the last term, we will use the following lemma.

Lemma 7.29 *Let $\boldsymbol{g}_1, \ldots, \boldsymbol{g}_T$ be an arbitrary sequence of vectors in $\mathbb{R}^d$ and $\mu, \lambda > 0$. Define $\boldsymbol{S}_t = \lambda \boldsymbol{I}_d + \mu \sum_{i=1}^t \boldsymbol{g}_i \boldsymbol{g}_i^\top$. Then, the following holds*

$$\sum_{t=1}^T \boldsymbol{g}_t^\top \boldsymbol{S}_t^{-1} \boldsymbol{g}_t \leq \frac{1}{\mu}\sum_{i=1}^d \ln\left(1 + \frac{\mu\lambda_i}{\lambda}\right),$$

where $\lambda_1, \ldots, \lambda_d$ are the eigenvalues of $\sum_{i=1}^T \boldsymbol{g}_i \boldsymbol{g}_i^\top$.

Proof First, we will prove the so-called *matrix determinant lemma* for any invertible $\boldsymbol{B} \in \mathbb{R}^{d\times d}$:

$$\det(\boldsymbol{B} + \boldsymbol{u}\boldsymbol{v}^\top) = \det(\boldsymbol{B})(1 + \boldsymbol{v}^\top \boldsymbol{B}^{-1}\boldsymbol{u}), \quad \forall \boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^d\,.$$

Observe that the matrix $\boldsymbol{u}\boldsymbol{v}^\top$ has one eigenvalue equal to $\boldsymbol{v}^\top\boldsymbol{u}$ and the remaining $d-1$ are 0. Hence, we have that

$$\det(\boldsymbol{I} + \boldsymbol{u}\boldsymbol{v}^\top) = 1 + \boldsymbol{v}^\top\boldsymbol{u}\,.$$

Using this equality, we have the matrix determinant lemma:

$$\det(\boldsymbol{B} + \boldsymbol{u}\boldsymbol{v}^\top) = \det(\boldsymbol{B})\det(\boldsymbol{I} + \boldsymbol{B}^{-1}\boldsymbol{u}\boldsymbol{v}^\top) = \det(\boldsymbol{B})(1 + \boldsymbol{v}^\top \boldsymbol{B}^{-1}\boldsymbol{u}) .$$

Now, setting $\boldsymbol{u} = -\sqrt{\mu}\boldsymbol{g}_t$, $\boldsymbol{v} = \sqrt{\mu}\boldsymbol{g}_t$ and $\boldsymbol{B} = \boldsymbol{S}_t$, we have

$$\det(\boldsymbol{S}_{t-1}) = \det(\boldsymbol{S}_t - \mu \boldsymbol{g}_t \boldsymbol{g}_t^\top) = \det(\boldsymbol{S}_t)(1 - \mu \boldsymbol{g}_t^\top \boldsymbol{S}_t^{-1} \boldsymbol{g}_t) .$$

Reordering the terms and summing over $t = 1, \dots, T$, we have

$$\begin{aligned}\sum_{t=1}^{T} \boldsymbol{g}_t^\top \boldsymbol{S}_t^{-1} \boldsymbol{g}_t &= \frac{1}{\mu}\sum_{t=1}^{T}\left(1 - \frac{\det(\boldsymbol{S}_{t-1})}{\det(\boldsymbol{S}_t)}\right) \le \frac{1}{\mu}\sum_{t=1}^{T} \ln \frac{\det(\boldsymbol{S}_t)}{\det(\boldsymbol{S}_{t-1})} = \frac{1}{\mu}\ln\frac{\det(\boldsymbol{S}_T)}{\det(\boldsymbol{S}_0)} \\ &= \frac{1}{\mu}\sum_{i=1}^{d} \ln\left(1 + \frac{\mu\lambda_i}{\lambda}\right),\end{aligned}$$

where in the inequality we used the elementary inequality $1 - x \le \ln 1/x$ for $x > 0$. □

Using this lemma and assuming $\|\boldsymbol{g}_t\|_2 \le L$ and (7.12) holds for the losses, then ONS satisfies the following regret

$$\begin{aligned}\sum_{t=1}^{T}\ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T}\ell_t(\boldsymbol{u}) &\le \frac{\lambda}{2}\|\boldsymbol{u}\|_2^2 - \min_{\boldsymbol{x}\in\mathcal{V}} \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 + \frac{1}{2\mu}\sum_{i=1}^{d}\ln\left(1 + \frac{\mu\lambda_i}{\lambda}\right) \\ &\le \frac{\lambda}{2}\|\boldsymbol{u}\|_2^2 - \min_{\boldsymbol{x}\in\mathcal{V}} \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 + \frac{d}{2\mu}\ln\left(1 + \frac{\mu T L^2}{d\lambda}\right),\end{aligned}$$

where in the second inequality we used the inequality of arithmetic and geometric means, $(\prod_{i=1}^{d} x_i)^{1/d} \le \frac{1}{d}\sum_{i=1}^{d} x_i$, and the fact that $\sum_{i=1}^{d}\lambda_i \le TL^2$.

Hence, if the losses satisfy (7.12), we can guarantee a logarithmic regret. However, unlike the strongly convex case, here the complexity of the update is at least quadratic in the number of dimensions. Moreover, the regret also depends linearly on the number of dimensions.

Remark 7.30 *Despite the name, the ONS algorithm should not be confused with the Newton algorithm. They are similar in spirit because they both construct quadratic approximations to the function, but the Newton algorithm uses the exact Hessian while the ONS uses an approximation that works only for a restricted class of functions. In this view, the ONS algorithm is more similar to Quasi-Newton methods. However, the best lens to understand the ONS is still through the generalized concept of strong convexity.*

In the next section, we introduce the *exp-concave* functions that satisfy (7.12).

7.10.1 Convex Analysis Bits: Exp-concavity

Definition 7.31 *Let $\mathcal{V} \subseteq \mathbb{R}^d$ be convex and $\alpha > 0$. A function $f : \mathbb{R}^d \to (-\infty, +\infty]$ is called α-**exp-concave** on $\mathcal{V}$ if $\boldsymbol{x} \mapsto \exp(-\alpha f(\boldsymbol{x}))$, with the convention $\exp(-\alpha\cdot(+\infty)) = 0$, is concave on $\mathcal{V}$.*

Proposition 7.32 *Let $\alpha > 0$. If $f : \mathbb{R}^d \to (-\infty, +\infty]$ is α-exp-concave on $\mathcal{V}$, then f is β-exp-concave on $\mathcal{V}$ for any $0 < \beta < \alpha$.*

Proof Observe that $h(\boldsymbol{x}) = \exp(-\beta f(\boldsymbol{x})) = (\exp(-\alpha f(\boldsymbol{x})))^{\beta/\alpha}$, where $0^{\beta/\alpha} = 0$. Since $x \mapsto x^{\beta/\alpha}$ is increasing and concave on $\mathbb{R}_+$, h is concave. □

Proposition 7.33 *Assume $f : \mathbb{R}^n \to (-\infty, +\infty]$ is α-exp-concave on $\mathcal{V} \subseteq \mathbb{R}^n$ and let $h(\boldsymbol{x}) = f(\boldsymbol{A}\boldsymbol{x} + \boldsymbol{u})$, where $\boldsymbol{A} \in \mathbb{R}^{n \times d}$ and $\boldsymbol{u} \in \mathbb{R}^n$. Then, h is α-exp-concave on $\{\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} + \boldsymbol{u} \in \mathcal{V}\}$.*

Proof Since $\exp(-\alpha f(\boldsymbol{x}))$ is concave, then $\exp(-\alpha h(\boldsymbol{x})) = \exp(-\alpha f(\boldsymbol{A}\boldsymbol{x} + \boldsymbol{u}))$ is also concave because it is the composition of a concave function with an affine transformation. □

The next proposition shows that exp-concavity is a stronger property than convexity.

Proposition 7.34 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be α-exp-concave on $\mathcal{V}$. Then, f is convex on $\mathcal{V}$.*

Proof Let $g(\boldsymbol{x}) = \exp(-\alpha f(\boldsymbol{x}))$. By hypothesis, g is concave and non-negative on $\mathcal{V}$. Fix $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{V}$ and $t \in [0, 1]$. If $f(\boldsymbol{x}) = +\infty$ or $f(\boldsymbol{y}) = +\infty$, the convexity inequality is trivial. Otherwise, $g(\boldsymbol{x}), g(\boldsymbol{y}) > 0$. Concavity of g and the weighted AM-GM inequality give

$$g(t\boldsymbol{x} + (1-t)\boldsymbol{y}) \geq tg(\boldsymbol{x}) + (1-t)g(\boldsymbol{y}) \geq g(\boldsymbol{x})^t g(\boldsymbol{y})^{1-t} .$$

Hence,

$$\exp(-\alpha f(t\boldsymbol{x} + (1-t)\boldsymbol{y})) \geq \exp\left[-\alpha \left(tf(\boldsymbol{x}) + (1-t)f(\boldsymbol{y})\right)\right] .$$

Taking $-\frac{1}{\alpha} \ln$ of both sides gives

$$f(t\boldsymbol{x} + (1-t)\boldsymbol{y}) \leq tf(\boldsymbol{x}) + (1-t)f(\boldsymbol{y}) .$$

So, f is convex on $\mathcal{V}$. □

We can also characterize the exp-concavity in terms of the curvature of the function: twice differentiable exp-concave functions have a minimal curvature in the direction of the gradient.

Theorem 7.35 *Let $\mathcal{V} \subset \mathbb{R}^d$ be convex and f be twice-differentiable and finite in an open set containing $\mathcal{V}$. Then, f is α-exp-concave on $\mathcal{V}$ iff*

$$\nabla^2 f(\boldsymbol{x}) \succeq \alpha \nabla f(\boldsymbol{x}) \nabla f(\boldsymbol{x})^\top, \quad \forall \boldsymbol{x} \in \mathcal{V} .$$

Proof The concavity of $g(\boldsymbol{x}) = \exp(-\alpha f(\boldsymbol{x}))$ is equivalent to $\nabla^2 g(\boldsymbol{x}) \preceq 0$ for all $\boldsymbol{x} \in \mathcal{V}$.

Let's compute ∇g and $\nabla^2 g$. Denote $g(\boldsymbol{x}) = \exp(-\alpha f(\boldsymbol{x}))$. The gradient is

$$\nabla g(\boldsymbol{x}) = \nabla \left(\exp(-\alpha f(\boldsymbol{x}))\right) = -\alpha \exp(-\alpha f(\boldsymbol{x})) \nabla f(\boldsymbol{x}) .$$

The Hessian is

$$\begin{aligned}
\nabla^2 g(\boldsymbol{x}) &= \nabla \left(-\alpha \exp(-\alpha f(\boldsymbol{x})) \nabla f(\boldsymbol{x})\right) \\
&= -\alpha \nabla \left(\exp(-\alpha f(\boldsymbol{x}))\right) \nabla f(\boldsymbol{x})^\top - \alpha \exp(-\alpha f(\boldsymbol{x})) \nabla^2 f(\boldsymbol{x}) \\
&= \alpha^2 \exp(-\alpha f(\boldsymbol{x})) \left(\nabla f(\boldsymbol{x}) \nabla f(\boldsymbol{x})^\top\right) - \alpha \exp(-\alpha f(\boldsymbol{x})) \nabla^2 f(\boldsymbol{x}) .
\end{aligned}$$

Re-ordering,

$$\nabla^2 g(\boldsymbol{x}) = \exp(-\alpha f(\boldsymbol{x})) \left(\alpha^2 \nabla f(\boldsymbol{x}) \nabla f(\boldsymbol{x})^\top - \alpha \nabla^2 f(\boldsymbol{x})\right) .$$

Since $\exp(-\alpha f(\boldsymbol{x})) > 0$ and $\alpha > 0$, the concavity condition $\nabla^2 g(\boldsymbol{x}) \preceq 0$ is equivalent to $\alpha \nabla f(\boldsymbol{x}) \nabla f(\boldsymbol{x})^\top \preceq \nabla^2 f(\boldsymbol{x})$. □

Example 7.36 *Consider $\mathcal{V} = \{\boldsymbol{x} : \|\boldsymbol{x}\|_2 \le R\}$, $\mathcal{Y} = \{\boldsymbol{y} : \|\boldsymbol{y}\|_2 \le R\}$, and $f : \mathbb{R}^d \to \mathbb{R}$ defined as $f(\boldsymbol{x}) = \|\boldsymbol{x} - \boldsymbol{y}\|_2^2$, where $\boldsymbol{y} \in \mathcal{Y}$. We have*

$$\nabla^2 f(\boldsymbol{x}) = 2\boldsymbol{I}_d$$

and

$$\nabla f(\boldsymbol{x}) = 2(\boldsymbol{x} - \boldsymbol{y}) .$$

Using Theorem 7.35, f is α-exp-concave on $\mathcal{V}$ iff

$$\boldsymbol{I}_d \succeq 2\alpha(\boldsymbol{x} - \boldsymbol{y})(\boldsymbol{x} - \boldsymbol{y})^\top, \ \forall \boldsymbol{x} \in \mathcal{V}, \boldsymbol{y} \in \mathcal{Y} .$$

Hence, we have that $\alpha = \frac{1}{2(2R)^2}$.

Example 7.37 *Let $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \le U\}$. The logistic loss of a linear predictor $\ell(\boldsymbol{x}) = \ln(1 + \exp(-\langle \boldsymbol{z}, \boldsymbol{x}\rangle))$, where $\|\boldsymbol{z}\|_2 \le 1$ is $\exp(-2U)/2$-exp-concave on $\mathcal{V}$. The proof is left to the reader, see Problem 7.7.*

Finally, we show that exp-concavity holds on the entire $\mathbb{R}^d$ only for constant functions.

Proposition 7.38 *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be α-exp-concave on $\mathbb{R}^d$. Then, f is constant, possibly equal to $+\infty$ everywhere.*

Proof Denote by $h(\boldsymbol{x}) = -\exp(-\alpha f(\boldsymbol{x}))$, which is convex by definition, and it is upper bounded by 0. Suppose that h is not constant, this means that there exist $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d$ such that $h(\boldsymbol{x}) > h(\boldsymbol{y})$. Since h is convex, we have

$$h(\boldsymbol{x}) \le \lambda h\left(\frac{\boldsymbol{x} - (1-\lambda)\boldsymbol{y}}{\lambda}\right) + (1-\lambda)h(\boldsymbol{y}), \quad \forall \lambda \in (0,1) .$$

Hence, we have

$$\frac{h(\boldsymbol{x}) - h(\boldsymbol{y})}{\lambda} + h(\boldsymbol{y}) = \frac{h(\boldsymbol{x}) - (1-\lambda)h(\boldsymbol{y})}{\lambda} \le h\left(\frac{\boldsymbol{x} - (1-\lambda)\boldsymbol{y}}{\lambda}\right) \le 0 .$$

Since $h(\boldsymbol{x}) > h(\boldsymbol{y})$, we have that the l.h.s. of the inequality goes to infinity as $\lambda \to 0^+$, giving a contradiction. Hence, h is constant. Therefore $\exp(-\alpha f)$ is constant, which implies that f is constant, with the possible constant value $+\infty$. □

Example 7.39 (ONS on exp-concave losses) *Assume a finite-valued function f to be α-exp-concave on a set $\mathcal{X}$.*

Choose $\beta \le \frac{\alpha}{2}$ such that $|\beta\langle \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{x}\rangle| \le \frac{1}{2}$ for all $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}$ and $\boldsymbol{g} \in \partial f(\boldsymbol{x})$. For β to exist, we need $\mathcal{X}$ to be a bounded domain. Then, this class of functions satisfies the property (7.12)*. In fact, given that f is α-exp-concave then it is also 2β-exp-concave. Hence, we have*

$$\exp(-2\beta f(\boldsymbol{y})) \le \exp(-2\beta f(\boldsymbol{x})) - 2\beta \exp(-2\beta f(\boldsymbol{x}))\langle \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{x}\rangle,$$

that is

$$\exp(-2\beta f(\boldsymbol{y}) + 2\beta f(\boldsymbol{x})) \leq 1 - 2\beta\langle \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{x}\rangle,$$

that implies

$$f(\boldsymbol{x}) - f(\boldsymbol{y}) \leq \frac{1}{2\beta}\ln\left(1 + 2\beta\langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{y}\rangle\right) \leq \langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{y}\rangle - \frac{\beta}{2}(\langle \boldsymbol{g}, \boldsymbol{y} - \boldsymbol{x}\rangle)^2,$$

where we used the elementary inequality $\ln(1+x) \leq x - x^2/4$*, for* $|x| \leq 1$*.*

For example, assuming the losses to be finite-valued and α*-exp-concave, the* L_2 *diameter of the feasible set* $\mathcal{V}$ *to be bounded by* D*, and the* L_2 *norm of the subgradient to be bounded by* G*, we can set* $\beta = \frac{1}{2}\min(\frac{1}{GD}, \alpha)$*.*

Remark 7.40 *Proposition 7.38 and the fact that exp-concave functions are often used only on bounded domains might induce someone to think that exp-concave functions only exist on bounded domains. This is not true: there exists* $f : \mathbb{R}^d \to \mathbb{R}$ *finite-valued and exp-concave on* $\mathcal{V}$*, where* $\mathcal{V} \subset \mathbb{R}^d$ *is not bounded. For example, let* $\mathcal{V} = \{x : x \geq 0\}$ *and* $f(x) = -\ln(x+1)$ *that is 1-exp-concave on* $\mathcal{V}$*.*

Remark 7.41 *Using the online-to-batch conversion in Theorem 3.1, the above theorem says that we can minimize stochastic exp-concave losses with a convergence rate in expectation of* $\mathcal{O}(\frac{d\ln T}{T})$*. Hence, up to logarithmic terms, we obtain a* $\mathcal{O}(1/T)$ *convergence rate without strong convexity.*

7.11 Online Linear Regression: Vovk–Azoury–Warmuth Forecaster

Algorithm 7.7 Vovk–Azoury–Warmuth Forecaster

Require: $\lambda > 0$

for $t = 1$ **to** T **do**
  Receive $\boldsymbol{z}_t \in \mathbb{R}^d$
  Set $\boldsymbol{x}_t = \operatorname{argmin}_{\boldsymbol{x}} \ \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 + \frac{1}{2}\sum_{i=1}^{t-1}(\langle \boldsymbol{z}_i, \boldsymbol{x}\rangle - y_i)^2 + \frac{1}{2}(\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle)^2$
  Receive $y_t \in \mathbb{R}$ and pay the loss $\ell_t(\boldsymbol{x}_t) = \frac{1}{2}(\langle \boldsymbol{z}_t, \boldsymbol{x}_t\rangle - y_t)^2$
end for

Let's now consider the specific case that $\ell_t(\boldsymbol{x}) = \frac{1}{2}(\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle - y_t)^2$ and $\mathcal{V} = \mathbb{R}^d$, which is the setting of *unconstrained online linear regression with square loss*. These losses are not strongly convex with respect to $\boldsymbol{x}$, but they are exp-concave when the domain is bounded. We could use the ONS algorithm, but it would not work in the unbounded case. Another possibility would be to run FTRL, but those losses are not strongly convex, and we would get only a $\mathcal{O}(\sqrt{T})$ regret.

It turns out we can still get a logarithmic regret if we make an additional assumption. We will assume to have access to $\boldsymbol{z}_t$ before predicting $\boldsymbol{x}_t$. Note that this is a reasonable assumption in many interesting applications in machine learning. Then, we will use the same strategy we used for the composite losses in Section 7.8, that is, we put in the regularizer

the future part of the loss we know we will receive, that is $\frac{1}{2}(\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle)^2$. This also has another equivalent interpretation as running FTRL over the past losses plus the loss on the received $\boldsymbol{z}_t$ hallucinating a label of 0. We will call this algorithm **Vovk–Azoury–Warmuth forecaster**, from the names of the inventors. The details are in Algorithm 7.7.

As we did for composite losses, we look closely at the loss functions to see if there are terms that we might move inside the regularizer. The motivation would be the same as in the composite losses case: the bound will depend only on the subgradients of the part of the losses that are outside of the regularizer.

So, observe that

$$\ell_t(\boldsymbol{x}) = \frac{1}{2}(\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle)^2 - y_t\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle + \frac{1}{2}y_t^2 \,.$$

From the above, we see that we could think of moving the terms $\frac{1}{2}(\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle)^2$ in the regularizer and leaving the linear terms in the loss: $\tilde{\ell}_t(\boldsymbol{x}) = -y_t\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle$. Hence, we will use

$$\psi_t(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top \left(\sum_{i=1}^{t} \boldsymbol{z}_i \boldsymbol{z}_i^\top\right) \boldsymbol{x} + \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 \,.$$

Note that the regularizer at time t contains the $\boldsymbol{z}_t$ that is revealed to the algorithm before it makes its prediction. For simplicity of notation, denote by $\boldsymbol{S}_t = \lambda \boldsymbol{I}_d + \sum_{i=1}^{t} \boldsymbol{z}_i \boldsymbol{z}_i^\top$.

Remark 7.42 *This splitting of the loss between the regularizer and the surrogate linear losses is important because now the gradient of the surrogate is just $-y_t\boldsymbol{z}_t$, which is controlled if $\boldsymbol{z}_t$ is controlled; instead, the gradient of the original losses is $y_t\boldsymbol{z}_t(\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle - y_t)$ that is not controlled for unbounded domains. However, now Lemma 7.5 will give us a strong convexity that depends on the strong convexity of ψ_t, and this is exactly the reason why we need $(\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle)^2$ in the regularizer.*

Using such a procedure, the prediction can be written in a closed form:

$$\begin{aligned}\boldsymbol{x}_t &= \operatorname*{argmin}_{\boldsymbol{x}} \ \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2 + \frac{1}{2}\sum_{i=1}^{t-1}(\langle \boldsymbol{z}_i, \boldsymbol{x}\rangle - y_i)^2 + \frac{1}{2}(\langle \boldsymbol{z}_t, \boldsymbol{x}\rangle)^2 \\ &= \left(\lambda \boldsymbol{I}_d + \sum_{i=1}^{t} \boldsymbol{z}_i \boldsymbol{z}_i^\top\right)^{-1} \sum_{i=1}^{t-1} y_i \boldsymbol{z}_i \,.\end{aligned}$$

Using an incremental update of the inverse using the Sherman–Morrison formula, the computational complexity of this update is $\mathcal{O}(d^2)$.

Hence, using the regret we proved for FTRL with strongly convex regularizers and $\psi_{T+1} =$

ψ_T, we get the following guarantee

$$\begin{aligned}
&\sum_{t=1}^{T}\left(\tilde{\ell}_t(\boldsymbol{x}_t)-\tilde{\ell}_t(\boldsymbol{u})\right)\\
&\quad=\sum_{t=1}^{T}\left(-y_t\langle\boldsymbol{z}_t,\boldsymbol{x}_t\rangle+y_t\langle\boldsymbol{z}_t,\boldsymbol{u}\rangle\right)\\
&\quad\le\psi_T(\boldsymbol{u})-\min_{\boldsymbol{x}}\ \psi_1(\boldsymbol{x})+\frac{1}{2}\sum_{t=1}^{T}y_t^2\boldsymbol{z}_t^\top\boldsymbol{S}_t^{-1}\boldsymbol{z}_t+\sum_{t=1}^{T-1}\left(\psi_t(\boldsymbol{x}_{t+1})-\psi_{t+1}(\boldsymbol{x}_{t+1})\right)\\
&\quad=\frac{1}{2}\boldsymbol{u}^\top\boldsymbol{S}_T\boldsymbol{u}+\frac{1}{2}\sum_{t=1}^{T}y_t^2\boldsymbol{z}_t^\top\boldsymbol{S}_t^{-1}\boldsymbol{z}_t-\frac{1}{2}\sum_{t=1}^{T-1}\left(\langle\boldsymbol{z}_{t+1},\boldsymbol{x}_{t+1}\rangle\right)^2\,.
\end{aligned}$$

Noting that $\boldsymbol{x}_1=\boldsymbol{0}$ and reordering the terms, we have

$$\begin{aligned}
&\sum_{t=1}^{T}\frac{1}{2}(\langle\boldsymbol{z}_t,\boldsymbol{x}_t\rangle-y_t)^2-\sum_{t=1}^{T}\frac{1}{2}(\langle\boldsymbol{z}_t,\boldsymbol{u}\rangle-y_t)^2\\
&\quad=\frac{1}{2}\sum_{t=1}^{T}(\langle\boldsymbol{z}_t,\boldsymbol{x}_t\rangle)^2+\sum_{t=1}^{T}\left(-y_t\langle\boldsymbol{z}_t,\boldsymbol{x}_t\rangle+y_t\langle\boldsymbol{z}_t,\boldsymbol{u}\rangle\right)-\frac{1}{2}\sum_{t=1}^{T}(\langle\boldsymbol{z}_t,\boldsymbol{u}\rangle)^2\\
&\quad\le\frac{\lambda}{2}\|\boldsymbol{u}\|_2^2+\frac{1}{2}\sum_{t=1}^{T}y_t^2\boldsymbol{z}_t^\top\boldsymbol{S}_t^{-1}\boldsymbol{z}_t\,.
\end{aligned}$$

Remark 7.43 *Note that, differently from the ONS algorithm, the strong convexity comes from the regularizer. Yet, the bound depends on $\boldsymbol{S}_t^{-1}$ instead of $\boldsymbol{S}_{t-1}^{-1}$ because the current sample $\boldsymbol{z}_t$ is used in the regularizer. Without the knowledge of $\boldsymbol{z}_t$ the last term would be $\frac{1}{2}\sum_{t=1}^{T}y_t^2\boldsymbol{z}_t^\top\boldsymbol{S}_{t-1}^{-1}\boldsymbol{z}_t$ that would have to be controlled using λ and a bound on the norms of $\boldsymbol{z}_t$.*

So, using again Lemma 7.29 and assuming $|y_t|\le Y,\ t=1,\dots,T$, we have

$$\sum_{t=1}^{T}\frac{1}{2}(\langle\boldsymbol{z}_t,\boldsymbol{x}_t\rangle-y_t)^2-\sum_{t=1}^{T}\frac{1}{2}(\langle\boldsymbol{z}_t,\boldsymbol{u}\rangle-y_t)^2\le\frac{\lambda}{2}\|\boldsymbol{u}\|_2^2+\frac{Y^2}{2}\sum_{i=1}^{d}\ln\left(1+\frac{\lambda_i}{\lambda}\right),$$

where $\lambda_1,\dots,\lambda_d$ are the eigenvalues of $\sum_{i=1}^{T}\boldsymbol{z}_i\boldsymbol{z}_i^\top$.

If we assume that $\|\boldsymbol{z}_t\|_2\le R$, we can reason as we did for the similar term in ONS, to have

$$\sum_{i=1}^{d}\ln\left(1+\frac{\lambda_i}{\lambda}\right)\le d\ln\left(1+\frac{R^2T}{\lambda d}\right)\,.$$

Putting everything together, we have the following theorem.

Theorem 7.44 *Let $\lambda>0$ and assume $\|\boldsymbol{z}_t\|_2\le R$ and $|y_t|\le Y$ for $t=1,\dots,T$. Then,*

using the prediction strategy in Algorithm 7.7, for all $\boldsymbol{u} \in \mathbb{R}^d$, we have

$$\sum_{t=1}^{T} \frac{1}{2}(\langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle - y_t)^2 - \sum_{t=1}^{T} \frac{1}{2}(\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle - y_t)^2 \leq \frac{\lambda}{2}\|\boldsymbol{u}\|_2^2 + \frac{dY^2}{2} \ln\left(1 + \frac{R^2 T}{\lambda d}\right) .$$

Remark 7.45 *It is possible to show that the regret of the Vovk–Azoury–Warmuth forecaster is optimal up to multiplicative factors (Cesa-Bianchi and Lugosi, 2006, Theorem 11.9).*

Remark 7.46 *We say that an online learning algorithm is **improper** if the prediction strategy can be chosen from a larger class than the competitor one. So, the Vovk–Azoury–Warmuth forecaster is improper because we measure its regret with respect to a linear predictor $\boldsymbol{u}$, but the Vovk–Azoury–Warmuth prediction depends in a non-linear way on $\boldsymbol{z}_t$.*

7.12 Optimistic Follow-the-Regularized-Leader

Algorithm 7.8 Optimistic Follow-the-Regularized-Leader (Optimistic FTRL)

Require: A sequence of regularizers $\psi_1, \dots, \psi_T : \mathcal{X} \to \mathbb{R}$, closed non-empty convex set $\mathcal{V} \subseteq \mathcal{X} \subseteq \mathbb{R}^d$

for $t = 1$ **to** T **do**
  Predict next loss $\tilde{\ell}_t : \mathcal{V} \to \mathbb{R}$
  Output $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \psi_t(\boldsymbol{x}) + \tilde{\ell}_t(\boldsymbol{x}) + \sum_{i=1}^{t-1} \ell_i(\boldsymbol{x})$
  Receive $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ such that $\boldsymbol{x}_t \in \operatorname{dom} \ell_t$, and pay the loss $\ell_t(\boldsymbol{x}_t)$
end for

In Section 6.11, we have seen that it is possible to achieve better regret guarantees through the so-called Optimistic OMD, which uses a prediction of the next gradient. Here, we will extend the same idea to FTRL, where we predict the next loss function. If our predicted loss is correct, we can expect the regret to decrease. However, if our prediction is wrong, we still want to recover the worst-case guarantee. Such an algorithm is called **Optimistic FTRL**, see Algorithm 7.8. As in Optimistic OMD, we do not make any assumption on how the prediction of the next loss is generated.

Let's see why this is a good idea. Remember that FTRL simply predicts with the minimizer of the previous losses plus a time-varying regularizer. Let's assume for a moment that instead we have the gift of predicting the future, so we do know the next loss ahead of time. Then, we could predict with its minimizer and suffer a negative regret. However, probably our foresight abilities are not so powerful, so our prediction of the next loss might be inaccurate. In this case, a better idea might be just to add our predicted loss to the previous ones and minimize the regularized sum. We would expect the regret guarantee to improve if our prediction of the future loss is precise. At the same time, if the prediction is wrong, we expect its influence to be limited, given that we use it together with all the past losses.

All these intuitions can be formalized in the following theorem.

Theorem 7.47 *With the notation in Algorithm 7.8, let $\mathcal{V}$ be convex, closed, non-empty. Denote by $F_t(\boldsymbol{x}) = \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1} \ell_i(\boldsymbol{x})$. For $t = 1, \dots, T$, if $F_t + \tilde{\ell}_t$ is closed, subdifferentiable, and strongly convex in $\mathcal{V}$, then $\boldsymbol{x}_t$ exists and is unique. In addition, for $t = 1, \dots, T$,*

assume $\partial(\ell_t - \tilde{\ell}_t)(\boldsymbol{x}_t)$ to be non-empty and $F_t + \ell_t$ to be closed, subdifferentiable, and λ_t-strongly convex with respect to $\|\cdot\|$ in $\mathcal{V}$. Then, we have

$$\begin{aligned}\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) &\leq \psi_T(\boldsymbol{u}) - \psi_1(\boldsymbol{x}_1) + \sum_{t=1}^{T}\left[\langle \hat{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{\lambda_t}{2}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2\right] \\ &\quad + \sum_{t=1}^{T-1}\left[\psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1})\right] \\ &\leq \psi_T(\boldsymbol{u}) - \psi_1(\boldsymbol{x}_1) + \sum_{t=1}^{T}\frac{\|\hat{\boldsymbol{g}}_t\|_\star^2}{2\lambda_t} + \sum_{t=1}^{T-1}\left[\psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1})\right],\end{aligned}$$

for all $\boldsymbol{u} \in \mathcal{V}$ and all $\hat{\boldsymbol{g}}_t \in \partial(\ell_t - \tilde{\ell}_t)(\boldsymbol{x}_t)$ for $t = 1, \ldots, T$.

Proof We can interpret the Optimistic FTRL as FTRL with a regularizer $\tilde{\psi}_t(\boldsymbol{x}) = \psi_t(\boldsymbol{x}) + \tilde{\ell}_t(\boldsymbol{x})$. So, as in Lemma 7.5, the existence and uniqueness of $\boldsymbol{x}_t$ is given by Theorem 6.9. Also, note that $\tilde{\ell}_{T+1}$ does not influence the regret of the algorithm over the T rounds, so we can set $\tilde{\ell}_{T+1} = \tilde{\ell}_T$.

Define $F_t(\boldsymbol{x}) = \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1}\ell_i(\boldsymbol{x})$. We can use Lemma 7.1 with $\psi_{T+1} = \psi_T$. Given that $\boldsymbol{u} \in \mathcal{V}$, we can discard the last term in the lemma because non-positive, to obtain

$$\begin{aligned}&\sum_{t=1}^{T}\ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T}\ell_t(\boldsymbol{u}) \\ &\quad\leq \tilde{\ell}_T(\boldsymbol{u}) + \psi_T(\boldsymbol{u}) - \min_{\boldsymbol{x}\in\mathcal{V}}\left(\tilde{\ell}_1(\boldsymbol{x}) + \psi_1(\boldsymbol{x})\right) \\ &\qquad + \sum_{t=1}^{T}[F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t)] + \sum_{t=1}^{T-1}[\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_{t+1}(\boldsymbol{x}_{t+1})] \\ &\quad= \psi_T(\boldsymbol{u}) - \psi_1(\boldsymbol{x}_1) + \sum_{t=1}^{T}[F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t)],\end{aligned}$$

where we used the fact that the terms $\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_{t+1}(\boldsymbol{x}_{t+1})$ form a telescopic sum and $\min_{\boldsymbol{x}\in\mathcal{V}}\left(\tilde{\ell}_1(\boldsymbol{x}) + \psi_1(\boldsymbol{x})\right) = \tilde{\ell}_1(\boldsymbol{x}_1) + \psi_1(\boldsymbol{x}_1)$. Now focus on the terms $F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t)$. Observe that $F_t(\boldsymbol{x}) + \ell_t(\boldsymbol{x}) + \iota_{\mathcal{V}}(\boldsymbol{x})$ is λ_t-strongly convex with respect to $\|\cdot\|$, hence from Lemma 4.2 we have

$$\begin{aligned}&F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t) \\ &\quad= (F_t(\boldsymbol{x}_t) + \ell_t(\boldsymbol{x}_t)) - (F_t(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_{t+1})) + \psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1}) \\ &\quad\leq \langle \hat{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{\lambda_t}{2}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 + \psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1}),\end{aligned}$$

for all $\hat{\boldsymbol{g}}_t \in \partial(F_t + \ell_t + \iota_{\mathcal{V}})(\boldsymbol{x}_t)$. Observing that $\boldsymbol{x}_t = \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} F_t(\boldsymbol{x}) + \tilde{\ell}_t(\boldsymbol{x})$, we have $\boldsymbol{0} \in \partial(F_t + \tilde{\ell}_t + \iota_{\mathcal{V}})(\boldsymbol{x}_t)$. Hence, using Theorem 2.23, $\partial(\ell_t - \tilde{\ell}_t)(\boldsymbol{x}_t) \subseteq \partial(F_t + \ell_t + \iota_{\mathcal{V}})(\boldsymbol{x}_t)$.

For the second inequality, by the definition of dual norms, we have that

$$\langle \hat{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{\lambda_t\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2}{2} \leq \|\hat{\boldsymbol{g}}_t\|_\star\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\| - \frac{\lambda_t\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2}{2} \leq \frac{\|\hat{\boldsymbol{g}}_t\|_\star^2}{2\lambda_t}\,. \quad \square$$

7.12.1 Improved Optimistic FTRL Bound for Linear Losses

We now prove a similar improved regret guarantee for Optimistic FTRL with linearized losses. The analysis will use an *auxiliary* sequence of predictions, and then relate these predictions to the ones of Optimistic FTRL.

Define $\boldsymbol{x}_t^{(a)}$ for $t = 1, \dots, T$ the auxiliary sequence of predictions obtained with Optimistic FTRL with linearized losses with losses $\ell_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$ and linear hints $\tilde{\ell}_t(\boldsymbol{x}) = \langle \tilde{\boldsymbol{g}}_t^{(a)}, \boldsymbol{x} \rangle$. Given that this auxiliary sequence is only used in the analysis, we do not have to specify the choice of the hints $\tilde{\boldsymbol{g}}_t^{(a)}$. Instead, we will leave them free, and state the final bound for any of these choices. Formally, defining $F_t^{(a)}(\boldsymbol{x}) := \psi_t(\boldsymbol{x}) + \langle \tilde{\boldsymbol{g}}_t^{(a)}, \boldsymbol{x} \rangle + \sum_{i=1}^{t-1} \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle$ we have $\boldsymbol{x}_t^{(a)} = \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \; F_t^{(a)}(\boldsymbol{x})$.

Theorem 7.48 *With the notation in Algorithm 7.8, let $\mathcal{V}$ be convex, closed, non-empty. Assume $\ell_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$ and $\tilde{\ell}_t(\boldsymbol{x}) = \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x} \rangle$ for $t = 1, \dots, T$. For $t = 1, \dots, T$, if ψ_t is closed, subdifferentiable, and λ_t-strongly convex with respect to $\|\cdot\|$ in $\mathcal{V}$, then $\boldsymbol{x}_t$ exists and is unique. Moreover, if in addition $\psi_{t+1} \geq \psi_t$ pointwise for all t, we have*

$$\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \leq \psi_T(\boldsymbol{u}) - \min_{\boldsymbol{x} \in \mathcal{V}} \psi_1(\boldsymbol{x}) + \sum_{t=1}^{T} \left[\frac{\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t^{(a)}\|_\star^2}{2\lambda_t} + \frac{\|\boldsymbol{g}_t\|_\star \|\tilde{\boldsymbol{g}}_t - \tilde{\boldsymbol{g}}_t^{(a)}\|_\star}{\lambda_t} \right],$$

for all $\boldsymbol{u} \in \mathcal{V}$ and all $\tilde{\boldsymbol{g}}_t^{(a)} \in \mathbb{R}^d$ for $t = 1, \dots, T$.

To prove this theorem, we will need the following stability lemma for FTRL.

Lemma 7.49 *Assume $\psi : \mathcal{V} \to \mathbb{R}$ to be λ-strongly convex with respect to $\|\cdot\|$. Define $\boldsymbol{x}^{(1)} := \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \; \psi(\boldsymbol{x}) + \langle \boldsymbol{\theta}^{(1)}, \boldsymbol{x} \rangle$ and $\boldsymbol{x}^{(2)} := \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \; \psi(\boldsymbol{x}) + \langle \boldsymbol{\theta}^{(2)}, \boldsymbol{x} \rangle$. Then, $\|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(2)}\| \leq \frac{1}{\lambda} \|\boldsymbol{\theta}^{(1)} - \boldsymbol{\theta}^{(2)}\|_\star$.*

Proof Define $F^{(1)}(\boldsymbol{x}) := \psi(\boldsymbol{x}) + \langle \boldsymbol{\theta}^{(1)}, \boldsymbol{x} \rangle$ and $F^{(2)}(\boldsymbol{x}) := \psi(\boldsymbol{x}) + \langle \boldsymbol{\theta}^{(2)}, \boldsymbol{x} \rangle$. From the strong convexity of ψ, we have

$$\begin{aligned} F^{(1)}(\boldsymbol{x}^{(2)}) - F^{(1)}(\boldsymbol{x}^{(1)}) &\geq \frac{\lambda}{2} \|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(2)}\|^2 \\ F^{(2)}(\boldsymbol{x}^{(1)}) - F^{(2)}(\boldsymbol{x}^{(2)}) &\geq \frac{\lambda}{2} \|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(2)}\|^2 \,. \end{aligned}$$

Hence, summing these two inequalities, we have

$$\|\boldsymbol{\theta}^{(1)} - \boldsymbol{\theta}^{(2)}\|_\star \, \|\boldsymbol{x}^{(2)} - \boldsymbol{x}^{(1)}\| \geq \langle \boldsymbol{\theta}^{(1)} - \boldsymbol{\theta}^{(2)}, \boldsymbol{x}^{(2)} - \boldsymbol{x}^{(1)} \rangle \geq \lambda \|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(2)}\|^2,$$

that completes the proof. □

We can now prove Theorem 7.48.

Proof of Theorem 7.48 Using Theorem 7.47 on the predictions $\boldsymbol{x}_t^{(a)}$, for all $\boldsymbol{u} \in \mathcal{V}$, we can

upper bound the regret of $\boldsymbol{x}_t$ as

$$\begin{aligned}\sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle &= \sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(a)} - \boldsymbol{u}\rangle + \sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_t^{(a)}\rangle \\ &\leq \psi_T(\boldsymbol{u}) - \psi_1(\boldsymbol{x}_1^{(a)}) + \sum_{t=1}^{T}\left(\frac{\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t^{(a)}\|_\star^2}{2\lambda_t} + \|\boldsymbol{g}_t\|_\star \|\boldsymbol{x}_t - \boldsymbol{x}_t^{(a)}\|\right) \\ &\leq \psi_T(\boldsymbol{u}) - \min_{\boldsymbol{x}\in\mathcal{V}}\psi_1(\boldsymbol{x}) + \sum_{t=1}^{T}\left(\frac{\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t^{(a)}\|_\star^2}{2\lambda_t} + \|\boldsymbol{g}_t\|_\star \|\boldsymbol{x}_t - \boldsymbol{x}_t^{(a)}\|\right).\end{aligned}$$

Now, we focus our attention on $\|\boldsymbol{x}_t - \boldsymbol{x}_t^{(a)}\|$. From Lemma 7.49, we have $\|\boldsymbol{x}_t^{(a)} - \boldsymbol{x}_t\| \leq \frac{1}{\lambda_t}\|\tilde{\boldsymbol{g}}_t - \tilde{\boldsymbol{g}}_t^{(a)}\|_\star$.

Using this inequality in the regret bound above, for any $\tilde{\boldsymbol{g}}_1^{(a)}, \ldots, \tilde{\boldsymbol{g}}_T^{(a)}$ and any $\boldsymbol{u} \in \mathcal{V}$, we have

$$\begin{aligned}\sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle &= \sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(a)} - \boldsymbol{u}\rangle + \sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{x}_t^{(a)}\rangle \\ &\leq \psi_T(\boldsymbol{u}) - \min_{\boldsymbol{x}\in\mathcal{V}}\psi_1(\boldsymbol{x}) + \sum_{t=1}^{T}\left(\frac{\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t^{(a)}\|_\star^2}{2\lambda_t} + \frac{\|\boldsymbol{g}_t\|_\star \|\tilde{\boldsymbol{g}}_t - \tilde{\boldsymbol{g}}_t^{(a)}\|_\star}{\lambda_t}\right).\end{aligned}$$

Remember that $\tilde{\boldsymbol{g}}_t^{(a)}$ is only used in the analysis. So, this upper bound holds for any choice of $\tilde{\boldsymbol{g}}_t^{(a)}$. □

Note that the minimization with respect to $\tilde{\boldsymbol{g}}_t^{(a)}$ can be solved only with the knowledge of the dual norm used. However, we have the following upper bound (the proof of the first inequality is left as an exercise, see Problem 7.14):

$$\begin{aligned}&\min_{\tilde{\boldsymbol{g}}_t^{(a)}} \frac{\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t^{(a)}\|_\star^2}{2} + \|\boldsymbol{g}_t\|_\star \|\tilde{\boldsymbol{g}}_t - \tilde{\boldsymbol{g}}_t^{(a)}\|_\star \\ &\quad\leq \frac{1}{2}\left[\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t\|_\star^2 - \max(\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t\|_\star - \|\boldsymbol{g}_t\|_\star, 0)^2\right] \\ &\quad\leq \frac{1}{2}\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t\|_\star \min(\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t\|_\star, 2\|\boldsymbol{g}_t\|_\star)\,.\end{aligned}$$

Let's compare this last expression to the similar one in the regret bound for FTRL. If the prediction of the subgradient of the next loss is good, that is, $\boldsymbol{g}_t \approx \tilde{\boldsymbol{g}}_t$, that term can become smaller and possibly even zero! On the other hand, if the predictions are bad, we only pay linearly in $\|\boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t\|_\star$. Overall, in the best case, we can gain a lot; in the worst case, we do not lose that much.

Example 7.50 *In the case of linear losses $\ell_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle$ and strongly convex regularizers, the update of optimistic FTRL becomes*

$$\boldsymbol{x}_t = \nabla\psi_{\mathcal{V},t}^\star\left(-\tilde{\boldsymbol{g}}_t - \sum_{i=1}^{t-1}\boldsymbol{g}_i\right),$$

where $\psi_{\mathcal{V},t} = \psi_t + \iota_{\mathcal{V}}$ *and we used the hint* $\tilde{\ell}_t(\boldsymbol{x}) = \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x} \rangle$. *For example, if* $\psi_t(\boldsymbol{x}) = \frac{\sqrt{t}}{2}\|\boldsymbol{x}\|_2^2$, *we have*

$$\boldsymbol{x}_t = \Pi_{\mathcal{V}} \left(\frac{-\tilde{\boldsymbol{g}}_t - \sum_{i=1}^{t-1} \boldsymbol{g}_i}{\sqrt{t}} \right),$$

where $\Pi_{\mathcal{V}}$ *is the projection onto* $\mathcal{V}$.

Despite the simplicity of the algorithm and its analysis, there are many applications of this principle. Here, we will only describe a few of them, while in Section 15.5 we describe the applications of optimistic algorithms for saddle-point optimization.

7.13 Application: FTRL with Delayed Feedback

In Section 6.12, we saw that it is possible to reduce OMD with delayed feedback to Optimistic OMD. Here, we will do the same for FTRL, obtaining a slightly different bound.

Similarly to the derivation in Section 6.12, we assume to receive the feedback with a delay of τ rounds. So, using FTRL with linearized losses with delays means that we predict with $\boldsymbol{x}_t = \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \psi_t(\boldsymbol{x})$ for $t \leq \tau + 1$ and

$$\boldsymbol{x}_t = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1-\tau} \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle, \quad t \geq \tau + 2 \,.$$

On the other hand, in Optimistic FTRL without delays and linear losses, we predict with

$$\boldsymbol{x}_t = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \psi_t(\boldsymbol{x}) + \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x} \rangle + \sum_{i=1}^{t-1} \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle \,.$$

Hence, the two updates are equivalent if we set $\tilde{\boldsymbol{g}}_t = -\sum_{i=\max(1,t-\tau)}^{t-1} \boldsymbol{g}_i$.

So, we can choose $\tilde{\boldsymbol{g}}_t^{(a)} = \boldsymbol{g}_t$ in Theorem 7.48. Then, using the delay-to-optimism conversion above, we have that FTRL with delayed linearized losses and increasing regularizer satisfies

$$\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \leq \psi_T(\boldsymbol{u}) - \min_{\boldsymbol{x} \in \mathcal{V}} \ \psi_1(\boldsymbol{x}) + \sum_{t=1}^{T} \frac{\|\boldsymbol{g}_t\|_\star \| \sum_{i=\max(1,t-\tau)}^{t} \boldsymbol{g}_i \|_\star}{\lambda_t}, \quad \forall \boldsymbol{u} \in \mathcal{V} \,. \tag{7.13}$$

That is, assuming that the $\boldsymbol{g}_t$ are bounded, we improved the dependence in the sum from $\mathcal{O}(\tau^2)$ to $\mathcal{O}(\tau)$. Choosing the strong convexity of the regularizer optimally, we obtain the optimal regret guarantee of $\mathcal{O}(\sqrt{\tau T})$.

7.14 Application: Regret that Depends on the Variance of the Subgradients

Consider running Optimistic-FTRL on the linearized losses $\ell_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$. We can gain something out of the Optimistic-FTRL compared to plain FTRL if we can predict the next $\boldsymbol{g}_t$. A simple possibility is to predict the average of the past values, $\bar{\boldsymbol{g}}_t = \frac{1}{t-1} \sum_{i=1}^{t-1} \boldsymbol{g}_i$.

Indeed, from the first chapter, we know that such a strategy is itself an online learning procedure! In particular, it corresponds to a FTL algorithm on the losses $\boldsymbol{x} \mapsto \|\boldsymbol{x} - \boldsymbol{g}_t\|_2^2$. Hence, from the strong convexity of these losses and assuming $\|\boldsymbol{g}_t\|_2 \leq 1$, we know that

$$\sum_{t=1}^{T} \|\bar{\boldsymbol{g}}_t - \boldsymbol{g}_t\|_2^2 - \sum_{t=1}^{T} \|\boldsymbol{u} - \boldsymbol{g}_t\|_2^2 \leq 4(1 + \ln T), \quad \forall \boldsymbol{u} \in \mathbb{R}^d \,.$$

This implies

$$\sum_{t=1}^{T} \|\bar{\boldsymbol{g}}_t - \boldsymbol{g}_t\|_2^2 \leq 4(1 + \ln T) + \min_{\boldsymbol{u}} \sum_{t=1}^{T} \|\boldsymbol{u} - \boldsymbol{g}_t\|_2^2 \,.$$

It is immediate to see that the minimizer is $\boldsymbol{u} = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{g}_t$, which results in T times the empirical "variance" of the subgradients, that is, the sum of the empirical variances of the coordinates of the subgradients. Plugging it into the Optimistic-FTRL regret, with $\psi_t = \psi$, we have

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \leq \psi(\boldsymbol{u}) - \psi(\boldsymbol{x}_1) + \frac{4 + 4\ln T + \min_{\boldsymbol{g}} \sum_{t=1}^{T} \|\boldsymbol{g}_t - \boldsymbol{g}\|_2^2}{2\lambda} \,.$$

Remark 7.51 *Instead of using the mean of the past subgradients, we could use any other strategy or even a mix of different strategies. For example, assuming the subgradients are bounded, we could use an algorithm to solve the LEA problem, where each expert is a strategy. Then, we would obtain a bound that depends on the predictions of the best strategy, plus the regret of the expert algorithm.*

7.15 Application: Online Convex Optimization with Gradual Variations

In this section, we consider the case where the losses we receive have small variations over time. We will show that in this case, it is possible to get constant regret in the case that the losses are equal.

In this case, the simple strategy we can use to predict the next subgradient is to use the previous one, that is $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_{t-1}, \boldsymbol{x} \rangle$ for $t \geq 2$ and $\tilde{\ell}_1(\boldsymbol{x}) = 0$.

Corollary 7.52 *Under the assumptions of Theorem 7.47, also assume that the losses ℓ_t are m-smooth with respect to $\|\cdot\|$. Define $\tilde{\ell}_t(\boldsymbol{x}) = \langle \nabla \ell_{t-1}(\boldsymbol{x}_{t-1}), \boldsymbol{x} \rangle$ for $t \geq 2$ and $\tilde{\ell}_1(\boldsymbol{x}) = 0$ for all $\boldsymbol{x} \in \mathcal{V}$. Set $\psi_t(\boldsymbol{x}) = \lambda_t \psi(\boldsymbol{x})$ where ψ is 1-strongly convex with respect to $\|\cdot\|$ and $\lambda_t > 0$ satisfies $\lambda_t \lambda_{t-1} \geq 8m^2$ and $\lambda_t \geq \lambda_{t-1}$ for $t = 2, \dots, T$. Then, $\forall \boldsymbol{u} \in \mathcal{V}$, we have*

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \leq \lambda_T \psi(\boldsymbol{u}) - \lambda_1 \psi(\boldsymbol{x}_1) + \frac{1}{\lambda_1} \|\nabla \ell_1(\boldsymbol{x}_1)\|_\star^2 + \sum_{t=2}^{T} \frac{2}{\lambda_t} \|\nabla \ell_t(\boldsymbol{x}_{t-1}) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1})\|_\star^2 \,.$$

In addition, assume that in each round t we can also query the gradient of ℓ_t in $\boldsymbol{x}_{t-1}$, fix

$\alpha > 0$, assume $\|\nabla \ell_t(\boldsymbol{x})\|_\star \le L$ for all $\boldsymbol{x} \in \mathcal{V}$, and set

$$\lambda_t = \frac{1}{\alpha}\sqrt{\max(8\alpha^2 m^2, 4L^2) + \sum_{i=2}^{t-1} \|\nabla \ell_i(\boldsymbol{x}_{i-1}) - \nabla \ell_{i-1}(\boldsymbol{x}_{i-1})\|_\star^2}\,.$$

Then, we have

$$\begin{aligned}
&\sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}) \\
&\le \left(\frac{\psi(\boldsymbol{u}) - \psi(\boldsymbol{x}_1)}{\alpha} + 4\alpha\right)\sqrt{\max(8\alpha^2 m^2, 4L^2) + \sum_{t=2}^T \|\nabla \ell_t(\boldsymbol{x}_{t-1}) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1})\|_\star^2} \\
&\quad + \frac{\alpha L}{2}\,.
\end{aligned}$$

Proof From the Optimistic-FTRL bound, we immediately get

$$\begin{aligned}
&\sum_{t=1}^T (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \\
&\quad \le \lambda_T \psi(\boldsymbol{u}) - \lambda_1 \psi(\boldsymbol{x}_1) + \sum_{t=1}^T \left[\langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{\lambda_t \|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2}{2}\right],
\end{aligned}$$

where $\boldsymbol{g}_t = \nabla \ell_t(\boldsymbol{x}_t)$. Now, consider the case that the losses ℓ_t are m-smooth. So, for any $\beta_t > 0$, we have

$$\begin{aligned}
&\langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{\lambda_t}{2}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 \\
&\quad = \langle \nabla \ell_t(\boldsymbol{x}_t) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1}), \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{\lambda_t}{2}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 \\
&\quad \le \frac{\beta_t}{2}\|\nabla \ell_t(\boldsymbol{x}_t) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1})\|_\star^2 + \frac{1}{2\beta_t}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 - \frac{\lambda_t}{2}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2\,.
\end{aligned}$$

Focusing on the first term, for $t = 2, \ldots, T$, we have

$$\begin{aligned}
&\frac{\beta_t}{2}\|\nabla \ell_t(\boldsymbol{x}_t) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1})\|_\star^2 \\
&\quad = \frac{\beta_t}{2}\|\nabla \ell_t(\boldsymbol{x}_t) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1}) - \nabla \ell_t(\boldsymbol{x}_{t-1}) + \nabla \ell_t(\boldsymbol{x}_{t-1})\|_\star^2 \\
&\quad \le \beta_t \|\nabla \ell_t(\boldsymbol{x}_t) - \nabla \ell_t(\boldsymbol{x}_{t-1})\|_\star^2 + \beta_t \|\nabla \ell_t(\boldsymbol{x}_{t-1}) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1})\|_\star^2 \\
&\quad \le \beta_t m^2 \|\boldsymbol{x}_t - \boldsymbol{x}_{t-1}\|^2 + \beta_t \|\nabla \ell_t(\boldsymbol{x}_{t-1}) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1})\|_\star^2\,.
\end{aligned}$$

Choose $\beta_t = \frac{2}{\lambda_t}$. We have for $t = 2, \ldots, T$

$$\begin{aligned}
&\langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{\lambda_t}{2}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 \\
&\quad \le \frac{2m^2}{\lambda_t}\|\boldsymbol{x}_t - \boldsymbol{x}_{t-1}\|^2 + \frac{2}{\lambda_t}\|\nabla \ell_t(\boldsymbol{x}_{t-1}) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1})\|_\star^2 - \frac{\lambda_t}{4}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2\,.
\end{aligned}$$

For $t = 1$, using the fact that $\tilde{\ell}_1$ is the null function, we have

$$\langle \boldsymbol{g}_1 - \tilde{\boldsymbol{g}}_1, \boldsymbol{x}_1 - \boldsymbol{x}_2 \rangle - \frac{\lambda_1}{2}\|\boldsymbol{x}_1 - \boldsymbol{x}_2\|^2 \leq \frac{1}{\lambda_1}\|\nabla \ell_1(\boldsymbol{x}_1)\|_\star^2 - \frac{\lambda_1}{4}\|\boldsymbol{x}_1 - \boldsymbol{x}_2\|^2 .$$

Now, observe that the assumption on λ_t implies $\frac{2m^2}{\lambda_t} \leq \frac{\lambda_{t-1}}{4}$ for $t = 2, \dots, T$. So, summing for $t = 1, \dots, T$, we have

$$\begin{aligned}
&\sum_{t=1}^{T} \left(\langle \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{x}_{t+1} \rangle - \frac{\lambda_t}{2}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 \right) \\
&\qquad \leq \frac{1}{\lambda_1}\|\nabla \ell_1(\boldsymbol{x}_1)\|_\star^2 + \sum_{t=2}^{T} \frac{2}{\lambda_t}\|\nabla \ell_t(\boldsymbol{x}_{t-1}) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1})\|_\star^2 .
\end{aligned}$$

Putting everything together, we have the first stated bound.

The second one is obtained by observing that

$$\begin{aligned}
&\sum_{t=2}^{T} \frac{\|\nabla \ell_t(\boldsymbol{x}_{t-1}) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1})\|_\star^2}{\lambda_t} \\
&\quad = \alpha \sum_{t=2}^{T} \frac{\|\nabla \ell_t(\boldsymbol{x}_{t-1}) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1})\|_\star^2}{\sqrt{\max(8\alpha^2 m^2, 4L^2) + \sum_{i=2}^{t-1} \|\nabla \ell_i(\boldsymbol{x}_{i-1}) - \nabla \ell_{i-1}(\boldsymbol{x}_{i-1})\|_\star^2}} \\
&\quad \leq \alpha \sum_{t=2}^{T} \frac{\|\nabla \ell_t(\boldsymbol{x}_{t-1}) - \nabla \ell_{t-1}(\boldsymbol{x}_{t-1})\|_\star^2}{\sqrt{\sum_{i=2}^{t} \|\nabla \ell_i(\boldsymbol{x}_{i-1}) - \nabla \ell_{i-1}(\boldsymbol{x}_{i-1})\|_\star^2}} \\
&\quad \leq 2\alpha \sqrt{\sum_{i=2}^{T} \|\nabla \ell_i(\boldsymbol{x}_{i-1}) - \nabla \ell_{i-1}(\boldsymbol{x}_{i-1})\|_\star^2} . \qquad \square
\end{aligned}$$

Note that if the losses are all the same, the regret becomes a constant! This is not surprising because the prediction of the next loss is a linear approximation of the previous loss. Indeed, looking back at the proof, the key idea is to use the smoothness to argue that, even if the past subgradient was taken in a different point than the current one, it is still a good prediction of the current subgradient.

Remark 7.53 *Note that the assumption of smoothness is necessary. Indeed, always passing the same function and using the online-to-batch conversion would result in a convergence rate of $\mathcal{O}(1/T)$ for a Lipschitz function, which is impossible by the lower bound in offline convex optimization.*

7.16 History Bits

The FTRL algorithm has a peculiar story. It was introduced by *four* different groups roughly around 1999-2006, following four different points of view. The algorithm was officially introduced in Abernethy et al. (2008); Hazan and Kale (2008) where at each step the prediction is computed as the minimizer of a regularization term plus the sum of losses on all past rounds. However, the key ideas were already presented years before. Gordon (1999a,b)

appear to have proposed it for the first time, naming it Maximum a Posteriori with a clear inspiration to Bayesian methods. Also, the analysis in Gordon (1999a,b) is the first one to use concepts from convex analysis. Later, Agarwal and Hazan (2005) proposed to add a regularizer to stabilize the FTL algorithm (Hannan, 1957) in an algorithm named Smooth Prediction for the specific application of Portfolio Selection. A little bit later, Shai Shalev-Shwartz and Yoram Singer proposed a very general analysis of FTRL using a dual perspective (Shalev-Shwartz and Singer, 2006a,b). In particular, the PhD thesis of Shai Shalev-Shwartz (Shalev-Shwartz, 2007) contains the most precise dual analysis of FTRL, which also allows multiple dual updates and regularizers with a time-varying weight. However, he called it "online mirror descent" because the name FTRL was only invented later! (I also contributed to the confusion, naming a general analysis of FTRL with time-varying regularizer and linear losses "generalized online mirror descent" (Orabona et al., 2015). So, now I am trying to set the record straight!) The offline optimization community also knows FTRL, but only with linear losses, and they call it Dual Averaging (Nesterov, 2009). Note that the paper Nesterov (2009) is actually from 2005[1], and in the paper Nesterov claims that these ideas are from 2001-2002, but he decided not to publish them for a while because he was convinced that "the importance of black-box approach in Convex Optimization will be irreversibly vanishing, and, finally, this approach will be completely replaced by other ones based on a clever use of problem's structure (interior-point methods, smoothing, etc.)." As Shai Shalev-Shwartz, Nesterov looks at FTRL from the dual point of view, but he focuses only on linear losses and only in the offline case.

Finally, McMahan (2017) gives the elegant equality result that I presented here (with minor improvements) that holds for general loss functions and regularizers. A similar equality, setting $\boldsymbol{u} = \boldsymbol{x}_{T+1}$ and specialized only to linear losses, was also proven in Abernethy et al. (2014), essentially matching the inequality in Orabona et al. (2015). Note that Dual Averaging stems from the dual interpretation of FTRL for linear losses, but Lemma 7.1 underlines the fact that FTRL is actually more general. People used to prove the regret bound for FTRL using the so-called Be-the-Leader-Follow-the-Leader lemma (see Lemma 1.2 and Problem 7.3). However, the proof is off by a factor of 2 in the case of fixed regularizers (McMahan, 2017). Moreover, it seems to fail in the case of generic strongly convex increasing regularizers, while it works for the particular case of proximal regularizers (McMahan, 2017).

Another source of confusion stems from the fact that some people differentiate between a "lazy" and "greedy" version of OMD. Zinkevich (2003) introduced both "greedy projection" and "lazy projection" algorithms. The greedy projection algorithm was later renamed projected online gradient descent. We now know that the lazy projection algorithm is FTRL with linearized losses in the special case of $\psi_t(\boldsymbol{x}) = \frac{\eta}{2}\|\boldsymbol{x}\|_2^2$. More generally, as proved in McMahan (2017), what people call the lazy version of OMD is just FTRL with linearized losses, and the greedy one is just OMD.

The concept of generalized Bregman divergence has been rediscovered many times in the literature, with different names too. It was first introduced as the canonical divergence of dually flat spaces in information geometry in Amari (1985, Eq. 3.21). Independently, it was proposed in (online) optimization by Gordon (1999a,b) to analyze FTRL for fixed

[1] https://papers.ssrn.com/sol3/papers.cfm?abstract_id=912637

regularizers. The equality in Remark 7.11 might be new, but immediate from the regret equality of FTRL.

The second result in Lemma 7.15 is a distillation of the reasoning used in Zimmert and Seldin (2019, 2021), but without using duality concepts.

The use of FTRL with the regularizer in (7.6) was proposed in Orabona and Pál (2015, 2018), while here I presented a simpler version of their proof that does not require Fenchel conjugates. The AdaHedge algorithm was introduced in van Erven et al. (2011) and refined in de Rooij et al. (2014). The analysis reported here is new, but inspired by the one in Orabona and Pál (2015, 2018), which generalized AdaHedge to arbitrary regularizers in AdaFTRL. Additional properties of AdaHedge for the stochastic case were proven in de Rooij et al. (2014).

The first implicit use of the $(2, 1)$ group norm I could find is in Bakin (1999), where it is used as a constraint in the first proposed formulation of Group Lasso. However, there is no mention of the fact that this constraint is a norm. Ding et al. (2006) propose the use of the $(2, 1)$ group norm for Principal Component Analysis, calling it R_1 norm. A few months later, the same group norm was proposed independently in Argyriou et al. (2006). Agarwal et al. (2008) extended the $(2, 1)$ group norm to (p, s) and used it in OMD for multitask online learning. Kakade et al. (2009) proved the smoothness and strong convexity of squared group norms. Jie et al. (2010) used the $(2, s)$ group norm FTRL for online multi-kernel learning, while Orabona et al. (2010, 2012b) used the same norm in a Perceptron-like algorithm.

Nesterov's Dual Averaging was extended to deal with composite losses by Xiao (2009, 2010) in the stochastic and online learning setting, essentially rediscovering the framework of Shalev-Shwartz, and calling the resulting framework Regularized Dual Averaging. The analysis presented here using the negative terms $\psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1})$ to easily prove regret bounds for FTRL for composite losses is from Orabona et al. (2015).

The first proof of FTRL for strongly convex losses was in Shalev-Shwartz and Singer (2007) (even if they do not call it FTRL). In the same paper, they also define strong convexity with respect to Bregman divergences, rather than just norms, and prove the same logarithmic regret bound (see Problem 6.7). Analogously, people later defined the concept of smoothness with respect to a Bregman divergence (Birnbaum et al., 2010, 2011), rediscovered in Bauschke et al. (2017). Later, both concepts were rebranded as "relative smoothness" and "relative strong convexity" (Lu et al., 2018).

FTRL-Proximal was proposed in McMahan and Streeter (2010) for AdaGrad and then further analyzed in McMahan (2011). There is an interesting bit about FTRL-Proximal: it became very famous in internet companies when Google disclosed in a very influential paper that they were using FTRL-Proximal to train the classifier for click prediction (McMahan et al., 2013). This generated some confusion because many people started conflating the term FTRL-Proximal (a specific algorithm) with FTRL (an entire family of algorithms).

The ONS algorithm was introduced in Hazan et al. (2006, 2007) and it is described for the particular case that the loss functions are exp-concave. The assumption in (7.10) with a fixed matrix $\boldsymbol{A}$ for all points $\boldsymbol{x}$ is called $\boldsymbol{A}$-restricted strong convexity in Ryu and Boyd (2014), and it was used for the first time in an online learning algorithm in this book. Exp-concave functions were introduced in Kivinen and Warmuth (1999). Here, I have extended them to extended-value functions. Theorem 7.35 and Example 7.36 are from Kivinen and Warmuth (1999). Here, I described a slight generalization for any sequence of functions

that satisfy (7.12), that in my view better shows the parallel between FTRL over strongly convex functions and ONS. Note that Hazan et al. (2006, 2007) also describes a variant of ONS based on OMD, but I find its analysis less interesting from a didactic point of view. Lemma 7.29 dates back to Lai and Wei (1982, Lemma 2). Orabona et al. (2012a) obtained a logarithmic $\mathscr{L}^\star$ bound, i.e., proportional to $\ln(1 + \sum_{t=1}^T \ell_t(\boldsymbol{u}))$, for exp-concave and smooth losses using ONS.

The Vovk–Azoury–Warmuth forecaster was introduced independently by Azoury and Warmuth (2001) and Vovk (2001), and its interpretation as hallucinating the future loss is by Azoury and Warmuth (2001). The proof presented here is from Orabona et al. (2015).

The Optimistic FTRL version was proposed in Rakhlin and Sridharan (2013b) but analyzed only with fixed self-concordant regularizers. Steinhardt and Liang (2014) proposed Optimistic FTRL as we know it, even if it was called "Optimistic Mirror Descent" for the misnaming problem I have explained above. Due to the misnaming, Mohri and Yang (2016) appeared to have rediscovered Optimistic FTRL with non-self-concordant regularizers. The proof of Theorem 7.47 I present here is new. Theorem 7.48 is from Flaspohler et al. (2021) as well as the reduction from delayed feedback to optimism in Section 7.13. See the History Bits of Chapter 6 for additional pointers to online learning with delayed feedback.

Corollary 7.52 was proved by Chiang et al. (2012) for Optimistic OMD and presented in a similar form for Optimistic FTRL in Joulani et al. (2017), but only for bounded domains.

7.17 Exercises

Problem 7.1 *Prove that the update of FTRL with linearized loss in Example 7.12 is correct.*

Problem 7.2 *Consider FTRL with linearized losses with regularizers $\psi_t(\boldsymbol{x})$. The regret equality in Lemma 7.1 has a term equal to $F_{T+1}(\boldsymbol{x}_{T+1}) - F_{T+1}(\boldsymbol{u})$. Assume that $\mathcal{V}$ is convex and non-empty, and ψ_{T+1} is strictly convex and differentiable at the point $\boldsymbol{x}_{T+1}$. Then, show that $F_{T+1}(\boldsymbol{x}_{T+1}) - F_{T+1}(\boldsymbol{u}) \leq -B_{\psi_{T+1}}(\boldsymbol{u}; \boldsymbol{x}_{T+1})$ for any $\boldsymbol{u} \in \mathcal{V}$.*

Problem 7.3 *Consider FTRL with losses $\ell_t(\boldsymbol{x})$ and regularizers $\psi_t(\boldsymbol{x})$. Assume that $\psi_{t+1} \geq \psi_t$ for $t = 1, \dots, T-1$. Denote by $F_t(\boldsymbol{x}) := \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t-1} \ell_i(\boldsymbol{x})$ and show that*

$$F_t(\boldsymbol{x}_t) - F_{t+1}(\boldsymbol{x}_{t+1}) + \ell_t(\boldsymbol{x}_t) \leq \ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{x}_{t+1}) \,.$$

Problem 7.4 *We can obtain $\mathscr{L}^\star$ bounds for smooth and Lipschitz losses with linearized FTRL, even without knowing the Lipschitz constant. In particular, show that under these assumptions the regularizers $\psi_t(\boldsymbol{x}) = \frac{\lambda_t}{2}\|\boldsymbol{x}\|_2^2$, where $\lambda_t = \sqrt{a + \sum_{i=1}^{t-1} \|\boldsymbol{g}_i\|_2^2}$ and $a > 0$, would give an $\mathscr{L}^\star$ bound. Hint: use Gaillard et al. (2014, Lemma 14).*

Problem 7.5 *Define $\lambda_t = a + \sum_{i=1}^{t-1} \frac{b_i}{\lambda_i}$, where $a > 0$. Let $b_t \in [0, m]$ and prove that $\sum_{t=1}^T \frac{b_t}{\lambda_t} \leq \sqrt{2\sum_{t=1}^T b_t} + \frac{m}{a}$. Use this inequality to design an alternative regularizer that solves the previous exercise. Hint: see Sachs et al. (2022, Proof of Theorem 5).*

Problem 7.6 *Let $\mathcal{V} = \mathbb{R}^d$ and prove that the update in (7.9) is equivalent to the one of OSD with learning rate $\eta_t = \frac{1}{\sum_{i=1}^t \mu_i}$, when both algorithms start with $\boldsymbol{x}_1 = 0$.*

Problem 7.7 *Prove the statement in Example 7.37.*

Problem 7.8 *Design and analyze the FTRL version of OMD with p-norms in Section 6.7.*

Problem 7.9 *Consider the LEA setting and assume $\boldsymbol{g}_{t,i} \in [0,1]$ for all $t = 1, \dots T$ and $i = 1, \dots, d$. From the analysis of EG with local norms, see (7.5), derive a time-varying regularizer that gives a regret upper bound proportional to $\ln d + \sqrt{\mathscr{L}^\star(\boldsymbol{u}) \ln d}$, where $\mathscr{L}^\star(\boldsymbol{u}) := \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle$ for any $\boldsymbol{u} \in \Delta^{d-1}$.*

Problem 7.10 *Prove a regret bound for optimistic FTRL with proximal regularizers.*

Problem 7.11 *Prove that the losses $\ell_t(\boldsymbol{x}) = \frac{1}{2}(\langle \boldsymbol{z}_t, \boldsymbol{x} \rangle - y_t)^2$, where $\|\boldsymbol{z}_t\|_2 \le 1$ and $|y_t| \le 1$, are exp-concave on $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \le 1\}$, and find the exp-concavity constant.*

Problem 7.12 *Prove that the ONS update is equivalent to the following two steps:*

$$\tilde{\boldsymbol{x}}_t = S_{t-1}^{-1} \left(\sum_{i=1}^{t-1} \boldsymbol{A}_i \boldsymbol{x}_i - \sum_{i=1}^{t-1} \boldsymbol{g}_i \right)$$
$$\boldsymbol{x}_t = \operatorname*{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \|\boldsymbol{x} - \tilde{\boldsymbol{x}}_t\|_{\boldsymbol{S}_{t-1}},$$

where $\boldsymbol{A}_t = \mu \boldsymbol{g}_t \boldsymbol{g}_t^\top$ and $\boldsymbol{S}_t = \lambda \boldsymbol{I} + \sum_{i=1}^t \boldsymbol{A}_i$.

Problem 7.13 *Consider the online learning algorithm for linear regression with square loss that predicts in each round with*

$$\boldsymbol{x}_t = \operatorname*{argmin}_{\boldsymbol{x}} \ \lambda \|\boldsymbol{x}\|_2^2 + \sum_{i=1}^{t-1} (\langle \boldsymbol{z}_i, \boldsymbol{x} \rangle - y_i)^2 \ .$$

Prove for it the equality

$$\sum_{t=1}^T \frac{(\langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle - y_t)^2}{1 + d_t} = \min_{\boldsymbol{u}} \ \lambda \|\boldsymbol{u}\|_2^2 + \sum_{t=1}^T (\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle - y_t)^2,$$

where $d_t = \boldsymbol{z}_t^\top (\sum_{i=1}^{t-1} \boldsymbol{z}_i \boldsymbol{z}_i^\top + \lambda \boldsymbol{I})^{-1} \boldsymbol{z}_t$. Hint: see Zhdanov and Kalnishkan (2010, 2013).

Problem 7.14 *Let $\boldsymbol{g}, \tilde{\boldsymbol{g}} \in \mathbb{R}^d$ and $\|\cdot\|_\star$ a norm. Then, show that*

$$\min_{\tilde{\boldsymbol{g}}^{(a)}} \ \frac{\|\boldsymbol{g} - \tilde{\boldsymbol{g}}^{(a)}\|_\star^2}{2} + \|\boldsymbol{g}\|_\star \|\tilde{\boldsymbol{g}} - \tilde{\boldsymbol{g}}^{(a)}\|_\star \le \frac{1}{2} \left[\|\boldsymbol{g} - \tilde{\boldsymbol{g}}\|_\star^2 - \max(\|\boldsymbol{g} - \tilde{\boldsymbol{g}}\|_\star - \|\boldsymbol{g}\|_\star, 0)^2 \right] \ .$$

Problem 7.15 *In Section 7.14, we showed that it is possible to have a regret bound that depends on the sum of the empirical variances of the coordinates of the subgradients, using the average of the past subgradients as a prediction of the next losses. Prove that a similar bound is possible simply using the previous subgradients as predictions of the next losses, i.e., $\tilde{\boldsymbol{g}}_t = \boldsymbol{g}_{t-1}$.*

8

Online Linear Classification

In this chapter, we will consider the problem of *online linear classification*. We consider the following setting:

- At each time step, we receive a sample $\boldsymbol{z}_t \in \mathbb{R}^d$
- We output a prediction of the binary label $y_t \in \{-1, 1\}$ of $\boldsymbol{z}_t$
- We receive the true label, and we see whether we made a mistake or not
- We update our online classifier

Ideally, the aim of the online algorithm is to minimize the number of mistakes it makes compared to some best fixed classifier.

We will focus on linear classifiers, which predict with the sign of the inner product between a vector $\boldsymbol{x}_t$ and the input features $\boldsymbol{z}_t$. Hence, $\tilde{y}_t = \operatorname{sign}(\langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle)$. However, we will also allow some form of randomization.

This problem could be written as a regret minimization problem:

$$\operatorname{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}),$$

where $\ell_t(\boldsymbol{x}) = \mathbf{1}\{\operatorname{sign}(\langle \boldsymbol{z}_t, \boldsymbol{x} \rangle) \neq y_t\}$, where $\operatorname{sign}(0) := 0$. However, these losses are non-convex. Hence, we need an alternative way to deal with them. In the following, we will see three possible approaches to this problem.

8.1 Online Randomized Classifier

As we did for the Learning with Expert Advice (LEA) framework, we might think of convexifying the losses using randomization. Hence, on each round we can predict a number $q_t \in [-1, 1]$ and output the label $+1$ with probability $\frac{1+q_t}{2}$ and the label -1 with probability $\frac{1-q_t}{2}$. So, define the random variable

$$\tilde{Y}(q) := \begin{cases} +1, & \text{with probability } \frac{1+q}{2}, \\ -1, & \text{with probability } \frac{1-q}{2}. \end{cases} \tag{8.1}$$

Now observe that $\mathbb{E}[\mathbf{1}\{\tilde{Y}(q_t) \neq y_t\}] = \frac{1}{2}|q_t - y_t|$. If we consider linear predictors, we can set $q_t = \langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle$ and similarly for the competitor $q'_t = \langle \boldsymbol{z}_t, \boldsymbol{u} \rangle$. Assuming we can constrain both the algorithm and the competitor to the space of vectors where $|\langle \boldsymbol{z}_t, \boldsymbol{x} \rangle| \leq 1$

Algorithm 8.1 Randomized Online Linear Classifier through FTRL

Require: $R > 0$ such that $\|\boldsymbol{z}_t\|_2 \leq R$ for $t = 1, \dots, T$, $\lambda_t > 0$ for $t = 1, \dots, T$
1: Set $\boldsymbol{\theta}_1 = \mathbf{0} \in \mathbb{R}^d$
2: **for** $t = 1$ **to** T **do**
3: $\quad \boldsymbol{x}_t = \frac{\frac{1}{\lambda_t}\boldsymbol{\theta}_t}{\max(R\frac{1}{\lambda_t}\|\boldsymbol{\theta}_t\|_2, 1)}$
4: $\quad$ Receive $\boldsymbol{z}_t \in \mathcal{X}$
5: $\quad$ Predict $\tilde{y}_t = \tilde{Y}(\langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle)$, where $\tilde{Y}$ is defined in (8.1)
6: $\quad$ Receive y_t and pay the loss $\mathbf{1}\{y_t \neq \tilde{y}_t\}$
7: $\quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{1}{2}\text{sign}(\langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle - y_t)\boldsymbol{z}_t$ where $\text{sign}(0) := 0$
8: **end for**

for $t = 1, \dots, T$, we can write

$$\mathbb{E}\left[\sum_{t=1}^{T} \mathbf{1}\left\{\tilde{Y}(\langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle) \neq y_t\right\} - \sum_{t=1}^{T} \mathbf{1}\left\{\tilde{Y}(\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle) \neq y_t\right\}\right]$$
$$= \mathbb{E}\left[\sum_{t=1}^{T} \frac{1}{2}\left|\langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle - y_t\right| - \sum_{t=1}^{T} \frac{1}{2}\left|\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle - y_t\right|\right] .$$

Hence, the surrogate convex loss becomes $\tilde{\ell}_t(\boldsymbol{x}) = \frac{1}{2}|\langle \boldsymbol{z}_t, \boldsymbol{x} \rangle - y_t|$ and the feasible set is any convex set where we have the property $|\langle \boldsymbol{z}_t, \boldsymbol{x} \rangle| \leq 1$ for $t = 1, \dots, T$.

Given that this problem is convex, assuming $\boldsymbol{z}_t$ to be bounded with respect to some norm, we can use almost any of the algorithms we have seen till now, from Online Mirror Descent (OMD) to Follow-the-Regularized-Leader (FTRL). All of them would result in $\mathcal{O}(\sqrt{T})$ regret upper bounds. The only caveat is to restrict $\langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle$ in $[-1, 1]$. One way to ensure this is to assume $\|\boldsymbol{z}_t\|_\star \leq R$ and choose the feasible set $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\| \leq \frac{1}{R}\}$.

Putting everything together, we can have, for example, using FTRL, Algorithm 8.1.

Theorem 8.1 *Let $\mathcal{X} \subset \mathbb{R}^d$ and $(\boldsymbol{z}_t, y_t)_{t=1}^T$ be an arbitrary sequence of sample-label pairs where $(\boldsymbol{z}_t, y_t) \in \mathcal{X} \times \{-1, 1\}$. Assume $\|\boldsymbol{z}_t\|_2 \leq R$, for $t = 1, \dots, T$. Then, running Algorithm 8.1 with $\lambda_t = \frac{R^2\sqrt{2t}}{2}$, for any $\boldsymbol{u} \in \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \leq \frac{1}{R}\}$ we have the following guarantee:*

$$\mathbb{E}\left[\sum_{t=1}^{T} \mathbf{1}\{\tilde{Y}(\langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle) \neq y_t\}\right] - \mathbb{E}\left[\sum_{t=1}^{T} \mathbf{1}\{\tilde{Y}(\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle) \neq y_t\}\right] \leq \frac{\sqrt{2T}}{2} .$$

The proof follows from the FTRL regret bound with the chosen increasing regularizer, and it is left as an exercise.

8.2 The Perceptron Algorithm

The above strategy has the shortcoming of restricting the feasible vectors to a possibly very small set. In turn, this could make the best competitor's performance poor, and the performance of the online algorithm is only guaranteed to be close to that of the best competitor.

Another way to deal with the non-convexity is to compare the number of mistakes that

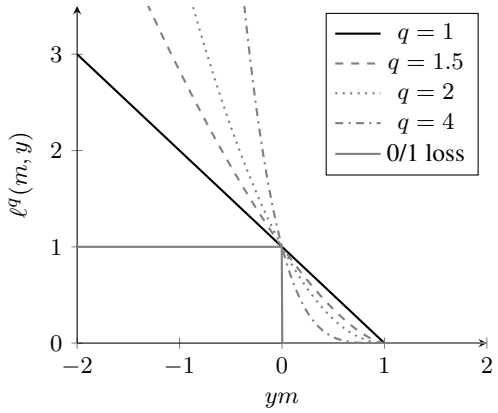


Figure 8.1 Powers of the hinge loss.

Algorithm 8.2 Perceptron

Set $\boldsymbol{x}_1 = \mathbf{0} \in \mathbb{R}^d$
for $t = 1$ **to** T **do**
  Receive $\boldsymbol{z}_t \in \mathcal{X}$
  Predict $\tilde{y}_t = \operatorname{sign}(\langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle)$ where $\operatorname{sign}(0) := 0$
  Receive y_t and pay the loss $\mathbf{1}\{y_t \tilde{y}_t \leq 0\}$
  $\boldsymbol{x}_{t+1} = \boldsymbol{x}_t + \mathbf{1}\{y_t \tilde{y}_t \leq 0\} y_t \boldsymbol{z}_t$
end for

the algorithm makes with a convex cumulative loss of the competitor. That is, we can try to prove a weaker regret guarantee, called **mistake bound**:

$$\sum_{t=1}^{T} \mathbf{1}\{y_t \neq \tilde{y}_t\} - \sum_{t=1}^{T} \ell(\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle, y_t) = \mathcal{O}\left(\sqrt{T}\right) . \tag{8.2}$$

In particular, the convex loss we consider is a *power* of the **hinge loss**: $\ell^q(m, y) := \max(1 - ym, 0)^q$. The hinge loss is a convex upper bound to the 0/1 loss, when the prediction $\tilde{y} = \operatorname{sign}(m)$, and it achieves the value of zero when the sign of the prediction is correct *and* the magnitude of the inner product is sufficiently large. Moreover, by taking powers, we obtain a family of functions that trades off the loss on misclassified samples against the loss on correctly classified samples whose prediction satisfies $|\langle \boldsymbol{z}_t, \boldsymbol{x} \rangle| \leq 1$, see Figure 8.1.

The oldest known algorithm to minimize the modified regret in (8.2) is the **Perceptron** algorithm, in Algorithm 8.2. The Perceptron algorithm updates the current prediction $\boldsymbol{x}_t$ by moving in the direction of the current sample multiplied by its label. Let's see why this is a good idea. Assume that $y_t = 1$ and the algorithm made a mistake. Then, the updated prediction $\boldsymbol{x}_{t+1}$ would predict a more positive number on the same sample $\boldsymbol{z}_t$. In fact, assuming $\|\boldsymbol{z}_t\|_2 > 0$, we have

$$\langle \boldsymbol{z}_t, \boldsymbol{x}_{t+1} \rangle = \langle \boldsymbol{z}_t, \boldsymbol{x}_t + y_t \boldsymbol{z}_t \rangle = \langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle + \|\boldsymbol{z}_t\|_2^2 > \langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle .$$

In the same way, if $y_t = -1$ and the algorithm made a mistake, the update would result in a more negative prediction on the same sample.

We can formalize the above intuition in the following guarantee.

Theorem 8.2 *Let $\mathcal{X} \subset \mathbb{R}^d$ and $(\boldsymbol{z}_t, y_t)_{t=1}^T$ be an arbitrary sequence of sample-label pairs where $(\boldsymbol{z}_t, y_t) \in \mathcal{X} \times \{-1, 1\}$. Assume $\|\boldsymbol{z}_t\|_2 \leq R$, for $t = 1, \dots, T$. Then, running the Perceptron algorithm, for all $\boldsymbol{u} \in \mathbb{R}^d$ and $q \geq 1$, we have the following guarantee*

$$\sum_{t=1}^{T} \mathbf{1}\{y_t \tilde{y}_t \leq 0\} - \sum_{t=1}^{T} \ell^q(\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle, y_t) \leq \frac{q^2 R^2 \|\boldsymbol{u}\|_2^2}{2} + qR\|\boldsymbol{u}\|_2 \sqrt{\frac{q^2 R^2 \|\boldsymbol{u}\|_2^2}{4} + \sum_{t=1}^{T} \ell^q(\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle, y_t)} \, .$$

Before proving the theorem, let's take a look at its meaning. If there exists a $\boldsymbol{u} \in \mathbb{R}^d$ such that $\sum_{t=1}^T \ell^q(\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle, y_t) = 0$, then the Perceptron algorithm makes a *finite* number of mistakes upper bounded by $R^2 \|\boldsymbol{u}\|_2^2$, by choosing $q = 1$. If there are many vectors $\boldsymbol{u}$ that achieve $\sum_{t=1}^T \ell^q(\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle, y_t) = 0$, we have that the finite number of mistakes is bounded by the norm of the smallest $\boldsymbol{u}$ among them. What is the meaning of this quantity?

Remember that a hyperplane represented by its normal vector $\boldsymbol{u}$ divides the space into two half-spaces: one with the points $\boldsymbol{z}$ that give a positive value for the inner product $\langle \boldsymbol{z}, \boldsymbol{u} \rangle$ and the other one where the same inner product is negative. Now, we have that the distance of a sample $\boldsymbol{z}_t$ from the hyperplane whose normal is $\boldsymbol{u}$ is

$$\frac{|\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle|}{\|\boldsymbol{u}\|_2} \, .$$

Also, given that we are considering a $\boldsymbol{u}$ that gives cumulative hinge loss zero, we have that this quantity is at least $\frac{1}{\|\boldsymbol{u}\|_2}$. So, *the norm of the minimal $\boldsymbol{u}$ that has cumulative hinge loss equal to zero is inversely proportional to the minimum distance between the points and the separating hyperplane*. This distance is called the **margin** of the samples $(\boldsymbol{z}_t, y_t)_{t=1}^T$. So, if the margin is small, the Perceptron algorithm can make more mistakes than when the margin is large.

If the problem is not linearly separable, the Perceptron satisfies a mistake bound of $\mathscr{L}^{(q)} + \mathcal{O}(q\sqrt{\mathscr{L}^{(q)}})$ when $T \to \infty$, uniformly for all $q \geq 1$, where $\mathscr{L}^{(q)}$ is the cumulative loss of the competitor, $\sum_{t=1}^T \ell^q(\langle \boldsymbol{z}_t, \boldsymbol{u} \rangle, y_t)$. Moreover, we measure the competitor with a *family of loss functions* and compete with the best $\boldsymbol{u}$ measured with the best loss. This adaptivity is achieved through two basic ingredients:

- *The Perceptron is independent of scaling of the update by a hypothetical learning rate η*, in the sense that the mistakes it makes are independent of the scaling. That is, we could update with $\boldsymbol{x}_{t+1} = \boldsymbol{x}_t + \eta \mathbf{1}\{y_t \tilde{y}_t \leq 0\} y_t \boldsymbol{z}_t$ and have the same mistakes and update on the same samples because they only depend on the sign of $\tilde{y}_t$. Hence, we can think of it as always using the best possible learning rate η.
- The weakened definition of regret allows us to consider a family of loss functions, because *the Perceptron is not using any of them in the update.* In this sense, it is worth stressing

that the Perceptron algorithm is *not* simply Online Subgradient Descent (OSD) on some sequence of loss functions.

Let's now prove the regret guarantee.

Proof of Theorem 8.2 Denote by $\tau_t := \mathbf{1}\{y_t\tilde{y}_t \le 0\}$, so the total number of mistakes of the Perceptron algorithm is $M := \sum_{t=1}^T \tau_t$.

First, note that the Perceptron algorithm can be thought of as running OSD with a fixed stepsize $\eta > 0$ over the surrogate losses $\tilde{\ell}_t(\boldsymbol{x}) := -\langle \tau_t y_t \boldsymbol{z}_t, \boldsymbol{x}\rangle$ over $\mathcal{V} = \mathbb{R}^d$. Indeed, OSD over such losses would update

$$\boldsymbol{x}_{t+1} = \boldsymbol{x}_t + \eta\tau_t y_t \boldsymbol{z}_t \,. \tag{8.3}$$

Now, as said above, η does not affect in any way the sign of the predictions, hence the Perceptron algorithm could be run with (8.3) and its predictions would be exactly the same. Hence, for all $\eta > 0$, we have

$$\sum_{t=1}^T -\langle \tau_t y_t \boldsymbol{z}_t, \boldsymbol{x}_t\rangle + \sum_{t=1}^T \langle \tau_t y_t \boldsymbol{z}_t, \boldsymbol{u}\rangle \le \frac{\|\boldsymbol{u}\|_2^2}{2\eta} + \frac{\eta}{2}\sum_{t=1}^T \tau_t \|\boldsymbol{z}_t\|_2^2 \,.$$

Given that this inequality holds for any η, we can choose the one that minimizes the r.h.s., to have

$$\begin{aligned}\sum_{t=1}^T (-\langle \tau_t y_t \boldsymbol{z}_t, \boldsymbol{x}_t\rangle + \langle \tau_t y_t \boldsymbol{z}_t, \boldsymbol{u}\rangle) &\le \|\boldsymbol{u}\|_2 \sqrt{\sum_{t=1}^T \tau_t \|\boldsymbol{z}_t\|_2^2} \le \|\boldsymbol{u}\|_2 R \sqrt{\sum_{t=1}^T \tau_t} \\ &\le \|\boldsymbol{u}\|_2 R\sqrt{M} \,. \end{aligned} \tag{8.4}$$

Note that $-\tau_t y_t \langle \boldsymbol{z}_t, \boldsymbol{x}_t\rangle \ge 0$. Also, we have

$$\langle y_t \boldsymbol{z}_t, \boldsymbol{u}\rangle = 1 - (1 - \langle y_t \boldsymbol{z}_t, \boldsymbol{u}\rangle) \ge 1 - \max(1 - \langle y_t \boldsymbol{z}_t, \boldsymbol{u}\rangle, 0) = 1 - \ell^1(\langle \boldsymbol{z}_t, \boldsymbol{u}\rangle, y_t) \,.$$

Denote by $\mathscr{L}^{(q)} := \sum_{t=1}^T \ell^q(\langle \boldsymbol{z}_t, \boldsymbol{u}\rangle, y_t)$, where for shortness of notation we hide the dependency on $\boldsymbol{u}$. We now rewrite (8.4) as

$$\begin{aligned} M = \sum_{t=1}^T \tau_t &\le \|\boldsymbol{u}\|_2 R\sqrt{M} + \sum_{t=1}^T \tau_t \ell^1(\langle \boldsymbol{z}_t, \boldsymbol{u}\rangle, y_t) \\ &\le \|\boldsymbol{u}\|_2 R\sqrt{M} + \mathscr{L}^{(1)} \,. \end{aligned} \tag{8.5}$$

Using Lemma 4.29, we have the stated bound for $q = 1$.

For $q > 1$, using Holder's inequality and $\frac{1}{p} + \frac{1}{q} = 1$ in (8.5), we have

$$\begin{aligned} M &\le \|\boldsymbol{u}\|_2 R\sqrt{M} + \left(\sum_{t=1}^T \tau_t^p\right)^{1/p} \left(\sum_{t=1}^T \ell^q(\langle \boldsymbol{z}_t, \boldsymbol{u}\rangle, y_t)\right)^{1/q} \\ &= \|\boldsymbol{u}\|_2 R\sqrt{M} + M^{1/p}(\mathscr{L}^{(q)})^{1/q} \,. \end{aligned}$$

Using Young's inequality (Example 6.20), we have

$$M \le \|\boldsymbol{u}\|_2 R\sqrt{M} + M^{1/p}(\mathscr{L}^{(q)})^{1/q} \le \|\boldsymbol{u}\|_2 R\sqrt{M} + \frac{1}{p}M + \frac{1}{q}\mathscr{L}^{(q)},$$

that implies

$$M\left(1-\frac{1}{p}\right) \leq \|\boldsymbol{u}\|_2 R\sqrt{M} + \frac{1}{q}\mathscr{L}^{(q)} .$$

Using the fact that $1-\frac{1}{p}=\frac{1}{q}$, we have

$$M \leq q\|\boldsymbol{u}\|_2 R\sqrt{M} + \mathscr{L}^{(q)} .$$

Finally, using Lemma 4.29, we have the stated bound. □

Remark 8.3 *The Perceptron algorithm can also be seen as a way to solve the* feasibility problem*. In particular, consider the problem*

$$\begin{aligned} \textit{Find} \quad & \boldsymbol{x} \\ \textit{s.t.} \quad & \langle \boldsymbol{q}_i, \boldsymbol{x}\rangle > 0,\ i=1,\dots,N \end{aligned}$$

where $\boldsymbol{q}_i \in \mathbb{R}^d$. Essentially, we have a set $\mathcal{V}$ defined by the intersection of the open half-spaces $\langle \boldsymbol{q}_i, \boldsymbol{x}\rangle > 0$ and we want to find any point in it. This is useful, for example, when we have a constrained convex optimization problem with feasible set $\mathcal{V}$ and we need a way to initialize our optimization algorithm.

To use the Perceptron to solve this problem, in each round t, we can find one constraint $\boldsymbol{q}_i$ that is violated and update

$$\boldsymbol{x}_{t+1} = \boldsymbol{x}_t + \boldsymbol{q}_i .$$

The algorithm will find a point in $\mathcal{V}$ after $R^2\|\boldsymbol{u}\|_2^2$ iterations, where $\|\boldsymbol{q}_i\|_2 \leq R$ and $\boldsymbol{u}$ is given by the following optimization problem

$$\begin{aligned} \boldsymbol{u} = \operatorname*{argmin}_{\boldsymbol{x}\in\mathbb{R}^d} \quad & \|\boldsymbol{x}\|_2^2 \\ \textit{s.t.} \quad & \langle \boldsymbol{q}_i, \boldsymbol{x}\rangle \geq 1,\ i=1,\dots,N . \end{aligned}$$

Given that each iteration has computational complexity proportional to dN, the total computational complexity is $\mathcal{O}(dNR^2\|\boldsymbol{u}\|_2^2)$.

8.3 The Gaptron Algorithm

In this section, we introduce an algorithm that merges ideas from the randomized strategy and the Perceptron algorithm: the **Gaptron** algorithm.

The Perceptron analysis bounds the regret with respect to surrogate losses. However, this can be a loose upper bound on the actual number of mistakes. The Gaptron algorithm is designed to tighten this analysis by actively managing the *surrogate gap*, which is the difference between the true zero-one loss and the convex surrogate loss function used for optimization, leading to better mistake bounds. For pedagogical reasons, in the following, we will focus on the binary version of the Gaptron algorithm.

The algorithm maintains a weight vector $\boldsymbol{x}_t$ and, at each round, computes a measure of confidence using the features $\boldsymbol{z}_t$ as $m_t = \langle \boldsymbol{z}_t, \boldsymbol{x}_t\rangle$. Based on this confidence measure and the loss function used, it decides whether to follow the prediction of the current weight vector or to explore by choosing a label uniformly at random. This exploration is controlled

Algorithm 8.3 Gaptron for Binary Classification

Require: Initial point $\boldsymbol{x}_1$, learning rate $\eta > 0$, loss $\ell : \mathbb{R} \times \{-1, 1\} \to \mathbb{R}_{\geq 0}$, gap map $a : \mathbb{R} \times \{-1, 1\} \to [0, 1]$

for $t = 1$ **to** T **do**
  Receive feature vector $\boldsymbol{z}_t \in \mathbb{R}^d$
  Set $m_t = \langle \boldsymbol{z}_t, \boldsymbol{x}_t \rangle$
  Set $\hat{y}_t = \begin{cases} 1, & \text{if } m_t > 0 \\ \text{any label}, & \text{if } m_t = 0 \\ -1, & \text{if } m_t < 0 \end{cases}$
  Set $a_t = a(m_t, \hat{y}_t) \in [0, 1]$
  Predict $Y_t' = \hat{y}_t$ with probability $1 - a_t$, and a uniform random label from $\{-1, 1\}$ with probability a_t
  Receive true label $y_t \in \{-1, 1\}$ and suffer loss $\mathbf{1}\{Y_t' \neq y_t\}$
  Compute subgradient $\boldsymbol{g}_t \in \boldsymbol{z}_t \cdot \partial(\ell(\cdot, y_t))(m_t)$
  Update $\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \eta \boldsymbol{g}_t$
end for

by a **gap map**, a_t, which allows us to exploit the surrogate gap. Intuitively, the algorithm will randomize its prediction each time it is unsure, and in this way, the expected number of mistakes approaches the value of the surrogate loss. Finally, the weight vector is updated using OSD on the surrogate loss. Algorithm 8.3 summarizes the entire procedure.

We now present a theorem on the expected number of mistakes for the Gaptron algorithm when using self-bounded losses.

Theorem 8.4 *Let $\ell_t(\boldsymbol{x}) = \ell(\langle \boldsymbol{z}_t, \boldsymbol{x} \rangle, y_t)$ where $\ell : \mathbb{R} \times \{-1, 1\} \to \mathbb{R}_{\geq 0}$. Assume that*

- *For each $y \in \{-1, 1\}$, the one-dimensional function $x \mapsto \ell(x, y)$ is s-self-bounded;*
- *$\ell(x, 1) + \ell(x, -1) \geq 2$ for all $x \in \mathbb{R}$;*
- *$\ell(x, \operatorname{sign}(x)) \in [0, 1]$ for $x \neq 0$;*
- *$\ell(0, 1) \in [0, 1]$ and $\ell(0, -1) \in [0, 1]$.*

Let $\|\boldsymbol{z}_t\|_2 \leq R$ for all t. Set the learning rate $\eta \leq \frac{1}{2sR^2}$ and the gap map $a(m, \hat{y}) = \ell(m, \hat{y})$. Then, the expected number of mistakes of the Gaptron algorithm (Algorithm 8.3) is upper bounded as

$$\mathbb{E}\left[\sum_{t=1}^{T} \mathbf{1}\{Y_t' \neq y_t\}\right] \leq \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) + \frac{\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2}{2\eta}, \quad \forall \boldsymbol{u} \in \mathbb{R}^d \,.$$

Proof Using the fact that self-boundedness implies convexity, the analysis starts from the standard regret bound for OSD, which for any $\boldsymbol{u}$ gives

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \leq \frac{\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2}{2\eta} + \frac{\eta}{2} \sum_{t=1}^{T} \|\boldsymbol{g}_t\|_2^2 \,.$$

The expected number of mistakes in round t, conditioned on the history up to round $t - 1$,

is

$$\mathbb{E}_t[\mathbf{1}\{Y_t' \neq y_t\}] = (1 - a_t)\mathbf{1}\{\hat{y}_t \neq y_t\} + \frac{a_t}{2} .$$

Hence, we have

$$\begin{aligned}
&\sum_{t=1}^{T} \mathbb{E}\left[\mathbf{1}\{Y_t' \neq y_t\}\right] - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \\
&\quad = \sum_{t=1}^{T} \mathbb{E}\left[\mathbf{1}\{Y_t' \neq y_t\}\right] - \sum_{t=1}^{T} \mathbb{E}[\ell_t(\boldsymbol{x}_t)] + \sum_{t=1}^{T} \mathbb{E}[\ell_t(\boldsymbol{x}_t)] - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \\
&\quad \leq \frac{\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2}{2\eta} + \sum_{t=1}^{T} \mathbb{E}\left[\mathbf{1}\{Y_t' \neq y_t\} - \ell_t(\boldsymbol{x}_t) + \frac{\eta}{2}\|\boldsymbol{g}_t\|_2^2\right] \\
&\quad = \frac{\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2}{2\eta} + \sum_{t=1}^{T} \mathbb{E}\left[(1 - a_t)\mathbf{1}\{\hat{y}_t \neq y_t\} + \frac{a_t}{2} - \ell_t(\boldsymbol{x}_t) + \frac{\eta}{2}\|\boldsymbol{g}_t\|_2^2\right] .
\end{aligned}$$

Let's analyze the term we call the **surrogate gap**:

$$\gamma_t := (1 - a_t)\mathbf{1}\{\hat{y}_t \neq y_t\} + \frac{a_t}{2} - \ell_t(\boldsymbol{x}_t) + \frac{\eta}{2}\|\boldsymbol{g}_t\|_2^2 .$$

The theorem is proven if we can show that $\gamma_t \leq 0$ for our choice of a_t and η.

The bound on the norm of the subgradient can be calculated through the self-boundedness of ℓ in its first argument:

$$\|\boldsymbol{g}_t\|_2^2 \leq 2s\|\boldsymbol{z}_t\|_2^2 \ell(m_t, y_t) \leq 2sR^2\ell(m_t, y_t) .$$

Hence, we have two cases.

Case $y_t \neq \hat{y}_t$:

$$\begin{aligned}
\gamma_t &= (1 - a_t)\mathbf{1}\{\hat{y}_t \neq y_t\} + \frac{a_t}{2} - \ell_t(\boldsymbol{x}_t) + \frac{\eta}{2}\|\boldsymbol{g}_t\|_2^2 \\
&\leq 1 - \ell(m_t, \hat{y}_t) + \frac{1}{2}\ell(m_t, \hat{y}_t) - \ell(m_t, y_t) + \eta sR^2\ell(m_t, y_t) \\
&= 1 - \frac{1}{2}\ell(m_t, \hat{y}_t) - \frac{1}{2}\ell(m_t, y_t) - \frac{1}{2}\ell(m_t, y_t) + \eta sR^2\ell(m_t, y_t) \\
&\leq -\frac{1}{2}\ell(m_t, y_t) + \eta sR^2\ell(m_t, y_t),
\end{aligned}$$

where in the second inequality we used the assumption on the loss. Hence, we need $\eta \leq \frac{1}{2sR^2}$ to have $\gamma_t \leq 0$.

Case $y_t = \hat{y}_t$:

$$\begin{aligned}
\gamma_t &= (1 - a_t)\mathbf{1}\{\hat{y}_t \neq y_t\} + \frac{a_t}{2} - \ell_t(\boldsymbol{x}_t) + \frac{\eta}{2}\|\boldsymbol{g}_t\|_2^2 \\
&\leq \frac{1}{2}\ell(m_t, \hat{y}_t) - \ell(m_t, y_t) + \eta sR^2\ell(m_t, y_t) = -\frac{1}{2}\ell(m_t, y_t) + \eta sR^2\ell(m_t, y_t) .
\end{aligned}$$

Hence, again we need $\eta \leq \frac{1}{2sR^2}$ to have $\gamma_t \leq 0$. □

Surprisingly, the use of the randomization allows us to obtain a *constant* additive term over the comparator's cumulative surrogate loss.

Remark 8.5 *Observe that if $\ell(x, y) = f(xy)$ for some convex function $f : \mathbb{R} \to \mathbb{R}$, then $f(0) \geq 1$ implies the condition $\ell(x, 1) + \ell(x, -1) \geq 2$ for all $x \in \mathbb{R}$ by Jensen's inequality (Theorem 2.9).*

We now show some examples of binary classification losses that satisfy the assumptions of the theorem.

Example 8.6 *We will use the squared hinge loss, $\ell_2(\tilde{y}, y) = \max(1-y\tilde{y}, 0)^2$, that satisfies all the assumptions in Theorem 8.4 and it is 2-self-bounded in its first argument. Hence, we have that the Gaptron with the squared hinge loss and $\eta = \frac{1}{4R^2}$ satisfies*

$$\mathbb{E}\left[\sum_{t=1}^{T} \mathbf{1}\{Y'_t \neq y_t\}\right] \leq \sum_{t=1}^{T} \ell_2(\langle \boldsymbol{z}_t, \boldsymbol{u}\rangle, y_t) + 2\|\boldsymbol{u} - \boldsymbol{x}_1\|_2^2 R^2, \quad \forall \boldsymbol{u} \in \mathbb{R}^d \,.$$

So, if the problem is not linearly separable, we can greatly beat (in expectation) the bound on the number of mistakes we gave for the Perceptron in terms of the squared hinge loss.

An even better choice is the smoothed hinge loss*:*

$$\ell^{sh}(\tilde{y}, y) := \begin{cases} 1 - 2y\tilde{y}, & \text{if } y\tilde{y} < 0 \\ (1 - y\tilde{y})^2, & \text{if } 0 \leq y\tilde{y} \leq 1 \\ 0, & \text{if } y\tilde{y} > 1 \end{cases} \tag{8.6}$$

This loss is always less than or equal to the squared hinge loss; it still satisfies all the assumptions of Theorem 8.4, and it is 2-self-bounded in its first argument.

Example 8.7 *One can also use loss functions that give different weights to the errors of the two classes. For example, we can use the following function*

$$\ell(\tilde{y}, y) = \begin{cases} 1 - 2w_y y\tilde{y}, & \text{if } y\tilde{y} < 0 \\ (1 - y\tilde{y})^2, & \text{if } 0 \leq y\tilde{y} \leq 1 \\ 0, & \text{if } y\tilde{y} > 1 \end{cases}$$

where $w_1 = 2$ and $w_{-1} = 1$. In this case, the loss function is 8-self-bounded, and it still satisfies the assumptions of Theorem 8.4.

8.4 History Bits

The Perceptron was proposed by Rosenblatt (1958). To be more precise, he introduced a *family* of algorithms characterized by a certain architecture. Also, he considered what we call now supervised and unsupervised training procedures. The particular class of Perceptron that we use nowadays, and that I described, was called *α-system* (Block, 1962). I hypothesize that the fact that the α-system survived the test of time is exactly due to the simple convergence proof in Block (1962) and Novikoff (1963). Both proofs are non-stochastic. For the sake of proper credit assignment, it seems that the convergence proof of the Perceptron was proved by many others before Block and Novikoff (see references in Novikoff, 1963). However, the proof in Novikoff (1963) seems to be the cleanest one. Aizerman et al. (1964) (essentially) described for the first time the Kernel Perceptron and proved a finite mistake

bound for it. The proof of convergence in the non-separable case for $q = 1$ is by Gentile and Littlestone (1999); Gentile (2003) and for $q = 2$ is from Freund and Schapire (1998, 1999a). Theorem 8.2 is from Beygelzimer et al. (2017), but with a new proof.

Let us also briefly mention the general combinatorial quantity that characterizes online binary classification in the realizable case. The *Littlestone dimension* of a class of classifiers $\mathcal{H} \subseteq \{-1, 1\}^{\mathcal{X}}$ is the largest depth of a complete binary tree, whose internal nodes are labeled by points in $\mathcal{X}$, such that every path of labels in $\{-1, 1\}$ is realized by some classifier in $\mathcal{H}$. This quantity was introduced by Littlestone (1988) and plays, in online learning, a role analogous to the one played by the Vapnik–Chervonenkis (VC) dimension in statistical learning: in the realizable case, a class admits an online algorithm with a finite worst-case number of mistakes if and only if its Littlestone dimension is finite, and the optimal mistake bound is exactly its Littlestone dimension.

The Gaptron was introduced in van der Hoeven (2020), in turn based on some of the ideas in Neu and Zhivotovskiy (2020). van der Hoeven (2020) also described a multiclass variant for different surrogate losses, as well as a bandit variant. The proof here and the conditions in Theorem 8.4 were developed in collaboration with Dirk van der Hoeven, and he was so kind to allow me to reproduce them here. Recently, Sakaue et al. (2024) extended the Gaptron algorithm to the structured prediction case, while Sakaue et al. (2025) extended it to the dynamic setting.

8.5 Exercises

Problem 8.1 *In the Perceptron mistake upper bound we let the competitor have any norm and we measure the loss of the competitor with the hinge loss* $\ell^{\text{hinge}}(\hat{y}, y) = \max(1 - \hat{y}y, 0)$*, where* $\hat{y} \in \mathbb{R}$ *and* $y \in \{-1, 1\}$*. Instead, we can equivalently constrain the competitor to have norm equal to 1 and measure the loss with the* hinge loss at margin γ*:* $\ell^{\text{hinge},\gamma}(\hat{y}, y) = \frac{1}{\gamma}\max(\gamma - \hat{y}y, 0)$*, where* $\gamma > 0$*. Prove that* $\ell^{\text{hinge},\gamma}(\hat{y}, y)$ *is still an upper bound on the zero-one loss over the prediction given by* $\operatorname{sign}(\hat{y})$*.*

Problem 8.2 *Use FTRL with an increasing regularizer instead of OSD in the Gaptron to get rid of the need to know* R *to set the learning rate, while still allowing unbounded feasible sets. Note that we make use of the knowledge of* $\boldsymbol{z}_t$ *to set the regularizer at time* t*.*

9

Multi-Armed Bandit

The multi-armed bandit setting is similar to the Learning with Expert Advice (LEA) setting: in each round, we select one expert $A_t \in \{1, \dots, d\}$, where $d \geq 2$, but, differently from the full-information setting, we only observe the loss g_{t,A_t} of that expert. The aim is still to compete with the cumulative loss of the best expert in hindsight. The observed losses can be adversarial or stochastic, giving rise to adversarial and stochastic multi-armed bandits.

9.1 Adversarial Multi-Armed Bandit

Problems where we do not receive the full-information feedback, i.e., we do not observe the loss vector, are called **bandit problems**. The name comes from the problem of a gambler who plays a pool of slot machines, which are often called "one-armed bandits". On each round, the gambler places his bet on a slot machine, and his goal is to win almost as much money as if he had known in advance which slot machine would return the maximum total reward. In this problem, we clearly have an *exploration-exploitation trade-off*. In fact, on the one hand, we would like to play the slot machine, which, based on previous rounds, we believe will give us the biggest win. On the other hand, we have to explore the slot machines to find the best ones. On each round, we have to solve this trade-off.

Given that we do not observe the whole loss vector, we cannot directly use Online Mirror Descent (OMD) and Follow-the-Regularized-Leader (FTRL) to solve such a problem, because they both need the loss functions or at least lower bounds to them. Moreover, the adversarial multi-armed bandit problem is a harder problem than Learning with Expert Advice (LEA), due to the lack of full information. So, as in the LEA case (Section 6.8), randomization will be necessary to have a sublinear regret, but we will also need ways to solve the exploration-exploitation trade-off.

Before showing how to solve this problem, we will weaken the adversary a bit by assuming that it is **oblivious**. That is, with the knowledge of the online algorithm being used, before the game starts, the adversary decides the losses of all the rounds. This makes the losses deterministic quantities, and it avoids the inadequacy in our definition of regret when the adversary is adaptive to the past algorithm's plays (see Arora et al. (2012)).

One way to solve the lack of full-information feedback is to construct *stochastic estimates* of the unknown losses. This is a natural choice given that the prediction strategy has to be randomized. So, in each round t, we construct a probability distribution over the arms $\boldsymbol{x}_t$, and we sample one action A_t according to this probability distribution. Then, we only observe the coordinate A_t of the loss vector $\boldsymbol{g}_t \in \mathbb{R}^d$. One possibility to have a stochastic estimate of the losses is to use an *importance-weighted estimator*: construct the estimator $\tilde{\boldsymbol{g}}_t$

of the unknown vector $\boldsymbol{g}_t$ in the following way:

$$\tilde{g}_{t,i} = \begin{cases} \frac{g_{t,i}}{x_{t,i}}, & i = A_t \\ 0, & \text{otherwise} \end{cases} \quad \text{for } i = 1, \dots, d \,.$$

Note that this estimator has all the coordinates equal to 0, except for the coordinate corresponding to the arm that was pulled.

This estimator is *unbiased*, that is $\mathbb{E}[\tilde{\boldsymbol{g}}_t | A_1, \dots, A_{t-1}] = \boldsymbol{g}_t$. To see why, note that we can rewrite $\tilde{g}_{t,i}$ as $\mathbf{1}\{A_t = i\}\frac{g_{t,i}}{x_{t,i}}$ for $i = 1, \dots, d$ and $\mathbb{E}[\mathbf{1}\{A_t = i\}|A_1, \dots, A_{t-1}] = x_{t,i}$. Hence, we have

$$\begin{aligned} \mathbb{E}[\tilde{g}_{t,i}|A_1, \dots, A_{t-1}] &= \mathbb{E}\left[\mathbf{1}\{A_t = i\}\frac{g_{t,i}}{x_{t,i}}\middle| A_1, \dots, A_{t-1}\right] \\ &= \frac{g_{t,i}}{x_{t,i}}\mathbb{E}[\mathbf{1}\{A_t = i\}|A_1, \dots, A_{t-1}] = g_{t,i}, \quad \forall i = 1, \dots, d \,. \end{aligned}$$

Let's also calculate the raw second moment of the coordinates of this estimator. We have

$$\mathbb{E}[\tilde{g}_{t,i}^2|A_1, \dots, A_{t-1}] = \mathbb{E}\left[\mathbf{1}\{A_t = i\}\frac{g_{t,i}^2}{x_{t,i}^2}\middle| A_1, \dots, A_{t-1}\right] = \frac{g_{t,i}^2}{x_{t,i}} \,.$$

We can now think of using OMD with these estimated losses and an entropic regularizer. Of course, a similar reasoning can also be done with FTRL. Hence, assume $\|\boldsymbol{g}_t\|_\infty \le L_\infty$ and set $\psi : \mathbb{R}^d_{\ge 0} \to \mathbb{R}$ defined as $\psi(\boldsymbol{x}) = \sum_{i=1}^d x_i \ln x_i$, that is the unnormalized negative entropy. Also, set $\boldsymbol{x}_1 = [1/d, \dots, 1/d]$. Using the OMD analysis, we have

$$\sum_{t=1}^T \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle \le \frac{\ln d}{\eta} + \frac{\eta}{2}\sum_{t=1}^T \|\tilde{\boldsymbol{g}}_t\|_\infty^2 \,.$$

We can now take the expectation on both sides and use the fact that

$$\mathbb{E}[\|\tilde{\boldsymbol{g}}_t\|_\infty^2] = \mathbb{E}\left[\frac{g_{t,A_t}^2}{x_{t,A_t}^2}\right] = \mathbb{E}\left[\mathbb{E}\left[\frac{g_{t,A_t}^2}{x_{t,A_t}^2}\middle| A_1, \dots, A_{t-1}\right]\right] = \mathbb{E}\left[\sum_{i=1}^d \frac{g_{t,i}^2}{x_{t,i}}\right],$$

to get

$$\begin{aligned} \mathbb{E}\left[\sum_{t=1}^T g_{t,A_t}\right] - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u}\rangle &= \mathbb{E}\left[\sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t\rangle\right] - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u}\rangle = \mathbb{E}\left[\sum_{t=1}^T \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle\right] \\ &\le \frac{\ln d}{\eta} + \frac{\eta}{2}\sum_{t=1}^T \mathbb{E}[\|\tilde{\boldsymbol{g}}_t\|_\infty^2] \\ &\le \frac{\ln d}{\eta} + \frac{\eta}{2}\sum_{t=1}^T \sum_{i=1}^d \mathbb{E}\left[\frac{g_{t,i}^2}{x_{t,i}}\right], \quad \forall \boldsymbol{u} \in \Delta^{d-1}, \end{aligned} \tag{9.1}$$

where in the last inequality we upper bounded the max over the squared coordinates with their sum. We are now in trouble, because the terms in the sum scale as $\max_{i=1,\dots,d} \frac{1}{x_{t,i}}$. So, we need a way to control the smallest probability over the arms.

One way to do it is to take a convex combination of $\boldsymbol{x}_t$ and a uniform probability. That is, we can predict with $\tilde{\boldsymbol{x}}_t = (1-\alpha)\boldsymbol{x}_t + \alpha[1/d, \dots, 1/d]^\top$, where α will be chosen in

Algorithm 9.1 Exponential Weights with Explicit Exploration for Multi-Armed Bandit

Require: $\eta, \alpha > 0$
1: Set $\boldsymbol{x}_1 = [1/d, \dots, 1/d]$
2: **for** $t = 1$ **to** T **do**
3: Set $\tilde{\boldsymbol{x}}_t = (1-\alpha)\boldsymbol{x}_t + \alpha[1/d, \dots, 1/d]^\top$
4: Draw A_t according to $\mathbb{P}\{A_t = i\} = \tilde{x}_{t,i}$
5: Select arm A_t
6: Observe *only* the loss of the selected arm $g_{t,A_t} \in [-L_\infty, L_\infty]$ and pay it
7: Construct the estimate $\tilde{g}_{t,i} = \begin{cases} \frac{g_{t,i}}{\tilde{x}_{t,i}}, & i = A_t \\ 0, & \text{otherwise} \end{cases}$ for $i = 1, \dots, d$
8: $x_{t+1,i} \propto x_{t,i} \exp(-\eta \tilde{g}_{t,i}),\ i = 1, \dots, d$
9: **end for**

the following. So, α can be seen as the minimum amount of exploration we require from the algorithm. Its value will be chosen by the regret analysis to optimally trade off exploration vs exploitation. The resulting algorithm is in Algorithm 9.1.

The same probability distribution is used in the estimator:

$$\tilde{g}_{t,i} = \begin{cases} \frac{g_{t,i}}{\tilde{x}_{t,i}}, & i = A_t \\ 0, & \text{otherwise} \end{cases} \quad \text{for } i = 1, \dots, d\,. \tag{9.2}$$

So, we now have that $\frac{1}{\tilde{x}_{t,i}} \leq \frac{d}{\alpha}$. The variance term in the regret is now controllable:

$$\|\tilde{\boldsymbol{g}}_t\|_\infty^2 = \frac{g_{t,A_t}^2}{\tilde{x}_{t,A_t}^2} \leq \frac{d^2}{\alpha^2} g_{t,A_t}^2 \leq \frac{d^2}{\alpha^2} L_\infty^2\,.$$

However, we pay a price for the bias introduced:

$$\sum_{t=1}^T \langle \tilde{\boldsymbol{g}}_t, \tilde{\boldsymbol{x}}_t - \boldsymbol{u}\rangle = (1-\alpha)\sum_{t=1}^T \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle + \frac{\alpha}{d}\sum_{t=1}^T\sum_{i=1}^d \tilde{g}_{t,i} - \alpha\sum_{t=1}^T \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{u}\rangle, \quad \forall \boldsymbol{u} \in \Delta^{d-1}\,.$$

From $\mathbb{E}[\sum_{i=1}^d \tilde{g}_{t,i}] = \sum_{i=1}^d g_{t,i} \leq dL_\infty$ and $\mathbb{E}[-\langle \tilde{\boldsymbol{g}}_t, \boldsymbol{u}\rangle] = -\langle \boldsymbol{g}_t, \boldsymbol{u}\rangle \leq L_\infty$, we have

$$\mathbb{E}\left[\sum_{t=1}^T \langle \tilde{\boldsymbol{g}}_t, \tilde{\boldsymbol{x}}_t - \boldsymbol{u}\rangle\right] \leq (1-\alpha)\mathbb{E}\left[\sum_{t=1}^T \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle\right] + 2\alpha L_\infty T, \quad \forall \boldsymbol{u} \in \Delta^{d-1}\,.$$

Putting together the last inequality and (9.1), we have

$$\begin{aligned}\mathbb{E}\left[\sum_{t=1}^T g_{t,A_t}\right] - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u}\rangle &\leq \frac{(1-\alpha)\ln d}{\eta} + \frac{(1-\alpha)\eta d^2 L_\infty^2 T}{2\alpha^2} + 2\alpha L_\infty T \\ &\leq \frac{\ln d}{\eta} + \frac{\eta d^2 L_\infty^2 T}{2\alpha^2} + 2\alpha L_\infty T, \quad \forall \boldsymbol{u} \in \Delta^{d-1}\,.\end{aligned}$$

Setting $\alpha \propto \frac{\sqrt{d}\ln^{\frac{1}{4}} d}{T^{\frac{1}{4}}}$ and $\eta \propto \sqrt{\frac{\alpha^2 \ln d}{d^2 L_\infty^2 T}}$, we obtain a regret of $\mathcal{O}(L_\infty \sqrt{d} T^{3/4} \ln^{1/4} d)$ as $T \to \infty$.

We did obtain a sublinear regret, but it is way worse than the $\mathcal{O}(\sqrt{T \ln d})$ of the full-information case. However, while it is expected that the bandit case must be more difficult than the full information one, it turns out that this is not the optimal strategy.

9.1.1 Exponential-weight algorithm for Exploration and Exploitation: Exp3

Algorithm 9.2 Exponential-weight Algorithm for Exploration and Exploitation (Exp3)

Require: $\eta > 0$

$\boldsymbol{x}_1 = [1/d, \dots, 1/d]$

for $t = 1$ **to** T **do**

  Draw A_t according to $\mathbb{P}\{A_t = i\} = x_{t,i}$

  Select arm A_t

  Observe *only* the loss of the selected arm $g_{t,A_t} \geq 0$ and pay it

  Construct the estimate $\tilde{g}_{t,i} = \begin{cases} \frac{g_{t,i}}{x_{t,i}}, & i = A_t \\ 0, & \text{otherwise} \end{cases}$ for $i = 1, \dots, d$

  $x_{t+1,i} \propto x_{t,i} \exp(-\eta \tilde{g}_{t,i}),\ i = 1, \dots, d$

end for

It turns out that the algorithm above actually works, even without mixing the output of the algorithm with the uniform distribution! We were just too loose in our regret guarantee. So, we will analyze Algorithm 9.2, **Exponential-weight algorithm for Exploration and Exploitation (Exp3)**, that is nothing else than OMD with entropic regularizer and stochastic estimates of the losses. Note that now we will assume that $g_{t,i} \geq 0$ for all t and i.

This time we will use the local norm regret bound for Exponentiated Gradient (EG) that we proved in (6.12). Let's consider the OMD with the entropic regularizer, learning rate η, and set $\tilde{\boldsymbol{g}}_t$ equal to the stochastic estimate of $\boldsymbol{g}_t$, as in Algorithm 9.2. Applying (6.12) and taking the expectation, for all $\boldsymbol{u} \in \Delta^{d-1}$, we have

$$\mathbb{E}\left[\sum_{t=1}^{T} g_{t,A_t}\right] - \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle = \mathbb{E}\left[\sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle\right] \leq \frac{\ln d}{\eta} + \frac{\eta}{2} \mathbb{E}\left[\sum_{t=1}^{T} \sum_{i=1}^{d} x_{t,i} \tilde{g}_{t,i}^2\right] .$$

Now, focusing on the terms $\mathbb{E}[x_{t,i} \tilde{g}_{t,i}^2]$, we have

$$\mathbb{E}\left[\sum_{i=1}^{d} x_{t,i} \tilde{g}_{t,i}^2\right] = \mathbb{E}\left[\mathbb{E}\left[\sum_{i=1}^{d} x_{t,i} \tilde{g}_{t,i}^2 \middle| A_1, \dots, A_{t-1}\right]\right] = \mathbb{E}\left[\sum_{i=1}^{d} g_{t,i}^2\right] \leq d L_\infty^2. \quad (9.3)$$

Finally, we can choose $\eta = \sqrt{\frac{2 \ln d}{d L_\infty^2 T}}$ to minimize the regret. Putting all together, we have the following theorem.

Theorem 9.1 *Assume $0 \leq g_{t,i} \leq L_\infty$ for $t = 1, \dots, T$ and $i = 1, \dots, d$. Then, Algorithm 9.2 with $\eta = \sqrt{\frac{2 \ln d}{d L_\infty^2 T}}$ guarantees*

$$\mathbb{E}\left[\sum_{t=1}^{T} g_{t,A_t}\right] - \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle \leq L_\infty \sqrt{2 d T \ln d}, \quad \forall \boldsymbol{u} \in \Delta^{d-1} .$$

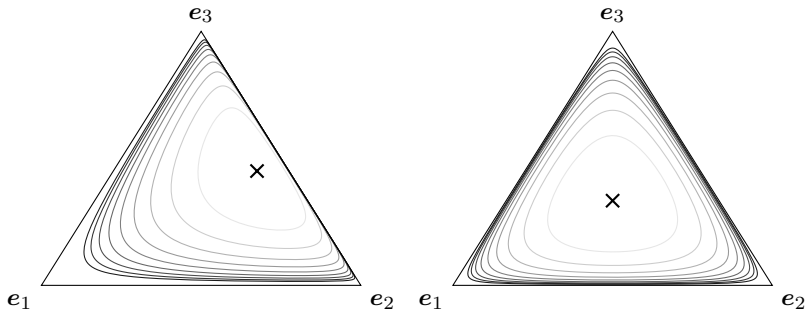


Figure 9.1 Contour plots of the KL divergence in 3-dimensions when $\boldsymbol{x}_t = [1/3, 1/3, 1/3]$ (left) and when $\boldsymbol{x}_t = [0.1, 0.45, 0.45]$ (right).

So, with a tighter analysis, we showed that, even without an explicit exploration term, OMD with an entropic regularizer solves the multi-armed bandit problem paying only a factor $\sqrt{d}$ more than the full information case. The intuition of why this algorithm works is in Figure 9.1: coordinates with small probability have large curvature of the KL divergence, which translates into small inverse-Hessian weights in the local dual norm.

However, one can show that this is still not the optimal regret! In the next section, we will change the regularizer to remove the $\sqrt{\ln d}$ term in the regret.

9.1.2 Optimal Regret Using OMD with Tsallis Entropy

Algorithm 9.3 Tsallis-INF (OMD version)

Require: $\eta > 0, q \in (0, 1)$

$\boldsymbol{x}_1 = [1/d, \ldots, 1/d]$

for $t = 1$ **to** T **do**

  Draw A_t according to $\mathbb{P}\{A_t = i\} = x_{t,i}$

  Select arm A_t

  Observe *only* the loss of the selected arm $g_{t,A_t} \geq 0$ and pay it

  Construct the estimate $\tilde{g}_{t,i} = \begin{cases} \frac{g_{t,i}}{x_{t,i}}, & i = A_t \\ 0, & \text{otherwise} \end{cases}$ for $i = 1, \ldots, d$

  $\boldsymbol{x}_{t+1} = \operatorname{argmin}_{\boldsymbol{x} \in \Delta^{d-1}} \ \eta \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x} \rangle - \frac{1}{1-q} \sum_{i=1}^{d} x_i^q + \frac{q}{1-q} \sum_{i=1}^{d} x_{t,i}^{q-1} x_i$

end for

In this section, we present the **Implicitly Normalized Forecaster (INF)**, which corresponds to OMD with Tsallis entropy for the multi-armed bandit.

Define $\psi_q : \mathbb{R}^d_{\geq 0} \to \mathbb{R}$ as $\psi_q(\boldsymbol{x}) = \frac{1}{1-q}\left(1 - \sum_{i=1}^{d} x_i^q\right)$. This is the negative **Tsallis entropy** of the vector $\boldsymbol{x}$. This is a strict generalization of the Shannon entropy, because when q goes to 1, $\psi_q(\boldsymbol{x})$ converges to the negative Shannon entropy for any $\boldsymbol{x} \in \Delta^{d-1}$. Note that $\operatorname{argmin}_{\boldsymbol{x} \in \Delta^{d-1}} \ \psi_q(\boldsymbol{x}) = [\frac{1}{d}, \ldots, \frac{1}{d}]$ and $\min_{\boldsymbol{x} \in \Delta^{d-1}} \psi_q(\boldsymbol{x}) = \frac{1-d^{1-q}}{1-q}$.

We will instantiate OMD with this regularizer for the multi-armed problem, obtaining the

Tsallis-INF algorithm in Algorithm 9.3. We will not use any interpretation of this regularizer from the information theory point of view. As we will see in the following, the only reason to choose it is its Hessian. In fact, the Hessian of this regularizer is still diagonal, and it is equal to $(\nabla^2 \psi_q(\boldsymbol{x}))_{ii} = \frac{q}{x_i^{2-q}}$. Once again, condition (6.6) holds, so we can use again the second part of Lemma 6.32. So, for any $\boldsymbol{u} \in \Delta^{d-1}$, we obtain

$$\begin{aligned}\sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle &\le \frac{B_{\psi_q}(\boldsymbol{u}; \boldsymbol{x}_1)}{\eta} + \frac{\eta}{2} \sum_{t=1}^T \boldsymbol{g}_t^\top (\nabla^2 \psi_q(\boldsymbol{z}'_t))^{-1} \boldsymbol{g}_t \\ &= \frac{d^{1-q} - \sum_{i=1}^d u_i^q}{\eta(1-q)} + \frac{\eta}{2q} \sum_{t=1}^T \sum_{i=1}^d g_{t,i}^2 (z'_{t,i})^{2-q},\end{aligned}$$

where $\boldsymbol{z}'_t = \alpha_t \boldsymbol{x}_t + (1-\alpha_t)\tilde{\boldsymbol{x}}_{t+1}$, $\alpha_t \in [0,1]$, and $\tilde{\boldsymbol{x}}_{t+1} = \operatorname{argmin}_{\boldsymbol{x} \in \mathbb{R}^d_{\ge 0}} \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle + \frac{1}{\eta} B_{\psi_q}(\boldsymbol{x}; \boldsymbol{x}_t)$.

As we did for Exp3, now we need an upper bound to the $z'_{t,i}$. From the definition of $\tilde{\boldsymbol{x}}_{t+1}$ and ψ_q, we have

$$-\frac{q}{1-q} \tilde{x}_{t+1,i}^{q-1} = -\frac{q}{1-q} x_{t,i}^{q-1} - \eta g_{t,i},$$

that is $\tilde{x}_{t+1,i} = x_{t,i} \left(1 + \frac{1-q}{q} \eta g_{t,i} x_{t,i}^{1-q}\right)^{-\frac{1}{1-q}}$. Assuming $g_{t,i} \ge 0$, we have $\tilde{x}_{t+1,i} \le x_{t,i}$.

Hence, putting everything together, we have

$$\sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \le \frac{d^{1-q} - \sum_{i=1}^d u_i^q}{\eta(1-q)} + \frac{\eta}{2q} \sum_{t=1}^T \sum_{i=1}^d g_{t,i}^2 x_{t,i}^{2-q}, \quad \forall \boldsymbol{u} \in \Delta^{d-1}.$$

We can now specialize the above reasoning, considering $q = 1/2$ in the Tsallis entropy, to obtain the following theorem.

Theorem 9.2 *Assume $0 \le g_{t,i} \le L_\infty$. Set $q = 1/2$. Then, for any $\eta > 0$, Algorithm 9.3 satisfies*

$$\mathbb{E}\left[\sum_{t=1}^T g_{t,A_t}\right] - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle \le \frac{2\sqrt{d}}{\eta} + \eta \sqrt{d} L_\infty^2 T, \quad \forall \boldsymbol{u} \in \Delta^{d-1}.$$

Proof We only need to calculate the terms $\mathbb{E}[\sum_{i=1}^d \tilde{g}_{t,i}^2 x_{t,i}^{3/2}]$. Proceeding as in (9.3), we obtain

$$\begin{aligned}\mathbb{E}\left[\sum_{i=1}^d \tilde{g}_{t,i}^2 x_{t,i}^{3/2}\right] &= \mathbb{E}\left[\mathbb{E}\left[\sum_{i=1}^d x_{t,i}^{3/2} \tilde{g}_{t,i}^2 \middle| A_1, \dots, A_{t-1}\right]\right] = \mathbb{E}\left[\sum_{i=1}^d x_{t,i}^{3/2} \frac{g_{t,i}^2}{x_{t,i}^2} x_{t,i}\right] \\ &= \mathbb{E}\left[\sum_{i=1}^d g_{t,i}^2 \sqrt{x_{t,i}}\right] \le \mathbb{E}\left[\sqrt{\sum_{i=1}^d g_{t,i}^2} \sqrt{\sum_{i=1}^d g_{t,i}^2 x_{t,i}}\right] \le L_\infty^2 \sqrt{d}. \quad \square\end{aligned}$$

Choosing $\eta \propto \frac{1}{L_\infty \sqrt{T}}$, we finally obtain an expected regret of $\mathcal{O}(L_\infty \sqrt{dT})$ as $T \to \infty$, that can be proved to be the optimal one.

There is one last thing: how do we compute the predictions of this algorithm? In each step,

we have to solve a constrained optimization problem. So, we can write the corresponding Lagrangian:

$$L(\boldsymbol{x}, \beta) = \sum_{i=1}^{d} \left(\eta \tilde{g}_{t,i} + \frac{q}{1-q} x_{t,i}^{q-1} \right) x_i - \frac{1}{1-q} \sum_{i=1}^{d} x_i^q + \beta \left(\sum_{i=1}^{d} x_i - 1 \right) .$$

From the Karush–Kuhn–Tucker conditions, we have

$$x_{t+1,i} = \left[\frac{1-q}{q} \left(\beta + \frac{q}{1-q} x_{t,i}^{q-1} + \eta \tilde{g}_{t,i} \right) \right]^{\frac{1}{q-1}} ,$$

and we also know that $\sum_{i=1}^{d} x_{t+1,i} = 1$. So, we have a one-dimensional problem in β that must be solved in each round.

9.2 Stochastic Bandits

We will now consider the *stochastic bandit* setting. Here, each arm is associated with an unknown probability distribution. At each time step, the algorithm selects one arm A_t. Then, for each $i = 1, \ldots, d$, losses $g_{t,i}$ are drawn independently over time from that arm's fixed distribution, and the algorithm only receives the loss g_{t,A_t}. We focus on minimizing the **pseudo-regret**, that is, the regret with respect to the optimal action in expectation, rather than the optimal action on the sequence of realized losses:

$$\text{P-Regret}_T := \mathbb{E}\left[\sum_{t=1}^{T} g_{t,A_t} \right] - \min_{i=1,\ldots,d} \mathbb{E}\left[\sum_{t=1}^{T} g_{t,i} \right] = \mathbb{E}\left[\sum_{t=1}^{T} g_{t,A_t} \right] - \min_{i=1,\ldots,d} \mu_i T,$$

where we denoted by μ_i the expectation of the distribution associated with the arm i.

Remark 9.3 *The usual notation in the stochastic bandit literature is to consider rewards instead of losses. Instead, to keep our notation coherent with the Online Convex Optimization (OCO) literature, we will consider losses. The two things are completely equivalent up to a multiplication by* -1.

Before presenting our first algorithm for stochastic bandits, we will introduce some basic notions on concentration inequalities that will be useful in our definitions and proofs.

9.2.1 Concentration Inequalities Bits

Suppose that $X_1, X_2, \ldots, X_n$ is a sequence of independent and identically distributed random variables with mean $\mu = \mathbb{E}[X_1]$ and variance $\sigma^2 = \text{Var}[X_1]$. Having observed $X_1, X_2, \ldots, X_n$, we would like to estimate the common mean μ. The most natural estimator is the *empirical mean*:

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^{n} X_i .$$

Linearity of expectation shows that $\mathbb{E}[\hat{\mu}] = \mu$, which means that $\hat{\mu}$ is an *unbiased estimator* of μ. Yet, $\hat{\mu}$ is a random variable itself. So, can we quantify how far $\hat{\mu}$ will be from μ?

We could use Chebyshev's inequality to upper bound the probability that $\hat{\mu}$ is far from μ:

$$\mathbb{P}\{|\hat{\mu} - \mu| \geq \epsilon\} \leq \frac{\mathrm{Var}[\hat{\mu}]}{\epsilon^2} .$$

Using the fact that $\mathrm{Var}[\hat{\mu}] = \frac{\sigma^2}{n}$, we have that

$$\mathbb{P}\{|\hat{\mu} - \mu| \geq \epsilon\} \leq \frac{\sigma^2}{n\epsilon^2} .$$

So, we can expect the probability of having a "bad" estimate to go to zero as one over the number of samples in our empirical mean. Is this the best we can get? To understand what we can hope for, let's take a look at the central limit theorem.

We know that, defining $S_n = \sum_{t=1}^{n}(X_t - \mu)$, $\frac{S_n}{\sqrt{n\sigma^2}} \to N(0,1)$ in distribution, as n goes to infinity. This means that

$$\mathbb{P}\{\hat{\mu} - \mu \geq \epsilon\} = \mathbb{P}\{S_n \geq n\epsilon\} = \mathbb{P}\left\{\frac{S_n}{\sqrt{n\sigma^2}} \geq \sqrt{\frac{n}{\sigma^2}}\epsilon\right\} \approx \int_{\epsilon\sqrt{\frac{n}{\sigma^2}}}^{\infty} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) \mathrm{d}x,$$

where the approximation comes from the central limit theorem. The integral cannot be calculated with a closed form, but we can easily upper-bound it. Indeed, for $a > 0$, we have

$$\begin{aligned}\int_a^{\infty} \exp\left(-\frac{x^2}{2}\right) \mathrm{d}x &= \int_a^{\infty} \frac{x}{x} \exp\left(-\frac{x^2}{2}\right) \mathrm{d}x \leq \frac{1}{a}\int_a^{\infty} x \exp\left(-\frac{x^2}{2}\right) \mathrm{d}x \\ &= \frac{1}{a}\exp\left(-\frac{a^2}{2}\right) .\end{aligned}$$

Hence, we have

$$\mathbb{P}\{\hat{\mu} - \mu \geq \epsilon\} \lessapprox \sqrt{\frac{\sigma^2}{2\pi\epsilon^2 n}} \exp\left(-\frac{n\epsilon^2}{2\sigma^2}\right) . \tag{9.4}$$

This bound has a better dependency on n than what we got with Chebyshev's inequality, and we would like to obtain a finite-time bound with a similar asymptotic rate. To do that, we will focus our attention on *subgaussian* random variables.

Definition 9.4 *We say that a random variable is* σ***-subgaussian*** *if for all* $\lambda \in \mathbb{R}$ *we have that* $\mathbb{E}[\exp(\lambda X)] \leq \exp(\lambda^2\sigma^2/2)$.

Example 9.5 *The following random variables are subgaussian:*

- *If X is Gaussian with mean zero and variance σ^2, then X is σ-subgaussian.*
- *If X has mean zero and $X \in [a, b]$ almost surely, then X is $(b-a)/2$-subgaussian.*

We have the following properties for subgaussian random variables, left as exercises (Problem 9.2).

Lemma 9.6 *Assume that the random variables X_1 and X_2 are independent, and σ_1-subgaussian and σ_2-subgaussian, respectively. Then,*

(a) $\mathbb{E}[X_1] = 0$ *and* $\mathrm{Var}[X_1] \leq \sigma_1^2$.
(b) cX_1 *is* $|c|\sigma_1$*-subgaussian.*

(c) $X_1 + X_2$ *is* $\sqrt{\sigma_1^2 + \sigma_2^2}$*-subgaussian.*

Subgaussian random variables behave like zero-mean Gaussian random variables, in the sense that their tail probabilities are upper bounded by the ones of a Gaussian of variance σ^2. To prove it, let's first state Markov's inequality.

Theorem 9.7 (Markov's inequality) *For a non-negative random variable* X *and* $\epsilon > 0$*, we have that* $\mathbb{P}\{X \geq \epsilon\} \leq \frac{\mathbb{E}[X]}{\epsilon}$.

With Markov's inequality, we can now formalize the above statement on subgaussian random variables.

Theorem 9.8 *Let* $\epsilon > 0$*. If a random variable is* σ*-subgaussian, then* $\mathbb{P}\{X \geq \epsilon\} \leq \exp\left(-\frac{\epsilon^2}{2\sigma^2}\right)$.

Proof For any $\lambda > 0$, we have

$$\mathbb{P}\{X \geq \epsilon\} = \mathbb{P}\{\exp(\lambda X) \geq \exp(\lambda\epsilon)\} \leq \frac{\mathbb{E}[\exp(\lambda X)]}{\exp(\lambda\epsilon)} \leq \exp(\lambda^2\sigma^2/2 - \lambda\epsilon) \,.$$

Minimizing the r.h.s. of the inequality with respect to λ, we have the stated result. □

An easy consequence of this theorem is that the empirical average of subgaussian random variables concentrates around its expectation, *with the same asymptotic rate as in* (9.4).

Corollary 9.9 *Assume that* $X_i - \mu$ *are independent,* σ*-subgaussian random variables. Let* $\hat{\mu} := \frac{1}{n}\sum_{i=1}^n X_i$*. Then, for any* $\epsilon \geq 0$*, we have*

$$\mathbb{P}\left\{\hat{\mu} \geq \mu + \epsilon\right\} \leq \exp\left(-\frac{n\epsilon^2}{2\sigma^2}\right) \qquad \textit{and} \qquad \mathbb{P}\left\{\hat{\mu} \leq \mu - \epsilon\right\} \leq \exp\left(-\frac{n\epsilon^2}{2\sigma^2}\right) \,.$$

Equating the upper bounds on the r.h.s. of the inequalities in the Corollary to δ, we have the equivalent statement that, with probability at least $1 - 2\delta$, we have

$$\mu \in \left[\hat{\mu} - \sqrt{\frac{2\sigma^2 \ln\frac{1}{\delta}}{n}}, \hat{\mu} + \sqrt{\frac{2\sigma^2 \ln\frac{1}{\delta}}{n}}\right] \,.$$

9.2.2 Explore-Then-Commit Algorithm

We are now ready to present the most natural algorithm for the stochastic bandit setting, called **Explore-then-Commit (ETC)** algorithm. That is, we first identify the best arm over md exploration rounds, and then we commit to it. This algorithm is summarized in Algorithm 9.4.

In the following, we will denote by $S_{t,i} = \sum_{j=1}^t \mathbf{1}\{A_j = i\}$, that is the number of times that the arm i was pulled in the first t rounds.

Define by $\mu^\star$ the expected loss of the arm with the smallest expectation, that is

$$\mu^\star := \min_{i=1,\ldots,d} \mu_i \,.$$

Algorithm 9.4 Explore-Then-Commit Algorithm

Require: $T, m \in \mathbb{N}, 1 \leq m \leq \frac{T}{d}$
$S_{0,i} = 0, \hat{\mu}_{0,i} = 0,\ i = 1, \ldots, d$
for $t = 1$ **to** T **do**
  Choose $A_t = \begin{cases} ((t-1) \mod d) + 1, & t \leq dm \\ \operatorname{argmin}_i\ \hat{\mu}_{dm,i}, & t > dm \end{cases}$
  Observe g_{t,A_t} and pay it
  $S_{t,i} = S_{t-1,i} + \mathbf{1}\{A_t = i\}$
  $\hat{\mu}_{t,i} = \frac{1}{S_{t,i}} \sum_{j=1}^{t} g_{j,A_j} \mathbf{1}\{A_j = i\}$ for i such that $S_{t,i} > 0$, and $\hat{\mu}_{t,i} = 0$ otherwise
end for

Critical quantities in our analysis will be the *gaps*, $\Delta_i := \mu_i - \mu^\star$ for $i = 1, \ldots, d$, that measure the expected difference in losses between the arms and the optimal one. In particular, we can decompose the regret as a sum over the arms of the expected number of times we pull an arm multiplied by its gap.

Lemma 9.10 *For any online policy of selection of the arms, the pseudo-regret is equal to*

$$\text{P-Regret}_T = \sum_{i=1}^{d} \mathbb{E}[S_{T,i}]\Delta_i \ .$$

Proof Observe that

$$\sum_{t=1}^{T} g_{t,A_t} = \sum_{t=1}^{T} \sum_{i=1}^{d} g_{t,i} \mathbf{1}\{A_t = i\} \ .$$

Hence,

$$\begin{aligned} \text{P-Regret}_T &= \mathbb{E}\left[\sum_{t=1}^{T} g_{t,A_t}\right] - T\mu^\star = \mathbb{E}\left[\sum_{t=1}^{T} (g_{t,A_t} - \mu^\star)\right] \\ &= \sum_{i=1}^{d} \sum_{t=1}^{T} \mathbb{E}[\mathbf{1}\{A_t = i\}(g_{t,i} - \mu^\star)] = \sum_{i=1}^{d} \sum_{t=1}^{T} \mathbb{E}[\mathbb{E}[\mathbf{1}\{A_t = i\}(g_{t,i} - \mu^\star)|A_t]] \\ &= \sum_{i=1}^{d} \sum_{t=1}^{T} \mathbb{E}[\mathbf{1}\{A_t = i\}\mathbb{E}[g_{t,i} - \mu^\star|A_t]] = \sum_{i=1}^{d} \sum_{t=1}^{T} \mathbb{E}[\mathbf{1}\{A_t = i\}](\mu_i - \mu^\star) \ . \ \square \end{aligned}$$

The above lemma quantifies the intuition that in order to have a small regret, we have to select the suboptimal arms less often than the best one.

We are now ready to prove the regret guarantee of the ETC algorithm.

Theorem 9.11 *Assume that the losses of the arms minus their expectations are* 1*-subgaussian and* $1 \leq m \leq T/d$*. Then, ETC guarantees a pseudo-regret of*

$$\text{P-Regret}_T \leq m \sum_{i=1}^{d} \Delta_i + (T - md) \sum_{i=1}^{d} \Delta_i \exp\left(-\frac{m\Delta_i^2}{4}\right) \ .$$

Proof Without loss of generality, let's assume that the optimal arm is the first one.

So, for $i \neq 1$, we have

$$\begin{aligned}\mathbb{E}[S_{T,i}] &= \sum_{t=1}^{T} \mathbb{E}[\mathbf{1}\{A_t = i\}] = m + (T - md)\mathbb{P}\left\{\hat{\mu}_{md,i} \leq \min_{j \neq i} \hat{\mu}_{md,j}\right\} \\ &\leq m + (T - md)\mathbb{P}\left\{\hat{\mu}_{md,i} \leq \hat{\mu}_{md,1}\right\} \\ &= m + (T - md)\mathbb{P}\left\{\hat{\mu}_{md,1} - \mu_1 - (\hat{\mu}_{md,i} - \mu_i) \geq \Delta_i\right\} \ .\end{aligned}$$

From Lemma 9.6, we have that $\hat{\mu}_{md,i} - \mu_i - (\hat{\mu}_{md,1} - \mu_1)$ is $\sqrt{2/m}$-subgaussian. So, from Theorem 9.8, we have

$$\mathbb{P}\left\{\hat{\mu}_{md,1} - \mu_1 - (\hat{\mu}_{md,i} - \mu_i) \geq \Delta_i\right\} \leq \exp\left(-\frac{m\Delta_i^2}{4}\right) \ .$$

Using Lemma 9.10 finishes the proof. □

The bound shows the trade-off between exploration and exploitation: if m is too big, we pay too much during the exploration phase (first term in the bound). On the other hand, if m is small, the probability of selecting a suboptimal arm increases (second term in the bound). Knowing all the gaps Δ_i, it is possible to choose m that minimizes the bound.

For example, in the case that $d = 2$, the regret is upper bounded by

$$m\Delta + (T - 2m)\Delta \exp\left(-m\frac{\Delta^2}{4}\right) \leq m\Delta + T\Delta \exp\left(-m\frac{\Delta^2}{4}\right),$$

that is minimized by $m = \frac{4}{\Delta^2} \ln \frac{T\Delta^2}{4}$. Remembering that m must be a natural number, we can choose

$$m = \max\left(\left\lceil \frac{4}{\Delta^2} \ln \frac{T\Delta^2}{4} \right\rceil, 1\right) \ .$$

When $\frac{T\Delta^2}{4} \leq 1$, we select $m = 1$. So, we have $\Delta + (T - 2)\Delta \leq T\Delta \leq \frac{4}{\Delta}$. Hence, the regret is upper-bounded by

$$\min\left(\Delta T, \frac{4}{\Delta}\left(1 + \max\left(\ln \frac{T\Delta^2}{4}, 0\right)\right) + \Delta\right) = \mathcal{O}\left(\frac{\ln T}{\Delta}\right) \text{ as } T \text{ goes to infinity.}$$

The main drawback of this algorithm is that its optimal tuning depends on the gaps Δ_i. Assuming knowledge of the gaps essentially makes the stochastic bandit problem trivial. However, its tuned regret bound gives us a baseline with which to compare other bandit algorithms. In particular, in the next section, we will present an algorithm that achieves the same asymptotic regret without any knowledge of the gaps.

9.2.3 Upper Confidence Bound Algorithm

The ETC algorithm has the disadvantage of requiring the knowledge of the gaps to tune the length of the exploration phase. Moreover, it solves the exploration vs. exploitation trade-off in a clunky way. It would be better to have an algorithm that smoothly mixes exploration and exploitation *in a data-dependent way*. So, we now describe an almost optimal and adaptive strategy called **Upper Confidence Bound (UCB)** algorithm. It employs the principle of

Algorithm 9.5 Upper Confidence Bound Algorithm

Require: $\alpha > 2, T \in \mathbb{N}$

$S_{0,i} = 0, \hat{\mu}_{0,i} = 0,\ i = 1, \dots, d$

for $t = 1$ **to** T **do**

  Choose $A_t = \text{argmin}_{i=1,\dots,d} \begin{cases} \hat{\mu}_{t-1,i} - \sqrt{\frac{2\alpha \ln t}{S_{t-1,i}}}, & \text{if } S_{t-1,i} \neq 0 \\ -\infty, & \text{otherwise} \end{cases}$

  Observe g_{t,A_t} and pay it

  $S_{t,i} = S_{t-1,i} + \mathbf{1}\{A_t = i\}$

  $\hat{\mu}_{t,i} = \frac{1}{S_{t,i}} \sum_{j=1}^{t} g_{j,A_j} \mathbf{1}\{A_j = i\}$ for i such that $S_{t,i} > 0$, and $\hat{\mu}_{t,i} = 0$ otherwise

end for

optimism in the face of uncertainty, to select in each round the arm that has the *potential to be the best one*.

UCB works by keeping an estimate of the expected loss of each arm and also a confidence interval at a certain probability. Roughly speaking, we have that with probability at least $1-\delta$

$$\mu_i \in \left[\hat{\mu}_i - \sqrt{\frac{2 \ln \frac{1}{\delta}}{S_{t-1,i}}}, +\infty \right),$$

where the "roughly" comes from the fact that $S_{t-1,i}$ is a random variable itself. Then, UCB will query the arm with the smallest lower bound, that is, the one that could potentially have the smallest expected loss. The regret bound below shows that this strategy automatically balances exploration (given by the use of the optimistic estimates through the confidence intervals) and exploitation (when the confidence intervals are small enough so that we query the arm with the smallest empirical mean).

Remark 9.12 *The name "Upper Confidence Bound" comes from the fact that traditionally stochastic bandits are defined over rewards, rather than losses. So, in our case, we actually use the lower confidence bound in the algorithm. However, to avoid confusion with the literature, we still call it Upper Confidence Bound algorithm.*

The proof works by proving that, once we have queried a suboptimal arm enough times, we will pull it again only if its confidence interval or the one around the best arm does not contain the true means. In turn, the probability that the confidence intervals are wrong is small because we use concentration inequalities to construct them. Moreover, a delicate point is to deal with the fact that the number of times we pull an arm is a random variable, making the application of the concentration inequality not immediate. We get around this by considering the worst number of pulls and taking a union bound on it.

The algorithm is summarized in Algorithm 9.5, and we can prove the following regret bound.

Theorem 9.13 *Assume that the losses of the arms minus their expectations are* 1*-subgauss-*

ian and let $\alpha > 2$. Then, Algorithm 9.5 guarantees a pseudo-regret of

$$\text{P-Regret}_T \leq \frac{\alpha}{\alpha - 2} \sum_{i=1}^{d} \Delta_i + \sum_{i:\Delta_i > 0} \frac{8\alpha \ln T}{\Delta_i} .$$

Proof We analyze one arm at a time. Also, without loss of generality, assume that the optimal arm is the first one. For each arm $i > 1$, we want to prove that $\mathbb{E}[S_{T,i}] \leq \frac{8\alpha \ln T}{\Delta_i^2} + \frac{\alpha}{\alpha-2}$.

The proof is based on the fact that once we have sampled a suboptimal arm enough times, the probability of selecting it again becomes small.

Let $t^\star$ be the largest time index such that $S_{t^\star-1,i} \leq \frac{8\alpha \ln T}{\Delta_i^2}$. If $t^\star = T$, then the statement above is true. Hence, we can safely assume $t^\star < T$. Now, for $t > t^\star$, we have

$$S_{t-1,i} > \frac{8\alpha \ln T}{\Delta_i^2} . \tag{9.5}$$

Consider $t > t^\star$ and such that $A_t = i$, then we claim that at least one of the two following equations must be true:

$$\hat{\mu}_{t-1,1} - \sqrt{\frac{2\alpha \ln t}{S_{t-1,1}}} \geq \mu_1, \tag{9.6}$$

$$\hat{\mu}_{t-1,i} + \sqrt{\frac{2\alpha \ln t}{S_{t-1,i}}} < \mu_i . \tag{9.7}$$

If the first one is true, the confidence interval around our estimate of the expectation of the optimal arm does not contain μ_1. On the other hand, if the second one is true, the confidence interval around our estimate of the expectation μ_i does not contain μ_i. So, we claim that if $t > t^\star$ and we selected the suboptimal arm i, then at least one of these two bad events happened.

Let's prove the claim by contradiction: *if both the inequalities above are false*, $t > t^\star$, and $A_t = i$, we have

$$\begin{aligned}\hat{\mu}_{t-1,1} - \sqrt{\frac{2\alpha \ln t}{S_{t-1,1}}} &\overset{(9.6)\text{ false}}{<} \mu_1 = \mu_i - \Delta_i \overset{(9.5)}{<} \mu_i - 2\sqrt{\frac{2\alpha \ln T}{S_{t-1,i}}} \\ &\leq \mu_i - 2\sqrt{\frac{2\alpha \ln t}{S_{t-1,i}}} \overset{(9.7)\text{ false}}{\leq} \hat{\mu}_{t-1,i} - \sqrt{\frac{2\alpha \ln t}{S_{t-1,i}}},\end{aligned}$$

that, by the selection strategy of the algorithm, would imply $A_t \neq i$.

Note that $S_{t^\star,i} \leq \frac{8\alpha \ln T}{\Delta_i^2} + 1$. Hence, we have

$$\begin{aligned} S_{T,i} &= S_{t^\star,i} + \sum_{t=t^\star+1}^{T} \mathbf{1}\{A_t = i\} \leq \frac{8\alpha \ln T}{\Delta_i^2} + 1 + \sum_{t=t^\star+1}^{T} \mathbf{1}\{\text{(9.6) or (9.7) true}\} \\ &\leq \frac{8\alpha \ln T}{\Delta_i^2} + 1 + \sum_{t=1}^{\infty} \mathbf{1}\{\text{(9.6) or (9.7) true}\} \\ &\leq \frac{8\alpha \ln T}{\Delta_i^2} + 1 + \sum_{t=1}^{\infty} \left(\mathbf{1}\{\text{(9.6) true}\} + \mathbf{1}\{\text{(9.7) true}\}\right) . \end{aligned}$$

Taking expectations on both sides, we have

$$\mathbb{E}[S_{T,i}] \leq \frac{8\alpha \ln T}{\Delta_i^2} + 1 + \sum_{t=1}^{\infty} \left(\mathbb{P}\{\text{(9.6) true}\} + \mathbb{P}\{\text{(9.7) true}\}\right) .$$

Now, we upper bound the probabilities in the sum. For each arm i, let $Y_{i,s}$ denote the s-th observed loss from arm i. Given that the losses on the arms are i.i.d. and using the union bound, we have

$$\begin{aligned} \mathbb{P}\left\{\hat{\mu}_{t-1,1} - \sqrt{\frac{2\alpha \ln t}{S_{t-1,1}}} \geq \mu_1\right\} &\leq \mathbb{P}\left\{\max_{s=1,\ldots,t-1} \frac{1}{s}\sum_{r=1}^{s} Y_{1,r} - \sqrt{\frac{2\alpha \ln t}{s}} \geq \mu_1\right\} \\ &= \mathbb{P}\left\{\bigcup_{s=1}^{t-1}\left\{\frac{1}{s}\sum_{r=1}^{s} Y_{1,r} - \sqrt{\frac{2\alpha \ln t}{s}} \geq \mu_1\right\}\right\} . \end{aligned}$$

Hence, using Corollary 9.9 and the union bound, we have

$$\mathbb{P}\{\text{(9.6) true}\} \leq \sum_{s=1}^{t-1} \mathbb{P}\left\{\frac{1}{s}\sum_{r=1}^{s} Y_{1,r} - \sqrt{\frac{2\alpha \ln t}{s}} \geq \mu_1\right\} \leq \sum_{s=1}^{t-1} t^{-\alpha} = (t-1)t^{-\alpha} .$$

Given that the same bound holds for $\mathbb{P}\{\text{(9.7) true}\}$, we have

$$\begin{aligned} \mathbb{E}\left[S_{T,i}\right] &\leq \frac{8\alpha \ln T}{\Delta_i^2} + 1 + \sum_{t=1}^{\infty} 2(t-1)t^{-\alpha} \leq \frac{8\alpha \ln T}{\Delta_i^2} + 1 + \sum_{t=2}^{\infty} 2t^{1-\alpha} \\ &\leq \frac{8\alpha \ln T}{\Delta_i^2} + 1 + 2\int_1^{\infty} x^{1-\alpha}\,\mathrm{d}x = \frac{8\alpha \ln T}{\Delta_i^2} + \frac{\alpha}{\alpha - 2} . \end{aligned}$$

Using Lemma 9.10, we have the stated bound. □

The bound above can become meaningless if the gaps are too small. So, here we prove another bound that does not depend on the inverse of the gaps.

Theorem 9.14 *Assume that the losses of the arms minus their expectations are* 1*-subgaussian and let* $\alpha > 2$*. Then, Algorithm 9.5 guarantees a pseudo-regret of*

$$\text{P-Regret}_T \leq 4\sqrt{2\alpha d T \ln T} + \frac{\alpha}{\alpha - 2}\sum_{i=1}^{d} \Delta_i .$$

Proof Let $\beta > 0$ be some value to be tuned subsequently, and recall from the proof of Theorem 9.13 that for each suboptimal arm i, we can bound

$$\mathbb{E}[S_{T,i}] \leq \frac{\alpha}{\alpha - 2} + \frac{8\alpha \ln T}{\Delta_i^2} .$$

Hence, using the pseudo-regret decomposition in Lemma 9.10, we have

$$\begin{aligned}\text{P-Regret}_T &= \sum_{i:\Delta_i<\beta} \Delta_i \mathbb{E}[S_{T,i}] + \sum_{i:\Delta_i\geq\beta} \Delta_i \mathbb{E}[S_{T,i}] \leq T\beta + \sum_{i:\Delta_i\geq\beta} \Delta_i \mathbb{E}[S_{T,i}] \\ &\leq T\beta + \sum_{i:\Delta_i\geq\beta} \left(\frac{\Delta_i \alpha}{\alpha - 2} + \frac{8\alpha \ln T}{\Delta_i} \right) \leq T\beta + \frac{\alpha}{\alpha - 2} \sum_{i=1}^{d} \Delta_i + \frac{8\alpha d \ln T}{\beta} .\end{aligned}$$

Choosing $\beta = \sqrt{\frac{8\alpha d \ln T}{T}}$, we have the stated bound. □

Remark 9.15 *Note that in the UCB algorithm, we have to know the subgaussian parameter of the arms. While this can be easily upper-bounded for stochastic arms with bounded support, it is unclear how to do it without any prior knowledge of the distribution of the arms.*

It is possible to prove that the UCB algorithm is asymptotically optimal, in the sense of the following theorem.

Theorem 9.16 (Bubeck and Cesa-Bianchi, 2012, Theorem 2.2) *Consider a strategy that satisfies $\mathbb{E}[S_{T,i}] = o(T^a)$ as $T \to \infty$ for any set of Bernoulli loss distributions, any arm i with $\Delta_i > 0$ and any $a > 0$. Then, for any set of Bernoulli loss distributions, we have*

$$\liminf_{T\to+\infty} \frac{\text{P-Regret}_T}{\ln T} \geq \sum_{i:\Delta_i>0} \frac{1}{2\Delta_i} .$$

9.3 Best-of-Both Worlds Guarantees with Tsallis-INF

We considered the two different settings for bandits: adversarial and stochastic. The optimal algorithms for these two settings were different. However, it is natural to ask if it is possible to have a *single* algorithm that achieves the best guarantee in both settings. It turns out that the answer is positive with the Tsallis-INF, but this time we will need the FTRL version, see Algorithm 9.6.

Theorem 9.17 *Assume that $0 \leq g_{t,i} \leq L_\infty$, for all $t = 1, \ldots, T$, $i = 1, \ldots, d$, where $d \geq 1$. Then, Algorithm 9.6 satisfies*

$$\mathbb{E}[\text{Regret}_T] \leq 32 L_\infty \sqrt{(d-1)T} .$$

Moreover, if in addition the $g_{t,i}$ are i.i.d. from a distribution ρ_i with mean μ_i for $i = 1, \ldots, d$, and $\operatorname{argmin}_i \mu_i$ is unique, then we also have

$$\text{P-Regret}_T \leq 256 L_\infty^2 \sum_{i:\Delta_i\neq 0} \frac{1 + \ln T}{\Delta_i} .$$

Algorithm 9.6 Tsallis-INF (FTRL version)

Require: $L_\infty > 0$

1: **for** $t = 1$ **to** T **do**
2: $\boldsymbol{x}_t = \operatorname{argmin}_{\boldsymbol{x} \in \Delta^{d-1}} \langle \sum_{n=1}^{t-1} \tilde{\boldsymbol{g}}_n, \boldsymbol{x} \rangle - 4L_\infty \sqrt{t} \sum_{i=1}^{d} \sqrt{x_i}$
3: Draw A_t according to $\mathbb{P}\{A_t = i\} = x_{t,i}$
4: Select arm A_t
5: Observe *only* the loss of the selected arm g_{t,A_t} and pay it
6: Construct the estimate $\tilde{g}_{t,i} = \begin{cases} \frac{g_{t,i}}{x_{t,i}}, & i = A_t \\ 0, & \text{otherwise} \end{cases}$ for $i = 1, \ldots, d$
7: **end for**

Proof Algorithm 9.6 is an instantiation of FTRL with linear losses $\ell_t(\boldsymbol{x}) = \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x} \rangle$ and regularizer $\psi_t(\boldsymbol{x}) = \lambda_t \psi(\boldsymbol{x})$, where $\psi(\boldsymbol{x}) = -\sum_{i=1}^{d} \sqrt{x_i}$ and $\lambda_t = 4L_\infty \sqrt{t}$. Note that λ_{T+1} has no influence on the regret, so we can set it equal to λ_T *ex post facto*. Moreover, given that ψ has a partial derivative that goes $-\infty$ if a coordinate goes to 0, we know from the first-order optimality condition that $\boldsymbol{x}_t$ cannot have any coordinate equal to 0 for any t. Hence, from Lemma 7.15 we have

$$\sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \le \psi_T(\boldsymbol{u}) - \min_{\boldsymbol{x} \in \mathcal{V}} \psi_1(\boldsymbol{x}) + \frac{1}{2} \sum_{t=1}^{T} \|\tilde{\boldsymbol{g}}_t\|^2_{(\nabla^2 \psi_t(\boldsymbol{z}_t))^{-1}} + \sum_{t=1}^{T-1} (\psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1})),$$

where $\boldsymbol{z}_t$ is on the line segment between $\boldsymbol{x}_t$ and $\tilde{\boldsymbol{x}}_{t+1} := \operatorname{argmin}_{\boldsymbol{x} \in \mathbb{R}^d_{\ge 0}} B_{\psi_t}(\boldsymbol{x}; \boldsymbol{x}_t) + \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x} \rangle$. Note that $\boldsymbol{x}_1 = \operatorname{argmin}_{\boldsymbol{x} \in \Delta^{d-1}} \psi_1(\boldsymbol{x}) = [1/d, \ldots, 1/d]$ and $\psi(\boldsymbol{x}_1) = -\sqrt{d}$. Also, for $\boldsymbol{e}_j$ being the j-th canonical basis, we have $\psi(\boldsymbol{e}_j) = -1$ for any $j = 1, \ldots, d$.

We now use the fact that the update of the algorithm remains the same if b_t is subtracted from all coordinates of the estimated loss vector in each round t. Hence, we can state the bound for a choice of b_t that helps reduce the first sum on the r.h.s. of the above inequality. The Hessian of the ψ is the diagonal matrix with entries $(\nabla^2 \psi(\boldsymbol{x}))_{ii} = \frac{1}{4x_i^{1.5}}$. Hence, we have

$$\sum_{t=1}^{T} \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \le \psi_T(\boldsymbol{u}) - \psi_1(\boldsymbol{x}_1) + \sum_{t=1}^{T} \frac{2}{\lambda_t} \sum_{i=1}^{d} (\tilde{g}_{t,i} - b_t)^2 z_{t,i}^{1.5} + \sum_{t=1}^{T-1} (\psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1})), \quad \forall \boldsymbol{u} \in \Delta^{d-1} .$$

If we constrain $0 \le b_t \le L_\infty$ and $\frac{L_\infty}{\lambda_t} \le 1/4$, we have that $\tilde{x}_{t+1,i}$ satisfies

$$\tilde{x}_{t+1,i} = \frac{x_{t,i}}{(1 + 2\frac{1}{\lambda_t}(\tilde{g}_{t,i} - b_t) x_{t,i}^{1/2})^2} \le 4 x_{t,i} .$$

Then, we select $b_t = \mathbf{1}\{A_t = j\} g_{t,j}$, where j is an arbitrary arm, and we take the expecta-

tion to have

$$\begin{aligned}
&\mathbb{E}\left[\sum_{i=1}^{d}(\tilde{g}_{t,i}-\mathbf{1}\{A_t=j\}g_{t,j})^2 z_{t,i}^{1.5}\right]\\
&\le 8\mathbb{E}\left[\sum_{i=1}^{d}(\tilde{g}_{t,i}-\mathbf{1}\{A_t=j\}g_{t,j})^2 x_{t,i}^{1.5}\right]\\
&= 8\mathbb{E}\left[\sum_{i=1}^{d}\left(\frac{g_{t,i}}{x_{t,i}}\mathbf{1}\{A_t=i\}-\mathbf{1}\{A_t=j\}g_{t,j}\right)^2 x_{t,i}^{1.5}\right]\\
&= 8\mathbb{E}\left[g_{t,j}^2\left(\frac{1}{x_{t,j}}-1\right)^2 x_{t,j}^{1.5}\mathbf{1}\{A_t=j\}\right]\\
&\quad + 8\mathbb{E}\left[\sum_{i\neq j}\left(\frac{g_{t,i}^2}{x_{t,i}^2}\mathbf{1}\{A_t=i\}+\mathbf{1}\{A_t=j\}g_{t,j}^2\right)x_{t,i}^{1.5}\right]\\
&= 8\mathbb{E}\left[g_{t,j}^2\left(\frac{1}{x_{t,j}}-1\right)^2 x_{t,j}^{1.5}x_{t,j}+\sum_{i\neq j}g_{t,i}^2 x_{t,i}^{0.5}+x_{t,j}\sum_{i\neq j}g_{t,j}^2 x_{t,i}^{1.5}\right]\\
&\le 24L_\infty^2\mathbb{E}\left[\sum_{i\neq j}\sqrt{x_{t,i}}\right],
\end{aligned}$$

where in the inequality we used $\left(\frac{1}{x_{t,j}}-1\right)^2 x_{t,j}^{1.5}x_{t,j} = x_{t,j}^{0.5}(1-x_{t,j})^2 \le 1-x_{t,j} \le \sum_{i\neq j}\sqrt{x_{t,i}}$ and $x_{t,j}\sum_{i\neq j}g_{t,j}^2 x_{t,i}^{1.5} \le L_\infty^2\sum_{i\neq j}x_{t,i}^{0.5}$.

Putting everything together, setting $\boldsymbol{u}=\boldsymbol{e}_k$, for all $k=1,\dots,d$, we have

$$\begin{aligned}
&\mathbb{E}\left[\sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t-\boldsymbol{e}_k\rangle\right]\\
&\le -\lambda_T-\lambda_1\psi(\boldsymbol{x}_1)+\sum_{t=1}^{T}\frac{48L_\infty^2}{\lambda_t}\sum_{i\neq j}\mathbb{E}[\sqrt{x_{t,i}}]+\sum_{t=1}^{T-1}(\lambda_{t+1}-\lambda_t)\left(-\mathbb{E}[\psi(\boldsymbol{x}_{t+1})]\right)\\
&= \lambda_1(-\psi(\boldsymbol{x}_1)-1)+\sum_{t=1}^{T}\frac{48L_\infty^2}{\lambda_t}\sum_{i\neq j}\mathbb{E}[\sqrt{x_{t,i}}]+\sum_{t=1}^{T-1}(\lambda_{t+1}-\lambda_t)\left(-\mathbb{E}[\psi(\boldsymbol{x}_{t+1})]-1\right)\\
&\le \lambda_1\sum_{i\neq j}\sqrt{x_{1,i}}+\sum_{t=1}^{T}\frac{48L_\infty^2}{\lambda_t}\sum_{i\neq j}\mathbb{E}[\sqrt{x_{t,i}}]+\sum_{t=1}^{T-1}(\lambda_{t+1}-\lambda_t)\sum_{i\neq j}\mathbb{E}[\sqrt{x_{t+1,i}}]\\
&= \sum_{t=1}^{T}\frac{48L_\infty^2}{\lambda_t}\sum_{i\neq j}\mathbb{E}[\sqrt{x_{t,i}}]+\sum_{t=0}^{T-1}(\lambda_{t+1}-\lambda_t)\sum_{i\neq j}\mathbb{E}[\sqrt{x_{t+1,i}}],
\end{aligned}$$

where in the first equality we have used the fact that $\sum_{t=1}^{T-1}(\lambda_{t+1}-\lambda_t)=\lambda_T-\lambda_1$, in the second inequality we used that $-1+\sum_{i=1}^{d}\sqrt{x_{t+1,i}} \le \sum_{i\neq j}\sqrt{x_{t+1,i}}$ for any $j=1,\dots,d$, and in the last equality we set $\lambda_0=0$. Using the fact that $\sqrt{t}-\sqrt{t-1} \le \frac{1}{\sqrt{t}}$ for $t=$

$1, 2, \dots$, we have

$$\mathbb{E}\left[\sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{e}_k\rangle\right] \le L_\infty \sum_{t=1}^{T}\left(4\sqrt{t} - 4\sqrt{t-1} + \frac{12}{\sqrt{t}}\right)\sum_{i\neq j}\mathbb{E}[\sqrt{x_{t,i}}]$$
$$\le L_\infty \sum_{t=1}^{T}\frac{16}{\sqrt{t}}\sum_{i\neq j}\mathbb{E}[\sqrt{x_{t,i}}]\,. \tag{9.8}$$

Using the fact that $\sum_{i\neq j}\sqrt{x_{t,i}} \le \sqrt{d-1}$ by Cauchy–Schwarz inequality and $\sum_{t=1}^{T}\frac{1}{\sqrt{t}} \le 2\sqrt{T}$, we get the first stated bound.

We now move to bound the pseudo-regret. Let j be the index of the best arm. From Lemma 9.10 and the fact that the expected time we pull arm i is $\sum_{t=1}^{T}\mathbb{E}[x_{t,i}]$, we have that $\mathbb{E}[\mathrm{Regret}_T(\boldsymbol{e}_j)] = \text{P-Regret}_T = \sum_{t=1}^{T}\sum_{i\neq j}\mathbb{E}[x_{t,i}]\Delta_i$. So, setting $k = j$, from (9.8) we have

$$\text{P-Regret}_T = \mathbb{E}[\mathrm{Regret}_T(\boldsymbol{e}_j)] = 2\mathbb{E}[\mathrm{Regret}_T(\boldsymbol{e}_j)] - \sum_{t=1}^{T}\sum_{i\neq j}\mathbb{E}[x_{t,i}]\Delta_i$$
$$\le \mathbb{E}\left[\sum_{t=1}^{T}\sum_{i\neq j}\left(32L_\infty\frac{\sqrt{x_{t,i}}}{\sqrt{t}} - x_{t,i}\Delta_i\right)\right] \le L_\infty^2\sum_{t=1}^{T}\sum_{i\neq j}\frac{256}{t\Delta_i}$$
$$\le 256L_\infty^2\sum_{i\neq j}\frac{1+\ln T}{\Delta_i},$$

where in the first inequality we used $a\sqrt{x} - bx \le \frac{a^2}{4b}$ for any $a, b > 0$. □

9.4 History Bits

The nonstochastic bandit problem was first considered by Baños (1968) (see also Megiddo (1980)). The algorithm in Algorithm 9.1 is from Cesa-Bianchi and Lugosi (2006, Theorem 6.9). The Exp3 algorithm was proposed in Auer et al. (2002b) and it used a small exploration rate to achieve the same regret we proved. The observation that the exploration in Exp3 can be removed completely is from Stoltz (2005).

The INF algorithm was proposed by Audibert and Bubeck (2009) and recast as an OMD procedure in Audibert et al. (2011). The connection with the Tsallis entropy was done in Abernethy et al. (2015). The specific proof presented here is new, and it builds on the proof by Abernethy et al. (2015), based on FTRL (which they call Gradient-Based Prediction Algorithm).

Stochastic multi-armed bandits were first proposed by Robbins (1952) and Lai and Robbins (1985). The ETC algorithm goes back to Robbins (1952), although Robbins proposed what is now called epoch-greedy (Langford and Zhang, 2008). For more history on ETC, take a look at Chapter 6 in Lattimore and Szepesvári (2020). The proofs in Section 9.2.2 are from Lattimore and Szepesvári (2020) as well.

The use of confidence bounds and the idea of optimism first appeared in the work by Lai and Robbins (1985). The first version of UCB is by Lai (1987). The version of UCB I

presented is by Auer et al. (2002a) under the name UCB1. Note that, rather than considering 1-subgaussian environments, Auer et al. (2002a) considers bandits where the rewards are confined to the $[0, 1]$ interval. The proof of Theorem 9.13 is a minor variation of that of Theorem 2.1 in Bubeck and Cesa-Bianchi (2012), which also popularized the subgaussian setup. Theorem 9.14 is from Bubeck and Cesa-Bianchi (2012).

Theorem 9.17 is from Zimmert and Seldin (2019, 2021), but here we use the proof from Lee and Orabona (2025) that does not use Fenchel conjugates and sacrifices constants for simpler calculations.

9.5 Exercises

Problem 9.1 *Design and analyse an FTRL version of Exp3 with non-decreasing regularizers.*

Problem 9.2 *Prove Lemma 9.6.*

Problem 9.3 *Prove a similar pseudo-regret bound to the one in Theorem 9.14 for an optimally tuned Explore-Then-Commit algorithm.*

10

Universal Portfolio Algorithms

We now describe another online learning problem: sequential investment in a market with $d \geq 2$ stocks. As in the rest of the book, we will assume that the stock market is adversarial and we will design no-regret online investment strategies.

10.1 Setting of Portfolio Selection

In this setting, each day we allocate our wealth across the stocks, and at the end of the day we sell all our shares. The behavior of the market is specified by arbitrarily chosen non-negative **market vectors** $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_T$, each of them in $\mathbb{R}^d_{\geq 0} \setminus \{\mathbf{0}\}$. The coordinates of the market vectors represent the ratio between the closing and opening prices for the stocks. An investment strategy is specified by a vector $\boldsymbol{x}_t \in \Delta^{d-1}$, and its elements specify the fraction of the current wealth invested in each stock at time t. We will also assume that the wealth is infinitely divisible. Hence, assuming an initial wealth of \$1, our wealth at the end of round T is

$$\mathrm{Wealth}_T := \sum_{i=1}^{d} \mathrm{Wealth}_{T-1}\, w_{T,i} x_{T,i} = \mathrm{Wealth}_{T-1} \langle \boldsymbol{w}_T, \boldsymbol{x}_T \rangle = \prod_{t=1}^{T} \langle \boldsymbol{w}_t, \boldsymbol{x}_t \rangle \,.$$

To define a regret, we have to decide what the class of comparators is. Here, we compare with the **best constantly rebalanced portfolio**. A constantly rebalanced portfolio follows the same strategy we use, but its allocation among the stocks is the same at each time step. Denoting by $\boldsymbol{u}$ the vector in the simplex of the constantly rebalanced portfolio, and setting the initial wealth to \$1, its wealth is

$$\mathrm{Wealth}_T(\boldsymbol{u}) := \prod_{t=1}^{T} \langle \boldsymbol{w}_t, \boldsymbol{u} \rangle \,.$$

Given the multiplicative nature of this game, the absolute difference between wealth values does not make much sense. Hence, we consider the ratio of the wealth of the best constantly rebalanced portfolio and that of the algorithm, or equivalently, the difference of the logarithms:

$$\mathrm{Regret}_T(\boldsymbol{u}) = \ln \mathrm{Wealth}_T(\boldsymbol{u}) - \ln \mathrm{Wealth}_T = \sum_{t=1}^{T} \ln \langle \boldsymbol{w}_t, \boldsymbol{u} \rangle - \sum_{t=1}^{T} \ln \langle \boldsymbol{w}_t, \boldsymbol{x}_t \rangle \,. \quad (10.1)$$

Rewritten in this way, it should be clear that this game is just an online convex optimization

game where $\mathcal{V} = \Delta^{d-1}$ and the convex losses are $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ defined as $\ell_t(\boldsymbol{x}) = -\ln\langle \boldsymbol{w}_t, \boldsymbol{x}\rangle$ if $\langle \boldsymbol{w}_t, \boldsymbol{x}\rangle > 0$ and $\boldsymbol{x} \in \Delta^{d-1}$, and $\ell_t(\boldsymbol{x}) = +\infty$ otherwise.

We will say that a portfolio algorithm is **universal** if the regret against any constantly rebalanced portfolio and with any sequence of market vectors is sublinear in time, that is, if the algorithm is no-regret.

Why Constantly Rebalanced Portfolios? This class of strategies has advantages that are not immediately obvious. First of all, despite the word "constant" this strategy is a very active one and very different from the buy-and-hold one that consists of buying a set of stocks in the first round and selling them at the end of the game. Let's consider an example that clearly shows the advantage of the constantly rebalanced portfolio. Consider only 2 stocks with a sequence of market vectors equal to $(1, \frac{1}{2}), (1, 2), (1, \frac{1}{2}), (1, 2), \ldots$. After any even number of rounds, the value of both stocks returns to the starting price, so a buy-and-hold strategy yields zero growth. Instead, the best constantly rebalanced portfolio here is $\boldsymbol{u} = (\frac{1}{2}, \frac{1}{2})$ and it gives an exponentially increasing wealth of $(\frac{9}{8})^{t/2} \approx 1.06^t$ after t days. This means that we would more than double our wealth every 12 days!

Another motivation to compare with constantly rebalanced portfolios is that it can be shown that in the case where the market vectors are i.i.d. from some fixed (unknown) distribution, then the constantly rebalanced portfolio is asymptotically optimal.

10.2 Portfolio Selection with the Exponentiated Gradient Algorithm

Given that the problem in (10.1) is an online convex optimization game, we could think of using any of the algorithms we saw so far. In particular, given that the feasible set is the probability simplex and that the losses are convex, we can try to use Exponentiated Gradient (EG). However, this problem turns out to be particularly challenging. In fact, the loss functions are not Lipschitz, so their gradients are unbounded.

In fact, using the upper bound on the regret of EG, we have

$$\sum_{t=1}^{T} \ln\langle \boldsymbol{w}_t, \boldsymbol{u}\rangle - \sum_{t=1}^{T} \ln\langle \boldsymbol{w}_t, \boldsymbol{x}_t\rangle \leq \frac{\ln d}{\eta} + \frac{\eta}{2}\sum_{t=1}^{T} \|\boldsymbol{g}_t\|_\infty^2 = \frac{\ln d}{\eta} + \frac{\eta}{2}\sum_{t=1}^{T} \frac{\|\boldsymbol{w}_t\|_\infty^2}{(\langle \boldsymbol{x}_t, \boldsymbol{w}_t\rangle)^2},$$

where $\boldsymbol{g}_t = \nabla \ell_t(\boldsymbol{x}_t)$. Unfortunately, the terms in the sum on the r.h.s. of the inequality are potentially unbounded. One way to avoid this issue is to assume that the market gains are in the range $[c, C]$, where $0 < c \leq C < +\infty$. In this way, we have $\frac{\|\boldsymbol{w}_t\|_\infty^2}{(\langle \boldsymbol{x}_t, \boldsymbol{w}_t\rangle)^2} \leq \frac{C^2}{c^2}$. Setting the learning rate to $\eta = \frac{\sqrt{2\ln d}}{\frac{C}{c}\sqrt{T}}$, we obtain a regret upper bound of

$$\frac{C}{c}\sqrt{2T\ln d}\,. \tag{10.2}$$

While the average regret vanishes, we had to use the additional assumption on the market gains, so the resulting algorithm is not universal, according to our definition above. In the next section, we will see that we can remove the constraint on the market gains and also achieve a much better dependence in T.

10.3 Universal Portfolio Selection with F-Weighted Portfolio Algorithms

Algorithm 10.1 F-Weighted Portfolio Selection

Require: F a probability measure on Δ^{d-1}
1: $\mathrm{Wealth}_0 = 1$
2: **for** $t = 1$ **to** T **do**
3: Set $\boldsymbol{x}_t = \frac{\int_{\Delta^{d-1}} \boldsymbol{x}\, \mathrm{Wealth}_{t-1}(\boldsymbol{x})\, F(\mathrm{d}\boldsymbol{x})}{\int_{\Delta^{d-1}} \mathrm{Wealth}_{t-1}(\boldsymbol{x})\, F(\mathrm{d}\boldsymbol{x})}$
4: Receive $\boldsymbol{w}_t \in \mathbb{R}^d_{\geq 0} \setminus \{\mathbf{0}\}$
5: $\mathrm{Wealth}_t = \mathrm{Wealth}_{t-1}\langle \boldsymbol{w}_t, \boldsymbol{x}_t \rangle$
6: **end for**

To solve the problem of the unbounded gradients, we will consider a different strategy. In particular, we will use the **F-weighted portfolio algorithms**, that predict at each step with

$$\boldsymbol{x}_t = \frac{\int_{\Delta^{d-1}} \boldsymbol{x}\, \mathrm{Wealth}_{t-1}(\boldsymbol{x})\, F(\mathrm{d}\boldsymbol{x})}{\int_{\Delta^{d-1}} \mathrm{Wealth}_{t-1}(\boldsymbol{x})\, F(\mathrm{d}\boldsymbol{x})},$$

where F is a probability measure on Δ^{d-1}. To understand this formula, consider a generic constantly rebalanced portfolio allocation $\boldsymbol{x} \in \Delta^{d-1}$ whose wealth after $t-1$ rounds is $\mathrm{Wealth}_{t-1}(\boldsymbol{x})$. Now, consider the probability distribution over all constantly rebalanced portfolios obtained by weighting the prior measure F proportionally to $\mathrm{Wealth}_{t-1}(\boldsymbol{x})$. We have that $\boldsymbol{x}_t$ defined above is nothing else than the average portfolio according to this distribution. Hence, we give more importance in the average to successful portfolios that are also weighted highly by the distribution F. The resulting strategy is summarized in Algorithm 10.1.

The above strategy also gives us an identity for the wealth of the algorithm:

$$\mathrm{Wealth}_t = \int_{\Delta^{d-1}} \mathrm{Wealth}_t(\boldsymbol{x})\, F(\mathrm{d}\boldsymbol{x}),$$

that is, *the wealth of the F-weighted portfolio is the average wealth of constantly rebalanced portfolios according to the distribution F*. Let's prove this expression by induction. At $t = 0$ the claim is true, so let's assume it is true at $t-1$ and let's prove it at t. We have

$$\begin{aligned}
\mathrm{Wealth}_t &= \mathrm{Wealth}_{t-1}\langle \boldsymbol{w}_t, \boldsymbol{x}_t \rangle \\
&= \int_{\Delta^{d-1}} \mathrm{Wealth}_{t-1}(\boldsymbol{x})\, F(\mathrm{d}\boldsymbol{x}) \frac{\int_{\Delta^{d-1}} \langle \boldsymbol{w}_t, \boldsymbol{x} \rangle\, \mathrm{Wealth}_{t-1}(\boldsymbol{x})\, F(\mathrm{d}\boldsymbol{x})}{\int_{\Delta^{d-1}} \mathrm{Wealth}_{t-1}(\boldsymbol{x})\, F(\mathrm{d}\boldsymbol{x})} \\
&= \int_{\Delta^{d-1}} \mathrm{Wealth}_t(\boldsymbol{x})\, F(\mathrm{d}\boldsymbol{x})\,. \qquad (10.3)
\end{aligned}$$

We will now state regret upper bounds for the Universal Portfolio algorithm using two possible settings of F: the Dirichlet$(1/2, \ldots, 1/2)$ distribution, whose density is

$$f(\boldsymbol{x}) = \frac{\Gamma(d/2)}{\pi^{d/2}} \prod_{i=1}^{d} x_i^{-1/2}, \quad \boldsymbol{x} \in \Delta^{d-1},$$

(where Γ is the gamma function, see Appendix A.2.2), and the uniform distribution.

Theorem 10.1 *Consider the F-weighted portfolio strategy and an arbitrary sequence of market gains $\boldsymbol{w}_t \in \mathbb{R}^d_{\geq 0} \setminus \{\mathbf{0}\}$ for $t = 1, \ldots, T$.*

- *If F is equal to the Dirichlet$(1/2, \ldots, 1/2)$ distribution, then we have*

$$\begin{aligned}\mathrm{Regret}_T &:= \max_{\boldsymbol{u} \in \Delta^{d-1}} \ln \mathrm{Wealth}_T(\boldsymbol{u}) - \ln \mathrm{Wealth}_T \leq \ln \frac{\sqrt{\pi}\Gamma\left(T + \frac{d}{2}\right)}{\Gamma\left(\frac{d}{2}\right)\Gamma\left(T + \frac{1}{2}\right)} \\ &\leq \frac{d-1}{2} \ln\left(\frac{T}{(d-1)/2} + 1\right) + \frac{d}{2} - 1 + \frac{1}{2}\ln\frac{\pi}{2}\,.\end{aligned}$$

- *If F is equal to the uniform distribution, then we have*

$$\begin{aligned}\mathrm{Regret}_T &:= \max_{\boldsymbol{u} \in \Delta^{d-1}} \ln \mathrm{Wealth}_T(\boldsymbol{u}) - \ln \mathrm{Wealth}_T \leq \ln \binom{T+d-1}{d-1} \\ &\leq (d-1)\left(\ln\left(\frac{T}{d-1} + 1\right) + 1\right)\,.\end{aligned}$$

Observe that both prediction strategies ensure that we never produce a vector where the loss is infinite, and allow us to show a vanishing average regret. Moreover, it can be shown that this first regret upper bound is optimal up to constant additive terms. Also, the proof shows that regret upper bounds are essentially tight in the case that in each round the market gains are vectors whose coordinates are all zero except for one coordinate equal to 1.

To prove this theorem, we will need some tools from information theory.

10.3.1 Information Theory Bits

We will need some basic results from information theory and, in particular, upper bounds used in the *method of types*.

Definition 10.2 *Consider a sequence of T symbols $\boldsymbol{s} = (x_1, \ldots, x_T)$, where each $x_t \in \{1, \ldots, d\}$. The **type of the sequence** $\boldsymbol{s}$ is the vector $\boldsymbol{n}$ of the fractions of times each symbol appears, i.e., $\boldsymbol{n} = [N(1;\boldsymbol{s})/T, \ldots, N(d;\boldsymbol{s})/T]$, where $N(j;\boldsymbol{s}) := \sum_{t=1}^{T} \mathbf{1}\{x_t = j\}$.*

Lemma 10.3 *Let $T \geq 1$, $\boldsymbol{x} \in \mathbb{R}^d_{\geq 0}$ such that $\sum_{i=1}^d x_i = T$, and $\boldsymbol{u} \in \Delta^{d-1}$. Then, we have*

$$\prod_{i=1}^{d} u_i^{x_i} \leq \prod_{i=1}^{d} \left(\frac{x_i}{T}\right)^{x_i}\,,$$

where $0^0 := 1$.

Proof If there exists i such that $u_i^{x_i} = 0$, the inequality is verified. So, in the following we assume that $u_i^{x_i} \neq 0$ for $i = 1, \ldots, d$. So, if for a given i we have $u_i = 0$, we must have $x_i = 0$ as well. Hence, we have

$$\begin{aligned}\prod_{i=1}^{d} u_i^{x_i} &= \prod_{i:u_i \neq 0} u_i^{x_i} = \prod_{i:u_i \neq 0} \exp(x_i \ln u_i) = \prod_{i:u_i \neq 0} \exp\left[T\left(\frac{x_i}{T}\ln\frac{x_i}{T} + \frac{x_i}{T}\ln\frac{u_i T}{x_i}\right)\right] \\ &= \exp\left[T(-\mathrm{H}(\boldsymbol{x}/T) - \mathrm{KL}(\boldsymbol{x}/T;\boldsymbol{u}))\right] \leq \exp(-T\ \mathrm{H}(\boldsymbol{x}/T)) = \prod_{i=1}^{d}\left(\frac{x_i}{T}\right)^{x_i}\,,\end{aligned}$$

where $0 \ln 0 := 0$, $0 \ln(0/0) := 0$, $\mathrm{H}(\boldsymbol{x}) := -\sum_{i=1}^{d} x_i \ln x_i$ is the Shannon entropy, and $\mathrm{KL}(\boldsymbol{x}; \boldsymbol{y}) := \sum_{i=1}^{d} x_i \ln \frac{x_i}{y_i}$ is the Kullback–Leibler (KL) divergence, and the inequality holds because the KL divergence is non-negative. □

Theorem 10.4 *Consider an alphabet of d symbols and consider the set of all sequences of T symbols. Let the set of all possible types of these sequences be $\mathcal{Q} \subset \Delta^{d-1}$. Denote by $\mathcal{T}_T(\boldsymbol{n})$ the set of all the sequences of length T symbols with a type $\boldsymbol{n}$, that is $\mathcal{T}_T(\boldsymbol{n}) := \{\boldsymbol{x} \in \{1, \dots, d\}^T : \boldsymbol{x} \text{ has type } \boldsymbol{n}\}$. Then, for any $\boldsymbol{n} \in \mathcal{Q}$, we have $|\mathcal{T}_T(\boldsymbol{n})| \leq \prod_{i=1}^{d} n_i^{-n_i T}$, where $0^0 := 1$.*

Proof We prove the inequality with a probabilistic argument. Consider all the possible sequences $\boldsymbol{y}$ of length T generated i.i.d. from a distribution over d symbols equal to $\boldsymbol{n} = [n_1, \dots, n_d]^\top \in \Delta^{d-1}$. The sum of the probabilities of all these sequences is 1. Among all the sequences, there are the ones with type $\boldsymbol{n}$. Hence, for any $\boldsymbol{n} \in \mathcal{Q}$, we have

$$1 \geq \sum_{\boldsymbol{y}: \boldsymbol{y} \text{ has type } \boldsymbol{n}} \mathbb{P}\{\boldsymbol{y}\} = \sum_{\boldsymbol{y}: \boldsymbol{y} \text{ has type } \boldsymbol{n}} \prod_{i=1}^{d} n_i^{n_i T} = |\mathcal{T}_T(\boldsymbol{n})| \prod_{i=1}^{d} n_i^{n_i T},$$

where $0^0 := 1$, the first equality is due to the i.i.d. assumption and the second one holds because all the sequences of type $\boldsymbol{n}$ have the same probability. □

10.3.2 Proof of Theorem 10.1

First of all, we need a technical lemma.

Lemma 10.5 *Let $a_1, \dots, a_T \geq 0$ and $b_1, \dots, b_T \geq 0$. Then, we have*

$$\frac{\sum_{t=1}^{T} a_t}{\sum_{t=1}^{T} b_t} \leq \max_{t=1,\dots,T} \frac{a_t}{b_t},$$

where we define $a/0 := +\infty$ for any $a > 0$ and $0/0 := 0$.

Proof Let $j^\star = \operatorname{argmax}_{t=1,\dots,T} \frac{a_t}{b_t}$. If $a_{j^\star} = 0$, then the theorem is true because both sides of the inequality are 0. In the same way, if $b_{j^\star} = 0$, then the inequality is true because the r.h.s. is $+\infty$. So, we can consider $a_{j^\star} > 0$ and $b_{j^\star} > 0$. Then, we have

$$\frac{\sum_{t=1}^{T} a_t}{\sum_{t=1}^{T} b_t} = \frac{a_{j^\star} \left(1 + \sum_{t \neq j^\star} \frac{a_t}{a_{j^\star}}\right)}{b_{j^\star} \left(1 + \sum_{t \neq j^\star} \frac{b_t}{b_{j^\star}}\right)} \leq \frac{a_{j^\star}}{b_{j^\star}},$$

because $\frac{a_{j^\star}}{b_{j^\star}} \geq \frac{a_t}{b_t}$ for all t, which implies $\frac{a_t}{a_{j^\star}} \leq \frac{b_t}{b_{j^\star}}$. □

Then, the following lemma is the key one.

Lemma 10.6 *For any constantly rebalanced portfolio $\boldsymbol{u} \in \Delta^{d-1}$ and any sequence of market gains $\boldsymbol{w}_t \in \mathbb{R}^d_{\geq 0} \setminus \{\boldsymbol{0}\}$ for $t = 1, \dots, T$, an F-weighted portfolio guarantees*

$$\frac{\mathrm{Wealth}_T(\boldsymbol{u})}{\mathrm{Wealth}_T} \leq \max_{\boldsymbol{j} \in \{1,\dots,d\}^T} \frac{\prod_{t=1}^{T} u_{j_t}}{\int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t} F(\mathrm{d}\boldsymbol{x})}.$$

Proof Observe that

$$\mathrm{Wealth}_T(\boldsymbol{u}) = \prod_{t=1}^{T} \langle \boldsymbol{w}_t, \boldsymbol{u} \rangle = \prod_{t=1}^{T} \sum_{i=1}^{d} w_{t,i} u_i = \sum_{\boldsymbol{j} \in \{1,\dots,d\}^T} \prod_{t=1}^{T} u_{j_t} w_{t,j_t},$$

where in the last equality we have expressed the products as a sum over all the possible combinations of the terms in the sum. In the same way, using (10.3), we have

$$\begin{aligned}\mathrm{Wealth}_T &= \int_{\Delta^{d-1}} \mathrm{Wealth}_T(\boldsymbol{x})\, F(\mathrm{d}\boldsymbol{x}) = \int_{\Delta^{d-1}} \prod_{t=1}^{T} \langle \boldsymbol{w}_t, \boldsymbol{x} \rangle\, F(\mathrm{d}\boldsymbol{x}) \\ &= \sum_{\boldsymbol{j} \in \{1,\dots,d\}^T} \int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t} w_{t,j_t}\, F(\mathrm{d}\boldsymbol{x})\,.\end{aligned}$$

We now use Lemma 10.5, to obtain

$$\begin{aligned}\frac{\mathrm{Wealth}_T(\boldsymbol{u})}{\mathrm{Wealth}_T} &= \frac{\sum_{\boldsymbol{j} \in \{1,\dots,d\}^T} \prod_{t=1}^{T} u_{j_t} w_{t,j_t}}{\sum_{\boldsymbol{j} \in \{1,\dots,d\}^T} \int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t} w_{t,j_t}\, F(\mathrm{d}\boldsymbol{x})} \\ &= \frac{\sum_{\boldsymbol{j} \in \{1,\dots,d\}^T, \prod_{t=1}^{T} w_{t,j_t} > 0} \prod_{t=1}^{T} u_{j_t} w_{t,j_t}}{\sum_{\boldsymbol{j} \in \{1,\dots,d\}^T, \prod_{t=1}^{T} w_{t,j_t} > 0} \int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t} w_{t,j_t}\, F(\mathrm{d}\boldsymbol{x})} \\ &\le \max_{\boldsymbol{j} \in \{1,\dots,d\}^T, \prod_{t=1}^{T} w_{t,j_t} > 0} \frac{\prod_{t=1}^{T} u_{j_t} w_{t,j_t}}{\int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t} w_{t,j_t}\, F(\mathrm{d}\boldsymbol{x})} \\ &= \max_{\boldsymbol{j} \in \{1,\dots,d\}^T, \prod_{t=1}^{T} w_{t,j_t} > 0} \frac{\prod_{t=1}^{T} u_{j_t}}{\int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t}\, F(\mathrm{d}\boldsymbol{x})} \\ &\le \max_{\boldsymbol{j} \in \{1,\dots,d\}^T} \frac{\prod_{t=1}^{T} u_{j_t}}{\int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t}\, F(\mathrm{d}\boldsymbol{x})}\,.\end{aligned}$$

□

This Lemma has a very easy and powerful interpretation: for a given $\boldsymbol{j} \in \{1,\dots,d\}^T$, we have that $\prod_{t=1}^{T} u_{j_t}$ is nothing else than the wealth of the constantly rebalanced portfolio $\boldsymbol{u}$ when the market gains for $t = 1,\dots,T$ are

$$\boldsymbol{w}_t = [0,\dots,0,\underbrace{1}_{\text{position } j_t},0,\dots,0] \in \mathbb{R}^d\,.$$

Similarly, for the wealth of the algorithm. This means that the lemma shows that the worst case regret ratio is achieved for market gains where only one coordinate is 1 and all the others are zero, so-called *Kelly market vectors* or "horse race" vectors. So, this lemma allows us to simplify the analysis by completely removing the market gains from the upper bound. As such, the upper bound is worst-case with respect to the sequence of market gains. Yet, given that with our choices of F distributions the final regret is logarithmic, it also shows that the market gains can only have a limited influence on the regret of the algorithm.

We can now prove the regret guarantees for the universal portfolio algorithm with two different distributions F.

Proof of Theorem 10.1 Our starting point is Lemma 10.6. Using Lemma 10.3, for any $\boldsymbol{j} \in \{1, \ldots, d\}^T$ and $\boldsymbol{u} \in \Delta^{d-1}$, we have

$$\prod_{t=1}^{T} u_{j_t} = \prod_{i=1}^{d} u_i^{N(i;\boldsymbol{j})} \leq \prod_{i=1}^{d} \left(\frac{N(i;\boldsymbol{j})}{T} \right)^{N(i;\boldsymbol{j})},$$

where $N(i;\boldsymbol{j})$ is $\sum_{t=1}^{T} \mathbf{1}\{j_t = i\}$.

Now, let's first consider the Dirichlet distribution. In this case, it can be shown that $\int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t} F(\mathrm{d}\boldsymbol{x})$ has a closed-form expression (see Problem 10.2):

$$\int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t} F(\mathrm{d}\boldsymbol{x}) = \frac{\pi^{-d/2} \Gamma\left(\frac{d}{2}\right)}{\Gamma\left(T + \frac{d}{2}\right)} \prod_{i=1}^{d} \Gamma\left(N(i;\boldsymbol{j}) + \frac{1}{2} \right). \tag{10.4}$$

We now use Lemma A.13 to maximize over the scaled simplex, to have

$$\begin{aligned} \max_{\boldsymbol{j} \in \{1,\ldots,d\}^T} \frac{\prod_{t=1}^{T} u_{j_t}}{\int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t} F(\mathrm{d}\boldsymbol{x})} &\leq \max_{a_1,\ldots,a_d \geq 0 : \sum_{i=1}^{d} a_i = T} \frac{\pi^{d/2} \Gamma\left(T + \frac{d}{2}\right)}{\Gamma\left(\frac{d}{2}\right) T^T} \prod_{i=1}^{d} \frac{a_i^{a_i}}{\Gamma\left(a_i + \frac{1}{2}\right)} \\ &= \frac{\sqrt{\pi} \Gamma\left(T + \frac{d}{2}\right)}{\Gamma\left(\frac{d}{2}\right) \Gamma\left(T + \frac{1}{2}\right)}. \end{aligned}$$

The second inequality in the stated bound is due to Lemma A.12.

Analogously, when F is the uniform distribution, it can be shown (see Problem 10.3) that

$$\int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t} F(\mathrm{d}\boldsymbol{x}) = \frac{1}{\binom{T+d-1}{d-1} |\, \mathcal{T}_T((N(1;\boldsymbol{j})/T, \ldots, N(d;\boldsymbol{j})/T))|}, \tag{10.5}$$

where $|\, \mathcal{T}_T(a_1, \ldots, a_d)|$ is the number of sequences of type $(a_1, \ldots, a_d)$. From Theorem 10.4, we have that $|\, \mathcal{T}_T(N(1;\boldsymbol{j})/T, \ldots, N(d;\boldsymbol{j})/T)| \leq \prod_{i=1}^{d} \left(\frac{N(i;\boldsymbol{j})}{T} \right)^{-N(i;\boldsymbol{j})}$. Hence, when F is the uniform distribution, we have

$$\max_{\boldsymbol{j} \in \{1,\ldots,d\}^T} \frac{\prod_{t=1}^{T} u_{j_t}}{\int_{\Delta^{d-1}} \prod_{t=1}^{T} x_{j_t} F(\mathrm{d}\boldsymbol{x})} \leq \binom{T+d-1}{d-1}.$$

The second inequality in the stated bound is due to the elementary inequality $\binom{n}{k} \leq \left(\frac{en}{k}\right)^k$. □

10.4 Portfolio Selection with the Online Newton Step Algorithm

While the F-weighted portfolio algorithm can have optimal regret, its computational complexity is very high, both in T and d. In fact, we have to calculate an integral over the simplex in each step. While it is possible to have a closed-form update with complexity $\mathcal{O}(t)$ (Cover and Ordentlich, 1996), it is also natural to look for alternatives with a lower computational complexity.

One might be tempted to use any other Online Convex Optimization (OCO) algorithm to solve the portfolio selection problem. However, as we have seen in Section 10.2, the gradients in general can be unbounded. Hence, we again consider an easier setting: we assume

Algorithm 10.2 Online Newton Step for Portfolio Selection

Require: $\lambda > 0, C < \infty, c > 0$
Set $\beta = \frac{c}{4C}$
for $t = 1$ **to** T **do**
  Set $\boldsymbol{x}_t = \operatorname{argmin}_{\boldsymbol{x} \in \Delta^{d-1}} \sum_{i=1}^{t-1} \langle \boldsymbol{g}_i, \boldsymbol{x} \rangle + \frac{\lambda}{2} \|\boldsymbol{x}\|_2^2 + \frac{\beta}{2} \sum_{i=1}^{t-1} (\langle \boldsymbol{g}_i, \boldsymbol{x} - \boldsymbol{x}_i \rangle)^2$
  Receive $\boldsymbol{w}_t \in [c, C]^d$
  Pay the loss $\ell_t(\boldsymbol{x}_t)$, where $\ell_t(\boldsymbol{x}) = -\ln \langle \boldsymbol{w}_t, \boldsymbol{x} \rangle$
  Set $\boldsymbol{g}_t = -\frac{\boldsymbol{w}_t}{\langle \boldsymbol{w}_t, \boldsymbol{x}_t \rangle}$
end for

that all market gains are lower bounded by $c > 0$ and upper bounded by $C < \infty$. We stress that Theorem 10.1 does not require this assumption, but we use it to derive a simpler algorithm, still with a logarithmic regret.

Now, we might be tempted to think that the losses are strongly convex, because Theorem 10.1 shows a logarithmic regret. This is clearly false: the Hessian of each function has rank 1, and unfortunately, in the worst case, the sum of T losses can also have a singular Hessian. Even assuming the Hessian of the sum of the losses is not singular, we would need its minimal eigenvalue to grow over time. Again, this does not happen in a worst-case scenario. Hence, we need another way.

Observe that the losses in portfolio selection are 1-exp-concave. In fact, $\exp(-\ell_t(\boldsymbol{x})) = \langle \boldsymbol{w}_t, \boldsymbol{x} \rangle$ is concave. Hence, if the gradients and the domain are bounded, we can apply the Online Newton Step (ONS) algorithm. In particular, following Example 7.39, we need to find $\beta \leq \frac{1}{2}$ such that $|\beta \langle \nabla \ell_t(\boldsymbol{x}), \boldsymbol{x} - \boldsymbol{u} \rangle| \leq \frac{1}{2}$ for any $\boldsymbol{x}, \boldsymbol{u} \in \Delta^{d-1}$. For the gradient, we have $\nabla \ell_t(\boldsymbol{x}) = -\frac{\boldsymbol{w}_t}{\langle \boldsymbol{w}_t, \boldsymbol{x} \rangle}$, hence $\|\nabla \ell_t(\boldsymbol{x})\|_\infty \leq \frac{C}{c}$. Moreover, $\|\boldsymbol{x} - \boldsymbol{u}\|_1 \leq 2$ for any $\boldsymbol{x}, \boldsymbol{u} \in \Delta^{d-1}$. Hence, we can set $\beta = \frac{c}{4C}$. The resulting algorithm is in Algorithm 10.2.

As shown in Problem 7.12, the update can also be divided into two steps: first minimizing over the entire space and then a Bregman projection.

As we have seen in Section 7.10, the regret upper bound of the ONS is

$$\begin{aligned} \operatorname{Regret}_T(\boldsymbol{u}) &= \ln \operatorname{Wealth}_T(\boldsymbol{u}) - \ln \operatorname{Wealth}_T = \sum_{t=1}^T \ln \langle \boldsymbol{w}_t, \boldsymbol{u} \rangle - \sum_{t=1}^T \ln \langle \boldsymbol{w}_t, \boldsymbol{x}_t \rangle \\ &\leq \frac{\lambda}{2} \|\boldsymbol{u}\|_2^2 + \frac{d}{2\beta} \ln \left(1 + \frac{\beta T L^2}{d\lambda} \right), \quad \forall \boldsymbol{u} \in \Delta^{d-1}, \end{aligned}$$

where $\|\nabla \ell_t(\boldsymbol{x}_t)\|_2 \leq L$. In our setting, this becomes

$$\begin{aligned} \operatorname{Regret}_T(\boldsymbol{u}) &= \ln \operatorname{Wealth}_T(\boldsymbol{u}) - \ln \operatorname{Wealth}_T = \sum_{t=1}^T \ln \langle \boldsymbol{w}_t, \boldsymbol{u} \rangle - \sum_{t=1}^T \ln \langle \boldsymbol{w}_t, \boldsymbol{x}_t \rangle \\ &\leq \frac{\lambda}{2} + \frac{2dC}{c} \ln \left(1 + \frac{CT}{4c\lambda} \right), \quad \forall \boldsymbol{u} \in \Delta^{d-1}, \end{aligned}$$

where we used the fact that $\|\nabla \ell_t(\boldsymbol{x}_t)\|_2 \leq \frac{\sqrt{d}C}{c}$.

10.5 History Bits

The distributional approach to betting and gambling was pioneered by Kelly (1956). This approach assumes that the market gains are i.i.d. from a (known) distribution. The F-weighted portfolios were proposed by Cover (1991). Cover and Ordentlich (1996) proposed the first portfolio algorithm with a minimax regret, without assumptions on the market gains, improving over the result in Cover (1991). The proof presented here follows the one in Cover and Ordentlich (1996), while Lemma 10.3 and Theorem 10.4 are adapted from the proofs of Cover and Thomas (2006, Theorem 11.1.2 and Theorem 11.1.3), respectively. Vovk and Watkins (1998) proved that F-weighted portfolio algorithms are instantiations of the Aggregating Algorithm (Vovk, 1990), which we will see in the next chapter.

The use of EG for portfolio selection was proposed by Helmbold et al. (1998). Stochastic approximations to the F-weighted portfolio update were proposed by Blum and Kalai (1997, 1999); Kalai and Vempala (2002). Blum and Kalai (1997, 1999) also present a simpler proof for the uniform distribution case. The use of the ONS algorithm for portfolio selection was proposed by Hazan et al. (2006, 2007). There is also a line of research that aims at designing algorithms on the efficiency-regret Pareto frontier (see, e.g., Tsai et al., 2023, and references therein).

10.6 Exercises

Problem 10.1 *Assume that the market gains $w_{t,i}$ are bounded in $[c, C]$. Find a variant of EG that does not need to know c and C, and it achieves up to constants the same guarantee in* (10.2)*.*

Problem 10.2 *Prove* (10.4)*. Hint: use the multivariate beta integral.*

Problem 10.3 *Prove* (10.5)*. Hint: use the identity* $|\mathcal{T}_T((N(1;\boldsymbol{j})/T, \dots, N(d;\boldsymbol{j})/T))| = \frac{T!}{\prod_{i=1}^d N(i;\boldsymbol{j})!}$.

Problem 10.4 *Let F be the Dirichlet$(\alpha, \dots, \alpha)$ distribution with $\alpha > 0$. Assume that the market vectors are Kelly market vectors, that is, $\boldsymbol{w}_t = \boldsymbol{e}_{j_t}$ for some $j_t \in \{1, \dots, d\}$. Show that the F-weighted portfolio algorithm predicts*

$$x_{t,i} = \frac{N_{t-1}(i) + \alpha}{t - 1 + d\alpha}, \quad i = 1, \dots, d,$$

where $N_{t-1}(i) := \sum_{s=1}^{t-1} \mathbf{1}\{j_s = i\}$. Then, prove that its wealth after T rounds is

$$\mathrm{Wealth}_T = \frac{\Gamma(d\alpha)}{\Gamma(T + d\alpha)} \prod_{i=1}^d \frac{\Gamma(N_T(i) + \alpha)}{\Gamma(\alpha)} \,.$$

11

Weighted Average Algorithm and Aggregating Algorithm

In this chapter, we will introduce the Weighted Average Algorithm (WAA) and the Aggregating Algorithm (AA). We will view both of them as instantiations of the Follow-the-Regularized-Leader (FTRL) algorithm on the space of distributions.

11.1 Follow-the-Regularized-Leader with Distributions

Our analysis of the Weighted Average Algorithm (WAA) and Aggregating Algorithm (AA) will be based on a generalization of the Follow-the-Regularized-Leader (FTRL) algorithm with the entropic regularizer, to work with distributions with infinite support, either countable or continuous.

Let $\mathcal{X} \subseteq \mathbb{R}^d$ be a measurable set, and let m be a reference measure on $\mathcal{X}$, used whenever we write densities with respect to a background measure. In the finite case, it is natural to take the counting measure $m(i) = 1$ for $i = 1, \dots, k$; in the continuous case, the natural choice of m is the Lebesgue measure; if the relevant set is not full-dimensional, one instead uses the Lebesgue measure on its affine hull.

As usual, we will denote by $\mathcal{V} \subseteq \mathcal{X}$ the feasible set of the problem. Also, we will make a subtle but important distinction between the support set, $\mathcal{X}$, and a smaller feasible set $\mathcal{V}$. This allows the flexibility to use priors defined on larger sets (like a Gaussian on $\mathbb{R}^d$), while still ensuring that the predictions $\boldsymbol{x}_t$ adhere to the problem's constraints $\boldsymbol{x}_t \in \mathcal{V}$. So, for a set $\mathcal{V}$, define $\Delta(\mathcal{V})$ **as the set of all probability distributions with support on** $\mathcal{V}$, and $\Delta_\mu(\mathcal{V})$ **as the set of all probability distributions supported on** $\mathcal{X}$ **whose expectation exists and is in** $\mathcal{V}$.

Throughout this chapter, losses $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ are measurable and may take the value $+\infty$ on $\mathcal{X}$. Allowing $+\infty$ lets us treat barriers or log-loss-type examples on closed domains without changing the feasible set.

For some set of distributions $\mathcal{P}$, in each round we output $P_t \in \mathcal{P}$, then we receive the loss functions $f_t(P) := \mathbb{E}_{\boldsymbol{x}\sim P}[\ell_t(\boldsymbol{x})]$. All expectations are understood in the extended-real sense, and the algorithms are assumed to select distributions for which the displayed quantities are well defined.

To simplify the notation, we introduce the **duality pairing**

$$\langle \ell, P \rangle := \int \ell(\boldsymbol{x}) P(\mathrm{d}\boldsymbol{x}) \, .$$

When the integrals are finite, the duality pairing is bilinear but not symmetric, so it should not be confused with the inner product. Yet, it reduces to the inner product in some cases,

for example, when $\mathcal{X}$ is a discrete set. With this notation, we can write $f_t(P) = \langle \ell_t, P \rangle$. We also have that the functional derivative of f_t is $\nabla_P f_t = \ell_t$.

The notation above illustrates the similarity between this formulation and the one we saw for FTRL. In particular, one would expect the same kind of results to hold here.

Let's use FTRL with regularizers $\psi_t : \mathcal{P} \to (-\infty, +\infty]$. Hence, at round t we produce a probability distribution P_t (assuming it exists) defined as

$$P_t := \operatorname*{argmin}_{P \in \mathcal{P}} \left(F_t(P) := \psi_t(P) + \left\langle \sum_{i=1}^{t-1} \ell_i, P \right\rangle \right) . \tag{11.1}$$

It is easy to see that the FTRL equality in Lemma 7.1 works even in this setting. Indeed, we have that

$$\left\langle \sum_{t=1}^{T} -\ell_t, Q \right\rangle = \psi_{T+1}(Q) - \min_{M \in \mathcal{P}} \psi_1(M) + \sum_{t=1}^{T} [F_t(P_t) - F_{t+1}(P_{t+1})] + F_{T+1}(P_{T+1}) - F_{T+1}(Q), \quad \forall Q \in \mathcal{P},$$

where we have canceled the algorithm's loss term on both sides. Define $\boldsymbol{x}_t := \mathbb{E}_{\boldsymbol{x} \sim P_t}[\boldsymbol{x}]$ and add $\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t)$ to both sides, to have

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \left\langle \sum_{t=1}^{T} \ell_t, Q \right\rangle = \psi_{T+1}(Q) - \min_{M \in \mathcal{P}} \psi_1(M) + F_{T+1}(P_{T+1}) - F_{T+1}(Q) + \sum_{t=1}^{T} [F_t(P_t) - F_{t+1}(P_{t+1}) + \ell_t(\boldsymbol{x}_t)], \quad \forall Q \in \mathcal{P}. \tag{11.2}$$

The above equality will serve as the starting point for analyzing WAA and AA.

11.2 The Weighted Average Algorithm

Algorithm 11.1 Weighted Average Algorithm (WAA)

Require: Non-empty closed set $\mathcal{V} \subseteq \mathcal{X} \subseteq \mathbb{R}^d$, $\lambda_1, \ldots, \lambda_T > 0$, $P_1 \in \Delta(\mathcal{X})$

1: **for** $t = 1$ **to** T **do**
2: Set $P_t \in \operatorname{argmin}_{P \in \Delta_\mu(\mathcal{V})} \lambda_t \operatorname{KL}(P; P_1) + \sum_{i=1}^{t-1} \mathbb{E}_{\boldsymbol{x} \sim P}[\ell_i(\boldsymbol{x})]$
3: Output $\boldsymbol{x}_t = \mathbb{E}_{\boldsymbol{x} \sim P_t}[\boldsymbol{x}]$
4: Receive $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ and pay $\ell_t(\boldsymbol{x}_t)$
5: **end for**

In this section, we introduce the **Weighted Average Algorithm**, stated in Algorithm 11.1. Essentially, WAA predicts with a weighted average of predictions—hence the name of the algorithm—where the weights are proportional to the negative exponential of the cumulative losses of each predictor. We can see WAA as a generalization of the Exponentiated Gradient (EG) algorithm with infinite distributions.

We will use FTRL over distributions from the previous section. Let P_1 be an arbitrary probability distribution on $\mathcal{X}$, and set $\psi_t(P) = \lambda_t \operatorname{KL}(P; P_1) = \lambda_t \mathbb{E}_{\boldsymbol{x} \sim P}[\ln(\frac{\mathrm{d}P}{\mathrm{d}P_1}(\boldsymbol{x}))]$,

where KL is the Kullback–Leibler (KL) divergence, with the convention $\mathrm{KL}(P;P_1) = +\infty$ if $P \not\ll P_1$. Also, we set $\mathcal{P} = \Delta_\mu(\mathcal{V})$. With this choice, the prediction rule in (11.1) becomes

$$\tilde{P}_t(\mathrm{d}\boldsymbol{x}) = \frac{\exp(-\frac{1}{\lambda_t}\sum_{i=1}^{t-1}\ell_i(\boldsymbol{x}))P_1(\mathrm{d}\boldsymbol{x})}{\int\exp(-\frac{1}{\lambda_t}\sum_{i=1}^{t-1}\ell_i(\boldsymbol{x}))P_1(\mathrm{d}\boldsymbol{x})}, \tag{11.3}$$

$$P_t = \operatorname*{argmin}_{P\in\Delta_\mu(\mathcal{V})} \mathrm{KL}(P;\tilde{P}_t)\,. \tag{11.4}$$

For this algorithm, we can show the following guarantee.

Lemma 11.1 *Set $\psi_t(P) = \lambda_t\,\mathrm{KL}(P;P_1)$, where $P_1 \in \Delta(\mathcal{X})$, $\mathrm{KL}(P;P_1) = +\infty$ if $P \not\ll P_1$, and $0 < \lambda_t \le \lambda_{t+1}$ for $t = 1,\dots,T-1$ and $\lambda_{T+1} = \lambda_T$. Define P_t as in* (11.1)*, where we set $\mathcal{P} = \Delta_\mu(\mathcal{V})$, and $\boldsymbol{x}_t := \mathbb{E}_{\boldsymbol{x}\sim P_t}[\boldsymbol{x}]$. Then, for all $Q \in \Delta_\mu(\mathcal{V})$, we have that*

$$\sum_{t=1}^T \ell_t(\boldsymbol{x}_t) \le \mathbb{E}_{\boldsymbol{x}\sim Q}\left[\sum_{t=1}^T \ell_t(\boldsymbol{x})\right] + \lambda_T\,\mathrm{KL}(Q;P_1) + \sum_{t=1}^T \lambda_t \ln \mathbb{E}_{\boldsymbol{x}\sim P_t}\left[e^{-\frac{\ell_t(\boldsymbol{x})-\ell_t(\boldsymbol{x}_t)}{\lambda_t}}\right]\,.$$

Before proving this lemma, observe that the following holds (proof left as an exercise):

$$\inf_{P\in\Delta(\mathcal{X})} \langle f,P\rangle + \lambda\,\mathrm{KL}(P;Q) = -\lambda \ln \mathbb{E}_{\boldsymbol{x}\sim Q}\left[e^{-\frac{1}{\lambda}f(\boldsymbol{x})}\right], \tag{11.5}$$

for every $\lambda > 0$, every $Q \in \Delta(\mathcal{X})$, and every measurable $f : \mathbb{R}^d \to (-\infty,+\infty]$ such that the expectation on the right-hand side is well defined, with the convention $\mathrm{KL}(P;Q) = +\infty$ if $P \not\ll Q$. This is called the Donsker–Varadhan variational formula.

Proof of Lemma 11.1 Given that $\psi_t(P) = \lambda_t\,\mathrm{KL}(P;P_1)$, one can verify that the Bregman divergence $B_{\psi_t}(P;Q)$ is equal to $\lambda_t\,\mathrm{KL}(P;Q)$.

Assuming $\frac{1}{\lambda_t}$ non-increasing, reasoning as in the proof of Lemma 7.15, we have

$$\begin{aligned} F_t(P_t) - F_{t+1}(P_{t+1}) &\le -\langle \ell_t, P_{t+1}\rangle - B_{F_t}(P_{t+1};P_t) = -\langle \ell_t, P_{t+1}\rangle - B_{\psi_t}(P_{t+1};P_t) \\ &\le \sup_{P\in\Delta(\mathcal{X})} -\langle \ell_t, P\rangle - B_{\psi_t}(P;P_t) = \lambda_t \ln \mathbb{E}_{\boldsymbol{x}\sim P_t}\left[e^{-\frac{\ell_t(\boldsymbol{x})}{\lambda_t}}\right], \end{aligned}$$

where in the first equality we used the fact that the terms with the losses are linear with respect to the distribution, and in the last equality we used (11.5). Observing that the term $F_{T+1}(P_{T+1}) - F_{T+1}(Q)$ in (11.2) is non-positive for all $Q \in \Delta_\mu(\mathcal{V})$, we obtain the stated bound. □

Remark 11.2 *If $\mathcal{A} \subseteq \mathcal{X}$ is measurable and $P_1(\mathcal{A}) = 0$, then $P_t(\mathcal{A}) = 0$ for all t. Hence, if $P_1 \in \Delta(\mathcal{V})$, then $P_t \in \Delta(\mathcal{V})$ for all t, and the constraint $P \in \Delta_\mu(\mathcal{V})$ is automatically satisfied.*

We now focus on the case that the WAA is used on exp-concave losses.

Theorem 11.3 *In Algorithm 11.1, assume that $\mathcal{X}$ is convex, and each $\ell_t : \mathbb{R}^d \to (-\infty,+\infty]$ is α-exp-concave on $\mathcal{X}$. Also, assume $\frac{1}{\alpha} \le \lambda_t \le \lambda_{t+1}$ for all $t = 1,\dots,T-1$, and*

$\lambda_{T+1} = \lambda_T$. *Then, we have that*

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) \le \mathbb{E}_{\boldsymbol{x}\sim Q}\left[\sum_{t=1}^{T} \ell_t(\boldsymbol{x})\right] + \lambda_T \operatorname{KL}(Q; P_1), \quad \forall Q \in \Delta_\mu(\mathcal{V}) .$$

Moreover, if in addition $P_1 \in \Delta(\mathcal{V})$ *and* $\mathcal{V}$ *is convex and bounded, then we also have*

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) \le -\lambda_T \ln \mathbb{E}_{\boldsymbol{x}\sim P_1}\left[e^{-\frac{1}{\lambda_T}\sum_{t=1}^{T}\ell_t(\boldsymbol{x})}\right],$$

where $e^{-\infty} := 0$.

Proof Given that ℓ_t is α-exp-concave, using Jensen's inequality (Theorem 2.9), we have

$$\lambda_t \ln \mathbb{E}_{\boldsymbol{x}\sim P_t}\left[e^{-\frac{\ell_t(\boldsymbol{x})}{\lambda_t}}\right] + \ell_t(\boldsymbol{x}_t) = \lambda_t \ln \mathbb{E}_{\boldsymbol{x}\sim P_t}\left[e^{-\frac{\ell_t(\boldsymbol{x})}{\lambda_t}}\right] + \ell_t\left(\mathbb{E}_{\boldsymbol{x}\sim P_t}[\boldsymbol{x}]\right) \le 0,$$

for all $\frac{1}{\lambda_t} \le \alpha$. Using this inequality in Lemma 11.1, we have the first result.

For the second bound, since $P_1 \in \Delta(\mathcal{V})$ and $\mathcal{V}$ is convex and bounded, every distribution Q with $\operatorname{KL}(Q; P_1) < \infty$ is supported on $\mathcal{V}$, and therefore belongs to $\Delta_\mu(\mathcal{V})$. Hence, by the first part of the theorem, for every such Q,

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) \le \mathbb{E}_{\boldsymbol{x}\sim Q}\left[\sum_{t=1}^{T} \ell_t(\boldsymbol{x})\right] + \lambda_T \operatorname{KL}(Q; P_1) .$$

Taking the infimum over all $Q \in \Delta(\mathcal{X})$, we obtain

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) \le \inf_{Q\in\Delta(\mathcal{X})}\left\{\mathbb{E}_{\boldsymbol{x}\sim Q}\left[\sum_{t=1}^{T} \ell_t(\boldsymbol{x})\right] + \lambda_T \operatorname{KL}(Q; P_1)\right\} .$$

Applying (11.5) with $f = \sum_{t=1}^{T} \ell_t$, $\lambda = \lambda_T$, and $Q = P_1$, gives us the stated bound. □

So, the WAA algorithm with λ_t constant over time has the surprising property of having a *constant* regret on exp-concave losses with respect to a stochastic competitor.

Unfortunately, the WAA update does not have a closed form in general. In fact, it requires the numerical evaluation of the expectation. However, if the distribution is supported on a finite discrete set, we can always calculate the prediction.

Example 11.4 *As a practical example, consider* k *experts* $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_k \in \mathbb{R}^d$ *and set* $\mathcal{X} = \mathcal{V} = \operatorname{conv}\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_k\}$. *Let the prior* P_1 *be supported on the experts, with* $P_1(\{\boldsymbol{u}_i\}) = P_{1,i} > 0$. *Then, WAA maintains weights* $P_{t,i}$ *on the experts and predicts* $\boldsymbol{x}_t = \sum_{i=1}^{k} P_{t,i}\boldsymbol{u}_i \in \mathcal{V}$, *and guarantees*

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) \le \min_{i=1,\ldots,k}\left(\sum_{t=1}^{T} \ell_t(\boldsymbol{u}_i) + \lambda_T \ln \frac{1}{P_{1,i}}\right) .$$

We have proved an upper bound on the regret of WAA that uses a randomized comparator, that is, $\mathbb{E}_{\boldsymbol{x}\sim Q}\left[\sum_{t=1}^{T} \ell_t(\boldsymbol{x})\right]$. However, sometimes one would like to prove an upper bound that depends on a deterministic one. Hence, now we show how to link the performance of the stochastic comparator to the performance of a deterministic one.

Theorem 11.5 *Assume that $\mathcal{V} \subseteq \mathbb{R}^d$ is a convex, closed, bounded set, full-dimensional in $\mathbb{R}^d$, and let P_1 be the uniform distribution on it. Assume that the losses ℓ_t are α-exp-concave on $\mathcal{V}$, and set $\frac{1}{\lambda_t} = \alpha$. Then, Algorithm 11.1 satisfies*

$$\sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \frac{d}{\alpha}\left[1 + \ln\left(\frac{T}{d}+1\right)\right], \quad \forall \boldsymbol{u} \in \mathcal{V}\,. \tag{11.6}$$

Proof Fix $\boldsymbol{u} \in \mathcal{V}$, and define $\mathcal{V}' = \{\boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{x} = \frac{T/d}{T/d+1}\boldsymbol{u} + \frac{1}{T/d+1}\boldsymbol{w}, \boldsymbol{w} \in \mathcal{V}\} \subseteq \mathcal{V}$. Choose Q to be the restriction of P_1 to $\mathcal{V}'$, renormalized by $1/\beta$, where $\beta = \int_{\mathcal{V}'} P_1(\mathrm{d}\boldsymbol{x})$.

By the exp-concavity of ℓ_t, for any $\boldsymbol{x} \in \mathcal{V}'$ there exists $\boldsymbol{w} \in \mathcal{V}$ such that

$$e^{-\alpha\ell_t(\boldsymbol{x})} \geq \frac{T/d}{T/d+1}e^{-\alpha\ell_t(\boldsymbol{u})} + \frac{1}{T/d+1}e^{-\alpha\ell_t(\boldsymbol{w})} \geq \frac{T/d}{T/d+1}e^{-\alpha\ell_t(\boldsymbol{u})}\,.$$

Hence, we have for any $\boldsymbol{x} \in \mathcal{V}'$

$$e^{-\alpha\sum_{t=1}^{T}\ell_t(\boldsymbol{x})} \geq \left(\frac{T/d}{T/d+1}\right)^T e^{-\alpha\sum_{t=1}^{T}\ell_t(\boldsymbol{u})} \geq \frac{e^{-\alpha\sum_{t=1}^{T}\ell_t(\boldsymbol{u})}}{e^d},$$

which implies

$$\sum_{t=1}^{T}\mathbb{E}_{\boldsymbol{x}\sim Q}[\ell_t(\boldsymbol{x})] \leq \frac{d}{\alpha} + \sum_{t=1}^{T}\ell_t(\boldsymbol{u})\,.$$

Moreover, the KL term becomes

$$\mathrm{KL}(Q;P_1) = \int_{\mathcal{V}} \ln\frac{Q(\mathrm{d}\boldsymbol{x})}{P_1(\mathrm{d}\boldsymbol{x})}Q(\mathrm{d}\boldsymbol{x}) = -\ln\beta = -\ln\int_{\mathcal{V}'}P_1(\mathrm{d}\boldsymbol{x})\,.$$

Given that P_1 is uniform on $\mathcal{V}$, we have that

$$\int_{\mathcal{V}'}P_1(\mathrm{d}\boldsymbol{x}) = \frac{Vol(\mathcal{V}')}{Vol(\mathcal{V})} = \frac{1}{(T/d+1)^d},$$

because $\mathcal{V}'$ is a scaling and translation of $\mathcal{V}$.

Using Theorem 11.3 and putting everything together, we have the stated bound. □

In Section 13.1, we will show an example of how to obtain a regret bound against a deterministic competitor without using a uniform prior.

11.2.1 ONS and OGD as instantiations of WAA

In this section, we show that WAA is more general than one might think. In fact, we will see that it can be equivalent to Online Gradient Descent (OGD) and to the Online Newton Step (ONS).

Here, we set $\mathcal{X} = \mathbb{R}^d$ and P_1 as the Gaussian distribution $\mathcal{N}(\boldsymbol{x}_1, \boldsymbol{\Sigma}_1)$, where $\boldsymbol{x}_1 \in \mathcal{V}$ and $\boldsymbol{\Sigma}_1 \succ 0$. We use the losses $\tilde{\ell}_t(\boldsymbol{x}) := \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle + \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}_t\|^2_{\boldsymbol{M}_t}$, where $\boldsymbol{M}_t \succeq 0$ for an arbitrary sequence of $\boldsymbol{g}_t$ in $\mathbb{R}^d$. Assume $\mathcal{V}$ is closed and convex. Finally, we set $\lambda_t = \lambda > 0$ for all t. Hence, we have

$$P_t = \operatorname*{argmin}_{P\in\Delta_\mu(\mathcal{V})} \mathrm{KL}(P;\hat{P}_t) = \mathcal{N}(\boldsymbol{x}_t, \boldsymbol{\Sigma}_t),$$

where

$$\begin{aligned}
\boldsymbol{x}_t &= \operatorname*{argmin}_{\boldsymbol{x}\in\mathcal{V}} (\boldsymbol{x}-\hat{\boldsymbol{x}}_t)^\top \boldsymbol{\Sigma}_t^{-1}(\boldsymbol{x}-\hat{\boldsymbol{x}}_t),\\
\hat{P}_t(\mathrm{d}\boldsymbol{x}) &\propto e^{-\frac{1}{2}\|\boldsymbol{x}-\boldsymbol{x}_1\|^2_{\boldsymbol{\Sigma}_1^{-1}} - \frac{1}{\lambda}\sum_{i=1}^{t-1}\left(\langle \boldsymbol{g}_i,\boldsymbol{x}\rangle + \frac{1}{2}\|\boldsymbol{x}-\boldsymbol{x}_i\|^2_{\boldsymbol{M}_i}\right)} = \mathcal{N}(\hat{\boldsymbol{x}}_t, \boldsymbol{\Sigma}_t)\,,\\
\hat{\boldsymbol{x}}_t &= \boldsymbol{\Sigma}_t\left(\boldsymbol{\Sigma}_1^{-1}\boldsymbol{x}_1 + \frac{1}{\lambda}\sum_{i=1}^{t-1}(\boldsymbol{M}_i\boldsymbol{x}_i - \boldsymbol{g}_i)\right),\\
\boldsymbol{\Sigma}_t &= \left(\boldsymbol{\Sigma}_1^{-1} + \frac{1}{\lambda}\sum_{i=1}^{t-1}\boldsymbol{M}_i\right)^{-1}.
\end{aligned}$$

From the above, we clearly have $\boldsymbol{x}_t = \mathbb{E}_{\boldsymbol{x}\sim P_t}[\boldsymbol{x}]$.

However, $\boldsymbol{x}_t$ can also be written as

$$\boldsymbol{x}_t = \operatorname*{argmin}_{\boldsymbol{x}\in\mathcal{V}} \frac{\lambda}{2}\|\boldsymbol{x}-\boldsymbol{x}_1\|^2_{\boldsymbol{\Sigma}_1^{-1}} + \sum_{i=1}^{t-1}\left(\langle \boldsymbol{g}_i,\boldsymbol{x}\rangle + \frac{1}{2}\|\boldsymbol{x}-\boldsymbol{x}_i\|^2_{\boldsymbol{M}_i}\right),$$

that is, the FTRL solution with regularizer $\psi(\boldsymbol{x}) = \frac{\lambda}{2}\|\boldsymbol{x}-\boldsymbol{x}_1\|^2_{\boldsymbol{\Sigma}_1^{-1}}$ on the surrogate losses $\tilde{\ell}_t$. This implies that $\boldsymbol{x}_t$ is equivalent to the output of the algorithm for the weaker notion of strong convexity we have analyzed in Section 7.10. So, using the notation $\boldsymbol{S}_t = \lambda\boldsymbol{\Sigma}_1^{-1} + \sum_{i=1}^{t}\boldsymbol{M}_i$, we can directly use the bound we proved for it:

$$\sum_{t=1}^{T}\tilde{\ell}_t(\boldsymbol{x}_t) - \sum_{t=1}^{T}\tilde{\ell}_t(\boldsymbol{u}) \le \frac{\lambda}{2}\|\boldsymbol{u}-\boldsymbol{x}_1\|^2_{\boldsymbol{\Sigma}_1^{-1}} - \min_{\boldsymbol{x}\in\mathcal{V}}\frac{\lambda}{2}\|\boldsymbol{x}-\boldsymbol{x}_1\|^2_{\boldsymbol{\Sigma}_1^{-1}} + \frac{1}{2}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|^2_{\boldsymbol{S}_t^{-1}}\,.$$

Consider now that we have losses $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$, set $\boldsymbol{g}_t \in \partial\ell_t(\boldsymbol{x}_t)$, and choose $\boldsymbol{M}_t$ such that

$$\sum_{t=1}^{T}\ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T}\ell_t(\boldsymbol{u}) \le \sum_{t=1}^{T}\tilde{\ell}_t(\boldsymbol{x}_t) - \sum_{t=1}^{T}\tilde{\ell}_t(\boldsymbol{u})\,.$$

So, we use the WAA algorithm with the surrogate losses $\tilde{\ell}_t$, to upper bound the regret on the losses ℓ_t. Hence, we have the following cases:

- If ℓ_t are exp-concave on $\mathcal{V}$, we can set $\boldsymbol{M}_t$ proportional to $\boldsymbol{g}_t\boldsymbol{g}_t^\top$ and we recover the regret upper bound of the ONS algorithm in Section 7.10.
- For convex losses, we have $\boldsymbol{M}_t = \boldsymbol{0}$. Hence, for all $\boldsymbol{u}\in\mathcal{V}$, we have

$$\begin{aligned}
\sum_{t=1}^{T}\ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T}\ell_t(\boldsymbol{u}) &\le \sum_{t=1}^{T}\tilde{\ell}_t(\boldsymbol{x}_t) - \sum_{t=1}^{T}\tilde{\ell}_t(\boldsymbol{u})\\
&\le \frac{\lambda}{2}\|\boldsymbol{u}-\boldsymbol{x}_1\|^2_{\boldsymbol{\Sigma}_1^{-1}} - \min_{\boldsymbol{x}\in\mathcal{V}}\frac{\lambda}{2}\|\boldsymbol{x}-\boldsymbol{x}_1\|^2_{\boldsymbol{\Sigma}_1^{-1}} + \frac{1}{2\lambda}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|^2_{\boldsymbol{\Sigma}_1}\,.
\end{aligned}$$

If in addition $\boldsymbol{\Sigma}_1 = \boldsymbol{I}_d$, for all $\boldsymbol{u} \in \mathcal{V}$, we have

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \le \sum_{t=1}^{T} \tilde{\ell}_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \tilde{\ell}_t(\boldsymbol{u}) \le \frac{\lambda}{2}\|\boldsymbol{x}_1 - \boldsymbol{u}\|_2^2 - \min_{\boldsymbol{x}\in\mathcal{V}} \frac{\lambda}{2}\|\boldsymbol{x} - \boldsymbol{x}_1\|_2^2 + \frac{1}{2\lambda}\sum_{t=1}^{T} \|\boldsymbol{g}_t\|_2^2 .$$

- For β-strongly convex losses with respect to $\|\cdot\|_2$ on $\mathcal{V}$, we have $\boldsymbol{M}_t = \beta \boldsymbol{I}_d$. Hence, for $\lambda = 1$ and any sequence of covariance matrices with $\boldsymbol{\Sigma}_{1,n}^{-1} \succ 0$ and $\boldsymbol{\Sigma}_{1,n}^{-1} \to \boldsymbol{0}$, for all $\boldsymbol{u} \in \mathcal{V}$, the bound above yields in the limit

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \le \sum_{t=1}^{T} \tilde{\ell}_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \tilde{\ell}_t(\boldsymbol{u}) \le \frac{1}{2}\sum_{t=1}^{T} \frac{\|\boldsymbol{g}_t\|_2^2}{\beta t} .$$

In the next theorem, we also show that the bound in Lemma 11.1 is powerful enough to capture all the cases above.

Theorem 11.6 *Assume that $\mathcal{V}$ is closed and convex. Let $\mathcal{X} = \mathbb{R}^d$ and run WAA on the losses $\tilde{\ell}_t(\boldsymbol{x}) := \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle + \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}_t\|_{\boldsymbol{M}_t}^2$, where $\boldsymbol{g}_t \in \mathbb{R}^d$ and $\boldsymbol{M}_t \succeq 0$ are arbitrary for all t. Set $P_1 = \mathcal{N}(\boldsymbol{x}_1, \boldsymbol{\Sigma}_1)$, where $\boldsymbol{x}_1 \in \mathcal{V}$ and $\boldsymbol{\Sigma}_1 \succ 0$. Set $\lambda_t = \lambda > 0$ for all t. Then, for all $\boldsymbol{u} \in \mathcal{V}$, we have*

$$\sum_{t=1}^{T} \left(\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{u})\right) \le \frac{\lambda}{2}\|\boldsymbol{x}_1 - \boldsymbol{u}\|_{\boldsymbol{\Sigma}_1^{-1}}^2 + \frac{1}{2\lambda}\sum_{t=1}^{T} \|\boldsymbol{g}_t\|_{\boldsymbol{\Sigma}_{t+1}}^2,$$

where $\boldsymbol{\Sigma}_{t+1} = \left(\boldsymbol{\Sigma}_1^{-1} + \frac{1}{\lambda}\sum_{i=1}^{t} \boldsymbol{M}_i\right)^{-1}$ for all t.

Proof Select $Q = \mathcal{N}(\boldsymbol{u}, \boldsymbol{C})$ for any $\boldsymbol{u} \in \mathcal{V}$ and we will specify $\boldsymbol{C}$ in the following.

Observe that the quadratic nature of the losses allows us to easily go from a stochastic to a deterministic competitor:

$$\mathbb{E}_{\boldsymbol{x}\sim Q}[\tilde{\ell}_t(\boldsymbol{x})] = \langle \boldsymbol{g}_t, \boldsymbol{u}\rangle + \frac{\mathrm{Tr}[\boldsymbol{C}\boldsymbol{M}_t]}{2} + \frac{\|\boldsymbol{u} - \boldsymbol{x}_t\|_{\boldsymbol{M}_t}^2}{2} = \tilde{\ell}_t(\boldsymbol{u}) + \frac{\mathrm{Tr}[\boldsymbol{C}\boldsymbol{M}_t]}{2} .$$

Hence, we have

$$\sum_{t=1}^{T} \left(\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{u})\right) = \sum_{t=1}^{T} \left(\tilde{\ell}_t(\boldsymbol{x}_t) - \mathbb{E}_{\boldsymbol{x}\sim Q}[\tilde{\ell}_t(\boldsymbol{x})]\right) + \frac{1}{2}\,\mathrm{Tr}\left(\boldsymbol{C}\sum_{t=1}^{T} \boldsymbol{M}_t\right) .$$

Now, we focus on the terms in the bound in Lemma 11.1. For the terms in the sum, we have

$$\lambda \ln \mathbb{E}_{\boldsymbol{x}\sim P_t}\left[e^{-\frac{1}{\lambda}\left(\tilde{\ell}_t(\boldsymbol{x}) - \tilde{\ell}_t(\boldsymbol{x}_t)\right)}\right] = \lambda \ln \mathbb{E}_{\boldsymbol{x}\sim P_t}\left[e^{-\frac{1}{\lambda}\left(\langle \boldsymbol{g}_t, \boldsymbol{x} - \boldsymbol{x}_t\rangle + \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}_t\|_{\boldsymbol{M}_t}^2\right)}\right] = \frac{\boldsymbol{g}_t^\top \boldsymbol{\Sigma}_{t+1}\boldsymbol{g}_t}{2\lambda} - \frac{\lambda}{2}\ln\frac{|\boldsymbol{\Sigma}_t|}{|\boldsymbol{\Sigma}_{t+1}|}, \quad (11.7)$$

where we leave the verification of the last equality as an exercise (see Problem 11.2). For the KL divergence term, we have

$$\mathrm{KL}(Q; P_1) = \frac{1}{2}\left(\ln\frac{|\boldsymbol{\Sigma}_1|}{|\boldsymbol{C}|} + \mathrm{Tr}(\boldsymbol{C}\boldsymbol{\Sigma}_1^{-1}) - d + (\boldsymbol{x}_1 - \boldsymbol{u})^\top \boldsymbol{\Sigma}_1^{-1}(\boldsymbol{x}_1 - \boldsymbol{u})\right) .$$

Putting everything together, we obtain

$$\frac{\lambda}{2}\left(\ln\frac{|\boldsymbol{\Sigma}_1|}{|\boldsymbol{C}|}+\operatorname{Tr}(\boldsymbol{C}\boldsymbol{\Sigma}_1^{-1})-d+(\boldsymbol{x}_1-\boldsymbol{u})^\top\boldsymbol{\Sigma}_1^{-1}(\boldsymbol{x}_1-\boldsymbol{u})\right)+\frac{1}{2\lambda}\sum_{t=1}^{T}\boldsymbol{g}_t^\top\boldsymbol{\Sigma}_{t+1}\boldsymbol{g}_t$$
$$-\frac{\lambda}{2}\ln\frac{|\boldsymbol{\Sigma}_1|}{|\boldsymbol{\Sigma}_{T+1}|}+\frac{1}{2}\operatorname{Tr}\left[\boldsymbol{C}\sum_{t=1}^{T}\boldsymbol{M}_t\right].$$

We now select $\boldsymbol{C}=\boldsymbol{\Sigma}_{T+1}=(\boldsymbol{\Sigma}_1^{-1}+\frac{1}{\lambda}\sum_{t=1}^{T}\boldsymbol{M}_t)^{-1}$, which minimizes the bound, to have the stated bound. □

11.3 The Aggregating Algorithm and Mixable Losses

Here, we show how to obtain a guarantee similar to the one of Theorem 11.3 for a larger class of loss functions. As above, P_1 is a probability distribution on $\mathcal{X}$; whenever densities with respect to a background measure are written, they are understood with respect to the reference measure m.

In the case of α-exp-concave losses, we used the fact that

$$\lambda_t\ln\mathbb{E}_{\boldsymbol{x}\sim P_t}\left[e^{-\frac{1}{\lambda_t}\ell_t(\boldsymbol{x})}\right]+\ell_t\left(\mathbb{E}_{\boldsymbol{x}\sim P_t}[\boldsymbol{x}]\right)\le 0,$$

for all $\frac{1}{\lambda_t}\le\alpha$. Now, we want to extend the class of functions where we can use a similar inequality. So, we introduce a more general class of losses, the α-*mixable* ones.

Definition 11.7 *Let $\mathcal{V}\subseteq\mathcal{X}$ and $f:\mathcal{X}\to\mathbb{R}$. We say that f is α-**mixable** on $\mathcal{V}$ if there exists a mapping $s:\Delta(\mathcal{X})\to\mathcal{V}$ called **substitution function** such that*

$$\frac{1}{\alpha}\ln\mathbb{E}_{\boldsymbol{x}\sim P}\left[e^{-\alpha f(\boldsymbol{x})}\right]+f\left(s(P)\right)\le 0,\quad\forall P\in\Delta(\mathcal{X}).$$

Proposition 11.8 *Let $\alpha>0$ and $\mathcal{X}\subseteq\mathbb{R}^d$. If $f:\mathcal{X}\to\mathbb{R}$ is α-mixable, then f is β-mixable for any $0<\beta<\alpha$.*

Proof To prove that f is β-mixable, it is enough to show that $h(\alpha)\ge h(\beta)$ for $h(y)=\left(\mathbb{E}_{\boldsymbol{x}\sim P}\left[e^{-yf(\boldsymbol{x})}\right]\right)^{\frac{1}{y}}$. Observe that

$$h(\alpha)=\left(\mathbb{E}_{\boldsymbol{x}\sim P}\left[e^{-\alpha f(\boldsymbol{x})}\right]\right)^{\frac{1}{\alpha}}=\left(\mathbb{E}_{\boldsymbol{x}\sim P}\left[\left(e^{-f(\boldsymbol{x})}\right)^{\alpha}\right]\right)^{\frac{1}{\alpha}}$$
$$=\left(\mathbb{E}_{\boldsymbol{x}\sim P}\left[\left(e^{-f(\boldsymbol{x})}\right)^{\beta\frac{\alpha}{\beta}}\right]\right)^{\frac{1}{\alpha}}\ge\left(\mathbb{E}_{\boldsymbol{x}\sim P}\left[\left(e^{-f(\boldsymbol{x})}\right)^{\beta}\right]\right)^{\frac{1}{\beta}}=h(\beta),$$

where we used Jensen's inequality (Theorem 2.9) on the convex function $x\mapsto x^{\frac{\alpha}{\beta}}$. □

There is an additional caveat: the substitution function here depends on f, and in online learning, we only know the loss function after producing our prediction. So, in our setting, we need to find a generic substitution function that holds for a *class* of loss functions.

Hence, we consider $\ell_t(\boldsymbol{x})=\ell(\boldsymbol{x},\boldsymbol{y}_t)$ where $\boldsymbol{y}_t\in\mathcal{Y}$, and we require

$$\frac{1}{\alpha}\ln\mathbb{E}_{\boldsymbol{x}\sim P}\left[e^{-\alpha\ell(\boldsymbol{x},\boldsymbol{y})}\right]+\ell\left(s(P),\boldsymbol{y}\right)\le 0,\quad\forall P\in\Delta(\mathcal{X}),\forall\boldsymbol{y}\in\mathcal{Y}.$$

An example of such losses is given in the following proposition.

Proposition 11.9 *Define the softmax* $\sigma : \mathbb{R}^K \to \Delta_{>0}^{K-1} := \{\boldsymbol{x} \in \Delta^{K-1} : x_i > 0, i = 1, \dots, K\}$ *as* $\sigma(\boldsymbol{z})_k = \frac{\exp(z_k)}{\sum_{i=1}^K \exp(z_i)}$. *The multiclass logistic loss, also referred to as softmax cross-entropy loss,* $\ell : \mathbb{R}^K \times \{1, \dots, K\} \to \mathbb{R}$, *defined as* $\ell(\boldsymbol{x}, y) = -\ln(\sigma(\boldsymbol{x})_y)$, *is 1-mixable.*

Proof The proof is by construction: define the mapping s as $s(P) = \ln(\mathbb{E}_{\boldsymbol{x}\sim P}[\sigma(\boldsymbol{x})])$ for any distribution $P \in \Delta(\mathbb{R}^K)$, where the logarithm is entry-wise. So, for any $y \in \{1, \dots, K\}$, we have

$$\mathbb{E}_{\boldsymbol{x}\sim P}\left[e^{-\ell(\boldsymbol{x},y)}\right] = \mathbb{E}_{\boldsymbol{x}\sim P}[\sigma(\boldsymbol{x})_y] = \sigma\left(\ln \mathbb{E}_{\boldsymbol{x}\sim P}[\sigma(\boldsymbol{x})]\right)_y = \sigma(s(P))_y = e^{-\ell(s(P),y)},$$

where the third equality uses $\sigma(\ln(\boldsymbol{p})) = \boldsymbol{p}$ for all $\boldsymbol{p} \in \Delta_{>0}^{K-1}$. Thus, ℓ is 1-mixable. □

It is worth stressing that not all convex losses are mixable, as proved next.

Proposition 11.10 *Let* $\mathcal{V} = \mathcal{X} = \mathbb{R}$. *The loss* $\ell : \mathbb{R} \times \{-1, 1\} \to \mathbb{R}$, $\ell(x, y) = |x - y|$, *is not mixable.*

Proof If the loss were mixable, there would exist $\alpha > 0$ and a substitution function s such that

$$\frac{1}{\alpha} \ln \mathbb{E}_{x\sim P}\left[e^{-\alpha|x-y|}\right] + |s(P) - y| \le 0, \quad \forall P \in \Delta(\mathcal{X}), y \in \{-1, 1\}\,.$$

Consider $P(-1) = P(1) = 1/2$. We need to find a $\mu \in \mathbb{R}$ and $\alpha > 0$ such that

$$\mathbb{E}_{x\sim P}\left[e^{-\alpha|x-1|}\right] \le e^{-\alpha|\mu-1|} \quad \text{and} \quad \mathbb{E}_{x\sim P}\left[e^{-\alpha|x+1|}\right] \le e^{-\alpha|\mu+1|}\,.$$

Let's calculate the expectations:

$$\mathbb{E}_{x\sim P}\left[e^{-\alpha|x-1|}\right] = \frac{1}{2} + \frac{1}{2}e^{-2\alpha} \quad \text{and} \quad \mathbb{E}_{x\sim P}\left[e^{-\alpha|x+1|}\right] = \frac{1}{2} + \frac{1}{2}e^{-2\alpha}\,.$$

Hence, we need to satisfy

$$-\frac{\ln\left(\frac{1}{2} + \frac{1}{2}e^{-2\alpha}\right)}{\alpha} \ge |\mu - 1| \quad \text{and} \quad -\frac{\ln\left(\frac{1}{2} + \frac{1}{2}e^{-2\alpha}\right)}{\alpha} \ge |\mu + 1|\,.$$

Summing these two inequalities, we have

$$-2\frac{\ln\left(\frac{1}{2} + \frac{1}{2}e^{-2\alpha}\right)}{\alpha} \ge |\mu - 1| + |\mu + 1| \ge 2\,.$$

Solving for α, we have $2 \ge e^{-\alpha} + e^{\alpha}$ which has the only solution $\alpha = 0$. Hence, the function is not mixable. □

It is clear that the proof of WAA holds for $\boldsymbol{x}_t = s(P_t)$ and $\lambda_t \ge \frac{1}{\alpha}$, instead of $\boldsymbol{x}_t = \mathbb{E}_{\boldsymbol{x}\sim P_t}[\boldsymbol{x}]$. Moreover, every α-exp-concave function is α-mixable when $\mathcal{V} \equiv \mathcal{X}$ is convex and bounded, because the substitution function is $s(P) = \mathbb{E}_{\boldsymbol{x}\sim P}[\boldsymbol{x}]$. Note that the boundedness ensures that the expectation exists. This implies that all the following regret guarantees also hold for exp-concave functions.

Equipped with the definition of mixability, we can introduce the AA in Algorithm 11.2.

Algorithm 11.2 Aggregating Algorithm (AA)

Require: Non-empty closed sets $\mathcal{V} \subseteq \mathcal{X} \subseteq \mathbb{R}^d$, $\lambda_1, \dots, \lambda_T > 0$, substitution function $s : \Delta(\mathcal{X}) \to \mathcal{V}$

Set P_1 to be a distribution supported on $\mathcal{X}$
for $t = 1$ **to** T **do**
  Set $P_t \in \operatorname{argmin}_{P \in \Delta(\mathcal{X})} \lambda_t \operatorname{KL}(P; P_1) + \sum_{i=1}^{t-1} \mathbb{E}_{\boldsymbol{x} \sim P}[\ell(\boldsymbol{x}, \boldsymbol{y}_i)]$
  Output $\boldsymbol{x}_t = s(P_t)$
  Get $\boldsymbol{y}_t$ and pay the loss $\ell(\boldsymbol{x}_t, \boldsymbol{y}_t)$
end for

Its regret guarantee is the following one, and its proof is identical to that of WAA using $\boldsymbol{x}_t = s(P_t)$.

Theorem 11.11 *Let $P_1 \in \Delta(\mathcal{X})$. Assume $\ell : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ is α-mixable in the first argument with substitution function $s : \Delta(\mathcal{X}) \to \mathcal{V}$, and $\frac{1}{\alpha} \le \lambda_t \le \lambda_{t+1}$ for $t = 1, \dots, T-1$, and $\lambda_{T+1} = \lambda_T$. Then, Algorithm 11.2 satisfies*

$$\sum_{t=1}^{T} \ell(\boldsymbol{x}_t, \boldsymbol{y}_t) \le \mathbb{E}_{\boldsymbol{x} \sim Q} \left[\sum_{t=1}^{T} \ell(\boldsymbol{x}, \boldsymbol{y}_t) \right] + \lambda_T \operatorname{KL}(Q; P_1), \quad \forall Q \in \Delta(\mathcal{X}) .$$

Moreover, we also have

$$\sum_{t=1}^{T} \ell(\boldsymbol{x}_t, \boldsymbol{y}_t) \le -\lambda_T \ln \mathbb{E}_{\boldsymbol{x} \sim P_1} \left[e^{-\frac{1}{\lambda_T} \sum_{t=1}^{T} \ell(\boldsymbol{x}, \boldsymbol{y}_t)} \right] .$$

As we did for WAA, under additional assumptions, we can also give a regret bound with respect to a deterministic competitor.

Theorem 11.12 *Let $\mathcal{V} \equiv \mathcal{X}$ be a non-empty closed convex set in $\mathbb{R}^d$, full-dimensional in $\mathbb{R}^d$, and $D := \max_{\boldsymbol{x}, \boldsymbol{y} \in \mathcal{V}} \|\boldsymbol{x} - \boldsymbol{y}\| < \infty$, where $\|\cdot\|$ is an arbitrary norm. Let $\ell : \mathcal{V} \times \mathcal{Y} \to \mathbb{R}$ and assume that $\ell(\cdot, \boldsymbol{y}_t)$ is L_t-Lipschitz with respect to $\|\cdot\|$ and α-mixable. Then, the Aggregating Algorithm in Algorithm 11.2 with a sequence of λ_t such that $\frac{1}{\alpha} \le \lambda_t \le \lambda_{t+1}$ for $t = 1, \dots, T-1$, $\lambda_{T+1} = \lambda_T$, and P_1 as the uniform distribution on $\mathcal{V}$ satisfies*

$$\begin{aligned} \sum_{t=1}^{T} (\ell(\boldsymbol{x}_t, \boldsymbol{y}_t) - \ell(\boldsymbol{u}, \boldsymbol{y}_t)) &\le d\lambda_T \ln \left(\max \left(1, \frac{\frac{1}{\lambda_T} D \sum_{t=1}^{T} L_t}{d} \right) \right) + d\lambda_T \\ &\le 2d\lambda_T \ln \left(\frac{\frac{1}{\lambda_T} D \sum_{t=1}^{T} L_t}{d} + e \right), \quad \forall \boldsymbol{u} \in \mathcal{V} . \end{aligned}$$

Proof We start from the second bound in Theorem 11.11. Fix $\theta \in [0, 1)$, $\boldsymbol{u} \in \mathcal{V}$, and define $\mathcal{V}' = \{\boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{x} = \theta \boldsymbol{u} + (1 - \theta)\boldsymbol{w}, \boldsymbol{w} \in \mathcal{V}\} \subseteq \mathcal{V}$. Choose P_1' to be the restriction of P_1 to $\mathcal{V}'$, renormalized to be a probability distribution. That is, $P_1'(\mathcal{A}) := \frac{P_1(\mathcal{A} \cap \mathcal{V}')}{\beta}$ for any $\mathcal{A} \subseteq \mathcal{V}$ measurable, where $\beta = P_1(\mathcal{V}') = (1 - \theta)^d$. Since P_1 is the uniform distribution,

P_1' is uniform over $\mathcal{V}'$. So, we have

$$\begin{aligned}
\mathbb{E}_{\boldsymbol{x}\sim P_1}\left[e^{-\frac{1}{\lambda_T}\sum_{t=1}^T \ell(\boldsymbol{x},\boldsymbol{y}_t)}\right] &\geq (1-\theta)^d \mathbb{E}_{\boldsymbol{x}\sim P_1'}\left[e^{-\frac{1}{\lambda_T}\sum_{t=1}^T \ell(\boldsymbol{x},\boldsymbol{y}_t)}\right] \\
&= (1-\theta)^d \mathbb{E}_{\boldsymbol{x}\sim P_1}\left[e^{-\frac{1}{\lambda_T}\sum_{t=1}^T \ell(\theta\boldsymbol{u}+(1-\theta)\boldsymbol{x},\boldsymbol{y}_t)}\right] \\
&= (1-\theta)^d \mathbb{E}_{\boldsymbol{x}\sim P_1}\left[e^{-\frac{1}{\lambda_T}\sum_{t=1}^T \ell((1-\theta)(\boldsymbol{x}-\boldsymbol{u})+\boldsymbol{u},\boldsymbol{y}_t)}\right] \\
&\geq (1-\theta)^d \mathbb{E}_{\boldsymbol{x}\sim P_1}\left[e^{-\frac{1}{\lambda_T}\sum_{t=1}^T (\ell(\boldsymbol{u},\boldsymbol{y}_t)+L_t(1-\theta)\|\boldsymbol{x}-\boldsymbol{u}\|)}\right] \\
&\geq (1-\theta)^d \mathbb{E}_{\boldsymbol{x}\sim P_1}\left[e^{-\frac{1}{\lambda_T}\sum_{t=1}^T (\ell(\boldsymbol{u},\boldsymbol{y}_t)+L_t(1-\theta)D)}\right] \\
&= (1-\theta)^d e^{-\frac{1}{\lambda_T}\sum_{t=1}^T \ell(\boldsymbol{u},\boldsymbol{y}_t)} e^{-\frac{1}{\lambda_T}\sum_{t=1}^T L_t(1-\theta)D} .
\end{aligned}$$

Putting everything together, we have

$$\sum_{t=1}^T (\ell(\boldsymbol{x}_t,\boldsymbol{y}_t)-\ell(\boldsymbol{u},\boldsymbol{y}_t)) \leq d\lambda_T \ln\frac{1}{1-\theta} + (1-\theta)D\sum_{t=1}^T L_t .$$

Now, we set θ such that $1-\theta = \min\left(\frac{d\lambda_T}{D\sum_{t=1}^T L_t}, 1\right)$. So, we have $(1-\theta)D\sum_{t=1}^T L_t \leq \lambda_T d$ and $\ln\frac{1}{1-\theta} = \ln\left(\max\left(1, \frac{\frac{1}{\lambda_T}D\sum_{t=1}^T L_t}{d}\right)\right)$, that gives the stated bound. □

11.4 Example of AA: Online Multiclass Logistic Regression

In this section, we show an application of AA to obtain logarithmic regret for online multiclass logistic regression. In this problem, in each round we receive a covariate $\boldsymbol{z}_t \in \mathcal{Z} \subseteq \mathbb{R}^d$ and we produce a discrete probability distribution over the K classes as $\hat{\boldsymbol{y}}_t \in \Delta^{K-1}$. Then, we receive the true class $y_t \in \{1,\dots,K\}$ and we pay the loss $-\ln \hat{\boldsymbol{y}}_{t,y_t}$.

We could use a linear strategy: $\hat{\boldsymbol{y}}_t = \sigma(\boldsymbol{X}_t^\top \boldsymbol{z}_t)$ where $\boldsymbol{X}_t \in \mathcal{V} \subseteq \mathbb{R}^{d\times K}$ is the linear classifier at time t. Assuming that the covariates $\boldsymbol{z}_t$ have bounded norm Z and that the columns of the matrices in $\mathcal{V}$ have bounded norm D, one can show that this problem has exp-concave losses, so we could use the ONS or the WAA algorithm. Unfortunately, the exp-concavity is of the order of $\exp(-ZD)$.

Here, we show that we can prove a logarithmic bound that depends only logarithmically on D. The price that we pay for this improved rate is that the algorithm we will use is *improper*, that is, our predictor will not be linear in the covariates $\boldsymbol{z}_t$, but we will still measure its regret with respect to a linear competitor.

We will proceed a little bit differently than in the standard AA algorithm because we will use two loss functions, $\ell_t : \mathcal{V}\times\mathcal{Y} \to \mathbb{R}$ and $\hat{\ell} : \mathbb{R}^K \times \mathcal{Y} \to \mathbb{R}$ that satisfy a *generalized form of α-mixability*:

$$\frac{1}{\alpha}\ln \mathbb{E}_{\boldsymbol{x}\sim P}\left[e^{-\alpha\ell_t(\boldsymbol{x},\boldsymbol{y})}\right] + \hat{\ell}\left(s_t(P),\boldsymbol{y}\right) \leq 0, \quad \forall P\in\Delta(\mathcal{V}), \forall \boldsymbol{y}\in\mathcal{Y} .$$

It is easy to see that Theorem 11.11 extends to this case:

$$\sum_{t=1}^T \hat{\ell}(\boldsymbol{x}_t,\boldsymbol{y}_t) \leq -\lambda_T \ln \mathbb{E}_{\boldsymbol{x}\sim P_1}\left[e^{-\frac{1}{\lambda_T}\sum_{t=1}^T \ell_t(\boldsymbol{x},\boldsymbol{y}_t)}\right] .$$

Algorithm 11.3 Aggregating Algorithm for Improper Online Multiclass Logistic Regression

Require: Non-empty bounded closed convex set $\mathcal{V} \subseteq \mathbb{R}^{d\times K}$, full-dimensional in $\mathbb{R}^{d\times K}$

Set P_1 to the uniform distribution over $\mathcal{V}$
for $t = 1$ **to** T **do**
  Set $P_t(\mathrm{d}\boldsymbol{X}) = \frac{\exp\left(\sum_{i=1}^{t-1} \ln(\sigma(\boldsymbol{X}^\top \boldsymbol{z}_i)_{y_i})\right) P_1(\mathrm{d}\boldsymbol{X})}{\int_{\mathcal{V}} \exp\left(\sum_{i=1}^{t-1} \ln(\sigma(\boldsymbol{U}^\top \boldsymbol{z}_i)_{y_i})\right) P_1(\mathrm{d}\boldsymbol{U})} \in \Delta(\mathcal{V})$
  Obtain $\boldsymbol{z}_t \in \mathcal{Z}$
  Output $\boldsymbol{x}_t = \ln \mathbb{E}_{\boldsymbol{X}\sim P_t}[\sigma(\boldsymbol{X}^\top \boldsymbol{z}_t)] \in \mathbb{R}^K$
  Obtain y_t and pay $-\ln \sigma(\boldsymbol{x}_t)_{y_t}$
end for

Similarly, we have that if ℓ_t is L_t-Lipschitz in its first argument, we have the corresponding version of Theorem 11.12:

$$\sum_{t=1}^{T} (\hat{\ell}(\boldsymbol{x}_t, \boldsymbol{y}_t) - \ell_t(\boldsymbol{u}, \boldsymbol{y}_t)) \le 2\dim(\mathcal{V})\lambda_T \ln\left(\frac{\frac{1}{\lambda_T} D \sum_{t=1}^{T} L_t}{\dim(\mathcal{V})} + e\right) \quad \forall \boldsymbol{u} \in \mathcal{V} \,. \tag{11.8}$$

In particular, from Proposition 11.9, we will define $\ell_t(\boldsymbol{X}, y) = -\ln(\sigma(\boldsymbol{X}^\top \boldsymbol{z}_t)_y)$, $s_t(P) = \ln \mathbb{E}_{\boldsymbol{X}\sim P}[\sigma(\boldsymbol{X}^\top \boldsymbol{z}_t)]$, and $\hat{\ell}(\boldsymbol{x}, y) = -\ln(\sigma(\boldsymbol{x})_y)$.

Theorem 11.13 *Let $\mathcal{B} \subset \mathbb{R}^d$ be closed, convex, and $\max_{\boldsymbol{x},\boldsymbol{y}\in\mathcal{B}} \|\boldsymbol{x} - \boldsymbol{y}\|_2 \le D$. Assume that $\|\boldsymbol{z}_t\|_2 \le Z$ for all t and $\mathcal{V} = \{\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_K] \in \mathbb{R}^{d\times K} : \boldsymbol{x}_i \in \mathcal{B}\}$, full-dimensional in $\mathbb{R}^{d\times K}$. Then, Algorithm 11.3 satisfies*

$$-\sum_{t=1}^{T} \ln(\sigma(\boldsymbol{x}_t)_{y_t}) + \sum_{t=1}^{T} \ln(\sigma(\boldsymbol{U}^\top \boldsymbol{z}_t)_{y_t}) \le 2d\,K \ln\left(\frac{2D\,Z\,T}{d\,K} + e\right), \quad \forall \boldsymbol{U} \in \mathcal{V} \,.$$

Proof Write $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_K]$ and, similarly, for any matrix $\boldsymbol{G} \in \mathbb{R}^{d\times K}$ write $\boldsymbol{G} = [\boldsymbol{g}_1, \ldots, \boldsymbol{g}_K]$, where $\boldsymbol{x}_j, \boldsymbol{g}_j \in \mathbb{R}^d$ denote the j-th columns. We have

$$\nabla_{\boldsymbol{x}_j} \ell_t(\boldsymbol{X}, y) = (\sigma(\boldsymbol{X}^\top \boldsymbol{z}_t)_j - \mathbf{1}\{y = j\})\boldsymbol{z}_t, \quad \forall j \in \{1, \ldots, K\} \,.$$

Define $\|\boldsymbol{X}\|_{2,\infty} := \max_{j\in\{1,\ldots,K\}} \|\boldsymbol{x}_j\|_2$. By Lemma 7.20, its dual norm is $\|\boldsymbol{G}\|_{2,1} := \sum_{j=1}^{K} \|\boldsymbol{g}_j\|_2$. Hence,

$$\|\nabla_{\boldsymbol{X}} \ell_t(\boldsymbol{X}, y_t)\|_{2,1} = \|\sigma(\boldsymbol{X}^\top \boldsymbol{z}_t) - \boldsymbol{e}_{y_t}\|_1 \|\boldsymbol{z}_t\|_2 \le 2Z \,.$$

Therefore, ℓ_t is $2Z$-Lipschitz with respect to $\|\cdot\|_{2,\infty}$. Moreover, if $\boldsymbol{X}, \boldsymbol{U} \in \mathcal{V}$, then each pair of corresponding columns satisfies $\|\boldsymbol{x}_j - \boldsymbol{u}_j\|_2 \le D$, and thus $\|\boldsymbol{X} - \boldsymbol{U}\|_{2,\infty} \le D$. So, the diameter of $\mathcal{V}$ with respect to $\|\cdot\|_{2,\infty}$ is at most D. Using (11.8) with $\dim(\mathcal{V}) = dK$, $L_t = 2Z$, and $\lambda_t = 1$, we obtain the stated result. □

11.5 History Bits

The WAA is from Kivinen and Warmuth (1999), as a simplification of the AA of Vovk (1990) (see also Vovk (1998, Appendix A) for an easier description of the AA). This algorithm is known by many names: Cesa-Bianchi and Lugosi (2006) calls it "exponentially

weighted mixture forecaster", Hazan et al. (2006, 2007) rediscovered it (see below) and named it "exponentially weighted online optimization algorithm", Wouter Koolen calls it simply "exponential weights algorithm" in his blog post in 2016 (see below). I prefer to use the name its designers gave it, also because its acronym nicely matches that of the Aggregating Algorithm.

The observation that the AA also works for infinite sets of experts was made by Freund (1996, 2003). Theorem 11.5 is from Hazan et al. (2006, 2007), where they seem to rediscover the WAA with uniform prior, but I improved it by adding a $1/d$ term in the logarithm. The proof of Hazan et al. (2006, 2007) is a generalization of that of Blum and Kalai (1997, 1999) for universal portfolio.

The observation that the WAA recovers the ONS and the gradient descent algorithm and its bounds is from a blog post[1] by Wouter Koolen, where it is done for OGD, while I derived it for FTRL. van der Hoeven and van Erven (2016) has extended this equivalence to Online Mirror Descent (OMD). The subtlety of defining the prior on $\mathcal{X}$ instead of on $\mathcal{V}$ is by me, and it allows us to state a single theorem that covers all the cases.

The concept of mixability is introduced in Vovk (2001). Proposition 11.8 is in the proof of Vovk (1998, Lemma 9). The mixability of the logistic loss, Proposition 11.9, and the content of Section 11.4 are from Foster et al. (2018). Theorem 11.12 is a generalization of a similar one for the logistic loss from Foster et al. (2018).

11.6 Exercises

Problem 11.1 *Prove the Donsker–Varadhan variational formula in* (11.5)*. Also identify the distribution attaining the infimum when the normalizing constant is finite and positive.*

Problem 11.2 *Prove the equality in* (11.7)*.*

Problem 11.3 *Let $\mathcal{X} = \mathcal{V} = [0, 1]$ and $\mathcal{Y} = [0, 1]$. Show that the square loss $\ell(x, y) = (x - y)^2$ is mixable for some positive constant α. Find an explicit value of α and construct a corresponding substitution function.*

[1] https://blog.wouterkoolen.info/EW4Quadratic/post.html

12

Black-Box Reductions

In this chapter, we show how to reduce one online learning problem into another, in a *black-box* way. That is, we will use an online convex optimization algorithm to solve a problem different from what it was meant to solve, without looking at its internal workings in any way. The only thing we will change is the input we pass to the algorithm. Why do it? Because you might have optimization software that you cannot modify, or just because designing and analyzing an online learning algorithm in one case might be easier than in another.

12.1 Solving Constrained Online Convex Optimization

We might have an Online Convex Optimization (OCO) algorithm designed for constrained online linear optimization over a feasible set $\mathcal{W} \subseteq \mathbb{R}^d$, and we might wonder how we can use it for online convex optimization on a feasible set $\mathcal{V} \subset \mathcal{W}$. How do we deal with the smaller feasible set?

Let's see a prototypical example of this approach. We have a OCO algorithm that outputs in each round $\boldsymbol{z}_t \in \mathcal{W} = \mathbb{R}^d_{\geq 0}$ and we want to use it for OCO in $\mathcal{V} = \Delta^{d-1}$. So, first of all, we have to transform $\boldsymbol{z}_t$ into a vector in the probability simplex. One way to do it could be to normalize it by its L_1 norm. So, we predict in each round by $\boldsymbol{x}_t = \frac{\boldsymbol{z}_t}{\|\boldsymbol{z}_t\|_1}$ if $\|\boldsymbol{z}_t\|_1 \neq 0$ and with any other vector in the simplex if $\|\boldsymbol{z}_t\|_1 = 0$. Given that we changed the prediction, the original regret upper bound no longer holds. So, one solution is to pass to the algorithm *surrogate losses*, so that the regret with the surrogate losses upper bounds the regret on the new problem with the modified predictions. In formulas, we have

$$\begin{aligned}\mathrm{Regret}_T(\boldsymbol{u}) &= \sum_{t=1}^T (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle \\ &= \sum_{t=1}^T (\langle \boldsymbol{g}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle),\end{aligned}$$

where $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$. Now, we want to find $\hat{\boldsymbol{g}}_t$ such that

$$\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle \leq \langle \hat{\boldsymbol{g}}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle, \quad \forall \boldsymbol{u} \in \mathcal{V}\,.$$

If we could do it, we would have

$$\mathrm{Regret}_T(\boldsymbol{u}) \leq \sum_{t=1}^T \langle \boldsymbol{g}_t + \hat{\boldsymbol{g}}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle,$$

that is the regret of the OCO algorithm on the surrogate linear losses $\boldsymbol{x} \mapsto \langle \boldsymbol{g}_t + \hat{\boldsymbol{g}}_t, \boldsymbol{x} \rangle$.

Let's see how to find $\hat{\boldsymbol{g}}_t$. Using our definition of $\boldsymbol{x}_t$, we have that if $\|\boldsymbol{z}_t\|_1 \neq 0$ then

$$\begin{aligned}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t \rangle &= \left\langle \boldsymbol{g}_t, \frac{\boldsymbol{z}_t}{\|\boldsymbol{z}_t\|_1} - \boldsymbol{z}_t \right\rangle = \left\langle \boldsymbol{g}_t, \frac{\boldsymbol{z}_t}{\|\boldsymbol{z}_t\|_1} \right\rangle (1 - \|\boldsymbol{z}_t\|_1) \\ &= -\langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \langle \mathbf{1}_d, \boldsymbol{z}_t - \boldsymbol{u} \rangle,\end{aligned}$$

where $\mathbf{1}_d = [1, \dots, 1]^\top \in \mathbb{R}^d$. Instead, if $\|\boldsymbol{z}_t\|_1 = 0$ we have

$$\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t \rangle = \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle = -\langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \langle \mathbf{1}_d, \boldsymbol{z}_t - \boldsymbol{u} \rangle \ .$$

Hence, we can set $\hat{\boldsymbol{g}}_t = -\langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \mathbf{1}_d$, and the surrogate linear losses we will pass to the OCO algorithm are

$$\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t - \mathbf{1}_d \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle, \boldsymbol{x} \rangle \ . \tag{12.1}$$

In this way, we have

$$\mathrm{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^T (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \sum_{t=1}^T (\tilde{\ell}_t(\boldsymbol{z}_t) - \tilde{\ell}_t(\boldsymbol{u})) \ .$$

It is worth noting that the norm of the gradients of the surrogate losses is controlled: $\|\boldsymbol{g}_t - \mathbf{1}_d \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle\|_\infty \leq 2\|\boldsymbol{g}_t\|_\infty$.

We solved our problem, but this approach does not seem to scale to arbitrarily complex feasible sets. Hence, we need a more general way to solve it. In the following, we will see two complementary ways to do it.

12.1.1 Dealing with Constraints using (Non-Euclidean) Projections

Now, we extend the procedure in the previous section to general convex sets and general norms. First of all, we need some definitions and basic results.

Definition 12.1 *Let $\mathcal{V} \subset \mathbb{R}^d$ be a non-empty, convex, and closed set. The **distance to a set** with respect to the norm $\|\cdot\|$ is the function $\mathrm{dist}_{\mathcal{V},\|\cdot\|} : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$ defined as $\mathrm{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x}) = \min_{\boldsymbol{y} \in \mathcal{V}} \|\boldsymbol{x} - \boldsymbol{y}\|$. We also define the **generalized projection** $\Pi_{\mathcal{V},\|\cdot\|}$ as the set-valued map $\Pi_{\mathcal{V},\|\cdot\|}(\boldsymbol{x}) = \mathrm{argmin}_{\boldsymbol{y} \in \mathcal{V}} \ \|\boldsymbol{x} - \boldsymbol{y}\|$.*

We have the following properties for the distance to a set function.

Lemma 12.2 *Let $\mathcal{V}$ be non-empty, convex, and closed. Then, we have*

- $\mathrm{dist}_{\mathcal{V},\|\cdot\|}$ *is convex.*
- $\mathrm{dist}_{\mathcal{V},\|\cdot\|}$ *is 1-Lipschitz with respect to $\|\cdot\|$.*
- *For all $\boldsymbol{x} \in \mathcal{V}$, we have $\mathbf{0} \in \partial \, \mathrm{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x})$.*
- *For all $\boldsymbol{x} \notin \mathcal{V}$, let $\boldsymbol{y} \in \Pi_{\mathcal{V},\|\cdot\|}(\boldsymbol{x})$. Then, we have*

$$\partial \, \mathrm{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x}) = \{\boldsymbol{g} : \ \|\boldsymbol{g}\|_\star = 1, \ \langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{y} \rangle = \|\boldsymbol{x} - \boldsymbol{y}\|, \ \langle \boldsymbol{g}, \boldsymbol{u} - \boldsymbol{y} \rangle \leq 0 \ \ \forall \boldsymbol{u} \in \mathcal{V}\} \ .$$

Proof For the first property, for any $\boldsymbol{x}' \in \Pi_{\mathcal{V},\|\cdot\|}(\boldsymbol{x})$ and any $\boldsymbol{z}' \in \Pi_{\mathcal{V},\|\cdot\|}(\boldsymbol{z})$, we have

$$\begin{aligned}\operatorname{dist}_{\mathcal{V},\|\cdot\|}(\lambda\boldsymbol{x}+(1-\lambda)\boldsymbol{z}) &= \min_{\boldsymbol{y}\in\mathcal{V}} \|\lambda\boldsymbol{x}+(1-\lambda)\boldsymbol{z}-\boldsymbol{y}\| \\ &\le \|\lambda\boldsymbol{x}+(1-\lambda)\boldsymbol{z}-(\lambda\boldsymbol{x}'+(1-\lambda)\boldsymbol{z}')\| \\ &= \|\lambda(\boldsymbol{x}-\boldsymbol{x}')+(1-\lambda)(\boldsymbol{z}-\boldsymbol{z}')\| \\ &\le \lambda \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x}) + (1-\lambda)\operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{z}),\end{aligned}$$

where in the first inequality we used the convexity of $\mathcal{V}$.

For the second property, let $\boldsymbol{x}, \boldsymbol{z} \in \mathbb{R}^d$ and $\boldsymbol{z}' \in \Pi_{\mathcal{V},\|\cdot\|}(\boldsymbol{z})$. Then,

$$\begin{aligned}\operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x}) &= \min_{\boldsymbol{y}\in\mathcal{V}} \|\boldsymbol{x}-\boldsymbol{y}\| \le \|\boldsymbol{x}-\boldsymbol{z}'\| \le \|\boldsymbol{x}-\boldsymbol{z}\| + \|\boldsymbol{z}-\boldsymbol{z}'\| \\ &= \|\boldsymbol{x}-\boldsymbol{z}\| + \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{z})\,.\end{aligned}$$

Hence, $\left|\operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x}) - \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{z})\right| \le \|\boldsymbol{x}-\boldsymbol{z}\|$, where the reverse inequality is obtained by exchanging the roles of $\boldsymbol{x}$ and $\boldsymbol{z}$.

The third property is immediate using the optimality condition in Theorem 6.27.

We now prove the fourth property. First, observe that $\|\boldsymbol{y}-\boldsymbol{x}\| = \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x})$. We now show that every subgradient $\boldsymbol{g} \in \partial \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x})$ belongs to the stated set. We have

$$0 = \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{y}) \ge \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x}) + \langle \boldsymbol{g}, \boldsymbol{y}-\boldsymbol{x}\rangle = \|\boldsymbol{x}-\boldsymbol{y}\| - \langle \boldsymbol{g}, \boldsymbol{x}-\boldsymbol{y}\rangle,$$

hence $\|\boldsymbol{x}-\boldsymbol{y}\| \le \langle \boldsymbol{g}, \boldsymbol{x}-\boldsymbol{y}\rangle$. Since $\operatorname{dist}_{\mathcal{V},\|\cdot\|}$ is 1-Lipschitz, by Theorem 4.21 every subgradient satisfies $\|\boldsymbol{g}\|_\star \le 1$. Therefore, by the definition of dual norms, we have

$$\|\boldsymbol{x}-\boldsymbol{y}\| \le \langle \boldsymbol{g}, \boldsymbol{x}-\boldsymbol{y}\rangle \le \|\boldsymbol{g}\|_\star \|\boldsymbol{x}-\boldsymbol{y}\| \le \|\boldsymbol{x}-\boldsymbol{y}\|\,.$$

As $\boldsymbol{x} \notin \mathcal{V}$, we have $\|\boldsymbol{x}-\boldsymbol{y}\| > 0$, and thus equality holds throughout:

$$\|\boldsymbol{g}\|_\star = 1, \qquad \langle \boldsymbol{g}, \boldsymbol{x}-\boldsymbol{y}\rangle = \|\boldsymbol{x}-\boldsymbol{y}\|\,.$$

It remains to prove the normal-cone condition. Let $\boldsymbol{u} \in \mathcal{V}$. Since $\boldsymbol{g} \in \partial \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x})$, the subgradient inequality gives

$$0 = \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{u}) \ge \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x}) + \langle \boldsymbol{g}, \boldsymbol{u}-\boldsymbol{x}\rangle\,.$$

Using $\operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x}) = \|\boldsymbol{x}-\boldsymbol{y}\|$ and $\langle \boldsymbol{g}, \boldsymbol{x}-\boldsymbol{y}\rangle = \|\boldsymbol{x}-\boldsymbol{y}\|$, we obtain

$$0 \ge \|\boldsymbol{x}-\boldsymbol{y}\| + \langle \boldsymbol{g}, \boldsymbol{u}-\boldsymbol{x}\rangle = \langle \boldsymbol{g}, \boldsymbol{x}-\boldsymbol{y}\rangle + \langle \boldsymbol{g}, \boldsymbol{u}-\boldsymbol{x}\rangle = \langle \boldsymbol{g}, \boldsymbol{u}-\boldsymbol{y}\rangle\,.$$

Hence, $\langle \boldsymbol{g}, \boldsymbol{u}-\boldsymbol{y}\rangle \le 0$, for all $\boldsymbol{u} \in \mathcal{V}$.

Conversely, let $\boldsymbol{g}$ satisfy

$$\|\boldsymbol{g}\|_\star = 1, \qquad \langle \boldsymbol{g}, \boldsymbol{x}-\boldsymbol{y}\rangle = \|\boldsymbol{x}-\boldsymbol{y}\|, \qquad \langle \boldsymbol{g}, \boldsymbol{u}-\boldsymbol{y}\rangle \le 0 \quad \forall \boldsymbol{u} \in \mathcal{V}\,.$$

Fix any $\boldsymbol{z} \in \mathbb{R}^d$ and any $\boldsymbol{u} \in \mathcal{V}$. Since $\|\boldsymbol{g}\|_\star = 1$,

$$\|\boldsymbol{z}-\boldsymbol{u}\| \ge \langle \boldsymbol{g}, \boldsymbol{z}-\boldsymbol{u}\rangle = \langle \boldsymbol{g}, \boldsymbol{z}-\boldsymbol{x}\rangle + \langle \boldsymbol{g}, \boldsymbol{x}-\boldsymbol{y}\rangle + \langle \boldsymbol{g}, \boldsymbol{y}-\boldsymbol{u}\rangle\,.$$

Using $\langle \boldsymbol{g}, \boldsymbol{x}-\boldsymbol{y}\rangle = \|\boldsymbol{x}-\boldsymbol{y}\| = \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x})$ and $\langle \boldsymbol{g}, \boldsymbol{y}-\boldsymbol{u}\rangle = -\langle \boldsymbol{g}, \boldsymbol{u}-\boldsymbol{y}\rangle \ge 0$, we obtain

$$\|\boldsymbol{z}-\boldsymbol{u}\| \ge \langle \boldsymbol{g}, \boldsymbol{z}-\boldsymbol{x}\rangle + \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x})\,.$$

Since this holds for every $\boldsymbol{u} \in \mathcal{V}$, minimizing over $\boldsymbol{u} \in \mathcal{V}$ gives

$$\operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{z}) \geq \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x}) + \langle \boldsymbol{g}, \boldsymbol{z} - \boldsymbol{x} \rangle \,.$$

Hence $\boldsymbol{g} \in \partial \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x})$. □

Example 12.3 *If we consider the L_2 norm, for all $\boldsymbol{x} \notin \mathcal{V}$, we have*

$$\partial \operatorname{dist}_{\mathcal{V},\|\cdot\|_2}(\boldsymbol{x}) = \left\{ \frac{\boldsymbol{x} - \Pi_{\mathcal{V}}(\boldsymbol{x})}{\|\boldsymbol{x} - \Pi_{\mathcal{V}}(\boldsymbol{x})\|_2} \right\},$$

where $\Pi_{\mathcal{V}}$ is the standard Euclidean projection.

Example 12.4 *Note that the projection with respect to a norm different from L_2 does not have to be unique. For example, we consider the L_1 norm and $\mathcal{V} = \Delta^{d-1}$ and we have that*

$$\Pi_{\mathcal{V},\|\cdot\|_1}(\boldsymbol{x}) = \operatorname*{argmin}_{\boldsymbol{y} \in \Delta^{d-1}} \|\boldsymbol{x} - \boldsymbol{y}\|_1 \,.$$

From this, we can see that the generalized projection is not unique. In fact, if $x_i > 1$ for all $i = 1, \ldots, d$, then $\operatorname{argmin}_{\boldsymbol{y} \in \Delta^{d-1}} \|\boldsymbol{x} - \boldsymbol{y}\|_1 = \Delta^{d-1}$. Indeed, we have

$$\operatorname*{argmin}_{\boldsymbol{y} \in \Delta^{d-1}} \|\boldsymbol{x} - \boldsymbol{y}\|_1 = \operatorname*{argmin}_{\boldsymbol{y} \in \Delta^{d-1}} \sum_{i=1}^{d} |x_i - y_i| = \operatorname*{argmin}_{\boldsymbol{y} \in \Delta^{d-1}} \sum_{i=1}^{d} (x_i - y_i) = \operatorname*{argmin}_{\boldsymbol{y} \in \Delta^{d-1}} \sum_{i=1}^{d} x_i - 1,$$

that is constant with respect to $\boldsymbol{y}$. Similarly, if $x_i \leq 0$ for all $i = 1, \ldots, d$, then we have $\operatorname{argmin}_{\boldsymbol{y} \in \Delta^{d-1}} \|\boldsymbol{x} - \boldsymbol{y}\|_1 = \Delta^{d-1}$ because

$$\operatorname*{argmin}_{\boldsymbol{y} \in \Delta^{d-1}} \|\boldsymbol{x} - \boldsymbol{y}\|_1 = \operatorname*{argmin}_{\boldsymbol{y} \in \Delta^{d-1}} \sum_{i=1}^{d} |x_i - y_i| = \operatorname*{argmin}_{\boldsymbol{y} \in \Delta^{d-1}} \sum_{i=1}^{d} (y_i - x_i) = \operatorname*{argmin}_{\boldsymbol{y} \in \Delta^{d-1}} 1 - \sum_{i=1}^{d} x_i \,.$$

Denoting by $\phi(\boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|_1$, from the optimality condition, we have that $\boldsymbol{z} = \Pi_{\Delta^{d-1},\|\cdot\|_1}(\boldsymbol{x})$ iff $\mathbf{0} \in \mathcal{N}_{\Delta^{d-1}}(\boldsymbol{z}) + \partial\phi(\boldsymbol{z})$, where $\mathcal{N}_{\Delta^{d-1}}(\boldsymbol{z})$ is the normal cone to Δ^{d-1} at $\boldsymbol{z}$ (see Example 2.25), that is

$$\mathcal{N}_{\Delta^{d-1}}(\boldsymbol{z}) = \{\alpha \mathbf{1}_d + \sum_{i:z_i=0} \beta_i \boldsymbol{e}_i : \alpha \in \mathbb{R}, \beta_i \leq 0\},$$

where $\mathbf{1}_d$ is a vector in $\mathbb{R}^d$ composed of all ones. This implies that if $\mathcal{S}_1(\boldsymbol{x}) = \{\boldsymbol{y} \in \Delta^{d-1} : y_i \geq x_i\}$ and $\mathcal{S}_2(\boldsymbol{x}) = \{\boldsymbol{y} \in \Delta^{d-1} : y_i \leq x_i\}$ are non-empty, then they contain solutions of the generalized projection. In particular, if $\|\boldsymbol{x}\|_1 < 1$ then $\mathcal{S}_1(\boldsymbol{x}) \neq \emptyset$ and if $\|\boldsymbol{x}\|_1 > 1$ and $x_i \geq 0$ then $\mathcal{S}_2(\boldsymbol{x}) \neq \emptyset$. This implies that if $\boldsymbol{x} \in \mathbb{R}^d_{\geq 0}$ then, for example, $\frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_1} \in \Pi_{\Delta^{d-1},\|\cdot\|_1}(\boldsymbol{x})$ when $\|\boldsymbol{x}\|_1 > 0$ and $\Pi_{\Delta^{d-1},\|\cdot\|_1}(\boldsymbol{x}) = \Delta^{d-1}$ if $\|\boldsymbol{x}\|_1 = 0$. Moreover, we can also calculate a subgradient $\boldsymbol{q}$ of $\operatorname{dist}_{\Delta^{d-1},\|\cdot\|_1}(\boldsymbol{x})$ for $\boldsymbol{x} \notin \Delta^{d-1}$ as $-\mathbf{1}_d$ if $\Pi_{\Delta^{d-1},\|\cdot\|_1}(\boldsymbol{x}) \in \mathcal{S}_1(\boldsymbol{x})$ and $\mathbf{1}_d$ if $\Pi_{\Delta^{d-1},\|\cdot\|_1}(\boldsymbol{x}) \in \mathcal{S}_2(\boldsymbol{x})$.

We can now state the main theorem.

Theorem 12.5 *Denote by $\operatorname{Regret}_T^{\mathscr{A}}(\boldsymbol{u})$ the regret of an OCO algorithm $\mathscr{A}$ with feasible set $\mathcal{W} \subseteq \mathbb{R}^d$. Let $\mathcal{V} \subset \mathcal{W}$ be non-empty, convex, and closed, and $\boldsymbol{q}_t \in \partial \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{z}_t)$. Set in Algorithm 12.1 $\Pi_{\mathcal{V}} = \Pi_{\mathcal{V},\|\cdot\|}$ and one of the following choices of surrogate losses $\tilde{\ell}_t : \mathcal{W} \to \mathbb{R}$:*

Algorithm 12.1 Constrained OCO through (Non-Euclidean) Projections

Require: Non-empty closed convex set $\mathcal{V} \subset \mathbb{R}^d$, OCO algorithm $\mathscr{A}$ with feasible set $\mathcal{W} \supset \mathcal{V}$, generic projection operator $\Pi_{\mathcal{V}}$, a way to construct surrogate losses $\tilde{\ell}_t$

1: **for** $t = 1$ **to** T **do**
2: Get $\boldsymbol{z}_t \in \mathcal{W}$ from $\mathscr{A}$
3: Output $\boldsymbol{x}_t \in \Pi_{\mathcal{V}}(\boldsymbol{z}_t)$
4: Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable on $\mathcal{V}$
5: Set $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$
6: Send the surrogate loss $\tilde{\ell}_t : \mathcal{W} \to \mathbb{R}$ to $\mathscr{A}$
7: **end for**

- $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle + \|\boldsymbol{g}_t\|_\star \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x})$
- $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t + \|\boldsymbol{g}_t\|_\star \boldsymbol{q}_t, \boldsymbol{x} \rangle$;
- $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle + \begin{cases} 0, & \text{if } \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t \rangle \leq 0 \\ \langle \boldsymbol{g}_t, \frac{\boldsymbol{x}_t - \boldsymbol{z}_t}{\|\boldsymbol{x}_t - \boldsymbol{z}_t\|} \rangle \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{x}), & \text{if } \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t \rangle > 0 \end{cases}$
- $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle + \begin{cases} 0, & \text{if } \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t \rangle \leq 0 \\ \langle \boldsymbol{g}_t, \frac{\boldsymbol{x}_t - \boldsymbol{z}_t}{\|\boldsymbol{x}_t - \boldsymbol{z}_t\|} \rangle \langle \boldsymbol{q}_t, \boldsymbol{x} \rangle, & \text{if } \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t \rangle > 0 \end{cases}$

Then, Algorithm 12.1 guarantees

$$\operatorname{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^T (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \operatorname{Regret}_T^{\mathscr{A}}(\boldsymbol{u}) = \sum_{t=1}^T (\tilde{\ell}_t(\boldsymbol{z}_t) - \tilde{\ell}_t(\boldsymbol{u})), \quad \forall \boldsymbol{u} \in \mathcal{V} .$$

Moreover, for any $\boldsymbol{x} \in \mathcal{W}$ and any $\tilde{\boldsymbol{g}}_t \in \partial \tilde{\ell}_t(\boldsymbol{x})$, we have $\|\tilde{\boldsymbol{g}}_t\|_\star \leq 2\|\boldsymbol{g}_t\|_\star$.

Proof Note that the second and fourth surrogates are just linearizations of the first and third surrogate losses at $\boldsymbol{z}_t$, respectively. Hence, it is enough to prove the regret guarantee for the first and third surrogate losses.

For all of them, we start by observing that

$$\begin{aligned} \operatorname{Regret}_T(\boldsymbol{u}) &= \sum_{t=1}^T (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \\ &= \sum_{t=1}^T (\langle \boldsymbol{g}_t, \boldsymbol{z}_t - \boldsymbol{u} \rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t \rangle) . \end{aligned} \tag{12.2}$$

For the first surrogate loss, we have

$$\begin{aligned}\sum_{t=1}^{T}(\langle \boldsymbol{g}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle) &\le \sum_{t=1}^{T}(\langle \boldsymbol{g}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle + \|\boldsymbol{g}_t\|_\star \|\boldsymbol{z}_t - \boldsymbol{x}_t\|) \\ &= \sum_{t=1}^{T}(\langle \boldsymbol{g}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle + \|\boldsymbol{g}_t\|_\star \min_{\boldsymbol{y}\in\mathcal{V}} \|\boldsymbol{z}_t - \boldsymbol{y}\|) \\ &= \sum_{t=1}^{T}(\tilde{\ell}_t(\boldsymbol{z}_t) - \tilde{\ell}_t(\boldsymbol{u}))\,.\end{aligned}$$

For the third surrogate loss, first of all, observe that this loss is convex because $\operatorname{dist}_{\mathcal{V},\|\cdot\|}$ is multiplied by a non-negative number. In the case that $\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle \le 0$, the second term in (12.2) is negative, so we can drop it. For the case that $\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle > 0$, we have

$$\begin{aligned}\langle \boldsymbol{g}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle &= \langle \boldsymbol{g}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle + \left\langle \boldsymbol{g}_t, \frac{\boldsymbol{x}_t - \boldsymbol{z}_t}{\|\boldsymbol{x}_t - \boldsymbol{z}_t\|}\right\rangle \|\boldsymbol{x}_t - \boldsymbol{z}_t\| \\ &= \langle \boldsymbol{g}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle + \left\langle \boldsymbol{g}_t, \frac{\boldsymbol{x}_t - \boldsymbol{z}_t}{\|\boldsymbol{x}_t - \boldsymbol{z}_t\|}\right\rangle \operatorname{dist}_{\mathcal{V},\|\cdot\|}(\boldsymbol{z}_t) \\ &= \tilde{\ell}_t(\boldsymbol{z}_t) - \tilde{\ell}_t(\boldsymbol{u})\,.\end{aligned}$$

Finally, the same norm bound holds for all the surrogate subgradients. For the first and third losses, this follows from the fact that every subgradient of $\operatorname{dist}_{\mathcal{V},\|\cdot\|}$ has dual norm at most 1, while the scalar multiplying the distance term is at most $\|\boldsymbol{g}_t\|_\star$. For the second and fourth losses, it follows from $\|\boldsymbol{q}_t\|_\star \le 1$. □

Remark 12.6 *In the case that $\|\cdot\| = \|\cdot\|_2$, we can sharpen the bound on the norm of $\tilde{\boldsymbol{g}}_t$ for the third and fourth loss. In fact, we have $\boldsymbol{q}_t = \frac{\boldsymbol{z}_t - \boldsymbol{x}_t}{\|\boldsymbol{z}_t - \boldsymbol{x}_t\|_2}$ for $\boldsymbol{z}_t \neq \boldsymbol{x}_t$ and $\boldsymbol{0}$ otherwise. So, we have*

$$\|\tilde{\boldsymbol{g}}_t\|_2^2 = \|\boldsymbol{g}_t\|_2^2 - \left(\left\langle \boldsymbol{g}_t, \frac{\boldsymbol{x}_t - \boldsymbol{z}_t}{\|\boldsymbol{x}_t - \boldsymbol{z}_t\|_2}\right\rangle\right)^2 \le \|\boldsymbol{g}_t\|_2^2\,.$$

Remark 12.7 *The first surrogate loss has a very natural interpretation: it is a Lipschitz penalty function that we add to the linearized original losses, where the Lipschitz constant is the same as that of the original loss. Hence, we essentially penalize the algorithm for predicting a point outside of the feasible set.*

Remark 12.8 *In the case that the loss functions ℓ_t satisfy a weaker form of strong convexity, we can show that the black-box reduction will preserve it. In particular, consider the case that we use an online learning algorithm with feasible set $\mathcal{W}$ with bounded diameter D with respect to $\|\cdot\|_2$, the losses ℓ_t are L-Lipschitz with respect to $\|\cdot\|_2$ on $\mathcal{V}$ and they also satisfy*

$$\ell_t(\boldsymbol{x}) - \ell_t(\boldsymbol{u}) \le \langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{u}\rangle - \frac{\beta}{2}\langle \boldsymbol{g}, \boldsymbol{x} - \boldsymbol{u}\rangle^2, \quad \forall \boldsymbol{u}, \boldsymbol{x} \in \mathcal{V},\ \forall \boldsymbol{g} \in \partial\ell_t(\boldsymbol{x}),$$

where $\beta \le \frac{1}{DL}$. For example, we saw that this is true if the functions ℓ_t are exp-concave on $\mathcal{V}$. Now, use projections with respect to $\|\cdot\|_2$ onto $\mathcal{V}$, and the fourth surrogate loss in Theorem 12.5, so we have $\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle \le \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle$, for all $\boldsymbol{u} \in \mathcal{V}$, and $\|\tilde{\boldsymbol{g}}_t\|_2 \le$

$\|\boldsymbol{g}_t\|_2 \le L$. Hence, we have $\langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle \le \|\tilde{\boldsymbol{g}}_t\|_2 \|\boldsymbol{z}_t - \boldsymbol{u}\|_2 \le LD \le \frac{1}{\beta}$ for all $\boldsymbol{u} \in \mathcal{V}$, which implies

$$\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u}) \le \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle - \frac{\beta}{2}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle^2 \le \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle - \frac{\beta}{2}\langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle^2, \quad \forall \boldsymbol{u} \in \mathcal{V},$$

where the second inequality is due to the fact that $x \mapsto x - \frac{\beta}{2}x^2$ is increasing for $x \le \frac{1}{\beta}$.

Example 12.9 *Let's start with a simple example: we have an online algorithm that works in $\mathbb{R}$, and we want to constrain it to $\mathbb{R}_{\ge 0}$. In this case, the choice of the norms does not really matter, so we can choose L_2. The projection is simply $x_t = \max(z_t, 0)$. The subgradient q_t of the distance function can be chosen as*

$$q_t = \begin{cases} -1, & z_t < 0 \\ 0, & z_t \ge 0 \end{cases}$$

So, using the fourth surrogate loss function in Theorem 12.5, the surrogate linear losses are $\tilde{\ell}_t(x) = \tilde{g}_t x$ where

$$\tilde{g}_t = \begin{cases} \min(g_t, 0), & z_t < 0 \\ g_t, & z_t \ge 0 \end{cases}$$

Example 12.10 (Regret Matching+) *Consider projected Online Gradient Descent (OGD) with learning rate $\eta = \frac{\alpha}{\sqrt{T}}$, over the feasible set $\mathcal{W} = \mathbb{R}^d_{\ge 0}$ and with initial prediction $\boldsymbol{z}_1 = \boldsymbol{0}$. Its regret upper bound is*

$$\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle \le \frac{1}{2}\left(\frac{\|\boldsymbol{u}\|_2^2}{\alpha} + \alpha \max_t \|\boldsymbol{g}_t\|_2^2\right)\sqrt{T}, \quad \forall \boldsymbol{u} \in \mathbb{R}^d_{\ge 0}\,.$$

From Example 12.4, we can project onto $\mathcal{V} = \Delta^{d-1}$ with respect to $\|\cdot\|_1$, using

$$\boldsymbol{x}_t = \begin{cases} \frac{\boldsymbol{z}_t}{\|\boldsymbol{z}_t\|_1}, & \boldsymbol{z}_t \ne \boldsymbol{0} \\ [1/d, \dots, 1/d]^\top, & \boldsymbol{z}_t = \boldsymbol{0} \end{cases}.$$

Then, using any of the above surrogate losses in Algorithm 12.1, we have

$$\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle \le \frac{1}{2}\left(\frac{\|\boldsymbol{u}\|_2^2}{\alpha} + \alpha \max_t \|\tilde{\boldsymbol{g}}_t\|_2^2\right)\sqrt{T}, \quad \forall \boldsymbol{u} \in \Delta^{d-1},$$

and $\|\tilde{\boldsymbol{g}}_t\|_2 \le \sqrt{d}\|\tilde{\boldsymbol{g}}_t\|_\infty \le 2\sqrt{d}\|\boldsymbol{g}_t\|_\infty$. Choosing α proportional to $1/\sqrt{d}$ gives an upper bound proportional to $\sqrt{dT}$, which is the expected one when using an algorithm with a Euclidean geometry.

However, we can do even better! Choose the specific surrogate losses in (12.1) *with the projection $\frac{\boldsymbol{z}_t}{\|\boldsymbol{z}_t\|_1}$, then it should be easy to realize that $\boldsymbol{x}_t$ is independent of $\eta > 0$ (left as an exercise for the reader). This means that we can set $\eta = 1$, but the regret guarantee of the*

resulting algorithm will be the one corresponding to the best $\eta > 0$ in hindsight:

$$\begin{aligned}\sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle &\le \min_{\eta>0}\ \frac{\|\boldsymbol{u}\|_2^2}{2\eta} + \frac{\eta}{2}\sum_{t=1}^{T}\|\boldsymbol{g}_t - \mathbf{1}_d\langle \boldsymbol{g}_t, \boldsymbol{x}_t\rangle\|_2^2 \\ &= \|\boldsymbol{u}\|_2\sqrt{\sum_{t=1}^{T}\|\boldsymbol{g}_t - \mathbf{1}_d\langle \boldsymbol{g}_t, \boldsymbol{x}_t\rangle\|_2^2} \\ &\le 2\sqrt{dT}\max_t \|\boldsymbol{g}_t\|_\infty, \quad \forall \boldsymbol{u} \in \Delta^{d-1}\,.\end{aligned}$$

The resulting algorithm is known as Regret Matching+ and, despite the suboptimal dependence on d, this is one of the best-performing algorithms in practice for the setting of Learning with Expert Advice (LEA). If instead of projected OGD we use Follow-the-Regularized-Leader (FTRL) with a fixed squared L_2 regularizer and feasible set $\mathcal{V} = \mathbb{R}^d_{\ge 0}$, we obtain the original Regret Matching algorithm, see Problem 12.2.

12.1.2 Dealing with Constraints using Bregman Projections

Now, we show an alternative approach based on Bregman projections.

Theorem 12.11 *With the notation in Algorithm 12.1, assume $\mathcal{V} \subset \mathcal{W} \subset \operatorname{int}\mathcal{X}$, where $\mathcal{V}$ and $\mathcal{W}$ are nonempty, closed, and convex. Let $\psi : \mathcal{X} \to \mathbb{R}$ be μ-strongly convex with respect to $\|\cdot\|$ and β-smooth with respect to $\|\cdot\|$ on $\operatorname{int}\mathcal{X}$. Also, set $\Pi_{\mathcal{V}}$ to the Bregman projection $\Pi_{\mathcal{V},B_\psi}$, and the surrogate losses*

$$\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle + \begin{cases} 0, & \text{if } \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle \le 0 \\ \frac{\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle}{\mu\|\boldsymbol{x}_t - \boldsymbol{z}_t\|^2}\langle \nabla\psi(\boldsymbol{z}_t) - \nabla\psi(\boldsymbol{x}_t), \boldsymbol{x}\rangle, & \text{if } \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle > 0 \end{cases}.$$

Then, we have

$$\operatorname{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \le \operatorname{Regret}_T^{\mathscr{A}}(\boldsymbol{u}), \quad \forall \boldsymbol{u} \in \mathcal{V}\,.$$

Moreover, the surrogate losses are linear, with gradients $\tilde{\boldsymbol{g}}_t$ satisfying $\|\tilde{\boldsymbol{g}}_t\|_\star \le (1 + \frac{\beta}{\mu})\|\boldsymbol{g}_t\|_\star$.

Proof Once again, we have

$$\begin{aligned}\operatorname{Regret}_T(\boldsymbol{u}) &= \sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \le \sum_{t=1}^{T}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle \\ &= \sum_{t=1}^{T}(\langle \boldsymbol{g}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle + \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle)\,.\end{aligned}$$

If $\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle \le 0$, we can just ignore this term. On the other hand, if $\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle > 0$,

we have

$$\begin{aligned}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle &\le \frac{\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle}{\mu\|\boldsymbol{x}_t - \boldsymbol{z}_t\|^2}\mu\|\boldsymbol{x}_t - \boldsymbol{z}_t\|^2 \\ &\le \frac{\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle}{\mu\|\boldsymbol{x}_t - \boldsymbol{z}_t\|^2}\langle \nabla\psi(\boldsymbol{z}_t) - \nabla\psi(\boldsymbol{x}_t), \boldsymbol{z}_t - \boldsymbol{x}_t\rangle,\end{aligned}$$

where in the second inequality we used the strong convexity of ψ. Hence, we have

$$\begin{aligned}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle &\le \langle \boldsymbol{g}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle + \frac{\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle}{\mu\|\boldsymbol{x}_t - \boldsymbol{z}_t\|^2}\langle \nabla\psi(\boldsymbol{z}_t) - \nabla\psi(\boldsymbol{x}_t), \boldsymbol{z}_t - \boldsymbol{x}_t\rangle \\ &\le \left\langle \boldsymbol{g}_t + \frac{\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle}{\mu\|\boldsymbol{x}_t - \boldsymbol{z}_t\|^2}(\nabla\psi(\boldsymbol{z}_t) - \nabla\psi(\boldsymbol{x}_t)), \boldsymbol{z}_t - \boldsymbol{u}\right\rangle,\end{aligned}$$

where in the second inequality we used the first-order optimality condition

$$\langle \nabla\psi(\boldsymbol{z}_t) - \nabla\psi(\boldsymbol{x}_t), \boldsymbol{u} - \boldsymbol{x}_t\rangle \le 0, \quad \forall \boldsymbol{u} \in \mathcal{V}\,.$$

For the bound on $\|\tilde{\boldsymbol{g}}_t\|_\star$, observe that when $\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle > 0$ we have

$$\begin{aligned}\|\tilde{\boldsymbol{g}}_t\|_\star &= \left\|\boldsymbol{g}_t + \frac{\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle}{\mu\|\boldsymbol{x}_t - \boldsymbol{z}_t\|^2}(\nabla\psi(\boldsymbol{z}_t) - \nabla\psi(\boldsymbol{x}_t))\right\|_\star \\ &\le \|\boldsymbol{g}_t\|_\star + \frac{\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle}{\mu\|\boldsymbol{x}_t - \boldsymbol{z}_t\|^2}\|\nabla\psi(\boldsymbol{z}_t) - \nabla\psi(\boldsymbol{x}_t)\|_\star \\ &\le \|\boldsymbol{g}_t\|_\star + \frac{\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle}{\mu\|\boldsymbol{x}_t - \boldsymbol{z}_t\|^2}\beta\|\boldsymbol{z}_t - \boldsymbol{x}_t\| \le \left(1 + \frac{\beta}{\mu}\right)\|\boldsymbol{g}_t\|_\star\,.\end{aligned}$$

The same bound clearly holds even in the case that $\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{z}_t\rangle \le 0$. □

Example 12.12 *Let's calculate* $\Pi_{\Delta^{d-1}, B_\psi}(\boldsymbol{x})$ *where* $\boldsymbol{x} \in \mathbb{R}^d_{>0}$, $\psi(\boldsymbol{x}) = \sum_{i=1}^d x_i \ln x_i$, *and we define* $0 \ln 0 = 0$. *Hence, we have to solve the following optimization problem*

$$\operatorname*{argmin}_{\boldsymbol{y}\in\Delta^{d-1}} \sum_{i=1}^d y_i \ln \frac{y_i}{x_i}\,.$$

We can rewrite it as an unconstrained optimization over $d-1$ *variables, as*

$$\operatorname*{argmin}_{\boldsymbol{y}\in\mathbb{R}^{d-1}_{\ge 0}} \sum_{i=1}^{d-1} y_i \ln \frac{y_i}{x_i} + \left(1 - \sum_{i=1}^{d-1} y_i\right) \ln \frac{1 - \sum_{i=1}^{d-1} y_i}{x_d}\,.$$

Since $x_i > 0$ *for all* i, *the optimal solution has* $y_1, \dots, y_{d-1}$ *and* $1 - \sum_{i=1}^{d-1} y_i$ *strictly positive. Hence, the inequality constraints are inactive, and we can use the first-order conditions:*

$$\ln \frac{y_i}{x_i} + 1 - \ln \frac{1 - \sum_{j=1}^{d-1} y_j}{x_d} - 1 = 0, \quad i = 1, \dots, d-1,$$

that gives

$$y_i = x_i \frac{1 - \sum_{j=1}^{d-1} y_j}{x_d}, \quad i = 1, \dots, d-1\,. \tag{12.3}$$

Summing over $i = 1, \dots, d-1$, *we have* $\sum_{i=1}^{d-1} y_i = \frac{1-\sum_{i=1}^{d-1} y_i}{x_d} \sum_{i=1}^{d-1} x_i$. *Solving it, we have* $\frac{1-\sum_{i=1}^{d-1} y_i}{x_d} = \frac{1}{\sum_{i=1}^{d} x_i}$ *and using it in* (12.3) *gives* $y_i = \frac{x_i}{\sum_{j=1}^{d} x_j}$, *for* $i = 1, \dots, d$.

12.2 Sleeping Experts

Consider now the setting of LEA where only a subset of the experts is active in each round. In particular, we have that $a_{t,i} = 1$ if expert i is active at time t, and $a_{t,i} = 0$ if the expert is inactive, that is, *sleeping*. This setting is useful in the case that some experts might become unavailable in some rounds, but also to model the case that the set of experts is growing over time.

The online algorithm receives at the beginning of each round the information about which experts will be active, and it outputs a distribution over the active experts. So, we constrain the LEA algorithm to produce a probability distribution only over the active experts. We also assume that there is always at least one active expert.

Here, we will use a different notion of regret, in particular, our regret with respect to any expert i is defined as

$$\mathrm{Regret}_T^{\text{sleeping}}(\boldsymbol{e}_i) := \sum_{t=1}^{T} a_{t,i}(\langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - g_{t,i}) \; .$$

In words, we measure the regret against expert i only on rounds where the expert was active. This notion can be generalized to $\boldsymbol{u} \in \mathbb{R}^d_{\geq 0}$ as

$$\mathrm{Regret}_T^{\text{sleeping}}(\boldsymbol{u}) := \sum_{t=1}^{T} \sum_{i=1}^{d} u_i a_{t,i}(\langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - g_{t,i}), \quad \forall \boldsymbol{u} \in \mathbb{R}^d_{\geq 0} \; .$$

If $\sum_i u_i a_{t,i} \leq 1$ for all t, then $\boldsymbol{u}$ can be interpreted as a fixed sleeping comparator that assigns total mass at most one to the active experts on each round. In the case that all the experts are active on all rounds and $\boldsymbol{u} \in \Delta^{d-1}$, this notion recovers the usual one. However, in the general case, this is a different notion than the one we used in Online Linear Optimization (OLO).

The reduction we consider transforms an OLO learner over $\mathcal{V} = \mathbb{R}^d_{\geq 0}$ to a sleeping expert algorithm. Hence, we will transform a vector $\boldsymbol{z}_t \in \mathbb{R}^d_{\geq 0}$ into a vector in the simplex defined by the active experts, and we also need appropriate surrogate losses. The reduction will be similar to the one we used in the previous section. For the first part, we construct a probability distribution as

$$x_{t,i} = \frac{a_{t,i} z_{t,i}}{\sum_{j=1}^{d} a_{t,j} z_{t,j}}, \quad \forall i = 1, \dots, d,$$

when $\sum_{j=1}^{d} a_{t,j} z_{t,j} \neq 0$ and an arbitrary one otherwise. For the second part, from the original linear losses $\ell_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$ we construct the modified losses $\tilde{\ell}_t(\boldsymbol{x}) = \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x} \rangle$, where $\tilde{g}_{t,i} = a_{t,i}(g_{t,i} - \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle)$. The overall algorithm is in Algorithm 12.2.

Algorithm 12.2 Sleeping Experts Reduction

Require: Online linear optimization algorithm $\mathscr{A}$ with feasible set $\mathcal{V} = \mathbb{R}^d_{\geq 0}$

for $t = 1$ **to** T **do**
  Get $\boldsymbol{z}_t \in \mathbb{R}^d_{\geq 0}$ from $\mathscr{A}$
  Receive the state of the experts $a_{t,i} \in \{0, 1\}, i = 1, \ldots, d$
  if $\sum_{j=1}^d a_{t,j} z_{t,j} \neq 0$ **then**
    Output $x_{t,i} = \frac{a_{t,i} z_{t,i}}{\sum_{j=1}^d a_{t,j} z_{t,j}},\ i = 1, \ldots, d$
  else
    Output an arbitrary distribution $\boldsymbol{x}_t$ over the active experts
  end if
  Receive $\boldsymbol{g}_t \in \mathbb{R}^d$
  Send the surrogate loss $\tilde{\ell}_t(\boldsymbol{x}) = \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{x} \rangle$ to $\mathscr{A}$, where $\tilde{g}_{t,i} = a_{t,i}(g_{t,i} - \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle),\ i = 1, \ldots, d$
end for

With the above definitions, we have

$$\langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t \rangle = \sum_{i=1}^d a_{t,i}(g_{t,i} - \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle) z_{t,i} = \sum_{i=1}^d a_{t,i} z_{t,i} g_{t,i} - \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \sum_{i=1}^d a_{t,i} z_{t,i} \,.$$

If $\sum_{i=1}^d a_{t,i} z_{t,i} = 0$, then, since $a_{t,i} \in \{0, 1\}$ and $z_{t,i} \geq 0$, we must have $a_{t,i} z_{t,i} = 0$, for all $i = 1, \ldots, d$. Therefore, $\langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t \rangle = 0$.

On the other hand, if $\sum_{i=1}^d a_{t,i} z_{t,i} \neq 0$, then by the definition of $\boldsymbol{x}_t$ we have

$$\sum_{i=1}^d a_{t,i} z_{t,i} g_{t,i} = \left(\sum_{j=1}^d a_{t,j} z_{t,j} \right) \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \,.$$

Substituting this into the previous expression gives $\langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t \rangle = 0$.

Hence, in all cases, $\langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t \rangle = 0$. In turn, this implies that

$$a_{t,i}(\langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - g_{t,i}) = -\tilde{g}_{t,i} = \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t \rangle - \tilde{g}_{t,i}, \quad i = 1, \ldots, d \,.$$

In words, we can construct surrogate losses to transform the sleeping expert problem into an OLO problem over $\mathbb{R}^d_{\geq 0}$, obtaining that

$$\mathrm{Regret}_T^{\text{sleeping}}(\boldsymbol{u}) = \sum_{t=1}^T \sum_{i=1}^d u_i a_{t,i}(\langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - g_{t,i}) = \sum_{t=1}^T \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t - \boldsymbol{u} \rangle, \quad \forall \boldsymbol{u} \in \mathbb{R}^d_{\geq 0} \,.$$

Formally, we have the following theorem.

Theorem 12.13 *Let $a_{t,i} \in \{0, 1\}$ denote whether expert i is active at time t, and assume that at least one expert is active on every round. Run Algorithm 12.2 with an OLO algorithm $\mathscr{A}$ over $\mathbb{R}^d_{\geq 0}$. If $\mathscr{A}$ guarantees*

$$\sum_{t=1}^T \langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t - \boldsymbol{u} \rangle \leq R_T^{\mathscr{A}}(\boldsymbol{u}), \quad \forall \boldsymbol{u} \in \mathbb{R}^d_{\geq 0},$$

Algorithm 12.3 Learning Magnitude and Direction Separately

Require: One-dimensional online learning algorithm $\mathscr{A}_{\text{1d}}$, Online learning algorithm $\mathscr{A}_B$ with feasible set equal to the unit ball $\mathcal{B} \subset \mathbb{R}^d$ with respect to $\|\cdot\|$

for $t = 1$ **to** T **do**
  Get point $z_t \in \mathbb{R}$ from $\mathscr{A}_{\text{1d}}$, and get point $\tilde{\boldsymbol{x}}_t \in \mathcal{B}$ from $\mathscr{A}_B$
  Play $\boldsymbol{x}_t = z_t \tilde{\boldsymbol{x}}_t \in \mathbb{R}^d$
  Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable on $\mathbb{R}^d$
  Set $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$ and $s_t = \langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_t \rangle$
  Send $\ell_t^{\mathscr{A}_{\text{1d}}}(x) = s_t x$ as the t-th linear loss to $\mathscr{A}_{\text{1d}}$
  Send $\ell_t^{\mathscr{A}_B}(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$ as the t-th linear loss to $\mathscr{A}_B$
end for

then Algorithm 12.2 satisfies

$$\text{Regret}_T^{\textit{sleeping}}(\boldsymbol{u}) = \sum_{t=1}^{T} \sum_{i=1}^{d} u_i a_{t,i} (\langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - g_{t,i}) \leq R_T^{\mathscr{A}}(\boldsymbol{u}), \quad \forall \boldsymbol{u} \in \mathbb{R}^d_{\geq 0} \,.$$

Moreover, if the active losses have range at most R on every round, that is, $\max_{i:a_{t,i}=1} g_{t,i} - \min_{i:a_{t,i}=1} g_{t,i} \leq R$, then $\|\tilde{\boldsymbol{g}}_t\|_\infty \leq R$ for all $t = 1, \dots, T$.

Remark 12.14 *Clearly, if we have an algorithm for LEA, we can use it in the reduction because $\Delta^{d-1} \subset \mathbb{R}^d_{\geq 0}$.*

Remark 12.15 *The above definitions and reduction generalize to the setting where $a_{t,i} \in [0, 1]$, see Problem 12.3. Here, $a_{t,i}$ denotes the confidence that expert i has in its prediction at time t, where 0 means that the expert abstains from producing a prediction.*

12.3 Reduction to Magnitude and Direction

Here, we will see how to use a one-dimensional algorithm for unconstrained OCO to solve an OCO problem in any number of dimensions and using any (reasonable) norm.

This reduction requires an unconstrained OCO algorithm for the one-dimensional case and an algorithm for learning in d-dimensional (or infinite-dimensional) balls. Given these two learners, we decompose the problem of learning a vector $\boldsymbol{x}_t$ into the problem of learning a *direction* and a *magnitude*. The regret of this procedure turns out to be just the sum of the regret of the two learners. We can formalize this idea in the following theorem.

Theorem 12.16 *Denote by $\text{Regret}_T^{\mathscr{A}_B}(\boldsymbol{u})$ the linear regret of algorithm $\mathscr{A}_B$ for any $\boldsymbol{u}$ in the unit ball with respect to a norm $\|\cdot\|$, and $\text{Regret}_T^{\mathscr{A}_{1d}}(u)$ the linear regret of algorithm $\mathscr{A}_{1d}$ for any competitor $u \in \mathbb{R}$. Then, for any $\boldsymbol{u} \in \mathbb{R}^d \setminus \{\boldsymbol{0}\}$, Algorithm 12.3 guarantees*

$$\begin{aligned}
\text{Regret}_T(\boldsymbol{u}) &= \sum_{t=1}^{T} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \\
&= \text{Regret}_T^{\mathscr{A}_{1d}}(\|\boldsymbol{u}\|) + \|\boldsymbol{u}\| \, \text{Regret}_T^{\mathscr{A}_B}\left(\frac{\boldsymbol{u}}{\|\boldsymbol{u}\|}\right),
\end{aligned}$$

and $\text{Regret}_T(\mathbf{0}) \le \text{Regret}_T^{\mathscr{A}_{1d}}(0)$. *Further, we have* $|s_t| \le \|\boldsymbol{g}_t\|_\star$ *for all* t.

Proof First, observe that $|s_t| \le \|\boldsymbol{g}_t\|_\star \|\tilde{\boldsymbol{x}}_t\| \le \|\boldsymbol{g}_t\|_\star$ since $\|\tilde{\boldsymbol{x}}_t\| \le 1$ for all t. Now, assuming $\boldsymbol{u} \neq \mathbf{0}$ compute:

$$\begin{aligned}
\text{Regret}_T(\boldsymbol{u}) &= \sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}) \le \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle = \sum_{t=1}^T \langle \boldsymbol{g}_t, z_t\tilde{\boldsymbol{x}}_t\rangle - \langle \boldsymbol{g}_t, \boldsymbol{u}\rangle \\
&= \underbrace{\sum_{t=1}^T \left(\langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_t\rangle z_t - \langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_t\rangle \|\boldsymbol{u}\|\right)}_{\text{linear regret of } \mathscr{A}_{1\text{d}} \text{ at } \|\boldsymbol{u}\| \in \mathbb{R}} + \sum_{t=1}^T \left(\langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_t\rangle \|\boldsymbol{u}\| - \langle \boldsymbol{g}_t, \boldsymbol{u}\rangle\right) \\
&= \text{Regret}_T^{\mathscr{A}_{1\text{d}}}(\|\boldsymbol{u}\|) + \sum_{t=1}^T \left(\langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_t\rangle \|\boldsymbol{u}\| - \langle \boldsymbol{g}_t, \boldsymbol{u}\rangle\right) \\
&= \text{Regret}_T^{\mathscr{A}_{1\text{d}}}(\|\boldsymbol{u}\|) + \|\boldsymbol{u}\| \underbrace{\sum_{t=1}^T \left(\langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_t\rangle - \left\langle \boldsymbol{g}_t, \frac{\boldsymbol{u}}{\|\boldsymbol{u}\|}\right\rangle\right)}_{\text{linear regret of } \mathscr{A}_B \text{ at } \frac{\boldsymbol{u}}{\|\boldsymbol{u}\|} \in \mathbb{R}^d} \\
&= \text{Regret}_T^{\mathscr{A}_{1\text{d}}}(\|\boldsymbol{u}\|) + \|\boldsymbol{u}\| \,\text{Regret}_T^{\mathscr{A}_B}\left(\frac{\boldsymbol{u}}{\|\boldsymbol{u}\|}\right).
\end{aligned}$$

The case $\boldsymbol{u} = \mathbf{0}$ follows similarly. □

Remark 12.17 *Note that the direction vector is not constrained to have norm equal to 1, which would be a non-convex constraint, yet this does not seem to affect the proof.*

We will use this theorem in Section 13.4.

12.4 History Bits

As far as I know, the idea of using a black-box reduction for constrained optimization does not appear in the offline and stochastic optimization literature. The reduction from learning from $\mathbb{R}^d_{\ge 0}$ to Δ^{d-1} is a classic approach hidden in most of the LEA algorithms. Cesa-Bianchi and Lugosi (2006, Section 2.1) describes a similar version but through the formalism of potential functions, which does not result in a black-box reduction. Cutkosky and Orabona (2018) proposed the first reduction from constrained OCO to unconstrained OCO described in Section 12.1.1. However, their characterization of the subgradient of the distance-to-a-set function is wrong, and I fixed it here. Cutkosky (2020a) proposed the third surrogate loss. The second and fourth surrogates are new. Farina et al. (2019b) independently of the above work proved Theorem 12.11. Remark 12.8 is taken from the method in Mhammedi and Gatmiry (2023).

Regret Matching (RM) was proposed by Hart and Mas-Colell (2000) and Regret Matching+ (RM+) by Tammelin et al. (2015). RM was developed to find correlated equilibria in two-player games and is commonly used to minimize regret over the simplex. RM+ is a modification of RM designed to accelerate convergence and used to effectively solve the game of Heads-up Limit Texas Hold'em poker (Bowling et al., 2015). While the exact reason why RM+ has better empirical convergence than any other similar algorithm is unclear,

I suggested on X.com on February 14th, 2020, that it might be due to its scale-freeness.[1] The observation that RM can be expressed as FTRL is due to Nicolò Cesa-Bianchi in an unpublished manuscript titled "The Joys of Regret Matching" dated October 14, 2015 (see Problem 12.2). That result was shared with a number of people (including me) and then later appeared in some papers too, unfortunately, without proper credit. The observation that RM+ can be obtained by projected OGD is by Farina et al. (2021) and independently by Flaspohler et al. (2021). RM and RM+ were extended to p-norms by Cesa-Bianchi and Lugosi (2001, 2003)[2] and D'Orazio (2020), respectively. Tuning p with knowledge of d, one gets the optimal dependence in d but still without a learning rate or a regularizer weight to tune. Later, Flaspohler et al. (2021) rediscovered both of these results.

The setting of sleeping experts has been proposed by Blum (1997) and Freund et al. (1997). There is also a different notion of regret for the sleeping-experts setting based on the best ordering of the experts (Kleinberg et al., 2008, 2010); however, it does not seem well suited to combining different algorithms as experts, as we will see in Section 14.2. The reduction in Section 12.2 is an extension of the one from Gaillard et al. (2014) that was designed to reduce the sleeping experts problem to the LEA problem rather than OLO in $\mathbb{R}^d_{\geq 0}$. I also added the minor improvement of removing the constant term from the surrogate losses. Such reduction is implicitly used by Luo and Schapire (2015) and by Jun et al. (2017a,b).

Theorem 12.16 is from Cutkosky and Orabona (2018). Note that the original theorem is more general because it works even in Banach spaces.

12.5 Exercises

Problem 12.1 *Find a solution to the projection from $\mathbb{R}^d$ to Δ^{d-1} with respect to $\|\cdot\|_1$.*

Problem 12.2 *The Regret Matching algorithm predicts with*

$$x_{t,i} = \frac{\max\left(\sum_{n=1}^{t-1}(g_{n,i} - \langle \boldsymbol{g}_n, \boldsymbol{x}_n\rangle), 0\right)}{\sum_{j=1}^{d}\max\left(\sum_{n=1}^{t-1}(g_{n,j} - \langle \boldsymbol{g}_n, \boldsymbol{x}_n\rangle), 0\right)}, \quad i = 1, \dots, d,$$

if $\sum_{j=1}^{d}\max\left(\sum_{n=1}^{t-1}(g_{n,j} - \langle \boldsymbol{g}_n, \boldsymbol{x}_n\rangle), 0\right) \neq 0$ and with an arbitrary distribution otherwise. Prove that it corresponds to FTRL with a squared L_2 regularizer and $\mathcal{V} = \mathbb{R}^d_{\geq 0}$ with projections onto Δ^{d-1} with respect to $\|\cdot\|_1$. Then, prove a regret upper bound proportional to $\sqrt{dT}$. Hint: see Farina et al. (2021) and Flaspohler et al. (2021).

Problem 12.3 *Extend Theorem 12.13 to the case where $a_{t,i} \in [0, 1]$ is the confidence of expert i at time t. Find a suitable prediction strategy $\boldsymbol{x}_t$ and surrogate losses $\tilde{\boldsymbol{g}}_t$ such that, for every $\boldsymbol{u} \in \mathbb{R}^d_{\geq 0}$, we have*

$$\sum_{t=1}^{T}\sum_{i=1}^{d} u_i a_{t,i}\big(\langle \boldsymbol{g}_t, \boldsymbol{x}_t\rangle - g_{t,i}\big) = \sum_{t=1}^{T}\langle \tilde{\boldsymbol{g}}_t, \boldsymbol{z}_t - \boldsymbol{u}\rangle \,.$$

[1] https://x.com/bremen79/status/1228318344195035142

[2] Note that their proof contains a small error that is fixed in the errata of Cesa-Bianchi and Lugosi (2006).

13

Parameter-free Online Linear Optimization

In the previous sections, we have shown that Online Mirror Descent (OMD) and Follow-the-Regularized-Leader (FTRL) achieve a regret of $\mathcal{O}(\sqrt{T})$ for convex Lipschitz losses. We have also shown that for bounded domains, these bounds are optimal up to constant multiplicative factors. However, in the unbounded case, the bounds we get are suboptimal with respect to the dependence on the competitor.

For example, Online Subgradient Descent (OSD), with $\mathcal{V} = \mathbb{R}^d$ and $\boldsymbol{x}_1 = \mathbf{0}$ over L-Lipschitz losses and learning rate $\eta = \frac{\alpha}{L\sqrt{T}}$, gives the regret guarantee

$$\mathrm{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \frac{\|\boldsymbol{u}\|_2^2}{2\eta} + \frac{\eta L^2 T}{2} = \frac{L\sqrt{T}}{2}\left(\frac{\|\boldsymbol{u}\|_2^2}{\alpha} + \alpha\right), \ \forall \boldsymbol{u} \in \mathbb{R}^d.$$

So, to get the best possible guarantee, we should know $\|\boldsymbol{u}\|_2$ and set $\alpha = \|\boldsymbol{u}\|_2$. As we said, this strategy does not work for a few reasons: (i) $\boldsymbol{u}$ is not a fixed vector, rather the regret is a function of $\boldsymbol{u} \in \mathcal{V}$; (ii) if we guessed any value of $\|\boldsymbol{u}\|_2$ the adversary could easily change the losses to make that value completely wrong; (iii) the resulting regret would violate the lower bound in Chapter 5.

Let's show another example. Use OSD with the online-to-batch conversion to minimize a function that is L-Lipschitz, whose minimizer is $\boldsymbol{x}^\star$. Setting $\boldsymbol{x}_1 = \mathbf{0}$, the convergence rate will be $\frac{L(\|\boldsymbol{x}^\star\|_2^2/\alpha + \alpha)}{\sqrt{T}}$ using a learning rate of $\eta = \frac{\alpha}{L\sqrt{T}}$. Consider the case that $\|\boldsymbol{x}^\star\|_2 = 100$, specifying $\alpha = 1$ will result in a convergence rate about 50 times slower than specifying the optimal choice in hindsight, $\alpha = 100$. Note that this is a real effect, not an artifact of the proof. Indeed, it is intuitive that the optimal learning rate should be proportional to the distance between the initial point and the optimal solution.

In the following, we will see that it is possible to reduce any Online Convex Optimization (OCO) game to betting on a non-stochastic coin. This will allow us to very easily design OCO algorithms that will enjoy (almost) *the optimal regret uniformly over all competitors*. We call these kinds of algorithms *parameter-free*.

13.1 The Coin-Betting Game

We now consider again the coin-betting game we described in Section 5.3. Later, we will show that the solution to this problem allows us to design optimal Online Convex Optimization (OCO) algorithms.

Consider the following repeated game:

- Set the initial wealth to ϵ: $\mathrm{Wealth}_0 = \epsilon$.

- In each round $t = 1, \dots, T$
  - You bet $|x_t|$ money on the side of the coin equal to $\operatorname{sign}(x_t)$. You cannot bet more money than what you currently have, hence $|x_t| \leq \text{Wealth}_{t-1}$.
  - The adversary reveals the outcome of the coin $c_t \in \{-1, 1\}$.
  - You gain money $x_t c_t$, that is $\text{Wealth}_t = \text{Wealth}_{t-1} + c_t x_t = \epsilon + \sum_{i=1}^{t} c_i x_i$.

Given that we cannot borrow money, we can codify the bets x_t as $\beta_t \, \text{Wealth}_{t-1}$, where $\beta_t \in [-1, 1]$ is the signed betting fraction. So, $|\beta_t|$ is the fraction of money to bet and $\operatorname{sign}(\beta_t)$ the side of the coin on which we bet.

The aim of the game is to make as much money as possible. However, given the game's adversarial nature, we cannot expect to always win money. Instead, we try to gain as much money as the strategy that bets a fixed signed fraction of money $\beta^\star \in [-1, 1]$ for the entire game.

Note that

$$\begin{aligned}\text{Wealth}_t &= \text{Wealth}_{t-1} + c_t x_t = \text{Wealth}_{t-1} + \beta_t \, \text{Wealth}_{t-1} \, c_t = \text{Wealth}_{t-1}(1 + \beta_t c_t) \\ &= \epsilon \prod_{i=1}^{t} (1 + \beta_i c_i) \, .\end{aligned}$$

So, given the multiplicative nature of the wealth, it is also useful to take the logarithm of the ratio of the wealth of the algorithm and the wealth of the optimal betting fraction. Hence, we want to minimize the following regret

$$\ln \max_{\beta \in [-1,1]} \epsilon \prod_{t=1}^{T} (1 + \beta c_t) - \ln \text{Wealth}_T = \max_{\beta \in [-1,1]} \sum_{t=1}^{T} \ln(1 + \beta c_t) - \sum_{t=1}^{T} \ln(1 + \beta_t c_t) \, .$$

This is nothing else than the regret of an OCO game where the losses are $\ell_t(x) = -\ln(1 + x c_t)$ and $\mathcal{V} = [-1, 1]$, but here the losses are allowed to take the value $+\infty$. We can also extend a bit the formulation allowing "continuous coins", where $c_t \in [-1, 1]$ rather than in $\{-1, 1\}$.

Remark 13.1 *The constraint to bet a fraction between -1 and 1 is not strictly necessary. We could allow the algorithm to bet more money than it currently has, lending it some money in each round. However, the restriction makes the analysis easier because it allows the transformation above into an OCO problem, using the non-negativity of $1 + \beta_t c_t$.*

Taking special care of the non-Lipschitzness of the functions, we could just use Online Mirror Descent (OMD) or Follow-the-Regularized-Leader (FTRL), but it turns out that there exists a better strategy specifically for this problem: the **Krichevsky–Trofimov (KT) bettor**. The KT strategy simply says that on each time step t you bet $\beta_t = \frac{\sum_{i=1}^{t-1} c_i}{t}$. So, the algorithm is the following one.

Theorem 13.2 *Let $c_t \in \{-1, 1\}$ for $t = 1, \dots, T$ arbitrarily chosen. Then, the KT bettor in Algorithm 13.1 guarantees*

$$\ln \text{Wealth}_T \geq \ln \max_{\beta \in [-1,1]} \epsilon \prod_{t=1}^{T} (1 + \beta c_t) - \frac{1}{2} \ln T - 1 \, .$$

Algorithm 13.1 Krichevsky–Trofimov (KT) Bettor

Require: Initial money $\mathrm{Wealth}_0 = \epsilon > 0$
1: **for** $t = 1$ **to** T **do**
2: Calculate the betting fraction $\beta_t = \frac{\sum_{i=1}^{t-1} c_i}{t}$
3: Bet $x_t = \beta_t \mathrm{Wealth}_{t-1}$, that is $|x_t|$ money on the side $\mathrm{sign}(x_t) = \mathrm{sign}(\beta_t)$
4: Receive the coin outcome c_t in $\{-1, 1\}$ or in $[-1, 1]$
5: Win/lose $x_t c_t$, that is $\mathrm{Wealth}_t = \mathrm{Wealth}_{t-1} + c_t x_t$
6: **end for**

Proof There are many ways to prove a regret bound for the KT algorithm. One instructive way is to show that the KT algorithm is an instantiation of the Weighted Average Algorithm (WAA) algorithm we saw in Chapter 11. First of all, we set $\epsilon = 1$ without loss of generality, because its value does not affect the regret. Then, consider the WAA algorithm where we set the losses to $\ell_t(x) = -\ln(1 + c_t x)$ where $c_t \in \{-1, 1\}$ with the feasible set $\mathcal{V} = [-1, 1]$, where we define $-\ln 0 := +\infty$. It is immediate to verify that these losses are 1-exp-concave on $\mathcal{V}$. Finally, we set $P_1(\mathrm{d}x) = \frac{1}{\pi\sqrt{(1-x)(1+x)}}$ over $\mathcal{V}$.

Given that $c_t \in \{-1, 1\}$, we define $a_t = \sum_{i=1}^{t} \mathbf{1}\{c_i = 1\}$ and $b_t = \sum_{i=1}^{t} \mathbf{1}\{c_i = -1\}$, to have the following closed-form expressions:

$$\mathbb{E}_{x\sim P_1}\left[\prod_{i=1}^{t}(1 + c_i x)\right] = \frac{2^{a_t+b_t}\Gamma(a_t + 1/2)\Gamma(b_t + 1/2)}{\pi\Gamma(a_t + b_t + 1)},$$

$$\mathbb{E}_{x\sim P_1}\left[x\prod_{i=1}^{t}(1 + c_i x)\right] = \frac{2^{a_t+b_t}\,(a_t - b_t)\,\Gamma(a_t + 1/2)\,\Gamma(b_t + 1/2)}{\pi\,(a_t + b_t + 1)\,\Gamma(a_t + b_t + 1)},$$

$$\begin{aligned}\max_v \prod_{t=1}^{T}(1 + c_t v) &= \max_v\,(1 + v)^{a_T}(1 - v)^{b_T} \\ &= \left(1 + \frac{a_T - b_T}{T}\right)^{a_T}\left(1 - \frac{a_T - b_T}{T}\right)^{b_T} = 2^T a_T^{a_T} b_T^{b_T} T^{-T},\end{aligned}$$

where $0^0 := 1$. With the above formulas, we can calculate that

$$P_{t+1}(\mathrm{d}x) = \frac{\pi\,\Gamma(a_t + b_t + 1)\prod_{i=1}^{t}(1 + c_i x)P_1(\mathrm{d}x)}{2^{a_t+b_t}\,\Gamma(a_t + 1/2)\,\Gamma(b_t + 1/2)},$$

and

$$x_{t+1} = \mathbb{E}_{x\sim P_{t+1}}[x] = \frac{a_t - b_t}{a_t + b_t + 1} = \frac{\sum_{i=1}^{t} c_i}{t + 1},$$

that corresponds to the one in Algorithm 13.1.

Using the second result in Theorem 11.3, for all $u \in [-1, 1]$, we have

$$\begin{aligned}\sum_{t=1}^{T}(\ell_t(x_t) - \ell_t(u)) &\le \ln \frac{\max_{v\in[-1,1]} \prod_{t=1}^{T}(1 + c_t v)}{\mathbb{E}_{x\sim P_1}[\prod_{t=1}^{T}(1 + c_t x)]} \\ &\le \ln \frac{2^T a_T^{a_T} b_T^{b_T} T^{-T}\, \pi\, \Gamma(a_T + b_T + 1)}{2^{a_T+b_T}\, \Gamma(a_T + 1/2)\, \Gamma(b_T + 1/2)} \\ &= \ln \frac{a_T^{a_T} b_T^{b_T} T^{-T} \pi\, \Gamma(T + 1)}{\Gamma(a_T + 1/2)\, \Gamma(b_T + 1/2)}\,.\end{aligned}$$

Using Lemma A.13, we have that this expression is maximized for $a_T = T$ and $b_T = 0$ (or equivalently $a_T = 0$ and $b_T = T$), so we obtain

$$\sum_{t=1}^{T}(\ell_t(x_t) - \ell_t(u)) \le \ln \frac{\pi\, \Gamma(T + 1)}{\Gamma(T + 1/2)\, \Gamma(1/2)} = \ln \frac{\sqrt{\pi}\, \Gamma(T + 1)}{\Gamma(T + 1/2)} \le \frac{1}{2} \ln T + 1,$$

where in the last inequality we used Lemma A.12. □

Note that if the outcomes of the coin are skewed towards one side, the a posteriori optimal betting fraction will gain an exponential amount of money, as proved in the next lemma.

Lemma 13.3 *Let $c_t \in \{-1, 1\}, t = 1, \ldots, T$ arbitrarily chosen and denote by $C_T = \sum_{t=1}^{T} c_t$. Then, we have*

$$\max_{\beta\in[-1,1]} \prod_{t=1}^{T}(1 + \beta c_t) = \left(1 + \frac{C_T}{T}\right)^{\frac{T+C_T}{2}} \left(1 - \frac{C_T}{T}\right)^{\frac{T-C_T}{2}} \ge \exp\left(\frac{C_T^2}{2T}\right)\,.$$

Proof Through a derivative argument, the equality is derived by finding that

$$\operatorname*{argmax}_{\beta\in[-1,1]} \sum_{t=1}^{T} \ln(1 + \beta c_t) = C_T/T\,.$$

The inequality is true because

$$\begin{aligned}&(1 + C_T/T)^{\frac{T+C_T}{2}}\, (1 - C_T/T)^{\frac{T-C_T}{2}} \\ &\quad = \exp\left[T \cdot \mathrm{KL}\left(\left[\frac{1}{2} + \frac{C_T}{2T}, \frac{1}{2} - \frac{C_T}{2T}\right]^\top ; \left[\frac{1}{2}, \frac{1}{2}\right]^\top\right)\right],\end{aligned}$$

where KL is the Kullback–Leibler (KL) divergence. Then, we have

$$\mathrm{KL}\left(\left[\frac{1}{2} + \frac{C_T}{2T}, \frac{1}{2} - \frac{C_T}{2T}\right]^\top ; \left[\frac{1}{2}, \frac{1}{2}\right]^\top\right) \ge \frac{C_T^2}{2T^2},$$

where we used (6.4) coupled with the fact that the KL divergence is 1-strongly convex with respect to the L_1 norm (Lemma 6.33) for $C_T \in (-T, T)$, and the same inequality holds for $C_T \in \{-T, T\}$ and $T \ge 1$. □

Hence, KT can guarantee an exponential amount of money, paying only a $\sqrt{T}$ penalty. It is possible to prove that the guarantee above for the KT algorithm is optimal to constant

additive factors. Moreover, observe that the KT strategy does not require any parameter to be set: no learning rates, nor a regularizer.

Also, we can extend the guarantee of the KT algorithm to the case in which the coins are "continuous", that is $c_t \in [-1, 1]$, and we have the following theorem.

Theorem 13.4 *Let $c_t \in [-1, 1]$ for $t = 1, \dots, T$. Then, the KT bettor in Algorithm 13.1 guarantees*

$$\mathrm{Wealth}_T \geq \frac{\epsilon 2^T \Gamma\left(\frac{T+1+\sum_{t=1}^T c_t}{2}\right) \Gamma\left(\frac{T+1-\sum_{t=1}^T c_t}{2}\right)}{\pi \Gamma(T+1)} \geq \epsilon \exp\left(\frac{(\sum_{t=1}^T c_t)^2}{2T} - \frac{\ln T}{2} - 1\right),$$

where Γ is the gamma function (see Appendix A.2.2).

Proof Let $t \geq 0$, and let $F_t : [-t, t] \to \mathbb{R}$ be defined by $F_t(x) = \epsilon \frac{2^t \Gamma\left(\frac{t+1+x}{2}\right) \Gamma\left(\frac{t+1-x}{2}\right)}{\pi \Gamma(t+1)}$.

First, we want to prove that

$$F_t(x+c) \leq \left(1 + c\frac{x}{t}\right) F_{t-1}(x), \quad \forall t \geq 1, \forall c \in [-1, 1], \forall x \in [-(t-1), t-1]\,. \tag{13.1}$$

This is equivalent to showing that $\phi_t(c, x) \leq \ln F_{t-1}(x)$ for all $c \in [-1, 1]$ and $x \in [-(t-1), t-1]$, where $\phi_t(c, x) := \ln F_t(x + c) - \ln\left(1 + c\frac{x}{t}\right)$.

One can verify that $F_t(x + c) = \left(1 + c\frac{x}{t}\right) F_{t-1}(x)$ for $c \in \{-1, 1\}$ (see Problem 13.1). Hence $\phi_t(1, x) = \phi_t(-1, x) = \ln F_{t-1}(x)$. Moreover, from the log-convexity of the Gamma function (Proposition A.10(a)) and the convexity of $c \mapsto -\ln\left(1 + c\frac{x}{t}\right)$, we have that ϕ_t is convex in its first argument. Hence, from the definition of convexity, we have

$$\begin{aligned}\phi_t(c, x) &= \phi_t\left(\frac{1+c}{2} \cdot 1 + \frac{1-c}{2}(-1), x\right) \leq \frac{1+c}{2}\phi_t(1, x) + \frac{1-c}{2}\phi_t(-1, x) \\ &= \ln F_{t-1}(x)\,.\end{aligned}$$

Now, we show that the wealth of the algorithm at the beginning of round t is lower bounded by $F_{t-1}(\sum_{i=1}^{t-1} c_i)$. We prove it by induction. The base case is

$$F_0(0) = \epsilon \frac{2^0 \Gamma\left(\frac{1}{2}\right) \Gamma\left(\frac{1}{2}\right)}{\pi \Gamma(1)} = \epsilon\,.$$

Then, we assume $\mathrm{Wealth}_{t-1} \geq F_{t-1}(\sum_{i=1}^{t-1} c_i)$ and prove that $\mathrm{Wealth}_t \geq F_t(\sum_{i=1}^{t} c_i)$.

$$\begin{aligned}\mathrm{Wealth}_t &= \mathrm{Wealth}_{t-1}\left(1 + c_t \frac{\sum_{i=1}^{t-1} c_i}{t}\right) \geq F_{t-1}\left(\sum_{i=1}^{t-1} c_i\right)\left(1 + c_t \frac{\sum_{i=1}^{t-1} c_i}{t}\right) \\ &\geq F_t\left(\sum_{i=1}^{t} c_i\right),\end{aligned}$$

where in the last inequality we used (13.1).

Finally, we show the second inequality. From Lemma A.13 and denoting by KL the KL

divergence, we have

$$\begin{aligned}\frac{2^t\Gamma\left(\frac{t+1+x}{2}\right)\Gamma\left(\frac{t+1-x}{2}\right)}{\pi\Gamma(t+1)} &\geq \frac{2^t\Gamma\left(t+\frac{1}{2}\right)\Gamma\left(\frac{1}{2}\right)\left(\frac{t+x}{2}\right)^{\frac{t+x}{2}}\left(\frac{t-x}{2}\right)^{\frac{t-x}{2}}}{\pi\Gamma(t+1)t^t}\\ &= \frac{\Gamma\left(t+\frac{1}{2}\right)\left(1+\frac{x}{t}\right)^{\frac{t+x}{2}}\left(1-\frac{x}{t}\right)^{\frac{t-x}{2}}}{\sqrt{\pi}\Gamma(t+1)}\\ &= \frac{\Gamma\left(t+\frac{1}{2}\right)\exp\left[t\cdot\mathrm{KL}\left(\left[\frac{1}{2}+\frac{x}{2t},\frac{1}{2}-\frac{x}{2t}\right]^\top;\left[\frac{1}{2},\frac{1}{2}\right]^\top\right)\right]}{\sqrt{\pi}\Gamma(t+1)}\\ &\geq \frac{\Gamma\left(t+\frac{1}{2}\right)\exp\left(\frac{x^2}{2t}\right)}{\sqrt{\pi}\Gamma(t+1)} \geq \exp\left(\frac{x^2}{2t}-\frac{1}{2}\ln t-1\right),\end{aligned}$$

where in the second to last inequality we used (6.4) coupled with the fact that the KL divergence is 1-strongly convex with respect to the L_1 norm (Lemma 6.33) for $x \in (-t, t)$, and the same inequality holds for $x \in \{-t, t\}$ and $t \geq 1$. Finally, Lemma A.12 gives in the last inequality. □

So, we have introduced the coin-betting game, extended it to continuous coins, and presented a simple and optimal strategy to solve it. In the next section, we show *how to use the KT bettor as a parameter-free one-dimensional OCO algorithm!*

13.2 Parameter-free One-Dimensional Online Convex Optimization

So, Theorem 13.2 tells us that we can win almost as much money as a strategy betting the optimal fixed fraction of money at each step. We only pay a logarithmic price in the log wealth, which corresponds to a $\frac{1}{\sqrt{T}}$ term in the actual wealth.

Now, let's see why this problem is interesting in OCO. It turns out that *solving the coin-betting game with continuous coins is equivalent to solving a one-dimensional unconstrained online linear optimization problem*. That is, a coin-betting algorithm is equivalent to an online learning algorithm that produces a sequence of $x_t \in \mathbb{R}$ that minimizes the one-dimensional regret with linear losses:

$$\mathrm{Regret}_T(u) := \sum_{t=1}^{T} g_t(x_t - u),$$

where the g_t are adversarial and bounded. Without loss of generality, we will assume $g_t \in [-L, L]$. Also, remembering that OCO games can be reduced to Online Linear Optimization (OLO) games, such a reduction would effectively reduce OCO to coin betting! Moreover, through online-to-batch conversion, any stochastic 1-d problem could be reduced to a coin-betting game!

The key theorem that allows the conversion between OLO and coin betting is the following one.

Theorem 13.5 (Regret-Reward Duality) *Let $\phi : \mathbb{R}^d \to (-\infty, +\infty]$ be a proper closed convex function and let $\phi^\star : \mathbb{R}^d \to (-\infty, +\infty]$ be its Fenchel conjugate. Consider two*

sequences $\boldsymbol{x}_1, \boldsymbol{x}_2, \dots, \boldsymbol{x}_T \in \mathbb{R}^d$ *and* $\boldsymbol{g}_1, \dots, \boldsymbol{g}_T \in \mathbb{R}^d$*. Then,*

$$-\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle \geq \phi\left(-\sum_{t=1}^{T} \boldsymbol{g}_t\right) \Leftrightarrow \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \leq \phi^\star(\boldsymbol{u}), \quad \forall \boldsymbol{u} \in \mathbb{R}^d \,.$$

Proof Let's prove the left-to-right implication. For any $\boldsymbol{u} \in \mathbb{R}^d$, we have

$$\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \leq -\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle - \phi\left(-\sum_{t=1}^{T} \boldsymbol{g}_t\right) \leq \sup_{\boldsymbol{\theta} \in \mathbb{R}^d} \langle \boldsymbol{\theta}, \boldsymbol{u} \rangle - \phi(\boldsymbol{\theta}) = \phi^\star(\boldsymbol{u}) \,.$$

For the other implication, we have

$$\begin{aligned} -\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle &= \sup_{\boldsymbol{u} \in \mathbb{R}^d} -\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle - \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \\ &\geq \sup_{\boldsymbol{u} \in \mathbb{R}^d} -\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle - \phi^\star(\boldsymbol{u}) = \phi\left(-\sum_{t=1}^{T} \boldsymbol{g}_t\right) \,. \end{aligned}$$

□

To make sense of the above theorem, assume that we are considering a 1-d problem and $g_t \in [-L, L]$. Then, guaranteeing a lower bound to $-\sum_{t=1}^{T} g_t x_t$ can be done through a betting strategy that bets x_t money on the continuous coins $c_t = -g_t/L \in [-1, 1]$. So, the theorem implies that *proving a lower bound for the wealth in a coin-betting game implies a regret upper bound for the corresponding one-dimensional OLO game*. However, proving a reward lower bound is easier because it does not depend on the competitor $\boldsymbol{u}$. Indeed, not knowing the norm of the competitor is exactly the reason why tuning the learning rates in OMD is hard!

This consideration immediately gives us the conversion between 1-d OLO and coin betting: *the outcome of the coin is the negative of the subgradient of the losses on the current prediction.* Indeed, setting $c_t = -g_t/L$, we have that a coin-betting algorithm that bets x_t would give us

$$\text{Wealth}_T = \epsilon + \sum_{t=1}^{T} x_t c_t = \epsilon - \sum_{t=1}^{T} x_t \frac{g_t}{L} \,.$$

So, a lower bound on the wealth corresponds to a lower bound that can be used in Theorem 13.5. To obtain a regret guarantee, we only need to calculate the Fenchel conjugate of the reward function, assuming it can be expressed as a function of $\sum_{t=1}^{T} c_t$.

The last step is to reduce 1-d OCO to 1-d OLO. But this is an easy step that we have done many times. Indeed, we have

$$\text{Regret}_T(u) = \sum_{t=1}^{T} \ell_t(x_t) - \sum_{t=1}^{T} \ell_t(u) \leq \sum_{t=1}^{T} x_t g_t - \sum_{t=1}^{T} g_t u, \quad \forall u \in \mathbb{R}, \, \forall g_t \in \partial \ell_t(x_t) \,.$$

So, to summarize, the Fenchel conjugate of the wealth lower bound for the coin-betting game becomes the regret guarantee for the OCO game. In the next section, we specialize all these considerations to the KT algorithm.

Algorithm 13.2 Krichevsky–Trofimov (KT) OCO Algorithm

Require: $\epsilon > 0$, $L > 0$
1: **for** $t = 1$ **to** T **do**
2: Predict $x_t = -\frac{\sum_{i=1}^{t-1} g_i}{tL} \left(\epsilon - \sum_{i=1}^{t-1} \frac{g_i}{L} x_i \right) \in \mathbb{R}$
3: Pay the loss $\ell_t(x_t)$, where ℓ_t is subdifferentiable on $\mathbb{R}$
4: Set $g_t \in \partial \ell_t(x_t)$ where $|g_t| \le L$
5: **end for**

13.2.1 KT as a One-Dimensional Online Convex Optimization Algorithm

Here, we want to use the considerations in the above section to use KT as a parameter-free 1-d OCO algorithm. First, let's see what such an algorithm looks like. KT bets $x_t = \beta_t \operatorname{Wealth}_{t-1}$, starting with ϵ money. Now, set $c_t = -g_t/L$ where $g_t \in \partial \ell_t(x_t)$ and assume the losses ℓ_t are L-Lipschitz. So, we get

$$x_t = -\frac{\sum_{i=1}^{t-1} g_i}{tL} \left(\epsilon - \sum_{i=1}^{t-1} \frac{g_i}{L} x_i \right) .$$

Algorithm 13.2 shows the **Krichevsky–Trofimov algorithm for OCO**.

Let's now see what kind of regret we get. From Theorem 13.4, we have that the KT bettor guarantees the following lower bound on the wealth when used with $c_t = -g_t/L$:

$$\epsilon - \sum_{t=1}^{T} x_t \frac{g_t}{L} \ge \frac{\epsilon}{e\sqrt{T}} \exp\left(\frac{\left(\sum_{t=1}^{T} g_t\right)^2}{2TL^2} \right) .$$

So, we found the function ϕ; we just need $\phi^\star$ or an upper bound to it, which can be found with the following lemma.

Lemma 13.6 *Define $f(x) = \beta \exp \frac{x^2}{2\alpha}$, for $\alpha, \beta > 0$, $x \in \mathbb{R}$. Then*

$$f^\star(y) = |y| \sqrt{\alpha W\left(\frac{\alpha y^2}{\beta^2}\right)} - \beta \exp\left(\frac{1}{2} W\left(\frac{\alpha y^2}{\beta^2}\right)\right) \le |y| \sqrt{\alpha W\left(\frac{\alpha y^2}{\beta^2}\right)} - \beta$$
$$\le |y| \sqrt{\alpha \ln\left(1 + \frac{\alpha y^2}{\beta^2}\right)} - \beta \le |y| \sqrt{2\alpha \ln\left(1 + \frac{\sqrt{\alpha}|y|}{\beta}\right)} - \beta,$$

where W is the Lambert function, i.e., $W : \mathbb{R}_{\ge 0} \to \mathbb{R}_{\ge 0}$ satisfies $x = W(x) \exp(W(x))$.

Proof From the definition of the Fenchel conjugate, we have

$$f^\star(y) = \max_{x \in \mathbb{R}} \; x\, y - f(x) = \max_{x} \; x\, y - \beta \exp \frac{x^2}{2\alpha}, = x^\star\, y - \beta \exp \frac{(x^\star)^2}{2\alpha},$$

where $x^\star = \operatorname{argmax}_x x\, y - f(x)$. We now use the fact that $x^\star$ satisfies $y = f'(x^\star)$, to have $x^\star = \operatorname{sign}(y)\sqrt{\alpha W(\frac{\alpha y^2}{\beta^2})}$, where $W(\cdot)$ is the Lambert function. Using Lemma A.6 in the Appendix, we obtain the second-to-last bound. The last one is obtained by using $\sqrt{a^2 + 1} \le |a| + 1$ for all $a \in \mathbb{R}$. □

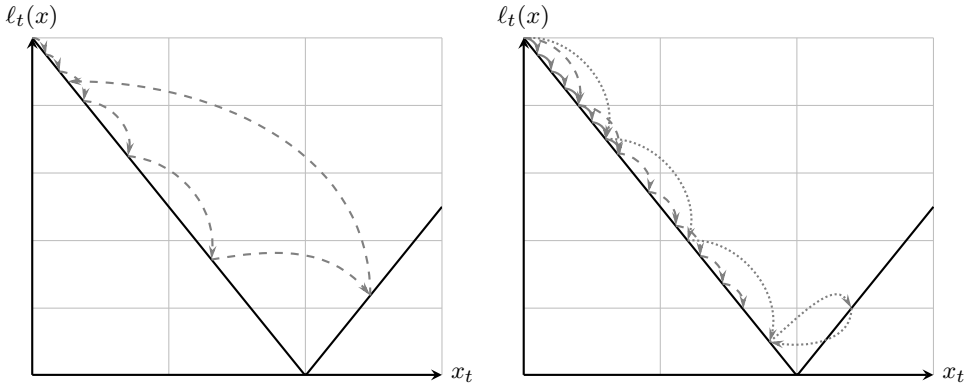


Figure 13.1 Behaviour of KT (left) and OSD with various learning rates and the same number of steps (right) when $\ell_t(x) = |x - 10|$ for all t.

So, the regret guarantee of KT used as a one-dimensional OCO algorithm is

$$\text{Regret}_T(u) = \sum_{t=1}^{T} (\ell_t(x_t) - \ell_t(u)) \leq |u| L \sqrt{2T \ln \left(\frac{e|u|T}{\epsilon} + 1 \right)} + \epsilon L, \quad \forall u \in \mathbb{R},$$

where the only assumption was that the first derivatives (or subderivatives) of ℓ_t are bounded in absolute value by L. The bound is almost optimal because the optimal one has $\sqrt{T}$ factor inside the logarithm, instead of T, see the lower bound in Theorem 5.8. Also, it is important to note that any setting of ϵ in $[1, \sqrt{T}]$ would not change the asymptotic rate.

To appreciate this guarantee, compare it to the one of OMD with learning rate $\eta = \frac{\alpha}{L\sqrt{T}}$:

$$\text{Regret}_T(u) = \sum_{t=1}^{T} \ell_t(x_t) - \sum_{t=1}^{T} \ell_t(u) \leq \frac{L}{2} \left(\frac{u^2}{\alpha} + \alpha \right) \sqrt{T}, \quad \forall u \in \mathbb{R} \,.$$

Hence, the coin-betting approach allows us to get almost the optimal bound, without having to solve the impossible task to choose the correct learning rate for each competitor, that is, in a *parameter-free* way. This motivates the following definition.

Definition 13.7 *We call **parameter-free** any online algorithm that satisfies the optimal regret bound with respect to T and to all the comparators in the feasible set $\mathcal{V}$, up to poly-logarithmic multiplicative factors.*

We note explicitly that in this definition, the algorithm is allowed to know the Lipschitz constant of the losses. Hence, Algorithm 13.2 is parameter-free.

It is also interesting to look at what the algorithm does on an easy problem, where $\ell_t(x) = |x - 10|$. In Figure 13.1, we show the different predictions that the KT and OSD algorithms make. The convergence rate of OSD critically depends on the learning rate: too large does not give convergence, and too small slows down the convergence. On the other hand, KT goes *exponentially fast* towards the minimum, and then it automatically backtracks. This exponential growth effectively works like a line search procedure. Later in the iterations, KT will oscillate around the minimum, *automatically shrinking its steps, without any parameter*

to tune. Of course, this is a simplified example. In a truly OCO game, the losses are different at each time step, and the intuition behind the algorithm becomes more difficult. Yet, the regret guarantee of KT assures us that this strategy is principled.

Remark 13.8 *Although we derived the KT algorithm for OCO via its connection to the coin-betting game, it remains an FTRL algorithm, and with a more involved proof, one can prove the same regret guarantee. Problem 13.5 shows a simplification of this idea, by designing a parameter-free algorithm directly from the FTRL equality in Lemma 7.1.*

Next, we reduce OCO in $\mathbb{R}^d$ and Learning with Expert Advice (LEA) to coin betting.

13.3 Coordinate-wise Parameter-free Online Convex Optimization

Algorithm 13.3 Coordinate-Wise KT for OCO

Require: $\epsilon > 0$, $L_\infty > 0$

for $t = 1$ **to** T **do**

  Output $\boldsymbol{x}_t \in \mathbb{R}^d$ such that $x_{t,i} = -\frac{\sum_{j=1}^{t-1} g_{j,i}}{L_\infty t}\left(\epsilon - \sum_{j=1}^{t-1} \frac{g_{j,i}}{L_\infty} x_{j,i}\right)$ for $i = 1, \dots, d$

  Pay the loss $\ell_t(\boldsymbol{x}_t)$, where ℓ_t is subdifferentiable in $\mathbb{R}^d$

  Set $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$ where $\|\boldsymbol{g}_t\|_\infty \leq L_\infty$

end for

With AdaGrad (Section 4.2.4), we have already seen that it is possible to decompose an OCO problem over the coordinates and use a different one-dimensional OLO algorithm on each coordinate. In particular, we saw that

$$\begin{aligned}\operatorname{Regret}_T(\boldsymbol{u}) &= \sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}) \leq \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle \\ &= \sum_{t=1}^T \sum_{i=1}^d g_{t,i}(x_{t,i} - u_i) = \sum_{i=1}^d \sum_{t=1}^T g_{t,i}(x_{t,i} - u_i),\end{aligned}$$

where the $\sum_{t=1}^T g_{t,i}(x_{t,i} - u_i)$ is exactly the regret with respect to the linear losses constructed by the coordinate i of the subgradient.

Hence, if we have a one-dimensional OLO algorithm, we can use d copies of it, each one fed with the coordinate i of the subgradient. In particular, we might think of using a **coordinate-wise KT** algorithm. The pseudo-code of this procedure is in Algorithm 13.3.

The regret bound we get is immediate: we just have to sum the regret over the coordinates.

Theorem 13.9 *With the notation in Algorithm 13.3, assume that $\|\boldsymbol{g}_t\|_\infty \leq L_\infty$. Then, $\forall \boldsymbol{u} \in \mathbb{R}^d$, the following regret bounds hold*

$$\begin{aligned}\sum_{t=1}^T (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) &\leq L_\infty \sum_{i=1}^d |u_i| \sqrt{2T \ln\left(1 + e|u_i|T/\epsilon\right)} + d\epsilon L_\infty \\ &\leq \|\boldsymbol{u}\|_1 L_\infty \sqrt{2T \ln\left(1 + e\|\boldsymbol{u}\|_\infty T/\epsilon\right)} + d\epsilon L_\infty \,.\end{aligned}$$

This theorem suggests that in high-dimensional settings ϵ should be proportional to $\frac{1}{d}$.

13.4 Parameter-free Algorithms in Any Norm

The above reduction works only in a finite-dimensional space. Moreover, it gives a dependence on the competitor with respect to the L_1 norm that might be undesirable. So, here we use the black-box magnitude/direction reduction from Section 12.3.

Assume the loss functions are L-Lipschitz with respect to the $\|\cdot\|_2$. We can instantiate the above theorem using the KT betting algorithm for the one-dimensional learner and OMD for the direction learner. For example, let $\mathscr{A}_B$ be OSD with $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \leq 1\}$, $\boldsymbol{x}_1 = \mathbf{0}$, and learning rate $\eta_t = \frac{\sqrt{2}}{2L\sqrt{t}}$. Let $\mathscr{A}_{\mathrm{1d}}$ be the KT algorithm for one-dimensional OCO. Then, using the construction in Algorithm 12.3, we have

$$\mathrm{Regret}_T(\boldsymbol{u}) \leq L\|\boldsymbol{u}\|_2 \left(\sqrt{\ln(e\|\boldsymbol{u}\|_2 T/\epsilon + 1)} + 1\right)\sqrt{2T} + L\epsilon, \quad \forall \boldsymbol{u} \in \mathbb{R}^d\,. \tag{13.2}$$

Using an online-to-batch conversion, this algorithm is a stochastic gradient descent procedure without learning rates to tune.

Once again, to better appreciate this kind of guarantee, let's take a look at the one of FTRL (remember that OSD can be used in unbounded domains only with constant learning rates). With the regularizer $\psi_t(\boldsymbol{x}) = \frac{L\sqrt{t}}{2\alpha}\|\boldsymbol{x}\|_2^2$ and L-Lipschitz losses we get a regret of

$$\mathrm{Regret}_T(\boldsymbol{u}) \leq L\sqrt{T}\left(\frac{\|\boldsymbol{u}\|_2^2}{2\alpha} + \alpha\right)\,.$$

So, to get the right dependence on $\|\boldsymbol{u}\|_2$, we need to tune α, but we saw that this is impossible. On the other hand, the regret in (13.2) suffers from a logarithmic factor, which is the price to pay not to have to tune parameters.

In the same way, we can even have a parameter-free regret bound for L_p norms. Assume that the loss functions are L_q-Lipschitz with respect to $\|\cdot\|_p$, where $1 < p \leq 2$ and q is such that $1/p + 1/q = 1$. Let $\mathscr{A}_B$ be OMD with $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_p \leq 1\}$ and learning rate $\eta_t = \frac{\sqrt{p-1}}{L_q\sqrt{2t}}$ (see Section 6.7). Let $\mathscr{A}_{\mathrm{1d}}$ be the KT algorithm for one-dimensional OCO. Then, using the construction in Algorithm 12.3, we have

$$\mathrm{Regret}_T(\boldsymbol{u}) \leq L_q\left(\|\boldsymbol{u}\|_p\sqrt{\ln(e\|\boldsymbol{u}\|_p T/\epsilon + 1)} + \frac{\|\boldsymbol{u}\|_p}{\sqrt{p-1}}\right)\sqrt{2T} + L_q\epsilon, \quad \forall \boldsymbol{u} \in \mathbb{R}^d\,.$$

Note that the regret against $\boldsymbol{u} = \mathbf{0}$ of the parameter-free construction is *constant* with respect to T. It is important to understand that there is nothing special in the origin in the unconstrained setting: we could translate the prediction by any offset and get a guarantee that treats the offset as the point with constant regret. This is shown in the next proposition.

Proposition 13.10 *Let $\mathscr{A}$ be an OLO algorithm that predicts $\boldsymbol{x}_t$ and has linear regret $\mathrm{Regret}_T^{OLO}(\boldsymbol{u})$ for any $\boldsymbol{u} \in \mathbb{R}^d$. We have that the regret of the predictions $\hat{\boldsymbol{x}}_t = \boldsymbol{x}_t + \boldsymbol{x}_0$ for OCO is*

$$\sum_{t=1}^T \ell_t(\hat{\boldsymbol{x}}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}) \leq \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t + \boldsymbol{x}_0 - \boldsymbol{u}\rangle = \mathrm{Regret}_T^{OLO}(\boldsymbol{u} - \boldsymbol{x}_0), \quad \forall \boldsymbol{u} \in \mathbb{R}^d,$$

where each $\boldsymbol{g}_t \in \partial \ell_t(\hat{\boldsymbol{x}}_t)$.

13.5 Combining Parameter-free Algorithms Almost for Free

We now show a useful application of the property of parameter-free OCO algorithms of having constant regret against $\boldsymbol{u} = \boldsymbol{0}$.

Theorem 13.11 *Let* $\mathscr{A}_1$ *and* $\mathscr{A}_2$ *be two OLO algorithms that produce the predictions* $\boldsymbol{x}_{t,1}$ *and* $\boldsymbol{x}_{t,2}$*, and have regrets equal to* $\mathrm{Regret}_T^{\mathscr{A}_1}(\boldsymbol{u})$ *and* $\mathrm{Regret}_T^{\mathscr{A}_2}(\boldsymbol{u})$*, respectively. Then, predicting with* $\boldsymbol{x}_t = \boldsymbol{x}_{t,1} + \boldsymbol{x}_{t,2}$*, we have for any* $\boldsymbol{u} \in \mathbb{R}^d$

$$\sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle = \min_{\boldsymbol{u}=\boldsymbol{u}_1+\boldsymbol{u}_2} \mathrm{Regret}_T^{\mathscr{A}_1}(\boldsymbol{u}_1) + \mathrm{Regret}_T^{\mathscr{A}_2}(\boldsymbol{u}_2) \,.$$

Moreover, if both algorithms have a constant regret of ϵ *against* $\boldsymbol{u} = \boldsymbol{0}$*, we have for any* $\boldsymbol{u} \in \mathbb{R}^d$

$$\sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle \leq \epsilon + \min \left(\mathrm{Regret}_T^{\mathscr{A}_1}(\boldsymbol{u}), \mathrm{Regret}_T^{\mathscr{A}_2}(\boldsymbol{u}) \right) \,.$$

Proof Set $\boldsymbol{u}_1 + \boldsymbol{u}_2 = \boldsymbol{u}$. Then,

$$\sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle = \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_{t,1} \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u}_1 \rangle + \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_{t,2} \rangle - \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{u}_2 \rangle \,.$$

Taking the minimum between the bounds with $\boldsymbol{u}_1 = \boldsymbol{0}$ and $\boldsymbol{u}_2 = \boldsymbol{0}$ concludes the proof. □

In words, the above theorem allows us to combine online learning algorithms. If the algorithms we combine have constant regret against the null competitor, then we always get the best of the two guarantees.

Example 13.12 *We can combine two parameter-free OCO algorithms, one that gives a bound that depends on the* L_2 *norm of the competitor and subgradients, and another one specialized to the* L_1*/*L_∞ *norm of competitor/subgradients. The above theorem assures us that we will also get the best guarantee between the two, paying only an additive constant factor in the regret.*

Of course, upper-bounding the OCO regret with the linear regret, the above theorem also upper bounds the OCO regret.

13.6 Reduction to Learning with Experts

In this section, we consider again the LEA setting from Section 6.8. First, remember that the regret we got from Exponentiated Gradient (EG) in Section 6.6 (and similarly for the FTRL version in Section 7.5) is

$$\mathrm{Regret}_T(\boldsymbol{u}) \leq \frac{\mathrm{KL}(\boldsymbol{u}; \boldsymbol{\pi})}{\eta} + \frac{\eta T L_\infty^2}{2}, \quad \forall \boldsymbol{u} \in \Delta^{d-1},$$

where $\boldsymbol{\pi}$ is the prior distribution on the experts and $\mathrm{KL}(\cdot;\cdot)$ is the KL divergence. As we reasoned in the OCO case, to set the learning rate, we should know the value of $\mathrm{KL}(\boldsymbol{u};\boldsymbol{\pi})$. If we could set η to $\sqrt{\frac{2\,\mathrm{KL}(\boldsymbol{u};\boldsymbol{\pi})}{L_\infty^2\,T}}$, we would obtain a regret of $L_\infty\sqrt{2T\;\mathrm{KL}(\boldsymbol{u};\boldsymbol{\pi})}$. However, given the adversarial nature of the game, this is impossible. So, as we did in the OCO case, we will see that even this problem can be reduced to betting on a continuous coin, obtaining optimal regret guarantees with a parameter-free algorithm.

Remark 13.13 *There exists a different notion of regret for LEA. Order the cumulative losses of all actions from lowest to highest and define the* ϵ***-quantile regret*** *to be the difference between the cumulative loss of the learner and the* $\lceil \epsilon d \rceil$*-th element in the sorted list. In formulas*

$$\mathrm{Regret}_T(\epsilon) = \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - \sum_{t=1}^T g_{t,i_\epsilon},$$

where $\epsilon \in [1/d, 1]$ *and* i_ϵ *is the* $\lceil \epsilon d \rceil$*-th best-performing action.*

This definition makes sense when the number of actions is very large, and there are many of them that are close to the optimal one. Note that the task of guaranteeing a small regret becomes easier with increasing ϵ*. So, we would like to design an algorithm whose regret depends on* $\ln \frac{1}{\epsilon}$ *and it does not depend on* d *in any way.*

We now show that a regret that depends on the KL divergence between a generic competitor $\boldsymbol{u}$ *and* $\boldsymbol{\pi}$ *implies such regret guarantee. Define* $\boldsymbol{u}_\epsilon$ *as the vector that has the coordinates corresponding to the best* $\lceil \epsilon d \rceil$ *experts equal to* $\frac{1}{\lceil \epsilon d \rceil}$ *and 0 in the other coordinates. Also, assume to have an algorithm that guarantees an upper bound* $\mathrm{Regret}_T(\boldsymbol{u})$ *equal to* $F_T(\mathrm{KL}(\boldsymbol{u};\boldsymbol{\pi}))$ *for a sequence of losses* $\boldsymbol{g}_t \in \mathbb{R}^d$*, where* F_T *is a non-decreasing function. Then, setting* $\boldsymbol{\pi} = [1/d, \dots, 1/d]$*, we have*

$$\begin{aligned}\mathrm{Regret}_T(\epsilon) &= \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - \sum_{t=1}^T g_{t,i_\epsilon} \le \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{p}_t - \boldsymbol{u}_\epsilon \rangle \le F_T(\mathrm{KL}(\boldsymbol{u}_\epsilon;\boldsymbol{\pi})) \\ &= F_T\left(\ln \frac{d}{\lceil \epsilon d \rceil}\right) \le F_T\left(\ln \frac{1}{\epsilon}\right),\end{aligned}$$

where in the first inequality we used the fact that the average of a set of numbers is smaller than the biggest number in the set.

First, let's introduce some notation. Let $d \ge 2$ be the number of experts and Δ^{d-1} be the probability simplex. Let $\boldsymbol{\pi} = [\pi_1, \pi_2, \dots, \pi_d] \in \Delta^{d-1}$ be any *prior* distribution. Let $\mathscr{A}$ be a coin-betting algorithm. We will instantiate d copies of $\mathscr{A}$, each of them with initial wealth π_i for $i = 1, \dots, d$.

Consider any round t. Let $x_{t,i} \in \mathbb{R}$ be the bet of the i-th copy of the coin-betting algorithm $\mathscr{A}$. The LEA algorithm computes $\widehat{\boldsymbol{p}}_t = [\widehat{p}_{t,1}, \widehat{p}_{t,2}, \dots, \widehat{p}_{t,d}] \in \mathbb{R}^d_{\ge 0}$ as

$$\widehat{p}_{t,i} = \max(x_{t,i}, 0)\,. \tag{13.3}$$

Then, the LEA advice algorithm predicts $\boldsymbol{p}_t = [p_{t,1}, p_{t,2}, \dots, p_{t,d}] \in \Delta^{d-1}$ as

$$\boldsymbol{p}_t = \begin{cases} \frac{\widehat{\boldsymbol{p}}_t}{\|\widehat{\boldsymbol{p}}_t\|_1}, & \text{if } \widehat{\boldsymbol{p}}_t \neq \boldsymbol{0}, \\ \boldsymbol{\pi}, & \text{otherwise}\,. \end{cases} \tag{13.4}$$

After the prediction, the algorithm receives the vector of losses $\boldsymbol{g}_t = [g_{t,1}, g_{t,2}, \ldots, g_{t,d}] \in [-L_\infty, L_\infty]^d$. From these losses, we construct the outcome of the continuous coin $c_{t,i} \in [-1, 1]$ for the i-th copy of the coin-betting algorithm $\mathscr{A}$, defined as

$$c_{t,i} = \frac{1}{2L_\infty} \begin{cases} \langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - g_{t,i} & \text{if } x_{t,i} > 0, \\ \max\left(\langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - g_{t,i}, 0\right) & \text{if } x_{t,i} \le 0\,. \end{cases} \tag{13.5}$$

The construction above defines a LEA algorithm that predicts $\boldsymbol{p}_t$, based on the algorithm $\mathscr{A}$. We can prove the following regret bound for it.

Theorem 13.14 (Regret Bound for Experts) *Let $\mathscr{A}$ be a coin-betting algorithm that guarantees a wealth after t rounds with initial money equal to $a > 0$ of at least $a \exp(f_t(\sum_{j=1}^t c'_j))$ for any sequence of continuous coin outcomes $c'_1, \ldots, c'_t \in [-1, 1]$. Assume each LEA loss to satisfy $|\boldsymbol{g}_{t,i}| \le L_\infty$ for $i = 1, \ldots, d$. Then, the regret of the LEA algorithm with prior $\boldsymbol{\pi} \in \Delta^{d-1}$ that predicts at each round with $\boldsymbol{p}_t$ in* (13.4) *satisfies*

$$\mathrm{Regret}_T(\boldsymbol{u}) = \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{p}_t - \boldsymbol{u} \rangle \le 2L_\infty h\left(\mathrm{KL}(\boldsymbol{u}; \boldsymbol{\pi})\right), \quad \forall T \ge 1, \forall \boldsymbol{u} \in \Delta^{d-1},$$

for any $h : \mathbb{R} \to \mathbb{R}$ concave and non-decreasing such that $x \le h(f_T(x))$.

Proof We first prove that $\langle \boldsymbol{c}_t, \boldsymbol{x}_t \rangle \le 0$. Indeed,

$$\begin{aligned} \langle \boldsymbol{c}_t, \boldsymbol{x}_t \rangle &= \sum_{i=1}^d c_{t,i} x_{t,i} \\ &= \sum_{i: x_{t,i} > 0} \frac{\max(x_{t,i}, 0)(\langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - g_{t,i})}{2L_\infty} + \sum_{i: x_{t,i} \le 0} \frac{x_{t,i} \max(\langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - g_{t,i}, 0)}{2L_\infty} \\ &= \frac{1}{2L_\infty} \|\widehat{\boldsymbol{p}}_t\|_1 \sum_{i=1}^d p_{t,i}(\langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - g_{t,i}) + \frac{1}{2L_\infty} \sum_{i: x_{t,i} \le 0} x_{t,i} \max(\langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - g_{t,i}, 0) \\ &= 0 + \frac{1}{2L_\infty} \sum_{i\,:\, x_{t,i} \le 0} x_{t,i} \max(\langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - g_{t,i}, 0) \;\le 0\,. \end{aligned}$$

The first equality follows from the definition of $c_{t,i}$. To see the second equality, consider two cases: if $x_{t,i} \le 0$ for all i, then $\|\widehat{p}_t\|_1 = 0$ and therefore both $\|\widehat{\boldsymbol{p}}_t\|_1 \sum_{i=1}^d p_{t,i}(g_{t,i} - \langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle)$ and $\sum_{i\,:\,x_{t,i}>0} \max(x_{t,i}, 0)(g_{t,i} - \langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle)$ are trivially zero. If $\|\widehat{\boldsymbol{p}}_t\|_1 > 0$ then $\max(x_{t,i}, 0) = \widehat{p}_{t,i} = \|\widehat{\boldsymbol{p}}_t\|_1 p_{t,i}$ for all i.

From the assumption on the coin-betting algorithm $\mathscr{A}$, for the arm i, we have that

$$\mathrm{Wealth}_{T,i} = \pi_i + \sum_{t=1}^T c_{t,i} x_{t,i} \ge \pi_i \exp\left(f_T\left(\sum_{t=1}^T c_{t,i}\right)\right)\,. \tag{13.6}$$

So, inequality $\langle \boldsymbol{c}_t, \boldsymbol{x}_t \rangle \le 0$ and (13.6) imply

$$\sum_{i=1}^d \pi_i \exp\left(f_T\left(\sum_{t=1}^T c_{t,i}\right)\right) \le 1 + \sum_{i=1}^d \sum_{t=1}^T c_{t,i} x_{t,i} \le 1\,. \tag{13.7}$$

Algorithm 13.4 Learning with Expert Advice based on KT Bettors

Require: Number of experts d, prior distribution $\boldsymbol{\pi} \in \Delta^{d-1}$, $L_\infty > 0$

for $t = 1, 2, \ldots, T$ **do**

Set $x_{t,i} = \frac{\sum_{j=1}^{t-1} c_{j,i}}{t} \left(\pi_i + \sum_{j=1}^{t-1} c_{j,i} x_{j,i} \right)$ for $i = 1, \ldots, d$

Set $\widehat{p}_{t,i} = \max(x_{t,i}, 0)$ for $i = 1, \ldots, d$

Predict with $\boldsymbol{p}_t = \begin{cases} \widehat{\boldsymbol{p}}_t / \|\widehat{\boldsymbol{p}}_t\|_1 & \text{if } \|\widehat{\boldsymbol{p}}_t\|_1 > 0 \\ \boldsymbol{\pi} & \text{if } \|\widehat{\boldsymbol{p}}_t\|_1 = 0 \end{cases}$

Receive loss vector $\boldsymbol{g}_t \in [-L_\infty, L_\infty]^d$

Set $c_{t,i} = \frac{1}{2L_\infty} \begin{cases} \langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - g_{t,i}, & \text{if } x_{t,i} > 0 \\ \max(\langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - g_{t,i}, 0), & \text{if } x_{t,i} \le 0 \end{cases}$ for $i = 1, \ldots, d$

end for

Now, for any competitor $\boldsymbol{u} \in \Delta^{d-1}$, we derive

$$
\begin{aligned}
\mathrm{Regret}_T(\boldsymbol{u}) &= \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{p}_t - \boldsymbol{u} \rangle \\
&= \sum_{t=1}^{T} \sum_{i=1}^{d} u_i \left(\langle \boldsymbol{g}_t, \boldsymbol{p}_t \rangle - g_{t,i} \right) \\
&\le 2L_\infty \sum_{t=1}^{T} \sum_{i=1}^{d} u_i c_{t,i} \quad \text{(definition of } c_{t,i}\text{)} \\
&\le 2L_\infty \sum_{i=1}^{d} u_i h \left(f_T \left(\sum_{t=1}^{T} c_{t,i} \right) \right) \quad \text{(definition of the } h(x)\text{)} \\
&\le 2L_\infty h \left(\sum_{i=1}^{d} u_i f_T \left(\sum_{t=1}^{T} c_{t,i} \right) \right) \quad \text{(Jensen's inequality)} \\
&\le 2L_\infty h \left(\mathrm{KL}(\boldsymbol{u}; \boldsymbol{\pi}) + \ln \sum_{i=1}^{d} \pi_i e^{f_T\left(\sum_{t=1}^{T} c_{t,i}\right)} \right) \quad \text{(Fenchel–Young)} \\
&\le 2L_\infty h \left(\mathrm{KL}(\boldsymbol{u}; \boldsymbol{\pi}) \right) \quad \text{(by (13.7))} .
\end{aligned}
$$

□

Now, we could think of using the KT bettor with this theorem, obtaining the Algorithm 13.4.

Unfortunately, this algorithm obtains a sub-optimal regret guarantee. In fact, remembering the lower bound on the wealth of KT and setting $h(x) = \sqrt{2T(x + \frac{1}{2} \ln T + 1)}$, we have

$$
\mathrm{Regret}_T(\boldsymbol{u}) \le L_\infty \sqrt{8T \left(\mathrm{KL}(\boldsymbol{u}; \boldsymbol{\pi}) + \frac{1}{2} \ln T + 1 \right)} .
$$

We might think that the $\ln T$ is the price we have to pay to adapt to the unknown competitor $\boldsymbol{u}$. However, it turns out it can be removed. In the next section, we see how to change the KT strategy to obtain the optimal guarantee.

13.7 Optimal Regret Bounds by Losing at Most a Constant Fraction of Money

In the reduction before, if we use the KT betting strategy, we would have a $\ln T$ term under the square root. It turns out that we can avoid that term if we know the number of rounds beforehand. Then, in case T is unknown, we can just use a doubling trick (see Problem 6.8), paying only a constant multiplicative factor in the regret.

More in detail, the logarithmic term in the regret comes from the $\frac{1}{\sqrt{T}}$ term in the lower bound on the wealth. Note that in the case in which the number of heads in the sequence is equal to the number of tails, so that $\sum_{t=1}^{T} c_t = 0$, the guaranteed wealth becomes proportional to $\frac{1}{\sqrt{T}}$. So, for T that goes to infinity, the bettor will lose all of their money.

Instead, we need a more conservative strategy that, in the worst case, loses only a constant fraction of the initial wealth. In this case, the betting strategy has to pace its betting, possibly with the knowledge of the duration of the game. At the same time, it must still gain an exponential amount of money when the coin outcomes are biased towards one side.

We will prove that this is possible, designing a new betting strategy, through a simple reduction of the coin-betting problem to an OLO one. First of all, observe that $\ln(1+x) \geq x - x^2$ for $|x| \leq 1/2$. This means that the log wealth of a coin-betting strategy that in each round bets a signed fraction $\beta_t \in [-1/2, 1/2]$ on the continuous coins $c_t \in [-1, 1]$ can be lower bounded as

$$\sum_{t=1}^{T} \ln(1 + c_t\beta_t) \geq \sum_{t=1}^{T} c_t\beta_t - \sum_{t=1}^{T} c_t^2\beta_t^2 \geq \sum_{t=1}^{T} c_t\beta_t - \sum_{t=1}^{T} \beta_t^2 \,. \tag{13.8}$$

This suggests a very simple strategy: run FTRL on the surrogate losses $\ell_t(\beta) = -c_t\beta$, with regularizer $\psi_t(\beta) = (t+a)\beta^2$ for $t \leq T$, $\psi_{T+1} = \psi_T$, and $\mathcal{V} = [-1/2, 1/2]$, where $a > 0$ will be chosen in the following. From Corollary 7.6, we have

$$\sum_{t=1}^{T} (-c_t)(\beta_t - u) \leq (T+a)u^2 + \sum_{t=1}^{T} \frac{c_t^2}{4(t+a)} - \sum_{t=1}^{T-1} \beta_{t+1}^2, \quad \forall u \in [-1/2, 1/2],$$

and $\beta_t = \frac{\sum_{i=1}^{t-1} c_i}{2(t+a)} \in [-1/2, 1/2]$. Reordering, using (13.8) and the fact that $\beta_1 = 0$, we have

$$\begin{aligned} -\sum_{t=1}^{T} \ln(1 + c_t\beta_t) &\leq \sum_{t=1}^{T} (-c_t\beta_t + \beta_t^2) \leq (T+a)u^2 - \sum_{t=1}^{T} c_t u + \sum_{t=1}^{T} \frac{c_t^2}{4(t+a)} \\ &\leq (T+a)u^2 - \sum_{t=1}^{T} c_t u + \frac{T}{4a} \,. \end{aligned}$$

Now, choose $u = \frac{\sum_{t=1}^{T} c_t}{2(T+a)}$, to have

$$\sum_{t=1}^{T} \ln(1 + c_t\beta_t) \geq \frac{1}{4(T+a)} \left(\sum_{t=1}^{T} c_t \right)^2 - \frac{T}{4a} \,.$$

Exponentiating, we have

$$\prod_{t=1}^{T}(1+c_t\beta_t) \geq \exp\left[\frac{1}{4(T+a)}\left(\sum_{t=1}^{T}c_t\right)^2 - \frac{T}{4a}\right] .$$

We can now use this betting strategy in the expert reduction in Theorem 13.14, setting $h(x) = \sqrt{4(T+a)(x+\frac{T}{4a})}$. Hence, choosing for example $a = \frac{T}{4}$, we have

$$\text{Regret}_T(\boldsymbol{u}) \leq L_\infty\sqrt{20T(\text{KL}(\boldsymbol{u};\boldsymbol{\pi})+1)}, \quad \forall \boldsymbol{u} \in \Delta^{d-1} . \tag{13.9}$$

Remark 13.15 *The update we obtain is very similar to the KT ones. The factor 2 comes from the approximation of the logarithm and the fact that FTRL requires bounded gradients, while we saw in Theorem 13.2 that KT is based on an algorithmic framework that allows unbounded gradients and uses a prior over the possible betting fractions. So, the additive factor a can also be obtained in KT by putting more mass around the 0 betting fraction in its prior, but the proof is more complex.*

Note that this betting strategy can also be used in the OCO reduction. Given that we removed the logarithmic term in the exponent, in the one-dimensional case, we obtain the optimal regret of

$$\text{Regret}_T(u) \leq |u|L\sqrt{5T\ln\left(\frac{|u|\sqrt{19T}}{\epsilon}+1\right)} + L\epsilon, \quad \forall u \in \mathbb{R}, \tag{13.10}$$

where we gained in the $\sqrt{T}$ term inside the logarithm, instead of the T term of the KT algorithm. This implies that now we can set ϵ to $\sqrt{T}$ and obtain an asymptotic rate of $\mathcal{O}(\sqrt{T})$ rather than $\mathcal{O}(\sqrt{T\ln T})$ for $T \to \infty$.

13.8 Example: A Parameter-free Algorithm for Sleeping Experts

Consider the setting of learning with sleeping experts from Section 12.2. We will show how to easily design a parameter-free algorithm for this setting.

Consider using KT to solve OLO in $\mathbb{R}$, but we slightly modify KT to ignore the rounds where the gradients are zero:

$$x_t = \frac{-\sum_{j=1}^{t-1}g_j}{L(1+\sum_{j=1}^{t-1}\mathbf{1}\{g_j \neq 0\})}\left(\epsilon - \sum_{j=1}^{t-1}\frac{g_j}{L}x_j\right) .$$

It is easy to show—see Problem 13.4—that this algorithm will guarantee the following regret upper bound:

$$\sum_{t=1}^{T}g_t(x_t-u) \leq |u|L\sqrt{2\sum_{t=1}^{T}\mathbf{1}\{g_t\neq 0\}\ln\left(1+\frac{e|u|\sum_{t=1}^{T}\mathbf{1}\{g_t\neq 0\}}{\epsilon}\right)} + L\epsilon, \ \forall u \in \mathbb{R} .$$

We now use *three* black-box reductions in a row: first, we transform this OLO algorithm

Algorithm 13.5 Sleeping Expert Algorithm based on KT Bettors

Require: Number of experts d, ϵ_i for $i = 1, \dots, d$, $R > 0$

for $t = 1, 2, \dots, T$ **do**

  Set $z_{t,i} = \frac{-\sum_{j=1}^{t-1} \tilde{g}_{j,i}}{R(1+\sum_{j=1}^{t-1} \mathbf{1}\{\tilde{g}_{j,i} \neq 0\})} \left(\epsilon_i - \sum_{j=1}^{t-1} \frac{\tilde{g}_{j,i}}{R} z_{j,i}\right),\ i = 1, \dots, d$

  Receive the state of the experts $a_{t,i} \in \{0, 1\}, i = 1, \dots, d$

  if $\sum_{j=1}^{d} a_{t,j} \max(z_{t,j}, 0) \neq 0$ **then**

    Predict with $x_{t,i} = \frac{a_{t,i} \max(z_{t,i}, 0)}{\sum_{j=1}^{d} a_{t,j} \max(z_{t,j}, 0)}$ for $i = 1, \dots, d$

  else

    Predict with $\boldsymbol{x}_t$ equal to the uniform distribution over the active experts

  end if

  Receive loss vector $\boldsymbol{g}_t \in \mathbb{R}^d$ such that $\max_{i:a_{t,i}=1} g_{t,i} - \min_{i:a_{t,i}=1} g_{t,i} \leq R$

  Set $\hat{g}_{t,i} = a_{t,i}(g_{t,i} - \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle),\ i = 1, \dots, d$

  $\tilde{g}_{t,i} = \begin{cases} \min(\hat{g}_{t,i}, 0), & z_{t,i} < 0 \\ \hat{g}_{t,i}, & z_{t,i} \geq 0 \end{cases},\ i = 1, \dots, d$

end for

for $\mathbb{R}$ to an OLO algorithm for $\mathbb{R}_{\geq 0}$. Second, we use d of such algorithms to produce predictions in $\mathbb{R}^d_{\geq 0}$. Finally, we use the sleeping expert reductions from Section 12.2.

Using a black-box reduction to constrain it to $\mathbb{R}_{\geq 0}$, as in Example 12.9, we have

$$\begin{aligned} \tilde{g}_t &= \begin{cases} \min(g_t, 0), & z_t < 0 \\ g_t, & z_t \geq 0 \end{cases} \\ z_t &= \frac{-\sum_{j=1}^{t-1} \tilde{g}_j}{L_\infty(1 + \sum_{j=1}^{t-1} \mathbf{1}\{\tilde{g}_j \neq 0\})} \left(\epsilon - \sum_{j=1}^{t-1} \frac{\tilde{g}_j}{L_\infty} z_j\right) \\ x_t &= \max(z_t, 0) \end{aligned}$$

and obtain a regret of $uL_\infty \sqrt{2T' \ln(eT'u/\epsilon + 1)} + \epsilon L_\infty$, where T' is the number of times that $\tilde{g}_t \neq 0$. Next, we use this algorithm to produce predictions over $\mathbb{R}^d_{\geq 0}$, using each one-dimensional algorithm on each coordinate, as seen in Section 13.3. So, we obtain a regret of

$$\text{Regret}_T(\boldsymbol{u}) \leq L_\infty \sum_{i=1}^{d} u_i \sqrt{2T'_i \ln(eu_i T'_i/\epsilon_i + 1)} + L_\infty \sum_{i=1}^{d} \epsilon_i, \quad \forall \boldsymbol{u} \in \mathbb{R}^d_{\geq 0}\,.$$

Finally, using the sleeping expert reduction from $\mathbb{R}^d_{\geq 0}$ from Section 12.2, we get Algorithm 13.5.

Given that $a_{t,i} = 0$ implies $\tilde{g}_{t,i} = 0$, the upper bound on the sleeping expert regret of Algorithm 13.5 against any expert i is $R\sqrt{2\sum_{t=1}^{T} a_{t,i} \ln(1 + e\sum_{t=1}^{T} a_{t,i}/\epsilon_i)} + R\sum_{j=1}^{d} \epsilon_j$, that depends on the number of rounds that the expert i was active.

Remark 13.16 *Algorithm 13.5 extends verbatim to a countable set of experts, provided that only finitely many experts are active on each round and the parameters satisfy* $\sum_{i=1}^{\infty} \epsilon_i <$

∞. *In this case, the same proof gives, for* $\boldsymbol{u} \in \mathbb{R}^{\mathbb{N}}_{\geq 0}$,

$$\mathrm{Regret}_T^{\mathrm{sleeping}}(\boldsymbol{u}) \leq R \sum_{i=1}^{\infty} u_i \sqrt{2 \sum_{t=1}^{T} a_{t,i} \ln \left(1 + u_i e \sum_{t=1}^{T} a_{t,i}/\epsilon_i\right)} + R \sum_{i=1}^{\infty} \epsilon_i \, .$$

13.9 From Portfolio Selection to Continuous-Coin Betting

Consider the online coin-betting problem again. We said that it corresponds to $\mathcal{V} = [-1, 1]$ and $\ell_t(x) = -\ln(1 + c_t x)$, where the coin outcome $c_t \in \{-1, 1\}$. We also considered an extension of this problem, the *continuous* coin-betting problem, where $c_t \in [-1, 1]$. Now, we show how to reduce the continuous coin-betting problem to the portfolio selection problem we saw in Chapter 10.

We consider portfolio selection with 2 stocks, and we set the market gains to $w_{t,1} = 1 + c_t$ and $w_{t,2} = 1 - c_t$. Note that $w_{t,1}, w_{t,2} \geq 0$, so they are legal market gains. Let $[x_t, 1 - x_t]$ be the play of a 2-stock portfolio algorithm, where $0 \leq x_t \leq 1$. Then, using $\beta_t = 2x_t - 1$ as the signed betting fraction of a continuous-coin-betting algorithm on c_t ensures that the gain in the coin-betting problem coincides with the gain in the portfolio selection problem.

The proof is immediate. We have

$$\begin{aligned} w_{t,1} x_t + w_{t,2}(1 - x_t) &= x_t + x_t c_t + (1 - c_t)(1 - x_t) \\ &= x_t + x_t c_t + 1 - x_t - c_t + c_t x_t = 1 + c_t \beta_t, \end{aligned}$$

where β_t is the signed betting fraction equal to $2x_t - 1$. Given that $x_t \in [0, 1]$, the range of the betting fractions is in $[-1, 1]$ as we wanted.

This implies that we can use the above reduction to transform any portfolio algorithm with 2 stocks into an algorithm for continuous-coin betting. In turn, given what we said in this chapter, this means that portfolio algorithms with 2 stocks can be used for OCO. This does not make KT and the optimal algorithm in Section 13.7 superfluous because the update rule of the universal portfolio algorithm, even with 2 stocks, is still linear in the iteration number.

13.10 History Bits

The keyword "parameter-free" was introduced in Chaudhuri et al. (2009) for a similar strategy for the LEA problem. It is now used as an umbrella term for all online algorithms that guarantee the optimal regret uniformly over the competitor class. Another less-used name to denote the same property is "comparator-adaptive" (e.g., van der Hoeven et al., 2020), but in my opinion, it is a misnomer because it implicitly assumes the existence of an optimal comparator, which is false in the unbounded OLO setting. Note that, given the allure of the name "parameter-free", the term has been adopted in many other domains, with many different meanings. However, when used in the online learning literature, it has *only* the meaning in Definition 13.7. This means that in this literature "parameter-free" algorithms might still require the knowledge of other characteristics of the problem, for example, the Lipschitz constant of the losses or the number of rounds. This is fine: it is a technical term, and we can assign to it any definition we like, as for "smooth" or "universal".

The first algorithm for 1-d parameter-free OCO is from Streeter and McMahan (2012), but the bound was suboptimal. The algorithm was then extended to Hilbert spaces in Orabona (2013), still with a suboptimal bound. The optimal bound in Hilbert space, matching the one in (13.10), was obtained in McMahan and Orabona (2014). This variant allows us to remove the factor T in the logarithm by setting $\epsilon = \mathcal{O}(\sqrt{T})$, at the price of having a $\mathcal{O}(\sqrt{T})$ regret versus $\boldsymbol{u} = \mathbf{0}$. The optimal constant for this kind of guarantee is obtained in Zhang et al. (2022) through a PDE-based analysis, proving also a matching lower bound.

The idea of using coin betting to do parameter-free OCO was introduced in Orabona and Pál (2016). A different reduction from a betting algorithm to the specific case of linear regression with square loss was proposed by Vovk (2006). The KT algorithm is from Krichevsky and Trofimov (1981) and its extension to the "continuous coin" is from Orabona and Pál (2016). The regret bound of the KT algorithm is optimal up to an additive constant because its regret upper bound matches the one of the minimax strategy for known T, the so-called *normalized maximum likelihood strategy* (Shtar'kov, 1987), up to an additive constant. The regret/reward duality relationship was proved for the first time in McMahan and Orabona (2014). Theorem 13.4 is new, and it is an improved version of a guarantee with a worse constant in Orabona and Pál (2016). There are also more refined betting algorithms that allow for obtaining parameter-free OCO algorithms that depend on the sum of the squared norms of the subgradients, rather than time (Cutkosky and Orabona, 2018), also in a scale-free way (Mhammedi and Koolen, 2020). Recently, parameter-free algorithms have also been extended to some non-convex functions in the stochastic setting (Orabona and Pál, 2021).

The approach of using a coordinate-wise version of the coin-betting algorithm was proposed in the first paper on parameter-free OLO in Streeter and McMahan (2012). Recently, the same approach with a special coin-betting algorithm was also used for optimization of deep neural networks (Orabona and Tommasi, 2017). The idea of combining two parameter-free OLO algorithms to obtain the best of the two guarantees is from Cutkosky (2019b).

Orabona and Pál (2016) proposed a different way to transform a coin-betting algorithm into an OCO algorithm that works in $\mathbb{R}^d$ or even in Hilbert spaces. However, that approach seems to work only for the L_2 norm, and it is not a black-box reduction. That said, the reduction in Orabona and Pál (2016) seems to have a better empirical performance compared to the one in Theorem 12.16.

The first parameter-free algorithm for experts is from Chaudhuri et al. (2009), named NormalHedge, where they introduced the concept of ϵ-quantile regret and obtained a bound of $\mathcal{O}(\sqrt{(T + \ln^2 d)(1 + \ln \frac{1}{\epsilon})})$ as $T \to \infty$, but without a closed-form update. Then, Chernov and Vovk (2010) removed the spurious dependence on d, again with an update without a closed form. Orabona and Pál (2016) showed that this guarantee can be efficiently obtained through the novel reduction to coin betting in Theorem 13.14. This kind of regret guarantee can be improved to depend on the sum of the squared losses rather than on time, but with an additional $\ln \ln T$ factor, as in the Squint algorithm (Koolen and van Erven, 2015). It is worth noting that the Squint algorithm can be interpreted exactly as a coin-betting algorithm plus a variant of the reduction in Theorem 13.14. The first lower bound for the ϵ-quantile regret is proved in Negrea et al. (2021).

The betting strategy in Section 13.7 is new, and it is a simplification of the methods in

Chen et al. (2022a) and in Orabona and Pál (2021). The arXiv version of this book (Orabona, 2019) contained yet another derivation, obtained from a distillation of the ideas of the "shifted-KT" potentials in Orabona and Pál (2016), yet more complex than the presented one. See also Orabona (2026a) for an explanation of the connection between shifted-KT potentials and gambling priors centered around the origin.

The variant of KT in Section 13.8 that updates only when $g_t \neq 0$ appeared for the first time in Chen et al. (2022b).

The reduction in Section 13.9 is folklore, and it appears in Orabona and Jun (2024).

Recently, coin-betting algorithms have found numerous applications, such as the design of parameter-free online multi-task learning algorithms (Denevi et al., 2020), parameter-free particle-based algorithms (Sharrock and Nemeth, 2023; Sharrock et al., 2023a,b), and the derivation of time-uniform concentration inequalities (see Chapter 16).

13.11 Exercises

Problem 13.1 *Define* $F_t(x) := \frac{2^t \Gamma\left(\frac{t+1+x}{2}\right)\Gamma\left(\frac{t+1-x}{2}\right)}{\pi\Gamma(t+1)}$ *for* $|x| \leq t$. *Prove that* $F_t(x+c) = F_{t-1}(x)(1+cx/t)$ *for all* $x \in [-(t-1), t-1]$ *and* $c \in \{-1, 1\}$.

Problem 13.2 *Prove that* $\ell_t(x) = -\ln(1 + z_t\, x)$ *with* $\mathcal{V} = \{x \in \mathbb{R} : |x| \leq 1/2\}$ *and* $|z_t| \leq 1,\ t = 1, \dots, T$ *are exp-concave. Then, using the Online Newton Step (ONS) algorithm, give an algorithm and a regret bound for a game with these losses. Finally, show a wealth guarantee of the corresponding coin-betting strategy. Hint: see Cutkosky and Orabona (2018).*

Problem 13.3 *Using the same proof technique in the section, find a betting strategy whose wealth depends on* $\sum_{t=1}^T |c_t|$ *rather than on* T.

Problem 13.4 *Consider the variant of KT for OLO that ignores the rounds where the* $g_t = 0$, *by predicting with* $x_t = \frac{-\sum_{j=1}^{t-1} g_j}{1+\sum_{j=1}^{t-1} \mathbf{1}\{g_j \neq 0\}} \left(\epsilon - \sum_{j=1}^{t-1} g_j x_j\right)$, *where each* $|g_t| \leq 1$. *Prove that it guarantees a regret of* $\mathcal{O}(|u|\sqrt{\sum_{t=1}^T \mathbf{1}\{g_t \neq 0\} \ln \frac{1+|u|\sum_{t=1}^T \mathbf{1}\{g_t \neq 0\}}{\epsilon}} + \epsilon)$ *for any* $u \in \mathbb{R}$ *and the linear losses* $\ell_t(x) = g_t x$.

Problem 13.5 *We want to design a one-dimensional FTRL parameter-free algorithm, just using Lemma 7.1. Let* $\epsilon > 0$, *set the number of rounds* $T \geq 4$, $\mathcal{V} = \mathbb{R}_{\geq 0}$, *and let* $\psi_t(x) = \sqrt{T} x \ln \frac{x\sqrt{T}}{\epsilon} + \frac{t-1}{\sqrt{T}} x - x\sqrt{T}$. *Show that the regret of this algorithm on linear losses* $\ell_t(x) = g_t x$, *where* $|g_t| \leq 1$ *is upper bounded by* $u\sqrt{T} \ln \frac{u\sqrt{T}}{\epsilon} + \epsilon$ *for all* $u \in \mathbb{R}_{\geq 0}$. *Hint: You might need the elementary inequality* $1 + x \geq \exp(x - x^2)$ *for all* $|x| \leq 1/2$.

14

Dynamic, Strongly Adaptive, and Tracking Regret

We saw that the definition of regret makes sense as a direct generalization of both the stochastic setting and the offline optimization. However, in some cases, we want to guarantee something stronger, that is, we would like the algorithm to have small regret with respect to multiple comparators. There are multiple ways to formalize this idea, and we will see three of these frameworks: *Dynamic*, *Strongly Adaptive*, and *Tracking* regret.

14.1 Dynamic Regret

Our first extension is to use multiple comparators, using the concept of *dynamic regret*.

Definition 14.1 *The **dynamic regret** is defined as*

$$\text{D-Regret}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) := \sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}_t),$$

where $\boldsymbol{u}_1, \dots, \boldsymbol{u}_T \in \mathcal{V}$ and $\mathcal{V} \subseteq \mathbb{R}^d$ is the feasible set.

For additional clarity, we will also refer to the standard notion of regret as the *static regret*.

The use of a sequence of competitors means that now our class of competitors is strictly larger than in the static case, so the cumulative loss of the competitors could be smaller. In turn, this means that the average loss of an algorithm with sublinear dynamic regret can converge to a potentially smaller average loss with respect to the static case. In this view, guarantees on the dynamic regret are strictly stronger than guarantees on the static regret.

Is it always possible to obtain sublinear dynamic regret? We already know that in the case that $\boldsymbol{u}_1 = \cdots = \boldsymbol{u}_T$ it is possible, because we recover the standard regret case. On the other hand, it should be intuitive that the problem becomes more and more difficult the more the sequence of $\boldsymbol{u}_t$ changes over time. There are various ways to quantify this complexity, and a common one is the **path length** $P^\phi : \mathcal{V}^T \to \mathbb{R}_{\geq 0}$ defined as

$$P^\phi(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) := \sum_{t=2}^{T} \phi(\boldsymbol{u}_t - \boldsymbol{u}_{t-1}),$$

where $\phi : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$ is a function that measures the shift from $\boldsymbol{u}_{t-1}$ to $\boldsymbol{u}_t$. In particular, we will instantiate ϕ to be a norm. In the following, we will see how to design an online algorithm whose dynamic regret depends on the path length in the case that the feasible set is bounded.

14.1.1 Dynamic Regret of Online Mirror Descent

It turns out that some online learning algorithms already satisfy a dynamic regret bound without any additional change. For Online Mirror Descent (OMD), we can state the following theorem.

Theorem 14.2 *Let B_ψ be the Bregman divergence with respect to $\psi : \mathcal{X} \to \mathbb{R}$ and assume ψ to be closed and λ-strongly convex with respect to $\|\cdot\|$ in $\mathcal{V} \cap \operatorname{int} \mathcal{X}$, where $\mathcal{V} \subseteq \mathcal{X}$ is a non-empty closed convex set. Assume that $\boldsymbol{x}_t \in \operatorname{int} \mathcal{X}$ for $t = 1, \dots, T+1$. Then, $\forall \boldsymbol{u}_1, \dots, \boldsymbol{u}_T \in \mathcal{V}$, OMD with constant learning rate η in Algorithm 6.1 satisfies*

$$\begin{aligned}\text{D-Regret}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) \le \frac{B_\psi(\boldsymbol{u}_T; \boldsymbol{x}_1) + QP^{\|\cdot\|}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T)}{\eta} + \frac{\eta}{2\lambda} \sum_{t=1}^T \|\boldsymbol{g}_t\|_\star^2 \\ - \frac{B_\psi(\boldsymbol{u}_T; \boldsymbol{x}_{T+1})}{\eta},\end{aligned}$$

where $Q = \max_{t=1,\dots,T} \|\nabla\psi(\boldsymbol{x}_t) - \nabla\psi(\boldsymbol{x}_1)\|_\star$.

Proof From the one-step inequality for OMD with the competitor $\boldsymbol{u}_t$ in Lemma 6.10, we have

$$\begin{aligned}\eta(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u}_t)) &\le B_\psi(\boldsymbol{u}_t; \boldsymbol{x}_t) - B_\psi(\boldsymbol{u}_t; \boldsymbol{x}_{t+1}) + \frac{\eta^2}{2\lambda}\|\boldsymbol{g}_t\|_\star^2 \\ &= B_\psi(\boldsymbol{u}_t; \boldsymbol{x}_t) - B_\psi(\boldsymbol{u}_{t+1}; \boldsymbol{x}_{t+1}) + B_\psi(\boldsymbol{u}_{t+1}; \boldsymbol{x}_{t+1}) - B_\psi(\boldsymbol{u}_t; \boldsymbol{x}_{t+1}) \\ &\quad + \frac{\eta^2}{2\lambda}\|\boldsymbol{g}_t\|_\star^2 .\end{aligned}$$

Dividing by η and summing over time, we have

$$\begin{aligned}\sum_{t=1}^T (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u}_t)) &\le \sum_{t=1}^T \frac{B_\psi(\boldsymbol{u}_t; \boldsymbol{x}_t) - B_\psi(\boldsymbol{u}_{t+1}; \boldsymbol{x}_{t+1})}{\eta} \\ &\quad + \sum_{t=1}^T \frac{B_\psi(\boldsymbol{u}_{t+1}; \boldsymbol{x}_{t+1}) - B_\psi(\boldsymbol{u}_t; \boldsymbol{x}_{t+1})}{\eta} + \sum_{t=1}^T \frac{\eta}{2\lambda}\|\boldsymbol{g}_t\|_\star^2 \\ &= \frac{B_\psi(\boldsymbol{u}_1; \boldsymbol{x}_1) - B_\psi(\boldsymbol{u}_{T+1}; \boldsymbol{x}_{T+1})}{\eta} \\ &\quad + \sum_{t=1}^T \frac{B_\psi(\boldsymbol{u}_{t+1}; \boldsymbol{x}_{t+1}) - B_\psi(\boldsymbol{u}_t; \boldsymbol{x}_{t+1})}{\eta} + \frac{\eta}{2\lambda} \sum_{t=1}^T \|\boldsymbol{g}_t\|_\star^2 .\end{aligned}$$

Now, observe that

$$\begin{aligned}&B_\psi(\boldsymbol{u}_{t+1}; \boldsymbol{x}_{t+1}) - B_\psi(\boldsymbol{u}_t; \boldsymbol{x}_{t+1}) \\ &\quad = \psi(\boldsymbol{u}_{t+1}) - \psi(\boldsymbol{x}_{t+1}) - \langle \nabla\psi(\boldsymbol{x}_{t+1}), \boldsymbol{u}_{t+1} - \boldsymbol{x}_{t+1}\rangle - \psi(\boldsymbol{u}_t) + \psi(\boldsymbol{x}_{t+1}) \\ &\qquad + \langle \nabla\psi(\boldsymbol{x}_{t+1}), \boldsymbol{u}_t - \boldsymbol{x}_{t+1}\rangle \\ &\quad = \psi(\boldsymbol{u}_{t+1}) - \psi(\boldsymbol{u}_t) + \langle \nabla\psi(\boldsymbol{x}_1), \boldsymbol{u}_t - \boldsymbol{u}_{t+1}\rangle + \langle \nabla\psi(\boldsymbol{x}_{t+1}) - \nabla\psi(\boldsymbol{x}_1), \boldsymbol{u}_t - \boldsymbol{u}_{t+1}\rangle.\end{aligned}$$

Hence, we have

$$\begin{aligned}
&\sum_{t=1}^{T}(B_\psi(\boldsymbol{u}_{t+1};\boldsymbol{x}_{t+1}) - B_\psi(\boldsymbol{u}_t;\boldsymbol{x}_{t+1})) \\
&\quad = \psi(\boldsymbol{u}_{T+1}) - \psi(\boldsymbol{u}_1) + \langle \nabla\psi(\boldsymbol{x}_1), \boldsymbol{u}_1 - \boldsymbol{u}_{T+1}\rangle \\
&\qquad + \sum_{t=1}^{T}\langle \nabla\psi(\boldsymbol{x}_{t+1}) - \nabla\psi(\boldsymbol{x}_1), \boldsymbol{u}_t - \boldsymbol{u}_{t+1}\rangle \\
&\quad = B_\psi(\boldsymbol{u}_{T+1};\boldsymbol{x}_1) - B_\psi(\boldsymbol{u}_1;\boldsymbol{x}_1) + \sum_{t=1}^{T}\langle \nabla\psi(\boldsymbol{x}_{t+1}) - \nabla\psi(\boldsymbol{x}_1), \boldsymbol{u}_t - \boldsymbol{u}_{t+1}\rangle \,.
\end{aligned}$$

Putting everything together, we have

$$\begin{aligned}
\text{D-Regret}(\boldsymbol{u}_1,\ldots,\boldsymbol{u}_T) \le \frac{B_\psi(\boldsymbol{u}_{T+1};\boldsymbol{x}_1) - B_\psi(\boldsymbol{u}_{T+1};\boldsymbol{x}_{T+1})}{\eta} + \frac{\eta}{2\lambda}\sum_{t=1}^{T}\|\boldsymbol{g}_t\|_\star^2 \\
+ \frac{1}{\eta}\sum_{t=1}^{T}\langle \nabla\psi(\boldsymbol{x}_{t+1}) - \nabla\psi(\boldsymbol{x}_1), \boldsymbol{u}_t - \boldsymbol{u}_{t+1}\rangle \,.
\end{aligned}$$

Observe that the choice of $\boldsymbol{u}_{T+1}$ does not affect the l.h.s. of the inequality, so we can set it equal to $\boldsymbol{u}_T$. Then, assuming that $\|\nabla\psi(\boldsymbol{x}_t) - \nabla\psi(\boldsymbol{x}_1)\|_\star \le Q$ and using the definition of dual norm, we have the stated bound. □

Assuming $\|\boldsymbol{g}_t\|_\star \le L$ and selecting the usual learning rate $\eta = \frac{\alpha\sqrt{2\lambda}}{L\sqrt{T}}$, we have

$$\text{D-Regret}(\boldsymbol{u}_1,\ldots,\boldsymbol{u}_T) \le \frac{L}{\sqrt{2\lambda}}\left(\frac{B_\psi(\boldsymbol{u}_T;\boldsymbol{x}_1) + QP^{\|\cdot\|}(\boldsymbol{u}_1,\ldots,\boldsymbol{u}_T)}{\alpha} + \alpha\right)\sqrt{T} \,.$$

In other words, we suffer an additional regret of

$$L\frac{QP^{\|\cdot\|}(\boldsymbol{u}_1,\ldots,\boldsymbol{u}_T)}{\alpha\sqrt{2\lambda}}\sqrt{T}$$

compared to the static case, for any $\boldsymbol{u}_1,\ldots,\boldsymbol{u}_T \in \mathcal{V}$.

Example 14.3 *Consider the case that the feasible set has diameter D with respect to the L_2 norm, i.e., $D = \max_{\boldsymbol{x},\boldsymbol{y}\in\mathcal{V}}\|\boldsymbol{x}-\boldsymbol{y}\|_2 < \infty$. Set $\psi(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{x}\|_2^2$ and assume that the subgradients satisfy $\|\boldsymbol{g}_t\|_\star \le L$ for $t = 1,\ldots,T$. In this case, we have that $Q \le D$ and setting $\eta = \frac{\alpha}{L\sqrt{T}}$ we have*

$$\text{D-Regret}(\boldsymbol{u}_1,\ldots,\boldsymbol{u}_T) \le \frac{1}{2}\left(\frac{D^2 + 2D\,P^{\|\cdot\|_2}(\boldsymbol{u}_1,\ldots,\boldsymbol{u}_T)}{\alpha} + \alpha\right)L\sqrt{T} \,.$$

This means that we get sublinear regret with respect to any competitor sequence whose path length is $o(\sqrt{T})$ as T goes to infinity.

Could we obtain a better regret guarantee? Setting $\eta = \frac{\sqrt{2\lambda}\sqrt{B_\psi(\boldsymbol{u}_T;\boldsymbol{x}_1)+QP^{\|\cdot\|}(\boldsymbol{u}_1,\ldots,\boldsymbol{u}_T)}}{L\sqrt{T}}$, we could obtain the dynamic regret

$$\text{D-Regret}(\boldsymbol{u}_1,\ldots,\boldsymbol{u}_T) \le \frac{\sqrt{2}}{\sqrt{\lambda}}L\sqrt{B_\psi(\boldsymbol{u}_T;\boldsymbol{x}_1) + QP^{\|\cdot\|}(\boldsymbol{u}_1,\ldots,\boldsymbol{u}_T)}\sqrt{T} \,.$$

This would give us a sublinear regret with respect to any competitor sequence whose path length is $o(T)$ as T goes to infinity, which is a larger set than the one in the example. However, assuming knowledge of the exact value of $P^{\|\cdot\|}$ is unreasonable because it violates the assumption of the adversarial nature of the problem. This mirrors the problem of tuning the learning rate with knowledge of $\|\boldsymbol{x}_1 - \boldsymbol{u}\|$ in Online Subgradient Descent (OSD). Fortunately, there is a solution, as we will see in the next section.

14.1.2 ADER: Optimal Dependence on the Path Length for Online Subgradient Descent

In the previous section, we saw that the improved bound requires the knowledge of the path length of the competitor sequence. Here, we show how to construct an online learning algorithm that achieves the same guarantees up to polylogarithmic terms. For simplicity, we will consider the Euclidean case, but it is easy to extend it with Bregman divergences. Moreover, we assume the losses to be L-Lipschitz with respect to the L_2 norm.

We will use an approach similar to the one in Section 6.10: running a number of projected OSD algorithms with different learning rates in parallel and using Exponentiated Gradient (EG) to learn online the best combination of their iterates. In this way, we will see that the cumulative loss of the resulting algorithm is close to the cumulative loss of the best learning rate, which in turn will give us the right dependence on the path length.

Consider a feasible set $\mathcal{V}$ with bounded L_2 diameter D and assume the losses ℓ_t to be L-Lipschitz on an open set containing $\mathcal{V}$. Using projected OSD with learning rate η, for all $\boldsymbol{u}_1, \dots, \boldsymbol{u}_T \in \mathcal{V}$, we obtain the following regret upper bound

$$\text{D-Regret}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) \leq \frac{D^2 + 2D\, P^{\|\cdot\|_2}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T)}{2\eta} + \frac{\eta L^2 T}{2} \,.$$

Using the fact that $P^{\|\cdot\|_2}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) \leq DT$, the choice of $\eta^\star = \frac{\sqrt{D^2 + 2D\, P^{\|\cdot\|_2}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T)}}{L\sqrt{T}}$ satisfies

$$\frac{D}{L\sqrt{T}} \leq \eta^\star \leq \frac{D\sqrt{1+2T}}{L\sqrt{T}} \,.$$

So, consider a grid of $N = 1 + \lceil \ln_2 \sqrt{4 + 8T} \rceil$ learning rates $\eta^{(i)} = \frac{D 2^{i-1}}{L\sqrt{T}}, i = 1, \dots, N$, so that $\eta^{(1)} = \frac{D}{L\sqrt{T}}$ and $\eta^{(N)} \geq 2\frac{D\sqrt{1+2T}}{L\sqrt{T}}$. This implies that there exists $i^\star$ such that

$$\eta^{(i^\star)} \leq \eta^\star \leq 2\eta^{(i^\star)} \,.$$

To combine the OSD algorithms with different learning rates, we use the EG algorithm where the OSD algorithms are our experts. Construct the loss vector for EG as $\boldsymbol{z}_t = [\ell_t(\boldsymbol{x}_t^{(1)}), \dots, \ell_t(\boldsymbol{x}_t^{(N)})]^\top$. The EG algorithm is invariant to adding the same constant to all coordinates of the loss vector (see Problem 6.10), hence we can consider it as running on the losses $\ell_t(\boldsymbol{x}_t^{(i)}) - \ell_t(\boldsymbol{x}_1)$ which makes them bounded in absolute value by DL. Hence,

using a uniform prior and learning rate $\beta = \frac{\sqrt{2\ln N}}{LD\sqrt{T}}$, we obtain the regret

$$\sum_{t=1}^{T}\langle \boldsymbol{p}_t, \boldsymbol{z}_t\rangle - \sum_{t=1}^{T} z_{t,i^\star} \leq \sqrt{2}L\,D\sqrt{T\ln N} \leq \sqrt{2}L\,D\sqrt{T\ln\left(2+\ln_2\sqrt{4+8T}\right)}\,.$$

Now, observe that by Jensen's inequality (Theorem 2.9) and the convexity of ℓ_t, we have

$$\langle \boldsymbol{p}_t, \boldsymbol{z}_t\rangle = \sum_{i=1}^{N} p_{t,i}\ell_t(\boldsymbol{x}_t^{(i)}) \geq \ell_t\left(\sum_{i=1}^{N} p_{t,i}\boldsymbol{x}_t^{(i)}\right) = \ell_t(\boldsymbol{x}_t)\,.$$

This motivates the choice of using a convex combination of the predictions of the OSD algorithms. Moreover, we have $\sum_{t=1}^{T} z_{t,i^\star} = \sum_{t=1}^{T}\ell_t(\boldsymbol{x}_t^{(i^\star)})$.

From the regret of OSD and the choice of $\eta^{(i^\star)}$, for all $\boldsymbol{u}_1,\dots,\boldsymbol{u}_T \in \mathcal{V}$, we have

$$\sum_{t=1}^{T}\ell_t(\boldsymbol{x}_t^{(i^\star)}) \leq \sum_{t=1}^{T}\ell_t(\boldsymbol{u}_t) + \frac{3}{2}L\sqrt{T(D^2 + 2D\,P^{\|\cdot\|_2}(\boldsymbol{u}_1,\dots,\boldsymbol{u}_T))}\,.$$

Putting everything together, for all $\boldsymbol{u}_1,\dots,\boldsymbol{u}_T \in \mathcal{V}$, we have

$$\begin{aligned}
&\text{D-Regret}(\boldsymbol{u}_1,\dots,\boldsymbol{u}_T)\\
&\quad= \sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u}_t))\\
&\quad= \sum_{t=1}^{T}\left(\ell_t(\boldsymbol{x}_t) - \ell_t\left(\boldsymbol{x}_t^{(i^\star)}\right)\right) + \sum_{t=1}^{T}\left(\ell_t\left(\boldsymbol{x}_t^{(i^\star)}\right) - \ell_t(\boldsymbol{u}_t)\right)\\
&\quad\leq \sqrt{2}LD\sqrt{T\ln\left(2+\ln_2\sqrt{4+8T}\right)} + \frac{3}{2}L\sqrt{T(D^2 + 2D\,P^{\|\cdot\|_2}(\boldsymbol{u}_1,\dots,\boldsymbol{u}_T))}\,.
\end{aligned}$$

We are still not completely done: this construction queries $N = \mathcal{O}(\ln T)$ subgradients in each step, so now we show how to reduce it to only one subgradient per iteration. This is easily achieved: it is enough to run the construction on the linear surrogate losses $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle$, where $\boldsymbol{g}_t \in \partial\ell_t(\boldsymbol{x}_t)$. The advantage is that $\partial\tilde{\ell}_t(\boldsymbol{x}) = \{\boldsymbol{g}_t\}$ for all $\boldsymbol{x}$, hence all the OSD algorithms receive the same subgradient! Once again, we have that $|\tilde{\ell}_t(\boldsymbol{x}_t^{(i)}) - \tilde{\ell}_t(\boldsymbol{x}_1)| \leq L\,D$ for all $i = 1,\dots,N$. Moreover, we have

$$\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u}_t) \leq \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}_t\rangle = \tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{u}_t), \quad \forall \boldsymbol{u}_t \in \mathcal{V}\,.$$

Hence, a dynamic regret guarantee on $\tilde{\ell}_t$ immediately translates to a dynamic regret guarantee on ℓ_t. The resulting algorithm is called **Adaptive learning for Dynamic EnviRonment (ADER)** and it is in Algorithm 14.1.

Formally, we have the following theorem.

Theorem 14.4 *Let $\mathcal{V} \subset \mathbb{R}^d$ be a non-empty closed convex set with bounded diameter with respect to the L_2 norm equal to D. Assume $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ to be convex functions that are subdifferentiable on $\mathcal{V}$ and L-Lipschitz with respect to the L_2 norm on an open set*

Algorithm 14.1 Adaptive learning for Dynamic EnviRonment (ADER)

Require: Non-empty, closed, and convex feasible set $\mathcal{V} \subset \mathbb{R}^d$, number of iterations T, diameter of the feasible set $D < \infty$, Lipschitz constant L, initial point $\boldsymbol{x}_1 \in \mathcal{V}$

Set $N = 1 + \lceil \ln_2(\sqrt{4+8T}) \rceil$
Set $\eta^{(i)} = \frac{D2^{i-1}}{L\sqrt{T}}, i = 1, \dots, N$
Set $\beta = \frac{\sqrt{2 \ln N}}{LD\sqrt{T}}$
$\boldsymbol{p}_1 = [1/N, \dots, 1/N] \in \mathbb{R}^N$
$\boldsymbol{x}_1^{(i)} = \boldsymbol{x}_1, i = 1, \dots, N$
for $t = 1$ **to** T **do**
    Output $\boldsymbol{x}_t = \sum_{i=1}^N p_{t,i} \boldsymbol{x}_t^{(i)}$ and pay $\ell_t(\boldsymbol{x}_t)$
    Receive $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$
    Update each OSD: $\boldsymbol{x}_{t+1}^{(i)} = \Pi_{\mathcal{V}}(\boldsymbol{x}_t^{(i)} - \eta^{(i)} \boldsymbol{g}_t),\ i = 1, \dots, N$
    Update EG: $p_{t+1,i} \propto p_{t,i} \exp\left(-\beta \langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(i)} \rangle\right),\ i = 1, \dots, N$
end for

containing $\mathcal{V}$. Then, for all $\boldsymbol{u}_1, \dots, \boldsymbol{u}_T \in \mathcal{V}$, Algorithm 14.1 satisfies

$$\text{D-Regret}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) \leq \frac{3}{2} L \sqrt{T(D^2 + 2D\, P^{\|\cdot\|_2}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T))} + \sqrt{2} LD \sqrt{T \ln\left(2 + \ln_2 \sqrt{4+8T}\right)}.$$

Note that while we query only one subgradient per round, the computational complexity of ADER per round is still $\mathcal{O}(\ln T)$ when T goes to infinity. This is due to the fact that in each round we need to update $\mathcal{O}(\ln T)$ different OSD algorithms.

14.1.3 Black-box Reduction for Dynamic Regret in Unbounded Feasible Sets

In the previous section, we showed how to obtain the optimal dynamic regret guarantee in bounded feasible sets. Here, we show a black-box reduction to obtain dynamic regret in unconstrained feasible sets, by using the black-box reduction to magnitude and direction of Section 12.3.

Theorem 14.5 *Denote by $\text{D-Regret}_T^{\mathscr{A}_B}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T)$ the linear dynamic regret of algorithm $\mathscr{A}_B$ for any $\boldsymbol{u}_1, \dots, \boldsymbol{u}_T$ in the unit ball with respect to a norm $\|\cdot\|$, and $\text{Regret}_T^{\mathscr{A}_{1d}}(u)$ the linear regret of algorithm $\mathscr{A}_{1d}$ for any competitor $u \in \mathbb{R}$. Then, for any sequence of convex losses $\ell_t : \mathbb{R}^d \to \mathbb{R}$ subdifferentiable in $\mathbb{R}^d$ for $t = 1, \dots, T$ and any $\boldsymbol{u}_1, \dots, \boldsymbol{u}_T \in \mathbb{R}^d$, Algorithm 12.3 guarantees regret*

$$\begin{aligned}\text{D-Regret}_T(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) &= \sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}_t) \leq \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}_t \rangle \\ &= \text{Regret}_T^{\mathscr{A}_{1d}}(U) + U \cdot \text{D-Regret}_T^{\mathscr{A}_B}\left(\frac{\boldsymbol{u}_1}{U}, \dots, \frac{\boldsymbol{u}_T}{U}\right),\end{aligned}$$

where $U := \max_{t=1,\dots,T} \|\boldsymbol{u}_t\|$ *and* $U \neq 0$. *If* $U = 0$, *then*

$$\text{D-Regret}_T(\mathbf{0},\dots,\mathbf{0}) = \text{Regret}_T(\mathbf{0}) \leq \text{Regret}_T^{\mathscr{A}_{1d}}(0)\ .$$

Further, the subgradients s_t *sent to* $\mathscr{A}_{1d}$ *satisfy* $|s_t| \leq \|\boldsymbol{g}_t\|_\star$.

Proof First, observe that $|s_t| \leq \|\boldsymbol{g}_t\|_\star \|\tilde{\boldsymbol{x}}_t\| \leq \|\boldsymbol{g}_t\|_\star$ since $\|\tilde{\boldsymbol{x}}_t\| \leq 1$ for all t. Now, assuming $U \neq 0$ compute:

$$\begin{aligned}
\text{D-Regret}_T(\boldsymbol{u}_1,\dots,\boldsymbol{u}_T) &= \sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{u}_t) \\
&\leq \sum_{t=1}^T \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}_t\rangle = \sum_{t=1}^T \langle \boldsymbol{g}_t, z_t\tilde{\boldsymbol{x}}_t\rangle - \langle \boldsymbol{g}_t, \boldsymbol{u}_t\rangle \\
&= \underbrace{\sum_{t=1}^T (\langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_t\rangle z_t - \langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_t\rangle U)}_{\text{linear regret of } \mathscr{A}_{1\text{d}} \text{ at } U \in \mathbb{R}} + \sum_{t=1}^T (\langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_t\rangle U - \langle \boldsymbol{g}_t, \boldsymbol{u}_t\rangle) \\
&= \text{Regret}_T^{\mathscr{A}_{1\text{d}}}(U) + \sum_{t=1}^T (\langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_t\rangle U - \langle \boldsymbol{g}_t, \boldsymbol{u}_t\rangle) \\
&= \text{Regret}_T^{\mathscr{A}_{1\text{d}}}(U) + U \underbrace{\sum_{t=1}^T \left(\langle \boldsymbol{g}_t, \tilde{\boldsymbol{x}}_t\rangle - \left\langle \boldsymbol{g}_t, \frac{\boldsymbol{u}_t}{U}\right\rangle\right)}_{\text{dynamic regret of } \mathscr{A}_B} \\
&= \text{Regret}_T^{\mathscr{A}_{1\text{d}}}(U) + U \cdot \text{D-Regret}_T^{\mathscr{A}_B}\left(\frac{\boldsymbol{u}_1}{U},\dots,\frac{\boldsymbol{u}_T}{U}\right)\ .
\end{aligned}$$

The case $U = 0$ is similarly proven. □

Example 14.6 *We can use the ADER algorithm in* $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \leq 1\}$ *as our direction learner and the parameter-free algorithm for Online Convex Optimization (OCO) in Section 13.7 as our magnitude learner. We assume that each* ℓ_t *is* L*-Lipschitz with respect to the* L_2 *norm on* $\mathbb{R}^d$. *In this way, for all* $\boldsymbol{u}_1,\dots,\boldsymbol{u}_T \in \mathbb{R}^d$, *we obtain*

$$\begin{aligned}
\text{D-Regret}_T(\boldsymbol{u}_1,\dots,\boldsymbol{u}_T) \leq\ & UL\sqrt{5T\ln\left(\frac{U\sqrt{19T}}{\epsilon}+1\right)} + \epsilon L \\
&+ 2UL\sqrt{2T\ln\left(2+\ln_2\sqrt{4+8T}\right)} \\
&+ 3UL\sqrt{T\left(1+P^{\|\cdot\|_2}\left(\frac{\boldsymbol{u}_1}{U},\dots,\frac{\boldsymbol{u}_T}{U}\right)\right)},
\end{aligned}$$

where $U = \max_{t=1,\dots,T} \|\boldsymbol{u}_t\|_2$. *The dependence on the path length is the same as for ADER, hence optimal, while the dependence on* U *is optimal.*

14.2 Strongly Adaptive Regret

We now introduce the concept of *strongly adaptive regret*.

Definition 14.7 *For any $\boldsymbol{u} \in \mathcal{V}$, we define the **strongly adaptive regret** as*

$$\text{SA-Regret}_{T,\tau}(\boldsymbol{u}) := \max_{[s,s+\tau-1]\subseteq\{1,2,\dots,T\}} \sum_{t=s}^{s+\tau-1} \ell_t(\boldsymbol{x}_t) - \sum_{t=s}^{s+\tau-1} \ell_t(\boldsymbol{u}) \, .$$

*We say that the algorithm is **strongly adaptive** if, for any fixed $\boldsymbol{u} \in \mathcal{V}$, the additional price we pay with respect to the optimal bound on any specific interval of length τ is at most polylogarithmic in T.*

This definition captures the fact that we want the performance of the algorithm to be good on any interval of length τ.

Remark 14.8 *One might be tempted to remove the dependence on τ and consider instead $\max_\tau \text{SA-Regret}_{T,\tau}(\boldsymbol{u})$. This notion is known as **adaptive regret**. However, it is clear that the worst-case value of $\max_\tau \text{SA-Regret}_{T,\tau}(\boldsymbol{u})$ is $\Omega(\sqrt{T})$ for bounded domains and Lipschitz losses (from the lower bound in Chapter 5), which is meaningless for intervals of size $\mathcal{O}(\sqrt{T})$. Hence, the adaptive regret does not allow us to reason about the performance of the algorithm on small intervals.*

In the next section, we show how to obtain strongly adaptive algorithms, using once again a combination of online learning algorithms.

14.2.1 CBCE: A Meta Algorithm for Strongly Adaptive Regret

We will introduce now the **Coin Betting for Changing Environment (CBCE)** algorithm, see Algorithm 14.2. We use T different Online Linear Optimization (OLO) algorithms, each one starting at a different time step, and we combine them with a sleeping-experts algorithm over a countably infinite number of experts. In particular, $\boldsymbol{x}_t^{(s)}$ is the output at time t of the OLO algorithm started at time s, while $\boldsymbol{x}_t$ is the convex combination of $\boldsymbol{x}_t^{(1)}, \dots, \boldsymbol{x}_t^{(t)}$ using the probability distribution produced by the sleeping expert algorithm. Using the sleeping expert algorithm in Section 13.8 immediately yields the following theorem.

Theorem 14.9 *Let $\mathcal{V} \subset \mathbb{R}^d$ be a non-empty bounded closed convex set and define $D = \max_{\boldsymbol{x},\boldsymbol{y}\in\mathcal{V}} \|\boldsymbol{x} - \boldsymbol{y}\|$. Assume the losses $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ to be subdifferentiable and L-Lipschitz with respect to $\|\cdot\|$ on an open set containing $\mathcal{V}$. Let $\mathscr{M}$ be the variant of Algorithm 13.5 with a countable number of experts in Remark 13.16, where $\epsilon_i = \frac{1}{i(i+1)}$ for $i = 1, 2, \dots$, and $\mathscr{B}$ an OLO algorithm whose regret we denote by $\text{Regret}_t^{OLO}(\boldsymbol{u})$ for all $t \in \mathbb{N}$. Let $I = [s, s+\tau-1] \subset [1, 2, \dots, T]$. Then, for all $\boldsymbol{u} \in \mathcal{V}$, Algorithm 14.2 satisfies*

$$\begin{aligned}\text{Regret}_I(\boldsymbol{u}) &:= \sum_{t\in I} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \\ &\leq \text{Regret}_\tau^{OLO}(\boldsymbol{u}) + DL\left(\sqrt{2\tau\ln(e\tau s(s+1)+1)}+1\right) \, .\end{aligned}$$

Algorithm 14.2 Coin Betting for Changing Environment (CBCE)

Require: Non-empty, closed, and convex feasible set $\mathcal{V} \subset \mathbb{R}^d$, OLO black-box algorithm $\mathscr{B}$, sleeping expert algorithm $\mathscr{M}$

for $t = 1$ **to** T **do**
  Pass the active set to $\mathscr{M}$ and get $\boldsymbol{p}_t$ from $\mathscr{M}$ such that $p_{t,i} = 0$ for $i > t$
  Get $\boldsymbol{x}_t^{(i)}$ from the copy i of $\mathscr{B}$, for $i = 1, \dots, t$
  Output $\boldsymbol{x}_t = \sum_{i=1}^{t} p_{t,i} \boldsymbol{x}_t^{(i)}$ and pay $\ell_t(\boldsymbol{x}_t)$
  Receive $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$
  Pass $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$ to the copy i of $\mathscr{B}$, for $i = 1, \dots, t$
  Let $\boldsymbol{v}_t = [\langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(1)} \rangle, \dots \langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(t)} \rangle, 0, 0, \dots] \in \mathbb{R}^\infty$
  Pass the loss $\ell_t^{\text{sleeping}}(\boldsymbol{x}) = \langle \boldsymbol{v}_t, \boldsymbol{x} \rangle$ to the sleeping expert algorithm
end for

Proof Like in the case of ADER, a simple implementation of this algorithm would require querying t values of the function ℓ_t at time t. However, we run the algorithm on the linearized losses $\langle \boldsymbol{g}_t, \cdot \rangle$, where $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$, so that the subgradient is only asked once. Hence, we upper bound the regret using the linearized losses:

$$\sum_{t=s}^{s+\tau-1} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \sum_{t=s}^{s+\tau-1} \langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle = \sum_{t=s}^{s+\tau-1} (\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{u})) \,.$$

Next, we decompose the regret in the contributions of $\mathscr{B}$ and $\mathscr{M}$ as

$$\sum_{t=s}^{s+\tau-1} (\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{u})) = \sum_{t=s}^{s+\tau-1} (\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{x}_t^{(s)})) + \sum_{t=s}^{s+\tau-1} (\tilde{\ell}_t(\boldsymbol{x}_t^{(s)}) - \tilde{\ell}_t(\boldsymbol{u})) \,.$$

The first sum can be written as

$$\sum_{t=s}^{s+\tau-1} (\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{x}_t^{(s)})) = \text{Regret}_{s+\tau-1}^{\text{sleeping}}(\boldsymbol{e}_s),$$

because the s-th copy of the algorithm $\mathscr{B}$ has been active only on rounds s to $s+\tau-1$. The losses passed to the sleeping-experts algorithm have an active range at most DL, because for any two active experts m, n,

$$|\langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(m)} - \boldsymbol{x}_t^{(n)} \rangle| \leq \|\boldsymbol{g}_t\|_\star \|\boldsymbol{x}_t^{(m)} - \boldsymbol{x}_t^{(n)}\| \leq DL \,.$$

Hence, with the choice of $\epsilon_i = \frac{1}{i(i+1)}$, Algorithm 13.5, through Remark 13.16, achieves a sleeping regret against the s-th expert of $DL\sqrt{2\tau \ln(e\tau s(s+1)+1)} + DL$, because the expert s has been active for τ rounds. For the second sum, we simply have $\text{Regret}_\tau^{\text{OLO}}(\boldsymbol{u})$ because the s-th copy of algorithm $\mathscr{B}$ was started on round s. □

Hence, assuming that $R_\tau(\boldsymbol{u}) = \mathcal{O}(\sqrt{\tau})$, the CBCE algorithm is strongly adaptive because the regret on any interval of length τ depends on $\sqrt{\tau}$ and, upper bounding s with T, only polylogarithmically on T.

However, while we ask only one subgradient per round, the computational complexity of CBCE in round t is proportional to t, because we have to update t OLO algorithms. So, in

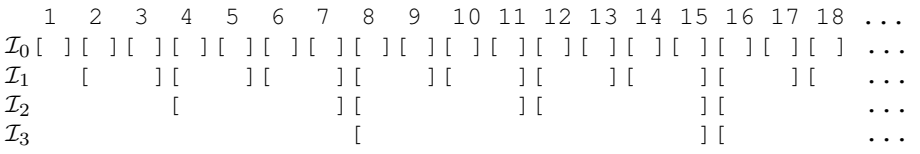


Figure 14.1 Geometric covering intervals. Each interval is denoted by `[ ]`.

the next section, we consider an efficient variant of the CBCE algorithm, whose update is proportional to $\ln t$, with the same strongly adaptive regret guarantee.

14.2.2 Efficient Version of CBCE

The previous algorithm has the disadvantage of requiring the start of a new copy of algorithm $\mathscr{B}$ in each iteration. Instead, here we will see how to reduce the number of copies to the logarithm of the number of rounds, using **geometric coverings**.

For $k = 0, 1, 2, \ldots$, define the intervals

$$\mathcal{I}_k := \{[i2^k, (i+1)2^k - 1] : i \in \mathbb{N}\} .$$

That is, each $\mathcal{I}_k$ is a partition of $\mathbb{N}\backslash\{1, 2, 3, \ldots, 2^k - 1\}$ into consecutive intervals of length 2^k. Also, define

$$\mathcal{I} := \cup_{k=0,1,2,\ldots}\mathcal{I}_k \qquad \text{and} \qquad ACTIVE(t) := \{I \in \mathcal{I} : t \in I\},$$

the set of the "active" intervals at time t, that is, the intervals that contain t. By the definition of $\mathcal{I}_k$, for every $t \leq 2^k - 1$ we have that no interval in $\mathcal{I}_k$ contains t, while for every $t \geq 2^k$ we have that a single interval in $\mathcal{I}_k$ contains t. Therefore, $|ACTIVE(t)| = \lfloor \log_2 t \rfloor + 1$. We will run a copy of an OCO algorithm for each active interval. This means that we have a number of algorithms at each time step t that is logarithmic in t. Given an interval J of $\mathbb{N}$, we will also use the definition of $\mathcal{I}|_J$

$$\mathcal{I}|_J := \{I \in \mathcal{I} : I \subseteq J\},$$

to denote the set of the intervals in $\mathcal{I}$ that are contained in J.

The next lemma allows us to decompose the regret over any interval using the regret over intervals in $\mathcal{I}$.

Lemma 14.10 *Let $J = [q, s] \subseteq \mathbb{N}$ be an arbitrary interval. Then, the interval J can be partitioned into two finite sequences of disjoint and consecutive intervals,*

$$(J_{-k}, \ldots, J_0) \subseteq \mathcal{I}|_J \quad \textit{and} \quad (J_1, J_2, \ldots, J_p) \subseteq \mathcal{I}|_J,$$

such that i) $|J_{-i}|/|J_{-i+1}| \leq 1/2, \forall i \geq 1$; ii) $|J_i|/|J_{i-1}| \leq 1/2, \forall i \geq 2$.

Proof The intuition of the proof is the following one: first, we choose the interval J_0 to be the biggest leftmost one in $\mathcal{I}|_J$. Then, we show that the others are selected so that they decrease in size to the left. On the right, J_1 can have the same size as J_0, and the others will decrease in size.

Denote the size of J_i by b_i for $i = -k, \ldots, 0, \ldots, p$ and let $b_0 = \max\{|J'| : J' \in \mathcal{I}|_J\}$

be the maximal size of any interval $J' \in \mathcal{I}$ that is contained in J. Among all of these intervals, let J_0 be the leftmost interval, i.e., we define

$$\begin{aligned} q_0 &:= \operatorname{argmin}\{q' : [q', q' + b_0 - 1] \in \mathcal{I}|_J\}, \\ s_0 &:= q_0 + b_0 - 1, \\ J_0 &:= [q_0, s_0]\,. \end{aligned}$$

Starting from $q_0 - 1$, we now define a sequence of intervals (in reverse order), denoted $(J_{-1}, \ldots, J_{-k})$ to cover the interval $[q, q_0 - 1]$, as follows:

$$\begin{aligned} [q_{-1}, s_{-1}] := J_{-1} &:= \operatorname*{argmax}_{J'=[q',s'] \in \mathcal{I}|_{[q, q_0 - 1]} : s' = q_0 - 1} |J'| \\ &\vdots \\ [q_{-i}, s_{-i}] := J_{-i} &:= \operatorname*{argmax}_{J'=[q',s'] \in \mathcal{I}|_{[q, q_{-i+1} - 1]} : s' = q_{-i+1} - 1} |J'| \\ &\vdots \end{aligned}$$

Clearly, this sequence is finite and the left endpoint of the leftmost interval, J_{-k}, is q. Denote the size of J_{-i} by b_{-i}. We next prove that for every $i \geq 1$, $b_{-i}/b_{-i+1} \leq 1/2$. Given that the interval lengths are powers of 2, it suffices to show that $b_{-i} < b_{-i+1}$ for every $i \geq 1$. We use induction. The base case follows from the fact that $b_0 > q_0 - q$, otherwise the interval $[q_0 - b_0, q_0 - 1]$ would belong to $\mathcal{I}|_J$, have the same size as J_0, and lie strictly to the left of J_0, contradicting the maximality of b_0 and the fact that J_0 is the leftmost interval of maximal size. We next assume that the claim holds for every $i \in \{1, \ldots, n-1\}$ and prove for n. Assume by contradiction that $b_{-n} \geq b_{-n+1}$. Consider the interval $\hat{J}_{-n+1}$ which is obtained by extending J_{-n+1} to its left by an additional size of b_{-n+1}, that is, $\hat{J}_{-n+1} := [q_{-n+1} - b_{-n+1}, q_{-n+1} - 1] \cup J_{-n+1}$. Hence, $\hat{J}_{-n+1}$ is an interval of size $2b_{-n+1}$ and it is contained in J. According to the induction hypothesis, $|\hat{J}_{-n+1}| = 2b_{-n+1} = 2^j b_{-n+2}$ for some $j \leq 0$. Given that $\hat{J}_{-n+1}$ is adjacent to the interval J_{-n+2} and its size is $2^j |J_{-n+2}|$ for some $j \leq 0$, from the definition of $\mathcal{I}$, we have that $\hat{J}_{-n+1} \in \mathcal{I}|_J$, contradicting the maximality of J_{-n+1}.

Similarly, we define a sequence of disjoint and consecutive intervals, denoted $(J_1, \ldots, J_p)$ that covers $[s_0 + 1, s]$.

$$\begin{aligned} [q_1, s_1] := J_1 &:= \operatorname*{argmax}_{J'=[q',s'] \in \mathcal{I}|_{[s_0 + 1, s]} : q' = s_0 + 1} |J'| \\ &\vdots \\ [q_i, s_i] := J_i &:= \operatorname*{argmax}_{J'=[q',s'] \in \mathcal{I}|_{[s_{i-1} + 1, s]} : q' = s_{i-1} + 1} |J'| \\ &\vdots \end{aligned}$$

Clearly, this sequence is finite, and the right endpoint of the rightmost interval, J_p, is s. Denote the size of J_i by b_i. We next prove that for every $i \geq 2$, $b_i/b_{i-1} \leq 1/2$ which is equivalent to $b_i < b_{i-1}$.

Algorithm 14.3 Efficient Coin Betting for Changing Environment

Require: Non-empty, closed, and convex feasible set $\mathcal{V} \subset \mathbb{R}^d$, OLO black-box algorithm $\mathscr{B}$, sleeping expert algorithm $\mathscr{M}$

1: **for** $t = 1$ **to** T **do**
2: Pass the active set $ACTIVE(t)$ to $\mathscr{M}$ and get $\boldsymbol{p}_t$ from $\mathscr{M}$ such that $p_{t,J} \neq 0$ only for $J \in ACTIVE(t)$
3: Get $\boldsymbol{x}_t^{(J)}$ from the copy of $\mathscr{B}$ for $J \in ACTIVE(t)$
4: Output $\boldsymbol{x}_t = \sum_{J \in ACTIVE(t)} p_{t,J} \boldsymbol{x}_t^{(J)}$ and pay $\ell_t(\boldsymbol{x}_t)$
5: Receive $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$
6: Pass $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$ to the copy of $\mathscr{B}$ for $J \in ACTIVE(t)$
7: Let $v_{t,J} = \begin{cases} \langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(J)} \rangle, & J \in ACTIVE(t) \\ 0, & \text{otherwise} \end{cases}$
8: Pass the loss $\ell_t^{\text{sleeping}}(\boldsymbol{x}) = \langle \boldsymbol{v}_t, \boldsymbol{x} \rangle$ to the sleeping expert algorithm $\mathscr{M}$
9: **end for**

We prove it again by induction. We assume that $b_i/b_{i-1} \in \{2^j : j \leq 0\}$ for every $i \in \{1, \dots, n-1\}$ and prove that $b_n < b_{n-1}$ for $n \geq 2$. For the base case, from the definition of b_0, we have that $b_1 \leq b_0$. Now, assume by contradiction that $b_n \geq b_{n-1}$. Then, we can consider the interval $\hat{J}_{n-1}$ which is obtained by extending J_{n-1} to its right by an additional size of b_{n-1}. It follows that $\hat{J}_{n-1}$ is an interval of size $2b_{n-1}$ and it is contained in J. According to the induction hypothesis, $|\hat{J}_{n-1}| = 2b_{n-1} = 2^j b_{n-2}$ for some $j \leq 1$. We need to consider the following two cases:

- Case $j \leq 0$ (i.e., $b_{n-1}/b_{n-2} \leq 1/2$). Then, $\hat{J}_{n-1}$ is consecutive to $J_{n-2} \in \mathcal{I}$ and its size is $2^j |J_{n-2}|$, hence $\hat{J}_{n-1} \in \mathcal{I}|_J$, contradicting the maximality of J_{n-1}.
- Case $j = 1$ (i.e., $b_{n-1} = b_{n-2}$). Then, $|\hat{J}_{n-1}| = 2b_{n-2}$ and this contradicts the maximality of J_{n-2}. □

As an example of the above Lemma, consider the interval $J = [1, 30]$ that is partitioned into the intervals $J_{-3} = [1, 1]$, $J_{-2} = [2, 3]$, $J_{-1} = [4, 7]$, $J_0 = [8, 15]$, $J_1 = [16, 23]$, $J_2 = [24, 27]$, $J_3 = [28, 29]$, $J_4 = [30, 30]$.

We will use an expert algorithm over a countable number of experts, hence we also have a prior over the infinite-dimensional simplex $\Delta^\infty := \{\boldsymbol{x} \in \mathbb{R}^\infty : x_i \geq 0, \sum_{i=1}^\infty x_i = 1\}$.

Theorem 14.11 *Let $\mathcal{V} \subset \mathbb{R}^d$ be a non-empty bounded closed convex set and define $D = \max_{\boldsymbol{x}, \boldsymbol{y} \in \mathcal{V}} \|\boldsymbol{x} - \boldsymbol{y}\|$. Assume the losses $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$ to be subdifferentiable and L-Lipschitz with respect to $\|\cdot\|$ on an open set containing $\mathcal{V}$. Let $\mathscr{M}$ be the variant of Algorithm 13.5 with a countable number of experts in Remark 13.16, where the copy associated to the interval $J = [i_1, i_2] \in \mathcal{I}$ has the parameter ϵ_J defined recursively as*

$$\epsilon_J = \begin{cases} \frac{1}{i_1(i_1+1)(\lfloor \log_2 i_1 \rfloor + 1)}, & J \in \mathcal{I}_0 \\ \sum_{I \in \mathcal{I}_0|_J} \epsilon_I, & J \in \mathcal{I}_k, k \geq 1 \end{cases}.$$

Let $\mathscr{B}$ be an OLO algorithm such that, for every $t \in \mathbb{N}$, every $\boldsymbol{u} \in \mathcal{V}$, and every sequence of linear losses $\tilde{\ell}_s(\boldsymbol{x}) = \langle \boldsymbol{g}_s, \boldsymbol{x} \rangle$ with $\|\boldsymbol{g}_s\|_\star \leq L$, its regret satisfies $\mathrm{Regret}_t(\boldsymbol{u}) \leq cDL\, t^\alpha$,

where $\alpha \in (0,1)$ and $c > 0$. Let $I = [s, s+\tau-1] \subset [1,2,\dots,T]$. Then, for all $\boldsymbol{u} \in \mathcal{V}$, Algorithm 14.3 satisfies

$$\mathrm{Regret}_I(\boldsymbol{u}) := \sum_{t\in I}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \le 2\frac{2^\alpha - (4\tau^2)^{-\alpha}}{2^\alpha - 1} cDL\tau^\alpha + DL + 10DL\sqrt{\tau \ln(e(s+\tau)^4+1)}\,.$$

Proof First of all, we upper bound the regret over the interval I with the one over linearized losses:

$$\sum_{t\in I}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \le \sum_{t\in I}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}\rangle = \sum_{t\in I}(\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{u})),$$

where $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x}\rangle$.

Now, suppose we denote by $\boldsymbol{x}_t^{(J)}$ the decision from the black-box run $\mathscr{B}_J$ associated with the interval J at time t and by $\boldsymbol{x}_t$ the combined decision of the meta algorithm at time t. Since the complete algorithm is a combination of a meta algorithm $\mathscr{M}$ and several black-box algorithms $\mathscr{B}$, its regret depends on both $\mathscr{M}$ and $\mathscr{B}$. We now decompose the two sources of regret additively through the geometric covering.

Let $\mathcal{J} = \bigcup_{i=-a}^{b} J_i$ be the partition of I obtained from Lemma 14.10. Then, the regret on I can be decomposed as follows:

$$\begin{aligned}
\mathrm{Regret}_I(\boldsymbol{u}) &= \sum_{t\in I}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \le \sum_{t\in I}\Big(\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{u})\Big) \\
&= \sum_{i=-a}^{b}\sum_{t\in J_i}\Big(\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{x}_t^{(J_i)}) + \tilde{\ell}_t(\boldsymbol{x}_t^{(J_i)}) - \tilde{\ell}_t(\boldsymbol{u})\Big) \\
&= \underbrace{\sum_{i=-a}^{b}\sum_{t\in J_i}\Big(\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{x}_t^{(J_i)})\Big)}_{\text{meta regret on } J_i} + \underbrace{\sum_{i=-a}^{b}\sum_{t\in J_i}\Big(\tilde{\ell}_t(\boldsymbol{x}_t^{(J_i)}) - \tilde{\ell}_t(\boldsymbol{u})\Big)}_{\text{black-box regret on } J_i}\,. \qquad (14.1)
\end{aligned}$$

The black-box regret on $J_i \in \mathcal{J}$ is exactly the regret of the black-box algorithm on $|J_i|$ rounds, since the black-box run $\mathscr{B}_{J_i}$ was started at the beginning of the interval J_i. In particular, we have

$$\begin{aligned}
\sum_{i=-a}^{b}\sum_{t\in J_i}(\tilde{\ell}_t(\boldsymbol{x}_t^{(J_i)}) - \tilde{\ell}_t(\boldsymbol{u})) = \sum_{i=-a}^{b}\sum_{t\in J_i}\langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(J_i)} - \boldsymbol{u}\rangle &\le cDL\sum_{i=-a}^{b}|J_i|^\alpha \\
&\le 2cDL\sum_{k=0}^{a+b+1}(2^{-k}|I|)^\alpha = 2c|I|^\alpha DL\frac{2^\alpha - 2^{-\alpha(a+b+1)}}{2^\alpha - 1} \\
&\le 2c|I|^\alpha DL\frac{2^\alpha - (4|I|^2)^{-\alpha}}{2^\alpha - 1},
\end{aligned}$$

where the second inequality is due to Lemma 14.10, and in the last one we used that the number of geometric cover intervals is less than $2(\lfloor \log_2 |I| \rfloor + 1)$ (again from Lemma 14.10) and $2^{-2(\lfloor \log_2 |I|\rfloor+1)} \ge \frac{1}{4|I|^2}$.

Now, we bound the meta-regret term in (14.1). We apply the sleeping-experts guarantee once with the fixed comparator that puts weight 1 on the experts associated with $J_{-a}, \dots, J_b$ and weight 0 on all the others. Since these intervals are disjoint, at each time $t \in I$ exactly one of these selected experts is active. Moreover, the losses passed to the sleeping-experts algorithm have an active range at most DL, because for any two active experts J, J',

$$|\langle \boldsymbol{g}_t, \boldsymbol{x}_t^{(J)} - \boldsymbol{x}_t^{(J')} \rangle| \le \|\boldsymbol{g}_t\|_\star \|\boldsymbol{x}_t^{(J)} - \boldsymbol{x}_t^{(J')}\| \le DL \,.$$

Hence,

$$\sum_{i=-a}^{b} \sum_{t \in J_i} (\tilde{\ell}_t(\boldsymbol{x}_t) - \tilde{\ell}_t(\boldsymbol{x}_t^{(J_i)})) \le DL \sum_{i=-a}^{b} \sqrt{2|J_i| \ln\left(1 + \frac{e|J_i|}{\epsilon_{J_i}}\right)} + DL \sum_{J \in \mathcal{I}} \epsilon_J \,.$$

Also, from Lemma 14.10, $|J_i| \le 2^i \tau$ for $i \le 0$. This implies that

$$|J_i|/\epsilon_{J_i} \le |J_i|(s+\tau-1)(s+\tau)(\log_2(s+\tau-1)+1) \le (s+\tau)^4, \forall i \le 0 \,.$$

Hence, we have

$$\begin{aligned}
\sum_{i=-a}^{0} \sqrt{2|J_i| \ln\left(1 + \frac{e|J_i|}{\epsilon_{J_i}}\right)} &\le \sum_{i=-a}^{0} \sqrt{2^{i+1}\tau \ln\left(1 + e(s+\tau)^4\right)} \\
&\le \sqrt{\tau \ln(e(s+\tau)^4 + 1)} \sum_{k=0}^{\infty} \sqrt{2^{-k+1}} \\
&\le 5\sqrt{\tau \ln(e(s+\tau)^4 + 1)}
\end{aligned}$$

Analogously, we have

$$\sum_{i=1}^{b} \sqrt{2|J_i| \ln\left(1 + \frac{e|J_i|}{\epsilon_{J_i}}\right)} \le 5\sqrt{\tau \ln(e(s+\tau)^4 + 1)} \,.$$

Finally, observe that $\sum_{J \in \mathcal{I}} \epsilon_J = 1$. Putting everything together, we have the stated bound. □

Remark 14.12 *Observe that the limit for $\alpha \to 0$ of the first term of the upper bound of the regret is $2cDL \log_2(8\tau^2)$. However, the second term is always of the order of $\sqrt{\tau}$.*

From the above theorem, it is immediate to get a guarantee on the strongly adaptive regret by observing that

$$\text{SA-Regret}_{T,\tau}(\boldsymbol{u}) = \max_{I \subseteq \{1,\dots,T\} : |I| = \tau} \text{Regret}_I(\boldsymbol{u}) \,.$$

14.3 Tracking Regret

We now see yet another way to measure the ability of an online algorithm to adapt to changes in the behaviour of the adversarial losses.

Definition 14.13 *For $m \in \{0, \dots, T-1\}$, define the **tracking regret** as*

$$\text{T-Regret}_T^m(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) := \text{D-Regret}_T(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T),$$

where the sequence $\boldsymbol{u}_1, \dots, \boldsymbol{u}_T \in \mathcal{V}$ changes at most m times. More formally, there exists an integer $K \leq m+1$, a partition of $\{1, 2, \dots, T\}$ into K consecutive intervals $(\mathcal{I}_i)_{i=1}^K$, and $\tilde{\boldsymbol{u}}_1, \dots, \tilde{\boldsymbol{u}}_K \in \mathcal{V}$ such that $\boldsymbol{u}_t = \tilde{\boldsymbol{u}}_i$ for all $t \in \mathcal{I}_i$.

The tracking regret can be upper-bounded both in terms of dynamic regret and strongly adaptive regret. Indeed, if we assume the diameter D of $\mathcal{V}$ to be bounded with respect to some norm, i.e., $D = \max_{\boldsymbol{u},\boldsymbol{v}\in\mathcal{V}} \|\boldsymbol{u} - \boldsymbol{v}\| < \infty$, then for a sequence $\boldsymbol{u}_1, \dots, \boldsymbol{u}_T \in \mathcal{V}$ that changes at most m times we have

$$P^{\|\cdot\|}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) \leq mD \,.$$

Hence, any algorithm with a dynamic regret guarantee that depends on $P^{\|\cdot\|}$ can also be used to obtain a tracking regret guarantee.

For example, we can consider the following algorithm.

Example 14.14 *Consider the setting of Learning with Expert Advice (LEA). In this setting, the tracking regret makes more sense because we are typically interested in competitors that are corners of the simplex.*

To tackle this problem, we will use OMD with the entropic regularizer over the truncated simplex $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : x_i \geq \alpha/d, \sum_{i=1}^d x_i = 1\}$, where $0 < \alpha \leq 1$ is a parameter that will be decided in the following. Hence, we have $x_{t,i} \geq \frac{\alpha}{d}$ and $(\nabla\psi(\boldsymbol{x}))_i = 1 + \ln x_i$. Hence, in the dynamic case, we have

$$Q = \max_t \|\nabla\psi(\boldsymbol{x}_t) - \nabla\psi(\boldsymbol{x}_1)\|_\infty = \max_t \|\ln \boldsymbol{x}_t - \ln \boldsymbol{x}_1\|_\infty \leq \ln \frac{d}{\alpha},$$

where the logarithm of vectors is taken element-wise.

Given that we want to consider $\boldsymbol{u}_t$ in the simplex rather than in the truncated simplex, we have

$$\begin{aligned}\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u}_t \rangle &= \left\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \left((1-\alpha)\boldsymbol{u}_t + \frac{\mathbf{1}_d \alpha}{d}\right)\right\rangle + \left\langle \boldsymbol{g}_t, \alpha\left(\frac{\mathbf{1}_d}{d} - \boldsymbol{u}_t\right)\right\rangle \\ &\leq \left\langle \boldsymbol{g}_t, \boldsymbol{x}_t - \left((1-\alpha)\boldsymbol{u}_t + \frac{\mathbf{1}_d \alpha}{d}\right)\right\rangle + 2\alpha\|\boldsymbol{g}_t\|_\infty \,.\end{aligned}$$

Denoting $\tilde{\boldsymbol{u}}_t = (1-\alpha)\boldsymbol{u}_t + \frac{\mathbf{1}_d \alpha}{d} \in \mathcal{V}$, the bound will depend on

$$\sum_{t=2}^T \|\tilde{\boldsymbol{u}}_t - \tilde{\boldsymbol{u}}_{t-1}\|_1 = (1-\alpha)\sum_{t=2}^T \|\boldsymbol{u}_t - \boldsymbol{u}_{t-1}\|_1 \,.$$

Hence, using the uniform prior $\boldsymbol{x}_1 = \mathbf{1}_d/d$, we have that the tracking regret is upper bounded by

$$\frac{\ln d + (1-\alpha)P^{\|\cdot\|_1}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T)\ln\frac{d}{\alpha}}{\eta} + \frac{\eta}{2}\sum_{t=1}^T \|\boldsymbol{g}_t\|_\infty^2 + 2\alpha\sum_{t=1}^T \|\boldsymbol{g}_t\|_\infty \,.$$

The sequence $\boldsymbol{u}_1, \dots, \boldsymbol{u}_T$ changes at most m times, so $P^{\|\cdot\|_1}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) \le 2m$. Hence, assuming $\|\boldsymbol{g}_t\|_\infty \le G_\infty$ for $t = 1, \dots, T$, we have

$$\text{T-Regret}_T^m(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) \le \frac{\ln d + 2m \ln \frac{d}{\alpha}}{\eta} + \frac{\eta T G_\infty^2}{2} + 2\alpha T G_\infty \,.$$

Assuming $m \ge 1$, choosing $\eta = \sqrt{\frac{2(\ln d + 2m \ln \frac{d}{\alpha})}{G_\infty^2 T}}$ and $\alpha = \sqrt{\frac{m}{T}}$, we have

$$\begin{aligned}\text{T-Regret}_T^m(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) &\le G_\infty \sqrt{2T} \left(\sqrt{\ln d + 2m \ln \frac{d\sqrt{T}}{\sqrt{m}}} + \sqrt{2m} \right) \\ &= G_\infty \sqrt{2Tm} \left(\sqrt{\left(\frac{1}{m} + 2\right) \ln d + \ln \frac{T}{m}} + \sqrt{2} \right) \,.\end{aligned}$$

We can also use the strongly adaptive regret guarantee to control the tracking regret, as detailed in the next theorem.

Theorem 14.15 *Consider an algorithm that guarantees $\text{SA-Regret}_{T,\tau}(\boldsymbol{u}) \le c\tau^\alpha$ for $\alpha \in (0, 1)$, for all $\boldsymbol{u} \in \mathcal{V}$ and all $\tau \in \{1, \dots, T\}$. Then,*

$$\text{T-Regret}_T^m(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) \le c(m+1)^{1-\alpha} T^\alpha, \quad \forall m \in \{0, \dots, T-1\} \,.$$

Proof

$$\begin{aligned}\text{T-Regret}_T^m(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T) &= \sum_{i=1}^K \sum_{t \in \mathcal{I}_i} (\ell_t(\boldsymbol{x}_t) - \ell_t(\tilde{\boldsymbol{u}}_i)) \le \sum_{i=1}^K c|\mathcal{I}_i|^\alpha \\ &\le c \left(\sum_{i=1}^K 1^{\frac{1}{1-\alpha}} \right)^{1-\alpha} \left(\sum_{i=1}^K |\mathcal{I}_i| \right)^\alpha = cK^{1-\alpha} T^\alpha \\ &\le c(m+1)^{1-\alpha} T^\alpha,\end{aligned}$$

where in the first inequality we used the assumption on the strongly adaptive regret and in the second one we used Hölder's inequality. □

14.4 History Bits

Littlestone and Warmuth (1994) introduced the tracking regret in the LEA game, but for upper bounding the number of mistakes, rather than the regret. The extension to arbitrary losses is in Herbster and Warmuth (1998). Zinkevich (2003) is often erroneously thought to have introduced the dynamic regret, while it first appeared in Herbster and Warmuth (2001), with upper bounds that depend on the drift $\sum_{t=2}^T \|\boldsymbol{u}_t - \boldsymbol{u}_{t-1}\|_p$.

Theorem 14.2 is a generalization of Cesa-Bianchi and Lugosi (2006, Theorem 11.4) to arbitrary distance generating functions and considering Lipschitz losses. Also, following Jacobsen and Cutkosky (2022, Lemma 4), I have fixed the offset in the proof so that we do not have to assume that $\mathbf{0} \in \mathcal{V}$.

Zhang et al. (2018) designed the ADER algorithm, and they also proved that it is optimal

in bounded domains. The idea of combining online algorithms with different learning rates comes directly from the MetaGrad algorithm (van Erven and Koolen, 2016; van Erven et al., 2021) that also showed how to query a single gradient per round. In turn, MetaGrad builds on prior work on using a grid of learning rates in EG (Koolen et al., 2014). By now, this is a well-known method that allows us to solve essentially all problems of tuning learning rates in bounded domains, at least theoretically. It is worth also stressing that the general idea of combining the outputs of different online learning algorithms through another online learning algorithm is instead much older, and it goes back at least to Blum and Mansour (2005, 2007).

I described a slightly simpler version of ADER with a flat prior, see Problem 14.1 for the original bound. I also removed the unnecessary assumption that $\mathbf{0} \in \mathcal{V}$. Recently, Zhao et al. (2020, 2024) improved the guarantees of ADER using optimistic algorithms, obtaining smaller bounds in easy environments.

Before ADER, there was also a line of work on the path length evaluated on $\boldsymbol{u}_t^\star \in \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{V}} \ell_t(\boldsymbol{x})$ for $t = 1, \dots, T$ (Jadbabaie et al., 2015). This is an easier setting because one can evaluate the set of minimizers of ℓ_t after receiving the loss, so one can use a simple doubling trick on the learning rate. Also, the resulting bound does not strictly generalize the static regret analysis. Hence, it is difficult to say if the resulting algorithm is better or worse than one with a static regret.

Theorem 14.5 is from Jacobsen and Cutkosky (2022).

The adaptive regret is defined in Hazan and Seshadhri (2007, 2009). The strongly-adaptive regret was defined by Adamskiy et al. (2012, 2016) (still calling it "adaptive regret") and then reinvented in Daniely et al. (2015) (that coined the name). Note that the strongly-adaptive regret is usually defined for bounded domains, taking the maximum with respect to $\boldsymbol{u} \in \mathcal{V}$, whereas I defined it more generally for any competitor in the feasible set. Lemma 14.10 is from Daniely et al. (2015) as well. The efficient version of the CBCE algorithm was proposed by Jun et al. (2017a,b) using a stronger version of the sleeping expert algorithm, still based on coin betting. Here, I simplified the sleeping expert algorithm for didactical reasons. CBCE improves over the guarantee in Daniely et al. (2015) by improving the logarithmic term. The idea of using linearized losses in CBCE to avoid querying more than one subgradient and the better behavior of the first term in the bound when $\alpha \to 0$ are new. The non-efficient version of CBCE is also new, but straightforward.

The algorithm in Example 14.14 is introduced in Herbster and Warmuth (2001) and generalized to arbitrary distance generating functions in Cesa-Bianchi et al. (2012). Theorem 14.15 is from Daniely et al. (2015).

14.5 Exercises

Problem 14.1 *Change the prior in the EG algorithm to obtain a dependence of* $\ln\ln(1 + P^{\|\cdot\|_2}(\boldsymbol{u}_1, \dots, \boldsymbol{u}_T))$ *instead of* $\ln\ln T$. *In this way, if the path length is zero, that is, we are in the static case, the regret is* $\mathcal{O}(\sqrt{T})$ *instead of* $\mathcal{O}(\sqrt{T \ln\ln T})$, *when* $T \to \infty$.

15

Saddle-Point Optimization and Online Algorithms

In this chapter, we talk about solving saddle-point problems with Online Convex Optimization (OCO) algorithms, and the connection with game theory.

15.1 Saddle-Point Problems

We want to solve the following saddle-point problem

$$\inf_{\boldsymbol{x}\in\mathcal{X}} \sup_{\boldsymbol{y}\in\mathcal{Y}} \; f(\boldsymbol{x},\boldsymbol{y}) \; . \tag{15.1}$$

Let's say from the beginning that we need $\inf$ and $\sup$ rather than $\min$ and $\max$ because the minimum or maximum might not exist. Every time we know for sure the $\inf$/$\sup$ are attained, we can substitute them with $\min$/$\max$.

While it is clear what it means to minimize a function, it might not be immediate to see the meaning of "solving" the saddle-point problem in (15.1). It turns out that the proper notion we are looking for is the one of *saddle point*.

Definition 15.1 *Let $\mathcal{X} \subseteq \mathbb{R}^n$, $\mathcal{Y} \subseteq \mathbb{R}^m$, and $f : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$. A point $(\boldsymbol{x}^\star, \boldsymbol{y}^\star) \in \mathcal{X} \times \mathcal{Y}$ is a **saddle point** of f in $\mathcal{X} \times \mathcal{Y}$ if*

$$f(\boldsymbol{x}^\star, \boldsymbol{y}) \leq f(\boldsymbol{x}^\star, \boldsymbol{y}^\star) \leq f(\boldsymbol{x}, \boldsymbol{y}^\star), \quad \forall \boldsymbol{x} \in \mathcal{X}, \boldsymbol{y} \in \mathcal{Y} \; .$$

We will now state conditions under which there *exists* a saddle point that solves (15.1). First, we need an easy lemma.

Lemma 15.2 *Let f be a function from a non-empty product set $\mathcal{X} \times \mathcal{Y}$ to $\mathbb{R}$. Then,*

$$\inf_{\boldsymbol{x}\in\mathcal{X}} \sup_{\boldsymbol{y}\in\mathcal{Y}} \; f(\boldsymbol{x},\boldsymbol{y}) \geq \sup_{\boldsymbol{y}\in\mathcal{Y}} \inf_{\boldsymbol{x}\in\mathcal{X}} \; f(\boldsymbol{x},\boldsymbol{y}) \; .$$

Proof For any $\boldsymbol{x}' \in \mathcal{X}$ and $\boldsymbol{y} \in \mathcal{Y}$ we have that $f(\boldsymbol{x}', \boldsymbol{y}) \geq \inf_{\boldsymbol{x}\in\mathcal{X}} \; f(\boldsymbol{x},\boldsymbol{y})$. This implies that $\sup_{\boldsymbol{y}\in\mathcal{Y}} \; f(\boldsymbol{x}', \boldsymbol{y}) \geq \sup_{\boldsymbol{y}\in\mathcal{Y}} \inf_{\boldsymbol{x}\in\mathcal{X}} \; f(\boldsymbol{x},\boldsymbol{y})$ for all $\boldsymbol{x}' \in \mathcal{X}$, which gives the stated inequality. □

We can now state the following theorem.

Theorem 15.3 *Let f be any function from a non-empty product set $\mathcal{X} \times \mathcal{Y}$ to $\mathbb{R}$. A point $(\boldsymbol{x}^\star, \boldsymbol{y}^\star)$ is a saddle point of f if and only if the supremum in*

$$\sup_{\boldsymbol{y}\in\mathcal{Y}} \inf_{\boldsymbol{x}\in\mathcal{X}} \; f(\boldsymbol{x},\boldsymbol{y}) \tag{15.2}$$

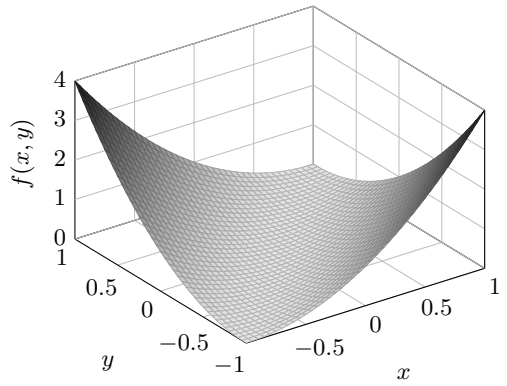


Figure 15.1 The function $f(x, y) = (x - y)^2$ does not have a saddle point.

is attained at $\boldsymbol{y}^\star$*, the infimum in*

$$\inf_{\boldsymbol{x}\in\mathcal{X}} \sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}, \boldsymbol{y}) \tag{15.3}$$

is attained at $\boldsymbol{x}^\star$*, and these two expressions are equal.*

Proof If $(\boldsymbol{x}^\star, \boldsymbol{y}^\star)$ is a saddle point, then we have

$$\begin{aligned} f(\boldsymbol{x}^\star, \boldsymbol{y}^\star) &= \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \boldsymbol{y}^\star) \leq \sup_{\boldsymbol{y}\in\mathcal{Y}} \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \boldsymbol{y}) \\ f(\boldsymbol{x}^\star, \boldsymbol{y}^\star) &= \sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}^\star, \boldsymbol{y}) \geq \inf_{\boldsymbol{x}\in\mathcal{X}} \sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}, \boldsymbol{y}) \,. \end{aligned}$$

From Lemma 15.2, we have that these quantities must be equal, so that the three conditions in the theorem are satisfied.

For the other direction, if the conditions are satisfied, then we have

$$f(\boldsymbol{x}^\star, \boldsymbol{y}^\star) \leq \sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}^\star, \boldsymbol{y}) = \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \boldsymbol{y}^\star) \leq f(\boldsymbol{x}^\star, \boldsymbol{y}^\star) \,.$$

Hence, $(\boldsymbol{x}^\star, \boldsymbol{y}^\star)$ is a saddle point. □

Remark 15.4 *The above theorem implies that the set of saddle points, when nonempty, is the Cartesian product* $\mathcal{X}^\star \times \mathcal{Y}^\star$*, where* $\mathcal{X}^\star$ *and* $\mathcal{Y}^\star$ *are the sets of optimal solutions of the optimization problems* (15.3) *and* (15.2)*, respectively. In other words,* $\boldsymbol{x}^\star$ *and* $\boldsymbol{y}^\star$ *can be independently chosen from the sets* $\mathcal{X}^\star$ *and* $\mathcal{Y}^\star$*, respectively, to form a saddle point.*

Remark 15.5 *The above theorem tells a surprising thing: if a saddle point exists, then there might be multiple ones, and all of them must have the same minimax value. This might seem surprising, but it is due to the fact that the definition of saddle point is a global and not a local property. Moreover, if the* $\inf \sup$ *and* $\sup \inf$ *problem have different values, no saddle point exists.*

Let's show a couple of examples that show that the conditions above are indeed necessary.

Example 15.6 *Let $f(x, y) = (x - y)^2$, $\mathcal{X} = [-1, 1]$, and $\mathcal{Y} = [-1, 1]$. Then, we have*

$$\inf_{x\in\mathcal{X}} \sup_{y\in\mathcal{Y}} (x - y)^2 = \inf_{x\in\mathcal{X}} (1 + |x|)^2 = 1,$$

while

$$\sup_{y\in\mathcal{Y}} \inf_{x\in\mathcal{X}} (x - y)^2 = \sup_{y\in\mathcal{Y}} 0 = 0 .$$

Indeed, from Figure 15.1 we can see that there is no saddle point.

Example 15.7 *Let $f(x, y) = xy$, $\mathcal{X} = (0, 1]$, and $\mathcal{Y} = (0, 1]$. Then, we have*

$$\inf_{x\in\mathcal{X}} \sup_{y\in\mathcal{Y}} xy = \inf_{x\in\mathcal{X}} x = 0$$

and

$$\sup_{y\in\mathcal{Y}} \inf_{x\in\mathcal{X}} xy = 0 .$$

Here, even if inf sup is equal to sup inf, the saddle point does not exist because the inf in the first expression is not attained at a point of $\mathcal{X}$.

Theorem 15.3 also tells us that to find a saddle point of a function f, we need to find the minimizer in $\boldsymbol{x}$ of $\sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}, \boldsymbol{y})$ and the maximizer in $\boldsymbol{y}$ of $\inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \boldsymbol{y})$. Let's now use this knowledge to design a proper measure of progress towards the saddle point.

We might be tempted to use $f(\boldsymbol{x}', \boldsymbol{y}') - f(\boldsymbol{x}^\star, \boldsymbol{y}^\star)$ as a measure of suboptimality of $(\boldsymbol{x}', \boldsymbol{y}')$ with respect to the saddle point $(\boldsymbol{x}^\star, \boldsymbol{y}^\star)$. Unfortunately, this quantity can be negative or equal to zero for an infinite number of points $(\boldsymbol{x}', \boldsymbol{y}')$ that are not saddle points. We might then think to use some notion of distance to the saddle point, like $\|\boldsymbol{x}' - \boldsymbol{x}^\star\|_2^2 + \|\boldsymbol{y}' - \boldsymbol{y}^\star\|_2^2$, but this quantity in general can go to zero at an arbitrarily slow rate. To see why, consider the case that $f(\boldsymbol{x}, \boldsymbol{y}) = h(\boldsymbol{x})$, so that the saddle-point problem reduces to minimizing a convex function. So, assuming only convexity, the rate of convergence to a minimizer of f can be arbitrarily slow because the function can be arbitrarily close to a flat surface, but not flat, around the minimizer. Hence, we need something different.

Observe that Theorem 15.3 says one of the problems we should solve is

$$\inf_{\boldsymbol{x}\in\mathcal{X}} h(\boldsymbol{x}),$$

where $h(\boldsymbol{x}) = \sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}, \boldsymbol{y})$. In this view, the problem looks like a standard offline convex optimization problem, where the objective function has a particular structure. Moreover, in this view, we only focus on the variables $\boldsymbol{x}$. The standard measure of convergence in this case, for a point $\boldsymbol{x}'$, the *suboptimality gap*, can be written as

$$h(\boldsymbol{x}') - \inf_{\boldsymbol{x}\in\mathcal{X}} h(\boldsymbol{x}) = \sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}', \boldsymbol{y}) - \inf_{\boldsymbol{x}\in\mathcal{X}} \sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}, \boldsymbol{y}) .$$

We also have to find the maximizer with respect to $\boldsymbol{y}$ of the function $\inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \boldsymbol{y})$, hence we have

$$\sup_{\boldsymbol{y}\in\mathcal{Y}} g(\boldsymbol{y}),$$

where $g(\boldsymbol{y}) = \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x},\boldsymbol{y})$. This also implies another measure of convergence in which we focus only on the variable $\boldsymbol{y}$:

$$\sup_{\boldsymbol{y}\in\mathcal{Y}} g(\boldsymbol{y}) - g(\boldsymbol{y}') = \sup_{\boldsymbol{y}\in\mathcal{Y}} \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x},\boldsymbol{y}) - \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x},\boldsymbol{y}') \,.$$

Finally, in case we are interested in studying the quality of a joint solution $(\boldsymbol{x}',\boldsymbol{y}')$, a natural measure is the sum of the two measures above:

$$\begin{aligned}&\sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}',\boldsymbol{y}) - \inf_{\boldsymbol{x}\in\mathcal{X}} \sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x},\boldsymbol{y}) + \sup_{\boldsymbol{y}\in\mathcal{Y}} \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x},\boldsymbol{y}) - \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x},\boldsymbol{y}') \\ &\quad = \sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}',\boldsymbol{y}) - \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x},\boldsymbol{y}'),\end{aligned}$$

where we assumed the existence of a saddle point to say that $\inf_{\boldsymbol{x}\in\mathcal{X}} \sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x},\boldsymbol{y}) = \sup_{\boldsymbol{y}\in\mathcal{Y}} \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x},\boldsymbol{y})$ from Theorem 15.3. This measure is called *duality gap*.

Definition 15.8 *For a function $f : \mathcal{X}\times\mathcal{Y} \to \mathbb{R}$, define the **duality gap** on $(\boldsymbol{x}',\boldsymbol{y}') \in \mathcal{X}\times\mathcal{Y}$ as*

$$\sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}',\boldsymbol{y}) - \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x},\boldsymbol{y}') \,.$$

The duality gap is always non-negative even when the saddle point does not exist, since $\sup_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}',\boldsymbol{y}) \geq f(\boldsymbol{x}',\boldsymbol{y}') \geq \inf_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x},\boldsymbol{y}')$, for all $\boldsymbol{x}' \in \mathcal{X}$ and $\boldsymbol{y}' \in \mathcal{Y}$.

Let's add even more intuition of the duality gap definition, using the notion of ϵ-*saddle point*.

Definition 15.9 *Let $\epsilon \geq 0$, $\mathcal{X} \subseteq \mathbb{R}^n$, $\mathcal{Y} \subseteq \mathbb{R}^m$, and $f : \mathcal{X}\times\mathcal{Y} \to \mathbb{R}$. A point $(\boldsymbol{x}^\star,\boldsymbol{y}^\star) \in \mathcal{X}\times\mathcal{Y}$ is an ϵ-**saddle point** of f in $\mathcal{X}\times\mathcal{Y}$ if*

$$f(\boldsymbol{x}^\star,\boldsymbol{y}) - \epsilon \leq f(\boldsymbol{x}^\star,\boldsymbol{y}^\star) \leq f(\boldsymbol{x},\boldsymbol{y}^\star) + \epsilon, \quad \forall \boldsymbol{x} \in \mathcal{X}, \boldsymbol{y} \in \mathcal{Y} \,.$$

This definition is useful because we cannot expect to numerically calculate a saddle point with infinite precision, but we can find something that satisfies the saddle point definition up to an ϵ. Obviously, any saddle point is also an ϵ-saddle point, yet we can have an ϵ-saddle point even when the saddle point does not exist. Indeed, in Example 15.7 the point (δ,δ), where $\delta \in (0,1]$, is a $\max\{\delta(1-\delta),\delta^2\}$-saddle point, even if the saddle-point does not exist.

Now, the notion of ϵ-saddle point is equivalent up to a multiplicative constant to the ϵ-duality gap, as detailed in the next lemma. The proof is left as an exercise (see Problem 15.1).

Lemma 15.10 *If $(\boldsymbol{x}^\star,\boldsymbol{y}^\star)$ is an ϵ-saddle point then its duality gap is upper bounded by 2ϵ. On the other hand, a duality gap at most ϵ implies that the point is a ϵ-saddle point.*

The above reasoning shows that finding the saddle point of the function f is equivalent to solving both a maximization and a minimization problem. However, as we said above, the saddle point might not exist. So, let's now move to easily checkable sufficient conditions for the existence of a saddle point. For this, we can state the following theorem, which is a weaker version of Sion's minimax theorem (Sion, 1958).

Theorem 15.11 *Let $\mathcal{X}$, $\mathcal{Y}$ be compact convex subsets of $\mathbb{R}^n$ and $\mathbb{R}^m$ respectively. Let $f : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ be a continuous function, convex in its first argument, and concave in its second, and with bounded subgradients with respect to both variables. Then, we have that*

$$\min_{\boldsymbol{x}\in\mathcal{X}} \max_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x},\boldsymbol{y}) = \max_{\boldsymbol{y}\in\mathcal{Y}} \min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x},\boldsymbol{y}) \,.$$

This theorem gives us sufficient conditions to have the min-max problem equal to the max-min one. So, for example, thanks to the Weierstrass theorem (Theorem A.3), the assumptions in Theorem 15.11, in light of Theorem 15.3, are sufficient conditions for the existence of a saddle point.

We defer the proof of this theorem for a bit, and we now turn to solving the saddle-point problem in (15.1).

15.2 Solving Saddle-Point Problems with Online Convex Optimization

Algorithm 15.1 Solving Saddle-Point Problems with Online Convex Optimization

Require: $\boldsymbol{x}_1 \in \mathcal{X}, \boldsymbol{y}_1 \in \mathcal{Y}$
1: **for** $t = 1, \dots, T$ **do**
2: $\quad\mathcal{X}$-Learner and $\mathcal{Y}$-Learner simultaneously decide their outputs $\boldsymbol{x}_t \in \mathcal{X}$ and $\boldsymbol{y}_t \in \mathcal{Y}$
3: $\quad\mathcal{X}$-Learner receives $\ell_t(\boldsymbol{x}) = f(\boldsymbol{x}, \boldsymbol{y}_t)$
4: $\quad\mathcal{Y}$-Learner receives $h_t(\boldsymbol{y}) = -f(\boldsymbol{x}_t, \boldsymbol{y})$
5: **end for**
6: **return** $\bar{\boldsymbol{x}}_T = \frac{1}{T}\sum_{t=1}^T \boldsymbol{x}_t, \bar{\boldsymbol{y}}_T = \frac{1}{T}\sum_{t=1}^T \boldsymbol{y}_t$

Let's show how to use Online Convex Optimization (OCO) algorithms to solve saddle-point problems. We will state a procedure that is a direct generalization of the online-to-batch conversion we saw in Chapter 3.

Suppose we use an OCO algorithm fed with losses $\ell_t(\boldsymbol{x}) = f(\boldsymbol{x}, \boldsymbol{y}_t)$ that produces the iterates $\boldsymbol{x}_t$ and another OCO algorithm fed with losses $h_t(\boldsymbol{y}) = -f(\boldsymbol{x}_t, \boldsymbol{y})$ that produces the iterates $\boldsymbol{y}_t$. Then, we can state the following theorem.

Theorem 15.12 *Let $\mathcal{X}$ and $\mathcal{Y}$ be non-empty closed convex sets. Let $f : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$. Then, with the notation in Algorithm 15.1, for any $\boldsymbol{x} \in \mathcal{X}$, we have*

$$\frac{1}{T}\sum_{t=1}^T f(\boldsymbol{x}_t, \boldsymbol{y}_t) - \frac{1}{T}\sum_{t=1}^T f(\boldsymbol{x}, \boldsymbol{y}_t) = \frac{\mathrm{Regret}_T^{\mathcal{X}}(\boldsymbol{x})}{T},$$

where $\mathrm{Regret}_T^{\mathcal{X}}(\boldsymbol{x}) = \sum_{t=1}^T \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^T \ell_t(\boldsymbol{x})$.

Moreover, for any $\boldsymbol{y} \in \mathcal{Y}$, we have

$$\frac{1}{T}\sum_{t=1}^T f(\boldsymbol{x}_t, \boldsymbol{y}) - \frac{1}{T}\sum_{t=1}^T f(\boldsymbol{x}_t, \boldsymbol{y}_t) = \frac{\mathrm{Regret}_T^{\mathcal{Y}}(\boldsymbol{y})}{T},$$

where $\mathrm{Regret}_T^{\mathcal{Y}}(\boldsymbol{y}) = \sum_{t=1}^T h_t(\boldsymbol{y}_t) - \sum_{t=1}^T h_t(\boldsymbol{y})$.

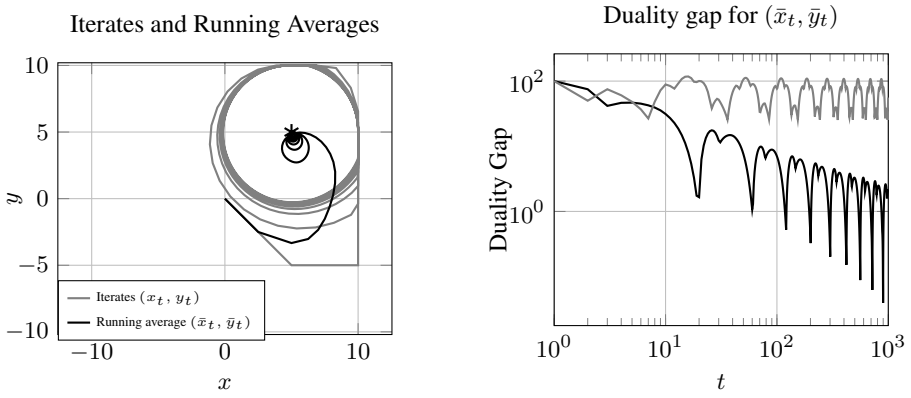


Figure 15.2 Left: behaviour of the iterates and running averages of the iterates for Example 15.14. The saddle point is marked with an asterisk. Right: the duality gap on a log-log plot for the same problem evaluated on the iterates and running averages of the iterates.

Also, if f is convex in the first argument, concave in the second, and $\arg\max_{\boldsymbol{y}\in\mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y})$ *and* $\arg\min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)$ *are non-empty, then we have*

$$\max_{\boldsymbol{y}\in\mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y}) - \min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T) \leq \frac{\mathrm{Regret}_T^{\mathcal{X}}(\boldsymbol{x}'_T) + \mathrm{Regret}_T^{\mathcal{Y}}(\boldsymbol{y}'_T)}{T},$$

for any $\boldsymbol{x}'_T \in \arg\min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)$ *and* $\boldsymbol{y}'_T \in \arg\max_{\boldsymbol{y}\in\mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y})$.

Proof The first two equalities are obtained by simply observing that $\ell_t(\boldsymbol{x}) = f(\boldsymbol{x}, \boldsymbol{y}_t)$ and $h_t(\boldsymbol{y}) = -f(\boldsymbol{x}_t, \boldsymbol{y})$.

For the stated inequality, using Jensen's inequality (Theorem 2.9), we obtain

$$f(\bar{\boldsymbol{x}}_T, \boldsymbol{y}) - f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T) \leq \frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{x}_t, \boldsymbol{y}) - \frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{x}, \boldsymbol{y}_t)\,.$$

Summing the first two equalities, using the above inequality, and taking $\boldsymbol{x} = \boldsymbol{x}'_T$ and $\boldsymbol{y} = \boldsymbol{y}'_T$, we get the stated inequality. □

From this theorem, we can immediately prove the following corollary.

Corollary 15.13 *Let $\mathcal{X}$ and $\mathcal{Y}$ be non-empty closed convex sets. Let $f : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ be continuous, convex in the first argument, and concave in the second one. Assume that $\mathcal{X}$ and $\mathcal{Y}$ are compact. Consider Algorithm 15.1 and assume that the two online algorithms guarantee that their maximum regret over competitors in their feasible set is sublinear in T. Then, we have*

$$\lim_{T\to\infty} \max_{\boldsymbol{y}\in\mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y}) - \min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T) = 0\,.$$

Example 15.14 *Consider the saddle-point problem*

$$\min_{|x|\leq 10} \max_{|y|\leq 10} (x-5)(y-5)\,.$$

The saddle point of this problem is $(x, y) = (5, 5)$. We can find it using, for example, projected Online Gradient Descent (OGD) with stepsizes $\eta_t = \frac{1}{\sqrt{t}}$. So, setting $\boldsymbol{x}_1 = \boldsymbol{y}_1 = 0$, we have the iterations

$$x_{t+1} = \max\left(\min\left(x_t - \frac{1}{\sqrt{t}}(y_t - 5), 10\right), -10\right)$$
$$y_{t+1} = \max\left(\min\left(y_t + \frac{1}{\sqrt{t}}(x_t - 5), 10\right), -10\right) .$$

According to Theorem 15.12, the duality gap in $(\frac{1}{T}\sum_{t=1}^{T} x_t, \frac{1}{T}\sum_{t=1}^{T} y_t)$ converges to 0. Hence, given that the saddle point is unique, $(\frac{1}{T}\sum_{t=1}^{T} x_t, \frac{1}{T}\sum_{t=1}^{T} y_t)$ converges to the saddle point of this problem. However, the iterates (x_t, y_t) will never converge, see Figure 15.2.

Surprisingly, we can even prove the simpler version of the minimax theorem in Theorem 15.11 from the above result!

Proof of Theorem 15.11 From Lemma 15.2, we have one inequality. Hence, we now have to prove the other inequality.

We will use a constructive proof. From the assumptions of the theorem, we know that we can use, for example, two projected Online Subgradient Descent (OSD) algorithms on the losses ℓ_t and h_t. Let's use Algorithm 15.1 and Theorem 15.12. For the first player, for any $\boldsymbol{x} \in \mathcal{X}$ we have

$$\frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{x}_t, \boldsymbol{y}_t) = \frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{x}, \boldsymbol{y}_t) + \frac{\mathrm{Regret}_T^{\mathcal{X}}(\boldsymbol{x})}{T} \le f\left(\boldsymbol{x}, \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{y}_t\right) + \frac{\mathrm{Regret}_T^{\mathcal{X}}(\boldsymbol{x})}{T} .$$

Observe that

$$\min_{\boldsymbol{x}\in\mathcal{X}} f\left(\boldsymbol{x}, \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{y}_t\right) \le \max_{\boldsymbol{y}\in\mathcal{Y}} \min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \boldsymbol{y}) .$$

Given that projected OSD has $o(T)$ regret for each $\boldsymbol{x} \in \mathcal{X}$, we have

$$\frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{x}_t, \boldsymbol{y}_t) \le \max_{\boldsymbol{y}\in\mathcal{Y}} \min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \boldsymbol{y}) + o(1) .$$

In the same way, we have

$$-\frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{x}_t, \boldsymbol{y}_t) \le -\min_{\boldsymbol{x}\in\mathcal{X}} \max_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}, \boldsymbol{y}) + o(1) .$$

Summing the two inequalities, taking $T \to \infty$, and using the sublinear regret assumption, we have

$$\min_{\boldsymbol{x}\in\mathcal{X}} \max_{\boldsymbol{y}\in\mathcal{Y}} f(\boldsymbol{x}, \boldsymbol{y}) \le \max_{\boldsymbol{y}\in\mathcal{Y}} \min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \boldsymbol{y}) .$$

Together with Lemma 15.2, this implies the stated equality. □

Algorithm 15.2 Saddle-Point Optimization with OCO and $\mathcal{Y}$-Best Response

Require: $\boldsymbol{x}_1 \in \mathcal{X}$
1: **for** $t = 1, \dots, T$ **do**
2: Set $\boldsymbol{y}_t \in \operatorname{argmax}_{\boldsymbol{y} \in \mathcal{Y}} f(\boldsymbol{x}_t, \boldsymbol{y})$
3: $\mathcal{X}$-Learner receives $\ell_t(\boldsymbol{x}) = f(\boldsymbol{x}, \boldsymbol{y}_t)$ and produces $\boldsymbol{x}_{t+1} \in \mathcal{X}$
4: **end for**
5: **return** $\bar{\boldsymbol{x}}_T = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{x}_t$, $\bar{\boldsymbol{y}}_T = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{y}_t$

15.2.1 Variations with Best Response and Alternation

In some cases, it is easy to compute the max with respect to $\boldsymbol{y} \in \mathcal{Y}$ of $f(\boldsymbol{x}_t, \boldsymbol{y})$ for a given $\boldsymbol{x}_t$. For example, this is trivial for bilinear games over the probability simplex that we will see in Section 15.3. In these cases, we can remove the second learner and just use its **best response** in each round, that is, the vector that minimizes the current loss on the feasible set. Note that in this way we are making one of the two players "stronger" through the knowledge of their loss in the next round. However, this is perfectly fine: the proof in Theorem 15.12 is still perfectly valid.

In this case, the $\mathcal{Y}$-player has an easy life: it knows the loss before making the prediction, hence it can just output the minimizer of the loss in $\mathcal{Y}$. Hence, we also have that the regret of the $\mathcal{Y}$-player will be non-positive, and it will not show up in Theorem 15.12. Putting everything together, we can state the following corollary.

Corollary 15.15 *Let $\mathcal{X}$ and $\mathcal{Y}$ be non-empty closed convex sets. Let $f : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ be convex in the first argument, and concave in the second. With the notation in Algorithm 15.2, assume that $\mathcal{X}$ is compact, the argmax of the $\mathcal{Y}$-player is never empty, and $\arg\min_{\boldsymbol{x} \in \mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)$ is non-empty. Then, we have*

$$\sup_{\boldsymbol{y} \in \mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y}) - \min_{\boldsymbol{x} \in \mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T) \leq \frac{\operatorname{Regret}_T^{\mathcal{X}}(\boldsymbol{x}'_T)}{T},$$

for any $\boldsymbol{x}'_T \in \arg\min_{\boldsymbol{x} \in \mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)$ and where $\operatorname{Regret}_T^{\mathcal{X}}(\boldsymbol{x}) = \sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{x})$.

This alternative seems interesting from a theoretical point of view because it allows us to avoid the complexity of learning in the $\mathcal{Y}$ space, for example, removing the dependence on its dimension.

Remark 15.16 *The best-response algorithm requires stronger access to the function f because we need to solve an optimization problem exactly. Hence, for example, we cannot just have access to subgradients of f to use it.*

Of course, an analogous result can be stated using best response for the $\mathcal{X}$-player and an OCO algorithm for the $\mathcal{Y}$-player, as in Algorithm 15.3.

There is a third variant, very common in empirical implementations, especially of Counterfactual Regret Minimization (CFR) (Zinkevich et al., 2007). It is called **alternation** and it breaks the simultaneous reveal of the actions of the two players. Instead, we use the updated prediction of the first player to construct the loss of the second player. Empirically, this variant seems to greatly speed up the convergence of the duality gap.

Algorithm 15.3 Saddle-Point Optimization with OCO and $\mathcal{X}$-Best Response

Require: $\boldsymbol{y}_1 \in \mathcal{Y}$
1: **for** $t = 1, \dots, T$ **do**
2: Set $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{X}} f(\boldsymbol{x}, \boldsymbol{y}_t)$
3: $\mathcal{Y}$-Learner receives $h_t(\boldsymbol{y}) = -f(\boldsymbol{x}_t, \boldsymbol{y})$ and produces $\boldsymbol{y}_{t+1} \in \mathcal{Y}$
4: **end for**
5: **return** $\bar{\boldsymbol{x}}_T = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{x}_t$, $\bar{\boldsymbol{y}}_T = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{y}_t$

Algorithm 15.4 Saddle-Point Optimization with OCO and Alternation

Require: $\boldsymbol{y}_1 \in \mathcal{Y}$
1: **for** $t = 1, \dots, T$ **do**
2: $\mathcal{X}$-Learner receives $\ell_t(\boldsymbol{x}) = f(\boldsymbol{x}, \boldsymbol{y}_t)$ and produces $\boldsymbol{x}_{t+1} \in \mathcal{X}$
3: $\mathcal{Y}$-Learner receives $h_t(\boldsymbol{y}) = -f(\boldsymbol{x}_{t+1}, \boldsymbol{y})$ and produces $\boldsymbol{y}_{t+1} \in \mathcal{Y}$
4: **end for**
5: **return** $\bar{\boldsymbol{x}}_T = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{x}_{t+1}$, $\bar{\boldsymbol{y}}_T = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{y}_t$

For this version, Theorem 15.12 does not hold anymore because the terms $f(\boldsymbol{x}_t, \boldsymbol{y}_t)$ and $f(\boldsymbol{x}_{t+1}, \boldsymbol{y}_t)$ are now different. However, we can prove a similar guarantee.

Theorem 15.17 *Let $\mathcal{X}$ and $\mathcal{Y}$ be non-empty closed convex sets. Let $f : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ be convex in the first argument and concave in the second. With the notation in Algorithm 15.4, assume that $\arg\min_{\boldsymbol{x} \in \mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)$ and $\arg\max_{\boldsymbol{y} \in \mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y})$ are non-empty. Then, for any $\boldsymbol{x}'_T \in \arg\min_{\boldsymbol{x} \in \mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)$ and any $\boldsymbol{y}'_T \in \arg\max_{\boldsymbol{y} \in \mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y})$, we have*

$$\begin{aligned}&\max_{\boldsymbol{y} \in \mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y}) - \min_{\boldsymbol{x} \in \mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T) \\ &\quad \le \frac{\operatorname{Regret}_T^{\mathcal{X}}(\boldsymbol{x}'_T) + \operatorname{Regret}_T^{\mathcal{Y}}(\boldsymbol{y}'_T) + \sum_{t=1}^{T}(f(\boldsymbol{x}_{t+1}, \boldsymbol{y}_t) - f(\boldsymbol{x}_t, \boldsymbol{y}_t))}{T},\end{aligned}$$

where $\operatorname{Regret}_T^{\mathcal{Y}}(\boldsymbol{y}) = \sum_{t=1}^{T}(h_t(\boldsymbol{y}_t) - h_t(\boldsymbol{y}))$, $\operatorname{Regret}_T^{\mathcal{X}}(\boldsymbol{x}) = \sum_{t=1}^{T}(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{x}))$, where $\boldsymbol{x}_1$ is the prediction of the $\mathcal{X}$-learner before seeing ℓ_1.

Proof Note that $\ell_t(\boldsymbol{x}) = f(\boldsymbol{x}, \boldsymbol{y}_t)$, and $h_t(\boldsymbol{y}) = -f(\boldsymbol{x}_{t+1}, \boldsymbol{y})$. By Jensen's inequality

(Theorem 2.9), we have

$$
\begin{aligned}
&T(f(\bar{\boldsymbol{x}}_T, \boldsymbol{y}) - f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)) \\
&\quad \le \sum_{t=1}^{T} f(\boldsymbol{x}_{t+1}, \boldsymbol{y}) - \sum_{t=1}^{T} f(\boldsymbol{x}, \boldsymbol{y}_t) \\
&\quad = \sum_{t=1}^{T} f(\boldsymbol{x}_{t+1}, \boldsymbol{y}) - \sum_{t=1}^{T} f(\boldsymbol{x}_{t+1}, \boldsymbol{y}_t) + \sum_{t=1}^{T} f(\boldsymbol{x}_t, \boldsymbol{y}_t) - \sum_{t=1}^{T} f(\boldsymbol{x}, \boldsymbol{y}_t) \\
&\qquad + \sum_{t=1}^{T} f(\boldsymbol{x}_{t+1}, \boldsymbol{y}_t) - \sum_{t=1}^{T} f(\boldsymbol{x}_t, \boldsymbol{y}_t) \\
&\quad = \sum_{t=1}^{T} (h_t(\boldsymbol{y}_t) - h_t(\boldsymbol{y})) + \sum_{t=1}^{T} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{x})) + \sum_{t=1}^{T} (f(\boldsymbol{x}_{t+1}, \boldsymbol{y}_t) - f(\boldsymbol{x}_t, \boldsymbol{y}_t)) \,.
\end{aligned}
$$

Taking $\boldsymbol{x} = \boldsymbol{x}'_T \in \arg\min_{\boldsymbol{x} \in \mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)$ and $\boldsymbol{y} = \boldsymbol{y}'_T \in \arg\max_{\boldsymbol{y} \in \mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y})$, we get the stated result. □

Remark 15.18 *In some cases, it is possible to show that the additional term in Theorem 15.17 is negative, showing a (marginal) improvement to the convergence rate. For example, using Online Mirror Descent (OMD) for the $\mathcal{X}$-player and assuming that the losses it receives are linear, we have that $f(\boldsymbol{x}_t, \boldsymbol{y}_t) = \ell_t(\boldsymbol{x}_t) \ge \ell_t(\boldsymbol{x}_{t+1}) = f(\boldsymbol{x}_{t+1}, \boldsymbol{y}_t)$.*

Next, we will see how to connect saddle-point problems with game theory.

15.3 Game-Theory Interpretation of Saddle-Point Problems

An instantiation of a saddle-point problem also has an interpretation in game theory as a **two-player zero-sum game**. Note that game theory is a vast field, and two-person zero-sum games are only a very small subset of the problems in this domain, and what I describe here is an even smaller subset of this subset of problems.

Game theory studies what happens when self-interested agents interact. By self-interested, we mean that each agent has an ideal state of things they want to reach, which can include a description of what should happen to other agents as well, and works towards this goal. In two-person games, the players act simultaneously, and then they receive their losses. In particular, the $\mathcal{X}$-player chooses the play $\boldsymbol{x}$ and the $\mathcal{Y}$-player chooses the play $\boldsymbol{y}$, the $\mathcal{X}$-player suffers the loss $f(\boldsymbol{x}, \boldsymbol{y})$ and the $\mathcal{Y}$-player the loss $-f(\boldsymbol{x}, \boldsymbol{y})$. It is important to understand that this is only one round, that is, there is only one play for each player.

Remark 15.19 *The standard game-theoretic terminology uses payoffs instead of losses, but we will keep using losses for coherence with the OCO notation used in this book.*

We consider the so-called **two-person normal-form games**, that is, when the first player has n possible actions and the second player m. A player can use a **pure strategy**, that is, a single fixed action, or randomize over a set of actions according to some probability distribution, a so-called **mixed strategy**. In this case, we consider $\mathcal{X} = \Delta^{n-1}$ and $\mathcal{Y} = \Delta^{m-1}$ and they are known as the **mixed-strategy spaces** for the two players. In this setting,

for a pair of pure strategies $(\boldsymbol{e}_i, \boldsymbol{e}_j)$, the first player receives the loss $f(\boldsymbol{e}_i, \boldsymbol{e}_j)$ and the second player $-f(\boldsymbol{e}_i, \boldsymbol{e}_j)$, where $\boldsymbol{e}_i$ is the vector with all zeros but a '1' in position i. The goal of each player is to minimize the received loss. Given the discrete nature of this game, the function $f(\boldsymbol{x}, \boldsymbol{y})$ is the bilinear function $\boldsymbol{x}^\top \boldsymbol{M}\boldsymbol{y}$, where $\boldsymbol{M}$ is a matrix with n rows and m columns. Hence, for a pair of mixed strategies $(\boldsymbol{x}, \boldsymbol{y})$, the *expected* loss of the first player is $\boldsymbol{x}^\top \boldsymbol{M}\boldsymbol{y}$ and the one of the second player is $-\boldsymbol{x}^\top \boldsymbol{M}\boldsymbol{y}$.

A fundamental concept in game theory is the notion of **Nash equilibrium**. We have a Nash equilibrium if all players are playing their best strategy against the other players' strategies. That is, none of the players has an incentive to change their strategy if the other player does not change it. For the zero-sum two-person game, this can be formalized by saying that $(\boldsymbol{x}^\star, \boldsymbol{y}^\star)$ is a Nash equilibrium if

$$(\boldsymbol{x}^\star)^\top \boldsymbol{M}\boldsymbol{y} \leq (\boldsymbol{x}^\star)^\top \boldsymbol{M}\boldsymbol{y}^\star \leq \boldsymbol{x}^\top \boldsymbol{M}\boldsymbol{y}^\star, \quad \forall \boldsymbol{x} \in \Delta^{n-1}, \boldsymbol{y} \in \Delta^{m-1} .$$

This is *exactly* the definition of saddle point for the function $f(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{x}^\top \boldsymbol{M}\boldsymbol{y}$ that we gave in the previous section. Given that $f(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{x}^\top \boldsymbol{M}\boldsymbol{y}$ is continuous, convex in the first argument and concave in the second one, the sets $\mathcal{X} = \Delta^{n-1}$ and $\mathcal{Y} = \Delta^{m-1}$ are convex and compact, we can deduce from Theorem 15.11 and Theorem 15.3 that a saddle point always exists. Hence, there is always at least one (possibly mixed) Nash equilibrium in two-person zero-sum games. The common value of the minimax and maxmin problem is called **value of the game** and we will denote it by $v^\star$.

For a zero-sum two-person game, the Nash equilibrium has an immediate interpretation: from the definition above, if the first player uses the strategy $\boldsymbol{x}^\star$ then their loss is at most the value of the game $v^\star$, regardless of the strategy of the second player. Analogously, if the second player uses the strategy $\boldsymbol{y}^\star$ then their loss is at most $-v^\star$, regardless of the strategy of the first player. Both players achieve the value of the game if they both play the Nash strategy. Moreover, even if one of the players announced their strategy in advance to the other player, they would not increase their loss in expectation.

Example 15.20 (Cake cutting) *Suppose there is a game between two people: the first player cuts the cake in two, and the second one chooses a piece; the first player receives the piece that was not chosen. We can formalize it with the following matrix:*

	larger piece	*smaller piece*
cut evenly	*0*	*0*
cut unevenly	*10*	*-10*

When the first player plays action i and the second player action j, the first player receives the loss $M_{i,j}$ and the second player receives $-M_{i,j}$. The losses represent how much less in percentage compared to half of the cake that the first player is receiving. The second player receives the negative of the same number. It should be easy to convince oneself that the strategy pair $(\textit{cut evenly}, \textit{larger piece})$ is an equilibrium with value of the game of 0. However, perhaps counterintuitively, this is not the only Nash equilibrium strategy. In fact, for $0.5 \leq a \leq 1$, one can verify numerically that any pair $\boldsymbol{x}^\star = [1, 0]^\top$, $\boldsymbol{y}^\star = [a, 1-a]$ is optimal, where the first player uses a pure strategy and the second player a mixed one.

Example 15.21 (Rock-Paper-Scissors) *Let's consider the game of Rock-Paper-Scissors.*

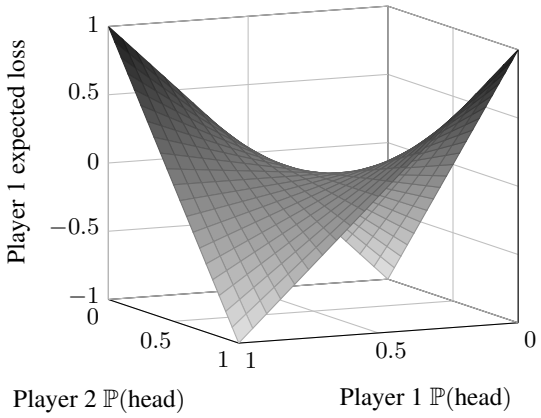


Figure 15.3 The saddle point in the Matching Pennies game.

We describe it with the following matrix:

	Rock	*Paper*	*Scissors*
Rock	*0*	*1*	*-1*
Paper	*-1*	*0*	*1*
Scissors	*1*	*-1*	*0*

It should be immediate to realize that there are no pure Nash equilibria for this game. However, there is a mixed Nash equilibrium when each player randomizes the action with a uniform probability distribution over the three actions, and the value of the game equals 0.

Example 15.22 (Matching Pennies) *In this game, both players show a face of a penny. If the two faces are the same, the first player wins both, otherwise the second player wins both. The associated matrix $\boldsymbol{M}$ is*

	head	*tail*
head	*-1*	*1*
tail	*1*	*-1*

It is easy to see that the Nash equilibrium is when both players randomize the face to show with equal probability.

In this simple case, we can visualize the saddle point associated with this problem in Figure 15.3.

Unless the game is very small, we find Nash equilibria using numerical procedures that typically give us only approximate solutions. Hence, as for ϵ-saddle points, for $\epsilon \geq 0$ we also define an **ϵ-Nash equilibrium** for a zero-sum two-person game when $\boldsymbol{x}^\star$ and $\boldsymbol{y}^\star$ satisfy

$$(\boldsymbol{x}^\star)^\top \boldsymbol{M}\boldsymbol{y} - \epsilon \leq (\boldsymbol{x}^\star)^\top \boldsymbol{M}\boldsymbol{y}^\star \leq \boldsymbol{x}^\top \boldsymbol{M}\boldsymbol{y}^\star + \epsilon, \quad \forall \boldsymbol{x} \in \Delta^{n-1}, \boldsymbol{y} \in \Delta^{m-1} .$$

Obviously, any Nash equilibrium is also an ϵ-Nash equilibrium.

From what we said in the previous section, it should be immediate to see how to numerically calculate the Nash equilibrium of a two-person zero-sum game. In fact, we know

that we can use online convex optimization algorithms to find ϵ-saddle points, so we can do the same for an ϵ-Nash equilibrium of two-person zero-sum games. Assuming that the average regret of both players is ϵ_T, Theorem 15.12 says that $\left(\frac{1}{T}\sum_{t=1}^{T}\boldsymbol{x}_t, \frac{1}{T}\sum_{t=1}^{T}\boldsymbol{y}_t\right)$ is a $2\epsilon_T$-Nash equilibrium.

15.3.1 Reducing Extensive to Normal-Form Zero-Sum Games

The connection between saddle-point optimization and two-person zero-sum games extends beyond the static case. Many strategic interactions are *sequential*: players move in turns, possibly with knowledge of previous actions. These games are naturally represented as **extensive-form games**, whose solution concepts can be related to the saddle-point structure of their normal forms.

A **perfect-information extensive-form zero-sum game** is described by a game tree in which nodes correspond to decision points, edges correspond to actions, and leaves specify the losses. Each non-terminal node is assigned to one of the two players, and all players observe all previous moves. The game is *zero-sum* if the sum of the players' losses at every terminal node is zero.

For example, consider the simple two-level perfect-information game in Figure 15.4. Player 1 moves first by choosing between A and B. Then, Player 2 observes the choice and selects L or R in one case and P and Q in another case. Each terminal leaf specifies the loss for Player 1 (Player 2's loss is its negation).

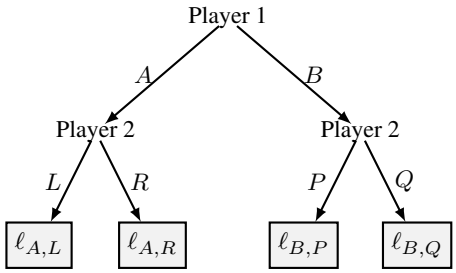


Figure 15.4 Example of a perfect-information extensive-form zero-sum game.

We can always convert such a game into the normal form by listing all the players' *pure strategies*—that is, complete contingent plans specifying what a player would do at every node where it could move.

In the example above, Player 1 has two possible pure strategies: $\{A, B\}$. Player 2, however, must decide what to do after each of Player 1's actions, hence has $2 \times 2 = 4$ pure strategies:

$$\mathcal{Y} = \{(L, P), (L, Q), (R, P), (R, Q)\},$$

where (L, P) means "play L after A and P after B," etc.

The extensive game can then be rewritten as a normal-form loss matrix

	(L,P)	(L,Q)	(R,P)	(R,Q)
A	$\ell_{A,L}$	$\ell_{A,L}$	$\ell_{A,R}$	$\ell_{A,R}$
B	$\ell_{B,P}$	$\ell_{B,Q}$	$\ell_{B,P}$	$\ell_{B,Q}$

Each entry corresponds to the leaf reached by the combination of both players' strategy profiles. Once this conversion is performed, standard saddle-point and minimax results (like Theorem 15.11) apply directly.

It should also be clear from the above example that the conversion from extensive form to the normal form can result in an exponential blowup of the representation of the game.

A foundational result on perfect-information extensive-form games is due to Zermelo (1913).

Theorem 15.23 (Zermelo's Theorem) *Every finite perfect-information game in extensive form has a pure-strategy Nash equilibrium.*

When players move without observing all past actions, the game has **imperfect information**. In this case, decision nodes are grouped into **information sets**, and a player must take the same action at all nodes within an information set. This also means that we will have the same set of actions in all nodes within an information set. A classic example is poker, where a player knows their own cards but not the opponent's.

Even imperfect-information extensive-form games can still be expressed as a normal-form game: each player's pure strategies again correspond to complete contingent plans, including choices at all information sets.

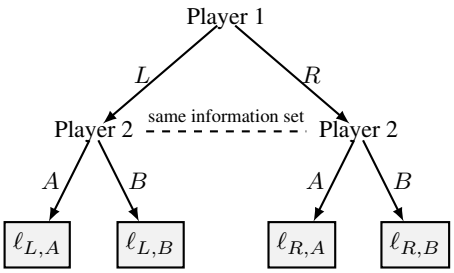


Figure 15.5 An imperfect-information game: Player 2 does not observe Player 1's action (L or R).

In Figure 15.5, Player 2's two decision nodes are connected by a dashed line, indicating that they form the same information set. Player 2 must choose the same move (A or B) regardless of whether the game reached the left or right subtree.

The corresponding normal-form representation again lists all pure strategies. However, now these plans must respect the information constraints. The resulting loss matrix is

	A	B
L	$\ell_{L,A}$	$\ell_{L,B}$
R	$\ell_{R,A}$	$\ell_{R,B}$

where Player 2's choice (A or B) must be the same across both branches.

15.4 Application: Boosting as a Two-Person Game

We will now show that the *boosting* problem can also be seen as the solution of a zero-sum two-person game.

Let $\mathcal{S} = \{\boldsymbol{z}_j, y_j\}_{j=1}^m$ be a training set of m feature/label pairs, where $\boldsymbol{z}_j \in \mathbb{R}^d$ and $y_j \in \{-1, 1\}$. Let $\mathcal{H} = \{h_i\}_{i=1}^n$ be a set of n functions $h_i : \mathbb{R}^d \to \{-1, 1\}$. The aim of boosting is to find a combination of the functions in $\mathcal{H}$ that has arbitrarily low misclassification error on $\mathcal{S}$. Of course, this is not always possible. However, we will make a (strong) assumption: we assume that it is always possible to find a function $h_i \in \mathcal{H}$ such that its misclassification error over $\mathcal{S}$ weighted by any probability distribution is better than chance by a constant $\gamma > 0$. We now show that this assumption guarantees that the boosting problem is solvable!

First, we construct a matrix of the misclassifications for each function: $\boldsymbol{M} \in \mathbb{R}^{n \times m}$ where

$$M_{i,j} = \begin{cases} 1, & \text{if } h_i(\boldsymbol{z}_j) \neq y_j \\ 0, & \text{otherwise.} \end{cases}$$

Setting $\mathcal{X} = \Delta^{n-1}$ and $\mathcal{Y} = \Delta^{m-1}$, we write the saddle-point problem/two-person zero-sum game

$$\min_{\boldsymbol{p} \in \Delta^{n-1}} \max_{\boldsymbol{q} \in \Delta^{m-1}} \boldsymbol{p}^\top \boldsymbol{M} \boldsymbol{q} \, .$$

Given the definition of the matrix $\boldsymbol{M}$, this is equivalent to

$$\min_{\boldsymbol{p} \in \Delta^{n-1}} \max_{\boldsymbol{q} \in \Delta^{m-1}} \sum_{i=1}^n \sum_{j=1}^m p_i q_j \mathbf{1}\{h_i(\boldsymbol{z}_j) \neq y_j\} \, .$$

Let's now formalize the assumption on the functions: we assume the existence of a **weak learning oracle** that, for any $\boldsymbol{q} \in \Delta^{m-1}$, returns $i^\star$ such that

$$\boldsymbol{e}_{i^\star}^\top \boldsymbol{M} \boldsymbol{q} = \sum_{j=1}^m q_j \mathbf{1}\{h_{i^\star}(\boldsymbol{z}_j) \neq y_j\} \leq \frac{1}{2} - \gamma, \tag{15.4}$$

where $\gamma > 0$. In words, $i^\star$ is the index of the function in $\mathcal{H}$ that gives a $\boldsymbol{q}$-weighted error better than chance. Moreover, given that

$$\min_{\boldsymbol{p} \in \Delta^{n-1}} \boldsymbol{p}^\top \boldsymbol{M} \boldsymbol{q} \leq \boldsymbol{e}_{i^\star}^\top \boldsymbol{M} \boldsymbol{q} \leq \frac{1}{2} - \gamma$$

and

$$v^\star = \max_{\boldsymbol{q} \in \Delta^{m-1}} \min_{\boldsymbol{p} \in \Delta^{n-1}} \boldsymbol{p}^\top \boldsymbol{M} \boldsymbol{q},$$

we have that the value of the game satisfies $v^\star \leq \frac{1}{2} - \gamma < \frac{1}{2}$. Using the inequality on the value of the game and the fact that the Nash equilibrium exists, we obtain that there exists $\boldsymbol{p}^\star \in \Delta^{n-1}$ such that

$$\sum_{i=1}^n p_i^\star \mathbf{1}\{h_i(\boldsymbol{z}_j) \neq y_j\} = (\boldsymbol{p}^\star)^\top \boldsymbol{M} \boldsymbol{e}_j \leq v^\star \leq \frac{1}{2} - \gamma < \frac{1}{2}, \quad \forall j = 1, \ldots, m \, . \tag{15.5}$$

In words, this means that every sample $\boldsymbol{z}_j$ is misclassified by less than half of the functions

Algorithm 15.5 Boosting through Online Convex Optimization

1: $q_{1,j} = 1/m, j = 1, \ldots, m$
2: **for** $t = 1, \ldots, T$ **do**
3: $\mathcal{X}$-Learner has two possible strategies:
(i) (Approximate best response) Set i_t such that $\boldsymbol{e}_{i_t}^\top \boldsymbol{M}\boldsymbol{q}_t = \sum_{j=1}^m q_{t,j}\mathbf{1}\{h_{i_t}(\boldsymbol{z}_j) \neq y_j\} \leq 1/2 - \gamma$
(ii) (Best response) Set $i_t = \operatorname{argmin}_{i=1,\ldots,n} \boldsymbol{e}_i^\top \boldsymbol{M}\boldsymbol{q}_t = \operatorname{argmin}_{i=1,\ldots,n} \sum_{j=1}^m q_{t,j}\mathbf{1}\{h_i(\boldsymbol{z}_j) \neq y_j\}$
4: $\mathcal{Y}$-Learner receives $\ell_t(\boldsymbol{q}) = -\boldsymbol{e}_{i_t}^\top \boldsymbol{M}\boldsymbol{q} = -\sum_{j=1}^m q_j \mathbf{1}\{h_{i_t}(\boldsymbol{z}_j) \neq y_j\}$ and produces $\boldsymbol{q}_{t+1} \in \Delta^{m-1}$
5: **end for**
6: **return** $\bar{h}_T(\boldsymbol{z}) = \frac{1}{T}\sum_{t=1}^T h_{i_t}(\boldsymbol{z})$

h_i when weighted by $\boldsymbol{p}^\star$. Hence, *we can correctly classify all the samples using a weighted majority vote rule where the weights over the function h_i are $\boldsymbol{p}^\star$*. This means that we can learn a perfect classifier rule using weak learners, through the solution of a minimax game. So, our job is to find a way to calculate this optimal distribution $\boldsymbol{p}^\star$ on the functions.

Given what we have said so far, a natural strategy is to use OCO algorithms. In particular, we can use Algorithm 15.3, where in each round the $\mathcal{X}$-player is the weak learning oracle, that knows the play $\boldsymbol{q}_t$ by the $\mathcal{Y}$-Learner and produces a best response or an approximate best response that returns a weighted classification error less than or equal to $\frac{1}{2} - \gamma$. In particular, from the assumption on the weak-learnability oracle in (15.4), the $\mathcal{X}$-learner can always find a classifier with small enough $\boldsymbol{q}_t$-weighted training error. Instead, the $\mathcal{Y}$-player is an OCO algorithm. Specialized to our setting, we have Algorithm 15.5. In words, the $\mathcal{X}$-player looks for the function that has a small enough weighted misclassification loss, where the weights are constructed by the $\mathcal{Y}$-player.

Let's show a guarantee on the misclassification error of this algorithm. From the second equality of Theorem 15.12, for any $\boldsymbol{q} \in \Delta^{m-1}$, we have

$$\frac{1}{T}\sum_{t=1}^T \boldsymbol{e}_{i_t}^\top \boldsymbol{M}\boldsymbol{q} = \frac{1}{T}\sum_{t=1}^T \boldsymbol{e}_{i_t}^\top \boldsymbol{M}\boldsymbol{q}_t + \frac{\operatorname{Regret}_T^{\mathcal{Y}}(\boldsymbol{q})}{T} .$$

From both strategies of the $\mathcal{X}$-player, we have $\frac{1}{T}\sum_{t=1}^T \boldsymbol{e}_{i_t}^\top \boldsymbol{M}\boldsymbol{q}_t \leq \frac{1}{2} - \gamma$. Moreover, choosing $\boldsymbol{q} = \boldsymbol{e}_j$ we have $\frac{1}{T}\sum_{t=1}^T \boldsymbol{e}_{i_t}^\top \boldsymbol{M}\boldsymbol{q} = \frac{1}{T}\sum_{t=1}^T \mathbf{1}\{h_{i_t}(\boldsymbol{z}_j) \neq y_j\}$. Putting everything together, we have

$$\frac{1}{T}\sum_{t=1}^T \mathbf{1}\{h_{i_t}(\boldsymbol{z}_j) \neq y_j\} \leq \frac{1}{2} - \gamma + \frac{\operatorname{Regret}_T^{\mathcal{Y}}(\boldsymbol{e}_j)}{T}, \quad \forall j = 1, \ldots, m .$$

If $\frac{\operatorname{Regret}_T^{\mathcal{Y}}(\boldsymbol{e}_j)}{T} < \gamma$, less than half of the functions selected by the boosting procedure will make a mistake on $(\boldsymbol{z}_j, y_j)$. Using the majority rule of the selected functions as a predictor, we will have 0 mistakes on the training samples.

Observe that in this scheme, we approximate $p_i^\star$ with the frequency with which i_t is equal to i.

Remark 15.24 *The use of a best response or approximate best response for the $\mathcal{X}$-player avoids the presence of its regret in the guarantee. In turn, this makes the number of rounds to reach training error 0 independent of the number of functions in $\mathcal{H}$.*

Let's now instantiate this framework with a specific OCO algorithm. For example, using Exponentiated Gradient (EG) as algorithm for the $\mathcal{Y}$-player, we have that $\operatorname{Regret}_T^{\mathcal{Y}}(\boldsymbol{e}_j) = \mathcal{O}(\sqrt{T \ln m})$ as $T \to \infty$ for any $j = 1, \dots, m$, that implies that after $T = \mathcal{O}(\frac{\ln m}{\gamma^2})$ rounds the error on the training samples of the classifier that uses the sign of $\bar{h}_T$ is exactly 0. This is exactly the same guarantee achieved by AdaBoost (Freund and Schapire, 1995, 1997).

Boosting and Margins. What happens if we keep boosting after the training error reaches 0? It turns out we maximize the **margin**, defined as $y_j \frac{1}{T} \sum_{t=1}^{T} h_{i_t}(\boldsymbol{z}_j)$. In fact, given that $\mathbf{1}\{h_{i_t}(\boldsymbol{z}_j) \neq y_j\} = \frac{1 - y_j h_{i_t}(\boldsymbol{z}_j)}{2}$, we have for any $j = 1, \dots, m$

$$2\gamma - \frac{2 \operatorname{Regret}_T^{\mathcal{Y}}(\boldsymbol{e}_j)}{T} \leq y_j \frac{1}{T} \sum_{t=1}^{T} h_{i_t}(\boldsymbol{z}_j) = y_j \bar{h}_T(\boldsymbol{z}_j) \; .$$

Hence, when the number of rounds goes to infinity, the minimum margin on the training samples reaches at least 2γ. The property of boosting of guaranteeing a minimum margin has been used as an explanation of the fact that in boosting additional rounds after the training error reaches 0 often keep improving the test error on test samples coming i.i.d. from the same distribution that generated the training samples.

The above reduction does not tell us how the training error precisely behaves. However, we can get this information by changing the Learning with Expert Advice (LEA) algorithm. Indeed, we have seen in Chapter 13 that there are LEA algorithms provably better than EG. We can use the parameter-free LEA algorithm in Section 13.7 that guarantees a regret $\mathcal{O}(\sqrt{T \cdot \operatorname{KL}(\boldsymbol{q}; \boldsymbol{\pi})})$ as $T \to \infty$ for a prior $\boldsymbol{\pi}$, where we set $\boldsymbol{\pi}$ equal to the uniform prior. These kinds of algorithms allow us to upper bound the fraction of mistakes after any T iterations. Denote by $\mathcal{I}_T$ the set of samples misclassified by $\operatorname{sign}(\bar{h}_T)$ after T iterations of boosting, let $k = |\mathcal{I}_T|$, and, if $k > 0$, set $\boldsymbol{q}_k$ as the vector whose coordinate j is $1/k$ if $j \in \mathcal{I}_T$ and 0 otherwise. Hence, we have

$$2\gamma - \frac{2 \operatorname{Regret}_T^{\mathcal{Y}}(\boldsymbol{q}_k)}{T} \leq \frac{1}{k} \sum_{j \in \mathcal{I}_T} y_j \bar{h}_T(\boldsymbol{z}_j) \leq 0 \; . \tag{15.6}$$

Using the expression of the regret, we have that the fraction of misclassified samples $\frac{k}{m}$ is bounded by $\mathcal{O}(\exp(-\gamma^2 T))$ as $T \to \infty$. That is, the fraction of misclassification error goes to zero exponentially fast in the number of boosting rounds. Again, a similar guarantee was proved for AdaBoost, but here we quickly derived it using a reduction from boosting to LEA, passing through zero-sum two-person games.

15.5 Faster Rates Through Optimism

We saw that it is possible to solve convex/concave saddle-point optimization problems using two OCO algorithms playing against each other. We obtained a rate of convergence for the duality gap of $\mathcal{O}(1/\sqrt{T})$. Now, we show that if the function is smooth, we can achieve a faster rate using *optimistic* algorithms.

Algorithm 15.6 Solving Saddle-Point Problems with Optimistic FTRL

Require: $\mathcal{X}$ and $\mathcal{Y}$ non-empty closed convex sets, regularizers $\psi_{\mathcal{X}}$ and $\psi_{\mathcal{Y}}$, $\lambda_{\mathcal{X}} > 0$, $\lambda_{\mathcal{Y}} > 0$

$\boldsymbol{g}_{\mathcal{X},0} = \mathbf{0}, \boldsymbol{g}_{\mathcal{Y},0} = \mathbf{0}$
for $t = 1, \dots, T$ **do**
  $\boldsymbol{x}_t = \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{X}} \ \psi_{\mathcal{X}}(\boldsymbol{x}) + \langle \boldsymbol{g}_{\mathcal{X},t-1}, \boldsymbol{x}\rangle + \sum_{i=1}^{t-1}\langle \boldsymbol{g}_{\mathcal{X},i}, \boldsymbol{x}\rangle$
  $\boldsymbol{y}_t = \operatorname{argmin}_{\boldsymbol{y}\in\mathcal{Y}} \ \psi_{\mathcal{Y}}(\boldsymbol{y}) + \langle \boldsymbol{g}_{\mathcal{Y},t-1}, \boldsymbol{y}\rangle + \sum_{i=1}^{t-1}\langle \boldsymbol{g}_{\mathcal{Y},i}, \boldsymbol{y}\rangle$
  Set $\boldsymbol{g}_{\mathcal{X},t} = \nabla_{\boldsymbol{x}} f(\boldsymbol{x}_t, \boldsymbol{y}_t)$
  Set $\boldsymbol{g}_{\mathcal{Y},t} = -\nabla_{\boldsymbol{y}} f(\boldsymbol{x}_t, \boldsymbol{y}_t)$
end for
return $\bar{\boldsymbol{x}}_T = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{x}_t, \bar{\boldsymbol{y}}_T = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{y}_t$

Assume that $f : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ is smooth in an open set containing its domain, in the sense that for any $\boldsymbol{x}, \boldsymbol{x}' \in \mathcal{X}$ and $\boldsymbol{y}, \boldsymbol{y}' \in \mathcal{Y}$, we have

$$\|\nabla_{\boldsymbol{x}} f(\boldsymbol{x}, \boldsymbol{y}) - \nabla_{\boldsymbol{x}} f(\boldsymbol{x}', \boldsymbol{y})\|_{\mathcal{X},\star} \le L_{\mathcal{X}\mathcal{X}} \|\boldsymbol{x} - \boldsymbol{x}'\|_{\mathcal{X}}, \tag{15.7}$$
$$\|\nabla_{\boldsymbol{x}} f(\boldsymbol{x}, \boldsymbol{y}) - \nabla_{\boldsymbol{x}} f(\boldsymbol{x}, \boldsymbol{y}')\|_{\mathcal{X},\star} \le L_{\mathcal{X}\mathcal{Y}} \|\boldsymbol{y} - \boldsymbol{y}'\|_{\mathcal{Y}}, \tag{15.8}$$
$$\|\nabla_{\boldsymbol{y}} f(\boldsymbol{x}, \boldsymbol{y}) - \nabla_{\boldsymbol{y}} f(\boldsymbol{x}', \boldsymbol{y})\|_{\mathcal{Y},\star} \le L_{\mathcal{X}\mathcal{Y}} \|\boldsymbol{x} - \boldsymbol{x}'\|_{\mathcal{X}}, \tag{15.9}$$
$$\|\nabla_{\boldsymbol{y}} f(\boldsymbol{x}, \boldsymbol{y}) - \nabla_{\boldsymbol{y}} f(\boldsymbol{x}, \boldsymbol{y}')\|_{\mathcal{Y},\star} \le L_{\mathcal{Y}\mathcal{Y}} \|\boldsymbol{y} - \boldsymbol{y}'\|_{\mathcal{Y}}, \tag{15.10}$$

where $\nabla_{\boldsymbol{x}}$ and $\nabla_{\boldsymbol{y}}$ denote the gradients with respect to the first and second variable respectively, and we have denoted by $\|\cdot\|_{\mathcal{X}}$ and $\|\cdot\|_{\mathcal{Y}}$ the norms on the ambient spaces of $\mathcal{X}$ and $\mathcal{Y}$ respectively, while the norms with the $\star$ are their duals.

Remark 15.25 *At this point, one might be tempted to consider the maximum between the three quantities "to simplify the math", but the units are different, and it does not make any sense to take the maximum between quantities with different units.*

Let's use two online algorithms again to solve the saddle-point problem. However, instead of using two standard no-regret algorithms, we will use two optimistic ones, as the ones we saw in Sections 6.11 and 7.12. We will use the same strategy and proof of the algorithm in Section 7.15, that is, we will use the previous observed gradient as a prediction for the next one.

For example, use two Optimistic Follow-the-Regularized-Leader (FTRL) algorithms with fixed strongly convex regularizers and hint at time t constructed using the previous observed gradient: $\tilde{\ell}_t(\boldsymbol{x}) = \langle \boldsymbol{g}_{t-1}, \boldsymbol{x}\rangle$ where we set $\boldsymbol{g}_0 = 0$, see Algorithm 15.6. We now show that these hints allow us to cancel out terms when we consider the sum of the regrets and obtain a faster rate of $\mathcal{O}(1/T)$ rather than just $\mathcal{O}(1/\sqrt{T})$.

From the regret of Optimistic FTRL in Theorem 7.47, for the $\mathcal{X}$-player, for any $\boldsymbol{u} \in \mathcal{X}$,

we have

$$\sum_{t=1}^{T}\langle \boldsymbol{g}_{\mathcal{X},t}, \boldsymbol{x}_t - \boldsymbol{u}\rangle \le \psi_{\mathcal{X}}(\boldsymbol{u}) - \psi_{\mathcal{X}}(\boldsymbol{x}_1) + \sum_{t=1}^{T}\left(\langle \boldsymbol{g}_{\mathcal{X},t} - \boldsymbol{g}_{\mathcal{X},t-1}, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{\lambda_{\mathcal{X}}}{2}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|_{\mathcal{X}}^2\right) .$$

From the Fenchel–Young's inequality, we have

$$\begin{aligned}
&\langle \boldsymbol{g}_{\mathcal{X},t} - \boldsymbol{g}_{\mathcal{X},t-1}, \boldsymbol{x}_t - \boldsymbol{x}_{t+1}\rangle - \frac{\lambda_{\mathcal{X}}}{2}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|_{\mathcal{X}}^2 \\
&\quad\le \frac{\lambda_{\mathcal{X}}}{4}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|_{\mathcal{X}}^2 + \frac{1}{\lambda_{\mathcal{X}}}\|\boldsymbol{g}_{\mathcal{X},t} - \boldsymbol{g}_{\mathcal{X},t-1}\|_{\mathcal{X},\star}^2 - \frac{\lambda_{\mathcal{X}}}{2}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|_{\mathcal{X}}^2 \\
&\quad= \frac{1}{\lambda_{\mathcal{X}}}\|\boldsymbol{g}_{\mathcal{X},t} - \boldsymbol{g}_{\mathcal{X},t-1}\|_{\mathcal{X},\star}^2 - \frac{\lambda_{\mathcal{X}}}{4}\|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|_{\mathcal{X}}^2 .
\end{aligned}$$

Note that there are multiple choices of the coefficient in the Fenchel–Young's inequality, but without additional information, all choices are equally good.

Now, using the smoothness assumption, for $t \ge 2$ we have

$$\begin{aligned}
&\|\boldsymbol{g}_{\mathcal{X},t} - \boldsymbol{g}_{\mathcal{X},t-1}\|_{\mathcal{X},\star}^2 \\
&= \|\nabla_{\boldsymbol{x}} f(\boldsymbol{x}_t, \boldsymbol{y}_t) - \nabla_{\boldsymbol{x}} f(\boldsymbol{x}_{t-1}, \boldsymbol{y}_{t-1})\|_{\mathcal{X},\star}^2 \\
&\le \left(\|\nabla_{\boldsymbol{x}} f(\boldsymbol{x}_t, \boldsymbol{y}_t) - \nabla_{\boldsymbol{x}} f(\boldsymbol{x}_{t-1}, \boldsymbol{y}_t)\|_{\mathcal{X},\star} + \|\nabla_{\boldsymbol{x}} f(\boldsymbol{x}_{t-1}, \boldsymbol{y}_t) - \nabla_{\boldsymbol{x}} f(\boldsymbol{x}_{t-1}, \boldsymbol{y}_{t-1})\|_{\mathcal{X},\star}\right)^2 \\
&\le 2L_{\mathcal{X}\mathcal{X}}^2\|\boldsymbol{x}_{t-1} - \boldsymbol{x}_t\|_{\mathcal{X}}^2 + 2L_{\mathcal{X}\mathcal{Y}}^2\|\boldsymbol{y}_{t-1} - \boldsymbol{y}_t\|_{\mathcal{Y}}^2 .
\end{aligned}$$

We can proceed in the exact same way for the $\mathcal{Y}$-player too.

Summing the regret of the two algorithms, we have

$$\begin{aligned}
\sum_{t=1}^{T}(f(\boldsymbol{x}_t, \boldsymbol{y}) - f(\boldsymbol{x}, \boldsymbol{y}_t)) \le{}& \psi_{\mathcal{X}}(\boldsymbol{x}) - \psi_{\mathcal{X}}(\boldsymbol{x}_1) + \psi_{\mathcal{Y}}(\boldsymbol{y}) - \psi_{\mathcal{Y}}(\boldsymbol{y}_1) \\
&+ \frac{\|\boldsymbol{g}_{\mathcal{X},1}\|_{\mathcal{X},\star}^2}{\lambda_{\mathcal{X}}} + \frac{\|\boldsymbol{g}_{\mathcal{Y},1}\|_{\mathcal{Y},\star}^2}{\lambda_{\mathcal{Y}}} \\
&+ \sum_{t=2}^{T}\left(\frac{2L_{\mathcal{X}\mathcal{X}}^2}{\lambda_{\mathcal{X}}} + \frac{2L_{\mathcal{X}\mathcal{Y}}^2}{\lambda_{\mathcal{Y}}} - \frac{\lambda_{\mathcal{X}}}{4}\right)\|\boldsymbol{x}_t - \boldsymbol{x}_{t-1}\|_{\mathcal{X}}^2 \\
&+ \sum_{t=2}^{T}\left(\frac{2L_{\mathcal{Y}\mathcal{Y}}^2}{\lambda_{\mathcal{Y}}} + \frac{2L_{\mathcal{X}\mathcal{Y}}^2}{\lambda_{\mathcal{X}}} - \frac{\lambda_{\mathcal{Y}}}{4}\right)\|\boldsymbol{y}_t - \boldsymbol{y}_{t-1}\|_{\mathcal{Y}}^2 .
\end{aligned}$$

Choosing $\lambda_{\mathcal{X}} \ge 2\sqrt{2}(L_{\mathcal{X}\mathcal{X}} + L_{\mathcal{X}\mathcal{Y}}\alpha)$ and $\lambda_{\mathcal{Y}} \ge 2\sqrt{2}(L_{\mathcal{Y}\mathcal{Y}} + L_{\mathcal{X}\mathcal{Y}}/\alpha)$ for any $\alpha > 0$ kills all the terms in the sum. In fact, we have

$$\frac{2L_{\mathcal{X}\mathcal{X}}^2}{\lambda_{\mathcal{X}}} + \frac{2L_{\mathcal{X}\mathcal{Y}}^2}{\lambda_{\mathcal{Y}}} \le \frac{2L_{\mathcal{X}\mathcal{X}}^2}{2\sqrt{2}L_{\mathcal{X}\mathcal{X}}} + \frac{2L_{\mathcal{X}\mathcal{Y}}^2\alpha}{2\sqrt{2}L_{\mathcal{X}\mathcal{Y}}} \le \frac{\lambda_{\mathcal{X}}}{4},$$

and similarly for the other term. One might wonder why we need to introduce α and if it can be just set to 1. However, α has units, and it allows the sum of the smoothness coefficients with different units, so it is better to keep it around to remember it.

Assuming that the regularizers are bounded over $\mathcal{X}$ and $\mathcal{Y}$ and using the usual online-to-batch conversion, we have that the duality gap evaluated at the pair $\left(\frac{1}{T}\sum_{t=1}^T \boldsymbol{x}_t, \frac{1}{T}\sum_{t=1}^T \boldsymbol{y}_t\right)$ goes to zero as $\mathcal{O}(1/T)$ when $T \to \infty$.

Overall, we can state the following theorem.

Theorem 15.26 *Let $\mathcal{X}$ and $\mathcal{Y}$ be non-empty closed convex sets. With the notation in Algorithm 15.6, let $f : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ be convex in the first argument and concave in the second, satisfying assumptions* (15.7)-(15.10)*. For a fixed $\alpha > 0$, let $\lambda_{\mathcal{X}} \geq 2\sqrt{2}(L_{\mathcal{X}\mathcal{X}} + L_{\mathcal{X}\mathcal{Y}}\alpha)$ and $\lambda_{\mathcal{Y}} \geq 2\sqrt{2}(L_{\mathcal{Y}\mathcal{Y}} + L_{\mathcal{X}\mathcal{Y}}/\alpha)$. Let $\psi_{\mathcal{X}} : \mathcal{X} \to \mathbb{R}$ be $\lambda_{\mathcal{X}}$-strongly convex with respect to $\|\cdot\|_{\mathcal{X}}$ and $\psi_{\mathcal{Y}} : \mathcal{Y} \to \mathbb{R}$ be $\lambda_{\mathcal{Y}}$-strongly convex with respect to $\|\cdot\|_{\mathcal{Y}}$. Assume that $\arg\max_{\boldsymbol{y}\in\mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y})$ and $\arg\min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)$ are non-empty. Then, we have*

$$\begin{aligned}&\max_{\boldsymbol{y}\in\mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y}) - \min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)\\ &\quad\leq \frac{\psi_{\mathcal{X}}(\boldsymbol{x}'_T) - \psi_{\mathcal{X}}(\boldsymbol{x}_1) + \psi_{\mathcal{Y}}(\boldsymbol{y}'_T) - \psi_{\mathcal{Y}}(\boldsymbol{y}_1) + \frac{\|\boldsymbol{g}_{\mathcal{X},1}\|^2_{\mathcal{X},\star}}{\lambda_{\mathcal{X}}} + \frac{\|\boldsymbol{g}_{\mathcal{Y},1}\|^2_{\mathcal{Y},\star}}{\lambda_{\mathcal{Y}}}}{T},\end{aligned}$$

for any $\boldsymbol{x}'_T \in \arg\min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)$ and $\boldsymbol{y}'_T \in \arg\max_{\boldsymbol{y}\in\mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y})$.

Looking back at the proof of the algorithm, we have a faster convergence because the regret of one player depends on the "stability" of the other player, measured by the terms $\|\boldsymbol{x}_t - \boldsymbol{x}_{t-1}\|^2_{\mathcal{X}}$ and $\|\boldsymbol{y}_t - \boldsymbol{y}_{t-1}\|^2_{\mathcal{Y}}$. Hence, we have a sort of stabilization loop in which the stability of one algorithm makes the other more stable, which in turn stabilizes the first one even more. Indeed, we can also show that the regret of the two algorithms is not growing over time. Note that such a result cannot be obtained just by looking at the fact that the sum of the regret does not grow over time.

In fact, setting for example $\lambda_{\mathcal{X}} \geq 4\sqrt{2}(L_{\mathcal{X}\mathcal{X}} + L_{\mathcal{X}\mathcal{Y}}\alpha)$ and $\lambda_{\mathcal{Y}} \geq 4\sqrt{2}(L_{\mathcal{Y}\mathcal{Y}} + L_{\mathcal{X}\mathcal{Y}}/\alpha)$, we have that

$$\frac{2L^2_{\mathcal{Y}\mathcal{Y}}}{\lambda_{\mathcal{Y}}} + \frac{2L^2_{\mathcal{X}\mathcal{Y}}}{\lambda_{\mathcal{X}}} - \frac{\lambda_{\mathcal{Y}}}{4} \leq -\frac{\lambda_{\mathcal{Y}}}{8}$$

and

$$\frac{2L^2_{\mathcal{X}\mathcal{X}}}{\lambda_{\mathcal{X}}} + \frac{2L^2_{\mathcal{X}\mathcal{Y}}}{\lambda_{\mathcal{Y}}} - \frac{\lambda_{\mathcal{X}}}{4} \leq -\frac{\lambda_{\mathcal{X}}}{8}\,.$$

Hence, using the fact that the existence of a saddle point $(\boldsymbol{x}^\star, \boldsymbol{y}^\star)$ guarantees that $f(\boldsymbol{x}_t, \boldsymbol{y}^\star) - f(\boldsymbol{x}^\star, \boldsymbol{y}_t) \geq 0$, we have

$$\begin{aligned}&\sum_{t=2}^T \left(\frac{\lambda_{\mathcal{X}}}{8}\|\boldsymbol{x}_t - \boldsymbol{x}_{t-1}\|^2_{\mathcal{X}} + \frac{\lambda_{\mathcal{Y}}}{8}\|\boldsymbol{y}_t - \boldsymbol{y}_{t-1}\|^2_{\mathcal{Y}}\right)\\ &\quad\leq \psi_{\mathcal{X}}(\boldsymbol{x}^\star) - \psi_{\mathcal{X}}(\boldsymbol{x}_1) + \psi_{\mathcal{Y}}(\boldsymbol{y}^\star) - \psi_{\mathcal{Y}}(\boldsymbol{y}_1) + \frac{\|\boldsymbol{g}_{\mathcal{X},1}\|^2_{\mathcal{X},\star}}{\lambda_{\mathcal{X}}} + \frac{\|\boldsymbol{g}_{\mathcal{Y},1}\|^2_{\mathcal{Y},\star}}{\lambda_{\mathcal{Y}}}\,. \qquad (15.11)\end{aligned}$$

Plugging this guarantee back into the regret of each algorithm, we have that their regret is bounded and independent of T. From (15.11), we also have that $\|\boldsymbol{x}_t - \boldsymbol{x}_{t-1}\|^2_{\mathcal{X}}$ and $\|\boldsymbol{y}_t - \boldsymbol{y}_{t-1}\|^2_{\mathcal{Y}}$ converge to 0. Hence, the algorithms are getting more and more stable over time, even if they use fixed regularizers.

Algorithm 15.7 Solving Saddle-Point Problems with Optimistic OMD

Require: $\lambda_{\mathcal{X}} > 0, \lambda_{\mathcal{Y}} > 0, \boldsymbol{x}_1 \in \mathcal{X}, \boldsymbol{y}_1 \in \mathcal{Y}$
1: $\boldsymbol{g}_{\mathcal{X},0} = \mathbf{0}, \boldsymbol{g}_{\mathcal{Y},0} = \mathbf{0}$
2: **for** $t = 1, \dots, T$ **do**
3: Set $\boldsymbol{g}_{\mathcal{X},t} = \nabla_{\boldsymbol{x}} f(\boldsymbol{x}_t, \boldsymbol{y}_t)$
4: Set $\boldsymbol{g}_{\mathcal{Y},t} = -\nabla_{\boldsymbol{y}} f(\boldsymbol{x}_t, \boldsymbol{y}_t)$
5: $\boldsymbol{x}_{t+1} = \operatorname{argmin}_{\boldsymbol{x}\in\mathcal{X}} \langle 2\boldsymbol{g}_{\mathcal{X},t} - \boldsymbol{g}_{\mathcal{X},t-1}, \boldsymbol{x}\rangle + \lambda_{\mathcal{X}} B_{\psi_{\mathcal{X}}}(\boldsymbol{x}; \boldsymbol{x}_t)$
6: $\boldsymbol{y}_{t+1} = \operatorname{argmin}_{\boldsymbol{y}\in\mathcal{Y}} \langle 2\boldsymbol{g}_{\mathcal{Y},t} - \boldsymbol{g}_{\mathcal{Y},t-1}, \boldsymbol{y}\rangle + \lambda_{\mathcal{Y}} B_{\psi_{\mathcal{Y}}}(\boldsymbol{y}; \boldsymbol{y}_t)$
7: **end for**
8: **return** $\bar{\boldsymbol{x}}_T = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{x}_t, \bar{\boldsymbol{y}}_T = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{y}_t$

Remark 15.27 *Note that the fact that* $\lim_{t\to\infty} \|\boldsymbol{x}_t - \boldsymbol{x}_{t-1}\|_{\mathcal{X}} = 0$ *is not enough to infer that* $\boldsymbol{x}_t$ *is converging. In fact, it is enough to consider the case of* $x_t = \ln t$ *on an unbounded domain.*

Version with Optimistic OMD. The exact same reasoning holds for Optimistic OMD, because the key terms of its regret bound are exactly the same as those of Optimistic FTRL. To better show this fact, we instantiate the Optimistic OMD with stepsizes equal to $\frac{1}{\lambda_{\mathcal{X}}}$ and $\frac{1}{\lambda_{\mathcal{Y}}}$ for $\mathcal{X}$-player and $\mathcal{Y}$-player respectively. Following the same reasoning as above and the regret bound of Optimistic OMD, we obtain the following theorem.

Theorem 15.28 *Let* $\mathcal{X}$ *and* $\mathcal{Y}$ *be non-empty closed convex sets. With the notation in Algorithm 15.7, let* $f : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ *be convex in the first argument and concave in the second, satisfying assumptions* (15.7)-(15.10)*. For a fixed* $\alpha > 0$*, let* $\lambda_{\mathcal{X}} \geq 2\sqrt{2}(L_{\mathcal{X}\mathcal{X}} + L_{\mathcal{X}\mathcal{Y}}\alpha)$ *and* $\lambda_{\mathcal{Y}} \geq 2\sqrt{2}(L_{\mathcal{Y}\mathcal{Y}} + L_{\mathcal{X}\mathcal{Y}}/\alpha)$*. Let* $\psi_{\mathcal{X}} : \mathcal{X} \to \mathbb{R}$ *be* 1*-strongly convex with respect to* $\|\cdot\|_{\mathcal{X}}$ *and* $\psi_{\mathcal{Y}} : \mathcal{Y} \to \mathbb{R}$ *be* 1*-strongly convex with respect to* $\|\cdot\|_{\mathcal{Y}}$*. Also, assume that* $\psi_{\mathcal{X}}$ *and* $\psi_{\mathcal{Y}}$ *are differentiable in the interior of their domains. Assume that* $\arg\max_{\boldsymbol{y}\in\mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y})$ *and* $\arg\min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)$ *are non-empty. Then, we have*

$$\max_{\boldsymbol{y}\in\mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y}) - \min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T) \leq \frac{B_{\psi_{\mathcal{X}}}(\boldsymbol{x}'_T; \boldsymbol{x}_1) + B_{\psi_{\mathcal{Y}}}(\boldsymbol{y}'_T; \boldsymbol{y}_1) + \frac{\|\boldsymbol{g}_{\mathcal{X},1}\|^2_{\mathcal{X},\star}}{\lambda_{\mathcal{X}}} + \frac{\|\boldsymbol{g}_{\mathcal{Y},1}\|^2_{\mathcal{Y},\star}}{\lambda_{\mathcal{Y}}}}{T},$$

for any $\boldsymbol{x}'_T \in \arg\min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}, \bar{\boldsymbol{y}}_T)$ *and* $\boldsymbol{y}'_T \in \arg\max_{\boldsymbol{y}\in\mathcal{Y}} f(\bar{\boldsymbol{x}}_T, \boldsymbol{y})$*.*

Example 15.29 *Consider the bilinear saddle-point problem*

$$\min_{\boldsymbol{x}\in\mathcal{X}} \max_{\boldsymbol{y}\in\mathcal{Y}} \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{y} \,.$$

In this case, we have that $\nabla_{\boldsymbol{x}} f(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{A}\boldsymbol{y}$*,* $\nabla_{\boldsymbol{y}} f(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{A}^\top \boldsymbol{x}$*,* $L_{\mathcal{X}\mathcal{X}} = 0$*,* $L_{\mathcal{Y}\mathcal{Y}} = 0$*, and* $L_{\mathcal{X}\mathcal{Y}} = \|\boldsymbol{A}\|_{op}$ *where* $\|\cdot\|_{op}$ *is the operator norm of the matrix* $\boldsymbol{A}$*. The specific shape of the operator norm depends on the norms we use on* $\mathcal{X}$ *and* $\mathcal{Y}$*. For example, if we choose the Euclidean norm on both* $\mathcal{X}$ *and* $\mathcal{Y}$*, the operator norm of* $\boldsymbol{A}$ *is the largest singular value of* $\boldsymbol{A}$*. On the other hand, if* $\mathcal{X} = \Delta^{n-1}$ *and* $\mathcal{Y} = \Delta^{m-1}$*, as in the two-person zero-sum games, and we use the* L_1 *norm on both, then the operator norm of a matrix* $\boldsymbol{A}$ *is the maximum absolute value of the entries of* $\boldsymbol{A}$*.*

Algorithm 15.8 Prescient Online Mirror Descent

Require: Non-empty closed convex $\mathcal{V} \subset \mathcal{X} \subseteq \mathbb{R}^d$, $\psi : \mathcal{X} \to \mathbb{R}$ strictly convex and differentiable on $\operatorname{int} \mathcal{X}$, $\boldsymbol{x}_0 \in \operatorname{int} \mathcal{X}$, $\eta_1, \dots, \eta_T > 0$

1: **for** $t = 1$ **to** T **do**
2: Receive $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$, where $\mathcal{V} \subseteq \operatorname{dom} \ell_t$
3: $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \ell_t(\boldsymbol{x}) + \frac{1}{\eta_t} B_\psi(\boldsymbol{x}; \boldsymbol{x}_{t-1})$
4: Pay the loss $\ell_t(\boldsymbol{x}_t)$
5: **end for**

15.5.1 Prescient Online Mirror Descent and Be-The-Regularized-Leader

The results in the previous section are interesting from a game-theoretic point of view, because they show that two players can converge to an equilibrium without any "communication". However, if we only care about converging to the saddle point without worrying about communication, we can easily do better. For example, we can use the fact that it is fine if one of the two players "cheats" by looking at the loss at the beginning of each round, making its regret non-positive, while the other uses optimistic updates. However, as discussed in Remark 15.16, this will change the oracle, because now one of the algorithms needs to access the functions rather than only their gradients.

For example, we saw the use of best response. However, best response only guarantees non-positive regret, while for the optimistic proof above we need some specific negative terms. It turns out we can achieve them with another algorithm: **Prescient Online Mirror Descent**, that predicts in each round with $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \ell_t(\boldsymbol{x}) + \frac{1}{\eta_t} B_\psi(\boldsymbol{x}; \boldsymbol{x}_{t-1})$, see Algorithm 15.8.

Theorem 15.30 *Let $\psi : \mathcal{X} \to \mathbb{R}$ differentiable in $\operatorname{int} \mathcal{X}$, closed, and strictly convex. Let $\mathcal{V} \subseteq \mathcal{X}$ be a non-empty closed convex set. Assume $\boldsymbol{x}_t \in \operatorname{int} \mathcal{X}$, ℓ_t subdifferentiable on $\mathcal{V}$, and $\eta_{t+1} \leq \eta_t$, for $t = 1, \dots, T$. Then, $\forall \boldsymbol{u} \in \mathcal{V}$, Algorithm 15.8 satisfies*

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \leq \max_{0 \leq t \leq T-1} \frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_t)}{\eta_T} - \sum_{t=1}^{T} \frac{1}{\eta_t} B_\psi(\boldsymbol{x}_t; \boldsymbol{x}_{t-1}) \ .$$

Moreover, if η_t is constant, i.e., $\eta_t = \eta \ \forall t = 1, \dots, T$, we have

$$\sum_{t=1}^{T} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) \leq \frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_0)}{\eta} - \frac{1}{\eta} \sum_{t=1}^{T} B_\psi(\boldsymbol{x}_t; \boldsymbol{x}_{t-1}) \ .$$

Proof From the first-order optimality condition of Theorem 2.8 on the update, we have that there exists $\boldsymbol{g}_t \in \partial \ell_t(\boldsymbol{x}_t)$ such that

$$\langle \eta_t \boldsymbol{g}_t + \nabla \psi(\boldsymbol{x}_t) - \nabla \psi(\boldsymbol{x}_{t-1}), \boldsymbol{u} - \boldsymbol{x}_t \rangle \geq 0, \quad \forall \boldsymbol{u} \in \mathcal{V} \ .$$

Hence, we have

$$\begin{aligned} \eta_t(\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) &\leq \langle \eta_t \boldsymbol{g}_t, \boldsymbol{x}_t - \boldsymbol{u} \rangle \leq \langle \nabla \psi(\boldsymbol{x}_{t-1}) - \nabla \psi(\boldsymbol{x}_t), \boldsymbol{x}_t - \boldsymbol{u} \rangle \\ &= B_\psi(\boldsymbol{u}; \boldsymbol{x}_{t-1}) - B_\psi(\boldsymbol{u}; \boldsymbol{x}_t) - B_\psi(\boldsymbol{x}_t; \boldsymbol{x}_{t-1}), \end{aligned}$$

Algorithm 15.9 Be-the-Regularized-Leader Algorithm

Require: Non-empty closed convex $\mathcal{V} \subset \mathcal{X} \subseteq \mathbb{R}^d$, $\psi_1, \dots, \psi_T$ regularizers
1: **for** $t = 1$ **to** T **do**
2: Receive $\ell_t : \mathbb{R}^d \to (-\infty, +\infty]$, where $\mathcal{V} \subseteq \operatorname{dom} \ell_t$
3: $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t} \ell_i(\boldsymbol{x})$
4: Pay the loss $\ell_t(\boldsymbol{x}_t)$
5: **end for**

where in the last equality we used Lemma 6.7. Dividing by η_t and summing over $t = 1, \dots, T$, we have

$$\begin{aligned}
\sum_{t=1}^{T} (\ell_t(\boldsymbol{x}_t) - \ell_t(\boldsymbol{u})) &\le \sum_{t=1}^{T} \left(\frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_{t-1})}{\eta_t} - \frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_t)}{\eta_t} \right) - \sum_{t=1}^{T} \frac{B_\psi(\boldsymbol{x}_t; \boldsymbol{x}_{t-1})}{\eta_t} \\
&= \frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_0)}{\eta_1} - \frac{B_\psi(\boldsymbol{u}; \boldsymbol{x}_T)}{\eta_T} + \sum_{t=1}^{T-1} \left(\frac{1}{\eta_{t+1}} - \frac{1}{\eta_t} \right) B_\psi(\boldsymbol{u}; \boldsymbol{x}_t) - \sum_{t=1}^{T} \frac{B_\psi(\boldsymbol{x}_t; \boldsymbol{x}_{t-1})}{\eta_t} \\
&\le \frac{D^2}{\eta_1} + D^2 \sum_{t=1}^{T-1} \left(\frac{1}{\eta_{t+1}} - \frac{1}{\eta_t} \right) - \sum_{t=1}^{T} \frac{B_\psi(\boldsymbol{x}_t; \boldsymbol{x}_{t-1})}{\eta_t} \\
&= \frac{D^2}{\eta_1} + D^2 \left(\frac{1}{\eta_T} - \frac{1}{\eta_1} \right) - \sum_{t=1}^{T} \frac{B_\psi(\boldsymbol{x}_t; \boldsymbol{x}_{t-1})}{\eta_t} \\
&= \frac{D^2}{\eta_T} - \sum_{t=1}^{T} \frac{B_\psi(\boldsymbol{x}_t; \boldsymbol{x}_{t-1})}{\eta_t},
\end{aligned}$$

where we denoted by $D^2 = \max_{0 \le t \le T-1} B_\psi(\boldsymbol{u}; \boldsymbol{x}_t)$.

The second statement is left as an exercise. □

The regret of Prescient Online Mirror Descent contains the negative terms we needed from the optimistic algorithms.

Analogously, we can obtain a version of FTRL that uses the knowledge of the current loss: **Be-The-Regularized-Leader (BTRL)**, that predicts in each time step with $\boldsymbol{x}_t \in \operatorname{argmin}_{\boldsymbol{x} \in \mathcal{V}} \ \psi_t(\boldsymbol{x}) + \sum_{i=1}^{t} \ell_i(\boldsymbol{x})$, see Algorithm 15.9.

If $\psi_t \equiv 0$, then Be-The-Regularized-Leader becomes the **Be-The-Leader** algorithm. BTRL can be thought of as Optimistic FTRL where $\tilde{\ell}_t = \ell_t$. Hence, from the regret of Optimistic FTRL in Theorem 7.47, we immediately have the following theorem.

Theorem 15.31 *Let $\mathcal{V} \subset \mathbb{R}^d$ be convex, closed, and non-empty. Assume for $t = 1, \dots, T$ that $\psi_t + \sum_{i=1}^{t} \ell_i$ is proper, closed, and λ_t-strongly convex with respect to $\|\cdot\|$. Then, for*

all $\boldsymbol{u} \in \mathcal{V}$, Algorithm 15.9 satisfies

$$\sum_{t=1}^{T} \ell_t(\boldsymbol{x}_t) - \sum_{t=1}^{T} \ell_t(\boldsymbol{u}) \\ \leq \psi_T(\boldsymbol{u}) - \psi_1(\boldsymbol{x}_1) - \sum_{t=1}^{T} \frac{\lambda_t}{2} \|\boldsymbol{x}_t - \boldsymbol{x}_{t+1}\|^2 + \sum_{t=1}^{T-1} (\psi_t(\boldsymbol{x}_{t+1}) - \psi_{t+1}(\boldsymbol{x}_{t+1})) \,.$$

Remark 15.32 *In the Be-The-Leader algorithm, if all the $\psi_t \equiv 0$, then the theorem states that the regret is non-positive.*

Notably, the non-negative gradient terms are missing in the bound of BTRL, but we still have the negative ones associated with the change in $\boldsymbol{x}_t$.

Using, for example, BTRL for the $\mathcal{X}$-player and Optimistic FTRL for the $\mathcal{Y}$-player, we have

$$\sum_{t=1}^{T} f(\boldsymbol{x}_t, \boldsymbol{y}) - \sum_{t=1}^{T} f(\boldsymbol{x}, \boldsymbol{y}_t) \leq \psi_{\mathcal{X}}(\boldsymbol{x}) + \psi_{\mathcal{Y}}(\boldsymbol{y}) + \frac{\|\boldsymbol{g}_{\mathcal{Y},1}\|_{\mathcal{Y},\star}^2}{\lambda_{\mathcal{Y}}} \\ + \sum_{t=2}^{T} \left(\left(\frac{2L_{\mathcal{X}\mathcal{Y}}^2}{\lambda_{\mathcal{Y}}} - \frac{\lambda_{\mathcal{X}}}{4} \right) \|\boldsymbol{x}_t - \boldsymbol{x}_{t-1}\|_{\mathcal{X}}^2 + \left(\frac{2L_{\mathcal{Y}\mathcal{Y}}^2}{\lambda_{\mathcal{Y}}} - \frac{\lambda_{\mathcal{Y}}}{4} \right) \|\boldsymbol{y}_t - \boldsymbol{y}_{t-1}\|_{\mathcal{Y}}^2 \right) \,.$$

15.6 History Bits

Theorem 15.3 is Rockafellar (1970, Lemma 36.2). The proof of Theorem 15.12 is from Liu and Orabona (2022) and, in turn, is based on the one in Abernethy and Wang (2017): Liu and Orabona (2022) stressed the dependence of the regret on a competitor that can be useful for refined bounds. Different variants of this theorem are known in the game theory community as "Folk Theorems", because such a result was widely known among game theorists in the 1950s, even though no one had published it.

The celebrated minimax theorem for zero-sum two-person games was first discovered by John von Neumann in the 1920s (Neumann, 1928; Neumann and Morgenstern, 1944). The version stated here is a simplification of the generalization due to Sion (1958). The proof here is from Abernethy and Wang (2017). A similar proof is in Cesa-Bianchi and Lugosi (2006) based on a discretization of the space that in turn is based on the one in Freund and Schapire (1996, 1999b).

Algorithms 15.2 and 15.3 are a generalization of the algorithm for boosting in Freund and Schapire (1996, 1999b). Algorithm 15.2 was also used in Abernethy and Wang (2017) to recover variants of the Frank–Wolfe algorithm (Frank and Wolfe, 1956).

It is not clear who invented alternation: it was a known trick used in implementations of CFR for the computer poker competition from 2010 or so[1]. Note that in CFR, the method of choice is Regret Matching (Hart and Mas-Colell, 2000). However, Kroer (2020) empirically shows that alternation improves even OGD for solving bilinear games. Tammelin et al. (2015) explicitly include this trick in their implementation of an improved version of

[1] Christian Kroer, 2021, personal communication.

CFR called CFR+, claiming that it would still guarantee convergence. However, Farina et al. (2019a) pointed out that averaging of the iterates in alternation might not produce a solution to the min-max problem, providing a counterexample. Theorem 15.17 is from Burch et al. (2019). Recently, Nan et al. (2025) showed that alternating gradient descent-ascent, without the use of optimism, gives a $\mathcal{O}(1/T)$ convergence for the duality gap of two-player zero-sum games when the saddle-point is in the interior of the set, confirming the empirical observation of Kroer (2020).

There is also a complementary view on alternation: Zhang et al. (2021) link alternating updates to Gauss-Seidel methods in numerical linear algebra, in contrast to the simultaneous updates of the Jacobi method. Also, they provide a good review of the optimization literature on the advantages of alternation, but this paper and the papers they cite do not seem to be aware of the use of alternation in CFR.

The reduction from boosting to LEA is from Freund and Schapire (1996). It seems[2] that the question if a weak learner can be boosted into a strong learner was originally posed by Kearns and Valiant (1988) (see also Kearns (1988)). It was answered in the positive by Schapire (1990). The AdaBoost algorithm is from Freund and Schapire (1995, 1997). The idea of using algorithms to guarantee a regret bound that depends on the Kullback–Leibler (KL) divergence, as in (15.6), is from Luo and Schapire (2014).

Daskalakis et al. (2011) proposed the first no-regret algorithm that achieved a rate of $\mathcal{O}(\frac{\ln T}{T})$ for the duality gap when used by the two players of a zero-sum game without any communication between the players. However, the algorithm was rather complex, and they posed the problem of obtaining the same or faster rate with a simpler algorithm. Rakhlin and Sridharan (2013a) solved this problem, showing that two Optimistic OMD algorithms can solve the problem in a simpler way. Theorems 15.26 and 15.28 derive directly from Rakhlin and Sridharan (2013a, Corollary 5). For some reason, Rakhlin and Sridharan (2013a, Corollary 5) was missed in recent years, so the $\mathcal{O}(1/T)$ convergence for smooth saddle-point problems using optimistic gradient descent/ascent has been rediscovered several times (e.g., Hsieh et al., 2019; Mokhtari et al., 2020). However, the optimistic gradient descent/ascent for saddle-point problems is much older: it was proposed for the first time by Popov (1980), as a modification of the Arrow-Hurwicz method (Arrow et al., 1958). Roughly 30 years later, the optimistic algorithms were rediscovered, first as pure online learning algorithms (Chiang et al., 2012; Rakhlin and Sridharan, 2013b) and then used to solve saddle-point problems (Rakhlin and Sridharan, 2013a).

The observation after Theorem 15.26 that it is possible to achieve constant regret for each player is from Luo (2022). For general games, it is possible to achieve logarithmic regret for each player by using modified versions of optimistic algorithms (see, e.g., Soleymani et al., 2025, and references therein).

The use of Prescient Online Mirror Descent in saddle-point optimization is from Wang et al. (2021), but when renaming $\boldsymbol{x}_t$ to $\boldsymbol{x}_{t+1}$ it is also equivalent to implicit online mirror descent (Kivinen and Warmuth, 1997; Kulis and Bartlett, 2010; Campolongo and Orabona, 2020). Theorem 15.30 is from the guarantee of implicit online mirror descent in Campolongo and Orabona (2020).

There is also a tight connection between optimistic updates using the previous gradi-

[2] I could not verify this claim because I could not find the paper anywhere.

ents and classic approaches to solving saddle-point optimization. In fact, Gidel et al. (2019) showed that using two optimistic gradient descent algorithms to solve a saddle-point problem can be seen as a variant of the Extra-gradient updates (Korpelevich, 1976), while Mokhtari et al. (2020) show that they can be interpreted as an approximated proximal point algorithm.

For generic saddle-point problems, Popov (1980) proved the asymptotic convergence of the iterates when using two optimistic OGD algorithms. This old result was unknown to the majority of the community until recently, and it implies some later weaker results (e.g., Daskalakis et al., 2018). The extension to the Mirror Descent case was done by Semenov (2017) to solve the more general problem of variational inequalities, but only for distance generating functions that are differentiable on the entire feasible set.[3] This means that this proof does not cover the optimistic EG. In turn, this little-known result was also recently rediscovered (e.g., Lee et al., 2021, Theorem 4). Hsieh et al. (2021, Theorem 7) proved the asymptotic convergence of the last iterate for Optimistic FTRL with linear losses and an adaptive regularization weight, assuming either strict convexity/concavity or that the regularizer is differentiable on the entire domain. Note that the proof in Hsieh et al. (2021, Theorem 7) can be easily adapted to prove the same result for optimistic OMD with a fixed and small enough learning rate. Lei et al. (2021) prove the asymptotic convergence of optimistic EG for saddle-point problems, under the assumption that some of the optimality conditions are satisfied in a strict way.

For the specific case of bilinear games, stronger results can be proven on the last-iterate convergence. Liang and Stokes (2019) proved that if the matrix $\boldsymbol{A}$ is square, full-rank, and the problem is unconstrained, then the iterates of two Optimistic OGD will converge exponentially fast to the saddle point in the origin. Daskalakis and Panageas (2019) proved the asymptotic convergence of optimistic OMD/FTRL EG with fixed stepsize for bilinear games over probability simplices, assuming a unique saddle point. Wei et al. (2021) proved an exponential rate for the same algorithm under the same assumptions. Hsieh et al. (2021, Theorem 8) removed the assumption of a unique saddle point, proving asymptotic convergence for Optimistic FTRL with entropic regularization and an adaptive regularization weight. Once again, this proof can be easily modified to prove the same result for optimistic OMD with a fixed and small enough learning rate.

15.7 Exercises

Problem 15.1 *Prove Lemma 15.10.*

Problem 15.2 *Let $f(\boldsymbol{x}, \boldsymbol{y})$ be a convex-concave function, Lipschitz with respect to both variables. Use two FTRL algorithms to find the saddle-point, using as regularizers $\psi_t(\boldsymbol{x}) \propto \sqrt{t}\psi(\boldsymbol{x})$, where ψ is differentiable and 1-strongly convex with respect to a norm $\|\cdot\|$. Then, using the inequality in Problem 7.2, show that the trajectory of the iterates of the above strategy is bounded on any convex-concave saddle-point problem with at least one saddle-point, even in unbounded domains and any number of dimensions.*

[3] Equation 17 in Semenov (2017) needs the fact that the distance generating function is continuously differentiable everywhere for a limit operation. This assumption is not stated, making the proof essentially wrong, for example, for the negative entropy function. As far as I know, no one had realized this issue in that paper before.

16

From Online Learning to X

In this chapter, we show that online learning is much more general than one might think. We already saw that online learning algorithms can be used for stochastic convex optimization (Chapter 3) and for saddle-point optimization (Chapter 15). Yet, these were still convex optimization problems. Instead, here we will show that online learning results can be used to prove generalization guarantees, design algorithms for the optimization of non-convex non-smooth functions, and construct confidence intervals.

16.1 From Online Learning to Rademacher Complexity

Here, we will quantify the statistical difficulty of a hypothesis class through its *Rademacher complexity*, which measures how well the functions in the class can correlate with pure noise. Rademacher complexity is the key quantity behind many uniform convergence and generalization results. In this section, *we show a simple reduction that upper bounds the Rademacher complexity of a class using the regret guarantee of an online learning algorithm.*

Rademacher Complexity. We consider again Vapnik's general setting of learning that we saw in Section 3.2, this time for generic function classes. Hence, we seek to minimize

$$\operatorname*{argmin}_{f \in \mathcal{H}} \mathbb{E}_{\boldsymbol{\xi} \sim \rho}[f(\boldsymbol{\xi})] \tag{16.1}$$

where $\mathcal{H}$ is a class of real-valued functions on $\mathcal{Z}$ and ρ is a distribution over $\mathcal{Z}$. One common way to solve this problem is through an Empirical Risk Minimization process. That is, we gather a sample of size T, $\{\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_T\}$, drawn i.i.d. from ρ, then we solve the empirical version of (16.1):

$$f_{\mathcal{S}} := \operatorname*{argmin}_{f \in \mathcal{H}} \frac{1}{T} \sum_{t=1}^{T} f(\boldsymbol{\xi}_t) \, .$$

It is now natural to ask if $f_{\mathcal{S}}$ will also minimize the objective function in (16.1).

One way to study this problem is through the concept of *Rademacher complexity* of a function class.

Definition 16.1 *Let $\mathcal{S} = \{\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_T\}$ be an arbitrary set of T vectors in some domain $\mathcal{Z}$. Let $\mathcal{H}$ be a class of real-valued functions on $\mathcal{Z}$. Let $\epsilon_1, \dots, \epsilon_T$ be i.i.d. Rademacher random variables, that is, $\mathbb{P}\{\epsilon_t = 1\} = \mathbb{P}\{\epsilon_t = -1\} = 1/2$. The* ***empirical Rademacher***

complexity *of $\mathcal{H}$ on $\mathcal{S}$ is defined as*

$$\widehat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H}) := \frac{1}{T}\,\mathbb{E}_{\epsilon_1,\dots,\epsilon_T}\left[\sup_{f\in\mathcal{H}}\sum_{t=1}^{T}\epsilon_t f(\boldsymbol{\xi}_t)\right]. \tag{16.2}$$

Intuitively, if $\widehat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H})$ is small, then no function in the class can fit random signs too well on the sample. Hence, we can expect the minimizers of the empirical risk to be close to the minimizers of the true risk.

This idea can be made formal and yields generalization guarantees; for example, bounds of the form

$$\sup_{f\in\mathcal{H}}\left|\mathbb{E}[f(\boldsymbol{\xi})] - \frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{\xi}_t)\right| \lesssim \widehat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H}) + \sqrt{\frac{\ln(1/\delta)}{T}}$$

with probability at least $1-\delta$ under mild boundedness assumptions. Given that the above bound applies uniformly over $\mathcal{H}$, it also applies to the empirical risk minimizer $f_{\mathcal{S}}$.

Remark 16.2 *One can easily obtain a generalization guarantee that depends on $f_{\mathcal{S}}$, rather than being uniform over a class. It is enough to consider a countable nested family $(\mathcal{H}_k)_{k=1}^{\infty}$ whose union covers the entire space of possible outcomes of the empirical risk minimization process. Then, apply the uniform bound to each $\mathcal{H}_k$ with a union bound, and choose the class with the smallest Rademacher complexity containing the empirical risk minimizer.*

We will not prove the generalization bounds here; instead, we focus on the following question: *how can we upper bound $\widehat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H})$ for interesting classes?*

We consider function classes given by composing Lipschitz functions with linear predictors. Given a prediction function class $\{\phi_{\boldsymbol{x}} : \boldsymbol{x}\in\mathcal{V}\}$ and a loss $\ell(\hat{y},y)$, we obtain the induced function class

$$\mathcal{H}_\ell := \{\boldsymbol{\xi}=(\boldsymbol{z},y)\mapsto \ell(\phi_{\boldsymbol{x}}(\boldsymbol{z}),y) \;:\; \boldsymbol{x}\in\mathcal{V}\},$$

where $(\boldsymbol{z},y)\in\mathcal{Z}\times\mathcal{Y}$. Hence, we would like to control $\widehat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H}_\ell)$. A common situation is that $\ell(\cdot,y)$ is Lipschitz, say L-Lipschitz for all y. In this case, it is intuitive that a Lipschitz loss cannot increase the ability to correlate with noise by more than a factor L. This is formalized by contraction inequalities, as in the next lemma.

Lemma 16.3 (Contraction Lemma) *Let $h:\mathbb{R}\to\mathbb{R}$ be L-Lipschitz. Consider the function class $h\circ\mathcal{H}=\{h\circ f : f\in\mathcal{H}\}$. Then, for any sample $\mathcal{S}$, we have $\widehat{\mathfrak{R}}_{\mathcal{S}}(h\circ\mathcal{H}) \le L\,\widehat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H})$.*

Hence, in the following, we will focus on controlling the complexity of linear functions.

Rademacher Complexity of Linear Classes through Regret Guarantees. We now state the core reduction. Let $\mathcal{V}\subset\mathbb{R}^d$ be a non-empty, bounded, closed set, and consider the linear class

$$\mathcal{H}_{\mathrm{lin}} := \{\boldsymbol{z}\mapsto\langle\boldsymbol{x},\boldsymbol{z}\rangle \;:\; \boldsymbol{x}\in\mathcal{V}\}.$$

Fix a sample $\mathcal{S}=\{\boldsymbol{z}_1,\dots,\boldsymbol{z}_T\}$ and define the empirical Rademacher complexity of $\mathcal{H}_{\mathrm{lin}}$ on $\mathcal{S}$ as in (16.2). The following theorem shows how to use the regret of an online learning algorithm to upper bound such empirical Rademacher complexity.

Theorem 16.4 *Let $\mathcal{Z} \subset \mathbb{R}^d$ such that for any $\boldsymbol{z} \in \mathcal{Z}$ we also have $-\boldsymbol{z} \in \mathcal{Z}$. Fix any sequence $\boldsymbol{z}_1, \dots, \boldsymbol{z}_T \in \mathcal{Z}$ and any non-empty, bounded, closed set $\mathcal{V} \subset \mathbb{R}^d$. Assume that there exists an online algorithm that, on any sequence of linear losses $\ell_t(\boldsymbol{x}) = \langle \boldsymbol{g}_t, \boldsymbol{x} \rangle$, where $\boldsymbol{g}_t \in \mathcal{Z}$, produces $\boldsymbol{x}_t \in \mathcal{V}$ and satisfies the regret guarantee*

$$\sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{x}_t \rangle - \min_{\boldsymbol{u} \in \mathcal{V}} \sum_{t=1}^{T} \langle \boldsymbol{g}_t, \boldsymbol{u} \rangle \le R(T) \, . \tag{16.3}$$

Then, the empirical Rademacher complexity of the linear class satisfies

$$\widehat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H}_{\text{lin}}) = \frac{1}{T} \mathbb{E}_{\epsilon_1, \dots, \epsilon_T} \left[\sup_{\boldsymbol{x} \in \mathcal{V}} \sum_{t=1}^{T} \epsilon_t \langle \boldsymbol{x}, \boldsymbol{z}_t \rangle \right] \le \frac{R(T)}{T} \, . \tag{16.4}$$

Proof Fix $\boldsymbol{z}_1, \dots, \boldsymbol{z}_T$ and draw $\epsilon_1, \dots, \epsilon_T$. Consider the online linear game with gradients $\boldsymbol{g}_t := -\epsilon_t \boldsymbol{z}_t$. Let $\boldsymbol{x}_1, \dots, \boldsymbol{x}_T$ be the iterates produced by the online algorithm when fed with $\boldsymbol{g}_1, \dots, \boldsymbol{g}_T$. By the regret guarantee (16.3) we have, for every realization of the signs,

$$\sum_{t=1}^{T} \langle -\epsilon_t \boldsymbol{z}_t, \boldsymbol{x}_t \rangle - \min_{\boldsymbol{u} \in \mathcal{V}} \sum_{t=1}^{T} \langle -\epsilon_t \boldsymbol{z}_t, \boldsymbol{u} \rangle \le R(T) \, . \tag{16.5}$$

Rearranging, we get

$$\max_{\boldsymbol{u} \in \mathcal{V}} \sum_{t=1}^{T} \epsilon_t \langle \boldsymbol{u}, \boldsymbol{z}_t \rangle \le R(T) + \sum_{t=1}^{T} \epsilon_t \langle \boldsymbol{x}_t, \boldsymbol{z}_t \rangle \, .$$

Taking expectation with respect to $\epsilon_1, \dots, \epsilon_T$ and dividing by T yields

$$\widehat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H}_{\text{lin}}) \le \frac{R(T)}{T} + \frac{1}{T} \mathbb{E}_{\epsilon_1, \dots, \epsilon_T} \left[\sum_{t=1}^{T} \epsilon_t \langle \boldsymbol{x}_t, \boldsymbol{z}_t \rangle \right] \, . \tag{16.6}$$

It remains to show that the second term is 0. The key observation is that $\boldsymbol{x}_t$ is measurable with respect to the past signs $\epsilon_1, \dots, \epsilon_{t-1}$ and possibly its internal randomization, while ϵ_t is independent of the past and has mean zero. Hence, conditioning on the past,

$$\mathbb{E}_{\epsilon_1, \dots, \epsilon_T} [\epsilon_t \langle \boldsymbol{x}_t, \boldsymbol{z}_t \rangle | \epsilon_1, \dots, \epsilon_{t-1}, \text{internal randomness}] = \langle \boldsymbol{x}_t, \boldsymbol{z}_t \rangle \, \mathbb{E}[\epsilon_t] = 0,$$

and by taking expectation again we obtain $\mathbb{E}[\epsilon_t \langle \boldsymbol{x}_t, \boldsymbol{z}_t \rangle] = 0$. Summing over t makes the last term in (16.6) equal to 0, so we obtain (16.4). □

Remark 16.5 *The proof of Theorem 16.4 has the same flavor as the probabilistic method argument used in lower bounds in Chapter 5: we introduce Rademacher signs, feed them to an online algorithm, and then exploit the fact that the algorithm cannot correlate with the* fresh *randomness at time t. The entire complexity term is paid by the regret upper bound $R(T)$.*

We can instantiate the above theorem using the Online Mirror Descent (OMD) algorithms with p-norms, see Section 6.7.

Corollary 16.6 *Let $q \in [2, \infty)$ and $p \in (1, 2]$ such that $1/p + 1/q = 1$. For $U_p > 0$, let*

$\mathcal{V}_p := \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_p \leq U_p\}$. Let $\mathcal{S} := \{\boldsymbol{z}_1, \dots, \boldsymbol{z}_T\}$ where $\boldsymbol{z}_t \in \mathbb{R}^d$ and $\|\boldsymbol{z}_t\|_q \leq G_q$ for $t = 1, \dots, T$. Let $\mathcal{H}_p := \{\boldsymbol{z} \mapsto \langle \boldsymbol{x}, \boldsymbol{z} \rangle \ : \ \boldsymbol{x} \in \mathcal{V}_p\}$. Then,

$$\frac{1}{T}\mathbb{E}_{\epsilon_1,\dots,\epsilon_T}\left[\sup_{\boldsymbol{x}\in\mathcal{V}_p} \sum_{t=1}^{T} \epsilon_t \langle \boldsymbol{x}, \boldsymbol{z}_t \rangle\right] \leq \frac{U_p G_q}{\sqrt{(p-1)T}} \, .$$

Proof We instantiate Theorem 16.4 with OMD with p-norms, learning rate $\eta = \frac{U_p\sqrt{p-1}}{G_q\sqrt{T}}$ and $\boldsymbol{x}_1 = \boldsymbol{0}$. □

We can also consider the case of classes of functions with finite cardinality.

Corollary 16.7 (Massart's Lemma) *Let $\mathcal{S} := \{\boldsymbol{z}_1, \dots, \boldsymbol{z}_T\}$ where $\boldsymbol{z}_t \in \mathbb{R}^d$ and $\|\boldsymbol{z}_t\|_\infty \leq G_\infty$ for $t = 1, \dots, T$. Let $\mathcal{H} := \left\{\boldsymbol{z} \mapsto \langle \boldsymbol{x}, \boldsymbol{z} \rangle \ : \ \boldsymbol{x} \in \Delta^{d-1}\right\}$. Then,*

$$\frac{1}{T}\mathbb{E}_{\epsilon_1,\dots,\epsilon_T}\left[\max_{i=1,\dots,d} \sum_{t=1}^{T} \epsilon_t z_{t,i}\right] \leq \frac{G_\infty\sqrt{2\ln d}}{\sqrt{T}} \, .$$

Proof We instantiate Theorem 16.4 with OMD with entropic regularizer, learning rate $\eta = \frac{\sqrt{2\ln d}}{G_\infty\sqrt{T}}$ and $\boldsymbol{x}_1 = [1/d, \dots, 1/d]^\top$. □

16.2 From Online Learning to PAC-Bayes

In this section, we show that one can directly obtain generalization bounds for any machine learning algorithm from upper bounds on linear regret.

We will consider the same setting we used in the online-to-batch reduction (Chapter 3), where one aims at minimizing the risk of a function $f : \mathcal{Z} \to [0, 1]$, defined as

$$\mathrm{Risk}(f) = \mathbb{E}_{\boldsymbol{\xi}\sim\rho}[f(\boldsymbol{\xi})] \, .$$

For example, in a regression setting, $f(\boldsymbol{\xi})$ is the composition of a loss with a prediction function, evaluated on the data $\boldsymbol{\xi} = (\boldsymbol{z}, y)$.

Given a training set $\mathcal{S} = \{\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_T\}$ drawn i.i.d. from ρ, here we are interested in the **generalization gap**, defined as the difference between the risk and the training error of a function f:

$$\mathrm{Gen}(f, \mathcal{S}) := \mathrm{Risk}(f) - \frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{\xi}_t) \, .$$

We can also consider the case where the predictor is randomized, in the sense that we draw f according to a probability distribution Q in $\Delta(\mathcal{H})$, that is the set of probability distributions over a set of functions $\mathcal{H} := \{f \mid f : \mathcal{Z} \to [0, 1]\}$. In this case, we study

$$\overline{\mathrm{Gen}}(Q, \mathcal{S}) := \mathbb{E}_{f\sim Q}[\mathrm{Gen}(f, \mathcal{S})] \, .$$

In particular, we are interested in upper bounding $\overline{\mathrm{Gen}}(Q_\mathcal{S}, \mathcal{S})$ in high probability, where $Q_\mathcal{S}$ is selected from $\Delta(\mathcal{H})$ after looking at $\mathcal{S}$.

We now describe the reduction from this problem to an online learning one. Consider

an online algorithm that at each round t produces a distribution $P_t \in \Delta(\mathcal{H})$ after observing the training samples $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_{t-1}$. Define $g_t(f) = f(\boldsymbol{\xi}_t) - \mathbb{E}_{\boldsymbol{\xi}\sim\rho}[f(\boldsymbol{\xi})]$ and $\ell_t(P) = \mathbb{E}_{f\sim P}[g_t(f)]$. So, for any $Q \in \Delta(\mathcal{H})$, we have that

$$\begin{aligned}\overline{\mathrm{Gen}}(Q,\mathcal{S}) &= \mathbb{E}_{f\sim Q}[\mathbb{E}_{\boldsymbol{\xi}\sim\rho}[f(\boldsymbol{\xi})]] - \frac{1}{T}\sum_{t=1}^{T}\mathbb{E}_{f\sim Q}[f(\boldsymbol{\xi}_t)] = \frac{1}{T}\sum_{t=1}^{T} -\ell_t(Q) \\ &= \frac{1}{T}\sum_{t=1}^{T}(\ell_t(P_t) - \ell_t(Q)) - \frac{1}{T}\sum_{t=1}^{T}\ell_t(P_t) = \frac{\mathrm{Regret}_T(Q)}{T} - \frac{1}{T}\sum_{t=1}^{T}\ell_t(P_t)\,.\end{aligned}$$

We use $\mathscr{F}_t$ to denote the σ-algebra generated by the data points $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_t$ and by all the random variables generated by the online learning algorithm up to the end of round t. Now, given that P_t is $\mathscr{F}_{t-1}$-measurable and $\boldsymbol{\xi}_t$ is independent of $\mathscr{F}_{t-1}$, we have

$$\mathbb{E}[\ell_t(P_t)|\mathscr{F}_{t-1}] = \mathbb{E}[\mathbb{E}_{f\sim P_t}[f(\boldsymbol{\xi}_t) - \mathbb{E}_{\boldsymbol{\xi}\sim\rho}[f(\boldsymbol{\xi})]]|\mathscr{F}_{t-1}] = 0\,.$$

Since $f(\boldsymbol{\xi}) \in [0,1]$, we have

$$g_t(f) = f(\boldsymbol{\xi}_t) - \mathbb{E}_{\boldsymbol{\xi}\sim\rho}[f(\boldsymbol{\xi})] \in [-1,1],$$

and therefore

$$\ell_t(P_t) = \mathbb{E}_{f\sim P_t}[g_t(f)] \in [-1,1]\,.$$

Hence, $\ell_1(P_1), \ell_1(P_1)+\ell_2(P_2), \dots, \sum_{t=1}^{T}\ell_t(P_t)$ is a martingale with bounded increments in $[-1,1]$, so we can apply the Hoeffding–Azuma inequality in Theorem 3.9. This concentration does not depend on Q, so it holds simultaneously for all $Q \in \Delta(\mathcal{H})$.

We are done! We can now put everything together to have the following theorem.

Theorem 16.8 *Let $\mathcal{H}$ be a measurable space of measurable functions $f : \mathcal{Z} \to [0,1]$, $\delta \in (0,1)$, and $\pi \in \Delta(\mathcal{H})$. Let $\mathcal{S} = \{\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_T\}$ be drawn i.i.d. from a distribution ρ over $\mathcal{Z}$. Consider any online learning algorithm that outputs a distribution over $\mathcal{H}$ and is fed with the linear losses $\ell_t(P) = \mathbb{E}_{f\sim P}[f(\boldsymbol{\xi}_t) - \mathbb{E}_{\boldsymbol{\xi}\sim\rho}[f(\boldsymbol{\xi})]]$. Then, with probability at least $1-\delta$, for all $Q \in \Delta(\mathcal{H})$ such that $Q \ll \pi$, even selected with the knowledge of $\mathcal{S}$, we have*

$$\overline{\mathrm{Gen}}(Q,\mathcal{S}) = \frac{\mathrm{Regret}_T(Q)}{T} - \frac{1}{T}\sum_{t=1}^{T}\ell_t(P_t) \le \frac{\mathrm{Regret}_T(Q)}{T} + \sqrt{\frac{2\ln\frac{1}{\delta}}{T}}\,.$$

We can now instantiate this theorem with the regret of any online learning algorithm. For example, we could use the Exponentiated Gradient (EG) algorithm, where we think of each $f \in \mathcal{H}$ as an expert. There are only two caveats: first, we would have to extend it to the case where the number of experts is infinite, possibly continuous; second, we would have to select an appropriate learning rate.

The first issue is easy to deal with: roughly speaking, it is enough to substitute any sum over the experts with integrals. Using the fact that $g_t(f) \in [-1,1]$, the regret guarantee of the continuous version of EG is

$$\frac{\mathrm{KL}(Q;\pi)}{\eta} + \frac{\eta T}{2},$$

where π is the prior distribution over the infinite experts and Q is the competitor distribution. We might not know how to run this algorithm because of the integrals, but we do not need to! We only need to know that such an algorithm and its regret bound exist.

The second problem, instead, is a difficult one: the optimal learning rate depends on Q. However, we want generalization guarantees that hold uniformly for any Q. This is *exactly* the problem we saw many times in online learning when the optimal choice of the learning rate depends on the unknown comparator. In standard EG, we upper bounded the Kullback–Leibler (KL) divergence term for the case of a uniform prior with $\ln d$, but here we cannot do it because the number of experts is infinite. One could construct a grid of learning rates, instantiate the bound for each of them, and use a union bound, but this approach could introduce additional poly-logarithmic terms in the final bound. However, we already know how to solve this problem in online learning, by simply using *parameter-free* algorithms.

So, we can consider a continuous version of the parameter-free algorithm for learning with expert advice in Section 13.7, and we can show this regret bound.

Theorem 16.9 *Let $\mathcal{H}$ be a measurable space of measurable functions $f : \mathcal{Z} \to [0, 1]$, and $\pi \in \Delta(\mathcal{H})$. At each round t, the adversary reveals a measurable loss function $g_t : \mathcal{H} \to [-1, 1]$. There exists an online learning algorithm that depends on π, outputs a distribution $P_t \in \Delta(\mathcal{H})$, and incurs the loss $\ell_t(P) = \mathbb{E}_{f\sim P}[g_t(f)]$. For any competitor distribution $Q \in \Delta(\mathcal{H})$ such that $Q \ll \pi$, its regret satisfies*

$$\mathrm{Regret}_T(Q) := \sum_{t=1}^{T} \left(\mathbb{E}_{f\sim P_t}[g_t(f)] - \mathbb{E}_{f\sim Q}[g_t(f)]\right) \le \sqrt{20T\left(\mathrm{KL}(Q;\pi) + 1\right)}\,.$$

The proof is a straightforward adaptation of the finite-expert case, but for completeness, we leave it as an exercise.

Combining the two previous theorems, we obtain the following so-called **PAC-Bayes** bound.

Corollary 16.10 (PAC-Bayes Bound) *Let $\mathcal{H}$ be a measurable space of measurable functions $f : \mathcal{Z} \to [0, 1]$, $\delta \in (0, 1)$, and $\pi \in \Delta(\mathcal{H})$. Let $\mathcal{S} = \{\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_T\}$ be drawn i.i.d. from a distribution ρ over $\mathcal{Z}$. Then, with probability at least $1 - \delta$, for all $Q \in \Delta(\mathcal{H})$ such that $Q \ll \pi$, even selected with the knowledge of $\mathcal{S}$, we have*

$$\overline{\mathrm{Gen}}(Q, \mathcal{S}) \le \frac{\sqrt{20(\mathrm{KL}(Q;\pi) + 1)}}{\sqrt{T}} + \sqrt{\frac{2\ln\frac{1}{\delta}}{T}}\,.$$

The role of π is to allow the bound to hold simultaneously for all posterior distributions Q. Indeed, the theorem guarantees that, with probability at least $1 - \delta$ over the draw of the sample $\mathcal{S}$, the inequality holds for every $Q \in \Delta(\mathcal{H})$ such that $Q \ll \pi$, even if Q is selected after observing the data. This uniformity is possible because π acts as a continuous analogue of the union bound. The divergence term $\mathrm{KL}(Q;\pi)$ plays the role of the logarithmic penalty that appears in the discrete union bound. Hence, choosing π corresponds to specifying how the bound is distributed over the different functions in $\mathcal{H}$.

Remark 16.11 *As in the previous section, the* existence *of an online learning algorithm with a regret upper bound is enough to prove our bound. We never need to actually run the*

online algorithm. Moreover, in this case, we could not run the reduction even if we wanted to, since the linear losses ℓ_t depend on the unknown distribution ρ.

16.3 From Online Learning to Non-Convex Non-Smooth Optimization

This section describes a simple (and surprisingly sharp) reduction from OLO to the task of finding approximate stationary points of *non-convex, non-smooth* objectives. The key idea is to feed stochastic gradients to an online algorithm, which then decides the *updates* rather than the *iterates*.

We consider the objective function $F : \mathbb{R}^d \to \mathbb{R}$, which is differentiable but not necessarily smooth, i.e., the gradient map might not be Lipschitz.

Example 16.12 *Consider $F : \mathbb{R} \to \mathbb{R}$ defined as $F(x) = \frac{2}{3}|x|^{\frac{3}{2}}$. Then, $F'(x) = \operatorname{sign}(x)\sqrt{|x|}$, where $\operatorname{sign}(0) := 0$. Hence, F is differentiable everywhere, but its derivative is not Lipschitz because $F''(x)$ is unbounded when $x \to 0$.*

To connect function values to gradients without convexity or smoothness, we isolate the only calculus identity needed by the reduction. So, we will define *well-behaved* functions as those that satisfy the specific equality we need.

Definition 16.13 *A differentiable function $F : \mathbb{R}^d \to \mathbb{R}$ is **well-behaved** if for every $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d$,*

$$F(\boldsymbol{y}) - F(\boldsymbol{x}) = \int_0^1 \langle \nabla F(\boldsymbol{x} + t(\boldsymbol{y} - \boldsymbol{x})), \boldsymbol{y} - \boldsymbol{x} \rangle \, \mathrm{d}t \; . \tag{16.7}$$

Remark 16.14 *Identity* (16.7) *is immediate for smooth functions (by the fundamental theorem of calculus applied to $t \mapsto F(\boldsymbol{x} + t(\boldsymbol{y} - \boldsymbol{x}))$), but it can hold well beyond smooth objectives. In particular, Cutkosky et al. (2023) show that if F is locally Lipschitz, then an arbitrarily small randomized smoothing yields a differentiable, well-behaved surrogate with a natural stochastic gradient oracle. Here, we focus on the differentiable, well-behaved case to keep the presentation simple.*

We now define our notion of optimality. Since F is non-convex and not necessarily smooth, small gradients are not obviously implied by function decrease; we instead certify a local averaged stationarity notion that measures how close we are to a local minimum by considering *local averages* of gradients.

Definition 16.15 *Let F be differentiable almost everywhere, and let $\delta > 0$. Let $\mathcal{Q}(\boldsymbol{x}, \delta)$ be the set of random variables $\boldsymbol{Q}$ with finite support in the ball of radius δ around $\boldsymbol{x}$, such that $\nabla F(\boldsymbol{y})$ exists for all $\boldsymbol{y}$ in the support of $\boldsymbol{Q}$. Define*

$$\|\nabla F(\boldsymbol{x})\|_\delta := \inf_{\boldsymbol{Q} \in \mathcal{Q}(\boldsymbol{x}, \delta)} \left\{ \left\| \mathbb{E}\left[\nabla F(\boldsymbol{Q})\right] \right\|_2 : \mathbb{E}[\boldsymbol{Q}] = \boldsymbol{x} \right\} \; .$$

*We say that $\boldsymbol{x}$ is a **barycentric** (δ, ϵ)**-stationary point** if $\|\nabla F(\boldsymbol{x})\|_\delta \le \epsilon$.*

Algorithm 16.1 Online-to-Non-Convex Conversion

Require: Well-behaved $F : \mathbb{R}^d \to \mathbb{R}$, OLO algorithm $\mathscr{A}$ over $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \le D\}$, cycle length K, horizon T as a multiple of K, initial point $\boldsymbol{x}_0$

$j = 0$
for $t = 1$ **to** T **do**
  if $t \equiv 1 (\mathrm{mod}\ K)$ **then**
    Reset $\mathscr{A}$
    $j = j + 1, \quad \bar{\boldsymbol{x}}_j = \boldsymbol{0}$
  end if
  Receive update $\boldsymbol{m}_t \in \mathcal{V}$ from $\mathscr{A}$
  $\boldsymbol{x}_t = \boldsymbol{x}_{t-1} - \boldsymbol{m}_t$
  Sample $s_t \sim \mathrm{Unif}[0, 1]$ and set $\boldsymbol{x}'_t = \boldsymbol{x}_{t-1} - s_t \boldsymbol{m}_t$
  Query a stochastic gradient $\boldsymbol{g}_t$ at $\boldsymbol{x}'_t$
  Define loss $\ell_t(\boldsymbol{m}) := -\langle \boldsymbol{g}_t, \boldsymbol{m} \rangle$ and pass it to $\mathscr{A}$
  $\bar{\boldsymbol{x}}_j = \bar{\boldsymbol{x}}_j + \boldsymbol{x}'_t / K$
end for
Return $\bar{\boldsymbol{x}}_J$ where J is uniform on $\{1, \dots, T/K\}$

Remark 16.16 *This notion is closely related to Goldstein-type δ-subdifferential stationarity, but we use gradients, and it is* barycentric*: the same convex weights used to average gradients must also average the sampled points back to $\boldsymbol{x}$.*

The Reduction Algorithm. We now present the reduction from non-convex optimization of well-behaved functions to online linear optimization.

Fix a cycle length $K \in \mathbb{N}$ and a radius $D > 0$. We will run an OLO algorithm $\mathscr{A}$ on the Euclidean ball $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \le D\}$, where the OLO algorithm outputs *updates* $\boldsymbol{m}_t \in \mathcal{V}$. The iterates are defined by $\boldsymbol{x}_t = \boldsymbol{x}_{t-1} - \boldsymbol{m}_t$. Then, at each step, we draw a random point $\boldsymbol{x}'_t$ on the segment from $\boldsymbol{x}_{t-1}$ to $\boldsymbol{x}_t$ and query a stochastic gradient there. The final output $\bar{\boldsymbol{x}}_J$ is the average of K points $\boldsymbol{x}'_t$ inside a random cycle J. The complete procedure is given in Algorithm 16.1.

Theorem 16.17 *Let F be well-behaved (Definition 16.13). Let $\mathscr{F}_t$ be the sigma-field generated by all randomness up to round t before querying $\boldsymbol{g}_t$, so that $\boldsymbol{x}'_t$ is $\mathscr{F}_t$-measurable. With the notation in Algorithm 16.1, we assume*

$$\mathbb{E}[\boldsymbol{g}_t \mid \mathscr{F}_t] = \nabla F(\boldsymbol{x}'_t), \qquad \mathbb{E}[\|\boldsymbol{g}_t\|_2^2 \mid \mathscr{F}_t] \le G^2, \tag{16.8}$$

for some $G > 0$. Let $T \in \mathbb{N}$ be a multiple of the cycle length K. Run Algorithm 16.1 for T steps with cycle length K, and instantiate $\mathscr{A}$ as projected Online Gradient Descent (OGD) over $\mathcal{V} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\|_2 \le D\}$ with learning rate $\eta = D/(G\sqrt{K})$ and initial point equal to $\boldsymbol{0}$ on each cycle (i.e., after each reset). Then,

$$\mathbb{E}\left[\frac{1}{T/K}\sum_{j=1}^{T/K}\left\|\frac{1}{K}\sum_{t=(j-1)K+1}^{jK}\nabla F(\boldsymbol{x}'_t)\right\|_2\right] \le \frac{F(\boldsymbol{x}_0) - \inf_{\boldsymbol{x}} F(\boldsymbol{x})}{D\,T} + \frac{2G}{\sqrt{K}}\,.$$

In particular, for $\delta > 0$, choose $D = \delta/K$ and then set

$$K \asymp \left(\frac{G\,T\,\delta}{F(\boldsymbol{x}_0) - \inf_{\boldsymbol{x}} F(\boldsymbol{x})} \right)^{2/3},$$

rounded to an integer. Then,

$$\mathbb{E}\big[\|\nabla F(\bar{\boldsymbol{x}}_J)\|_\delta\big] = \mathcal{O}\left(G^{\frac{2}{3}} \left(\frac{F(\boldsymbol{x}_0) - \inf_{\boldsymbol{x}} F(\boldsymbol{x})}{T\delta} \right)^{1/3} \right).$$

Proof We first prove the key identity linking change in function value to an *expected* gradient. By Definition 16.13, we have

$$\begin{aligned} F(\boldsymbol{x}_t) - F(\boldsymbol{x}_{t-1}) &= \int_0^1 \langle \nabla F(\boldsymbol{x}_{t-1} + a(\boldsymbol{x}_t - \boldsymbol{x}_{t-1})), \boldsymbol{x}_t - \boldsymbol{x}_{t-1} \rangle \, \mathrm{d}a \\ &= \int_0^1 \langle \nabla F(\boldsymbol{x}_{t-1} - a\boldsymbol{m}_t), -\boldsymbol{m}_t \rangle \, \mathrm{d}a \\ &= -\left\langle \mathbb{E}_{s_t}\big[\nabla F(\boldsymbol{x}_{t-1} - s_t \boldsymbol{m}_t)\big], \boldsymbol{m}_t \right\rangle = -\left\langle \mathbb{E}_{s_t}\big[\nabla F(\boldsymbol{x}'_t)\big], \boldsymbol{m}_t \right\rangle, \end{aligned}$$

where the second-to-last equality is due to the fact that s_t is uniform on $[0, 1]$.

Taking expectation over all randomness up to time t (including s_t) and for any $\boldsymbol{u}_j \in \mathcal{V}$, we have

$$\begin{aligned} \mathbb{E}\left[F(\boldsymbol{x}_t) - F(\boldsymbol{x}_{t-1})\right] &= -\mathbb{E}\left[\langle \nabla F(\boldsymbol{x}'_t), \boldsymbol{m}_t \rangle\right] \\ &= \mathbb{E}\left[\langle -\boldsymbol{g}_t, \boldsymbol{m}_t - \boldsymbol{u}_j \rangle\right] + \mathbb{E}\left[\langle \boldsymbol{g}_t - \nabla F(\boldsymbol{x}'_t), \boldsymbol{m}_t \rangle\right] - \mathbb{E}\left[\langle \boldsymbol{g}_t, \boldsymbol{u}_j \rangle\right] \\ &= \mathbb{E}\left[\langle -\boldsymbol{g}_t, \boldsymbol{m}_t - \boldsymbol{u}_j \rangle\right] - \mathbb{E}\left[\langle \boldsymbol{g}_t, \boldsymbol{u}_j \rangle\right] \\ &= \mathbb{E}\left[\langle -\boldsymbol{g}_t, \boldsymbol{m}_t - \boldsymbol{u}_j \rangle\right] + \mathbb{E}\left[\langle -\nabla F(\boldsymbol{x}'_t), \boldsymbol{u}_j \rangle\right] \\ &\quad + \mathbb{E}\left[\langle \nabla F(\boldsymbol{x}'_t) - \boldsymbol{g}_t, \boldsymbol{u}_j \rangle\right], \end{aligned} \tag{16.9}$$

where in the second-to-last equality we used that $\boldsymbol{m}_t$ is determined before querying $\boldsymbol{g}_t$, together with the unbiasedness condition $\mathbb{E}[\boldsymbol{g}_t \mid \mathscr{F}_t] = \nabla F(\boldsymbol{x}'_t)$.

We now analyze one cycle, and then sum over cycles. Fix a cycle j and let its time indices be $t = (j-1)K+1, \ldots, jK$. Summing (16.9) over $t = (j-1)K+1, \ldots, jK$, we obtain

$$\begin{aligned} \mathbb{E}\left[F(\boldsymbol{x}_{jK}) - F(\boldsymbol{x}_{(j-1)K})\right] &= \mathbb{E}\left[\sum_{t=(j-1)K+1}^{jK} \langle -\boldsymbol{g}_t, \boldsymbol{m}_t - \boldsymbol{u}_j \rangle\right] \\ &\quad - \mathbb{E}\left[\sum_{t=(j-1)K+1}^{jK} \langle \nabla F(\boldsymbol{x}'_t), \boldsymbol{u}_j \rangle\right] \\ &\quad + \mathbb{E}\left[\sum_{t=(j-1)K+1}^{jK} \langle \nabla F(\boldsymbol{x}'_t) - \boldsymbol{g}_t, \boldsymbol{u}_j \rangle\right]. \end{aligned} \tag{16.10}$$

We now focus on each term on the r.h.s. of this equality.

For the first term on the r.h.s. of (16.10), for any $\boldsymbol{u}_j \in \mathcal{V}$, from the regret guarantee of

projected OGD with learning rate $\eta = D/(G\sqrt{K})$ and initial point equal to $\mathbf{0}$, we have

$$\sum_{t=(j-1)K+1}^{jK} \langle -\boldsymbol{g}_t, \boldsymbol{m}_t - \boldsymbol{u}_j \rangle \leq \frac{\|\boldsymbol{u}_j\|_2^2}{2\eta} + \frac{\eta}{2} \sum_{t=(j-1)K+1}^{jK} \|\boldsymbol{g}_t\|_2^2 \leq \frac{D^2}{2\eta} + \frac{\eta}{2} \sum_{t=(j-1)K+1}^{jK} \|\boldsymbol{g}_t\|_2^2 \,.$$

Since this inequality holds for every $\boldsymbol{u}_j$, we can use a hindsight choice depending on the gradients in cycle j. Moreover, taking expectation and using the definition of η, we have

$$\mathbb{E}\left[\sum_{t=(j-1)K+1}^{jK} \langle -\boldsymbol{g}_t, \boldsymbol{m}_t - \boldsymbol{u}_j \rangle\right] \leq \frac{D^2}{2\eta} + \frac{\eta}{2} K G^2 = DG\sqrt{K} \,. \tag{16.11}$$

Now, we choose $\boldsymbol{u}_j$ as

$$\boldsymbol{u}_j := D \frac{\sum_{t=(j-1)K+1}^{jK} \nabla F(\boldsymbol{x}'_t)}{\left\| \sum_{t=(j-1)K+1}^{jK} \nabla F(\boldsymbol{x}'_t) \right\|_2} \quad \text{(and } \boldsymbol{u}_j = \mathbf{0} \text{ if the numerator is } \mathbf{0}) \,.$$

Our choice of $\boldsymbol{u}_j$ guarantees $\|\boldsymbol{u}_j\|_2 \leq D$, and the second term of the r.h.s. of (16.10) becomes

$$\mathbb{E}\left[\sum_{t=(j-1)K+1}^{jK} \langle \nabla F(\boldsymbol{x}'_t), \boldsymbol{u}_j \rangle\right] = D\mathbb{E}\left[\left\| \sum_{t=(j-1)K+1}^{jK} \nabla F(\boldsymbol{x}'_t) \right\|_2\right] \,.$$

Now, we upper bound the last term in (16.10). First, observe that

$$\mathbb{E}[\|\boldsymbol{g}_t - \nabla F(\boldsymbol{x}'_t)\|_2^2 | \mathscr{F}_t] = \mathbb{E}[\|\boldsymbol{g}_t\|_2^2 | \mathscr{F}_t] - \|\nabla F(\boldsymbol{x}'_t)\|_2^2 \leq G^2,$$

and

$$\mathbb{E}[\langle \boldsymbol{g}_t - \nabla F(\boldsymbol{x}'_t), \boldsymbol{g}_n - \nabla F(\boldsymbol{x}'_n) \rangle | \mathscr{F}_t] = 0, \quad \forall n < t \,.$$

Hence, we have

$$\begin{aligned}
\mathbb{E}\left[\sum_{t=(j-1)K+1}^{jK} \langle \nabla F(\boldsymbol{x}'_t) - \boldsymbol{g}_t, \boldsymbol{u}_j \rangle\right] &\leq D\mathbb{E}\left[\left\| \sum_{t=(j-1)K+1}^{jK} (\boldsymbol{g}_t - \nabla F(\boldsymbol{x}'_t)) \right\|_2\right] \\
&\leq D\sqrt{\mathbb{E}\left[\left\| \sum_{t=(j-1)K+1}^{jK} (\boldsymbol{g}_t - \nabla F(\boldsymbol{x}'_t)) \right\|_2^2\right]} \\
&= D\sqrt{\mathbb{E}\left[\sum_{t=(j-1)K+1}^{jK} \|\boldsymbol{g}_t - \nabla F(\boldsymbol{x}'_t)\|_2^2\right]} \\
&\leq GD\sqrt{K},
\end{aligned}$$

where we used the Cauchy–Schwarz inequality and Jensen's inequality (Theorem 2.9).

Putting everything together, we have

$$D\,\mathbb{E}\left[\left\| \sum_{t=(j-1)K+1}^{jK} \nabla F(\boldsymbol{x}'_t) \right\|_2\right] \leq 2DG\sqrt{K} + \mathbb{E}\left[F(\boldsymbol{x}_{(j-1)K}) - F(\boldsymbol{x}_{jK})\right] \,.$$

Summing over $j = 1, \dots, T/K$ telescopes the function values:

$$D \sum_{j=1}^{T/K} \mathbb{E}\left[\left\| \sum_{t=(j-1)K+1}^{jK} \nabla F(\boldsymbol{x}'_t) \right\|_2\right] \le 2\frac{T}{K} DG\sqrt{K} + F(\boldsymbol{x}_0) - \mathbb{E}[F(\boldsymbol{x}_T)]$$
$$\le 2\frac{T}{K} DG\sqrt{K} + F(\boldsymbol{x}_0) - \inf_{\boldsymbol{x}} F(\boldsymbol{x}) \; .$$

Dividing by DT gives the stated bound.

Now observe that within each cycle we have $\|\boldsymbol{x}_t - \boldsymbol{x}_{t-1}\|_2 = \|\boldsymbol{m}_t\|_2 \le D$. Hence all points $\boldsymbol{x}'_t$ lie in the convex hull of $\{\boldsymbol{x}_{(j-1)K}, \boldsymbol{x}_{(j-1)K+1}, \dots, \boldsymbol{x}_{jK}\}$ whose diameter is at most KD (triangle inequality over at most K steps). Since $\bar{\boldsymbol{x}}_j = \frac{1}{K}\sum_{t=(j-1)K+1}^{jK} \boldsymbol{x}'_t$, every $\boldsymbol{x}'_t$ lies at a distance at most the diameter of the set from $\bar{\boldsymbol{x}}_j$, i.e., KD. Therefore, if $D = \delta/K$, then for all t in cycle j we have $\boldsymbol{x}'_t \in \mathcal{B}(\bar{\boldsymbol{x}}_j, \delta)$. By Definition 16.15,

$$\|\nabla F(\bar{\boldsymbol{x}}_j)\|_\delta \le \left\| \frac{1}{K} \sum_{t=(j-1)K+1}^{jK} \nabla F(\boldsymbol{x}'_t) \right\|_2 \; .$$

Sampling J uniformly over cycles and taking expectation yields the stated barycentric (δ, ϵ)-stationarity bound. Choosing K to balance the two terms gives the stated rate. □

It is interesting to compute the update $\boldsymbol{m}_t$ in Algorithm 16.1 when $\mathscr{A}$ is projected OGD:

$$\boldsymbol{m}_{t+1} = \Pi_{\mathcal{V}}(\boldsymbol{m}_t + \eta \boldsymbol{g}_t) \; .$$

This is reminiscent of the SGD update with momentum and clipping, which are common heuristics for optimizing non-convex objectives in deep learning. Yet, here the update comes naturally from the theory.

We can also prove that this bound is optimal. To see this, we consider smooth functions and the following lemma.

Lemma 16.18 *Suppose that $F : \mathbb{R}^d \to \mathbb{R}$ is H-smooth and $\boldsymbol{x}$ satisfies $\|\nabla F(\boldsymbol{x})\|_\delta \le \epsilon$. Then, $\|\nabla F(\boldsymbol{x})\|_2 \le \epsilon + H\delta$.*

Proof Using the assumption that $\|\nabla F(\boldsymbol{x})\|_\delta \le \epsilon$, for any $p > 0$ there exists $Q \in \mathcal{Q}(\boldsymbol{x}, \delta)$ such that $\|\mathbb{E}_{\boldsymbol{y}\sim Q}[\nabla F(\boldsymbol{y})]\|_2 \le \epsilon + p$. By definition of $\mathcal{Q}(\boldsymbol{x}, \delta)$, we have $\text{supp}(Q) \subseteq \mathcal{B}(\boldsymbol{x}, \delta)$. So, by H-smoothness, for every $\boldsymbol{y} \in \text{supp}(Q)$, we have $\|\nabla F(\boldsymbol{y}) - \nabla F(\boldsymbol{x})\|_2 \le H\|\boldsymbol{y} - \boldsymbol{x}\|_2 \le H\delta$. Hence,

$$\begin{aligned}
\epsilon + p &\ge \|\mathbb{E}_{\boldsymbol{y}\sim Q}[\nabla F(\boldsymbol{y})]\|_2 = \big\|\nabla F(\boldsymbol{x}) + \mathbb{E}_{\boldsymbol{y}\sim Q}\big[\nabla F(\boldsymbol{y}) - \nabla F(\boldsymbol{x})\big]\big\|_2 \\
&\ge \|\nabla F(\boldsymbol{x})\|_2 - \big\|\mathbb{E}_{\boldsymbol{y}\sim Q}\big[\nabla F(\boldsymbol{y}) - \nabla F(\boldsymbol{x})\big]\big\|_2 \\
&\ge \|\nabla F(\boldsymbol{x})\|_2 - \mathbb{E}_{\boldsymbol{y}\sim Q}\left[\|\nabla F(\boldsymbol{y}) - \nabla F(\boldsymbol{x})\|_2\right] \ge \|\nabla F(\boldsymbol{x})\|_2 - H\delta \; .
\end{aligned}$$

Therefore, $\|\nabla F(\boldsymbol{x})\|_2 \le \epsilon + H\delta + p$. Since this holds for every $p > 0$, we conclude that $\|\nabla F(\boldsymbol{x})\|_2 \le \epsilon + H\delta$. □

Now, recall that Theorem 16.17 shows that we can find a (δ, ϵ) barycentric stationary point in $\mathcal{O}(\epsilon^{-3}\delta^{-1})$ iterations. Thus, Lemma 16.18 implies that by setting $\delta = \epsilon/H$, we can find a 2ϵ-stationary point of an H-smooth objective F in $\mathcal{O}(\epsilon^{-4})$ iterations, which matches

the optimal guarantee of standard SGD. Hence, the $\mathcal{O}(\epsilon^{-3}\delta^{-1})$ rate in Theorem 16.17 is optimal for all ϵ of the order of δH.

16.4 From Online Learning to Time-Uniform Concentration Inequalities

In this section, we will use the existence of a universal portfolio algorithm to derive time-uniform concentration inequalities.

For a sequence of random variables $Z_1, Z_2, \ldots$, assume that $\mathbb{E}[Z_t|Z_1, \ldots, Z_{t-1}] = \mu$ for all t. Also, assume that Z_t takes values in $[0, 1]$. We want to estimate μ, giving confidence intervals $[l_t, u_t]$ that are valid for any t with probability at least $1 - \delta$, that is, $\mathbb{P}\{\exists t : \mu \notin [l_t, u_t]\} \leq \delta$. This kind of confidence intervals are usually called *confidence sequences*. The classic approach is to estimate μ with $\hat{\mu}_t = \frac{1}{t}\sum_{i=1}^{t} Z_i$, then invoke a concentration inequality that holds uniformly over time to obtain the confidence intervals. However, most of the well-known concentration inequalities produce *vacuous* confidence intervals when t is small, that is, $u_t - l_t > 1$ for all t smaller than some constant.

Here, we will see how a portfolio algorithm immediately gives rise to non-vacuous time-uniform confidence intervals, even with a single sample!

The idea is the following: we will construct a continuous coin-betting game on a fair coin from the problem of estimating the unknown mean. Then, we will use the following theorem, which says that the probability of making a large amount of money at any moment in time is small.

Theorem 16.19 (Ville's inequality) *Let $Y_0, Y_1, \ldots,$ be a non-negative supermartingale and $\delta \in (0, 1]$. Then, we have*

$$\mathbb{P}\left\{\max_t\ Y_t \geq \frac{1}{\delta}\right\} \leq \mathbb{E}[Y_0]\delta\ .$$

Warm-up: From Krichevsky–Trofimov (KT) to a concentration inequality. As a warm-up example, consider the KT algorithm that bets $\beta_t \operatorname{Wealth}_{t-1}$ in round t on the outcome of the continuous coin $c_t = Z_t - \mu$. Let $\mathscr{F}_t := \sigma(Z_1, \ldots, Z_t)$. Then, $(\operatorname{Wealth}_t)_{t\geq 0}$ is a martingale, and hence also a supermartingale, with respect to the filtration $(\mathscr{F}_t)_{t\geq 0}$. As in Example 3.8, we have

$$\begin{aligned}\mathbb{E}[\operatorname{Wealth}_t \mid \mathscr{F}_{t-1}] &= \mathbb{E}[\operatorname{Wealth}_{t-1}(1 + \beta_t c_t) \mid \mathscr{F}_{t-1}] \\ &= \operatorname{Wealth}_{t-1}\mathbb{E}[1 + \beta_t(Z_t - \mu) \mid \mathscr{F}_{t-1}] = \operatorname{Wealth}_{t-1}\ .\end{aligned}$$

Moreover, the wealth is non-negative because KT guarantees a non-negative wealth on any sequence of coins.

So, starting with \$1, using Ville's inequality and Theorem 13.4, we obtain

$$\begin{aligned}\delta &\geq \mathbb{P}\left\{\max_t \operatorname{Wealth}_t \geq \frac{1}{\delta}\right\} = \mathbb{P}\left\{\max_t \ln \operatorname{Wealth}_t \geq \ln\frac{1}{\delta}\right\} \\ &\geq \mathbb{P}\left\{\max_t \frac{1}{2t}\left(\sum_{i=1}^{t} c_i\right)^2 - \frac{1}{2}\ln(e^2 t) \geq \ln\frac{1}{\delta}\right\}\ .\end{aligned}$$

Equivalently, with probability at least $1-\delta$, we have uniformly over t that

$$\left|\frac{1}{t}\sum_{i=1}^{t} Z_i - \mu\right| = \frac{1}{t}\left|\sum_{i=1}^{t} c_i\right| \leq \sqrt{\frac{2}{t}\ln\frac{\sqrt{e^2 t}}{\delta}}\ .$$

Observe that $Y_t = \sum_{i=1}^{t}(Z_i - \mu)$ is a martingale with respect to the filtration $\mathscr{F}_t := \sigma(Z_1, \ldots, Z_t)$. So, let's compare this concentration to Hoeffding–Azuma inequality in Theorem 3.9. We almost get the same thing, but here we have an additional $\ln\sqrt{t}$ term. However, this concentration is uniform over time, which justifies the additional term in the logarithm. To summarize, the regret/reward guarantee of KT implies a concentration inequality.

From Universal Portfolio to a concentration inequality. Let's now improve this reasoning using a universal portfolio algorithm to bet on the same outcomes. Similarly to Section 13.9, we will transform the random variables Z_t to a sequence of two market gains. Set $w_{t,1} = 1 + \frac{Z_t - \mu}{\mu}$ and $w_{t,2} = 1 - \frac{Z_t - \mu}{1-\mu}$. Given that $Z_t - \mu \in [-\mu, 1-\mu]$, we have that $w_{t,1}$ and $w_{t,2}$ are non-negative. As before, we have that the wealth will be a martingale. So, we have

$$\max_{\boldsymbol{u}\in\Delta}\sum_{i=1}^{t}\ln\langle \boldsymbol{w}_i, \boldsymbol{u}\rangle = \max_{\beta\in[-\frac{1}{1-\mu},\frac{1}{\mu}]}\sum_{i=1}^{t}\ln\left(1+(Z_i-\mu)\beta\right)\ .$$

Reasoning as above, we have

$$\mathbb{P}\left\{\max_t \max_{\beta\in[-\frac{1}{1-\mu},\frac{1}{\mu}]}\sum_{i=1}^{t}\ln\left(1+(Z_i-\mu)\beta\right) - \mathrm{Regret}_t \geq \ln\frac{1}{\delta}\right\} \leq \delta,$$

where Regret_t is the regret of the portfolio algorithm with 2 stocks. If we use a portfolio algorithm with logarithmic regret, given that $[-1,1] \subset [-\frac{1}{1-\mu}, \frac{1}{\mu}]$, this expression always gives a bigger wealth than the KT strategy we have just seen. In turn, a bigger wealth corresponds to a tighter concentration.

Let's be more precise by instantiating the above idea with an F-weighted portfolio algorithm from Chapter 10, with F equal to the Dirichlet(1/2,1/2) distribution.

Theorem 16.20 *Let $\delta \in (0,1)$. Assume $Z_1, Z_2, \ldots$ are a sequence of random variables such that for each i we have $0 \leq Z_i \leq 1$ and $\mathbb{E}[Z_i|Z_1,\ldots,Z_{i-1}] = \mu$ almost surely and $\mu \in (0,1)$. Let $G_t(\beta,\mu) := \sum_{i=1}^{t}\ln\left(1+\beta(Z_i-\mu)\right)$, $R_t := \ln\frac{\sqrt{\pi}\Gamma(t+1)}{\Gamma(t+\frac{1}{2})}$, and*

$$\mathcal{S}_t := \left\{m \in (0,1) : \max_{\beta\in[-\frac{1}{1-m},\frac{1}{m}]} G_t(\beta, m) - R_t \leq \ln\frac{1}{\delta}\right\}\ .$$

Then, with probability at least $1-\delta$ and uniformly over t, after observing t random variables we have $\mu \in \cap_{i=1}^{t}\mathcal{S}_i$.

Moreover, we have that $\mathcal{S}_t$ is an interval $[l_t, u_t] \subseteq [0,1]$ for all $t \geq 1$.

Proof Following the reasoning above and the fact that the regret of the F-weighted portfolio algorithm with 2 stocks and $F = Dirichlet(1/2, 1/2)$ is upper bounded by R_t using Theorem 10.1, we get that $\mu \in \mathcal{S}_t$. From the fact that this holds with probability $1-\delta$ uniformly over time, we get that at time t μ must be in the intersection of the sets $\mathcal{S}_1, \ldots, \mathcal{S}_t$.

For the second claim, first denote by $\beta^\star(m) := \operatorname{argmax}_{\beta\in[-\frac{1}{1-m},\frac{1}{m}]} G_t(\beta, m)$ and $\hat{G}_t(m) = \max_{\beta\in[-\frac{1}{1-m},\frac{1}{m}]} G_t(\beta, m)$.

Observe that the derivative of G_t with respect to its first argument is

$$G_t'(\beta, m) = \sum_{i=1}^{t} \frac{Z_i - m}{1 + \beta(Z_i - m)} .$$

So, we have that $G_t'(0, \hat{\mu}_t) = 0$. Given that $G_t(\beta, m)$ is concave in β, then $G_t(\beta, \hat{\mu}_t)$ has a maximum with respect to the first argument in $\beta = 0$ and the value of the function is 0.

For $m' > \hat{\mu}_t$, $G_t'(0, m') < 0$. Since $G_t(\beta, m')$ is concave in β, we have $\beta^\star(m') < 0$. In the same way, for $m' < \hat{\mu}_t$ we have $\beta^\star(m') > 0$.

Now, let us start with $m' > \hat{\mu}_t$ and prove that $\hat{G}_t(m)$ is nondecreasing; the other side is analogous. Consider $m_1 > m_2 > \hat{\mu}_t$. Given that $\beta^\star(m_2) < 0$, we have

$$\hat{G}_t(m_2) = G_t(\beta^\star(m_2), m_2) \le G_t(\beta^\star(m_2), m_1) \le G_t(\beta^\star(m_1), m_1) = \hat{G}_t(m_1),$$

where the first inequality holds because $G_t(\beta, m)$ is nondecreasing in m when $\beta < 0$ and the second inequality holds because the negative part of the interval $[-\frac{1}{1-m_1}, \frac{1}{m_1}]$ contains the negative part of the interval $[-\frac{1}{1-m_2}, \frac{1}{m_2}]$ and we know the maximum β is negative. Hence, $\hat{G}_t(m)$ is a quasiconvex function of m, and hence $\mathcal{S}_t$ is an interval. □

Remark 16.21 *Given that $\mathcal{S}_t$ is an interval, we can find $\mathcal{S}_t = [l_t, u_t]$ efficiently using the bisection algorithm.*

Now, we gather some more intuition on the inequality of Theorem 16.20. It may not seem obvious if the maximum log wealth is a better candidate for constructing a confidence sequence than the standard ones like Bernoulli KL-divergence-based bound (e.g., Garivier and Cappé, 2011, Theorem 10), which works for random variables supported in $[0, 1]$:

$$\mathbb{P}\left\{\max_t \; t \cdot \mathrm{KL}_{\text{Bern}}(\hat{\mu}_t; \mu) - \ln f(t) \ge \ln \frac{1}{\delta}\right\} \le \delta,$$

where $\hat{\mu}_t = \frac{1}{t}\sum_{i=1}^t Z_i$, $\mathrm{KL}_{\text{Bern}}(p; q) := p \ln \frac{p}{q} + (1-p) \ln \frac{1-p}{1-q}$, and $f(t)$ grows polynomially in t or slower. So, in the following proposition we show that the maximum log wealth is never worse than the KL divergence, which supports a viewpoint that the KL divergence is a special case of the maximum log wealth and that confidence bounds constructed with the maximum wealth are never worse than those with KL divergence, ignoring the minor difference in $\ln f(t)$.

Proposition 16.22 *Let $Z_1, \dots, Z_t \in [0, 1]$, $\hat{\mu}_t = \frac{1}{t}\sum_{i=1}^t Z_i$, and $\mu \in (0, 1)$. Then,*

$$\max_{\beta\in[-\frac{1}{1-\mu},\frac{1}{\mu}]} \sum_{i=1}^{t} \ln\left(1 + \beta(Z_i - \mu)\right) \ge t \cdot \mathrm{KL}_{\textit{Bern}}\left(\hat{\mu}_t; \mu\right),$$

where we achieve the equality if $Z_1, \dots, Z_t \in \{0, 1\}$ almost surely.

Proof Using Jensen's inequality (Theorem 2.9), we have for any $Z \in [0, 1]$ that

$$\begin{aligned}\ln(1 + \beta(Z - \mu)) &= \ln[Z(1 + \beta(1 - \mu)) + (1 - Z)(1 + \beta(0 - \mu))] \\ &\ge Z \ln(1 + \beta(1 - \mu)) + (1 - Z) \ln(1 + \beta(0 - \mu)) .\end{aligned}$$

Note that we achieve equality when $Z = 1$ or $Z = 0$. Then, we have

$$\begin{aligned}
&\max_{\beta\in[-\frac{1}{1-\mu},\frac{1}{\mu}]} \sum_{i=1}^{t} \ln(1+\beta(Z_i-\mu)) \\
&\quad\geq \max_{\beta\in[-\frac{1}{1-\mu},\frac{1}{\mu}]} \sum_{i} Z_i \ln(1+\beta(1-\mu)) + (1-Z_i)\ln(1-\beta\mu) \\
&\quad= \max_{\beta\in[-\frac{1}{1-\mu},\frac{1}{\mu}]} t[\hat{\mu}_t \ln(1+\beta(1-\mu)) + (1-\hat{\mu}_t)\ln(1-\beta\mu)] \,.
\end{aligned}$$

As the r.h.s. is concave in β, it remains to maximize the r.h.s. over β. The solution is $\beta = \frac{\hat{\mu}_t-\mu}{\mu(1-\mu)}$ with which the maximum becomes $t \cdot \mathrm{KL}_{\text{Bern}}(\hat{\mu}_t, \mu)$. □

Second, we show that the confidence intervals obtained by the numerical inversion of Theorem 16.20 are *never vacuous*.

Theorem 16.23 *Under the assumptions of Theorem 16.20, we have that* $u_1 - l_1 = 1 - \frac{\delta}{2}$.

Proof With only one sample, the upper and lower bounds have a closed form. Indeed, the argmax of $G_t(\beta, m)$ with respect to β over $[-\frac{1}{1-m}, \frac{1}{m}]$ is achieved in $\beta = \frac{1}{m}$ if $Z_1 - m > 0$ and in $\beta = -\frac{1}{1-m}$ for $Z_1 - m < 0$. This implies that

$$l_1 = \frac{Z_1}{\exp(R_1 + \ln\frac{1}{\delta})} \qquad \text{and} \qquad u_1 = 1 - \frac{1-Z_1}{\exp(R_1 + \ln\frac{1}{\delta})} \,.$$

Given that $R_1 = \ln 2$, subtracting the lower bound from the upper bound, we get the stated bound for any Z_1. □

Given that we can return the intersection of all intervals $[l_t, u_t]$, the above theorem implies the same bound for all t.

Finally, besides implying the KL bound above, the implicit concentration in Theorem 16.20 also implies an empirical Bernstein time-uniform concentration. It is worth stressing that the intersection of the numerically evaluated intervals $[l_i, u_i]$ from $i = 1, \ldots, t$ is strictly smaller than the upper bound in the following theorem, given that Theorem 16.23 tells us that the widths of numerically calculated intervals are always strictly smaller than 1.

Theorem 16.24 *Under the assumptions of Theorem 16.20, denote by* $\hat{\mu}_i = \frac{1}{i}\sum_{j=1}^{i} Z_j$, $V_i = \sum_{j=1}^{i}(Z_j - \hat{\mu}_i)^2$, $\hat{R}_i = \ln\frac{\sqrt{\pi}\Gamma(i+1)}{\delta\Gamma(i+\frac{1}{2})}$, *and*

$$\epsilon_i = \frac{(4/3)i\hat{R}_i + \sqrt{16/9 i^2\hat{R}_i^2 + 8V_i\hat{R}_i(i^2 - 2i\hat{R}_i)}}{2i^2 - 4i\hat{R}_i} \,.$$

Then, with probability at least $1-\delta$ *uniformly for all* t *such that* $t > 2\hat{R}_t$, *we have*

$$\max_{i=1,\ldots,t} \hat{\mu}_i - \epsilon_i \leq \mu \leq \min_{i=1,\ldots,t} \hat{\mu}_i + \epsilon_i \,.$$

For sufficiently large t the deviation is roughly $\frac{(4/3)\ln(\sqrt{t}/\delta)}{t} + \frac{\sqrt{2V_t\ln(\sqrt{t}/\delta)}}{t}$, similarly to the inequalities first proposed in Audibert et al. (2009); Maurer and Pontil (2009).

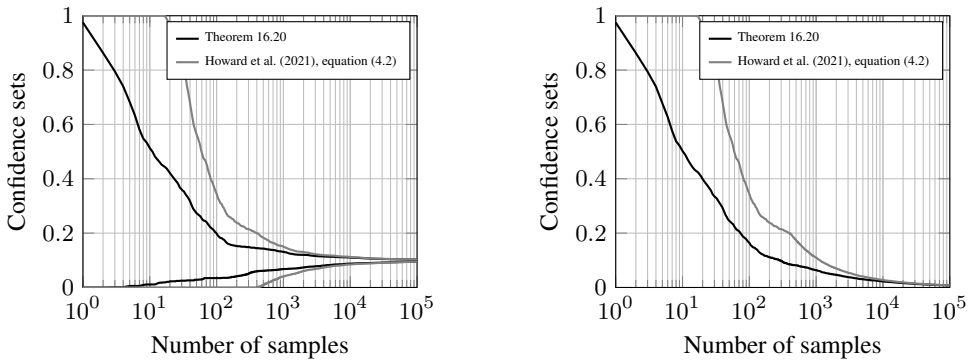


Figure 16.1 Confidence intervals for Theorem 16.20 and Howard et al. (2021) for a sequence of random variables drawn i.i.d. from Bernoulli(0.1).

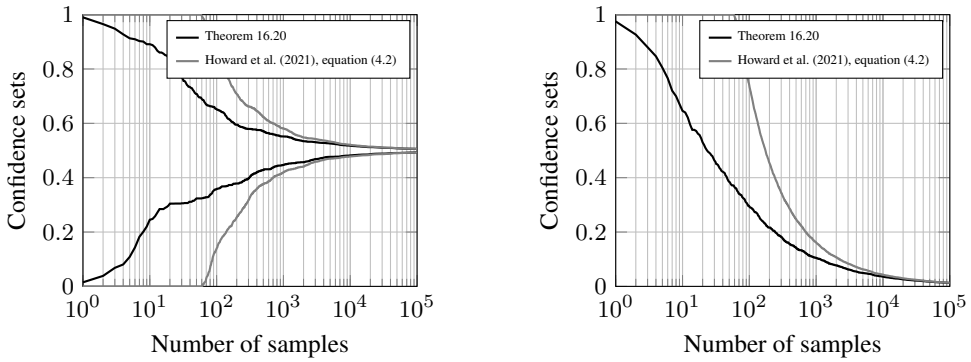


Figure 16.2 Confidence intervals for Theorem 16.20 and Howard et al. (2021) for a sequence of random variables drawn i.i.d. from Bernoulli(0.5).

Remark 16.25 *One can easily see that, in terms of the scaling with* $\ln(1/\delta)$*, the factor* $\sqrt{2(\frac{1}{t}V_t)/t}$ *is the optimal one due to the central limit theorem. For the scaling with* t*, by changing the weight distribution* F *it is also possible to obtain* $\sqrt{\frac{2(\frac{1}{t}V_t)\ln(\ln(V_t))+o(\ln\ln t)}{t}}$ *as* $t \to \infty$*, which matches the law of the iterated logarithm (thus asymptotically optimal), see Orabona and Jun (2024).*

We can see the performance of this approach compared to a state-of-the-art closed-form concentration in Figures 16.1-16.4. We repeat each experiment 10 times and use the first 5 in which all the algorithms do not fail. In fact, all the algorithms/inequalities will fail with probability δ, and averaging over their failing runs would result in confidence sequences with smaller widths. We set $\delta = 0.05$ in all the experiments.

To prove Theorem 16.24, we first need a technical lemma.

Lemma 16.26 *Let* $f(x) = ax + b(\ln(1 - |x|) + |x|)$*, where* $a \in \mathbb{R}$ *and* $b \geq 0$*. Then,*

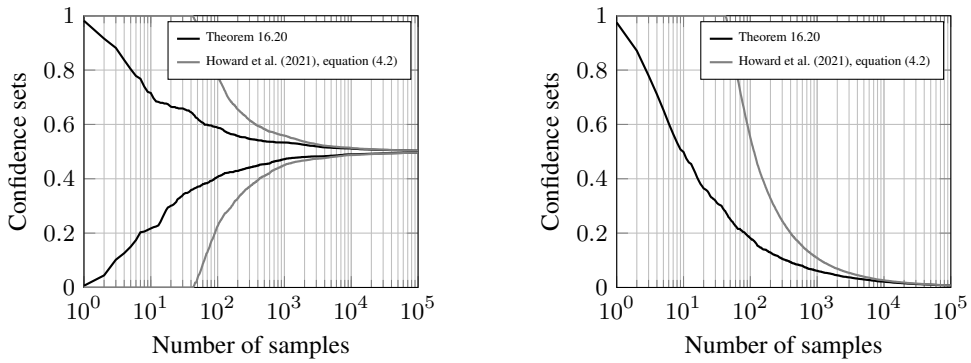


Figure 16.3 Confidence intervals for Theorem 16.20 and Howard et al. (2021) for a sequence of random variables drawn i.i.d. from Beta(1,1).

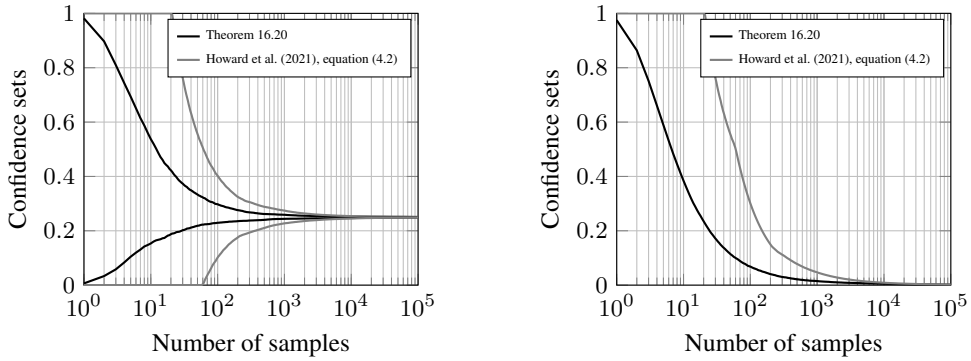


Figure 16.4 Confidence intervals for Theorem 16.20 and Howard et al. (2021) for a sequence of random variables drawn i.i.d. from Beta(10,30).

$\operatorname{argmax}_{x\in[-1,1]} f(x) = \frac{a}{|a|+b}$ *and* $\max_{x\in[-1,1]} f(x) = b\psi(\frac{a}{b}) \geq \frac{a^2}{(4/3)|a|+2b}$, *where* $\psi(x) = |x| - \ln(|x|+1)$.

Proof If $b = 0$, we have that argmax is $\operatorname{sign}(a)$. If $a = 0$, the argmax is 0. Hence, in the following, we can assume a and b are nonzero.

We can rewrite the maximization problem as

$$\operatorname*{argmax}_x f(x) = b \operatorname*{argmax}_x \frac{a}{b}x + \ln(1-|x|) + |x| \,.$$

From the optimality condition, we have that $\frac{a}{b} - \frac{\operatorname{sign}(x^\star)}{1-|x^\star|} + \operatorname{sign}(x^\star) = 0$, which implies $x^\star = \frac{a}{|a|+b}$. Substituting this expression in f, we obtain the stated expression. The inequality is obtained by the elementary inequality $\ln(1+x) \leq x \cdot \frac{6+x}{6+4x}$ for $x \geq 0$. □

We can now prove Theorem 16.24.

Proof of Theorem 16.24 For a given t, set ϵ_t equal to $\mu - \hat{\mu}_t$, so that $\epsilon_t + \hat{\mu}_t \in [0,1]$.

Consider the function $f(a) = \frac{\ln(1+a)-a}{a^2/2}$ for $a > -1$. Set $|x| \leq 1$ and $|\beta| < 1$, so

we have $\beta x \geq -|\beta| > -1$. From the sign of the first derivative, we have that $f(a)$ is increasing. Hence, we have

$$\ln(1+\beta x) = \beta x + \frac{1}{2}(\beta x)^2 f(\beta x) \geq \beta x + \frac{1}{2}(\beta x)^2 f(-|\beta|) = \beta x + x^2(|\beta| + \ln(1-|\beta|)) .$$

Define $\phi(\beta) := \ln(1-|\beta|) + |\beta|$, so that, for any $\beta \in (-1,1)$, we have

$$\begin{aligned}
&\sum_{i=1}^{t} \ln(1+\beta(Z_i - \mu)) \\
&\quad = \sum_{i=1}^{t} \ln(1+\beta(Z_i - \hat{\mu}_t - \epsilon_t)) \\
&\quad \geq \beta \sum_{i=1}^{t} (Z_i - \hat{\mu}_t - \epsilon_t) + \phi(\beta) \left(\sum_{i=1}^{t} (Z_i - \hat{\mu}_t)^2 + \epsilon_t^2 t - 2\epsilon_t \sum_{i=1}^{t} (Z_i - \hat{\mu}_t) \right) \\
&\quad = -\epsilon_t \beta t + \phi(\beta) \left(\sum_{i=1}^{t} (Z_i - \hat{\mu}_t)^2 + \epsilon_t^2 t \right) .
\end{aligned}$$

Hence, we have

$$\begin{aligned}
&\max_{\beta \in [-1,1]} \sum_{i=1}^{t} \ln(1+\beta(Z_i - \hat{\mu}_t - \epsilon_t)) \\
&\quad = \left(\sum_{i=1}^{t} (Z_i - \hat{\mu}_t)^2 + \epsilon_t^2 t \right) \psi \left(\frac{|\epsilon_t| t}{\sum_{i=1}^{t} (Z_i - \hat{\mu}_t)^2 + \epsilon_t^2 t} \right) ,
\end{aligned}$$

where $\psi(x) = |x| - \ln(|x|+1)$ and the equality is due to Lemma 16.26. From the inequality in Lemma 16.26 we also obtain

$$\max_{\beta \in [-1,1]} \sum_{i=1}^{t} \ln(1+\beta(Z_i - \hat{\mu}_t - \epsilon_t)) \geq \frac{\epsilon_t^2 t^2}{(4/3)|\epsilon_t| t + 2\sum_{i=1}^{t} (Z_i - \hat{\mu}_t)^2 + 2\epsilon_t^2 t} .$$

Now, note that for any $\mu \in (0,1)$ the interval $[-1,1]$ is contained in $[-\frac{1}{1-\mu}, \frac{1}{\mu}]$. Hence, from Theorem 16.20, uniformly on all t with probability at least $1-\delta$, we have

$$\frac{\epsilon_t^2 t^2}{(4/3)|\epsilon_t| t + 2\sum_{i=1}^{t} (Z_i - \hat{\mu}_t)^2 + 2\epsilon_t^2 t} \leq R_t + \ln \frac{1}{\delta} = \hat{R}_t .$$

Assuming ϵ_t is positive and solving for it, we have the stated upper bound. By the symmetry of the formula, the expression for negative ϵ_t has the opposite sign. □

16.5 History Bits

The contraction lemma (Lemma 16.3) appears in many places in the literature (see, e.g., Ledoux and Talagrand, 1991; Bartlett and Mendelson, 2001, 2002; Meir and Zhang, 2003). Kakade et al. (2008) proved Theorem 16.4 using strong convexity, while Kakade et al. (2009) proved it through the proof of Follow-the-Regularized-Leader (FTRL). Here, I simply used

a black-box conversion to the linear regret upper bound for any online learning algorithm. Massart's lemma was originally proved by Massart (2000).

PAC-Bayes bounds were first proposed by McAllester (1998) and, as explained by van Erven (2014), they can be seen as a continuous generalization of the union bound. There is now a vast literature on this subject, and we refer the reader to the recent review by Alquier (2024) for an introduction. Recently, PAC-Bayes bounds gained popularity because they can yield non-vacuous generalization bounds for deep neural networks. Yet, one should not confuse a certificate of generalization with an "explanation of generalization" in deep neural networks (see, e.g., Picard-Weibel et al., 2025). The reduction from online learning to PAC-Bayes bounds, as well as the idea of using parameter-free algorithms, is from Lugosi and Neu (2023). Despite what is claimed in Lugosi and Neu (2023), the PAC-Bayes bound in Corollary 16.10 is not new; it has been obtained by Kakade et al. (2008). Such a bound shaves off a $\ln T$ term under the square root compared to previous PAC-Bayes guarantees. Lugosi and Neu (2023) also present many more results based on the same reduction. Independently, Jang et al. (2023) proposed another reduction from coin betting to PAC-Bayes. While less general than the one presented here, it allows one to prove tighter results by changing the surrogate loss ℓ_t to a non-linear one. Later, Kuzborskij et al. (2024) used a similar approach to show the first PAC-Bayes bound with a better-than-KL divergence.

Zhang et al. (2020) proposed to use the Goldstein stationarity condition (Goldstein, 1977) for non-convex non-smooth objectives. They also proved a suboptimal bound of $\mathcal{O}(\epsilon^{-4}\delta^{-1})$. The barycentric definition, Theorem 16.17, and Lemma 16.18 are from Cutkosky et al. (2023). Note that I changed Definition 16.15 slightly because Algorithm 16.1 may return repeated points, which would not be allowed by the original definition in Cutkosky et al. (2023). The barycentric part of the definition is not strictly necessary for the results I presented, yet it is necessary for the additional results in Cutkosky et al. (2023) on functions whose Hessian is Lipschitz. A nonsmooth generalization of the identity used in the definition of well-behaved functions is given by Bolte and Pauwels (2021), who replace gradients with conservative set-valued fields satisfying an analogous integration identity along absolutely continuous paths. Zhang and Cutkosky (2024) improved the reduction in Algorithm 16.1 using an exponentially distributed s_t, which allows one to query the gradient at $\boldsymbol{x}_t$ rather than at the intermediate point $\boldsymbol{x}'_t$, and removes the feasible set $\mathcal{V}$. Jordan et al. (2023) proved that the randomization is essential in this class of problems to achieve dimension-free rates.

The results and proofs from Section 13.9 are taken from Orabona and Jun (2024). Note that the computational complexity to calculate $\mathcal{S}_t$ in Theorem 16.20 is $\mathcal{O}(t^2)$. Looser confidence intervals from portfolio algorithms but with complexity $\mathcal{O}(t)$ have been proposed in Orabona and Jun (2024) and Ryu and Bhatt (2024). Recently, Voráček and Orabona (2025) obtained state-of-the-art fixed-time confidence intervals as well, using a betting strategy that takes explicitly into account the temporal horizon and the threshold the algorithm's wealth must pass to test each value of the unknown mean.

Ville's inequality is proved in Ville (1939, Page 84), where its meaning is exactly associated with betting. In fact, Ville (1939) also introduced the concept of martingale as the wealth of a betting strategy on a fair coin. The ideas of Ville were later used to design ideal tests for randomness of infinite sequences (Schnorr, 1971; Levin, 1976; Gács, 2005), "ideal" because none of these tests is computable.

There are two papers that, at the same time and independently, explicitly link hypothe-

sis testing on a finite sequence of outcomes to betting. One is Cover (1974, Example 3), where an optimal betting strategy is defined in terms of the null hypothesis. The other one is Robbins and Siegmund (1974) that constructs confidence sequences from novel betting schemes and explicitly recognizes the connection between the sequential probability ratio test (Wald, 1945) and betting. Very surprisingly, Robbins and Siegmund (1974, Section 9) seem to have proposed and analyzed the famous strategy of Krichevsky and Trofimov (1981) 7 years before them. However, while in the information theory literature these ideas flourished and gave birth to results on coding, compression, minimum description length, and gambling, they seem to have disappeared from the statistics community for 30 years. In this view, it is remarkable that Cover (1974) was submitted to Annals of Statistics and probably rejected, as it can be inferred from the footnote on its first page. In fact, in the statistics literature, gambling strategies reappeared only between the 1990s and 2000s, thanks to the book and papers by Shafer and Vovk. In particular, Vovk (1993); Shafer and Vovk (2001) aimed to found probabilities on a game-theoretic ground through betting schemes. However, the foundational approach in Shafer and Vovk (2001) also means that all the betting strategies they propose do not have closed-form expressions, and they cannot be easily implemented. In addition, they do not use the regret analysis for the study of the proposed betting schemes. Finally, Shafer and Vovk (2001) does not contain any explicit concentration inequality, while the first concentration for game-theoretic probability derived by a betting scheme is in Vovk (2007), which derives a game-theoretic Hoeffding's inequality.

As far as I know, the first work to derive an explicit new concentration from the finite-time worst-case guarantee of an online gambling algorithm is in an unpublished work of mine from 2017, shared with a number of researchers in 2017–2018, and later published on arXiv (Orabona, 2026b). This work was based on the seminal work of Rakhlin and Sridharan (2017), which showed an *equivalence* between the regret guarantees of online learning algorithms with linear losses and concentration inequalities. However, the proof technique in Orabona (2026b) is different from the one in Rakhlin and Sridharan (2017), and it is specific to online algorithms that guarantee a non-negative exponential wealth for biased inputs. In particular, it allows us to derive time-uniform concentrations, like the law of the iterated logarithm, that are not possible with the method in Rakhlin and Sridharan (2017).

The first paper to consider an implementable strategy for testing through betting is by Hendriks (2018). Directly building on Shafer and Vovk (2001) and Shafer et al. (2011), Hendriks (2018) proposes to construct testing martingales and confidence sequences for bounded random variables as uniform mixtures of constant betting strategies, effectively a F-weighted portfolio with the uniform distribution. Hendriks (2018) also showed the good performance of the proposed approach empirically in a simple statistical test. Waudby-Smith and Ramdas (2024) seem to follow the same approach as Hendriks (2018) but propose several heuristic betting algorithms to maximize the wealth, as well as a discrete version of the uniform mixture that appeared in Hendriks (2018). Jun and Orabona (2019), based on the ideas in Orabona (2026b), show how to easily derive a Law of Iterated Logarithm for sub-Gaussian random vectors in Banach spaces from the regret analysis of a one-dimensional betting algorithm and the direction/magnitude reduction in Section 13.4. More recently, Jang et al. (2023) showed how to derive tighter PAC-Bayes bounds from the regret of portfolio algorithms.

Appendix

Appendix

A.1 More Convex Analysis

Theorem A.1 (Fundamental Theorem of Calculus for Extended-real-valued Convex Functions (Hiriart-Urruty and Lemaréchal, 2004, Theorem 2.3.4, page 179)) *Let $f : \mathbb{R}^d \to (-\infty, +\infty]$ be proper and convex. For $\boldsymbol{x}, \boldsymbol{y} \in \operatorname{int}\operatorname{dom} f$ and $t \in [0,1]$, define $\boldsymbol{x}_t = t\boldsymbol{y} + (1-t)\boldsymbol{x}$. Then, for any $\boldsymbol{g}_t \in \partial f(\boldsymbol{x}_t)$, we have*

$$f(\boldsymbol{y}) - f(\boldsymbol{x}) = \int_0^1 \langle \boldsymbol{g}_t, \boldsymbol{y} - \boldsymbol{x} \rangle \, \mathrm{d}t \, .$$

The above theorem hinges on the fact that the derivative of $\phi(t) = f(t\boldsymbol{y} + (1-t)\boldsymbol{x})$ exists except possibly for a countable set.

Definition A.2 *A topological space $\mathcal{X}$ is a **Hausdorff space** if any two distinct points can be separated by disjoint open sets. That is, for any $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}$ with $\boldsymbol{x} \neq \boldsymbol{y}$, there exist open sets $\mathcal{U}, \mathcal{V} \subseteq \mathcal{X}$ such that $\boldsymbol{x} \in \mathcal{U}$, $\boldsymbol{y} \in \mathcal{V}$, and $\mathcal{U} \cap \mathcal{V} = \emptyset$.*

Note that Euclidean spaces are Hausdorff spaces.

Theorem A.3 (Weierstrass theorem for extended-real-valued functions (Bauschke et al., 2003, Theorem 1.29)) *Let $\mathcal{X}$ be a Hausdorff space, let $f : \mathcal{X} \to [-\infty, +\infty]$ be lower semicontinuous, and let $\mathcal{V}$ be a compact subset of $\mathcal{X}$. Suppose $\mathcal{V} \cap \operatorname{dom} f \neq \emptyset$. Then, f achieves its infimum over $\mathcal{V}$.*

A.2 Inequalities for Transcendental Functions

A.2.1 The Lambert Function

The Lambert function $W : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$ is defined by the equality

$$x = W(x) \exp\left(W(x)\right), \quad \text{for } x \geq 0 \, . \tag{A.1}$$

Hence, from the definition we have that $\exp(W(x)/2) = \sqrt{\frac{x}{W(x)}}$ for $x > 0$.

We have the following property of the Lambert function, stated without proof.

Theorem A.4 *The Lambert function satisfies* $\lim_{x \to +\infty} \frac{W(x)}{\ln x} = 1$

Theorem A.5 (Hoorfar and Hassani, 2008, Theorem 2.3)

$$W(x) \leq \ln \frac{x + C}{1 + \ln(C)}, \quad \forall x \geq 0, \; \forall C > \frac{1}{e}.$$

The following theorem provides upper and lower bounds on $W(x)$.

Theorem A.6 (Orabona and Pál, 2016, Lemma 17) *The Lambert function $W(x)$ satisfies*

$$0.6321 \ln(x+1) \leq W(x) \leq \ln(x+1), \quad \forall x \geq 0 \,.$$

Proof The inequalities are satisfied for $x = 0$, hence in the following we assume $x > 0$. We first prove the lower bound. From (A.1) we have

$$W(x) = \ln\left(\frac{x}{W(x)}\right) \,. \tag{A.2}$$

From this equality, using the elementary inequality $\ln(u) \leq \frac{1}{ae} u^a$ for any $a > 0$, we get

$$W(x) \leq \frac{1}{a\,e}\left(\frac{x}{W(x)}\right)^a, \quad \forall a > 0,$$

that is

$$W(x) \leq \left(\frac{1}{a\,e}\right)^{\frac{1}{1+a}} x^{\frac{a}{1+a}}, \quad \forall a > 0. \tag{A.3}$$

Using (A.3) in (A.2), we have

$$W(x) \geq \ln\left(\frac{x}{\left(\frac{1}{a\,e}\right)^{\frac{1}{1+a}} x^{\frac{a}{1+a}}}\right) = \frac{1}{1+a}\ln\left(a\,e\,x\right), \quad \forall a > 0 \,.$$

Consider now the function $g(x) = \frac{x}{x+1} - \frac{b}{\ln(1+b)(b+1)}\ln(x+1)$ defined in $[0, b]$ where b is a positive number that will be decided in the following. This function has a maximum in $x^\star = (1+\frac{1}{b})\ln(1+b) - 1$, the derivative is positive in $[0, x^\star]$ and negative in $[x^\star, b]$. Hence, the minimum is attained at $x = 0$ and $x = b$, where it is equal to 0. Using the property just proved on g, setting $a = \frac{1}{x}$, we have

$$W(x) \geq \frac{x}{x+1} \geq \frac{b}{\ln(1+b)(b+1)}\ln(x+1), \quad \forall x \leq b \,.$$

For $x > b$, setting $a = \frac{x+1}{ex}$, we have

$$W(x) \geq \frac{e\,x}{(e+1)x+1}\ln(x+1) \geq \frac{e\,b}{(e+1)b+1}\ln(x+1) \,.$$

Hence, we set b such that

$$\frac{e\,b}{(e+1)b+1} = \frac{b}{\ln(1+b)(b+1)} \,.$$

Numerically, $b = 1.71825\ldots$, so $W(x) \geq 0.6321\ln(x+1)$.

For the upper bound, we use Theorem A.5 and set $C = 1$. □

Theorem A.7 *Let $a, b > 0$. Then, the Fenchel conjugate of $f : \mathbb{R} \to \mathbb{R}$ defined as $f(x) = b\exp(x^2/(2a))$ is*

$$\begin{aligned} f^\star(\theta) &= \sqrt{a}|\theta|\sqrt{W(a\theta^2/b^2)} - b\exp\left(\frac{W(a\theta^2/b^2)}{2}\right) \\ &= \sqrt{a}|\theta|\left(\sqrt{W(a\theta^2/b^2)} - \frac{1}{\sqrt{W(a\theta^2/b^2)}}\right) . \end{aligned}$$

Moreover,

$$\sqrt{a}|\theta|\left(\sqrt{0.6321\ln(a\theta^2/b^2+1)}-1\right) - b \leq f^\star(\theta) \leq \sqrt{a}|\theta|\sqrt{\ln(a\theta^2/b^2+1)} - b\,.$$

Proof First, observe that

$$\max_x\ \theta x - b\exp(x^2/(2a)) = \max_y\ b\left(\frac{\sqrt{a}\theta}{b}y - \exp(y^2/2)\right) .$$

Also, by the definition of the Lambert function, we have

$$\operatorname*{argmax}_y\ uy - \exp(y^2/2) = \operatorname{sign}(u)\sqrt{W(u^2)},$$

where $\operatorname{sign}(u) = 0$ for $u = 0$. Hence, we have

$$\operatorname*{argmax}_x\ \theta x - b\exp(x^2/(2a)) = \operatorname{sign}(\theta)\sqrt{a}\sqrt{W(a\theta^2/b^2)}\,.$$

So, we obtain

$$\begin{aligned} \max_x\ \theta x - b\exp(x^2/(2a)) &= \max_y\ b\left(\frac{\sqrt{a}\theta}{b}y - \exp(y^2/2)\right) \\ &= \sqrt{a}|\theta|\sqrt{W(a\theta^2/b^2)} - b\exp\left(\frac{W(a\theta^2/b^2)}{2}\right) \\ &= \sqrt{a}|\theta|\sqrt{W(a\theta^2/b^2)} - b\sqrt{\frac{a\theta^2/b^2}{W(a\theta^2/b^2)}} \\ &= \sqrt{a}|\theta|\left(\sqrt{W(a\theta^2/b^2)} - \frac{1}{\sqrt{W(a\theta^2/b^2)}}\right) . \end{aligned}$$

The upper bound is obtained using Theorem A.6 and upper bounding $-b\exp(\frac{W(\frac{a\theta^2}{b^2})}{2})$ with $-b$.

For the lower bound, we use again Theorem A.6 and observe that

$$\begin{aligned} -b\exp\left(\frac{W(a\theta^2/b^2)}{2}\right) &\geq -b\exp\left(\frac{1}{2}\ln\left(a\theta^2/b^2+1\right)\right) = -b\sqrt{a\theta^2/b^2+1} \\ &= -\sqrt{a\theta^2+b^2} \geq -\sqrt{a}|\theta| - b\,. \end{aligned}$$

□

A.2.2 The Gamma Function

Definition A.8 *For $\Re(x) > 0$, the **gamma function** is defined as*

$$\Gamma(x) := \int_0^\infty t^{x-1} e^{-t} \, \mathrm{d}t \, .$$

Definition A.9 *The **digamma function** is defined as*

$$\psi(x) := \frac{d}{dx} \ln \Gamma(x) = \frac{\Gamma'(x)}{\Gamma(x)}, \quad \forall x > 0 \, .$$

Proposition A.10 *For the gamma and digamma functions, we have the following properties:*

(a) The gamma function is log-convex on $\mathbb{R}_{>0}$, i.e., $\ln \Gamma$ is convex.
(b) Wendel's double inequality (Wendel, 1948)

$$\left(\frac{x}{x+s} \right)^{1-s} \leq \frac{\Gamma(x+s)}{x^s \Gamma(x)} \leq 1, \quad \forall 0 < s < 1, \; \forall x > 0 \, .$$

(c) $\Gamma(x) \geq \left(\frac{x-1/2}{e} \right)^{x-1/2} \sqrt{2e}, \quad x \geq 1$ (Batir, 2008, Theorem 1.5).
(d) The digamma function is strictly increasing and strictly concave on $(0, \infty)$.
(e) For all $x > 0$, we have $\ln x - \frac{1}{x} \leq \psi(x) \leq \ln x - \frac{1}{2x}$.
(f) For all $x > 0$, we have $\psi'(x) = \sum_{k=0}^{\infty} \frac{1}{(x+k)^2}$.

Lemma A.11 *Let $T \geq 1$ and $d \geq 2$ be integers. Then, we have*

$$\frac{\Gamma(T + d/2)}{\Gamma(T + 1/2)} \leq (T + d/2 - 1/2)^{(d-1)/2} \, .$$

Proof From the definition of the digamma function ψ, we have

$$\begin{aligned} \ln \frac{\Gamma(T + d/2 - 1/2)}{\Gamma(T + 1/2)} &= \int_{T+1/2}^{T+d/2-1/2} \psi(t) \, \mathrm{d}t \leq \int_{T+1/2}^{T+d/2-1/2} \psi\left(T + d/2 - 1/2\right) \, \mathrm{d}t \\ &= \frac{d-2}{2} \psi\left(T + d/2 - 1/2\right) \leq \frac{d-2}{2} \ln\left(T + d/2 - 1/2\right), \end{aligned}$$

where we used the fact that the digamma is increasing (Proposition A.10(d)) and the upper bound $\psi(x) < \ln(x)$ (Proposition A.10(e)). Then, from Proposition A.10(b), we use the fact that $\Gamma(T + d/2) \leq \sqrt{T + d/2 - 1/2} \, \Gamma(T + d/2 - 1/2)$. Hence, we have

$$\ln \frac{\Gamma(T + d/2)}{\Gamma(T + 1/2)} \leq \frac{d-1}{2} \ln\left(T + d/2 - 1/2\right) \, . \qquad \square$$

Lemma A.12 *Let $T \geq 1$ and $d \geq 2$ be integers. Then, we have*

$$\ln \frac{\sqrt{\pi} \Gamma(T + d/2)}{\Gamma(d/2) \Gamma(T + 1/2)} \leq \frac{d-1}{2} \ln\left(\frac{T}{(d-1)/2} + 1 \right) + \frac{d}{2} - 1 + \frac{1}{2} \ln \frac{\pi}{2} \, .$$

Hence, for $d = 2$, the upper bound is $\frac{1}{2} \ln\left(\pi T + \frac{\pi}{2}\right) \leq \frac{1}{2} \ln T + 1$.

Proof We use Proposition A.10(c):

$$\Gamma(d/2) \geq \left(\frac{d/2 - 1/2}{e}\right)^{\frac{d-1}{2}} \sqrt{2e}\,.$$

Hence, using Lemma A.11, we have

$$\begin{aligned}\ln \frac{\sqrt{\pi}\Gamma\left(T + \frac{d}{2}\right)}{\Gamma\left(\frac{d}{2}\right)\Gamma\left(T + \frac{1}{2}\right)} &\leq \frac{1}{2}\ln \pi + \frac{d-1}{2}\ln\left(T + \frac{d-1}{2}\right) - \frac{d-1}{2}\ln\frac{d-1}{2e} - \frac{1}{2}\ln(2e) \\ &= \frac{d-1}{2}\ln\left(\frac{T}{(d-1)/2} + 1\right) + \frac{d}{2} - 1 + \frac{1}{2}\ln\frac{\pi}{2}\,. \end{aligned}$$

□

Lemma A.13 *Let $T \geq 1$ and let $n \geq 1$ be an integer. Define $f : \mathbb{R}^n_{\geq 0} \to \mathbb{R}$ as*

$$f(a_1, \ldots, a_n) = \prod_{i=1}^{n} \frac{a_i^{a_i}}{\Gamma\left(a_i + \frac{1}{2}\right)},$$

where $0^0 = 1$. Then, f is maximized over the set $\mathcal{V} = \{(a_1, \ldots, a_n) \in \mathbb{R}^n_{\geq 0} : \sum_{i=1}^n a_i = T\}$ at the extreme points of the simplex, namely when one coordinate is equal to T and all the others are equal to 0.

Proof Define

$$h(x) := x \ln x - \ln \Gamma\left(x + \frac{1}{2}\right), \quad \forall x > 0\,.$$

Then, $\ln f(a_1, \ldots, a_n) = \sum_{i=1}^n h(a_i)$. So, it is enough to prove that h is convex on $(0, \infty)$.

We have

$$h''(x) = \frac{1}{x} - \psi'\left(x + \frac{1}{2}\right),$$

where ψ is the digamma function.

Using Proposition A.10(f), we have

$$\psi'\left(x + \frac{1}{2}\right) = \sum_{k=0}^{\infty} \frac{1}{\left(x + k + \frac{1}{2}\right)^2}\,.$$

Now, define

$$g_x(t) := \frac{1}{(x+t)^2}, \quad t \geq 0\,.$$

Since g_x is convex on $[0, \infty)$, using Jensen's inequality (Theorem 2.9), for every $k \geq 0$ we have

$$g_x\left(k + \frac{1}{2}\right) = g_x\left(\int_k^{k+1} t\,\mathrm{d}t\right) \leq \int_k^{k+1} g_x(t)\,\mathrm{d}t\,.$$

Therefore,

$$\psi'\left(x + \frac{1}{2}\right) = \sum_{k=0}^{\infty} g_x\left(k + \frac{1}{2}\right) \leq \sum_{k=0}^{\infty} \int_k^{k+1} g_x(t)\,dt = \int_0^{\infty} \frac{\mathrm{d}t}{(x+t)^2} = \frac{1}{x}\,.$$

Hence

$$h''(x) = \frac{1}{x} - \psi'\left(x + \frac{1}{2}\right) \geq 0\,.$$

Therefore, $\ln f(a_1, \ldots, a_n)$ is convex on the relative interior of $\mathcal{V}$. We can extend the function $\sum_i h(a_i)$ by continuity to all of $\mathcal{V}$.

A convex function on a simplex attains its maximum at an extreme point. This proves the claim. □

A.3 Probabilistic Inequalities

A.3.1 Khintchine's inequality

Theorem A.14 (Khintchine's inequality) *Let $\{\epsilon_n\}_{n=1}^N$ be i.i.d. Rademacher random variables, i.e.,*

$$\mathbb{P}\{\epsilon_n = 1\} = \mathbb{P}\{\epsilon_n = -1\} = \frac{1}{2}, \quad \forall n = 1, \ldots, N\,.$$

Let $0 < p < \infty$ and let $x_1, \ldots, x_N \in \mathbb{C}$. Then, we have

$$A_p \left(\sum_{n=1}^N |x_n|^2\right)^{1/2} \leq \left(\mathbb{E}\left|\sum_{n=1}^N \epsilon_n x_n\right|^p\right)^{1/p} \leq B_p \left(\sum_{n=1}^N |x_n|^2\right)^{1/2},$$

where

$$A_p = \begin{cases} 2^{1/2-1/p} & 0 < p \leq p_0, \\ 2^{1/2}(\Gamma((p+1)/2)/\sqrt{\pi})^{1/p} & p_0 < p < 2 \\ 1 & 2 \leq p < \infty \end{cases}$$

$$B_p = \begin{cases} 1 & 0 < p \leq 2 \\ 2^{1/2}(\Gamma((p+1)/2)/\sqrt{\pi})^{1/p} & 2 < p < \infty \end{cases},$$

where $p_0 \approx 1.847$ and Γ is the gamma function.

The constants in the above theorem are due to Haagerup (1981).

A.3.2 Lower Bound for the Tail of the Binomial Distribution

First, we prove a new handy result for the sum of binomial coefficients.

Lemma A.15 *Let $n \geq 1$ and $n/2 \leq k \leq n$ be integers. Define*

$$\phi(k, n) := \frac{1}{2}\sqrt{4k^2 - 4kn + (n+2)^2} - k - n/2 - 1\,.$$

Then, $\sum_{i=k}^n \binom{n}{i} \geq \binom{n}{k} \frac{k}{2k+\phi(k,n)}$. Moreover, $\phi(k, n)$ is non-increasing in the first argument.

Proof First of all, observe that $\phi(k,n)$ is non-increasing in the first argument. Indeed, the first derivative is

$$\begin{aligned}\frac{2k-n}{\sqrt{4k^2-4kn+(n+2)^2}}-1 &= \frac{2k-n-\sqrt{4k^2-4kn+(n+2)^2}}{\sqrt{4k^2-4kn+(n+2)^2}} \\ &\leq \frac{2k-n-\sqrt{4k^2-4kn+n^2}}{\sqrt{4k^2-4kn+(n+2)^2}} = 0\,.\end{aligned}$$

We will prove the statement by backward induction. The base case is easy: $\sum_{i=n}^{n}\binom{n}{i} = 1 = \binom{n}{n}\frac{n}{2n+\phi(n,n)}$. So, now assume that $\sum_{i=k+1}^{n}\binom{n}{i} \geq \binom{n}{k+1}\frac{k+1}{2(k+1)+\phi(k+1,n)}$. We have that

$$\begin{aligned}\sum_{i=k}^{n}\binom{n}{i} &= \sum_{i=k+1}^{n}\binom{n}{i}+\binom{n}{k} \geq \binom{n}{k+1}\frac{k+1}{2(k+1)+\phi(k+1,n)}+\binom{n}{k} \\ &= \binom{n}{k}\left(\frac{n-k}{k+1}\frac{k+1}{2k+2+\phi(k+1,n)}+1\right) \\ &= \binom{n}{k}\left(\frac{n-k}{2k+2+\phi(k+1,n)}+1\right) \geq \binom{n}{k}\left(\frac{n-k}{2k+2+\phi(k,n)}+1\right) \\ &= \binom{n}{k}\frac{k}{2k+\phi(k,n)},\end{aligned}$$

where the last equality is due to the definition of $\phi(k,n)$. □

Remark A.16 *For n even, this lower bound gives us $\sum_{i=n/2+q}^{n}\binom{n}{i} \geq \binom{n}{n/2+q}\frac{n/2+q}{q+\sqrt{q^2+n+1}-1}$. Hence, it is tight in $q = n/2$, while in $q = 0$ it is*

$$\sum_{i=n/2}^{n}\binom{n}{i} \geq \binom{n}{n/2}\frac{n/2}{\sqrt{n+1}-1} \geq \binom{n}{n/2}\frac{n/2}{\sqrt{n}} = \binom{n}{n/2}\frac{\sqrt{n}}{2},$$

that is larger by a factor $\frac{\sqrt{n}}{2}$ than the trivial lower bound $\sum_{i=n/2}^{n}\binom{n}{i} \geq \binom{n}{n/2}$.

We denote the Kullback–Leibler (KL) divergence between two Bernoulli distributions with parameters $p \in [0,1]$ and $q \in (0,1)$ as

$$\mathrm{KL}_{\text{Bern}}(p;q) := p\ln\left(\frac{p}{q}\right)+(1-p)\ln\left(\frac{1-p}{1-q}\right),$$

where we define $0\ln 0 := 0$.

We will also denote by $B(n,p)$ the binomial distribution with parameters n and p.

We can now state the bound on the tail of the Binomial distribution.

Lemma A.17 (Bound on Binomial Tail) *Let $n \geq 1$ be an even integer and $0 \leq q \leq$*

$n/2-1$ *integer. Then,* $X_n \sim B(n, \frac{1}{2})$ *satisfies*

$$\mathbb{P}\left\{X_n \geq \frac{n}{2}+q\right\} \geq \frac{1}{3}\frac{1}{2q+\sqrt{n}}\sqrt{n\frac{n/2+q}{n/2-q}}\,\exp\left(-n\cdot \mathrm{KL}_{\mathit{Bern}}\left(\frac{1}{2}+\frac{q}{n};\frac{1}{2}\right)\right)$$
$$\geq \frac{1}{3}\frac{1}{\frac{2q}{\sqrt{n}}+1}\exp\left(-n\cdot \mathrm{KL}_{\mathit{Bern}}\left(\frac{1}{2}+\frac{q}{n};\frac{1}{2}\right)\right).$$

Note that the second lower bound also holds in $q = n/2$.

Proof By definition, we have

$$\mathbb{P}\left\{X_n \geq \frac{n}{2}+q\right\} = 2^{-n}\sum_{i=n/2+q}^{n}\binom{n}{i}.$$

From Lemma A.15, we have

$$\sum_{i=n/2+q}^{n}\binom{n}{i} \geq \binom{n}{n/2+q}\frac{n/2+q}{n+2q+\phi(n/2+q,n)} \geq \binom{n}{n/2+q}\frac{n/2+q}{2q+\sqrt{n}}.$$

We bound the binomial coefficient $\binom{n}{k}$ using Stirling's formula for factorials. In particular, we use the explicit upper and lower bounds due to Robbins (1955) valid for any $n \geq 1$,

$$\sqrt{2\pi n}\left(\frac{n}{e}\right)^n < n! < e^{1/12}\sqrt{2\pi n}\left(\frac{n}{e}\right)^n.$$

Hence, for any $1 \leq k \leq n-1$, we have

$$\begin{aligned}\binom{n}{k} &= \frac{n!}{k!(n-k)!}\\ &> \frac{\sqrt{2\pi n}\,n^n e^{-n}}{\sqrt{2\pi k}\,k^k e^{-k}e^{1/12}\cdot\sqrt{2\pi(n-k)}\,(n-k)^{n-k}e^{-(n-k)}e^{1/12}}\\ &= \frac{1}{e^{1/6}\sqrt{2\pi}}\left(\frac{n}{n-k}\right)^{n-k}\left(\frac{n}{k}\right)^k\sqrt{\frac{n}{k(n-k)}}\\ &= \frac{1}{e^{1/6}\sqrt{2\pi}}\,2^n\exp\left(-n\cdot\mathrm{KL}_{\text{Bern}}\left(\frac{k}{n};\frac{1}{2}\right)\right)\sqrt{\frac{n}{k(n-k)}},\end{aligned}$$

where in the equality we used the definition of $\mathrm{KL}_{\text{Bern}}(p;q)$. Hence, we have

$$\binom{n}{n/2+q} \geq \frac{2^n}{e^{1/6}\sqrt{2\pi}}\exp\left(-n\ \mathrm{KL}_{\text{Bern}}\left(\frac{1}{2}+\frac{q}{n};\frac{1}{2}\right)\right)\sqrt{\frac{n}{(n/2+q)(n/2-q)}}.$$

Combining all the inequalities, using the fact that $\sqrt{\frac{n/2+q}{n/2-q}} \geq 1$, and overapproximating, we get the stated bound.

For $q = n/2$, we verify the statement of the lemma by direct substitution. The l.h.s. is $\mathbb{P}[X_n \geq n] = 2^{-n}$. Since $\exp\left(-n\cdot\mathrm{KL}_{\text{Bern}}(1;\frac{1}{2})\right) = 2^{-n}$, it is easy to see that the r.h.s. is smaller than 2^{-n}. □

References

J. Abernethy, C. Lee, A. Sinha, and A. Tewari. Online linear optimization via smoothing. In *Conference on Learning Theory (COLT)*, pages 807–823, 2014. 137

J. D. Abernethy and J.-K. Wang. On Frank-Wolfe and equilibrium computation. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. 267

J. D. Abernethy, E. Hazan, and A. Rakhlin. Competing in the dark: An efficient algorithm for bandit linear optimization. In R. A. Servedio and T. Zhang, editors, *Proc. of Conference on Learning Theory (COLT)*, pages 263–274. Omnipress, 2008. 96, 136

J. D. Abernethy, C. Lee, and A. Tewari. Fighting bandits with a new kind of smoothness. In *Advances in Neural Information Processing Systems 28*, pages 2197–2205. Curran Associates, Inc., 2015. 168

D. Adamskiy, W. M. Koolen, A. Chernov, and V. Vovk. A closer look at adaptive regret. In *Algorithmic Learning Theory: 23rd International Conference, ALT 2012, Lyon, France, October 29-31, 2012. Proceedings 23*, pages 290–304. Springer, 2012. 243

D. Adamskiy, W. M. Koolen, A. Chernov, and V. Vovk. A closer look at adaptive regret. *Journal of Machine Learning Research*, 17(23):1–21, 2016. 243

A. Agarwal and E. Hazan. New algorithms for repeated play and universal portfolio management. Technical Report TR-740-05, Princeton University Technical Report, 2005. 137

A. Agarwal, A. Rakhlin, and P. Bartlett. Matrix regularization techniques for online multitask learning. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2008-138*, 2008. 138

N. Agarwal, R. Anil, E. Hazan, T. Koren, and C. Zhang. Disentangling adaptive gradient methods from learning rates. *arXiv preprint arXiv:2002.11803*, 2020. 49

M. A. Aizerman, E. M. Braverrnan, and L. I. Rozonoer. Theoretical foundations of the potential function method in pattern recognition learning. *Automation and remote control*, 25(6):917–936, 1964. 149

P. Alquier. User-friendly introduction to PAC-Bayes bounds. *Foundations and Trends in Machine Learning*, 17(2):174–303, 2024. 288

S. Amari. *Differential-Geometrical Methods in Statistics*. Springer-Verlag, 1985. 137

A. Argyriou, T. Evgeniou, and M. Pontil. Multi-task feature learning. In B. Schölkopf, J. Platt, and T. Hoffman, editors, *Advances in Neural Information Processing Systems*, volume 19. MIT Press, 2006. 138

R. Arora, O. Dekel, and A. Tewari. Online bandit learning against an adaptive adversary: from regret to policy regret. In *Proc. of the International Conference on Machine Learning (ICML)*, January 2012. 151

K. J. Arrow, L. Hurwicz, and H. Uzawa. *Studies in linear and non-linear programming*, volume 2. Stanford University Press Stanford, 1958. 268

J.-Y. Audibert and S. Bubeck. Minimax policies for adversarial and stochastic bandits. In *Proc. of the Conference on Learning Theory (COLT)*, 2009. 168

J.-Y. Audibert, R. Munos, and C. Szepesvári. Exploration-exploitation tradeoff using variance estimates in multi-armed bandits. *Theoretical Computer Science*, 410(19):1876–1902, 2009. 284

J.-Y. Audibert, S. Bubeck, and G. Lugosi. Minimax policies for combinatorial prediction games. In *Proceedings of the Conference on Learning Theory (COLT)*, pages 107–132, 2011. 168

P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47:235–256, 2002a. 169

P. Auer, N. Cesa-Bianchi, Y. Freund, and R. E. Schapire. The nonstochastic multiarmed bandit problem. *SIAM journal on computing*, 32(1):48–77, 2002b. 168

P. Auer, N. Cesa-Bianchi, and C. Gentile. Adaptive and self-confident on-line learning algorithms. *J. Comput. Syst. Sci.*, 64(1):48–75, 2002c. 48

K. S. Azoury and M. K. Warmuth. Relative loss bounds for on-line density estimation with the exponential family of distributions. *Machine Learning*, 43(3):211–246, 2001. 97, 139

A. Baños. On pseudo-games. *Annals of Mathematical Statistics*, 39:1932—1945, 1968. 168

S. Bakin. *Adaptive regression and model selection in data mining problems*. PhD thesis, The Australian National University, 1999. 138

P. L. Bartlett and S. Mendelson. Rademacher and Gaussian complexities: Risk bounds and structural results. In *International Conference on Computational Learning Theory*, pages 224–240. Springer, 2001. 287

P. L. Bartlett and S. Mendelson. Rademacher and Gaussian complexities: Risk bounds and structural results. *Journal of machine learning research*, 3(Nov):463–482, 2002. 287

N. Batir. Inequalities for the gamma function. *Archiv der Mathematik*, 91(6):554–563, 2008. 294

H. H. Bauschke and J. M. Borwein. Legendre functions and the method of random Bregman projections. *Journal of convex analysis*, 4(1):27–67, 1997. 96

H. H. Bauschke and P. L. Combettes. *Convex analysis and monotone operator theory in Hilbert spaces*. Springer, 2017. Second edition. 16, 18, 63, 69, 72, 77

H. H. Bauschke, J. M. Borwein, and P. L. Combettes. Bregman monotone optimization algorithms. *SIAM Journal on Control and Optimization*, 42(2):596–636, 2003. 95, 291

H. H. Bauschke, J. Bolte, and M. Teboulle. A descent lemma beyond Lipschitz gradient continuity: first-order methods revisited and applications. *Mathematics of Operations Research*, 42(2):330–348, 2017. 138

A. Beck and M. Teboulle. Mirror descent and nonlinear projected subgradient methods for convex optimization. *Operations Research Letters*, 31(3):167–175, 2003. 95, 96

A. Ben-Tal, T. Margalit, and A. Nemirovski. The ordered subsets mirror descent optimization method with applications to tomography. *SIAM Journal on Optimization*, 12(1):79–108, 2001. 96

A. Beygelzimer, F. Orabona, and C. Zhang. Efficient online bandit multiclass learning with $\tilde{O}(\sqrt{T})$ regret. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 488–497. JMLR. org, 2017. 150

B. Birnbaum, N. R. Devanur, and L. Xiao. New convex programs and distributed algorithms for Fisher markets with linear and spending constraint utilities. Technical Report MSR-TR-2010-112, Microsoft Research, August 2010. 138

B. Birnbaum, N. R. Devanur, and L. Xiao. Distributed algorithms via gradient descent for Fisher markets. In *Proceedings of the 12th ACM conference on Electronic commerce*, pages 127–136, 2011. 138

D. Blackwell and D. Freedman. On the amount of variance needed to escape from a strip. *The Annals of Probability*, 1(5):772–787, 1973. 48

H.-D. Block. The Perceptron: A model for brain functioning. I. *Reviews of Modern Physics*, 34(1):123–135, 1962. 149

A. Blum. Empirical support for Winnow and weighted-majority algorithms: Results on a calendar scheduling domain. *Machine Learning*, 26:5–23, 1997. 205

A. Blum and A. Kalai. Universal portfolios with and without transaction costs. In *Proceedings of the Tenth Annual Conference on Computational Learning Theory*, pages 309–313, 1997. 178, 191

A. Blum and A. Kalai. Universal portfolios with and without transaction costs. *Machine Learning*, 35: 193–205, 1999. 178, 191

A. Blum and Y. Mansour. From external to internal regret. In *Learning Theory: 18th Annual Conference on Learning Theory, COLT 2005, Bertinoro, Italy, June 27-30, 2005. Proceedings 18*, pages 621–636. Springer, 2005. 243

A. Blum and Y. Mansour. From external to internal regret. *Journal of Machine Learning Research*, 8(6), 2007. 243

A. Blum, A. Kalai, and J. Langford. Beating the hold-out: Bounds for k-fold and progressive cross-validation. In *COLT*, volume 99, pages 203–208, 1999. 33

J. Bolte and E. Pauwels. Conservative set valued fields, automatic differentiation, stochastic gradient methods and deep learning. *Mathematical Programming*, 188(1):19–51, 2021. 288

M. Bowling, N. Burch, M. Johanson, and O. Tammelin. Heads-up limit hold'em poker is solved. *Science*, 347(6218):145–149, 2015. 204

S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, New York, NY, USA, 2004. ISBN 0521833787. 9

L. M. Bregman. The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming. *USSR Computational Mathematics and Mathematical Physics*, 7(3):200–217, 1967. 95

S. Bubeck and N. Cesa-Bianchi. Regret analysis of stochastic and nonstochastic multi-armed bandit problems. *Foundations and Trends® in Machine Learning*, 5(1):1–122, 2012. 165, 169

S. Bubeck, N. R. Devanur, Z. Huang, and R. Niazadeh. Online auctions and multi-scale online learning. In *Proceedings of the 2017 ACM Conference on Economics and Computation*, pages 497–514, 2017. 96

S. Bubeck, N. R. Devanur, Z. Huang, and R. Niazadeh. Multi-scale online learning: Theory and applications to online auctions and pricing. *Journal of Machine Learning Research*, 20(62):1–37, 2019. 96

N. Burch, M. Moravcik, and M. Schmid. Revisiting CFR+ and alternating updates. *Journal of Artificial Intelligence Research*, 64:429–443, 2019. 268

N. Campolongo and F. Orabona. Temporal variability in implicit online learning. In *Advances in Neural Information Processing Systems*, volume 33. Curran Associates, Inc., 2020. 268

Y. Carmon and O. Hinder. The price of adaptivity in stochastic convex optimization. In *Conference on Learning Theory*, 2024. 59

N. Cesa-Bianchi. Analysis of two gradient-based algorithms for on-line regression. *Journal of Computer and System Sciences*, 59(3):392–411, 1999. 23

N. Cesa-Bianchi and G. Lugosi. Potential-based algorithms in online prediction and game theory. In *International Conference on Computational Learning Theory*, pages 48–64. Springer, 2001. 205

N. Cesa-Bianchi and G. Lugosi. Potential-based algorithms in on-line prediction and game theory. *Machine Learning*, 51(3):239–261, 2003. 205

N. Cesa-Bianchi and G. Lugosi. *Prediction, learning, and games*. Cambridge University Press, 2006. 23, 95, 96, 129, 168, 190, 204, 205, 242, 267

N. Cesa-Bianchi, Y. Freund, D. P. Helmbold, D. Haussler, R. E. Schapire, and M. K Warmuth. How to use expert advice. In *Proceedings of the twenty-fifth annual ACM symposium on Theory of Computing*, pages 382–391, 1993. 96

N. Cesa-Bianchi, P. M. Long, and M. K. Warmuth. Worst-case quadratic loss bounds for prediction using linear functions and gradient descent. *IEEE Transactions on Neural Networks*, 7(3):604–619, 1996. 48

N. Cesa-Bianchi, Y. Freund, D. Haussler, D. P. Helmbold, R. E. Schapire, and M. K. Warmuth. How to use expert advice. *J. ACM*, 44(3):427–485, 1997. 96

N. Cesa-Bianchi, A. Conconi, and C. Gentile. On the generalization ability of on-line learning algorithms. *IEEE Trans. Inf. Theory*, 50(9):2050–2057, 2004. 33

N. Cesa-Bianchi, Y. Mansour, and G. Stoltz. Improved second-order bounds for prediction with expert advice. In *International Conference on Computational Learning Theory*, pages 217–232. Springer, 2005. 49

N. Cesa-Bianchi, Y. Mansour, and G. Stoltz. Improved second-order bounds for prediction with expert advice. *Machine Learning*, 66(2):321–352, 2007. 49

N. Cesa-Bianchi, P. Gaillard, G. Lugosi, and G. Stoltz. Mirror descent meets fixed share (and feels no regret). In *Advances in Neural Information Processing Systems*, volume 25, 2012. 243

K. Chaudhuri, Y. Freund, and D. J. Hsu. A parameter-free hedging algorithm. In *Advances in neural information processing systems*, pages 297–305, 2009. 224, 225

G. Chen and M. Teboulle. Convergence analysis of a proximal-like minimization algorithm using Bregman functions. *SIAM Journal on Optimization*, 3(3):538–543, 1993. 64

K. Chen, A. Cutkosky, and F. Orabona. Implicit parameter-free online learning with truncated linear models. In *International Conference on Algorithmic Learning Theory*, pages 148–175. PMLR, 2022a. 226

K. Chen, J. Langford, and F. Orabona. Better parameter-free stochastic optimization with ODE updates for coin-betting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 6239–6247, 2022b. 226

L. Chen, H. Luo, and C.-Y. Wei. Impossible tuning made possible: A new expert algorithm and its applications. In *Proc. of the Conference on Learning Theory*, 2021. 96

A. Chernov and V. Vovk. Prediction with advice of unknown number of experts. In *Proc. of the Conference on Uncertainty in Artificial Intelligence (UAI)*, 2010. 225

C.-K. Chiang, T. Yang, C.-J. Lee, M. Mahdavi, C.-J. Lu, R. Jin, and S. Zhu. Online optimization with gradual variations. In *Proc. of the Conference on Learning Theory (COLT)*, volume 23, pages 6.1–6.20, 2012. 97, 139, 268

L. Condat. Fast projection onto the simplex and the $\boldsymbol{l_1}$ ball. *Mathematical Programming*, 158(1):575–585, 2016. 81

T. M. Cover. Behavior of sequential predictors of binary sequences. In *Proc. of the 4th Prague Conference on Information Theory, Statistical Decision Functions and Random Processes*, pages 263–272. Publishing House of the Czechoslovak Academy of Sciences, 1965. 96

T. M Cover. Universal gambling schemes and the complexity measures of Kolmogorov and Chaitin. Technical Report 12, Department of Statistics, Stanford University, 1974. 289

T. M. Cover. Universal portfolios. *Mathematical Finance*, pages 1–29, 1991. 178

T. M. Cover and E. Ordentlich. Universal portfolios with side information. *IEEE Transactions on Information Theory*, 42(2):348–363, 1996. 176, 178

T. M. Cover and J. A. Thomas. *Elements of information theory*. Wiley-Interscience, 2006. 178

A. Cutkosky. *Algorithms and Lower Bounds for Parameter-free Online Learning*. PhD thesis, Stanford University, 2018. 59

A. Cutkosky. Anytime online-to-batch, optimism and acceleration. In K. Chaudhuri and R. Salakhutdinov, editors, *Proc. of the 36th International Conference on Machine Learning*, volume 97 of *Proc. of Machine Learning Research*, pages 1446–1454, Long Beach, California, USA, 09–15 Jun 2019a. PMLR. 33

A. Cutkosky. Combining online learning guarantees. In *Proc. of the Conference on Learning Theory (COLT)*, 2019b. 225

A. Cutkosky. Artificial constraints and hints for unbounded online learning. In A. Beygelzimer and D. Hsu, editors, *Conference on Learning Theory, COLT 2019, 25-28 June 2019, Phoenix, AZ, USA*, volume 99 of *Proceedings of Machine Learning Research*, pages 874–894. PMLR, 2019c. 97

A. Cutkosky. Parameter-free, dynamic, and strongly-adaptive online learning. In *International Conference on Machine Learning*, pages 2250–2259. PMLR, 2020a. 204

A. Cutkosky. Better full-matrix regret via parameter-free online learning. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 8836–8846. Curran Associates, Inc., 2020b. 49

A. Cutkosky and F. Orabona. Black-box reductions for parameter-free online learning in Banach spaces. In *Proc. of the Conference on Learning Theory (COLT)*, 2018. 96, 204, 205, 225, 226

A. Cutkosky, H. Mehta, and F. Orabona. Optimal stochastic non-smooth non-convex optimization through online-to-non-convex conversion. In *International Conference on Machine Learning*, pages 6643–6670. PMLR, 2023. 276, 288

A. Daniely, A. Gonen, and S. Shalev-Shwartz. Strongly adaptive online learning. In *International Conference on Machine Learning*, pages 1405–1411. PMLR, 2015. 243

C. Daskalakis and I. Panageas. Last-iterate convergence: Zero-sum games and constrained min-max optimization. In *10th Innovations in Theoretical Computer Science (ITCS) conference*, 2019. 269

C. Daskalakis, A. Deckelbaum, and A. Kim. Near-optimal no-regret algorithms for zero-sum games. In *Proceedings of the twenty-second annual ACM-SIAM symposium on Discrete Algorithms*, pages 235–254. SIAM, 2011. 268

C. Daskalakis, A. Ilyas, V. Syrgkanis, and H. Zeng. Training GANs with optimism. In *International Conference on Learning Representations*, 2018. 269

S. de Rooij, T. van Erven, P. D. Grünwald, and W. M. Koolen. Follow the leader if you can, hedge if you must. *Journal of Machine Learning Research*, 15(37):1281–1316, 2014. 138

G. Denevi, M. Pontil, and D. Stamos. Online parameter-free learning of multiple low variance tasks. In *Conference on Uncertainty in Artificial Intelligence*, pages 889–898. PMLR, 2020. 226

C. Ding, D. Zhou, X. He, and H. Zha. r_1-PCA: rotational invariant l_1-norm principal component analysis for robust subspace factorization. In *Proceedings of the 23rd international conference on Machine learning*, pages 281–288, 2006. 138

R. D'Orazio. Regret minimization with function approximation in extensive-form games. Master's thesis, University of Alberta, 2020. 205

J. Duchi, E. Hazan, and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. In *COLT*, 2010. 48

G. Farina, C. Kroer, and T. Sandholm. Online convex optimization for sequential decision processes and extensive-form games. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 1917–1925, 2019a. 268

G. Farina, C. Kroer, and T. Sandholm. Regret circuits: Composability of regret minimizers. In *International conference on machine learning*, pages 1863–1872. PMLR, 2019b. 204

G. Farina, C. Kroer, and T. Sandholm. Faster game solving via predictive Blackwell approachability: Connecting regret matching and mirror descent. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 5363–5371, 2021. 205

G. E. Flaspohler, F. Orabona, J. Cohen, S. Mouatadid, M. Oprescu, P. Orenstein, and L. Mackey. Online learning with optimism and delay. In M. Meila and T. Zhang, editors, *Proc. of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 3363–3373. PMLR, 18–24 Jul 2021. 97, 139, 205

D. J. Foster, S. Kale, M. Mohri, and K. Sridharan. Parameter-free online learning via model selection. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 6020–6030, 2017. 96, 97

D. J. Foster, S. Kale, H. Luo, M. Mohri, and K. Sridharan. Logistic regression: The importance of being improper. In *Conference on learning theory*, pages 167–208. PMLR, 2018. 191

M. Frank and P. Wolfe. An algorithm for quadratic programming. *Naval research logistics quarterly*, 3 (1-2):95–110, 1956. 267

Y. Freund. Predicting a binary sequence almost as well as the optimal biased coin. In *Proceedings of the ninth annual conference on Computational learning theory*, pages 89–98, 1996. 191

Y. Freund. Predicting a binary sequence almost as well as the optimal biased coin. *Information and Computation*, 182(2):73–94, 2003. 191

Y. Freund and R. E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. In *Computational Learning Theory: Second European Conference, EuroCOLT'95 Barcelona, Spain, March 13–15*, pages 23–37. Springer, 1995. 96, 260, 268

Y. Freund and R. E. Schapire. Game theory, on-line prediction and boosting. In *Proceedings of the Ninth Annual Conference on Computational Learning Theory*, COLT '96, pages 325—-332, New York, NY, USA, 1996. Association for Computing Machinery. 267, 268

Y. Freund and R. E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of computer and system sciences*, 55(1):119–139, 1997. 96, 260, 268

Y. Freund and R. E. Schapire. Large margin classification using the Perceptron algorithm. In *Proceedings of the Eleventh Annual Conference on Computational Learning Theory*, COLT' 98, pages 209—-217, New York, NY, USA, 1998. Association for Computing Machinery. 150

Y. Freund and R. E. Schapire. Large margin classification using the Perceptron algorithm. *Machine Learning*, pages 277–296, 1999a. 150

Y. Freund and R. E. Schapire. Adaptive game playing using multiplicative weights. *Games and Economic Behavior*, 29:79–103, 1999b. 267

Y. Freund, R. E. Schapire, Y. Singer, and M. K. Warmuth. Using and combining predictors that specialize. In *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, pages 334–343, 1997. 205

D. Fudenberg and D. K. Levine. Consistency and cautious fictitious play. *Journal of Economic Dynamics and Control*, 19:1065—-1089, 1995. 96

P. Gács. Uniform test of algorithmic randomness over a general space. *Theoretical Computer Science*, 341 (1-3):91–137, 2005. 288

P. Gaillard, G. Stoltz, and T. Van Erven. A second-order bound with excess losses. In *Conference on Learning Theory*, pages 176–196. PMLR, 2014. 139, 205

A. Garivier and O. Cappé. The KL-UCB algorithm for bounded stochastic bandits and beyond. In *Proceedings of the Conference on Learning Theory (COLT)*, pages 359–376, 2011. 283

C. Gentile. The robustness of the p-norm algorithms. *Machine Learning*, 53(3):265–299, 2003. 96, 150

C. Gentile and N. Littlestone. The robustness of the p-norm algorithms. In *Proc. of the Twelfth Annual Conference on Computational Learning Theory*, COLT '99, pages 1–11, New York, NY, USA, 1999. ACM. 96, 150

G. Gidel, H. Berard, G. Vignoud, P. Vincent, and S. Lacoste-Julien. A variational inequality perspective on generative adversarial networks. In *International Conference on Learning Representations*, 2019. 269

A. A. Goldstein. Optimization of Lipschitz continuous functions. *Mathematical Programming*, 13(1): 14–22, 1977. 288

G. J. Gordon. *Approximate solutions to Markov decision processes*. PhD thesis, Carnegie Mellon University, 1999a. 23, 136, 137

G. J. Gordon. Regret bounds for prediction problems. In *Proc. of the twelfth annual conference on Computational learning theory (COLT)*, pages 29–40, 1999b. 23, 136, 137

A. J. Grove, N. Littlestone, and D. Schuurmans. General convergence results for linear discriminant updates. In *Proc. of the 10th Annual Conference on Computational Learning Theory*, pages 171–183, 1997. 96

A. J. Grove, N. Littlestone, and D. Schuurmans. General convergence results for linear discriminant updates. *Machine Learning*, 43(3):173–210, 2001. 96

U. Haagerup. The best constants in the Khintchine inequality. *Studia Mathematica*, 70(3):231–283, 1981. 296

J. Hannan. Approximation to Bayes risk in repeated play. In *Contributions to the Theory of Games, Volume III*, pages 97–140. Princeton University Press, 1957. 7, 137

S. Hart and A. Mas-Colell. A simple adaptive procedure leading to correlated equilibrium. *Econometrica*, 68(5):1127–1150, 2000. 204, 267

E. Hazan and S. Kale. Extracting certainty from uncertainty: Regret bounded by variation in costs. In *Proc. of the 21st Conference on Learning Theory*, 2008. 96, 136

E. Hazan and S. Kale. Extracting certainty from uncertainty: Regret bounded by variation in costs. *Machine learning*, 80(2):165–188, 2010. 96

E. Hazan and C. Seshadhri. Adaptive algorithms for online decision problems. In *Electronic colloquium on computational complexity (ECCC)*, volume 14, 2007. 243

E. Hazan and C. Seshadhri. Efficient learning algorithms for changing environments. In *Proceedings of the 26th annual international conference on machine learning*, pages 393–400, 2009. 243

E. Hazan, A. Kalai, S. Kale, and A. Agarwal. Logarithmic regret algorithms for online convex optimization. In *International Conference on Computational Learning Theory*, pages 499–513. Springer, 2006. 48, 138, 139, 178, 191

E. Hazan, A. Agarwal, and S. Kale. Logarithmic regret algorithms for online convex optimization. *Machine Learning*, 69(2-3):169–192, 2007. 48, 138, 139, 178, 191

E. Hazan, A. Rakhlin, and P. L. Bartlett. Adaptive online gradient descent. In *Advances in Neural Information Processing Systems*, pages 65–72, 2008. 48

D. P. Helmbold, R. E. Schapire, Y. Singer, and M. K. Warmuth. On-line portfolio selection using multiplicative updates. *Mathematical Finance*, 8(4):325–347, 1998. 178

H. Hendriks. Test martingales for bounded random variables. *arXiv preprint arXiv:2109.08923*, 2018. 289

M. Herbster and M. K. Warmuth. Tracking the best expert. *Machine learning*, 32(2):151–178, 1998. 242

M. Herbster and M. K. Warmuth. Tracking the best linear predictor. *Journal of Machine Learning Research*, 1(281-309):10–1162, 2001. 242, 243

Jean-Baptiste Hiriart-Urruty and Claude Lemaréchal. *Fundamentals of convex analysis*. Springer Science & Business Media, 2004. 291

P. Hitczenko and S. Kwapień. On the Rademacher series. In *Probability in Banach spaces, 9*, pages 31–36. Springer, 1994. 59

L. Hollom and J. Portier. Tight lower bounds for anti-concentration of Rademacher sums and Tomaszewski's counterpart problem. *arXiv preprint arXiv:2306.07811*, 2023. 59

A. Hoorfar and M. Hassani. Inequalities on the Lambert W function and hyperpower function. *J. Inequal. Pure and Appl. Math*, 9(2):5–9, 2008. 291

S. R. Howard, A. Ramdas, J. McAuliffe, and J. Sekhon. Time-uniform, nonparametric, nonasymptotic confidence sequences. *The Annals of Statistics*, 49(2):1055–1080, 2021. 285, 286

Y.-G. Hsieh, F. Iutzeler, J. Malick, and P. Mertikopoulos. On the convergence of single-call stochastic extra-gradient methods. *Advances in Neural Information Processing Systems*, 32, 2019. 268

Y.-G. Hsieh, K. Antonakopoulos, and P. Mertikopoulos. Adaptive learning in continuous games: Optimal regret bounds and convergence to Nash equilibrium. In *Conference on Learning Theory*, pages 2388–2422. PMLR, 2021. 269

A. Jacobsen and A. Cutkosky. Parameter-free mirror descent. In *Conference on Learning Theory*, pages 4160–4211. PMLR, 2022. 242, 243

A. Jadbabaie, A. Rakhlin, S. Shahrampour, and K. Sridharan. Online optimization: Competing with dynamic comparators. In *Proc. of the Eighteenth International Conference on Artificial Intelligence and Statistics*, pages 398–406. PMLR, 2015. 243

K. Jang, K.-S. Jun, I. Kuzborskij, and F. Orabona. Tighter PAC-Bayes bounds through coin-betting. In *The Thirty Sixth Annual Conference on Learning Theory*, pages 2240–2264. PMLR, 2023. 288, 289

Z. Ji and M. Telgarsky. The implicit bias of gradient descent on nonseparable data. In *Conference on Learning Theory*, 2019. 33

L. Jie, F. Orabona, M. Fornoni, B. Caputo, and N. Cesa-Bianchi. OM-2: An online multi-class multi-kernel learning algorithm. In *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition-Workshops*, pages 43–50. IEEE, 2010. 138

M. Jordan, G. Kornowski, T. Lin, O. Shamir, and M. Zampetakis. Deterministic nonsmooth nonconvex optimization. In *The Thirty Sixth Annual Conference on Learning Theory*, pages 4570–4597. PMLR, 2023. 288

P. Joulani, A. Gyorgy, and C. Szepesvári. Online learning under delayed feedback. In *International conference on machine learning*, pages 1453–1461. PMLR, 2013. 97

P. Joulani, A. Gyorgy, and C. Szepesvári. Delay-tolerant online convex optimization: Unified analysis and adaptive-gradient algorithms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016. 97

P. Joulani, A. György, and C. Szepesvári. A modular analysis of adaptive (non-)convex optimization: Optimism, composite objectives, and variational bounds. In *Proc. of the International Conference on Algorithmic Learning Theory (ALT)*, volume 76, pages 681–720, 2017. 97, 139

K.-S. Jun and F. Orabona. Parameter-free online convex optimization with sub-exponential noise. In *Proc. of the Conference on Learning Theory (COLT)*, 2019. 289

K.-S. Jun, F. Orabona, S. Wright, and R. Willett. Improved strongly adaptive online learning using coin betting. In *Artificial Intelligence and Statistics*, pages 943–951. PMLR, 2017a. 205, 243

K.-S. Jun, F. Orabona, S. Wright, and R. Willett. Online learning for changing environments using coin betting. *Electronic Journal of Statistics*, 11:5282—-5310, 2017b. 205, 243

S. Kakade, S. Shalev-Shwartz, and A. Tewari. On the duality of strong convexity and strong smoothness: Learning applications and matrix regularization. Technical report, TTIC, 2009. 96, 115, 116, 138, 287

S. M. Kakade, K. Sridharan, and A. Tewari. On the complexity of linear prediction: Risk bounds, margin bounds, and regularization. *Advances in neural information processing systems*, 21, 2008. 287, 288

A. T. Kalai and S. Vempala. Efficient algorithms for universal portfolios. *Journal of Machine Learning Research*, pages 423–440, 2002. 178

M. Kearns. Thoughts on hypothesis boosting. Machine Learning class project, December 1988. 268

M. Kearns and L. G. Valiant. Learning boolean formulae or finite automata is as hard as factoring. Technical Report TR-14-88, Harvard University Aikem Computation Laboratory, 1988. 268

J. L. Kelly, jr. A new interpretation of information rate. *IRE Transactions on Information Theory*, 2(3): 185–189, 1956. 178

J. Kivinen and M. Warmuth. Exponentiated gradient versus gradient descent for linear predictors. *Information and Computation*, 132(1):1–63, January 1997. 48, 96, 268

J. Kivinen and M. K. Warmuth. Averaging expert predictions. In *European Conference on Computational Learning Theory*, pages 153–167. Springer, 1999. 138, 190

R. Kleinberg, A. Niculescu-Mizil, and Y. Sharma. Regret bounds for sleeping experts and bandits. In *Proceedings of the 21st Annual Conference on Learning Theory (COLT)*, pages 421–432, 2008. 205

R. Kleinberg, A. Niculescu-Mizil, and Y. Sharma. Regret bounds for sleeping experts and bandits. *Machine learning*, 80(2):245–272, 2010. 205

W. M. Koolen and T. van Erven. Second-order quantile methods for experts and combinatorial games. In *Proc. of the Conference On Learning Theory*, pages 1155–1175, 2015. 97, 225

W. M. Koolen, T. van Erven, and P. Grünwald. Learning the learning rate for prediction with expert advice. In Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 27*, pages 2294–2302. Curran Associates, Inc., 2014. 243

G. M. Korpelevich. The extragradient method for finding saddle points and other problems. *Matecon*, 12: 747–756, 1976. 269

R. Krichevsky and V. Trofimov. The performance of universal encoding. *IEEE Trans. on Information Theory*, 27(2):199–207, 1981. 225, 289

C. Kroer. IEOR8100: Economics, AI, and optimization. lecture note 5: Computing Nash equilibrium via regret minimization. Technical report, Columbia University, 2020. 267, 268

B. Kulis and P. L. Bartlett. Implicit online learning. In *International Conference on Machine Learning*, pages 575–582, 2010. 268

I. Kuzborskij, K.-S. Jun, Y. Wu, K. Jang, and F. Orabona. Better-than-KL PAC-Bayes bounds. In *The Thirty Seventh Annual Conference on Learning Theory*, pages 3325–3352. PMLR, 2024. 288

S. Lacoste-Julien, M. Schmidt, and F. Bach. A simpler approach to obtaining an $O(1/t)$ convergence rate for the projected stochastic subgradient method. *arXiv preprint arXiv:1212.2002*, 2012. 48

T. L. Lai. Adaptive treatment allocation and the multi-armed bandit problem. *The Annals of Statistics*, pages 1091–1114, 1987. 168

T. L. Lai and H. Robbins. Asymptotically efficient adaptive allocation rules. *Advances in applied mathematics*, 6(1):4–22, 1985. 168

T. L. Lai and C. Z. Wei. Least squares estimates in stochastic regression models with applications to identification and control of dynamic systems. *The Annals of Statistics*, 10(1):154–166, 1982. 139

J. Langford and T. Zhang. The epoch-greedy algorithm for multi-armed bandits with side information. In *Advances in neural information processing systems*, pages 817–824, 2008. 168

T. Lattimore and C. Szepesvári. *Bandit algorithms*. Cambridge University Press, 2020. 168

M. Ledoux and M. Talagrand. *Probability in Banach Spaces: isoperimetry and processes*, volume 23. Springer, 1991. 287

C.-W. Lee, C. Kroer, and H. Luo. Last-iterate convergence in extensive-form games. In *Advances in Neural Information Processing Systems*, volume 34, pages 14293–14305, 2021. 269

W.-C. Lee and F. Orabona. A best-of-both-worlds proof for Tsallis-INF without Fenchel conjugates. *arXiv preprint arXiv:2511.11211*, 2025. 169

Q. Lei, S. G. Nagarajan, I. Panageas, and X. Wang. Last iterate convergence in no-regret learning: constrained min-max optimization for convex-concave landscapes. In *International Conference on Artificial Intelligence and Statistics*, pages 1441–1449. PMLR, 2021. 269

L. A. Levin. Uniform tests of randomness. *Doklady Akademii Nauk*, 227(1):33–35, 1976. English version available at `https://www.cs.bu.edu/fac/lnd/dvi/rnd76.pdf`. 288

T. Liang and J. Stokes. Interaction matters: A note on non-asymptotic local convergence of generative adversarial networks. In K. Chaudhuri and M. Sugiyama, editors, *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 907–915. PMLR, 16–18 Apr 2019. 269

N. Littlestone. Learning quickly when irrelevant attributes abound: a new linear-threshold algorithm. *Machine Learning*, 2(4):285–318, 1988. 150

N. Littlestone and M. K. Warmuth. The weighted majority algorithm. *Information and computation*, 108 (2):212–261, 1994. 96, 242

M. Liu and F. Orabona. On the initialization for convex-concave min-max problems. In S. Dasgupta and N. Haghtalab, editors, *Proc. of The 33rd International Conference on Algorithmic Learning Theory*, volume 167 of *Proceedings of Machine Learning Research*, pages 743–767. PMLR, 29 Mar–01 Apr 2022. 267

H. Lu, R. M. Freund, and Y. Nesterov. Relatively smooth convex optimization by first-order methods, and applications. *SIAM Journal on Optimization*, 28(1):333–354, 2018. 138

G. Lugosi and G. Neu. Online-to-PAC conversions: Generalization bounds via regret analysis. *arXiv preprint arXiv:2305.19674*, 2023. 288

H. Luo. CSCI 659: Introduction to online optimization/learning, lecture 4, 2022. 268

H. Luo and R. E. Schapire. A drifting-games analysis for online learning and applications to boosting. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014. 268

H. Luo and R. E. Schapire. Achieving all with no parameters: AdaNormalHedge. In *Proc. of the Conference on Learning Theory (COLT)*, pages 1286–1304, 2015. 205

Y. Malitsky. Projected reflected gradient methods for monotone variational inequalities. *SIAM Journal on Optimization*, 25(1):502–520, 2015. 97

P. Massart. Some applications of concentration inequalities to statistics. In *Annales de la Faculté des sciences de Toulouse: Mathématiques*, volume 9, pages 245–303, 2000. 288

A. Maurer and M. Pontil. Empirical Bernstein bounds and sample variance penalization. In *Proc. of the Conference on Learning Theory*, 2009. 284

D. A. McAllester. Some PAC-Bayesian theorems. In *Proceedings of the eleventh annual conference on Computational learning theory*, pages 230–234, 1998. 288

H. B. McMahan. Follow-the-regularized-leader and mirror descent: Equivalence theorems and L1 regularization. In *Proc. of the Fourteenth International Conference on Artificial Intelligence and Statistics, AISTATS*, pages 525–533, 2011. 138

H. B. McMahan. A survey of algorithms and analysis for adaptive online learning. *The Journal of Machine Learning Research*, 18(1):3117–3166, 2017. 137

H. B. McMahan and J. Abernethy. Minimax optimal algorithms for unconstrained linear optimization. In C. J. C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 26*, pages 2724–2732. Curran Associates, Inc., 2013. 59

H. B. McMahan and F. Orabona. Unconstrained online linear learning in Hilbert spaces: Minimax algorithms and normal approximations. In *Proc of the Annual Conference on Learning Theory, COLT*, 2014. 59, 225

H. B. McMahan and M. Streeter. Delay-tolerant algorithms for asynchronous distributed online learning. In *Advances in neural information processing systems*, volume 27, 2014. 97

H. B. McMahan and M. J. Streeter. Adaptive bound optimization for online convex optimization. In *COLT*, 2010. 48, 138

H. B. McMahan, G. Holt, D. Sculley, M. Young, D. Ebner, J. Grady, L. Nie, T. Phillips, E. Davydov, D. Golovin, S. Chikkerur, D. Liu, M. Wattenberg, A. M. Hrafnkelsson, T. Boulos, and J. Kubica. Ad click prediction: a view from the trenches. In *Proc. of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1222–1230. ACM, 2013. 138

N. Megiddo. On repeated games with incomplete information played by non-Bayesian players. *International Journal of Game Theory*, 9:157—167, 1980. 168

R. Meir and T. Zhang. Generalization error bounds for Bayesian mixture algorithms. *Journal of Machine Learning Research*, 4(Oct):839–860, 2003. 287

Z. Mhammedi and K. Gatmiry. Quasi-Newton steps for efficient online exp-concave optimization. In *The Thirty Sixth Annual Conference on Learning Theory*, pages 4473–4503. PMLR, 2023. 204

Z. Mhammedi and W. M Koolen. Lipschitz and comparator-norm adaptivity in online learning. In *Conference on Learning Theory*, pages 2858–2887. PMLR, 2020. 225

J. Milnor. Games against nature. Technical Report RM-679, RAND PROJECT AIR FORCE, 1951. 6

M. Mohri and S. Yang. Accelerating online convex optimization via adaptive prediction. In *Artificial Intelligence and Statistics*, pages 848–856. PMLR, 2016. 139

A. Mokhtari, A. Ozdaglar, and S. Pattathil. A unified analysis of extra-gradient and optimistic gradient methods for saddle point problems: Proximal point approach. In *International Conference on Artificial Intelligence and Statistics*, pages 1497–1507. PMLR, 2020. 268, 269

T. Nan, S. D. Gupta, G. Iyengar, and C. Kroer. On the $O(1/T)$ convergence of alternating gradient descent-ascent in bilinear games. *arXiv preprint arXiv:2510.03855*, 2025. 268

J. Negrea, B. Bilodeau, N. Campolongo, F. Orabona, and D. Roy. Minimax optimal quantile and semi-adversarial regret via root-logarithmic regularizers. In *Advances in Neural Information Processing Systems*, volume 34, 2021. 225

A. Nemirovski. Information-based complexity of convex programming. Technical report, Technion, 1995. Lecture notes. 62

A. S. Nemirovskij and D. Yudin. *Problem complexity and method efficiency in optimization*. Wiley, New York, NY, USA, 1983. 95

Y. Nesterov. *Introductory lectures on convex optimization: A basic course*, volume 87. Springer, 2004. 58

Y. Nesterov. Primal-dual subgradient methods for convex problems. *Mathematical programming*, 120(1): 221–259, 2009. Received: 29 September 2005 / Accepted: 13 January 2007 / Published online: 19 June 2007. 137

Y. Nesterov and V. Shikhman. Quasi-monotone subgradient methods for nonsmooth convex minimization. *Journal of Optimization Theory and Applications*, 165(3):917–940, 2015. 33

G. Neu and N. Zhivotovskiy. Fast rates for online prediction with abstention. In *Conference on Learning Theory*, pages 3030–3048. PMLR, 2020. 150

J. von Neumann. Zur theorie der gesellschaftsspiele. *Mathematische annalen*, 100(1):295–320, 1928. 267

J. von Neumann and O. Morgenstern. *Theory of games and economic behavior*. Princeton University Press, 1944. 267

A. B Novikoff. On convergence proofs for perceptrons. In *Proc. of the Symposium of the Mathematical Theory of Automata*, volume XII, pages 615–622. Wiley, New York, 1963. 149

F. Orabona. Dimension-free exponentiated gradient. In *Advances in Neural Information Processing Systems 26*, pages 1806–1814. Curran Associates, Inc., 2013. 59, 225

F. Orabona. Simultaneous model selection and optimization through parameter-free stochastic learning. In *Advances in Neural Information Processing Systems 27*, 2014. 97

F. Orabona. A modern introduction to online learning. *arXiv preprint arXiv:1912.13213*, 2019. Version 9. 226

F. Orabona. A note on how to remove the $\ln \ln t$ term from the Squint bound. *arXiv preprint arXiv:2604.26926*, 2026a. 226

F. Orabona. From betting to empirical Bernstein LIL. *arXiv preprint arXiv:2605.22124*, 2026b. 289

F. Orabona and K.-S. Jun. Tight concentrations and confidence sequences from the regret of universal portfolio. *IEEE Trans. Inf. Theory*, 70(1):436–455, 2024. 226, 285, 288

F. Orabona and D. Pál. Scale-free algorithms for online linear optimization. In *International Conference on Algorithmic Learning Theory*, pages 287–301. Springer, 2015. 49, 58, 138

F. Orabona and D. Pál. Optimal non-asymptotic lower bound on the minimax regret of learning with expert advice. *arXiv preprint arXiv:1511.02176*, 2015. 59

F. Orabona and D. Pál. Coin betting and parameter-free online learning. In D. D. Lee, M. Sugiyama, U. V. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems 29*, pages 577–585. Curran Associates, Inc., 2016. 59, 225, 226, 292

F. Orabona and D. Pál. Scale-free online learning. *Theoretical Computer Science*, 716:50–69, 2018. Special Issue on ALT 2015. 49, 58, 59, 138

F. Orabona and D. Pál. Parameter-free stochastic optimization of variationally coherent functions. *arXiv preprint arXiv:2102.00236*, 2021. 225, 226

F. Orabona and T. Tommasi. Training deep networks without learning rates through coin betting. In *Advances in Neural Information Processing Systems*, pages 2160–2170, 2017. 225

F. Orabona, L. Jie, and B. Caputo. Online-batch strongly convex multi kernel learning. In *2010 IEEE computer society conference on computer vision and pattern recognition*, pages 787–794. IEEE, 2010. 138

F. Orabona, N. Cesa-Bianchi, and C. Gentile. Beyond logarithmic bounds in online learning. In N. D. Lawrence and M. Girolami, editors, *Proc. of the 15th International Conference on Artificial Intelligence and Statistics, AISTATS, La Palma, Canary Islands, April 21-23, 2012*, volume 22 of *JMLR Proceedings*, pages 823–831. JMLR.org, 2012a. 139

F. Orabona, L. Jie, and B. Caputo. Multi kernel learning with online-batch optimization. *Journal of Machine Learning Research*, 13(2), 2012b. 138

F. Orabona, K. Crammer, and N. Cesa-Bianchi. A generalized online mirror descent with applications to classification and regression. *Machine Learning*, 99:411–435, 2015. 137, 138, 139

A. Picard-Weibel, E. Clerico, R. Moscoviz, and B. Guedj. How good is PAC-Bayes at explaining generalisation? *arXiv preprint arXiv:2503.08231*, 2025. 288

B. Polyak. Existence theorems and convergence of minimizing sequences in extremum problems with restrictions. *Dokl. Akad. Nauk SSSR*, 166(2):287—290, 1966. 48

L. D. Popov. A modification of the Arrow-Hurwicz method for search of saddle points. *Mathematical notes of the Academy of Sciences of the USSR*, 28(5):845–848, 1980. 97, 268, 269

K. Quanrud and D. Khashabi. Online learning with adversarial delays. In *Advances in neural information processing systems*, volume 28, 2015. 97

A. Rakhlin and K. Sridharan. Optimization, learning, and games with predictable sequences. In *Advances in Neural Information Processing Systems*, pages 3066–3074, 2013a. 268

A. Rakhlin and K. Sridharan. Online learning with predictable sequences. In *Proc. of the Conference on Learning Theory (COLT)*, volume 30, pages 993–1019, 2013b. 97, 139, 268

A. Rakhlin and K. Sridharan. On equivalence of martingale tail bounds and deterministic regret inequalities. In *Proc. of the Conference On Learning Theory (COLT)*, pages 1704–1722, 2017. 289

S. J. Reddi, S. Kale, and S. Kumar. On the convergence of Adam and beyond. In *International Conference on Learning Representations*, 2018. 9

H. Robbins. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58(5):527–535, 1952. 168

H. Robbins. A remark on Stirling's formula. *The American mathematical monthly*, 62(1):26–29, 1955. 298

H. Robbins and D. Siegmund. The expected sample size of some tests of power one. *The Annals of Statistics*, 2(3):415–436, 1974. 289

R. T. Rockafellar. *Convex functions and dual extremum problems*. PhD thesis, Harvard University, 1963. 22

R. T. Rockafellar. *Convex Analysis*. Princeton University Press, 1970. 10, 11, 17, 69, 95, 267

F. Rosenblatt. The Perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65:386–407, 1958. 149

E. K. Ryu and S. Boyd. Stochastic proximal iteration: a non-asymptotic improvement upon stochastic gradient descent. Technical report, Stanford, 2014. 138

J. J. Ryu and A. Bhatt. On confidence sequences for bounded random processes via universal gambling strategies. *IEEE Transactions on Information Theory*, 2024. 288

S. Sachs, H. Hadiji, T. van Erven, and C. Guzmán. Between stochastic and adversarial online convex optimization: Improved regret bounds via smoothness. In *Advances in Neural Information Processing Systems*, 2022. 139

S. Sakaue, H. Bao, T. Tsuchiya, and T. Oki. Online structured prediction with fenchel–young losses and improved surrogate regret for online multiclass classification with logistic loss. In *The Thirty Seventh Annual Conference on Learning Theory*, pages 4458–4486. PMLR, 2024. 150

S. Sakaue, H. Bao, and Y. Cao. Non-stationary online structured prediction with surrogate losses. *arXiv preprint arXiv:2510.07086*, 2025. 150

L. J. Savage. The theory of statistical decision. *Journal of the American Statistical Association*, 46(253): 55–67, 1951. 6, 7

L. J. Savage. *The foundations of statistics*. Wiley, 1954. 6, 7

R. E. Schapire. The strength of weak learnability. *Machine learning*, 5(2):197–227, 1990. 268

C.-P. Schnorr. A unified approach to the definition of random sequences. *Mathematical systems theory*, 5 (3):246–258, 1971. 288

V. V. Semenov. A version of the mirror descent method to solve variational inequalities. *Cybernetics and Systems Analysis*, 53(2):234–243, 2017. 269

G. Shafer and V. Vovk. *Probability and finance: it's only a game!* John Wiley & Sons, 2001. 289

G. Shafer, A. Shen, N. Vereshchagin, and V. Vovk. Test martingales, Bayes factors and p-values. *Statistical Science*, 26(1):84–101, 2011. 289

S. Shalev-Shwartz. *Online Learning: Theory, Algorithms, and Applications*. PhD thesis, The Hebrew University, 2007. 96, 137

S. Shalev-Shwartz and Y. Singer. Online learning meets optimization in the dual. In *International Conference on Computational Learning Theory*, pages 423–437. Springer, 2006a. 137

S. Shalev-Shwartz and Y. Singer. Convex repeated games and Fenchel duality. In *Advances in neural information processing systems*, pages 1265–1272, 2006b. 23, 137

S. Shalev-Shwartz and Y. Singer. Logarithmic regret algorithms for strongly convex repeated games. Technical report, The Hebrew University, 2007. 138

S. Shalev-Shwartz, Y. Singer, and N. Srebro. Pegasos: Primal Estimated sub-GrAdient SOlver for SVM. In *Proc. of the International Conference on Machine Learning*, pages 807–814, 2007. 48

L. Sharrock and C. Nemeth. Coin sampling: Gradient-based Bayesian inference without learning rates. In *International Conference on Machine Learning*, 2023. 226

L. Sharrock, D. Dodd, and C. Nemeth. CoinEM: Tuning-free particle-based variational inference for latent variable models. *arXiv preprint arXiv:2305.14916*, 2023a. 226

L. Sharrock, L. Mackey, and C. Nemeth. Learning rate free Bayesian inference in constrained domains. *arXiv preprint arXiv:2305.14943*, 2023b. 226

N. Shor. *On the structure of algorithms for the numerical solution of optimal planning and design problems (in Russian)*. Phd, Cybernetics Institute, Academy of Sciences of the Ukrainian SSR, Kiev, 1964. 22

Y. M. Shtar'kov. Universal sequential coding of single messages. *Problemy Peredachi Informatsii*, 23(3): 3–17, 1987. 225

M. Sion. On general minimax theorems. *Pacific Journal of mathematics*, 8(1):171–176, 1958. 247, 267

A. Soleymani, G. Piliouras, and G. Farina. Cautious optimism: A meta-algorithm for near-constant regret in general games. In *Proceedings of the 26th ACM Conference on Economics and Computation*, EC '25, page 870, New York, NY, USA, 2025. Association for Computing Machinery. 268

J. Steinhardt and P. Liang. Adaptivity and optimism: An improved exponentiated gradient algorithm. In *Proc. of the International Conference on Machine Learning (ICML)*, pages 1593–1601, 2014. 96, 139

G. Stoltz. *Incomplete information and internal regret in prediction of individual sequences*. PhD thesis, Université Paris Sud-Paris XI, 2005. 168

M. Streeter and B. McMahan. No-regret algorithms for unconstrained online convex optimization. In *Advances in Neural Information Processing Systems 25*, pages 2402–2410. Curran Associates, Inc., 2012. 59, 225

M. Streeter and H. B. McMahan. Less regret via online conditioning. *arXiv preprint arXiv:1002.4862*, 2010. 48

O. Tammelin, N. Burch, M. Johanson, and M. Bowling. Solving heads-up limit Texas hold'em. In *Twenty-fourth international joint conference on artificial intelligence*, 2015. 204, 267

C.-E. Tsai, H.-C. Cheng, and Y.-H. Li. Online self-concordant and relatively smooth minimization, with applications to online portfolio selection and learning quantum states. In *International Conference on Algorithmic Learning Theory*, pages 1481–1483. PMLR, 2023. 178

D. van der Hoeven. Exploiting the surrogate gap in online multiclass classification. *Advances in Neural Information Processing Systems*, 33:9562–9572, 2020. 150

D. van der Hoeven and T. van Erven. Is mirror descent a special case of exponential weights? Master's thesis, Leiden University, 2016. 191

D. van der Hoeven, A. Cutkosky, and H. Luo. Comparator-adaptive convex bandits. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 19795–19804. Curran Associates, Inc., 2020. 224

T. van Erven. PAC-Bayes mini-tutorial: A continuous union bound. *arXiv preprint arXiv:1405.1580*, 2014. 288

T. van Erven and W. M. Koolen. MetaGrad: Multiple learning rates in online learning. In D. D. Lee, M. Sugiyama, U. V. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems 29*, pages 3666–3674. Curran Associates, Inc., 2016. 97, 243

T. van Erven, W. M. Koolen, S. D. Rooij, and P. Grünwald. Adaptive Hedge. In *Advances in Neural Information Processing Systems*, pages 1656–1664, 2011. 138

T. van Erven, W. M. Koolen, and D. van der Hoeven. MetaGrad: Adaptation using multiple learning rates in online learning. *Journal of Machine Learning Research*, 22(161):1–61, 2021. 97, 243

S. Vansover-Hager, T. Koren, and R. Livni. Rapid overfitting of multi-pass stochastic gradient descent in stochastic convex optimization. *arXiv preprint arXiv:2505.08306*, 2025. 29

J. Ville. *Étude critique de la notion de collectif*. Gauthier-Villars, Paris, 1939. 288

V. Voráček and F. Orabona. STaR-Bets: Sequential target-recalculating bets for tighter confidence intervals. In *Advances in Neural Information Processing Systems*, 2025. 288

V. Vovk. A game of prediction with expert advice. *Journal of Computer and System Sciences*, 56(2): 153–173, 1998. 190, 191

V. Vovk. Competitive on-line statistics. *International Statistical Review*, 69(2):213–248, 2001. 139, 191

V. Vovk. On-line regression competitive with Reproducing Kernel Hilbert Spaces. In J.-Y. Cai, S. B. Cooper, and A. Li, editors, *Theory and Applications of Models of Computation*, volume 3959 of *Lecture Notes in Computer Science*, pages 452–463. Springer Berlin Heidelberg, 2006. 225

V. Vovk. Hoeffding's inequality in game-theoretic probability. *arXiv preprint arXiv:0708.2502*, 2007. 289

V. Vovk and C. Watkins. Universal portfolio selection. In *Proc. of the Conference on Computational Learning Theory*, pages 12–23, 1998. 178

V. G. Vovk. Aggregating strategies. *Proc. of Computational Learning Theory, 1990*, pages 371–386, 1990. 96, 178, 190

V. G. Vovk. A logic of probability, with application to the foundations of statistics. *Journal of the Royal Statistical Society. Series B (Methodological)*, 55(2):317–351, 1993. 289

A. Wald. Sequential tests of statistical hypotheses. *The annals of mathematical statistics*, 16(2):117–186, 1945. 289

A. Wald. *Statistical decision functions*. Wiley, 1950. 6

J.-K. Wang, J. Abernethy, and K. Y. Levy. No-regret dynamics in the fenchel game: A unified framework for algorithmic convex optimization. *arXiv preprint arXiv:2111.11309*, 2021. 268

M. K. Warmuth and A. K. Jagota. Continuous and discrete-time nonlinear gradient descent: Relative loss bounds and convergence. In *Electronic proceedings of the 5th International Symposium on Artificial Intelligence and Mathematics*, volume 326, 1997. 95

I. Waudby-Smith and A. Ramdas. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86(1):1–27, 2024. 289

C.-Y. Wei, C.-W. Lee, M. Zhang, and H. Luo. Linear last-iterate convergence in constrained saddle-point optimization. In *International Conference on Learning Representations*, 2021. 269

M. J. Weinberger and E. Ordentlich. On delayed prediction of individual sequences. *IEEE Transactions on Information Theory*, 48(7):1959–1976, 2002. 97

J. G. Wendel. Note on the gamma function. *The American Mathematical Monthly*, 55(9):563–564, 1948. 294

L. Xiao. Dual averaging method for regularized stochastic learning and online optimization. In Y. Bengio, D. Schuurmans, J. Lafferty, C. Williams, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 22, pages 2116–2124. Curran Associates, Inc., 2009. 138

L. Xiao. Dual averaging methods for regularized stochastic learning and online optimization. *Journal of Machine Learning Research*, 11:2543–2596, 2010. 138

E. Zermelo. Über eine anwendung der mengenlehre auf die theorie des schachspiels. In *Proceedings of the fifth international congress of mathematicians*, volume 2, pages 501–504. Cambridge University Press Cambridge, 1913. 257

G. Zhang, Y. Wang, L. Lessard, and R. Grosse. Near-optimal local convergence of alternating gradient descent-ascent for minimax optimization. *arXiv preprint arXiv:2102.09468*, 2021. 268

J. Zhang, H. Lin, S. Jegelka, S. Sra, and A. Jadbabaie. Complexity of finding stationary points of nonconvex nonsmooth functions. In *International Conference on Machine Learning*, pages 11173–11182. PMLR, 2020. 288

L. Zhang, S. Lu, and Z.-H. Zhou. Adaptive online learning in dynamic environments. *Advances in neural information processing systems*, 31, 2018. 242

Q. Zhang and A. Cutkosky. Random scaling and momentum for non-smooth non-convex optimization. *arXiv preprint arXiv:2405.09742*, 2024. 288

T. Zhang. Solving large scale linear prediction problems using stochastic gradient descent algorithms. In *Proc. of International Conference on Machine Learning*, pages 919–926, New York, NY, USA, 2004. ACM. 23, 33, 48

Z. Zhang, A. Cutkosky, and I. Paschalidis. PDE-based optimal strategy for unconstrained online learning. In *International Conference on Machine Learning*, pages 26085–26115. PMLR, 2022. 59, 225

P. Zhao, Y.-J. Zhang, L. Zhang, and Z.-H. Zhou. Dynamic regret of convex and smooth functions. *Advances in Neural Information Processing Systems*, 33:12510–12520, 2020. 243

P. Zhao, Y.-J. Zhang, L. Zhang, and Z.-H. Zhou. Adaptivity and non-stationarity: Problem-dependent dynamic regret for online convex optimization. *Journal of Machine Learning Research*, 25(98):1–52, 2024. 243

F. Zhdanov and Y. Kalnishkan. An identity for kernel ridge regression. In M. Hutter, F. Stephan, V. Vovk, and T. Zeugmann, editors, *Algorithmic Learning Theory*, pages 405–419, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg. 140

F. Zhdanov and Y. Kalnishkan. An identity for kernel ridge regression. *Theor. Comput. Sci.*, 473:157–178, February 2013. 140

J. Zimmert and Y. Seldin. An optimal algorithm for stochastic and adversarial bandits. In K. Chaudhuri and M. Sugiyama, editors, *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 467–475. PMLR, 2019. 138, 169

J. Zimmert and Y. Seldin. Tsallis-INF: An optimal algorithm for stochastic and adversarial bandits. *Journal of Machine Learning Research*, 22(28):1–49, 2021. 138, 169

M. Zinkevich. Online convex programming and generalized infinitesimal gradient ascent. In *Proc. of the International Conference on Machine Learning*, pages 928–936, 2003. 23, 137, 242

M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione. Regret minimization in games with incomplete information. In *Advances in neural information processing systems*, volume 20, pages 1729–1736, 2007. 251

M. Zinkevich, J. Langford, and A. Smola. Slow learners are fast. *Advances in neural information processing systems*, 22, 2009. 97

Index

$\boldsymbol{A}$-restricted strong convexity, 138
AA, *see* Aggregating Algorithm
AdaGrad algorithm, **45–48**, 48–49, 97, 138
AdaHedge algorithm, **110–115**, 138
adaptive regret, **234**
ADER algorithm, **230–232**, 233, 242–243
adversary, 3
Aggregating Algorithm, **186–190**, 190, 191
algorithm
  coordinate-wise scale-free, **47**, 49
  improper, **129**
  scale-free, **47**, 111, 205, 225
  self-confident, 48
alternation, **251–253**, 267–268

barycentric (δ, ϵ)-stationary point, **276**
Be-the-Leader
  algorithm, **266**
  lemma, **4**, 5–7, 137
Be-the-Regularized-Leader algorithm, **266–267**
best response, **251**, 259, 260, 265
binomial coefficient, 296, 298
black-box reduction, **192–205**
  for constrained online convex optimization, **192–201**, 204
  for dynamic regret in unbounded feasible sets, **232–233**, 243
  for sleeping experts, **201–203**, 205
  to magnitude and direction, **203–204**, 205
boosting, **258–260**, 267, 268
Bregman divergence, **63–64**, 65–67, 74, 76, 77, 92, 94–98, 106–108, 138, 228, 230
  generalized, **106**, 137
buy-and-hold strategy, 171

CBCE algorithm, **234–240**
coin-betting game, 53–56, **206–211**
comparator, *see also* competitor, **2**
competitor, *see also* comparator, **2**
composite loss, **117–119**, 126, 127, 138
confidence sequence, 281
constantly rebalanced portfolio, **170**, 170–172, 175
continuous coin, 207, **210**, 211, 225
contraction lemma, **271**, 287
convex hull, **18**, 280
convex set, **9**
Counterfactual Regret Minimization algorithm, 251, 267
cutting plane method, 61

delayed feedback, **94–95**, 97
digamma function, **294**, 294
distance generating function, **64**, 79, 81, 82, 98, 106, 109, 242, 243, 269
distance to a set, **193**, 193
domain, **9**
Donsker–Varadhan variational formula, **181**, 191
doubling trick, 98, 221, 243
Dual Averaging algorithm, 137
dual map, 106
duality gap, **247**, 247, 249–251, 260, 263, 268
duality map, *see also* mirror map, **75**, 76, 104
duality pairing, **179**
duality strong convexity-smoothness, **73–74**, 75, 96, 104, 116

empirical risk, **31**
  minimization, **31**
  minimizer of, 31
entropy
  negative Shannon, 64, 78, 110, 112, 155
  negative weighted, 80
  Shannon, 174
  Tsallis, **155**, 156, 168
  unnormalized negative, 152
epigraph, **10**
ϵ-insensitive loss, 23
excess risk, **30**
Exp3 algorithm, **154–155**, 168
exploration-exploitation trade-off, 151
Explore-then-Commit algorithm, **159–161**
Exponentiated Gradient algorithm
  FTRL version, **109–110**
  OMD version, **78–82**, 85, 96, 98, 171

feasibility problem, 146
feasible set, *see also* valid set, **2**
Fenchel conjugate, **68–74**
first-order optimality condition, **74**
Follow-the-Leader algorithm, **3**, 11, 49
Follow-the-Regularized-Leader algorithm, **99–140**
  optimistic, **129–136**, 139
  regret equality, **100–101**

with linear losses, **103–107**
with local norms, **107–110**
Forward Algorithm, 97
Frank–Wolfe algorithm, 267
FTRL, *see* Follow-the-Regularized-Leader algorithm
FTRL-Proximal algorithm, 103, 138
function
absolute symmetric, **115**
affine, 10, 18
closed, **16**
convex, **10**, 17
exp-concave, **123–126**, 126, 138–140, 177, 181–184, 186, 187, 189, 197, 208, 226
extended-real-valued, **9**, 9, 10, 138, 179, 291
Legendre, 95
Lipschitz, **19**, 19
lower semicontinuous, 16
mixable, **186–187**, 187, 188, 191
proper, **16**, 16, 17, 69
self-bounded, **43**, 43–45, 48, 49, 147–149
smooth, **42–43**, 44, 45, 49, 73, 116, 134, 136, 138, 139, 261, 276, 280
strictly convex, **63**, 63, 66, 76, 108, 139
strongly convex, **34–36**, 37–39, 48, 49, 64, 66, 73, 80, 82, 93, 101, 102, 119–121, 131, 138, 185, 209, 211, 261
subdifferentiable, **17**, 17
well-behaved, **276**, 277
fundamental theorem of calculus for extended-real-valued convex functions, 36, **291**

game
extensive form, 256–257
two-player zero-sum, 253–256
gamma function, 172, 210, **294–296**
gap map, **147**
Gaptron algorithm, **146–149**, 150
generalization gap, 273
geometric covering, **236–238**, 239
guessing game, **1**, 3–6, 15, 20, 38

Hausdorff space, 16, **291**
Hedge algorithm, 96
hinge loss, **16**, 25, 30, 150
at margin γ, 150
conjugate, 71
powers, 143
smoothed, 149
squared, 45, 149
subgradients of, 18
Huber loss, 15
hypothesis class, **29**, 270
sample complexity of, 30

Implicitly Normalized Forecaster, **155**, 168
importance-weighted estimator, 151
indicator function, **9**, 16
subgradients of, 18
inequality
Cauchy–Schwarz, **7**, 19, 41, 47, 168, 279
Chebyshev's, 158
Fenchel–Young's, **69**, 70, 72, 78, 98, 105, 106, 108, 220, 262
Hoeffding–Azuma, **27**, 274, 282
Jensen's, **11**, 25, 31, 80, 149, 182, 186, 220, 231, 249, 252, 279, 283, 295
Khintchine's, 51, **296**
Markov's, **159**
Ville's, **281**, 281, 288
Wendel's, **294**, 294
Young's, **71–72**, 145

Karush–Kuhn–Tucker conditions, 79, 157
KL, *see* Kullback–Leibler divergence
Krichevsky–Trofimov
bettor, **207–211**
online convex optimization algorithm, **213–216**, 222
KT, *see* Krichevsky–Trofimov
Kullback–Leibler divergence, 174, 209, 211, 218, 275, 283, 288
between Bernoulli distributions, **54**, 283, 297
generalized, **64**

$\mathscr{L}^\star$ bound, 44–45, 48, 140
Lambert function, 57, 213, **291–293**
law of iterated logarithm, 289
LEA, *see* learning with expert advice
learning rate, *see also* stepsize, 12
adaptive, 39–40, 46, 48–49
learning with expert advice, **84–86**, 88, 96
linear classifier, 141
Littlestone dimension, **150**
logistic loss, **26**, 30, 49, 125, 191
multiclass, 187
lower bound
bounded online linear optimization, **50–51**, 58
unconstrained online linear optimization, **53–58**, 59
unconstrained online subgradient descent, **51–53**, 58–59

margin
in boosting, **260**
in linear classification, **144**
market vector, **170**, 170, 171
Kelly, 175
martingale, **27**, 27, 28, 274, 281, 282, 288, 289
Massart's lemma, **273**, 288
matrix determinant lemma, 122
MetaGrad algorithm, 243
method of types, 173–174
minimax theorem, **247–248**, 267
Mirror Descent algorithm, 95
mirror map, *see also* duality map, **75**
mistake bound, **143**, 144, 147
mixed strategy, **253**
mixed-strategy space, **253**

multi-armed bandit, 151–169
  adversarial, 151–157
  best-of-both worlds, 165–168
  stochastic, 157–165
Multi-Scale Expert algorithm, **86–91**, 96–97
Multiplicative Weights algorithm, 96

Nash equilibrium, **254–256**
  ϵ-, 255
no-regret algorithm, **2**
norm
  p-, 41, 82
  dual, **41**, 41
  group, **115**
  local, 64, 77, 96, 107, 110, 140, 154
normal cone, **18**, 195
normal-form game, 253, 257
NormalHedge algorithm, 225
normalized maximum likelihood strategy, 225

oblivious adversary, **151**
OCO, *see* online convex optimization
OGD, *see* Online Gradient Descent algorithm
OLO, *see* online linear optimization
OMD, *see* Online Mirror Descent algorithm
online convex optimization, 9, 23
Online Gradient Descent algorithm, **11–16**
online linear optimization, 22
Online Mirror Descent algorithm, **60–98**
  one-step lemma, 66–67
  optimistic, **91–94**, 94–95, 97, 264
  prescient, **265–266**, 268
  with local norms, **77–78**, 81
Online Newton Step algorithm, **122–123**, 125, 138–140, 177–178, 184, 189, 191
online non-convex optimization, 98
Online Subgradient Descent algorithm, **19–20**
online-to-batch conversion, **24–29**, 33, 38–39, 48
ONS, *see* Online Newton Step algorithm
optimism in the face of uncertainty, 162
OSD, *see* Online Subgradient Descent algorithm

PAC learning
  agnostic, 30
PAC-Bayes generalization bound, 273–276, 288
parameter-free, **214**, 224
  learning with continuous experts, 275
  learning with experts, 217–222, 225
  online convex optimization, 211–217, 224–225
  sleeping experts, 222–223
path length, **227**
Pegasos algorithm, 48
Perceptron algorithm, **142–146**, 149–150
  with kernels, 149
portfolio algorithm
  F-weighted, **172**
  definition of universal, **171**
portfolio selection problem, 170–171
potential function, 106, 204
probabilistic method, 50
probability simplex, **78**, 81, 110, 171, 192, 218, 251
projection
  Bregman, **76**, 77, 199–201
  Euclidean, **12**
  generalized, **193**, 195
proximal operator, **119**
pseudo-regret decomposition, **160**
pure strategy, **253**

Rademacher complexity
  empirical, **271**
random variable
  Bernoulli, 55
  Binomial, 297
  Dirichlet, 172
  Rademacher, 51, 59, 270, 296
  subgaussian, **158**, 158–160, 163, 164
regret, 2, 6, **8**
  ϵ-quantile, **218**, 225
  adaptive, **234**, 243
  dynamic, **227–233**, 242
  linear, **22**
  pseudo-, **157**
  sleeping experts, **201**
  strongly adaptive, **234–240**, 243
  tracking, **240–242**, 242
Regret Matching algorithm, 204–205
Regret Matching+ algorithm, **198–199**, 204–205
regret-reward duality, 211
regularizer, 99
  proximal, **102–103**, 137, 140
  self-concordant, 96, 139
relative smoothness, 138
relative strong convexity, 138
risk, **29**

saddle point, **244**
  ϵ-, **247**
Sherman–Morrison formula, 127
signed betting fraction, 53, 207
sleeping experts, **201–203**, 205
soft-thresholding operator, 119
softmax, 187
Squint algorithm, 225
stepsize, *see also* learning rate, 12
Stirling's formula, **298**
Stochastic Gradient Descent algorithm, 3, 12
subdifferential set, **17**
subgradient, **16–19**
  affine transformations, 18–19
  Lipschitz convex function, 19, 42
  max of functions, 18
  sum of functions, 17–18
suboptimality gap, 246
substitution function, **186**
supermartingale, **27**, 27, 281
surrogate gap, 146, **148**

Tsallis-INF algorithm
  FTRL version, **165–168**, 168–169

OMD version, **155–157**, 168
type of a sequence, **173**
unbiased estimator, 157
union bound, **29**, 32, 164, 271, 275, 288
unit analysis, **20–22**
Universal Portfolio algorithm, **171–178**, 282–283
Upper Confidence Bound algorithm, **161–165**, 168
valid set, *see also* feasible set, **2**
validation set, 31, 32
value of the game, **254**
Vapnik's general setting of learning, 29, 270
VC dimension, 150
Vovk–Azoury–Warmuth forecaster, **126–129**, 139
WAA, *see* Weighted Average algorithm
weak learning oracle, **258**
wealth
in coin-betting game, 207
in portfolio selection, 170
Weierstrass theorem for extended-real-valued functions, 66, 248, **291**
Weighted Average Algorithm, **180–186**, 190, 191
Zermelo's theorem, 257